Vertex Operator Algebras in Lean

The Brief

Statements, Lean names and verbatim Lean, without proofs

Fedor Popov and Leonardo Rastelli
with Claude (Anthropic)

Companion to the release edition of the book; documents commit (`recorded at release`)
(built 2026-11-08)

How to read the brief

This is a companion to the release edition of *Vertex Operator Algebras in Lean* (`book.pdf`). For the headline results of every chapter, and for every definition (in any chapter) that their statements use, it prints only: the mathematical statement, as in the book; under **Lean.**, the fully qualified names of the Lean declarations that formalise it; the Lean file(s); every declaration of the book's quotation, verbatim; and the hypotheses and status recorded in the chapter's Referee's checklist. Chapter, theorem and page numbers, and all cross-references, are the *book's*, so every item can be looked up there with its proof, its fidelity discussion ("From the mathematics to the Lean statement") and its physical meaning. Chapter 1 of the book (how to read Lean) has no statements and is omitted.

Every quoted Lean excerpt occurs verbatim (up to whitespace and comments) in the library at the commit above. The qualified names are computed from the `namespace` blocks of the cited files.

A clean `#print axioms` does not make a theorem unconditional: explicit hypotheses are listed in the status line. The library has no project axiom (`VOA.pbw` is a theorem) (book Chapter 3). Part VIII (monstrous moonshine) has exactly one named hypothesis, `GmatEqAut0`: AxiomMath's generation theorem for their model of the Monster, a theorem of their library that does not run on this library's toolchain. It enters only the identification of Aut $V^\natural$ with the whole Monster (`monstrousMoonshine`). The Conway–Norton theorem for every automorphism of $V^\natural$, and the isomorphism of Aut $V^\natural$ with the subgroup of the Monster that acts on $V^\natural$, do not use it.

Contents

I Foundations 1

2 Formal calculus and locality 2

3 Lie-theoretic inputs and the trust base 6

4 Vertex algebra axioms, Borcherds identity, reconstruction 11

5 Structure theory and invariant forms 15

II Examples, deformations and rigidity 19

6 Heisenberg and lattice vertex algebras in the library 20

7 The Virasoro VOA: Verma modules, Kac determinant, screenings 24

8 First-order deformations and the graded first-order rigidity of Virasoro minimal models 28

9 Affine $\mathfrak{sl}_2$: construction, Wakimoto, Verma, Kac 31

10 Graded first-order rigidity of affine $\mathfrak{sl}_2$: Linshaw–Qi and admissible levels 35

11 Filtrations, the C_2 algebra and presentations 37

III Representation theory 43

12 Modules, Zhu's algebra, and the rationality hierarchy 44

13 Zhu's bijection via the tower $A_n(V)$ 53

IV Modularity 57

14 Modular forms and the analytic toolkit 58

15 Zhu's modularity theorem 61

16 Quasi-lisse vertex algebras and the Arakawa–Kawasetsu theorem 67

V Super, logarithmic and unitary theories 73

17 The super generalization 74

18 Logarithmic vertex algebras: the triplet and doublet algebras 77

19 Graded unitarity and Lee–Yang 84

VI Two recent theorems, formalized 88

20 Classical freeness at the boundary level: Arakawa–Dai 89

21 Huang's modular tensor category and the congruence property 97

22 The classification of nice central-charge-one vertex operator algebras (Gannon–Rayhaun) 102

23 Affine VOAs for every simple Lie algebra; the exceptional Lie algebras 105

24 The Deligne series I: presentations, carriers, divisor comparison 110

25 The Deligne series II: characters, the Arakawa–Kawasetsu MLDE and q-series identities 122

26 The Deligne series III: fusion modules and Ravinder's kernel 133

27 The Deligne series IV: final theorems and named inputs 145

28 Graded unitarity through the BRST coset 149

29 The (2,2n+3) series: coset transfer at every boundary level 158

VII The Moonshine module 166

30 Lattices, codes and the Leech lattice 167

31 Heisenberg and lattice vertex algebras 170

32 The involution θ and the fixed-point algebra V_Λ^+ 174

33 Graded dimensions and the modular function J 177

34 Twisted modules, the twisted Fock space and the module T 180

35 The twisted Heisenberg module 184

36 The twisted Leech lattice module and $(V_\Lambda^T)^+$ 189

37 Intertwiners and invariant forms: the operators W and $\overline{W}$ 192

38 The vertex algebra on $V^\natural$ modulo one relation 195

39 The Golay frame and the untwisted triality 198

40 Existence of the twisted triality I: reduction to Zhu generation 201

41 Existence of the twisted triality II: the explicit construction 205

42 The key identity and the completion of the proof 208

VIII Monstrous moonshine 211

43 Monstrous moonshine: McKay–Thompson series, Hauptmoduln and the statement 212

44 The vertex algebra of the hyperbolic plane and the tensor theory of central charge 26 216

45 The Monster Lie algebra: string states, the no-ghost theorem and the Borcherds–Kac–Moody structure 223

46 Denominator identities, replication and genus zero 226

47 Conway–Norton for the automorphism group of the moonshine module 232

48 The Griess algebra of the moonshine module and AxiomMath's Monster 238

49 The Monster acts on the moonshine module: lifts, triality, the commutant and the converse 244

IX Verification 249

50 Verification, dead ends, numerics and open problems 250

Part I

Foundations

Chapter 2

Formal calculus and locality

10 of the chapter's 56 numbered statements: its headline results and every definition they use.

Definition 2.1 (Formal distribution). Let $\mathcal{W}$ be a set. A *formal distribution* in one variable z with coefficients in $\mathcal{W}$ is a function $f : \mathbb{Z} \to \mathcal{W}$, written as the doubly infinite formal series

$$f(z) = \sum_{n \in \mathbb{Z}} f_n \, z^n, \qquad f_n := f(n) \in \mathcal{W}.$$

The set of formal distributions is denoted $\mathcal{W}[[z^{\pm 1}]]$. When $\mathcal{W}$ is an abelian group, respectively a module over a semiring R, $\mathcal{W}[[z^{\pm 1}]]$ is an abelian group, respectively an R-module, under coefficientwise addition and scalar multiplication. Two formal distributions are equal iff all their coefficients agree. No product of two formal distributions is defined.

Lean. `VOA.FormalDist`, `VOA.FormalDist.instAddCommGroup`, `VOA.FormalDist.instModule`, `VOA.FormalDist.ext`
File `VOA/Foundations/FormalSeries.lean`.

```
def FormalDist (𝓦 : Type*) : Type _ := ℤ → 𝓦

@[inherit_doc] notation:max 𝓦 "⟦z^±⟧" => FormalDist 𝓦

instance instAddCommGroup [AddCommGroup 𝓦] : AddCommGroup (FormalDist 𝓦) :=
  inferInstanceAs (AddCommGroup (ℤ → 𝓦))

instance instModule {R : Type*} [Semiring R] [AddCommGroup 𝓦] [Module R 𝓦] :
    Module R (FormalDist 𝓦) := inferInstanceAs (Module R (ℤ → 𝓦))

@[ext]
theorem ext {f g : FormalDist 𝓦} (h : ∀ n, f n = g n) : f = g := funext h
```

Hypotheses / status. Definition

Definition 2.7 (Modes, residue, derivative). Let $f \in \mathcal{W}[[z^{\pm 1}]]$.

(i) The *m-th mode* of f ($m \in \mathbb{Z}$) is $f_{(m)} := f_{-m-1}$, so that $f(z) = \sum_{m \in \mathbb{Z}} f_{(m)} \, z^{-m-1}$.

(ii) If $\mathcal{W}$ is a complex vector space, the *residue* is the linear functional $\mathrm{Res}_z f := f_{-1}$, the coefficient of z^{-1}.

(iii) The *formal derivative* is the linear map ∂_z with $(\partial_z f)_n := (n+1) f_{n+1}$, i.e. $\partial_z \sum_n f_n z^n = \sum_n n f_n z^{n-1}$.

Lean. `VOA.mode`, `VOA.res`, `VOA.deriv`
File `VOA/Foundations/FormalCalculus.lean`.

```
def mode (f : FormalDist 𝒲) (m : ℤ) : 𝒲 := f (-m - 1)

def res : FormalDist 𝒲 →ₗ[ℂ] 𝒲 where
  toFun f := f (-1)

def deriv : FormalDist 𝒲 →ₗ[ℂ] FormalDist 𝒲 where
  toFun f := fun n => (n + 1 : ℤ) • f (n + 1)
```

Hypotheses / status. Definition; **key convention**

Proposition 2.8 (Modes are residues; residues of derivatives vanish). *For every $f \in \mathcal{W}[[z^{\pm 1}]]$ and $m \in \mathbb{Z}$,*

$$f_{(m)} = \operatorname{Res}_z\big(z^m f(z)\big), \qquad \operatorname{Res}_z\big(\partial_z f\big) = 0.$$

Lean. `VOA.mode_eq_res`, `VOA.res_deriv`
File `VOA/Foundations/FormalCalculus.lean`.

```
theorem mode_eq_res (f : FormalDist 𝒲) (m : ℤ) : mode f m = res (mulByMonomial m f) := by

@[simp] theorem res_deriv (f : FormalDist 𝒲) : res (deriv f) = 0 := by
```

Hypotheses / status. None; unconditional

Theorem 2.12 (Integration by parts; Gannon (5.1.3b)). *If $g \in \mathbb{C}[z^{\pm 1}]$ is a Laurent polynomial and $f \in \mathcal{W}[[z^{\pm 1}]]$, then*

$$\operatorname{Res}_z\big(g \cdot \partial_z f\big) = -\operatorname{Res}_z\big((\partial_z g) \cdot f\big).$$

Lean. `VOA.res_lpSMul_deriv`
File `VOA/Foundations/FormalCalculus.lean`.

```
theorem res_lpSMul_deriv (g : FormalDist ℂ) (hg : (Function.support g).Finite)
    (f : FormalDist 𝒲) : res (lpSMul g (deriv f)) = - res (lpSMul (deriv g) f) := by
```

Hypotheses / status. g Laurent polynomial

Theorem 2.27 (Structure theorem; Gannon Prop. 5.1.2, Kac Thm. 2.3). *Let $\mathcal{W}$ be a complex vector space, $f \in \mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$ and $N \in \mathbb{N}$. Then*

$$(z_1 - z_2)^N f = 0 \quad \Longleftrightarrow \quad \exists\, c^0, \dots, c^{N-1} \in \mathcal{W}[[z^{\pm 1}]] :\ f(z_1, z_2) = \sum_{j=0}^{N-1} c^j(z_2)\, \partial_{z_2}^{j} \delta(z_1, z_2).$$

Moreover, in the forward direction the coefficients can be taken to be $c^j = \frac{1}{j!} \operatorname{Res}_{z_1}(z_1 - z_2)^j f$ (Corollary 2.28).

Lean. `VOA.exists_deltaSum_of_mulBySubPow_eq_zero`, `VOA.local_iff_deltaSum`
File `VOA/Foundations/Locality.lean`.

```
theorem exists_deltaSum_of_mulBySubPow_eq_zero (N : ℕ) (f : FormalDist₂ 𝒲)
    (hf : mulBySubPow N f = 0) :
    ∃ c : Fin N → FormalDist 𝒲, f = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j) := by

theorem local_iff_deltaSum (f : FormalDist₂ 𝒲) (N : ℕ) :
    mulBySubPow N f = 0 ↔
      ∃ c : Fin N → FormalDist 𝒲, f = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j) := by
```

Hypotheses / status. None; unconditional

Definition 2.30 (The two expansions of $(z_1-z_2)^n$). For $n \in \mathbb{Z}$ define two elements of $\mathbb{C}[[z_1^{\pm 1}, z_2^{\pm 1}]]$:

$$\iota_{z_1,z_2}(z_1-z_2)^n := \sum_{k\geq 0}(-1)^k\binom{n}{k} z_1^{n-k}z_2^k \qquad \text{(nonnegative powers of } z_2\text{; ``}|z_1|>|z_2|\text{''),}$$

$$\iota_{z_2,z_1}(z_1-z_2)^n := \sum_{k\geq 0}(-1)^{n+k}\binom{n}{k} z_1^{k}z_2^{n-k} \qquad \text{(nonnegative powers of } z_1\text{; ``}|z_2|>|z_1|\text{''}).$$

Also let $\delta(z_1,z_2)=\sum_k z_1^k z_2^{-k-1}$ be the delta function, with coefficient $[m+n+1=0]$.

Lean. `VOA.expandPos`, `VOA.expandNeg`, `VOA.deltaForm`
File `VOA/Foundations/Expansion.lean`.

```
noncomputable def expandPos (n : ℤ) : FormalDist₂ ℂ :=
  fun a b => if 0 ≤ b ∧ a = n - b then negOnePow b * gBinom n b.toNat else 0

noncomputable def expandNeg (n : ℤ) : FormalDist₂ ℂ :=
  fun a b => if 0 ≤ a ∧ b = n - a then negOnePow (n + a) * gBinom n a.toNat else 0

def deltaForm : FormalDist₂ ℂ := fun m n => if m + n + 1 = 0 then 1 else 0
```

Hypotheses / status. Definition

Theorem 2.33 (The delta-function jump; Gannon §5.1.2, FLM §8.10).

$$\iota_{z_1,z_2}\frac{1}{z_1-z_2}-\iota_{z_2,z_1}\frac{1}{z_1-z_2}=\delta(z_1,z_2), \qquad (z_1-z_2)\,\delta(z_1,z_2)=0.$$

Lean. `VOA.expand_jump`, `VOA.mulBySub_deltaForm`
File `VOA/Foundations/Expansion.lean`.

```
theorem expand_jump : expandPos (-1) - expandNeg (-1) = deltaForm := by

theorem mulBySub_deltaForm : mulBySub (deltaForm : FormalDist₂ ℂ) = 0 := by
```

Hypotheses / status. None; unconditional

Theorem 2.36 (Multiplicativity of the expansions). *For all $m,n\in\mathbb{Z}$,*

$$\iota_{12}(z_1-z_2)^m\cdot\iota_{12}(z_1-z_2)^n=\iota_{12}(z_1-z_2)^{m+n}, \qquad \iota_{21}(z_1-z_2)^m\cdot\iota_{21}(z_1-z_2)^n=\iota_{21}(z_1-z_2)^{m+n}.$$

Lean. `VOA.expandPos_mul`, `VOA.expandNeg_mul`
File `VOA/Foundations/IotaZExpansion.lean`.

```
theorem expandPos_mul (m n : ℤ) : convolPos m n = expandPos (m + n) := by

theorem expandNeg_mul (m n : ℤ) : convolNeg m n = expandNeg (m + n) := by
```

Hypotheses / status. None; `convolPos` is the genuine product (paper)

Theorem 2.38 (All-order delta jump). *For every $k\in\mathbb{N}$,*

$$\iota_{12}(z_1-z_2)^{-k-1}-\iota_{21}(z_1-z_2)^{-k-1}=\frac{1}{k!}\,\partial_{z_2}^k\delta(z_1,z_2).$$

Lean. `VOA.mulBySub_negPowJump`, `VOA.negPowJump_residue_zero`, `VOA.deltaDeriv_succ_residue_zero`, `VOA.negPowJump_eq_deltaDeriv`
File `VOA/Foundations/DeltaJump.lean`.

```
theorem mulBySub_negPowJump (k : ℕ) :
    mulBySub (negPowJump (k + 1)) = negPowJump k := by

theorem negPowJump_residue_zero (k : ℕ) (n : ℤ) :
    negPowJump (k + 1) (-1) n = 0 := by

theorem deltaDeriv_succ_residue_zero (k : ℕ) (c : FormalDist ℂ) (n : ℤ) :
    deltaDeriv (k + 1) c (-1) n = 0 := by

theorem negPowJump_eq_deltaDeriv (k : ℕ) :
    negPowJump k = (((k.factorial : ℂ))⁻¹) • deltaDeriv k deltaCoeff := by
```

Hypotheses / status. None; unconditional

Theorem 2.62 (Outer-to-inner transport). *For all orientations o_1, o_2 and $N \in \mathbb{N}$,*

$$(z_{o_1} - z_{o_2})^N K_{o_1 o_2} = (x_1 - x_2)^N K_{o_1 o_2}.$$

Lean. `VOA.FormalDist₄Delta.mulBySub₁₂_comm_mulBySub₃₄`, `VOA.FormalDist₄Delta.mulBySub₃₄_doubleDelta`, `VOA.FormalDist₄Delta.mulBySub₃₄Pow_doubleDelta`
File `VOA/Foundations/FormalDist4Delta.lean`.

```
theorem mulBySub₁₂_comm_mulBySub₃₄ :
    Commute
      (FormalDist₄.mulBySub₁₂ : FormalDist₄ (FormalDist ℂ) →ₗ[ℂ] _)
      (FormalDist₄.mulBySub₃₄ : FormalDist₄ (FormalDist ℂ) →ₗ[ℂ] _) := by

theorem mulBySub₃₄_doubleDelta (o₁ o₂ : Orientation) :
    FormalDist₄.mulBySub₃₄ (doubleDelta o₁ o₂) =
      FormalDist₄.mulBySub₁₂ (doubleDelta o₁ o₂) := by

theorem mulBySub₃₄Pow_doubleDelta (o₁ o₂ : Orientation) (N : ℕ) :
    FormalDist₄.mulBySub₃₄Pow N (doubleDelta o₁ o₂) =
      FormalDist₄.mulBySub₁₂Pow N (doubleDelta o₁ o₂) := by
```

Hypotheses / status. None; unconditional

Chapter 3

Lie-theoretic inputs and the trust base

7 of the chapter's 25 numbered statements: its headline results and every definition they use.

Theorem 3.9 (Poincaré–Birkhoff–Witt). *Let $\mathfrak{g}$ be a Lie algebra over a field F (of any dimension), and let $(e_i)_{i\in I}$ be a basis of $\mathfrak{g}$ indexed by a totally ordered set I. Then the ordered monomials*

$$\iota(e_{i_1})\iota(e_{i_2})\cdots\iota(e_{i_r}), \qquad r\geq 0, \quad i_1\leq i_2\leq \ldots \leq i_r,$$

form a basis of $U(\mathfrak{g})$ (the empty monomial being 1). Equivalently, the associated graded algebra of $U(\mathfrak{g})$ for its natural filtration is the symmetric algebra $S(\mathfrak{g})$.

Lean. `VOA.PBWPrinciple`, `VOA.pbw`
`VOA/Foundations/AffineLie.lean`; the same declaration is now a theorem:

```
structure PBWPrinciple : Prop where
  /-- PBW nonvanishing for every affine vacuum module. -/
  affineVacuum : ∀ {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
      (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) (k : ℂ),
    PBW κ hκ k
  /-- The canonical bottom embeds in every module induced from a Lie subalgebra. -/
  inducedBottom : ∀ {g U : Type u} [LieRing g] [LieAlgebra ℂ g]
      [AddCommGroup U] [Module ℂ U] (p : LieSubalgebra ℂ g)
      (rho : p →ₗ[ℂ] Module.End ℂ U),
    Function.Injective (PBWInduction.bottom p rho)

theorem pbw : PBWPrinciple :=
  ⟨fun κ hκ k => AffineLie.vacuumVector_ne_zero_of_induction κ hκ k,
    fun p rho => PBWTheorem.bottom_injective p rho⟩
```

Proposition 3.11 (The Lean induced module is the induced module). *Let $\mathfrak{p}\subseteq\mathfrak{g}$ be a Lie subalgebra, ρ a representation of $\mathfrak{p}$ on U, and $\varphi : U(\mathfrak{p})\to U(\mathfrak{g})$ the algebra map induced by the inclusion. Make $U(\mathfrak{g})$ a right $U(\mathfrak{p})$-module through φ and U a left $U(\mathfrak{p})$-module through ρ. Let $N\subseteq U(\mathfrak{g})\otimes_{\mathbb{C}} U$ be the span of $\{y\iota(x)\otimes u - y\otimes\rho(x)u : y\in U(\mathfrak{g}), x\in\mathfrak{p}, u\in U\}$. Then N equals the span N' of $\{y\varphi(a)\otimes u - y\otimes a\cdot u : y\in U(\mathfrak{g}), a\in U(\mathfrak{p}), u\in U\}$, so*

$$\texttt{Induced p rho} = (U(\mathfrak{g})\otimes_{\mathbb{C}} U)/N = U(\mathfrak{g})\otimes_{U(\mathfrak{p})} U.$$

Lean. `VOA.PBWInduction.Cover`, `VOA.PBWInduction.balancingSet`, `VOA.PBWInduction.balancing`, `VOA.PBWInduction.Induced`, `VOA.PBWInduction.bottom`
File `VOA/Foundations/AffineLie.lean`.

```
universe u
```

```
namespace PBWInduction

variable {g U : Type u} [LieRing g] [LieAlgebra ℂ g]
variable [AddCommGroup U] [Module ℂ U]

abbrev Cover (g U : Type u) [LieRing g] [LieAlgebra ℂ g]
    [AddCommGroup U] [Module ℂ U] :=
  UniversalEnvelopingAlgebra ℂ g ⊗[ℂ] U

def balancingSet (p : LieSubalgebra ℂ g) (rho : p →ₗ[ℂ] Module.End ℂ U) :
    Set (Cover g U) :=
  {t | ∃ (y : UniversalEnvelopingAlgebra ℂ g) (x : p) (u : U),
    t = (y * UniversalEnvelopingAlgebra.ι ℂ (x : g)) ⊗ₜ[ℂ] u - y ⊗ₜ[ℂ] (rho x u)}

noncomputable def balancing (p : LieSubalgebra ℂ g)
    (rho : p →ₗ[ℂ] Module.End ℂ U) : Submodule ℂ (Cover g U) :=
  Submodule.span ℂ (balancingSet p rho)

abbrev Induced (p : LieSubalgebra ℂ g) (rho : p →ₗ[ℂ] Module.End ℂ U) : Type u :=
  Cover g U ⧸ balancing p rho

noncomputable def bottom (p : LieSubalgebra ℂ g)
    (rho : p →ₗ[ℂ] Module.End ℂ U) : U →ₗ[ℂ] Induced p rho :=
  (balancing p rho).mkQ.comp
    (TensorProduct.mk ℂ (UniversalEnvelopingAlgebra ℂ g) U 1)

end PBWInduction
```

Hypotheses / status. Definitions

Theorem 3.15 (Relative PBW representation; Humphreys §17.4, Lemma A, relative form). *Let $\mathfrak{g}$ be a Lie algebra over a field K (of any dimension), $\mathfrak{p} \subseteq \mathfrak{g}$ a Lie subalgebra, $\mathfrak{c}$ a vector-space complement of $\mathfrak{p}$ in $\mathfrak{g}$, $(y_a)_{a\in A}$ a basis of $\mathfrak{c}$ indexed by a totally ordered set A, and ρ a representation of $\mathfrak{p}$ on a K-vector space U. Let M be the vector space of finitely supported functions from finite multisets of A to U; write $m \otimes u$ for the function with value u at the multiset m and 0 elsewhere, $|m|$ for the number of elements of m counted with multiplicity, and $a \le m$ for "$a \le b$ for every letter b of m". Then there is a representation σ of $\mathfrak{g}$ on M such that, for all $u \in U$, $x \in \mathfrak{p}$ and all multisets m:*

$$\begin{aligned}
&\textit{(E1)} && \sigma(y_a)(m \otimes u) = (a\,m) \otimes u && \text{if } a \le m,\\
&\textit{(E2)} && \sigma(y_a)\big((b\,m) \otimes u\big) = \sigma(y_b)\sigma(y_a)(m \otimes u) + \sigma([y_a, y_b])(m \otimes u) && \text{if } b < a,\ b \le m,\\
&\textit{(E3)} && \sigma(x)(\varnothing \otimes u) = \varnothing \otimes \rho(x)u, &&\\
&\textit{(E4)} && \sigma(x)\big((b\,m) \otimes u\big) = \sigma(y_b)\sigma(x)(m \otimes u) + \sigma([x, y_b])(m \otimes u) && \text{if } b \le m,
\end{aligned}$$

where $a\,m$ denotes the multiset m with one more copy of a. In particular $u \mapsto \varnothing \otimes u$ is an injective linear map $U \to M$ which intertwines $\rho(x)$ with $\sigma(x)$ for every $x \in \mathfrak{p}$.

Lean. `VOA.PBWTheorem.Car`, `VOA.PBWTheorem.act_y_low`, `VOA.PBWTheorem.act_p_zero`, `VOA.PBWTheorem.act_y_cons`, `VOA.PBWTheorem.act_p_cons`, `VOA.PBWTheorem.defect_eq_zero`, `VOA.PBWTheorem.rep`, `VOA.PBWTheorem.rep_apply`, `VOA.PBWTheorem.rep_p_zero`
File `VOA/Foundations/PBWTheorem/Construction.lean`.

```
variable {K : Type*} [Field K]
variable {g : Type*} [LieRing g] [LieAlgebra K g]
variable {U : Type*} [AddCommGroup U] [Module K U]
variable {A : Type*} [LinearOrder A]

abbrev Car (A U : Type*) [AddCommGroup U] [Module K U] := Multiset A →₀ U
```

```
theorem act_y_low {a : A} {m : Multiset A} (h : ∀ b ∈ m, a ≤ b) (u : U) :
    act p hc bc ρ (bc a : g) (single m u) = single (a ::ₘ m) u := by

theorem act_p_zero (x : p) (u : U) :
    act p hc bc ρ (x : g) (single 0 u) = single 0 (ρ x u) := by

theorem act_y_cons {a b : A} {m : Multiset A} (hba : b < a) (hb : ∀ c ∈ m, b ≤ c) (u : U) :
    act p hc bc ρ (bc a : g) (single (b ::ₘ m) u) =
      act p hc bc ρ (bc b : g) (act p hc bc ρ (bc a : g) (single m u)) +
        act p hc bc ρ ⁅(bc a : g), (bc b : g)⁆ (single m u) := by

theorem act_p_cons (x : p) {b : A} {m : Multiset A} (hb : ∀ c ∈ m, b ≤ c) (u : U) :
    act p hc bc ρ (x : g) (single (b ::ₘ m) u) =
      act p hc bc ρ (bc b : g) (act p hc bc ρ (x : g) (single m u)) +
        act p hc bc ρ ⁅(x : g), (bc b : g)⁆ (single m u) := by
```

File `VOA/Foundations/PBWTheorem/LieRep.lean`.

```
variable (p : LieSubalgebra K g) {c : Submodule K g} (hc : IsCompl p.toSubmodule c)
variable (bc : Basis A K c) (ρ : p →ₗ⁅K⁆ Module.End K U)

theorem defect_eq_zero (z z' : g) (v : Car (K := K) A U) : defect p hc bc ρ z z' v = 0 :=
 ↪ by

noncomputable def rep : g →ₗ⁅K⁆ Module.End K (Car (K := K) A U) :=

theorem rep_apply (z : g) (v : Car (K := K) A U) : rep p hc bc ρ z v = act p hc bc ρ z v :=
 ↪ rfl

theorem rep_p_zero (x : p) (u : U) :
    rep p hc bc ρ (x : g) (single 0 u) = single 0 (ρ x u) :=
  act_p_zero p hc bc ρ x u
```

Hypotheses / status. Unconditional, any field

Theorem 3.17 (Bottom injectivity, relative PBW). *Let $\mathfrak{g}$ be a Lie algebra over a field K, $\mathfrak{p}$ a Lie subalgebra and ρ a representation of $\mathfrak{p}$ on U. Then the map $U \to (U(\mathfrak{g}) \otimes_K U)/N$, $u \mapsto [1 \otimes u]$, is injective, where N is the span of the balancing relations $y\,\iota(x) \otimes u - y \otimes \rho(x)u$. By Proposition 3.11 this is the map $U \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$, and over $K = \mathbb{C}$ it is field (B) of* `PBWPrinciple`*. More generally: if there are a representation σ of $\mathfrak{g}$ on some M and an injective linear map $j : U \to M$ with $\sigma(x)j(u) = j(\rho(x)u)$ for $x \in \mathfrak{p}$, then the bottom map is injective.*

Lean. `VOA.PBWTheorem.balancingSet`, `VOA.PBWTheorem.balancing`, `VOA.PBWTheorem.bottom`, `VOA.PBWTheorem.bottom_injective_of_embedding`, `VOA.PBWTheorem.bottom_injective`
File `VOA/Foundations/PBWTheorem/Induced.lean`.

```
def balancingSet (p : LieSubalgebra K g) (rho : p →ₗ⁅K⁆ Module.End K U) :
    Set (UniversalEnvelopingAlgebra K g ⊗[K] U) :=
  {t | ∃ (y : UniversalEnvelopingAlgebra K g) (x : p) (u : U),
    t = (y * UniversalEnvelopingAlgebra.ι K (x : g)) ⊗ₜ[K] u - y ⊗ₜ[K] (rho x u)}

noncomputable def balancing (p : LieSubalgebra K g) (rho : p →ₗ⁅K⁆ Module.End K U) :
    Submodule K (UniversalEnvelopingAlgebra K g ⊗[K] U) :=
  Submodule.span K (balancingSet p rho)

noncomputable def bottom (p : LieSubalgebra K g) (rho : p →ₗ⁅K⁆ Module.End K U) :
    U →ₗ[K] (UniversalEnvelopingAlgebra K g ⊗[K] U) ⧸ balancing p rho :=
  (balancing p rho).mkQ.comp (TensorProduct.mk K (UniversalEnvelopingAlgebra K g) U 1)
```

```
theorem bottom_injective_of_embedding (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K
 ↪  U)
    {M : Type*} [AddCommGroup M] [Module K M] (σ : g →ₗ[K] Module.End K M) (j : U →ₗ[K] M)
    (hj : Function.Injective j) (hσ : ∀ (x : p) (u : U), σ (x : g) (j u) = j (rho x u)) :
    Function.Injective (bottom p rho) := by

theorem bottom_injective (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K U) :
    Function.Injective (bottom p rho) := by
```

Hypotheses / status. Unconditional

Theorem 3.18 (Nonvanishing of the affine vacuum, from bottom injectivity). *For every complex Lie algebra $\mathfrak{g}$ with symmetric invariant form κ and every $k \in \mathbb{C}$, the vacuum vector of $V^k(\mathfrak{g})$ is nonzero. The proof uses the subalgebra $\hat{\mathfrak{p}} = \mathfrak{g}[t] \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ of $\hat{\mathfrak{g}}$ and its character $\chi_k(f + aC + p\ell_0) = ka$.*

Lean. `VOA.AffineLie.nonnegSub`, `VOA.AffineLie.vacuumChar`, `VOA.AffineLie.tmul_one_mem_balancing_of_mem_vacuumIdeal`, `VOA.AffineLie.vacuumVector_ne_zero_of_induction`
File `VOA/Foundations/AffineLie.lean`.

```
noncomputable def nonnegSub : LieSubalgebra ℂ (AffineLie 𝔤 κ hκ) where
  carrier := {X | X.carrier.1 ∈ supported 𝔤 ℂ (Set.Ici (0 : ℤ))}
  ...

noncomputable def vacuumChar (k : ℂ) : nonnegSub κ hκ →ₗ[ℂ] Module.End ℂ ℂ where
  toFun X := (k * (X : AffineLie 𝔤 κ hκ).carrier.2.1) • LinearMap.id
  ...

theorem tmul_one_mem_balancing_of_mem_vacuumIdeal (k : ℂ) {a : UEnv 𝔤 κ hκ}
    (ha : a ∈ vacuumIdeal κ hκ k) :
    a ⊗ₜ[ℂ] (1 : ℂ) ∈ PBWTheorem.balancing (nonnegSub κ hκ) (vacuumChar κ hκ k) := by

theorem vacuumVector_ne_zero_of_induction (k : ℂ) : vacuumVector κ hκ k ≠ 0 := by
```

Hypotheses / status. Unconditional

Definition 3.20 (Ordered creation word; PBW basis hypothesis). Let $(b_i)_{i \in I}$ be a basis of $\mathfrak{g}$ indexed by a totally ordered set I, and order $I \times \mathbb{N}$ lexicographically. For a finitely supported exponent function $\mu : I \times \mathbb{N} \to \mathbb{N}$, the *ordered creation word* lists the pairs (i, n) in the support in increasing order, each repeated $\mu(i, n)$ times; the corresponding *ordered creation monomial* is

$$v_\mu = \prod_{(i,n)\ \text{increasing}} (b_i \otimes t^{-n-1})^{\mu(i,n)} |0\rangle \ \in V^k(\mathfrak{g}),$$

with the smallest pair leftmost. PBWBasis(κ, b, k) is the statement that $(v_\mu)_\mu$ is a basis of $V^k(\mathfrak{g})$.

Lean. `VOA.pbwWord`, `VOA.PBWBasis`, `VOA.PBWBasis.linearIndependent`, `VOA.PBWBasis.vacuumVector_ne_zero`, `VOA.pbwBasis_iff_of_subsingleton`
File `VOA/Foundations/PBWBasis.lean`.

```
noncomputable def pbwWord {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤] {ι : Type*}
 ↪  [LinearOrder ι]
    (b : Module.Basis ι ℂ 𝔤) (μ : (ι ×ₗ ℕ) →₀ ℕ) : List (𝔤 × ℕ) :=
  (μ.support.sort (· ≤ ·)).flatMap fun p => List.replicate (μ p) (b (ofLex p).1, (ofLex
   ↪  p).2)
```

```
def PBWBasis {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant)
    {ι : Type*} [LinearOrder ι] (b : Module.Basis ι ℂ 𝔤) (k : ℂ) : Prop :=
  ∃ B : Module.Basis ((ι ×ₗ ℕ) →₀ ℕ) ℂ (AffineLie.vacuumSpace κ hκ k),
    ∀ μ, B μ = AffineLie.negWord κ hκ k (pbwWord b μ)
```

```
theorem linearIndependent (h : PBWBasis κ hκ b k) :
    LinearIndependent ℂ fun μ : (ι ×ₗ ℕ) →₀ ℕ => AffineLie.negWord κ hκ k (pbwWord b μ) :=
     ↪ by

theorem vacuumVector_ne_zero (h : PBWBasis κ hκ b k) :
    AffineLie.vacuumVector κ hκ k ≠ 0 := by

theorem pbwBasis_iff_of_subsingleton :
    PBWBasis κ hκ (Module.Basis.empty 𝔤 : Module.Basis (Fin 0) ℂ 𝔤) k
      ↔ AffineLie.vacuumVector κ hκ k ≠ 0 :=
```

Theorem 3.21 (PBW basis of the affine vacuum module). *For every complex Lie algebra $\mathfrak{g}$ (of any dimension), every symmetric invariant bilinear form κ, every basis $(b_i)_{i\in I}$ of $\mathfrak{g}$ indexed by a totally ordered set, and every $k \in \mathbb{C}$ (including the critical level),* PBWBasis(κ, b, k) *holds: the ordered creation monomials v_μ of Definition 3.20 form a basis of $V^k(\mathfrak{g})$. More precisely, the linear map $\Phi : V^k(\mathfrak{g}) \to S(\mathfrak{n}_-)$, $[a] \mapsto a\cdot 1$, given by the relative PBW representation of $\hat{\mathfrak{g}}$ on $S(\mathfrak{n}_-)\otimes\mathbb{C}_{\chi_k} \cong S(\mathfrak{n}_-)$, is a linear isomorphism.*

Lean. `VOA.VacuumBasis.yMode`, `VOA.VacuumBasis.negComp`, `VOA.VacuumBasis.negComp_isCompl`, `VOA.VacuumBasis.Φ`, `VOA.VacuumBasis.Φ_rho`, `VOA.VacuumBasis.ΦEquiv`, `VOA.pbwBasis_holds`
File `VOA/Foundations/PBWTheorem/VacuumBasis.lean`.

```
noncomputable def yMode (a : ι ×ₗ ℕ) : AffineLie 𝔤 κ hκ :=
  ofLoop κ hκ (b (ofLex a).1) (-((ofLex a).2 : ℤ) - 1)

noncomputable def negComp : Submodule ℂ (AffineLie 𝔤 κ hκ) :=
  Submodule.span ℂ (Set.range (yMode κ hκ b))

theorem negComp_isCompl : IsCompl (nonnegSub κ hκ).toSubmodule (negComp κ hκ b) := by

noncomputable def Φ : vacuumSpace κ hκ k →ₗ[ℂ] (Multiset (ι ×ₗ ℕ) →₀ ℂ) :=

theorem Φ_rho (z : AffineLie 𝔤 κ hκ) (v : vacuumSpace κ hκ k) :
    Φ κ hκ b k (vacuumRho κ hκ k z v) = ACT z (Φ κ hκ b k v) := by

noncomputable def ΦEquiv : vacuumSpace κ hκ k ≃ₗ[ℂ] (Multiset (ι ×ₗ ℕ) →₀ ℂ) :=
  LinearEquiv.ofLinear (Φ κ hκ b k) (Ψ κ hκ b k)
    (Finsupp.lhom_ext fun m u => Φ_Ψ_single κ hκ b k m u)
    (LinearMap.ext_on_range (negSpan_eq_top κ hκ k) fun L => Ψ_Φ_negWord κ hκ b k L)

theorem pbwBasis_holds {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant)
    {ι : Type*} [LinearOrder ι] (b : Module.Basis ι ℂ 𝔤) (k : ℂ) : PBWBasis κ hκ b k := by
```

Hypotheses / status. Unconditional

Chapter 4

Vertex algebra axioms, the Borcherds identity, and reconstruction

6 of the chapter's 51 numbered statements: its headline results and every definition they use.

Theorem 4.14 (Goddard uniqueness, three forms). *Let V be a vertex algebra in the library's sense, $a \in V$ and $A(z) \in \mathrm{End}(V)[[z^{\pm 1}]]$ with $A(z)\mathbf{1} = Y(a,z)\mathbf{1}$ coefficientwise.*

(i) If A is local with every $Y(b,z)$, then $A = Y(a,\cdot)$.

(ii) It suffices that for every b some $(z_1 - z_2)^N$ kills $[A(z_1), Y(b,z_2)]\mathbf{1}$ (the commutator applied to the vacuum only).

*(iii) (*Abstract test fields.*) Let V be any vector space with a vector $\mathbf{1}$, A a distribution with $A(z)\mathbf{1} = 0$, and $(G_b)_{b \in V}$ distributions with $G_b(w)\mathbf{1} \in V[[w]]$ of constant term b. If A is local with every G_b, then $A = 0$.*

Lean. `VOA.PreVertexAlgebra.goddard_uniqueness`, `VOA.goddard_zero`, `VOA.PreVertexAlgebra.goddard_uniqueness_vac`

```
theorem goddard_uniqueness (V : PreVertexAlgebra 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    (hAloc : ∀ b : 𝒱, Local A (V.Y b))
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) :
    A = V.Y a

theorem goddard_zero (vac : 𝒱) (A : FormalDist (End ℂ 𝒱)) (G : 𝒱 → FormalDist (End ℂ 𝒱))
    (hloc : ∀ b, Local A (G b))
    (hcre0 : ∀ b, (G b) 0 vac = b)
    (hcreneg : ∀ b, ∀ q : ℤ, q < 0 → (G b) q vac = 0)
    (hAvac : ∀ p : ℤ, A p vac = 0)
    (m : ℤ) (b : 𝒱) : A m b = 0
```

File `VOA/Core/Goddard.lean` (`VOA.PreVertexAlgebra.goddard_uniqueness`, `VOA.goddard_zero`).

```
theorem goddard_uniqueness_vac (V : PreVertexAlgebra 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    (hAloc : ∀ b : 𝒱, ∃ N : ℕ, mulBySubPow N (applyVec V.vacuum (bracketDist A (V.Y b))) =
    ↪ 0)
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) :
    A = V.Y a
```

File `VOA/Core/WeakAssoc.lean` (`VOA.PreVertexAlgebra.goddard_uniqueness_vac`).
Hypotheses / status. VA level / abstract; unconditional

Theorem 4.16 (Dong's lemma, all products). *Let a, b, c be pairwise local distributions on V. Then for every $k \geq 0$ the field product $a_{(k)}b := \mathrm{Res}_{z_1}(z_1 - z_2)^k[a(z_1), b(z_2)]$ is local with c. If moreover a, b are fields, then $:ab:$ is local with c, and hence so are all negative products $a_{(-m-1)}b = \frac{1}{m!}:(\partial^m a)b:$.*

Lean. `VOA.jacobi₃`, `VOA.dong_core`, `VOA.dong_nonneg`, `VOA.dong_neg`

```
theorem jacobi₃ (a b c : FormalDist (End ℂ V)) :
    outerC₁₂ (bracketDist a b) c
      = outerA a (bracketDist b c) - outerB b (bracketDist a c)

theorem dong_core (a b c : FormalDist (End ℂ V)) {r : ℕ}
    (hab : mulBySubPow r (bracketDist a b) = 0)
    (hac : mulBySubPow r (bracketDist a c) = 0)
    (hbc : mulBySubPow r (bracketDist b c) = 0)
    (k M : ℕ) (hM : 2 * r ≤ M) :
    mulBySub₂₃Pow (M + r) (mulBySub₁₂Pow k (outerC₁₂ (bracketDist a b) c)) = 0

theorem dong_nonneg {a b c : FormalDist (End ℂ V)}
    (hab : Local a b) (hac : Local a c) (hbc : Local b c) (k : ℕ) :
    Local (cCoeff k (bracketDist a b)) c

theorem dong_neg {a b c : FormalDist (End ℂ V)} (ha : IsField a) (hb : IsField b)
    (hab : Local a b) (hac : Local a c) (hbc : Local b c) :
    Local (nprod ha hb) c
```

File `VOA/Core/Dong.lean`.
Hypotheses / status. Pairwise locality (+ field); unconditional

Theorem 4.20 (The n-th product theorem, $n \geq 0$). *In a VOA, for all u, v and $k \in \mathbb{N}$:*

$$Y(u_{(k)}v, z) = \mathrm{Res}_{z_1}(z_1 - z)^k[Y(u, z_1), Y(v, z)].$$

Lean. `VOA.VertexOperatorAlgebra.cCoeff_vacuum_eq`, `VOA.VertexOperatorAlgebra.nth_product`, `VOA.VertexOperatorAlgebra.assoc_nonneg`

```
theorem cCoeff_vacuum_eq (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) (m : ℤ) :
    cCoeff k (bracketDist (W.Y u) (W.Y v)) m W.vacuum = (W.Y (W.smul u k v)) m W.vacuum

theorem nth_product (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) :
    cCoeff k (bracketDist (W.Y u) (W.Y v)) = W.Y (W.smul u k v)

theorem assoc_nonneg (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) :
    W.Y (W.smul u k v) = cCoeff k (bracketDist (W.Y u) (W.Y v))
```

File `VOA/Core/Reconstruction.lean`.
Hypotheses / status. VOA; unconditional

Theorem 4.21 (Commutator formula). *In a VOA, for all u, v and $a, b \in \mathbb{Z}$, $[u_{(a)}, v_{(b)}] = \sum_{j \geq 0} \binom{a}{j}(u_{(j)}v)_{(a+b-j)}$ (a finite sum).*

Lean. `VOA.VertexOperatorAlgebra.mode_commutator`, `VOA.VertexOperatorAlgebra.commBound`

```
theorem mode_commutator (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (a b : ℤ) :
    ∃ N : ℕ, ⁅mode (W.Y u) a, mode (W.Y v) b⁆
      = ∑ j : Fin N, gBinom a (j : ℕ) • mode (W.Y (W.smul u (j : ℕ) v)) (a + b - (j : ℕ))

theorem commBound (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (m n : ℤ) :
    ∃ N : ℕ, (∀ k : ℕ, N ≤ k → W.smul u k v = 0)
      ∧ ⁅mode (W.Y u) m, mode (W.Y v) n⁆
        = ∑ j : Fin N, gBinom m (j : ℕ) • mode (W.Y (W.smul u (j : ℕ) v)) (m + n - (j : ℕ))
```

File `VOA/Core/Reconstruction.lean` (`mode_commutator`) and `VOA/Core/Borcherds.lean` (`commBound`). The theorem `jacobi_commutator` of `VOA/Core/BorcherdsIdentity.lean` is `mode_commutator` under another name.

Hypotheses / status. VOA; $\exists N$ made exhaustive by `commBound`

Theorem 4.26 (The Borcherds identity). *In a VOA, for all $u, v, x \in V$ and all $\ell, m, n \in \mathbb{Z}$, $B_{\mathrm{L}}(\ell, m, n) = B_{\mathrm{R}}(\ell, m, n)$, i.e.*

$$\sum_{i \geq 0} (-1)^i \binom{\ell}{i} \Big(u_{(\ell+m-i)} v_{(n+i)} x - (-1)^{\ell} v_{(\ell+n-i)} u_{(m+i)} x \Big) = \sum_{i \geq 0} \binom{m}{i} (u_{(\ell+i)} v)_{(m+n-i)} x. \tag{4.1}$$

Lean. `VOA.VertexOperatorAlgebra.borcherds_base`, `VOA.VertexOperatorAlgebra.borcherds_boundary`, `VOA.VertexOperatorAlgebra.borcherds`, `VOA.VertexOperatorAlgebra.jacobi`

```
theorem borcherds_base (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) (ℓ n : ℤ) :
    BL W u v x ℓ 0 n = BR W u v x ℓ 0 n

theorem borcherds_boundary (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) (m n : ℤ) :
    BL W u v x 0 m n = BR W u v x 0 m n

theorem borcherds (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) :
    BL W u v x ℓ m n = BR W u v x ℓ m n

theorem jacobi (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (W.smul u (ℓ + m - i) (W.smul v (n + i) x)
          - negOnePow ℓ • W.smul v (ℓ + n - i) (W.smul u (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • W.smul (W.smul u (ℓ + i) v) (m + n - i) x
```

File `VOA/Core/Borcherds.lean`.

Hypotheses / status. VOA; unconditional, axiom-clean

Theorem 4.41 (Operator-first reconstruction). *Let V be a vector space, $\mathbf{1} \in V$, $T \in \operatorname{End} V$ with $T\mathbf{1} = 0$, and $(s_i, f_i)_{i \in I}$ states and distributions such that each f_i is a field, the f_i are pairwise local and T-covariant, $f_i(z)\mathbf{1} \in V[[z]]$ with constant term s_i, and the s_i span V. Then there is a linear $Y : V \to \operatorname{End}(V)[[z^{\pm 1}]]$ with $Y(s_i, \cdot) = f_i$, all $Y(u, \cdot)$ fields, pairwise local, T-covariant, with $Y(u, z)\mathbf{1} \in u + zV[[z]]$.*

Lean. `VOA.Reconstruction.reconstruct`

```
theorem reconstruct {ι : Type*} (vac : V) (st : ι → V) (fld : ι → FormalDist (End ℂ V))
    (T : End ℂ V)
    (hspan : Submodule.span ℂ (Set.range st) = ⊤)
    (hfield : ∀ i, IsField (fld i))
    (hlocal : ∀ i j, Local (fld i) (fld j))
    (hcre0 : ∀ i, (fld i) 0 vac = st i)
    (hcreneg : ∀ i, ∀ q : ℤ, q < 0 → (fld i) q vac = 0)
    (hTvac : T vac = 0)
    (hTcov : ∀ i, DCovariant T (fld i)) :
    ∃ YA : V →ₗ[ℂ] FormalDist (End ℂ V),
      (∀ i, YA (st i) = fld i) ∧
      (∀ u : V, IsField (YA u)) ∧
      (∀ u v : V, Local (YA u) (YA v)) ∧
      (∀ u : V, (YA u) 0 vac = u) ∧
      (∀ u : V, ∀ q : ℤ, q < 0 → (YA u) q vac = 0) ∧
      (∀ u : V, DCovariant T (YA u))
```

File `VOA/Core/ReconstructionExistence.lean` (`VOA.Reconstruction.reconstruct`).
Hypotheses / status. All hypotheses explicit; no VA2/VA1 in conclusion

Chapter 5

Structure theory and invariant forms

7 of the chapter's 55 numbered statements: its headline results and every definition they use.

Theorem 5.2 (Zero modes are derivations; Gannon (5.2.4)). *For all $u, v, w \in V$ and $n \in \mathbb{Z}$: $u_{(0)}(v_{(n)}w) - v_{(n)}(u_{(0)}w) = (u_{(0)}v)_{(n)}w$.*

Lean. `VOA.VertexOperatorAlgebra.zeroMode_isDerivation`, `VOA.VertexOperatorAlgebra.zeroMode_leibniz`
File `VOA/Core/LieAlgebraV1.lean`.

```
theorem zeroMode_isDerivation (W : VertexOperatorAlgebra 𝒱) (u v w : 𝒱) (n : ℤ) :
    W.smul u 0 (W.smul v n w) - W.smul v n (W.smul u 0 w) = W.smul (W.smul u 0 v) n w := by
```

File `VOA/Core/Automorphism.lean` restates it as a Leibniz rule:

```
theorem zeroMode_leibniz (W : VertexOperatorAlgebra 𝒱) (u a b : 𝒱) (n : ℤ) :
    W.smul u 0 (W.smul a n b) = W.smul (W.smul u 0 a) n b + W.smul a n (W.smul u 0 b) := by
```

Hypotheses / status. None; unconditional

Theorem 5.17 (Ideals are two-sided). *If I is an ideal, then $u_{(n)}v \in I$ for all $u \in I$, $v \in V$, $n \in \mathbb{Z}$.*

Lean. `VOA.QuotVOA.AbsorbsLeft`, `VOA.QuotVOA.absorbsLeft_of_isIdeal`
File `VOA/Core/QuotientVOA.lean`.

```
def AbsorbsLeft (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) : Prop :=
  ∀ (u v : 𝒱) (n : ℤ), u ∈ I → W.smul u n v ∈ I

theorem absorbsLeft_of_isIdeal (hI : W.IsIdeal I) : AbsorbsLeft W I := by
```

Hypotheses / status. None

Theorem 5.18 (Ideals are graded). *Every ideal I satisfies $I = \bigoplus_n (I \cap V_n)$.*

Lean. `VOA.QuotVOA.IsGradedIdeal`, `VOA.QuotVOA.isGradedIdeal_of_isIdeal`
File `VOA/Core/QuotientVOA.lean`.

```
def IsGradedIdeal (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) : Prop :=
  I = ⨆ n : ℤ, I ⊓ W.grading n

theorem isGradedIdeal_of_isIdeal (hI : W.IsIdeal I) : IsGradedIdeal W I := by
```

Hypotheses / status. None

Definition 5.26 (Generated subalgebra). For $S \subseteq V$, $\langle S \rangle$ is the intersection of all vertex operator subalgebras (in the sense just recalled) containing S.

Lean. `VOA.VOASubalgebra.generated`, `VOA.VOASubalgebra.generated_le`, `VOA.VOASubalgebra.mem_generated`, `VOA.VOASubalgebra.generatedVOA`
File `VOA/Core/VOASubalgebraGenerated.lean`.

```
def generated (S : Set 𝒱) : VOASubalgebra W :=
  sInf {A : VOASubalgebra W | S ⊆ A.carrier}

theorem generated_le {S : Set 𝒱} (A : VOASubalgebra W) (hA : S ⊆ A.carrier) :
    (generated W S).carrier ≤ A.carrier :=

theorem mem_generated {S : Set 𝒱} {motive : 𝒱 → Prop}
    (base : ∀ x ∈ S, motive x)
    (zero : motive 0)
    (add : ∀ x y : 𝒱, motive x → motive y → motive (x + y))
    (smul : ∀ (c : ℂ) (x : 𝒱), motive x → motive (c • x))
    (vac : motive W.vacuum)
    (conf : motive W.conformalVector)
    (prod : ∀ u : 𝒱, motive u → ∀ (n : ℤ) (v : 𝒱), motive v → motive (W.smul u n v))
    (proj : ∀ (n : ℤ) (v : 𝒱), motive v → motive (piHom W n v))
    {x : 𝒱} (hx : x ∈ (generated W S).carrier) : motive x := by

noncomputable def generatedVOA (S : Set 𝒱) :
    VertexOperatorAlgebra (generated W S).carrier :=
  (generated W S).toVOA
```

Hypotheses / status. always contains ω

Theorem 5.50 (Orthogonality, radical, uniqueness, nondegeneracy, evaluation on V_1). *Let B be an invariant form. (i) $(V_m|V_n) = 0$ for $m \neq n$. Now let B be fully invariant. (ii) Its left radical is an ideal. (iii) If B_1, B_2 are fully invariant and normalised, $B_1 = B_2$. (iv) If V is simple and B is fully invariant and normalised, its radical is 0. (v) If B is fully invariant and normalised, $u_{(1)}v = -(u|v)\mathbf{1}$ for $u, v \in V_1$.*

Lean. `VOA.VertexOperatorAlgebra.InvariantForm.orthogonal_of_ne`, `VOA.VertexOperatorAlgebra.InvariantForm.radical_isIdeal`, `VOA.VertexOperatorAlgebra.InvariantForm.invariantForm_unique`, `VOA.VertexOperatorAlgebra.InvariantForm.invariantForm_nondegenerate`, `VOA.VertexOperatorAlgebra.InvariantForm.eval_oneMode_grading1`
File `VOA/Core/BilinearForm.lean`.

```
theorem orthogonal_of_ne (B : InvariantForm W) {m n : ℤ} (hmn : m ≠ n) {a b : 𝒱}
    (ha : a ∈ W.grading m) (hb : b ∈ W.grading n) : B.form a b = 0 := by

theorem radical_isIdeal {B : InvariantForm W} (hB : B.IsFullyInvariant) :
    W.IsIdeal (leftRadical B) := by

theorem invariantForm_unique {B₁ B₂ : InvariantForm W}
    (h₁ : B₁.IsFullyInvariant) (h₂ : B₂.IsFullyInvariant)
    (hn₁ : B₁.Normalized) (hn₂ : B₂.Normalized) : B₁.form = B₂.form := by

theorem invariantForm_nondegenerate {B : InvariantForm W} (hsimple : W.IsSimple)
    (hB : B.IsFullyInvariant) (hn : B.Normalized) : leftRadical B = ⊥ := by

theorem eval_oneMode_grading1 {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) {u v : 𝒱} (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    W.smul u 1 v = -B.form u v • W.vacuum := by
```

Hypotheses / status. None

Theorem 5.56 (FHL, Li: existence and uniqueness of the invariant form). *Let V be a VOA (of CFT type) with $\mathbf{1} \neq 0$ and $L_1 V_1 = 0$. There is a fully invariant form with $(\mathbf{1}|\mathbf{1}) = 1$; every fully invariant normalised form equals it; it is nondegenerate if V is simple; and $u_{(1)}v = -(u|v)\mathbf{1}$ on V_1.*

Lean. `VOA.canonicalInvariantForm`, `VOA.canonicalInvariantForm_isFullyInvariant`, `VOA.canonicalInvariantForm_normalized`, `VOA.invariantForm_unique_FHL`, `VOA.invariantForm_nondegenerate_FHL`, `VOA.eval_oneMode_grading1_FHL`
File `VOA/Core/InvariantFormFHL2.lean`.

```
noncomputable def canonicalInvariantForm (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) : InvariantForm W :=
  canFormFHL hne (canDiagonal_unconditional hne hqp)

theorem canonicalInvariantForm_isFullyInvariant (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) :
    (canonicalInvariantForm hne hqp).IsFullyInvariant :=

theorem canonicalInvariantForm_normalized (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) :
    (canonicalInvariantForm hne hqp).Normalized :=

theorem invariantForm_unique_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {B : InvariantForm W}
    (hB : B.IsFullyInvariant) (hn : B.Normalized) :
    B.form = (canonicalInvariantForm hne hqp).form :=

theorem invariantForm_nondegenerate_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (hsimple : W.IsSimple) :
    InvariantForm.leftRadical (canonicalInvariantForm hne hqp) = ⊥ :=

theorem eval_oneMode_grading1_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {u v : V}
    (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    W.smul u 1 v = -(canonicalInvariantForm hne hqp).form u v • W.vacuum :=
```

Hypotheses / status. $\mathbf{1} \neq 0$, $L_1 V_1 = 0$; symmetry **not proved**

Theorem 5.57 (Li, Cor. 3.2). *Let B be a fully invariant normalised form on V. Then*

1. $\operatorname{rad}_L B = \mathfrak{m}(V) = \operatorname{rad}_R B$*;*
2. *V is simple if and only if $\operatorname{rad}_L B = 0$;*
3. *for homogeneous $x \in V_n$: $x \in \mathfrak{m}(V) \iff (x|y) = 0$ for all $y \in V_n$; equivalently, the weight-n part of $\mathfrak{m}(V)$ is the radical of the restriction of B to $V_n \times V_n$.*

Lean. `VOA.LiRadical.leftRadical_eq_maxGradedIdeal`, `VOA.LiRadical.leftRadical_eq_rightRadical`, `VOA.LiRadical.isSimple_iff_leftRadical_eq_bot`, `VOA.LiRadical.mem_maxGradedIdeal_iff_grading`, `VOA.LiRadical.gradedForm`, `VOA.LiRadical.ker_gradedForm`
File `VOA/Core/InvariantFormRadical.lean`.

```
theorem leftRadical_eq_maxGradedIdeal {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    InvariantForm.leftRadical B = QuotVOA.maxGradedIdeal W :=
```

```
theorem leftRadical_eq_rightRadical {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    InvariantForm.leftRadical B = SingVec.rightRadical B := by

theorem isSimple_iff_leftRadical_eq_bot {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    W.IsSimple ↔ InvariantForm.leftRadical B = ⊥ := by

theorem mem_maxGradedIdeal_iff_grading {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) {n : ℤ} {x : 𝒱} (hx : x ∈ W.grading n) :
    x ∈ QuotVOA.maxGradedIdeal W ↔ ∀ y ∈ W.grading n, B.form x y = 0 := by

def gradedForm (n : ℤ) : LinearMap.BilinForm ℂ (W.grading n) :=
  B.form.compl₁₂ (W.grading n).subtype (W.grading n).subtype

theorem ker_gradedForm {B : InvariantForm W} (hB : B.IsFullyInvariant) (hn : B.Normalized)
    (n : ℤ) :
    LinearMap.ker (gradedForm B n) = (QuotVOA.maxGradedIdeal W).comap (W.grading n).subtype
     ↪  := by
```

Hypotheses / status. B fully inv., normalised

Part II

Examples, deformations and rigidity

Chapter 6

Heisenberg and lattice vertex algebras in the library

6 of the chapter's 34 numbered statements: its headline results and every definition they use.

Theorem 6.7 (The axioms VA1–VA5 of the library's `PreVertexAlgebra`). *For every $k \in \mathbb{C}$: $Y(\mathbf{1}, z) = \mathrm{id}$; $Y(a_{-1}\mathbf{1}, z) = \alpha(z)$; for every u, $u_{(-1)}\mathbf{1} = u$ and $u_{(n)}\mathbf{1} = 0$ for $n \geq 0$; any two $Y(u, z)$, $Y(v, z)$ are mutually local; for all u, v there is N with $u_{(n)}v = 0$ for $n \geq N$; and if $u \in \mathcal{F}_{(p)}$, $v \in \mathcal{F}_{(q)}$ then $u_{(n)}v \in \mathcal{F}_{(p+q-n-1)}$.*

Lean. `VOA.Y_vacuum`, `VOA.Y_current`, `VOA.Y_creation`, `VOA.Y_local`, `VOA.va5_regularity`, `VOA.va1_grading_mode`
Files `VOA/Examples/HeisenbergVertex.lean`, `VOA/Examples/HeisenbergCreation.lean`, `VOA/Examples/HeisenbergStructure.lean`, `VOA/Examples/HeisenbergGrading.lean`.

```
theorem Y_vacuum (k : ℂ) : Y k vacuum = idField

theorem Y_current (k : ℂ) : Y k (osc k (-1) vacuum) = current k

theorem Y_creation (k : ℂ) (u : Fock) :
    mode (Y k u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y k u) n vacuum = 0

theorem Y_local (k : ℂ) (u v : Fock) : Local (Y k u) (Y k v)

theorem va5_regularity (k : ℂ) (u v : Fock) :
    ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y k u) n v = 0

theorem va1_grading_mode (kk : ℂ) {k ℓ : ℤ} (u : Fock) (hu : u ∈ gradingZ k)
    (v : Fock) (hv : v ∈ gradingZ ℓ) (n : ℤ) :
    mode (Y kk u) n v ∈ gradingZ (k + ℓ - n - 1)
```

Hypotheses / status. Unconditional (any k)

Theorem 6.10 (The Heisenberg vertex operator algebra at level k). *For every $k \in \mathbb{C} \setminus \{0\}$, $(\mathcal{F}, Y, \mathbf{1}, \omega)$ with the grading $(\mathcal{F}_{(n)})_{n\in\mathbb{Z}}$ is a vertex operator algebra of CFT type with central charge* 1.

Lean. `VOA.heisVA`, `VOA.heisenbergVOA`, `VOA.heisenberg_central_charge`
File `VOA/Examples/HeisenbergVOA.lean`.

```
noncomputable def heisVA (k : ℂ) (_hk : k ≠ 0) : PreVertexAlgebra Fock where
  grading := gradingZ
  isInternal := gradingZ_isInternal
  finrank := gradingZ_finrank
  Y := Y k
```

```
  vacuum := vacuum
  vacuum_mem := mem_gradingZ_of_gradingW vacuum_mem_gradingW
  grading_mode := fun u hu v hv n => va1_grading_mode k u hu v hv n
  vacuum_Y := by rw [Y_vacuum, idField_eq_one]
  creation := fun u => Y_creation k u
  locality := fun u v => Y_local k u v
  regularity := fun u v => va5_regularity k u v

noncomputable def heisenbergVOA (k : ℂ) (hk : k ≠ 0) : VertexOperatorAlgebra Fock where
  toPreVertexAlgebra := heisVA k hk
  conformalVector := ω k
  conformal_mem := omega_mem_gradingZ_two k
  centralCharge := 1
  virasoro := fun m n => conf_virasoro k hk m n
  L0_grading := fun v hv => conf_L0_grading k hk v hv
  translation := fun v => conf_translation k hk v
  vacuum_grade := gradingZ_zero_eq_span_vacuum
  positivity := fun n hn => gradingZ_positivity n hn

theorem heisenberg_central_charge (k : ℂ) (hk : k ≠ 0) :
    (heisenbergVOA k hk).centralCharge = 1 := rfl
```

Hypotheses / status. $k \neq 0$

Theorem 6.18 (The free boson at general central charge). *For every $\lambda \in \mathbb{C}$, the vertex algebra $\mathcal{F}$ of Theorem 6.10 at level 1, with conformal vector ω^{λ}, is a vertex operator algebra of CFT type with central charge $1 - 12\lambda^2$; its vertex algebra, its grading, L_0 and L_{-1} are those of the undeformed algebra. Consequently every complex number is the central charge of a vertex operator algebra structure on $\mathcal{F}$.*

Lean. `VOA.FBgc.freeBosonVOA`, `VOA.FBgc.freeBosonVOA_centralCharge`, `VOA.FBgc.freeBosonVOA_toPreVertexAlgebra`, `VOA.FBgc.freeBosonVOA_L_zero`, `VOA.FBgc.exists_centralCharge`, `VOA.FBgc.centralCharge_surjective`
File `VOA/Examples/FreeBosonGeneralC.lean`.

```
noncomputable def freeBosonVOA (lam : ℂ) : VertexOperatorAlgebra Fock where
  toPreVertexAlgebra := heisVA 1 one_ne_zero
  conformalVector := omegaLD lam
  conformal_mem := omegaLD_mem_gradingZ_two lam
  centralCharge := 1 - 12 * lam ^ 2
  virasoro := fun m n => confLD_virasoro lam m n
  L0_grading := fun v hv => confLD_L0_grading lam v hv
  translation := fun v => confLD_translation lam v
  vacuum_grade := gradingZ_zero_eq_span_vacuum
  positivity := fun n hn => gradingZ_positivity n hn

theorem freeBosonVOA_centralCharge (lam : ℂ) :
    (freeBosonVOA lam).centralCharge = 1 - 12 * lam ^ 2 := rfl

theorem freeBosonVOA_toPreVertexAlgebra (lam : ℂ) :
    (freeBosonVOA lam).toPreVertexAlgebra = heisVA 1 one_ne_zero := rfl

theorem freeBosonVOA_L_zero (lam : ℂ) :
    (freeBosonVOA lam).L 0 = (heisenbergVOA 1 one_ne_zero).L 0

theorem exists_centralCharge (c : ℂ) :
    ∃ lam : ℂ, (freeBosonVOA lam).centralCharge = c

theorem centralCharge_surjective :
    Function.Surjective (fun lam : ℂ => (freeBosonVOA lam).centralCharge) :=
```

```
  fun c => exists_centralCharge c
```

Hypotheses / status. Unconditional

Theorem 6.20 (The grading is the expected one). *For any integral lattice L, $V_L = \bigoplus_n (V_L)_n$ (no hypothesis). If L is even and positive-definite, then for every $\alpha \in L$ the charged vacuum $e^\alpha = 1 \otimes e^\alpha$ lies in the weight space $(V_L)_{\langle\alpha,\alpha\rangle/2}$ of the vertex operator algebra V_L; more generally $p \otimes e^\alpha$ with p of oscillator level w has weight $w + \frac{1}{2}\langle\alpha,\alpha\rangle$.*

Lean. `VOA.LatGen.bigradα_le_gradingLatGen`
File `VOA/Examples/LatticeGenGrading.lean`.

```
theorem bigradα_le_gradingLatGen (w : ℤ) (α : L.carrier) :
    bigradα L (w, α) ≤ gradingLatGen L (w + halfNorm L α)
```

Hypotheses / status. Unconditional

Theorem 6.21 (Lattice vertex operator algebras). *Let L be an even positive-definite integral lattice of rank $r \geq 0$. Then V_L, with the FLM vertex operators, vacuum $1 \otimes e^0$, conformal vector $\omega = \frac{1}{2}\sum_{a=1}^{r} h_a(-1)^2\mathbf{1} \otimes e^0$ and the grading of Theorem 6.20, is a vertex operator algebra of CFT type with central charge r.*

Lean. `VOA.LatGen.latticeVOAGen`, `VOA.LatGen.latticeVOAGen_centralCharge`
File `VOA/Examples/LatticeGenInstance.lean`.

```
noncomputable def latticeVOAGen (hEven : L.Even) (hPos : L.PosDef) :
    VertexOperatorAlgebra (LatModuleGen L) where
  grading := gradingLatGen L
  isInternal := gradingLatGen_isInternal_of_posDef L hEven hPos
  finrank := gradingLatGen_finrank_of_posDef L hEven hPos
  Y := YfullGen L hEven hPos
  vacuum := latVacuumGen L
  vacuum_mem := by
    rw [gradingLatGen_zero_eq_span_vacuum L hEven hPos]
    exact Submodule.mem_span_singleton_self (latVacuumGen L)
  grading_mode := fun u hu v hv j => YfullGen_grading_mode L hEven hPos u hu v hv j
  vacuum_Y := YfullGen_vacuum L hEven hPos
  creation := YfullGen_creation L hEven hPos
  locality := YfullGen_local L hEven hPos
  regularity := YfullGen_regularity L hEven hPos
  conformalVector := omegaLatGen L
  conformal_mem := omegaLatGen_mem L
  centralCharge := (rank L : ℂ)
  virasoro := YfullGen_virasoro L hEven hPos
  L0_grading := fun v hv => YfullGen_L0_grading L hEven hPos v hv
  translation := YfullGen_translation_omega L hEven hPos
  vacuum_grade := gradingLatGen_zero_eq_span_vacuum L hEven hPos
  positivity := fun n hn => gradingLatGen_positivity L hEven hPos n hn

theorem latticeVOAGen_centralCharge (hEven : L.Even) (hPos : L.PosDef) :
    (latticeVOAGen L hEven hPos).centralCharge = (rank L : ℂ) := rfl
```

Hypotheses / status. E, P

Theorem 6.27 (Charge additivity). *Let L be even and positive-definite. The boson currents shift charge by 0, $Y(e^\alpha, z)$ shifts charge by α, and for every state u in the charge-α sector, $Y(u, z)$ shifts charge by α.*

Lean. `VOA.LatGen.alphaLatGen_shiftsCharge`, `VOA.LatGen.Yeαvec_shiftsCharge`, `VOA.LatGen.YfullGen_shiftsCharge`
File `VOA/Examples/LatticeCharge.lean`.

```
theorem alphaLatGen_shiftsCharge (a : Fin (rank L)) :
    ShiftsCharge L (alphaLatGen L a) 0

theorem Yeαvec_shiftsCharge (α : L.carrier) :
    ShiftsCharge L (Yeαvec L α) α

theorem YfullGen_shiftsCharge (hEven : L.Even) (hPos : L.PosDef) (α : L.carrier)
    {u : LatModuleGen L} (hu : u ∈ chargeSec L α) :
    ShiftsCharge L (YfullGen L hEven hPos u) α
```

Hypotheses / status. E, P

Chapter 7

The Virasoro vertex operator algebra: Verma modules, the Kac determinant, screenings

6 of the chapter's 45 numbered statements: its headline results and every definition they use.

Theorem 7.8 (PBW basis of $V(c,0)$, at every c). *For every $c \in \mathbb{C}$ and $n \in \mathbb{N}$, the monomials $L_w\mathbf{1}$ with w sorted of energy n form a basis of $V(c,0)_n$. Hence* $\dim V(c,0)_n = \mathrm{p}_2(n)$*, and the sorted monomials of all energies form a basis of $V(c,0)$. In particular* $\dim V(c,0)_n = 1, 0, 1, 1, 2, 2, 4, 4, 7$ *for* $n = 0, \dots, 8$ *and* $\dim V(c,0)_{16} = 55$.

Lean. `VOA.VirModule.gradedBasis`, `VOA.VirModule.finrank_gradingVir`, `VOA.VirModule.sortedWord_linearIndependent`, `VOA.VirModule.sortedWords`, `VOA.VirModule.mem_sortedWords`, `VOA.VirModule.finrank_gradingVir_eq_card`, `VOA.VirModule.card_sortedWords_le_eight`, `VOA.VirModule.finrank_gradingVir_sixteen`
File `VOA/Examples/VirasoroPartitionBasis.lean`.

```
noncomputable def gradedBasis (c : ℂ) (n : ℕ) : Module.Basis (SortedOfEnergy n) ℂ
    (gradingVir c (n : ℤ)) :=
  Module.Basis.mk (gradedWord_linearIndependent c n) (gradedWord_span c n)

theorem finrank_gradingVir (c : ℂ) (n : ℕ) :
    Module.finrank ℂ (gradingVir c (n : ℤ)) = Fintype.card (SortedOfEnergy n) :=

theorem sortedWord_linearIndependent (c : ℂ) :
    LinearIndependent ℂ (fun w : {w : List ℕ // SortedWord w} => negWordVir c w.1) := by
```

File `VOA/Examples/VirasoroTopSymbol.lean`.

```
def sortedWords (n : ℕ) : Finset (List ℕ) := sortedAux n n n

theorem mem_sortedWords (n : ℕ) (w : List ℕ) :
    w ∈ sortedWords n ↔ (SortedWord w ∧ energyVir w = n) := by

theorem finrank_gradingVir_eq_card (c : ℂ) (n : ℕ) :
    Module.finrank ℂ (gradingVir c (n : ℤ)) = (sortedWords n).card := by

theorem card_sortedWords_le_eight :
    (sortedWords 0).card = 1 ∧ (sortedWords 1).card = 0 ∧ (sortedWords 2).card = 1 ∧
      (sortedWords 3).card = 1 ∧ (sortedWords 4).card = 2 ∧ (sortedWords 5).card = 2 ∧
      (sortedWords 6).card = 4 ∧ (sortedWords 7).card = 4 ∧ (sortedWords 8).card = 7 :=

theorem finrank_gradingVir_sixteen (c : ℂ) : Module.finrank ℂ (gradingVir c 16) = 55 := by
```

Hypotheses / status. None; unconditional

Theorem 7.18 (Graded dimension of $L(c,0)$ is a Gram rank). *Let $G_n(c)$ be the matrix of $\langle\cdot,\cdot\rangle_c$ on $V(c,0)_n$ in the partition basis. Then $\dim L(c,0)_n = \operatorname{rank} G_n(c)$ for every c and n. In particular $\dim L(c,0)_n = \mathrm{p}_2(n)$ if $\det G_n(c) \neq 0$, and $\dim L(c,0)_n < \mathrm{p}_2(n)$ if $\det G_n(c) = 0$.*

Lean. `VOA.VirSimpleGram.finrank_virasoroSimple_grading_partition`, `VOA.VirSimpleGram.finrank_virasoroSimple_lt_of_det_eq_zero`, `VOA.GramMat.canGram`
File `VOA/Examples/VirasoroSimpleGram.lean`.

```
theorem finrank_virasoroSimple_grading_partition (n : ℕ) :
    finrank ℂ ((virasoroSimple c).grading (n : ℤ))
      = (GramMat.canGram (vacVec_ne_zero c) (virasoroVOA_qp c) (n : ℤ)
          (gradedBasis c n)).rank :=

theorem finrank_virasoroSimple_lt_of_det_eq_zero {ι : Type*} [Fintype ι] [DecidableEq ι] (n
↪  : ℤ)
    (e : Basis ι ℂ ((virasoroVOA c).grading n))
    (h : (GramMat.canGram (vacVec_ne_zero c) (virasoroVOA_qp c) n e).det = 0) :
    finrank ℂ ((virasoroSimple c).grading n) < finrank ℂ ((virasoroVOA c).grading n) :=
```

File `VOA/Core/GradedFormMatrix.lean`.

```
noncomputable def canGram : Matrix ι ι ℂ := gramMatrix (canonicalInvariantForm hne hqp) n e
```

Hypotheses / status. None; unconditional

Theorem 7.30 (Singular vectors versus the Gram determinant). *Let $n > 0$.*

1. *If $M(c,h)$ has a singular vector at level n, then $\det G_n(c,h) = 0$; conversely if $\det G_n(c,h) \neq 0$ there is none.*
2. *If $\det G_n(c,h) = 0$ and $\det G_k(c,h) \neq 0$ for all $k < n$, then $M(c,h)$ has a singular vector at level n.*
3. *A singular vector v at level n generates an injective image of $M(c,h+n)$; consequently $(X-h)^{\mathrm{p}(m)}$ divides $\det G_{n+m}(c,X)$ as a polynomial in the highest weight X, for every $m \geq 0$.*
4. *Singular vectors at a given level are unique up to scalar.*

Lean. `VOA.InverseDS.VirasoroVerma.vermaGramMatrix_nonsingular_no_singular`, `VOA.InverseDS.VirasoroVerma.verma_singular_of_first_gram_root`, `VOA.InverseDS.VirasoroVerma.vermaSingularMap`, `VOA.InverseDS.VirasoroVerma.vermaSingularMap_injective`, `VOA.InverseDS.VirasoroVerma.vermaWeightGramPolynomial`, `VOA.InverseDS.VirasoroVerma.vermaWeightDeterminant_dvd_of_singular`, `VOA.InverseDS.VirasoroVerma.verma_singular_eq_smul`
File `VOA/Examples/VirasoroVermaGramPolynomial.lean`.

```
theorem vermaGramMatrix_nonsingular_no_singular (c h : ℂ) (n : ℕ)
    [DecidableEq (VermaPartitions n)] (hn : 0 < n)
    (hd : (vermaGramMatrix c h n).det ≠ 0)
    (v : Verma c h) (hv : v ∈ sortedSpan c h n)
    (h1 : vermaMode c h 1 v = 0) (h2 : vermaMode c h 2 v = 0) : v = 0 := by

theorem verma_singular_of_first_gram_root (c h : ℂ) (n : ℕ)
    [∀ k : ℕ, DecidableEq (VermaPartitions k)]
    (hroot : (vermaGramMatrix c h n).det = 0)
    (hlower : ∀ k < n, (vermaGramMatrix c h k).det ≠ 0) :
    ∃ v : Verma c h, v ≠ 0 ∧ v ∈ sortedSpan c h n ∧
      ∀ m : ℤ, 0 < m → vermaMode c h m v = 0 := by
```

File VOA/Examples/VirasoroVermaSingularEmbedding.lean.

```
noncomputable def vermaSingularMap (c h : ℂ) (n : ℕ) (v : Verma c h)
    (hv : v ∈ sortedSpan c h n)
    (h1 : vermaMode c h 1 v=0) (h2 : vermaMode c h 2 v=0) :
    Verma c (h+(n:ℂ)) →ₗ[ℂ] Verma c h :=
  universalMap (vermaRep c h) v (verma_singular_isHighest c h n v hv h1 h2)

theorem vermaSingularMap_injective (c h : ℂ) (n : ℕ) (v : Verma c h)
    (hv : v ∈ sortedSpan c h n) (hne : v≠0)
    (h1 : vermaMode c h 1 v=0) (h2 : vermaMode c h 2 v=0) :
    Function.Injective (vermaSingularMap c h n v hv h1 h2) :=

noncomputable def vermaWeightGramPolynomial (c : ℂ) (n : ℕ) :
    Matrix (VermaPartitions n) (VermaPartitions n) (Polynomial ℂ) :=
  vermaAlgebraGramMatrix (Polynomial.C c) Polynomial.X n

theorem vermaWeightDeterminant_dvd_of_singular (c h : ℂ) (n m : ℕ)
    [DecidableEq (VermaPartitions (n+m))] (hn : 0<n)
    (v : sortedSpan c h n) (hv : v≠0)
    (h1 : vermaMode c h 1 (v : Verma c h)=0)
    (h2 : vermaMode c h 2 (v : Verma c h)=0) :
    (Polynomial.X-Polynomial.C h)^(Fintype.card (VermaPartitions m)) ∣
      (vermaWeightGramPolynomial c (n+m)).det := by
```

File VOA/Examples/VirasoroSingularUniqueness.lean.

```
theorem verma_singular_eq_smul (c h : ℂ) (n : ℕ) (v w : Verma c h)
    (hv : v ∈ sortedSpan c h n) (hw : w ∈ sortedSpan c h n) (hwne : w ≠ 0)
    (hv1 : vermaMode c h 1 v = 0) (hv2 : vermaMode c h 2 v = 0)
    (hw1 : vermaMode c h 1 w = 0) (hw2 : vermaMode c h 2 w = 0) :
    v = (vermaTopCoordinate c h n v / vermaTopCoordinate c h n w) • w := by
```

Hypotheses / status. Singular vector; unconditional as implication

Theorem 7.34 (Kac formula from singular vectors (conditional form)). *Let $c \in \mathbb{C}$, $n \in \mathbb{N}$ and $t \in \mathbb{C}$ with $\operatorname{Im} t \neq 0$. Suppose that for every label (r,s) with $rs \leq n$ the Verma module $M(c, h_{r,s}(t))$ has a singular vector at level rs. Then*

$$\det G_n(c,X) = K_n \cdot \mathcal{K}_n(t;X).$$

Lean. VOA.InverseDS.VirasoroVerma.fusionKacWeight_injective_positive, VOA.InverseDS.VirasoroVerma.vermaWeightDeterminant_eq_kac_of_singular
File VOA/Examples/VirasoroKacProduct.lean.

```
theorem fusionKacWeight_injective_positive (t : ℂ) (ht : t.im ≠ 0)
    (r s r' s' : ℕ) (hr : 0 < r) (hr' : 0 < r')
    (he : fusionKacWeight r s t = fusionKacWeight r' s' t) : r = r' ∧ s = s' := by

theorem vermaWeightDeterminant_eq_kac_of_singular (c t : ℂ) (n : ℕ) (ht : t.im ≠ 0)
    (hs : ∀ i ∈ kacLabels n,
      ∃ v : sortedSpan c (fusionKacWeight (i.1.val+1) (i.2.val+1) t)
        ((i.1.val+1)*(i.2.val+1)), v ≠ 0 ∧
        vermaMode c (fusionKacWeight (i.1.val+1) (i.2.val+1) t) 1 v = 0 ∧
        vermaMode c (fusionKacWeight (i.1.val+1) (i.2.val+1) t) 2 v = 0) :
    (vermaWeightGramPolynomial c n).det =
      Polynomial.C (∏ w : VermaPartitions n, vermaDiagonalContraction w.val) *
        kacWeightProduct t n := by
```

Hypotheses / status. $\operatorname{Im} t \neq 0$, `hs`; conditional (discharged)

Theorem 7.42 (Kac roots and singular vectors at all labels). *For all $r, s \geq 1$:*

1. *for every integer $m \geq 1$, $\det G_{rs}(c(m^2), h_{r,s}(m^2)) = 0$;*
2. *hence for every $t \neq 0$, $\det G_{rs}(c(t), h_{r,s}(t)) = 0$;*
3. *for every nonreal t, $M(c(t), h_{r,s}(t))$ has a singular vector at level rs.*

Lean. `VOA.VirasoroScreening.integer_screening_kac_root`, `VOA.VirasoroScreening.infinite_integer_screening_roots`, `VOA.VirasoroScreening.screening_kac_root`, `VOA.VirasoroScreening.screening_nonreal_kac_singular`
File `VOA/Examples/VirasoroScreeningKacRoots.lean`.

```
theorem integer_screening_kac_root (r s m : ℕ) (hr : 0 < r) (hs : 0 < s)
    (hm : 0 < m) :
    (vermaGramMatrix (vermaBSACharge ((m:ℂ)^2))
      (fusionKacWeight r s ((m:ℂ)^2)) (r*s)).det = 0 := by

theorem infinite_integer_screening_roots (r s : ℕ) (hr : 0 < r) (hs : 0 < s) :
    ∃ S : Set ℂ, S.Infinite ∧ (∀ t ∈ S, t ≠ 0) ∧
      ∀ t ∈ S, (vermaGramMatrix (vermaBSACharge t)
        (fusionKacWeight r s t) (r*s)).det = 0 := by

theorem screening_kac_root (r s : ℕ) (hr : 0 < r) (hs : 0 < s)
    (t : ℂ) (ht : t ≠ 0) :
    (vermaGramMatrix (vermaBSACharge t) (fusionKacWeight r s t) (r*s)).det = 0 := by

theorem screening_nonreal_kac_singular (t : ℂ) (ht : t.im ≠ 0)
    (r s : ℕ) (hr : 0 < r) (hs : 0 < s) :
    ∃ v : sortedSpan (vermaBSACharge t) (fusionKacWeight r s t) (r*s),
      v ≠ 0 ∧ vermaMode (vermaBSACharge t) (fusionKacWeight r s t) 1 v = 0 ∧
        vermaMode (vermaBSACharge t) (fusionKacWeight r s t) 2 v = 0 := by
```

Hypotheses / status. $r, s > 0$; unconditional

Theorem 7.43 (Kac determinant formula). *For every $t \in \mathbb{C} \setminus \{0\}$, every $n \in \mathbb{N}$ and every $h \in \mathbb{C}$,*

$$\det G_n\big(c(t), h\big) = K_n \prod_{\substack{r,s \geq 1 \\ rs \leq n}} \big(h - h_{r,s}(t)\big)^{\mathrm{p}(n-rs)}.$$

Since every $c \in \mathbb{C}$ is some $c(t)$, this is the Kac determinant for all Virasoro Verma modules.
(The chapter quotes no Lean declaration for this statement; see book p. 343.)
Hypotheses / status. $t \neq 0$; unconditional (check not in this release)

Chapter 8

First-order deformations and the graded first-order rigidity of Virasoro minimal models

7 of the chapter's 19 numbered statements: its headline results and every definition they use.

Theorem 8.4 (Coboundaries are cocycles). *For every linear φ with $\varphi(\mathbf{1}) = 0$, the cochain $\delta\varphi$ satisfies (D1)–(D4). In particular every coboundary is the Y_1 of a first-order deformation.*

Lean. `VOA.Deform.cocycles`, `VOA.Deform.coboundaries_le_cocycles`, `VOA.NonzeroDeform.deltaCochain_mem_cocycles`

```
def cocycles (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ (Cochain 𝒱) where
  carrier := {f | Truncated f ∧ f W.vacuum = 0 ∧ CreationAx W f ∧ JacobiAx W f}

theorem coboundaries_le_cocycles (W : VertexOperatorAlgebra 𝒱) :
    coboundaries W ≤ cocycles W := by

theorem deltaCochain_mem_cocycles (W : VertexOperatorAlgebra 𝒱) {φ : 𝒱 →ₗ[ℂ] 𝒱}
    (hφ0 : φ W.vacuum = 0) : deltaCochain W φ ∈ Deform.cocycles W := by
```

Files `VOA/Core/DeformationGraded.lean`, `VOA/Examples/NonzeroDeformation.lean`. **Hypotheses / status.** None; unconditional

Theorem 8.14 (The gauge line). *Let W be any VOA with $\mathcal{V}_2 = \mathbb{C}\omega$ and let f be a coboundary. Then there is $\mu \in \mathbb{C}$ with $f(\omega)_{(1)}\omega = 2\mu\,\omega$ and $f(\omega)_{(3)}\omega = \mu c\,\mathbf{1}$. Consequently, for a trivial graded deformation the scalars of (8.6) satisfy $\alpha = \mu c$, $\beta = 2\mu$, hence*

$$\delta := 2\alpha - \beta c = 0. \tag{8.1}$$

Lean. `VOA.VirCDef.cmode_conformal_of_isCoboundary`

```
theorem cmode_conformal_of_isCoboundary (W : VertexOperatorAlgebra 𝒱)
    (hV2 : W.grading 2 = ℂ • W.conformalVector) {f : Cochain 𝒱} (hf : IsCoboundary W f) :
    ∃ μ : ℂ,
      cmode f W.conformalVector 1 W.conformalVector = (2 * μ) • W.conformalVector ∧
      cmode f W.conformalVector 3 W.conformalVector = (μ * W.centralCharge) • W.vacuum :=
       ↪ by
```

File `VOA/Examples/VirasoroCDeformation.lean`. **Hypotheses / status.** $\mathcal{V}_2 = \mathbb{C}\omega$; unconditional

Theorem 8.16 (Reconstruction from a closed lift). *Let W be simple and D a first-order deformation. Suppose $S \subseteq \mathcal{V} \times \mathcal{V}$ is mode-closed, grade-closed, contains $(\mathbf{1}, 0)$, does not contain $(0, \mathbf{1})$, and its reduction is surjective. Then D is trivial.*

Lean. `VOA.Deform.Reconstruction.isTrivial_of_closed_lift`

```
theorem isTrivial_of_closed_lift (S : Submodule ℂ (V × V))
    (hsimple : W.IsSimple) (hclosed : ModeClosed D S)
    (hsurj : Function.Surjective (reduction S))
    (hsep : (0, W.vacuum) ∉ S) (hvac : (W.vacuum, 0) ∈ S)
    (hgrade : GradeClosed W S) : D.IsTrivial := by
```

File `VOA/Core/DeformationReconstruction.lean`. **Hypotheses / status.** W simple + 5 lift properties

Theorem 8.17 (Fixed-charge reconstruction). *Let D be a graded first-order deformation of $L(c, 0)$, $c \in \mathbb{C}$ arbitrary, with scalars α, β as in (8.6). If $2\alpha - \beta c = 0$ then D is trivial.*

Lean. `VOA.VirDeformModes.isTrivial_of_zero_central_variation`

```
variable (c : ℂ) (D : FirstOrderDeformation (virasoroSimple c))

theorem isTrivial_of_zero_central_variation (hD : D.IsGraded) (α β : ℂ)
    (ha : D.smul₁ (virasoroSimple c).conformalVector 3
      (virasoroSimple c).conformalVector = α • (virasoroSimple c).vacuum)
    (hb : D.smul₁ (virasoroSimple c).conformalVector 1
      (virasoroSimple c).conformalVector = β • (virasoroSimple c).conformalVector)
    (hz : 2*α-β*c = 0) : D.IsTrivial := by
```

File `VOA/Examples/VirasoroFixedReconstruction.lean`. **Hypotheses / status.** Graded; unconditional

Proposition 8.18 (Lifting criterion). *For symmetric B_0 on finite-dimensional E and any $s \in E$: s lifts to first order if and only if $B_1(s, v) = 0$ for all v in the radical of B_0. Hence, writing $F_1 = \operatorname{rad} B_0$ and $F_2 = \{s \in F_1 : B_1(s, F_1) = 0\}$ (the first two Jantzen layers), F_2 is exactly the set of relations that lift.*

Lean. `VOA.DefForm.radBil`, `VOA.DefForm.lift_iff_pairing_zero`, `VOA.DefForm.F1`, `VOA.DefForm.F2`, `VOA.RigidBridge.F2_lt_F1_of_self_pairing_ne_zero`

```
def radBil (B : E →ₗ[K] E →ₗ[K] K) : Submodule K E := LinearMap.ker B

theorem lift_iff_pairing_zero [FiniteDimensional K E] {B₀ B₁ : E →ₗ[K] E →ₗ[K] K}
    (hB₀ : ∀ x y : E, B₀ x y = B₀ y x) (s : E) :
    (∃ s₁ : E, ∀ v : E, B₁ s v + B₀ s₁ v = 0) ↔ ∀ v ∈ radBil B₀, B₁ s v = 0 := by

def F1 (B₀ : E →ₗ[K] E →ₗ[K] K) : Submodule K E := radBil B₀

def F2 (B₀ B₁ : E →ₗ[K] E →ₗ[K] K) : Submodule K E where
  carrier := {s | s ∈ F1 B₀ ∧ ∀ v ∈ F1 B₀, B₁ s v = 0}
```

```
theorem F2_lt_F1_of_self_pairing_ne_zero (hvac : W.vacuum ≠ 0)
    (D : FirstOrderDeformation W) (hD : D.IsGraded) {k : ℤ} {s : W.grading k}
    (hs : s ∈ DefForm.radBil (shapForm W hvac k))
    (hself : shapForm₁ D hD hvac k s s ≠ 0) :
    DefForm.F2 (shapForm W hvac k) (shapForm₁ D hD hvac k) < DefForm.F1 (shapForm W hvac k)
     ↪ :=
```

Files `VOA/Core/DeformedForm.lean`, `VOA/Core/RigidityBridge.lean`. **Hypotheses / status.** B_0 symmetric, fin.-dim.; unconditional

Theorem 8.20 (Relation obstruction). *Let $c \in \mathbb{C}$. Suppose $s := \sum_i r_i L_{w_i}\mathbf{1} = 0$ in $L(c,0)$ for a finite family of words w_i and coefficients r_i, and suppose the norm polynomial satisfies $P'_{w,r}(c) \neq 0$. Then every graded first-order deformation of $L(c,0)$ has central variation $\delta = 0$, and therefore $L(c,0)$ is graded rigid.*

Lean. `VOA.VirDeformModes.virWord_pairing_obstruction`, `VOA.VirDeformModes.NormalOrdering.simple_normalized_word_tangent`, `VOA.VirDeformModes.NormalOrdering.graded_rigidity_of_relation`

```
theorem virWord_pairing_obstruction {W : VertexOperatorAlgebra V}
    (D : FirstOrderDeformation W) (β : ℂ) (B : InvariantForm W)
    {ι : Type*} [Fintype ι] (w : ι → List ℤ) (r : ι → ℂ)
    (hr : ∑ i, r i • virWord W.L (w i) W.vacuum = 0) :
    B.form W.vacuum
      ((∑ i, r i • virWord (normalizedL D β) ((w i).reverse.map Neg.neg))
        (∑ j, r j • virWord (normalizedL D β) (w j) (W.vacuum,0))).2 = 0 := by

theorem simple_normalized_word_tangent (c : ℂ)
    (D : FirstOrderDeformation (VirSimple.virasoroSimple c)) (hD : D.IsGraded)
    (α β : ℂ)
    (ha : D.smul₁ (VirSimple.virasoroSimple c).conformalVector 3
      (VirSimple.virasoroSimple c).conformalVector = α • (VirSimple.virasoroSimple
      ↪ c).vacuum)
    (hb : D.smul₁ (VirSimple.virasoroSimple c).conformalVector 1
      (VirSimple.virasoroSimple c).conformalVector = β • (VirSimple.virasoroSimple
      ↪ c).conformalVector)
    (w : List ℤ) :
    (VirQuotForm.Bsimp c).form (VirSimple.virasoroSimple c).vacuum
      (virWord (normalizedL D β) w ((VirSimple.virasoroSimple c).vacuum,0)).2 =
        (wordPolynomial w).derivative.eval c * (2*α-β*c) := by

theorem graded_rigidity_of_relation (c : ℂ) (w : ι → List ℤ) (r : ι → ℂ)
    (hr : ∑ i, r i • virWord (VirSimple.virasoroSimple c).L (w i)
      (VirSimple.virasoroSimple c).vacuum = 0)
    (hd : (relationPolynomial w r).derivative.eval c ≠ 0) :
    IsRigidGraded (VirSimple.virasoroSimple c) := by
```

Files `VOA/Examples/VirasoroWordPairing.lean`, `VOA/Examples/VirasoroUniformObstruction.lean`. **Hypotheses / status.** Relation in $L(c,0)$, $P'(c) \neq 0$; unconditional

Theorem 8.22 (Graded first-order rigidity of the Virasoro minimal models). *Let $p, q \geq 2$ be coprime integers and $c_{p,q} = 1 - 6(p-q)^2/(pq)$. Every graded first-order deformation of the simple Virasoro vertex operator algebra $L(c_{p,q},0)$ is trivial: there is a grade-preserving linear map φ with $\varphi(\mathbf{1}) = 0$ such that $u^1_{(n)}v = (\varphi u)_{(n)}v - \varphi(u_{(n)}v) + u_{(n)}\varphi v$ for all u, v, n.*

Lean. `VOA.VirModule.minimal_graded_rigidity`

```
theorem minimal_graded_rigidity (p q : ℕ) (hp : 2 ≤ p) (hq : 2 ≤ q)
    (hcop : p.Coprime q) :
    IsRigidGraded (virasoroSimple (minimalCentralCharge p q)) := by
  apply minimal_rigidity_of_infinite_kac_roots p q hp hq hcop
  intro r s hr hs _
  exact infinite_integer_screening_roots r s hr hs
```

File `VOA/Examples/VirasoroMinimalRigidityComplete.lean`. **Hypotheses / status.** $p, q \geq 2$ coprime; **unconditional**, no `pbw`; non-vacuity machine-checked ($V(c,0)$ not graded rigid)

Chapter 9

Affine $\mathfrak{sl}_2$: construction, Verma modules, Wakimoto realisation, screening, non-simplicity

7 of the chapter's 43 numbered statements: its headline results and every definition they use.

Definition 9.1 ($\mathfrak{sl}_2$, its structure constants and normalised Killing form). $\mathfrak{sl}_2$ is the complex Lie algebra with basis e, f, h and brackets $[h,e] = 2e$, $[h,f] = -2f$, $[e,f] = h$. Writing $[x_a, x_b] = \sum_c f^{ab}{}_c\, x_c$ for $(x_0, x_1, x_2) = (e, f, h)$, the nonzero structure constants are $f^{he}{}_e = 2$, $f^{eh}{}_e = -2$, $f^{hf}{}_f = -2$, $f^{fh}{}_f = 2$, $f^{ef}{}_h = 1$, $f^{fe}{}_h = -1$. The *normalised Killing form* is the symmetric invariant form with $\kappa(e,f) = \kappa(f,e) = 1$, $\kappa(h,h) = 2$ and all other basis pairings zero; it equals the trace form $\mathrm{tr}(xy)$ in the defining representation and $\frac{1}{4}$ of the Killing form $\mathrm{tr}(\mathrm{ad}\,x\ \mathrm{ad}\,y)$.

Lean. `VOA.AffineVOA.colE`, `VOA.AffineVOA.colF`, `VOA.AffineVOA.colH`, `VOA.AffineVOA.fabc`, `VOA.AffineVOA.kappa`, `VOA.AffineVOA.Sl2`, `VOA.AffineVOA.basis`, `VOA.AffineVOA.brComp`, `VOA.AffineVOA.AffSl2`, `VOA.AffineVOA.WakModule`, `VOA.AffineVOA.J`
File `VOA/Examples/AffineSl2.lean`.

```
abbrev colE : Fin 3 := 0
abbrev colF : Fin 3 := 1
abbrev colH : Fin 3 := 2

def fabc : Fin 3 → Fin 3 → Fin 3 → ℂ
  | 2, 0, 0 => 2    -- [h,e] = 2e
  | 0, 2, 0 => -2   -- [e,h] = -2e
  | 2, 1, 1 => -2   -- [h,f] = -2f
  | 1, 2, 1 => 2    -- [f,h] = 2f
  | 0, 1, 2 => 1    -- [e,f] = h
  | 1, 0, 2 => -1   -- [f,e] = -h
  | _, _, _ => 0

def kappa : Fin 3 → Fin 3 → ℂ
  | 0, 1 => 1
  | 1, 0 => 1
  | 2, 2 => 2
  | _, _ => 0

abbrev Sl2 : Type := Fin 3 → ℂ

noncomputable def basis (a : Fin 3) : Sl2 := Pi.single a 1

def brComp (x y : Sl2) (c : Fin 3) : ℂ := ∑ a, ∑ b, x a * y b * fabc a b c
```

File `VOA/Examples/AffineSl2.lean`.

```
noncomputable abbrev AffSl2 : Type := AffineLie Sl2 kappaForm kappaForm_isInvariant

noncomputable abbrev WakModule (k : ℂ) : Type :=
  vacuumSpace kappaForm kappaForm_isInvariant k

noncomputable def J (k : ℂ) (a : Fin 3) (m : ℤ) : Module.End ℂ (WakModule k) :=
  vacuumRho kappaForm kappaForm_isInvariant k (ofLoop kappaForm kappaForm_isInvariant
  ↪ (basis a) m)
```

Theorem 9.2 (The affine bracket on $V^k(\mathfrak{sl}_2)$). *For every $k \in \mathbb{C}$, all colours a, b and all $m, n \in \mathbb{Z}$, as operators on $V^k(\mathfrak{sl}_2)$,*

$$J^a_m J^b_n - J^b_n J^a_m = \sum_c f^{ab}{}_c J^c_{m+n} + \delta_{m+n,0}\, m\, k\, \kappa(a,b)\,\mathrm{id}.$$

Lean. `VOA.AffineVOA.current_comm`
File `VOA/Examples/AffineSl2.lean`.

```
theorem current_comm (k : ℂ) (a b : Fin 3) (m n : ℤ) :
    J k a m * J k b n - J k b n * J k a m
      = (∑ c, fabc a b c • J k c (m + n))
        + (if m + n = 0 then ((m : ℂ) * k * kappa a b) • LinearMap.id else 0) := by
```

Hypotheses / status. None

Theorem 9.9 (Translation covariance of the currents). *Let $k \neq -2$. Then $[L_{-1}, J^c_\mu] = -\mu\, J^c_{\mu-1}$ for every colour c and $\mu \in \mathbb{Z}$. Equivalently, $[L_{-1}, J^c(z)] = \partial_z J^c(z)$.*

Lean. `VOA.AffineVOA.Ltrans`, `VOA.AffineVOA.SugawaraCov.Lcomm_current`, `VOA.AffineVOA.SugawaraCov.Jfield_DCovariant`, `VOA.Reconstruction.DCovariant`
File `VOA/Examples/AffineSl2Reconstruction.lean`.

```
noncomputable def Ltrans (k : ℂ) : End ℂ (WakModule k) := Lsug k (-1)
```

File `VOA/Examples/AffineSl2CurrentCovStruct.lean`.

```
theorem Lcomm_current (k : ℂ) (hk : k + 2 ≠ 0) (c : Fin 3) (μ : ℤ) (x : WakModule k) :
    (Ltrans k * J k c μ - J k c μ * Ltrans k) x = ((-μ : ℤ) : ℂ) • J k c (μ - 1) x := by

theorem Jfield_DCovariant (k : ℂ) (hk : k + 2 ≠ 0) (a : Fin 3) :
    Reconstruction.DCovariant (Ltrans k) (Jfield k a) := by
```

File `VOA/Core/ReconstructionExistence.lean`.

```
def DCovariant (T : End ℂ V) (a : FormalDist (End ℂ V)) : Prop :=
  ∀ m : ℤ, T * a m - a m * T = ((m + 1 : ℤ)) • a (m + 1)
```

Hypotheses / status. $k \neq -2$ (necessary)

Theorem 9.10 (The Sugawara L_0 is the energy). *Let $k \neq -2$. Then $L_0 = -\ell_0$ on $V^k(\mathfrak{sl}_2)$; equivalently $L_0 v = n v$ for $v \in V^k_n$.*

Lean. `VOA.AffineVOA.SugawaraCov.Lsug0_comm_current`, `VOA.AffineVOA.SugawaraCov.Lsug0_eq_negEll0OpA`, `VOA.AffineVOA.SugawaraCov.hYA_L0`
File `VOA/Examples/AffineSl2L0Cov.lean`.

```
theorem Lsug0_comm_current (k : ℂ) (hk : k + 2 ≠ 0) (c : Fin 3) (μ : ℤ) (x : WakModule k) :
    (Lsug k 0 * J k c μ - J k c μ * Lsug k 0) x = ((-μ : ℤ) : ℂ) • J k c μ x := by

theorem Lsug0_eq_negEll0OpA (k : ℂ) (hk : k + 2 ≠ 0) : Lsug k 0 = -ell0OpA k := by

theorem hYA_L0 (k : ℂ) (hk : k + 2 ≠ 0) {n : ℤ} (v : WakModule k) (hv : v ∈ gradingZA k n)
 ↪  :
    mode (YA k hk (omegaA k)) 1 v = (n : ℂ) • v := by
```

Hypotheses / status. $k \neq -2$

Theorem 9.25 (PBW for $V^k(\mathfrak{sl}_2)$, by Wakimoto triangularity). *For every $k \neq -2$ the ordered creation monomials form a basis of $V^k(\mathfrak{sl}_2)$. Consequently the universal C_2-symbol map $\mathbb{C}[e,f,h] \to R_{V^k(\mathfrak{sl}_2)} = V^k/C_2(V^k)$ is injective.*

Lean. `VOA.AffineSl2WakimotoLiLeading.affineSl2PBWBasis`, `VOA.AffineSl2WakimotoLiLeading.universalSymbolAlg_injective`
File `VOA/Examples/AffineSl2WakimotoLiLeading.lean`.

```
theorem affineSl2PBWBasis (k : ℂ) (hk : k + 2 ≠ 0) :
    PBWBasis AffineVOA.kappaForm AffineVOA.kappaForm_isInvariant
      (Pi.basisFun ℂ (Fin 3)) k :=
  pbwBasis_of_wakimotoOrderedCreationState_linearIndependent k hk
    (wakimotoOrderedCreationState_linearIndependent k)

theorem universalSymbolAlg_injective (k : ℂ) (hk : k + 2 ≠ 0) :
    Function.Injective (Sl2Pres.symbolAlg k hk) :=
  AffineSl2UniversalC2PBW.symbolAlg_injective_of_PBWBasis k hk
    (affineSl2PBWBasis k hk)
```

Hypotheses / status. $k \neq -2$

Theorem 9.28 (The level is forced; the singular vector). *(i) Every solution of the singularity system has $b = kg$, $c = -(k+2)g$, $d = 4g$, $a = -(k-2)(k+2)g$ and $(3k+4)g = 0$; a nonzero solution exists iff $3k+4 = 0$. (ii) At $k = -4/3$ the vector*

$$s = 20\,e_{-3}|0\rangle - 12\,e_{-2}h_{-1}|0\rangle - 6\,h_{-2}e_{-1}|0\rangle + 36\,e_{-1}e_{-1}f_{-1}|0\rangle + 9\,e_{-1}h_{-1}h_{-1}|0\rangle$$

has weight 3, satisfies $h_0 s = 2s$, and is killed by every current mode J^a_m, $m \geq 1$, and by e_0. (iii) $s \neq 0$.

Lean. `VOA.Sl2Sing.SingSys_param`, `VOA.Sl2Sing.exists_nonzero_solution_iff`, `VOA.Sl2Sing.sl2Singular`, `VOA.Sl2Sing.sl2Singular_E0`, `VOA.Sl2Sing.sl2Singular_H0`, `VOA.Sl2Sing.sl2Singular_mem`, `VOA.Sl2Sing.sl2Singular_pos`, `VOA.Wak.Gam3_valW_wSing`, `VOA.Wak.sl2Singular_ne_zero`
File `VOA/Examples/AffineSl2Singular.lean`.

```
theorem SingSys_param {k a b c d g : ℂ} (h : SingSys k a b c d g) :
    b = k * g ∧ c = -(k + 2) * g ∧ d = 4 * g ∧ a = -((k - 2) * (k + 2)) * g
      ∧ (3 * k + 4) * g = 0 := by

theorem exists_nonzero_solution_iff (k : ℂ) :
    (∃ a b c d g : ℂ, ¬(a = 0 ∧ b = 0 ∧ c = 0 ∧ d = 0 ∧ g = 0) ∧ SingSys k a b c d g)
      ↔ 3 * k + 4 = 0 := by

noncomputable def sl2Singular : WakModule (-4 / 3 : ℂ) := ansatz (-4 / 3) 20 (-12) (-6) 36
 ↪  9

theorem sl2Singular_E0 : J (-4 / 3 : ℂ) colE 0 sl2Singular = 0 :=
```

```
theorem sl2Singular_H0 : J (-4 / 3 : ℂ) colH 0 sl2Singular = (2 : ℂ) • sl2Singular :=

theorem sl2Singular_mem : sl2Singular ∈ gradingZA (-4 / 3 : ℂ) 3 := ansatz_mem _ _ _ _ _ _

theorem sl2Singular_pos (a : Fin 3) {m : ℤ} (hm : 1 ≤ m) :
    J (-4 / 3 : ℂ) a m sl2Singular = 0 :=
```

File `VOA/Examples/WakimotoFree.lean`.

```
theorem Gam3_valW_wSing (k : ℂ) (hk : k + 2 ≠ 0) :
    Gam 3 (valW k hk wSing) = (-8 : ℂ) • (1 : WakFock) := by

theorem sl2Singular_ne_zero : Sl2Sing.sl2Singular ≠ 0 := by
```

Hypotheses / status. None

Theorem 9.34 (PBW for Verma modules, at every (k,λ)). *For every $k,\lambda \in \mathbb{C}$: $v_\lambda \neq 0$; the* occupation words *— one fixed product of lower modes for each finite multiset of lower modes — form a basis of $M(k,\lambda)$; and for every nonzero $v \in M(k,\lambda)$ the vectors (occupation word)·v are linearly independent.*

Lean. `VOA.AffineVOA.HighestModule.highestVector_ne_zero`, `VOA.AffineVOA.HighestModule.occupationWords_independent`, `VOA.AffineVOA.HighestModule.occupationBasis`, `VOA.Wak.Charged.lowerLetter`, `VOA.Wak.Charged.occupationWord`, `VOA.AffineVOA.HighestModule.descendants_independent`
File `VOA/Examples/AffineSl2Verma.lean`.

```
theorem highestVector_ne_zero (k h : ℂ) : highestVector k h≠0 := by

theorem occupationWords_independent (k h : ℂ) :
    LinearIndependent ℂ (fun d : Occupation => currentWord (mode k h)
      (occupationWord d) (highestVector k h)) := by
```

File `VOA/Examples/AffineSl2VermaStraightening.lean`.

```
def occupationBasis (k h : ℂ) : Module.Basis Occupation ℂ (Verma k h) :=
  Module.Basis.mk (occupationWords_independent k h)
    (by change ⊤≤occupationSpan k h; rw [occupationSpan_eq_top])
```

File `VOA/Examples/AffineSl2HighestWords.lean`.

```
def lowerLetter (v : Fin 3×ℕ) : Fin 3×ℤ :=
  (v.1,if v.1=1 then -(v.2 : ℤ) else -(v.2 : ℤ)-1)

def occupationWord (d : Occupation) : List (Fin 3×ℤ) := d.toMultiset.toList.map lowerLetter
```

File `VOA/Examples/AffineSl2VermaDescendants.lean`.

```
theorem descendants_independent (k h : ℂ) (v : Verma k h) (hv : v≠0) :
    LinearIndependent ℂ (fun d : Occupation => currentWord (mode k h) (occupationWord d) v)
     ↪  := by
```

Hypotheses / status. None

Chapter 10

Graded first-order rigidity of affine $\mathfrak{sl}_2$: Linshaw–Qi and admissible levels

6 of the chapter's 12 numbered statements: its headline results and every definition they use.

Theorem 10.8 (Whitehead's second lemma for $\mathfrak{sl}_2$). *If a complex Lie algebra $\mathfrak{g}$ has a basis (e, h, f) forming an $\mathfrak{sl}_2$-triple, then every 2-cocycle $\omega \in Z^2(\mathfrak{g}, \mathfrak{g})$ is $d_{12}\psi$ for some ψ; and $Z(\mathfrak{g}) = 0$.*

Lean. `VOA.Sl2Whitehead.whitehead2_of_sl2Basis`, `VOA.VertexOperatorAlgebra.weightOneH2Vanishes_of_sl2Basis`

```
theorem whitehead2_of_sl2Basis (b : Basis (Fin 3) ℂ g) (ht : IsSl2Triple (b 1) (b 0) (b 2))
    (ω : twoCochain ℂ g g) (hω : ω ∈ twoCocycle ℂ g g) :
    ∃ ψ : oneCochain ℂ g g, d₁₂ ℂ g g ψ = ω := by
```

```
theorem weightOneH2Vanishes_of_sl2Basis (W : VertexOperatorAlgebra 𝒱)
    (b : Basis (Fin 3) ℂ ₜ(W.grading 1)) (ht : IsSl2Triple (b 1) (b 0) (b 2)) :
    WeightOneH2Vanishes W :=
  fun ω hω => Sl2Whitehead.whitehead2_of_sl2Basis b ht ω hω
```

File `VOA/Core/WeightOneCohomology.lean`. **Hypotheses / status.** $\mathfrak{sl}_2$-triple basis

Theorem 10.10 (split form of Linshaw–Qi Theorem 5.1). *For any VOA W with centreless V_1: if (Theorem 3.1) every graded regular deformation is trivial, (Piece 1) $H^2(V_1, V_1) = 0$, and (Piece 2) every graded deformation in pole form has vanishing pole on V_1, then W is graded rigid.*

Lean. `VOA.LQ43.isRigidGraded_of_split`
File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
theorem isRigidGraded_of_split {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (W : VertexOperatorAlgebra 𝒱) (hZ : W.CentrelessWeightOne)
    (h31 : RegularImpliesTrivialGraded W) (h1 : W.WeightOneH2Vanishes)
    (h2 : NormalisedPoleVanishes W) : Deform.IsRigidGraded W := by
```

Hypotheses / status. `hZ`,`h31`,`h1`,`h2`

Theorem 10.11 (Graded first-order rigidity of $L_{-4/3}(\mathfrak{sl}_2)$; Linshaw–Qi, Theorem 5.1). *Every graded first-order deformation of $L_{-4/3}(\mathfrak{sl}_2)$ is trivial.*

Lean. `VOA.LQ43.L43_graded_rigidity`

```
theorem L43_graded_rigidity : Deform.IsRigidGraded L43 :=
  L43_isRigidGraded_of_normalisedPoleVanishes L43_normalisedPoleVanishes
```

File `VOA/Examples/AffineSl2RigidityComplete.lean`. **Hypotheses / status.** None

Theorem 10.12 (relation obstruction). *Let $k \neq 0, -2$. Suppose $\sum_i c_i J_{u_i}\mathbf{1} = 0$ and $\sum_j d_j J_{v_j}\mathbf{1} = 0$ in $L_k(\mathfrak{sl}_2)$ (two* relations*), and let $P(C) = \sum_{i,j} c_i d_j\, G_{u_i,v_j}(C)$ be their Gram polynomial, where $G_{u,v}(k) = (J_u\mathbf{1} \mid J_v\mathbf{1})$. If $P'(k) \neq 0$, then $L_k(\mathfrak{sl}_2)$ is graded rigid.*

Lean. `VOA.AffineSl2GeneralRigidity.graded_rigidity_of_relations`
File `VOA/Examples/AffineSl2UniformObstruction.lean`.

```
theorem graded_rigidity_of_relations {ι κ : Type*} [Fintype ι] [Fintype κ]
    (u : ι → List (Fin 3 × ℤ)) (c : ι → ℂ)
    (v : κ → List (Fin 3 × ℤ)) (d : κ → ℂ)
    (hc : ∑ i, c i • currentWord k hk (u i) (simple k hk).vacuum = 0)
    (hd : ∑ j, d j • currentWord k hk (v j) (simple k hk).vacuum = 0)
    (hp : (relationPairPolynomial u c v d).derivative.eval k ≠ 0) :
    IsRigidGraded (simple k hk) := by
```

Hypotheses / status. Relations, $P'(k) \neq 0$

Lemma 10.13 (kernel pair). *Let $M(X)$ be a square matrix of polynomials over an integral domain, and z a simple root of* $\det M$*: $\det M(z) = 0 \neq (\det M)'(z)$. Then there are vectors r, s with $r^{\mathsf{T}} M(z) = 0$, $M(z)s = 0$ and $r^{\mathsf{T}} M'(z) s \neq 0$.*

Lean. `VOA.PolynomialMatrix.kernel_pair_of_simple_root`, `VOA.AffineSl2GeneralRigidity.graded_rigidity_of_vacuum_simple_root`
File `VOA/Core/PolynomialMatrixSimpleRoot.lean`.

```
theorem kernel_pair_of_simple_root [NoZeroDivisors R]
    (M : Matrix ι ι R[X]) (z : R)
    (hz : M.det.eval z = 0) (hd : M.det.derivative.eval z ≠ 0) :
    ∃ r c : ι → R,
      (∀ j, ∑ i, r i * (M i j).eval z = 0) ∧
      (∀ i, ∑ j, (M i j).eval z * c j = 0) ∧
      (∑ i, ∑ j, (r i * c j) * (M i j).derivative.eval z) ≠ 0 := by
```

```
theorem graded_rigidity_of_vacuum_simple_root (hk0 : k≠0) (n : ℕ) (c : ℤ)
    (hz : (vacuumChevalleyPolynomial n c).det.eval k=0)
    (hd : (vacuumChevalleyPolynomial n c).det.derivative.eval k≠0) :
    Deform.IsRigidGraded (simple k hk) := by
```

Hypotheses / status. Simple root

Theorem 10.15 (graded first-order rigidity at negative admissible levels). *Let $p, q \in \mathbb{N}$ with $p \geq 2$, $q \geq 1$, $\gcd(p, q) = 1$ and $p < 2q$, and let $k = p/q - 2$. Then every graded first-order deformation of the simple affine vertex operator algebra $L_k(\mathfrak{sl}_2)$ is trivial: for every Y_1 satisfying the order-t vertex-algebra axioms and the weight rule, there is a linear $\varphi : L_k \to L_k$ preserving each weight space, with $\varphi\mathbf{1} = 0$ and $Y_1 = \delta\varphi$.*

Lean. `VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity`

```
theorem negative_admissible_graded_rigidity (p q : ℕ) (hp : 2≤p) (hq : 0<q)
    (hcop : p.Coprime q) (hneg : p<2*q) :
    Deform.IsRigidGraded (simple ((p : ℂ)/q-2) (admissible_level_noncritical p q hp hq)) :=
```

File `VOA/Examples/AffineSl2AdmissibleRigidity.lean`. **Hypotheses / status.** $2 \leq p < 2q$, $q \geq 1$, coprime

Chapter 11

Filtrations, the C_2 algebra and presentations

8 of the chapter's 58 numbered statements: its headline results and every definition they use.

Theorem 11.8 (Li's multiplicativity estimates). *Let $x \in F^pV$, $y \in F^qV$. Then*

1. *for every $N \in \mathbb{Z}$, $x_{(N)}y \in F^{p+q-N-1}V$ (indices ≤ 0 meaning V);*
2. *$x_{(-1)}y \in F^{p+q}V$ and, more generally, $x_{(-d-1)}y \in F^{p+q+d}V$ for $d \in \mathbb{N}$;*
3. *for $n \in \mathbb{N}$ with $n \leq p+q$, $x_{(n)}y \in F^{p+q-n}V$;*
4. *$Tx \in F^{p+1}V$; and $F^0V = V$.*

Lean. `VOA.adjointGfil_smul_mem`, `VOA.liFiltration_smul_negSucc_mem`, `VOA.liFiltration_smul_nonneg_mem`, `VOA.liFiltration_L_neg_one_mem`, `VOA.liFiltration_zero_eq_top`
File `VOA/Filtered/LiMultiplicativity.lean`.

```
theorem adjointGfil_smul_mem (W : VertexOperatorAlgebra V)
    (hLW : LWGenerated (adjointWeakModule W) ({W.vacuum} : Set V))
    {p q N : ℤ} {x y : V}
    (hx : x ∈ Gfil (adjointWeakModule W) ({W.vacuum} : Set V) p)
    (hy : y ∈ Gfil (adjointWeakModule W) ({W.vacuum} : Set V) q) :
    W.smul x N y ∈
      Gfil (adjointWeakModule W) ({W.vacuum} : Set V) (p + q - N - 1) := by

theorem liFiltration_smul_negSucc_mem (W : VertexOperatorAlgebra V)
    {p q : ℕ} (d : ℕ) {x y : V}
    (hx : x ∈ liFiltration W p) (hy : y ∈ liFiltration W q) :
    W.smul x (-(d : ℤ) - 1) y ∈ liFiltration W (p + q + d) := by

theorem liFiltration_smul_nonneg_mem (W : VertexOperatorAlgebra V)
    {p q n : ℕ} (hn : n ≤ p + q) {x y : V}
    (hx : x ∈ liFiltration W p) (hy : y ∈ liFiltration W q) :
    W.smul x (n : ℤ) y ∈ liFiltration W (p + q - n) := by

theorem liFiltration_L_neg_one_mem (W : VertexOperatorAlgebra V) {p : ℕ} {x : V}
    (hx : x ∈ liFiltration W p) : W.L (-1) x ∈ liFiltration W (p + 1) := by

theorem liFiltration_zero_eq_top (W : VertexOperatorAlgebra V) : liFiltration W 0 = ⊤ := by
```

Hypotheses / status. Unconditional

Theorem 11.10 (Li: $F^1V = C_2(V)$; $\mathrm{gr}_0^F V = R_V$). *$C_2(V) = F^1V$. Consequently the degree-zero layers $\bigoplus_N \mathrm{gr}^F_{0,N} V$ form the algebra $R_V = V/C_2(V)$, and the class map $V_N \to \mathrm{gr}^F_{0,N} V$ kills $C_2(V) \cap V_N$.*
Lean. `VOA.O₂_le_liFiltration_one`, `VOA.liDegreeZeroClass`, `VOA.liDegreeZeroClass_eq_zero_of_mem_O₂`, `VOA.liFiltration_one_le_O₂`, `VOA.liFiltration_one_eq_O₂`
File `VOA/Filtered/ClassicalFreeness.lean` (the inclusion $C_2 \subseteq F^1$).

```
theorem O₂_le_liFiltration_one (W : VertexOperatorAlgebra V) :
    O₂ W ≤ liFiltration W 1 := by

noncomputable def liDegreeZeroClass (W : VertexOperatorAlgebra V) (N : ℕ)
    (x : W.grading (N : ℤ)) : liAssociatedGraded W :=
  DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (0, N)

theorem liDegreeZeroClass_eq_zero_of_mem_O₂ (W : VertexOperatorAlgebra V) (N : ℕ)
    (x : W.grading (N : ℤ)) (hx : x.1 ∈ O₂ W) :
    liDegreeZeroClass W N x = 0 := by
```

File `VOA/Filtered/LiC2Bridge.lean` (the reverse inclusion and the equality, now library theorems):

```
theorem liFiltration_one_le_O₂ (W : VertexOperatorAlgebra V) :
    liFiltration W 1 ≤ O₂ W := by

theorem liFiltration_one_eq_O₂ (W : VertexOperatorAlgebra V) :
    liFiltration W 1 = O₂ W :=
```

Hypotheses / status. None; equality proved in library (`LiC2Bridge.lean`)

Theorem 11.16 (Generator words hit every Li layer). *Let $a^1, \dots, a^g \in V$ be homogeneous of weights w_i, and suppose*

$$V = \mathrm{span}\{a^1, \dots, a^g, \mathbf{1}\} + C_2(V).$$

Let I be any ideal of relations vanishing on the degree-zero classes $[a^i] \in \mathrm{gr}_0^F V$. Then for every $p \le m$ and every N, every element of the layer $\mathrm{gr}^F_{p,N} V$ is in the image of the Li jet map with $x_i = [a^i]$. Moreover the jet monomial $\prod x_{i_k}^{(r_k)}$ maps to the class of the state $a^{i_1}_{(-r_1-1)} \cdots a^{i_s}_{(-r_s-1)}\mathbf{1}$, exactly.
Lean. `VOA.genWord`, `VOA.pairsLevel`, `VOA.dividedTranslateState`, `VOA.prod_dividedIterate_degreeZeroClass_eq_generatorWord`, `VOA.liPresentedJetMap_hits_homogeneous_class_of_generators`
File `VOA/Filtered/LiGeneratorWords.lean`; generator words `VOA/Modularity/OrderedSpan.lean` (Chapter 15).

```
def genWord {g : ℕ} (gen : Fin g → 𝒱) (wt : Fin g → ℕ) (l : List (Fin g × ℕ)) :
    List (Letter 𝒱) :=
  l.map (fun q => (gen q.1, wt q.1, q.2))

def pairsLevel {g : ℕ} (l : List (Fin g × ℕ)) : ℕ := (l.map Prod.snd).sum

noncomputable def dividedTranslateState (W : VertexOperatorAlgebra V) (r : ℕ) (a : V) : V
 ↪  :=
  ((r.factorial : ℂ)⁻¹) • ((W.toPreVertexAlgebra.T ^ r) a)

theorem prod_dividedIterate_degreeZeroClass_eq_generatorWord
    (W : VertexOperatorAlgebra V)
    {g : ℕ} (gen : Fin g → V) (wt : Fin g → ℕ)
    (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ)) (l : List (Fin g × ℕ)) :
    (l.map fun q =>
```

```
      FiniteJet.dividedIterate (liAssociatedGradedTranslation W) q.2
        (liDegreeZeroClass W (wt q.1) ⟨gen q.1, hgen q.1⟩)).prod =
      liHomogeneousClass W (pairsLevel l) (wordDeg (genWord gen wt l))
        ⟨wordAct (adjointWeakModule W) (genWord gen wt l) W.vacuum,
          generatorWord_mem_liWeightPiece W gen wt hgen l⟩ := by

theorem liPresentedJetMap_hits_homogeneous_class_of_generators
    (W : VertexOperatorAlgebra V) (m : ℕ)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (gen : Fin g → V) (wt : Fin g → ℕ)
    (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom)
    (hdec : (τ : Submodule ℂ V) =
      Submodule.span ℂ (Set.range gen ∪ {W.vacuum}) ⊔ O₂ W)
    (p N : ℕ) (hp : p ≤ m) :
    ∀ z : liWeightLayer W p N,
      ∃ y : FiniteJet.PresentedJet (R := ℂ) m I,
```

Hypotheses / status. hdec: **forces C_2-cofinite**

Theorem 11.19 (Current words and Li's filtration for $V^k(\mathfrak{sl}_2)$). *Let $k \neq -2$ and let $C_p \subseteq V^k(\mathfrak{sl}_2)$ be the span of the current words $J^{a_1}_{(-r_1-1)} \cdots J^{a_s}_{(-r_s-1)}\mathbf{1}$ with $r_1 + \cdots + r_s \geq p$. Then $F^pV^k(\mathfrak{sl}_2) = C_p$ for all p. For any graded ideal I, the Li filtration of the quotient $V^k(\mathfrak{sl}_2)/I$ is the image of that of $V^k(\mathfrak{sl}_2)$, and equals the image of C_p.*

Lean. VOA.AffineVOA.CurrentLi.currentWordPiece, VOA.AffineVOA.CurrentLi.liFiltration_eq_currentWordPiece, VOA.AffineVOA.CurrentLi.quotientLiFiltration_eq_map, VOA.AffineVOA.CurrentLi.quotientLiFiltration_eq_currentWordPiece
File VOA/Filtered/AffineCurrentLi.lean.

```
def currentWordPiece (k : ℂ) (p : ℕ) : Submodule ℂ (WakModule k) :=
  Submodule.span ℂ
    {x | ∃ w : List (Fin 3 × ℕ), p ≤ currentWordLevel w ∧ wordStateA k w = x}

theorem liFiltration_eq_currentWordPiece (k : ℂ) (hk : k + 2 ≠ 0) (p : ℕ) :
    liFiltration (affineSl2VOA_closed k hk) p = currentWordPiece k p :=

theorem quotientLiFiltration_eq_map (p : ℕ) :
    liFiltration (QuotVOA.quotientVOA W I hI hgr) p =
      (liFiltration W p).map I.mkQ :=

theorem quotientLiFiltration_eq_currentWordPiece (p : ℕ) :
    liFiltration (QuotVOA.quotientVOA (affineSl2VOA_closed k hk) I hI hgr) p =
      quotientCurrentWordPiece k I p :=
```

Hypotheses / status. $k \neq -2$; unconditional

Definition 11.20 (Generator words and the word filtration). Let V be a VOA and $a^1, \dots, a^g \in V$ a finite family of states, with weight labels $w_1, \dots, w_g \in \mathbb{N}$. Encode a word by a finite list $l = [(i_1, r_1), \dots, (i_s, r_s)]$ of pairs $(i, r) \in \{1, \dots, g\} \times \mathbb{N}$. Its *state* is

$$a_l := a^{i_1}_{(-r_1-1)} a^{i_2}_{(-r_2-1)} \cdots a^{i_s}_{(-r_s-1)}\mathbf{1},$$

and its *level* is $|l| := r_1 + \cdots + r_s$ (the total number of derivatives, since $a_{(-r-1)}b = (T^r a/r!)_{(-1)}b$). For $p \in \mathbb{N}$ the *word piece* is

$$\mathcal{W}_p := \operatorname{span}_{\mathbb{C}}\{\, a_l : |l| \geq p \,\} \subseteq V.$$

The family *spans Li's filtration* if $F^pV \subseteq \mathcal{W}_p$ for every $p \in \mathbb{N}$.

Lean. `VOA.genWordState`, `VOA.genWordPiece`, `VOA.GeneratorWordsSpanLiFiltration`
File `VOA/Filtered/GeneratorWordLiSurjectivity.lean`.

```
abbrev genWordState (l : List (Fin g × ℕ)) : V :=
  wordAct (adjointWeakModule W) (genWord gen wt l) W.vacuum

def genWordPiece (p : ℕ) : Submodule ℂ V :=
  Submodule.span ℂ {u | ∃ l : List (Fin g × ℕ), p ≤ pairsLevel l ∧ genWordState W gen wt l
   ↪  = u}

def GeneratorWordsSpanLiFiltration : Prop :=
  ∀ p : ℕ, liFiltration W p ≤ genWordPiece W gen wt p
```

Hypotheses / status. Definition; `wt` unused until `hgen`

Theorem 11.23 (Spanning of Li's filtration gives surjective jet maps). *Let V be a VOA, let $a^1, \dots, a^g$ be homogeneous of weights $w_1, \dots, w_g$, and assume that they span Li's filtration (Definition 11.20). Let $x_i := [a^i] \in \mathrm{gr}^F_{0,w_i} V$ be their degree-zero classes, and let $I \subseteq \mathbb{C}[x_1, \dots, x_g]$ be any ideal with $f([a^1], \dots, [a^g]) = 0$ for all $f \in I$. Then:*

1. *for all p, N, the layer $\mathrm{gr}^F_{p,N} V$ is spanned by the classes of the words a_l with $|l| = p$ and $\sum_k (w_{i_k} + r_k) = N$;*
2. *for all p, N, every element of $\mathrm{gr}^F_{p,N} V$ is the image of an element of $J_p(\mathbb{C}[x]/I)$ under the order-p Li jet map;*
3. *for all p, N, the bidegree jet map $J_p(\mathbb{C}[x]/I)_{(p,N)} \to \mathrm{gr}^F_{p,N} V$ (order $m = p$, base weights w) is surjective;*
4. *the infinite Li jet map $J_\infty(\mathbb{C}[x]/I) \to \mathrm{gr}^F V$ is surjective.*

Lean. `VOA.genWordClassSet`, `VOA.genWordClassSet_spans_layer`, `VOA.liPresentedJetMap_hits_homogeneous_class_of_genWordSpan`, `VOA.liPresentedJetBidegreeMap_surjective_of_genWordSpan`, `VOA.liInfinitePresentedJetMap_surjective_of_genWordSpan`
File `VOA/Filtered/GeneratorWordLiSurjectivity.lean`.

```
def genWordClassSet (p N : ℕ) : Set (liAssociatedGraded W) :=
  {x | ∃ l : List (Fin g × ℕ),
      pairsLevel l = p ∧ wordDeg (genWord gen wt l) = N ∧
      x = liHomogeneousClass W (pairsLevel l) (wordDeg (genWord gen wt l))
        ⟨genWordState W gen wt l, generatorWord_mem_liWeightPiece W gen wt hgen l⟩}

theorem genWordClassSet_spans_layer (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (p N : ℕ) (z : liWeightLayer W p N) :
    DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p, N) z ∈
      Submodule.span ℂ (genWordClassSet W gen wt hgen p N) := by

theorem liPresentedJetMap_hits_homogeneous_class_of_genWordSpan
    (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom) :
    ∀ p N (z : liWeightLayer W p N),
      ∃ y : FiniteJet.PresentedJet (R := ℂ) p I,
        liPresentedJetMap W p (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩) I hI y
         ↪  =
```

```
          DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p, N) z := by

theorem liPresentedJetBidegreeMap_surjective_of_genWordSpan
    (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom)
    (p N : ℕ) :
    Function.Surjective
      (liPresentedJetBidegreeMap W p (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩)
        I hI wt p N) :=

theorem liInfinitePresentedJetMap_surjective_of_genWordSpan
    (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom) :
    Function.Surjective
      (liInfinitePresentedJetMap W (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩) I
       ↪ hI) := by
```

Hypotheses / status. `hgen`, `hspan` (= Li spanning), `hI`; unconditional; any VOA, no C_2-cofiniteness; comparison with `hdec` on paper only (Rem. 11.24)

Theorem 11.26 (Word spanning plus the tracked estimate give spanning of Li's filtration). *If the words a_l span V and the family satisfies the tracked estimate, then it spans Li's filtration, i.e. $F^pV \subseteq \mathcal{W}_p$ for every p.*

Lean. `VOA.generatorWordsSpanLiFiltration_of_tracked`
File `VOA/Filtered/GeneratorCurrentLiSpanning.lean`.

```
theorem generatorWordsSpanLiFiltration_of_tracked (htrack : GeneratorModeTracked W gen wt)
    (hspan : Submodule.span ℂ (Set.range (genWordState W gen wt)) = ⊤) :
    GeneratorWordsSpanLiFiltration W gen wt := by
```

Hypotheses / status. `htrack`, `hspan` (here = word spanning, $\mathcal{W}_0 = V$); unconditional

Definition 11.38 (OPE data: De Sole–Kac Definition 3.1, free-generator form). An *OPE datum* on generators ι consists of weights $\Delta_i \geq 1$, parities $p_i \in \mathbb{Z}/2$, and elements $a^i_{(n)}a^j \in \mathcal{T}(R)$ for $n \in \mathbb{N}$ (the singular OPE table) such that $a^i_{(n)}a^j \in \mathrm{filt}(\Delta_i + \Delta_j - n - 1)$. In particular $a^i_{(n)}a^j = 0$ for $n \geq \Delta_i + \Delta_j$.

Lean. `VOA.NonlinearConformal.OPEData`, `VOA.NonlinearConformal.OPEData.ope_eq_zero`, `VOA.NonlinearConformal.virasoroOPE`
File `VOA/Core/NonlinearConformal.lean`.

```
structure OPEData (ι : Type u) where
  wt : ι → ℕ
  wt_pos : ∀ i, 0 < wt i
  par : ι → ZMod 2
  ope : ι → ι → ℕ → TR ι
  ope_wt : ∀ i j (n : ℕ), ope i j n ∈ filt wt ((wt i : ℤ) + (wt j : ℤ) - (n : ℤ) - 1)

theorem ope_eq_zero (i j : ι) {n : ℕ} (h : D.wt i + D.wt j ≤ n) : D.ope i j n = 0 := by

noncomputable def virasoroOPE (c : ℂ) : OPEData Unit where
  wt := fun _ => 2
  wt_pos := fun _ => by norm_num
```

```
  par := fun _ => 0
  ope := fun _ _ n =>
    if n = 0 then gv ((), 1)
    else if n = 1 then (2 : ℂ) • gv ((), 0)
    else if n = 3 then (c / 2) • (1 : TR Unit)
    else 0
  ope_wt := by
    intro i j n
    split_ifs with h0 h1 h3
    · subst h0
      exact gv_mem_filt _ (by norm_num [wtIdx])
    · subst h1
      exact Submodule.smul_mem _ _ (gv_mem_filt _ (by norm_num [wtIdx]))
    · subst h3
      exact Submodule.smul_mem _ _ (one_mem_filt _ (by norm_num))
    · exact Submodule.zero_mem _
```

Hypotheses / status. Definition

Part III

Representation theory

Chapter 12

Modules, Zhu's algebra, and the rationality hierarchy

18 of the chapter's 43 numbered statements: its headline results and every definition they use.

Definition 12.1 (Weak module; Gannon Def. 5.3.1, FHL). Let V be a VOA. A *weak V-module* is a vector space M together with a linear map $Y_M : V \to \mathrm{End}(M)[[z^{\pm 1}]]$, $Y_M(u,z) = \sum_{n\in\mathbb{Z}} u^M_{(n)} z^{-n-1}$, such that

(M1) $Y_M(\mathbf{1},z) = \mathrm{id}_M$;

(M2) each $Y_M(u,z)$ is a field: for all $u \in V$ and $x \in M$, $u^M_{(n)}x = 0$ for $n \gg 0$;

(M3) the Borcherds (Jacobi) identity holds: for all $u, v \in V$, $x \in M$ and $\ell, m, n \in \mathbb{Z}$,

$$\sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u^M_{(\ell+m-i)}v^M_{(n+i)}x - (-1)^\ell v^M_{(\ell+n-i)}u^M_{(m+i)}x\Big) = \sum_{i\ge 0}\binom{m}{i}(u_{(\ell+i)}v)^M_{(m+n-i)}x.$$

Lean. `VOA.WeakModule`, `VOA.WeakModule.act`
File `VOA/Modules/WeakModule.lean`.

```
structure WeakModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] where
  YM : 𝒱 →ₗ[ℂ] (End ℂ M)⟦z^±⟧
  vacuum_YM : YM W.vacuum = 1
  truncation : ∀ (u : 𝒱) (x : M), ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (YM u) n x = 0
  jacobi : ∀ (u v : 𝒱) (x : M) (ℓ m n : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (mode (YM u) (ℓ + m - i) (mode (YM v) (n + i) x)
          - negOnePow ℓ • mode (YM v) (ℓ + n - i) (mode (YM u) (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • mode (YM (W.smul u (ℓ + i) v)) (m + n - i) x

def act (A : WeakModule W M) (u : 𝒱) (n : ℤ) (x : M) : M := mode (A.YM u) n x
```

Hypotheses / status. Definition; $\sum^{\mathrm{f}}$ finite

Definition 12.4 (Admissible module; Zhu 1996, §1.4; Dong–Li–Mason 1998, Def. 2.2). An *admissible V-module* is a weak module M together with a decomposition $M = \bigoplus_{k\in\mathbb{N}} M(k)$ such that for every homogeneous $a \in V_w$, every $p \in \mathbb{Z}$ and every k,

$$a^M_{(p)}M(k) \subseteq M(k + w - p - 1)$$

(with $M(j) = 0$ for $j < 0$). It is *irreducible* if $M \neq 0$ and its only weak submodules are 0 and M; *completely reducible* if every weak submodule has a complementary weak submodule.

Lean. `VOA.AdmissibleModule`, `VOA.AdmissibleModule.IsIrreducible`, `VOA.AdmissibleModule.IsCompletelyReducible`, `VOA.adjointAdmissibleModule`
File `VOA/Modules/AdmissibleModule.lean`.

```
structure AdmissibleModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] extends WeakModule W M where
  lev : ℤ → Submodule ℂ M
  lev_eq_bot_of_neg : ∀ k : ℤ, k < 0 → lev k = ⊥
  isInternal : DirectSum.IsInternal lev
  act_lev : ∀ {{w p k : ℤ}} {{a : 𝒱}}, a ∈ W.grading w → ∀ {{x : M}}, x ∈ lev k →
    mode (YM a) p x ∈ lev (k + (w - p - 1))

def IsIrreducible (N : AdmissibleModule W M) : Prop :=
  Nontrivial M ∧ ∀ P : WeakModule.Submodule' N.toWeakModule, P.carrier = ⊥ ∨ P.carrier = ⊤

def IsCompletelyReducible (N : AdmissibleModule W M) : Prop :=
  ∀ P : WeakModule.Submodule' N.toWeakModule,
    ∃ Q : WeakModule.Submodule' N.toWeakModule, IsCompl P.carrier Q.carrier

def adjointAdmissibleModule (W : VertexOperatorAlgebra 𝒱) : AdmissibleModule W 𝒱 where
  toWeakModule := adjointWeakModule W
  lev := W.grading
```

Hypotheses / status. Definitions

Definition 12.5 (Ordinary module; irreducible; completely reducible; Gannon Def. 5.3.1(c)). A *V-module* (an *ordinary* module) is a weak module M together with a decomposition $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$ such that $L_0^M = \omega_{(1)}^M$ acts on M_α as the scalar α, each M_α is finite-dimensional, and the real parts of the weights α with $M_\alpha \neq 0$ are bounded below. It is *irreducible* if $M \neq 0$ and its only weak submodules are 0 and M, and *completely reducible* if every weak submodule has a complementary weak submodule.

Lean. `VOA.OrdVModule`, `VOA.OrdVModule.IsIrreducible`, `VOA.OrdVModule.IsCompletelyReducible`
File `VOA/Modules/WeakModule.lean`.

```
structure OrdVModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] extends WeakModule W M where
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → mode (YM W.conformalVector) 1 x = α • x
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re

def IsIrreducible (N : OrdVModule W M) : Prop :=
  Nontrivial M ∧ ∀ P : WeakModule.Submodule' N.toWeakModule, P.carrier = ⊥ ∨ P.carrier = ⊤

def IsCompletelyReducible (N : OrdVModule W M) : Prop :=
  ∀ P : WeakModule.Submodule' N.toWeakModule,
    ∃ Q : WeakModule.Submodule' N.toWeakModule, IsCompl P.carrier Q.carrier
```

Hypotheses / status. Definitions; **global** lower bound on weights

Definition 12.6 (Adjoint module). Every VOA V is a module over itself, the *adjoint module*, with $Y_V = Y$ and weight spaces $V_\alpha = V_n$ if $\alpha = n \in \mathbb{Z}$ and $V_\alpha = 0$ if $\alpha \notin \mathbb{Z}$.

Lean. `VOA.adjWeight`, `VOA.adjointWeakModule`, `VOA.adjointModule`
File `VOA/Modules/WeakModule.lean`.

```
def adjWeight (W : VertexOperatorAlgebra 𝒱) (α : ℂ) : Submodule ℂ 𝒱 :=
  ⨆ (n : ℤ) (_ : (n : ℂ) = α), W.grading n

def adjointWeakModule (W : VertexOperatorAlgebra 𝒱) : WeakModule W 𝒱 where
  YM := W.Y
  vacuum_YM := W.vacuum_Y
  truncation := fun u x => W.regularity u x
  jacobi := W.jacobi

def adjointModule (W : VertexOperatorAlgebra 𝒱) : OrdVModule W 𝒱 where
  toWeakModule := adjointWeakModule W
  weight := adjWeight W
```

Hypotheses / status. Unconditional

Theorem 12.16 (Zhu: zero modes act on the lowest-weight space through $A(V)$). *Let M be an ordinary V-module with lowest weight h. For all $u, v \in V$ and $x \in M_h$, $o(u * v)x = o(u)o(v)x$. Moreover $o(O(V))M_h = 0$, so $u \mapsto o(u)|_{M_h}$ factors through $A(V) = V/O(V)$. For homogeneous $a \in V_M$ ($M \in \mathbb{N}$) and all $b \in V$, all the residues $\sum_i \binom{M+n}{i} a_{(i-2-n)}b$, $n \geq 0$, lie in $O(V)$.*

Lean. `VOA.oMod_zhuMul_lowest_all`, `VOA.rhoLW`, `VOA.O_le_ker_rhoLW`, `VOA.Pres`, `VOA.Pres_mem_O`
File `VOA/Modules/ZhuModule.lean`.

```
theorem oMod_zhuMul_lowest_all (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h)
    (u v : 𝒱) {x : Mod} (hx : x ∈ N.weight h) :
    oMod N (zhuMul W u v) x = oMod N u (oMod N v x)

noncomputable def rhoLW (N : OrdVModule W Mod) (h : ℂ) : 𝒱 →ₗ[ℂ] End ℂ (N.weight h) where
  toFun u := weightBlock N (oMod N u) h

theorem O_le_ker_rhoLW (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h) :
    O W ≤ LinearMap.ker (rhoLW N h)

noncomputable def Pres (W : VertexOperatorAlgebra 𝒱) (M : ℕ) (a b : 𝒱) (n : ℕ) : 𝒱 :=
  ∑ i ∈ Finset.range (M + n + 1), cbz ((M : ℤ) + n) (i : ℤ) • W.smul a ((i : ℤ) - 2 - n) b

theorem Pres_mem_O (W : VertexOperatorAlgebra 𝒱) {M : ℕ} {a : 𝒱}
    (ha : a ∈ W.grading (M : ℤ)) (b : 𝒱) (n : ℕ) : Pres W M a b n ∈ O W
```

Hypotheses / status. `IsLowestWeight` (M_h **may be** 0); unconditional

Theorem 12.17 (Zhu 1996, Theorem 2.1.1). *For every VOA V: $O(V)$ is a two-sided ideal for $*$, the product $*$ is associative modulo $O(V)$, and the class of $\mathbf{1}$ is a two-sided unit. Hence $A(V)$ is an associative unital $\mathbb{C}$-algebra with product $[u][v] = [u * v]$.*

Lean. `VOA.ZhuRingData`, `VOA.zhuRingData_holds`, `VOA.instRingA`, `VOA.instAlgebraA`
Files `VOA/Modules/ZhuRing.lean` (the predicate) and `VOA/Modules/ZhuRingInstance.lean` (the theorem).

```
structure ZhuRingData (W : VertexOperatorAlgebra 𝒱) : Prop where
  left_ideal : ∀ {u : 𝒱}, u ∈ O W → ∀ v, zhuMul W u v ∈ O W
  right_ideal : ∀ (u : 𝒱) {v : 𝒱}, v ∈ O W → zhuMul W u v ∈ O W
  assoc : ∀ u v w : 𝒱, zhuMul W (zhuMul W u v) w - zhuMul W u (zhuMul W v w) ∈ O W

theorem zhuRingData_holds (W : VertexOperatorAlgebra 𝒱) : ZhuRingData W :=
  zhuRingData_of_cores W
    (fun _ _ _ _ hu ha b => ZhuHR.right_ideal_core W hu ha b)
    (fun _ _ _ _ hu hv w => associator_core W hu hv w)
```

```
@[reducible] noncomputable def instRingA (W : VertexOperatorAlgebra 𝒱) : Ring (A W) :=
  AsRing (zhuRingData_holds W)

@[reducible] noncomputable def instAlgebraA (W : VertexOperatorAlgebra 𝒱) :
    @Algebra ℂ (A W) _ (instRingA W).toSemiring :=
  AsAlgebra (zhuRingData_holds W)
```

Hypotheses / status. None; unconditional

Theorem 12.23 (R_V is a Poisson algebra; Zhu, Li). *$C_2(V)$ is a two-sided ideal for both products $u_{(-1)}v$ and $u_{(0)}v$, and on R_V the product $[u][v] = [u_{(-1)}v]$ is commutative, associative and unital, the bracket $\{[u],[v]\} = [u_{(0)}v]$ is a Lie bracket, and the Leibniz rule $\{a,bc\} = \{a,b\}c + b\{a,c\}$ holds.*

Lean. `VOA.poissonBracket_leibniz`, `VOA.A₂CommRing`, `VOA.A₂LieRing`
Files `VOA/Modules/C2Cofinite.lean`, `VOA/Modules/PoissonDescent.lean`.

```
theorem poissonBracket_leibniz (W : VertexOperatorAlgebra 𝒱) (u v w : 𝒱) :
    poissonBracket W u (poissonMul W v w) - poissonMul W v (poissonBracket W u w)
      = poissonMul W (poissonBracket W u v) w := by

noncomputable instance A₂CommRing (W : VertexOperatorAlgebra 𝒱) : CommRing (A₂ W) :=

noncomputable instance A₂LieRing (W : VertexOperatorAlgebra 𝒱) : LieRing (A₂ W) :=
```

Hypotheses / status. Definitions; unconditional

Proposition 12.31 (`Unitary` is unsatisfiable in dimension ≥ 2). *The library's "unitary module" structure on an ordinary module M asks for a $\mathbb{C}$-bilinear form $(\cdot|\cdot)$ on M with $(u_{(n)}x|y) = (x|u_{(n)}y)$ for all u, n, x, y and $(x|x) \neq 0$ for every $x \neq 0$. If M contains two linearly independent vectors, no such structure exists.*

Lean. `VOA.Unitary`
File `VOA/Modules/Contragredient.lean`; the proposition is the library theorem `VOA.old_unitary_isEmpty` (`VOA/Modules/CoreFixUnitary.lean`).

```
structure Unitary [StarRing ℂ] (N : OrdVModule W M) where
  form : M →ₗ[ℂ] M →ₗ[ℂ] ℂ
  invariant : ∀ (u : 𝒱) (n : ℤ) (x y : M),
    form (N.toWeakModule.act u n x) y = form x (N.toWeakModule.act u n y)
  posDef : ∀ x : M, x ≠ 0 → form x x ≠ 0
```

Definition 12.32 (The library's intertwining operators and intertwiner space). Let M, N, P be ordinary V-modules. *Mode data* of type $\binom{P}{M}N$ is a family of bilinear maps $\mathcal{Y}_s : M \otimes N \to P$, $s \in \mathbb{C}$ (the coefficient of z^{-s-1} of a would-be field $\mathcal{Y}(u,z)x$ with arbitrary complex powers of z). Its *intertwining defect* is

$$\Delta(a,p,s,u,x) = a^P_{(p)}\mathcal{Y}_s(u)x - \mathcal{Y}_s(u)a^N_{(p)}x - \sum_{j\geq 0}\binom{p}{j}\mathcal{Y}_{s+p-j}(a^M_{(j)}u)x,$$

for $a \in V$, $p \in \mathbb{Z}$, $s \in \mathbb{C}$, $u \in M$, $x \in N$. The library's *intertwiner space* $\mathrm{Int}(M,N;P)$ is the subspace of mode data with $\Delta \equiv 0$; an `Intertwiner` is an element of it which in addition is lower truncated ($\mathcal{Y}_s(u)x = 0$ for $\mathrm{Re}\, s \gg 0$); and fusionCoeff $= \dim \mathrm{Int}(M,N;P)$.

Lean. `VOA.IntertwinerModes`, `VOA.intertwinerSummand`, `VOA.intertwiningDefect`, `VOA.IntertwinerSpace`, `VOA.Intertwiner`
File `VOA/Modules/Fusion.lean`.

```
abbrev IntertwinerModes (M N P : Type u) [AddCommGroup M] [Module ℂ M] [AddCommGroup N]
    [Module ℂ N] [AddCommGroup P] [Module ℂ P] : Type u := ℂ → (M →ₗ[ℂ] N →ₗ[ℂ] P)

noncomputable def intertwinerSummand (A : OrdVModule W M) (Y : IntertwinerModes M N P)
    (a : 𝒱) (p : ℤ) (s : ℂ) (u : M) (x : N) (j : ℕ) : P :=
  cbz p j • Y (s + p - j) (A.toWeakModule.act a (j : ℤ) u) x

noncomputable def intertwiningDefect (A : OrdVModule W M) (B : OrdVModule W N) (C :
↪  OrdVModule W P) :
    IntertwinerModes M N P →ₗ[ℂ] (𝒱 → ℤ → ℂ → M → N → P) where
  toFun Y := fun a p s u x =>
    C.toWeakModule.act a p (Y s u x) - Y s u (B.toWeakModule.act a p x)
      - ∑ᶠ j : ℕ, intertwinerSummand A Y a p s u x j

noncomputable def IntertwinerSpace (A : OrdVModule W M) (B : OrdVModule W N) (C :
↪  OrdVModule W P) :
    Submodule ℂ (IntertwinerModes M N P) :=
  LinearMap.ker (intertwiningDefect A B C)

structure Intertwiner (A : OrdVModule W M) (B : OrdVModule W N) (C : OrdVModule W P) where
  modes : IntertwinerModes M N P
  truncation : ∀ (u : M) (x : N), ∃ s₀ : ℝ, ∀ s : ℂ, s₀ ≤ s.re → modes s u x = 0
  isIntertwiner : modes ∈ IntertwinerSpace A B C
```

Hypotheses / status. Definitions; **weaker than FHL** (no iterate, no L_{-1} axiom)

Theorem 12.33 (`fusionCoeff` is identically zero). *For all ordinary modules A, B, C,* `fusionCoeff A B C = 0`. *More precisely,* `IntertwinerSpace A B C` *is either the zero space or infinite-dimensional.*

Lean. `VOA.fusionCoeff`

```
noncomputable def fusionCoeff (A : OrdVModule W M) (B : OrdVModule W N) (C : OrdVModule W
↪  P) : ℕ :=
  Module.finrank ℂ (IntertwinerSpace A B C)
```

Hypotheses / status. $\equiv 0$ (proved in the library)

Definition 12.35 (Rational; Zhu 1996, Dong–Li–Mason 1997–98). V is *(admissibly) rational* if every admissible V-module is completely reducible.

Lean. `VOA.AdmissiblyRational`, `VOA.IsRational`
File `VOA/Modules/AdmissibleModule.lean`.

```
def AdmissiblyRational (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
    N.IsCompletelyReducible

abbrev IsRational (W : VertexOperatorAlgebra 𝒱) : Prop := AdmissiblyRational W
```

Hypotheses / status. Definitions

Theorem 12.36 (DLM rationality implies `AdmissiblyRational`, under C_2). *Let V be C_2-cofinite and rational in the sense of Dong–Li–Mason: every admissible module is the internal direct sum of graded submodules that are minimal among nonzero graded submodules. Then V is* `AdmissiblyRational`.

Lean. `VOA.Classification.COne.DLMRat.GradedMinimal`, `VOA.Classification.COne.DLMRat.DLMRational`, `VOA.Classification.COne.DLMRat.admissiblyRational_of_dlmRational`
File `VOA/Classification/COne/DLMRationality.lean`, namespace `VOA.Classification.COne.DLMRat`.

```
def GradedMinimal (N : AdmissibleModule W M) (P : WeakModule.Submodule' N.toWeakModule) :
↪  Prop :=
  N.IsLevelGraded P ∧ P.carrier ≠ ⊥ ∧
    ∀ Q : WeakModule.Submodule' N.toWeakModule, N.IsLevelGraded Q → Q.carrier ≤ P.carrier →
      Q.carrier = ⊥ ∨ Q.carrier = P.carrier

def DLMRational (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
    ∃ (ι : Type u) (P : ι → WeakModule.Submodule' N.toWeakModule),
      iSupIndep (fun i => (P i).carrier) ∧ ⨆ i, (P i).carrier = ⊤ ∧ ∀ i, GradedMinimal N (P
       ↪  i)

theorem admissiblyRational_of_dlmRational (hC2 : C2Cofinite W) (h : DLMRational W) :
    AdmissiblyRational W := by
```

Hypotheses / status. `hC2`; proved; converse open

Definition 12.37 (Gannon-rational and holomorphic; Gannon, Def. 5.3.2). V is *Gannon-rational* if (1) every ordinary module is completely reducible, (2) $\Phi(V)$ is finite, and (3) every irreducible admissible module is ordinary (on the same weak module). V is *holomorphic* if it is Gannon-rational and $|\Phi(V)| = 1$.

Lean. `VOA.GannonAdmissibleIsOrdinary`, `VOA.GannonRational`, `VOA.Holomorphic`
File `VOA/Modules/Rationality.lean`.

```
def GannonAdmissibleIsOrdinary (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
    N.IsIrreducible → ∃ O : OrdVModule W M, O.toWeakModule = N.toWeakModule

structure GannonRational (W : VertexOperatorAlgebra 𝒱) : Prop where
  completelyReducible : ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M),
    OrdVModule.IsCompletelyReducible N
  finitelyManyIrred : Finite (Φ W)
  admissibleIsOrdinary : GannonAdmissibleIsOrdinary W

def Holomorphic (W : VertexOperatorAlgebra 𝒱) : Prop :=
  GannonRational W ∧ Nat.card (Φ W) = 1
```

Definition 12.38 (Regular; Dong–Li–Mason 1997, Def. 3.1). V is *regular* if every weak module is completely reducible and every irreducible weak module is ordinary. $\Phi_{\mathrm{adm}}(V)$ is the set of isomorphism classes of irreducible admissible modules (isomorphism of the underlying weak modules).

Lean. `VOA.Regular`, `VOA.IrrAdmModule`, `VOA.Φadm`
File `VOA/Modules/RationalityLattice.lean`.

```
structure Regular (W : VertexOperatorAlgebra 𝒱) : Prop where
  weakCompletelyReducible : ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : WeakModule W
   ↪  M)
    (P : WeakModule.Submodule' A), ∃ Q : WeakModule.Submodule' A, IsCompl P.carrier
     ↪  Q.carrier
  irreducibleWeakIsOrdinary : ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : WeakModule
   ↪  W M),
    (Nontrivial M ∧ ∀ P : WeakModule.Submodule' A, P.carrier = ⊥ ∨ P.carrier = ⊤) →
    ∃ N : OrdVModule W M, N.toWeakModule = A

structure IrrAdmModule (W : VertexOperatorAlgebra 𝒱) where
  carrier : Type u
```

```
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : AdmissibleModule W carrier
  irreducible : isMod.IsIrreducible

def Φadm (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) := Quotient (IrrAdmModule.setoid W)
```

Hypotheses / status. Definitions

Theorem 12.39 (Zhu 1996, Theorem 2.2.3; Dong–Li–Mason: rational ⇒ $A(V)$ semisimple). *Let V be a vertex operator algebra of the library. If V is admissibly rational (Definition 12.35), then Zhu's algebra $A(V)$, with Zhu's product, is a semisimple ring. Equivalently, the predicate* `ZhuAdmissibleSemisimple W` *holds for every V.*

Lean. `VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational`, `VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr`
File `VOA/Huang/ZhuSemisimple.lean` (Chapter 21), namespace `VOA.Huang.ZhuSemisimple`. The section variables are `{W : VertexOperatorAlgebra 𝒱}` and `(Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))`, and each theorem is preceded by `include Pr in`.

```
theorem isSemisimpleRing_A_of_admissiblyRational (hrat : AdmissiblyRational W) :
    letI := instRingA W; IsSemisimpleRing (A W) := by

theorem zhuAdmissibleSemisimple_of_Pr : ZhuAdmissibleSemisimple W :=
  fun hrat => isSemisimpleRing_A_of_admissiblyRational Pr hrat
```

Hypotheses / status. `Pr` (from `VOA.pbw`), `AdmissiblyRational`; proved; used by the Zhu headline

Theorem 12.40 (Abe–Buhl–Dong Prop. 5.7, Cor. 5.8, Thm. 4.5; Dong–Li–Mason Thm. 8.1(b),(c); Zhu). *Let V be a vertex operator algebra of the library (of CFT type, Chapter 1).*

1. (ABD Prop. 5.7) *If V is C_2-cofinite, then every weak V-module M carries an admissible-module structure (Definition 12.4) whose underlying weak module is M itself, with the same vertex operators.*
2. (ABD Cor. 5.8) *If V is C_2-cofinite, then every irreducible weak V-module M (that is, $M \neq 0$ and its only weak submodules are 0 and M) carries an ordinary-module structure (Definition 12.5) with the same vertex operators.*
3. (ABD Thm. 4.5, hard direction) *If V is C_2-cofinite and admissibly rational, then V is regular.*
4. (Zhu) *If V is admissibly rational, then $A(V)$ is a semisimple ring (Theorem 12.39, restated).*
5. (DLM Thm. 8.1(c)) *If V is C_2-cofinite, then every irreducible admissible V-module carries an ordinary-module structure with the same vertex operators. No rationality is assumed.*
6. (DLM Thm. 8.1(b)) *If V is C_2-cofinite and admissibly rational, then V has only finitely many isomorphism classes of irreducible admissible modules: $\Phi_{\mathrm{adm}}(V)$ is finite.*

Lean. `VOA.Classification.COne.ABDWeakAdmissible.abdWeakIsAdmissible`, `VOA.Classification.COne.AdmissibleOrdinary.irreducible_admissible_isOrdinary`, `VOA.Classification.COne.AdmissibleOrdinary.dlmSimpleAdmissibleOrdinary_of_c2Cofinite`, `VOA.Classification.COne.LiteratureInputs.abdIrreducibleWeakIsOrdinary_holds`, `VOA.Classification.COne.LiteratureInputs.abdRegularOfC2AndRational_holds`, `VOA.Classification.COne.LiteratureInputs.zhuAdmissibleSemisimple_holds`, `VOA.Classification.COne.LiteratureInputs.dlmFinitelyManyIrredAdmissible_of_c2Cofinite`
File `VOA/Classification/COne/ABDWeakAdmissibleComplete.lean`, namespace `VOA.Classification.COne.ABDWeakAdmissible`, with `W` an implicit section variable (item 1):

```
theorem abdWeakIsAdmissible : ABDWeakIsAdmissible W :=
  abdWeakIsAdmissible_of_degreeNormalForm_boundedBelow
    weakDegreeNormalForm weakSpectrumBoundedBelow
```

File `VOA/Classification/COne/AdmissibleOrdinaryC2.lean`, namespace `VOA.Classification.COne.AdmissibleOrdinary`, with `W` implicit (item 5):

```
theorem irreducible_admissible_isOrdinary (hC2 : C2Cofinite W)
    (N : AdmissibleModule W M) (hirr : N.IsIrreducible) :
    ∃ O : OrdVModule W M, O.toWeakModule = N.toWeakModule := by

theorem dlmSimpleAdmissibleOrdinary_of_c2Cofinite (hC2 : C2Cofinite W) :
    DLMSimpleAdmissibleOrdinary W := by
```

File `VOA/Classification/COne/LiteratureInputsProved.lean`, namespace `VOA.Classification.COne.LiteratureInputs` (items 2, 3, 4, 6):

```
variable {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] {W : VertexOperatorAlgebra 𝒱}

theorem abdIrreducibleWeakIsOrdinary_holds : ABDIrreducibleWeakIsOrdinary W :=
  abdIrreducibleWeakIsOrdinary_of_weakIsAdmissible abdWeakIsAdmissible

theorem abdRegularOfC2AndRational_holds : ABDRegularOfC2AndRational W :=
  abdRegularOfC2AndRational_of_components abdWeakIsAdmissible
    abdIrreducibleWeakIsOrdinary_holds

theorem zhuAdmissibleSemisimple_holds : ZhuAdmissibleSemisimple W :=
  VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))

theorem dlmFinitelyManyIrredAdmissible_of_c2Cofinite (hC2 : C2Cofinite W) :
    DLMFinitelyManyIrredAdmissible W := fun hAR =>
  c2SemisimpleAdmissibleFiniteness_of_bottomProjection
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))) hC2
    (VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational
      (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))) hAR)
```

Hypotheses / status. None; **proved**, unconditional

Definition 12.43 (Strongly finite, strongly rational; Creutzig–Gannon, Dong–Mason, McRae). V is *strongly finite* if it is simple, self-dual and C_2-cofinite (and of CFT type, which is automatic here); *strongly rational* if moreover it is admissibly rational.

Lean. `VOA.StronglyFinite`, `VOA.StronglyRational`, `VOA.stronglyFinite_iff`, `VOA.stronglyRational_of_regular`, `VOA.stronglyRational_of_span_vacuum`
File `VOA/Modules/StronglyRational.lean`.

```
structure StronglyFinite (W : VertexOperatorAlgebra 𝒱) : Prop where
  simple : W.IsSimple
  selfDual : W.SelfDual
  c2Cofinite : C2Cofinite W

structure StronglyRational (W : VertexOperatorAlgebra 𝒱) : Prop extends StronglyFinite W
 ↪ where
  rational : AdmissiblyRational W

theorem stronglyFinite_iff {W : VertexOperatorAlgebra 𝒱} (hsimple : W.IsSimple)
    (hc2 : C2Cofinite W) :
```

```
    StronglyFinite W ↔ ∀ v, v ∈ W.grading 1 → W.L 1 v = 0 :=

theorem stronglyRational_of_regular {W : VertexOperatorAlgebra 𝒱} (h : StronglyFinite W)
    (hreg : Regular W) : StronglyRational W where

theorem stronglyRational_of_span_vacuum {W : VertexOperatorAlgebra 𝒱} (hne : W.vacuum ≠ 0)
    (hspan : Submodule.span ℂ ({W.vacuum} : Set 𝒱) = ⊤) : StronglyRational W where
```

Hypotheses / status. Definitions; satisfiable

Theorem 12.44 (Strongly rational VOAs: regular, finitely many admissible irreducibles, Gannon-rational). *Let V be strongly rational (Definition 12.43). Then*

1. *V is regular (Definition 12.38);*
2. *V satisfies the predicate* `DLMFinitelyManyIrredAdmissible`*, and V has only finitely many isomorphism classes of irreducible admissible modules;*
3. *V is Gannon-rational (Definition 12.37).*

No named literature hypothesis is assumed.

Lean. `VOA.StronglyRational.regular_of_abd`, `VOA.StronglyRational.gannonRational`, `VOA.StronglyRational.regular`, `VOA.StronglyRational.dlmFinitelyManyIrredAdmissible`, `VOA.StronglyRational.finite_Φadm`, `VOA.StronglyRational.gannonRational'`
File `VOA/Modules/StronglyRational.lean` (the older forms, which carry the literature predicates as hypotheses):

```
theorem StronglyRational.regular_of_abd {W : VertexOperatorAlgebra 𝒱} (h : StronglyRational
 ↪  W)
    (hABD : ABDRegularOfC2AndRational W) : Regular W :=
  hABD h.c2Cofinite h.rational

theorem gannonRational (h : StronglyRational W) (hFin : DLMFinitelyManyIrredAdmissible W)
    (hOrd : DLMSimpleAdmissibleOrdinary W) : GannonRational W :=
  gannonRational_of_admissiblyRational h.rational hFin hOrd
```

File `VOA/Classification/COne/LiteratureInputsProved.lean` (the discharged forms; same section variables as in Theorem 12.40):

```
theorem _root_.VOA.StronglyRational.regular (h : StronglyRational W) : Regular W :=
  h.regular_of_abd abdRegularOfC2AndRational_holds

theorem _root_.VOA.StronglyRational.dlmFinitelyManyIrredAdmissible (h : StronglyRational W)
 ↪  :
    DLMFinitelyManyIrredAdmissible W :=
  dlmFinitelyManyIrredAdmissible_of_c2Cofinite h.c2Cofinite

theorem _root_.VOA.StronglyRational.finite_Φadm (h : StronglyRational W) : Finite (Φadm W)
 ↪  :=
  h.dlmFinitelyManyIrredAdmissible h.rational

theorem _root_.VOA.StronglyRational.gannonRational' (h : StronglyRational W) :
    GannonRational W :=
  h.gannonRational h.dlmFinitelyManyIrredAdmissible
    (dlmSimpleAdmissibleOrdinary_of_stronglyRational h)
```

Hypotheses / status. `StronglyRational`; proved, no literature hypothesis

Chapter 13

Zhu's bijection via the tower $A_n(V)$

7 of the chapter's 33 numbered statements: its headline results and every definition they use.

Theorem 13.4 (Zhu's correspondence, uniqueness form). *Let V be any vertex operator algebra of the library and assume the datum* `Pr` *of Definition 13.13 for the regular $A(V)$-module. Then* fwd $:\ \Phi(V) \to \mathrm{Irr}A(V)$ *is a bijection. Equivalently: if P, Q are irreducible bundled lowest-weight V-modules and $P_h \cong Q_{h'}$ as $A(V)$-modules, then $P \cong Q$. The datum* `Pr` *is supplied, for every V, by the theorem* `VOA.pbw` *(through* `voaPBWBottomProjection`*).*

Lean. `VOA.zhuBijective`, `VOA.zhuBijective_of_voaPBW`, `VOA.IrredBottomRigidity`, `VOA.forwardMapIrr_surjective_uncond`, `VOA.forwardMapIrr_bijective_iff_rigidity`
Files `VOA/Modules/ZhuVermaAnArchLUBijection.lean` and `VOA/Modules/ZhuVermaAnArchLUBijectionPBW.lean`.

```
theorem zhuBijective (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    Function.Bijective (forwardMapIrr W) :=
theorem zhuBijective_of_voaPBW (W : VertexOperatorAlgebra 𝒱) :
    Function.Bijective (forwardMapIrr W) :=
  zhuBijective
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

```
def IrredBottomRigidity (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ P Q : LWModule W, P.IsIrreducibleLW → Q.IsIrreducibleLW →
    Nonempty (AModEquiv W P.toAMod Q.toAMod) → LWModule.Iso W P Q
theorem forwardMapIrr_surjective_uncond (W : VertexOperatorAlgebra 𝒱) :
    Function.Surjective (forwardMapIrr W) := by
theorem forwardMapIrr_bijective_iff_rigidity :
    Function.Bijective (forwardMapIrr W) ↔ IrredBottomRigidity W :=
  ⟨fun hb => irredBottomRigidity_of_injective hb.1, forwardMapIrr_bijective_of_rigidity⟩
```

Hypotheses / status. `Pr`; conditional

Theorem 13.6 (Bottoms of irreducible modules are simple; Zhu 1996, Theorem 2.2.1(1)). *For every vertex operator algebra V of the library and every irreducible bundled lowest-weight module P, the bottom P_h is a simple $A(V)$-module. More precisely, every lowest-weight module P has the* bottom-recovery property*: for every $A(V)$-submodule $S \subseteq P_h$, the V-submodule of P generated by S meets P_h in exactly S.*

Lean. `VOA.BottomRecovers`, `VOA.AllBottomRecovers`, `VOA.Huang.LWBottomRecovery.allBottomRecovers`, `VOA.Huang.LWBottomRecovery.isSimpleAMod_toAMod_of_irreducible`

```
def BottomRecovers : Prop :=
  ∀ S : AModSubmodule (P.toAMod),
    vSubmoduleBottomCarrier S.genVSubmodule ≤ S.carrier
```

```
def AllBottomRecovers (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ P : LWModule W, BottomRecovers P
```

```
theorem allBottomRecovers (W : VertexOperatorAlgebra 𝒱) : AllBottomRecovers W :=
  fun P => bottomRecovers P

theorem isSimpleAMod_toAMod_of_irreducible (hP : P.IsIrreducibleLW) : IsSimpleAMod P.toAMod
 ↪  :=
  isSimpleAMod_of_bottomRecovers P hP (bottomRecovers P)
```

Files `VOA/Modules/ZhuVermaAnArchLeaf3.lean`, `VOA/Modules/ZhuVermaAnArchAModScope.lean`, `VOA/Huang/LWBottomRecovery.lean`. **Hypotheses / status.** None; unconditional

Theorem 13.8 (Zhu's correspondence as one bijection; Zhu 1996, Theorem 2.2.2 / Gannon, Theorem 5.3.4, for C_2-cofinite V). *Let V be C_2-cofinite. There is a bijection*

$$e : \Phi(V) \xrightarrow{\sim} \mathrm{Simple}A(V)/\cong$$

such that for every irreducible lowest-weight module P with bottom P_h, $e([P]) = [P_h]$.

Lean. `VOA.zhuCorrespondence_of_C2Cofinite`
File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
theorem zhuCorrespondence_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱) (hC2 : C2Cofinite W)
 ↪  :
    ∃ e : Φ W ≃ SimpleAModIsoClass W, ∀ P : IrrLWModule W,
      e (forgetClass W (Quotient.mk (IrrLWModule.setoid W) P)) =
        Quotient.mk (SimpleAMod.setoid W) (IrrLWModule.toSimpleAMod P) := by
```

Hypotheses / status. C_2; conditional; via theorem `VOA.pbw`

Definition 13.13 (Bottom projection; DLM's PBW step). A *bottom projection* for a level-n parabolic representation P on U is a linear map $\pi : M_n(U) \to U$ with $\pi([1 \otimes u]) = u$ for all u, and $\pi = 0$ on every level $M_n(U)(k)$, $k \neq n$. The *PBW freeness* of P is the conjunction of: the bottom map is injective, and the levels are independent.

Lean. `VOA.VHat.DLM3.BottomProjection`, `VOA.VHat.DLM3.PBWFreeness`, `VOA.VHat.DLM3.pbwFreeness_iff_bottom_injective`, `VOA.VHat.DLM3.bottomProjectionOfInjective`, `VOA.VHat.DLM3.nonempty_bottomProjection_iff_bottom_injective`
Files `VOA/Modules/ZhuVermaAnArchDLM3Survival.lean` and `VOA/Modules/ZhuVermaAnArchDLM3PBWSplit.lean`.

```
structure BottomProjection (P : ParabolicRep W n U) where
  /-- The projection onto the bottom `U` of `M_n(U)`. -/
  proj : IndMod P →ₗ[ℂ] U
  /-- It retracts the bottom map `u ↦ [1 ⊗ u]`. -/
  proj_bottom : ∀ u : U, proj (indBottom P u) = u
  /-- It annihilates every level other than `n` (DLM3: "`U*` annihilates `⊕_{i≠n}
   ↪  M(U)(i)`"). -/
  proj_levCover : ∀ {k : ℤ}, k ≠ (n : ℤ) → ∀ {z : IndMod P}, z ∈ levCover P k → proj z = 0
structure PBWFreeness (P : ParabolicRep W n U) : Prop where
  /-- **PBW, part 1** — the bottom `U` embeds: `u ↦ [1 ⊗ u]` is injective. -/
  bottom_injective : Function.Injective (indBottom P)
  /-- **Historical PBW part 2, now a theorem** — the levels `M_n(U)(k) = U(V̂)_{k-n}U` are
  independent. `levCover_iSupIndep` supplies this field unconditionally; together with
  `levCover_spanning` it gives the internal `ℤ`-grading of `M_n(U)`. -/
  levels_iSupIndep : iSupIndep (levCover P)
```

```
theorem pbwFreeness_iff_bottom_injective (P : ParabolicRep W n U) :
    PBWFreeness P ↔ Function.Injective (indBottom P) :=
noncomputable def bottomProjectionOfInjective (Q : ParabolicRep W 0 U)
    (hinj : Function.Injective (indBottom Q)) : BottomProjection Q :=
theorem nonempty_bottomProjection_iff_bottom_injective (Q : ParabolicRep W 0 U) :
    Nonempty (BottomProjection Q) ↔ Function.Injective (indBottom Q) := by
```

Hypotheses / status. Definitions; $\Longleftrightarrow$ bottom injective

Theorem 13.25 (Survival of the bottom; the gate). *Let* `Pr` *be a bottom projection for the parabolic representation of the regular $A(V)$-module. Then:*

1. *(Dong–Li–Mason, Theorem 4.1 at $n = 0$.) For every $A(V)$-module U with a bottom projection, U embeds into the weak module $\overline{M}_0(U) = M_0(U)/(\text{defects})$ as its bottom.*
2. *(Dong–Jiang, Proposition 4.6 at $(0,0)$.) $O_{0,0}(V) = O(V)$; hence $A_{0,0}(V) = A(V)$, and for every $A(V)$-module U the bottom level of $M(U)$ is $M(U)(0) \cong U$.*

Lean. `VOA.VHat.DLM3.IndBar`, `VOA.VHat.DLM3.indBarBottom_injective_of_bottomProjection`, `VOA.gate_of_pbwDatum`, `VOA.djAnnEqAn_zero_of_pbwDatum`, `VOA.djCarrierDegZeroEquivOfPbwDatum`
Files `VOA/Modules/ZhuVermaAnArchDLM3Survive.lean` and `VOA/Modules/ZhuVermaAnArchDLM3Induced.lean`.

```
abbrev IndBar (P : ParabolicRep W n U) : Type u :=
  IndMod P / VHatRep.defectSubmodule (indRho P)
theorem indBarBottom_injective_of_bottomProjection (W : VertexOperatorAlgebra 𝒱) (U : AMod
 ↪  W)
    (Pr : BottomProjection (parabolicRepOfAMod W U)) :
    Function.Injective (indBarBottom (parabolicRepOfAMod W U)) :=
theorem gate_of_pbwDatum (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    OnmFull W 0 0 = O W :=
theorem djAnnEqAn_zero_of_pbwDatum
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) : DJAnnEqAn W 0 :=
noncomputable def djCarrierDegZeroEquivOfPbwDatum
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) (V : AmMod W 0) :
    DJCarrier.deg W V 0 ≃ₗ[ℂ] V.carrier :=
```

Hypotheses / status. `Pr`

Theorem 13.31 ($P \cong L(\Omega(P))$; Dong–Jiang, Theorem 4.13). *Every irreducible bundled lowest-weight module P is isomorphic to $L(\Omega(P))$.*

Lean. `VOA.pRealizesLWIso`
File `VOA/Modules/ZhuVermaAnArchLURealizesIrr.lean`.

```
theorem pRealizesLWIso (P : LWModule W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hU : Nontrivial (pBotAmMod P).carrier)
    (hL0 : ∀ (n : ℕ) (x : DJCarrier W (pBotAmMod P)), x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x)
    (hP : P.IsIrreducibleLW)
    (hfin : (djLU Pr (pBotAmMod P) hU P.h hL0).FiniteWeights) :
    LWModule.Iso W ((djLU Pr (pBotAmMod P) hU P.h hL0).toLWModule hfin) P :=
```

Hypotheses / status. `Pr`, irreducible

Theorem 13.32 ($L(\cdot)$ is invariant under isomorphism of bottoms). *If $U_1 \cong U_2$ as $A(V)$-modules (both nonzero, with L_0 acting on level n by $h_i + n$, $h_1 = h_2$), then $L(U_1) \cong L(U_2)$.*

Lean. `VOA.djFunctorEquiv`, `VOA.djLUFunctorLWIso'`, `VOA.pH_eq`
Files `VOA/Modules/ZhuVermaAnArchLUFunctorial.lean` and `VOA/Modules/ZhuVermaAnArchLUBijection.lean`.

```
noncomputable def djFunctorEquiv (U₁ U₂ : AmMod W 0) (e : AmModHom U₁ U₂) :
    DJCarrier W U₁ ≃ₗ[ℂ] DJCarrier W U₂ :=
  DirectSum.congrLinearEquiv (fun n => djFunctorPiece U₁ U₂ e n)
theorem djLUFunctorLWIso' (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    {U₁ U₂ : AmMod W 0} (e : AmModHom U₁ U₂) (hU₁ : Nontrivial U₁.carrier)
    (hU₂ : Nontrivial U₂.carrier) (h₁ h₂ : ℂ) (heq : h₁ = h₂)
    (hL0₁ : ∀ (n : ℕ) (x : DJCarrier W U₁), x ∈ DJCarrier.deg W U₁ n →
      djModeZero W U₁ 1 W.conformalVector x = (h₁ + n) • x)
    (hL0₂ : ∀ (n : ℕ) (x : DJCarrier W U₂), x ∈ DJCarrier.deg W U₂ n →
      djModeZero W U₂ 1 W.conformalVector x = (h₂ + n) • x)
    (hfin₁ : (djLU Pr U₁ hU₁ h₁ hL0₁).FiniteWeights)
    (hfin₂ : (djLU Pr U₂ hU₂ h₂ hL0₂).FiniteWeights) :
    LWModule.Iso W ((djLU Pr U₁ hU₁ h₁ hL0₁).toLWModule hfin₁)
      ((djLU Pr U₂ hU₂ h₂ hL0₂).toLWModule hfin₂) := by
theorem pH_eq {P Q : LWModule W} (ψ : AModEquiv W P.toAMod Q.toAMod) : P.h = Q.h := by
```

Hypotheses / status. `Pr`

Part IV

Modularity

Chapter 14

Modular forms and the analytic toolkit

9 of the chapter's 68 numbered statements: its headline results and every definition they use.

Theorem 14.2 (The derivative does not commute with slashing). *Let $k \in \mathbb{Z}$, $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, $z \in \mathbb{H}$, and let f be complex-differentiable at γz. Then*

$$(f')|_{k+2}\gamma\,(z) \;=\; \frac{d}{dz}\big(f|_k\gamma\big)(z) \;+\; \frac{k\,c}{cz+d}\,(f|_k\gamma)(z).$$

Lean. `VOA.slashC_deriv_eq`
File `VOA/Modularity/SerreDerivative.lean`.

```
theorem slashC_deriv_eq (k : ℤ) (γ : SL(2, ℤ)) (f : ℂ → ℂ) {z : ℂ} (hz : z ∈ imGT 0)
    (hf : DifferentiableAt ℂ f (mobiusC γ z)) :
    slashC (k + 2) γ (_root_.deriv f) z
      = _root_.deriv (slashC k γ f) z
        + (k : ℂ) * (γ 1 0 : ℂ) * (denomC γ z)⁻¹ * slashC k γ f z
```

Hypotheses / status. Differentiability; unconditional

Theorem 14.4 (Quasimodularity of G_2; Mathlib's `G2_slash_action`). *For $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ and $z \in \mathbb{H}$:* $(G_2|_2\gamma)(z) = G_2(z) - \dfrac{2\pi i\,c}{cz+d}$.

Lean. `VOA.slashC_G2C`
File `VOA/Modularity/SerreDerivative.lean`.

```
theorem slashC_G2C (γ : SL(2, ℤ)) {z : ℂ} (hz : z ∈ imGT 0) :
    slashC 2 γ G2C z
      = G2C z - 2 * (Real.pi : ℂ) * Complex.I * (γ 1 0 : ℂ) * (denomC γ z)⁻¹
```

Hypotheses / status. Differentiability; unconditional

Theorem 14.8 (Identification with Mathlib's Eisenstein series). *For $j \geq 2$ and $z \in \mathbb{H}$, $\mathcal{G}_{2j}(z) = g_j(0)\,E_{2j}(z)$, where E_{2j} is Mathlib's normalised Eisenstein modular form of weight $2j$. Consequently, for all $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, $\mathcal{G}_{2j}|_{2j}\gamma = \mathcal{G}_{2j}$ on $\mathbb{H}$. (Note $g_j(0) = -B_{2j}/(2j)! \neq 1$: Zhu's $\mathcal{G}_{2j}$ is* not *E_{2j}, but a nonzero constant multiple of it.)*

Lean. `VOA.zhuGC_eq_mul_E`, `VOA.slashC_zhuGC`
File `VOA/Modularity/EisensteinAnalytic.lean`.

```
theorem zhuGC_eq_mul_E {j : ℕ} (hj : 2 ≤ j) {z : ℂ} (hz : z ∈ imGT 0) :
    zhuGC j z = zhuG j 0 * ModularForm.E (show 3 ≤ 2 * j by omega) (⟨z, hz⟩ :
     ↪ UpperHalfPlane)

theorem slashC_zhuGC {j : ℕ} (hj : 2 ≤ j) (γ : SL(2, ℤ)) {z : ℂ} (hz : z ∈ imGT 0) :
    slashC (2 * j : ℤ) γ (zhuGC j) z = zhuGC j z
```

Hypotheses / status. $j \geq 2$; unconditional

Definition 14.14 (Three named propositions: homogeneous representatives, monomial and full Serre closure). (a) *Homogeneous representatives*: every $f \in \mathcal{E}_{46}$ of weight k has a homogeneous polynomial representative of weight k. (b) *Monomial Serre closure*: for all a, b, $\partial_{4a+6b}(E_4^a E_6^b)$ equals on $\mathbb{H}$ an element of $\mathcal{E}_{46}$ of weight $4a + 6b + 2$. (c) *Serre closure*: for every f of weight k there is $g \in \mathcal{E}_{46}$ of weight $k + 2$ with $g = \partial_k f$ on $\mathbb{H}$.

Lean. `VOA.E46HasWeightedPolynomialReps`, `VOA.E46SerreMonomialClosed`, `VOA.E46SerreClosed`
File `VOA/Modularity/EisensteinRamanujan.lean`.

```
def E46HasWeightedPolynomialReps : Prop :=
  ∀ (k : ℤ) (f : E46), E46Weight k f → E46WeightedPolynomialRep k f

def E46SerreMonomialClosed : Prop :=
  ∀ (a b : ℕ), ∃ g : E46,
    E46Weight (4 * (a : ℤ) + 6 * (b : ℤ) + 2) g ∧
      ∀ z ∈ imGT 0,
        e46C g z = serreDerivC (4 * (a : ℤ) + 6 * (b : ℤ))
          (e46C (e46Monomial a b)) z

def E46SerreClosed : Prop :=
  ∀ (k : ℤ) (f : E46), E46Weight k f →
    ∃ g : E46, E46Weight (k + 2) g ∧
      ∀ z ∈ imGT 0, e46C g z = serreDerivC k (e46C f) z
```

Theorem 14.15 (Homogeneous representatives exist). *Every $f \in \mathcal{E}_{46}$ of weight k is a linear combination of monomials $E_4^a E_6^b$ with $4a + 6b = k$; that is,* `E46HasWeightedPolynomialReps` *holds.*

File `VOA/Modularity/EisensteinSerreClosure.lean`.
(The chapter quotes no Lean declaration for this statement; see book p. 687.)

Theorem 14.23 (Holomorphy and term-by-term differentiation). *Let χ be a formal q-series with $\|\chi(h+n)\|$ subexponential. Then $z \mapsto \chi_h^{\mathbb{C}}(z)$ is holomorphic on $\mathbb{H}$, and for $z \in \mathbb{H}$*

$$\frac{d}{dz}\chi_h^{\mathbb{C}}(z) = 2\pi i\,(D\chi)_h^{\mathbb{C}}(z), \qquad (D\chi)(e) := e\,\chi(e).$$

In particular the character of a C_2-cofinite VOA (with $\mathbf{1} \neq 0$) is holomorphic on $\mathbb{H}$.

Lean. `VOA.qDeriv`, `VOA.differentiableOn_qEvalC`, `VOA.deriv_qEvalC`, `VOA.differentiableOn_char_adjoint`
File `VOA/Modularity/ZhuBlockConditions.lean` (a Chapter 15 file).

```
noncomputable def qDeriv (χ : FormalQSeries) : FormalQSeries := fun e => e * χ e
```

File `VOA/Modularity/QEvalHolomorphic.lean`.

```
theorem differentiableOn_qEvalC (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + (n : ℂ))‖)) :
    DifferentiableOn ℂ (qEvalC χ h) (imGT 0)

theorem deriv_qEvalC (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + (n : ℂ))‖)) {z : ℂ} (hz : z ∈ imGT 0) :
    _root_.deriv (qEvalC χ h) z = 2 * (Real.pi : ℂ) * Complex.I * qEvalC (qDeriv χ) h z

theorem differentiableOn_char_adjoint (W : VertexOperatorAlgebra 𝒱) (hne : W.vacuum ≠ 0)
    (hC2 : C2Cofinite W) :
    DifferentiableOn ℂ
      (qEvalC (gradedDim (adjointModule W)) (0 - W.centralCharge / 24)) (imGT 0)
```

Hypotheses / status. Growth

Theorem 14.35 (Equivariance of the Serre derivative). *For $k \in \mathbb{Z}$, $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, $z \in \mathbb{H}$, and f differentiable at γz: $(\partial_k f)|_{k+2}\gamma\,(z) = \partial_k(f|_k\gamma)(z)$. (No modularity of f is required. This is equivariance only: it does* not *show that ∂_k maps M_k into M_{k+2}, which also needs holomorphy at the cusp; that is Serre closure, Definition 14.14, proved separately in* `EisensteinSerreClosure.lean`*.)*

Lean. `VOA.serreDerivC_slashC`
File `VOA/Modularity/SerreDerivative.lean`.

```
theorem serreDerivC_slashC (k : ℤ) (γ : SL(2, ℤ)) (f : ℂ → ℂ) {z : ℂ} (hz : z ∈ imGT 0)
    (hf : DifferentiableAt ℂ f (mobiusC γ z)) :
    slashC (k + 2) γ (serreDerivC k f) z = serreDerivC k (slashC k γ f) z
```

Hypotheses / status. Differentiability; unconditional

Theorem 14.41 (The majorant theorem). *Let $\chi_n, a_n, b_n \geq 0$ with a, b subexponential, let $p_n \to +\infty$, and suppose that for some N_0 and all $n \geq N_0$*

$$p_n\,\chi_n \;\leq\; \sum_{m<n} a_{n-m}\,\chi_m + b_n.$$

Then χ is subexponential.

Lean. `VOA.subexponential_of_majorant_recursion`
File `VOA/Modularity/Majorant.lean`.

```
theorem subexponential_of_majorant_recursion {χ a b p : ℕ → ℝ}
    (hχ0 : ∀ n, 0 ≤ χ n) (ha0 : ∀ n, 0 ≤ a n) (hb0 : ∀ n, 0 ≤ b n)
    (ha : Subexponential a) (hb : Subexponential b)
    (hp : Tendsto p atTop atTop) (N₀ : ℕ)
    (hrec : ∀ n, N₀ ≤ n → p n * χ n ≤ (∑ m ∈ Finset.range n, a (n - m) * χ m) + b n) :
    Subexponential χ
```

Hypotheses / status. $p_n \to \infty$

Proposition 14.64 (Check for the template warning). *For every super character system S:* `CompleteRationalFamily` $S\,P \iff P$; `Conjecture_HalfWeight` S False *holds; and the zero family satisfies* `SuperRealisesLawOn` *for every Γ, J, ρ.*

(The chapter quotes no Lean declaration for this statement; see book p. 722.)

Chapter 15

Zhu's modularity theorem

10 of the chapter's 42 numbered statements: its headline results and every definition they use.

Theorem 15.1 (Zhu 1996, Theorem 5.3.3; Dong–Li–Mason 2000). *Let V be rational and C_2-cofinite, with irreducible modules $M_1, \dots, M_r$. Then:*

1. *each $Z_{M_i}(v,\tau)$ converges to a holomorphic function on $\mathbb{H}$;*
2. *there is a representation $\rho : SL_2(\mathbb{Z}) \to GL_r(\mathbb{C})$ such that for every v in the square-bracket weight space $V_{[n]}$ (Section 15.3)*

$$Z_{M_i}(v,\gamma\tau) = (c\tau+d)^n \sum_j \rho(\gamma)_{ij} Z_{M_j}(v,\tau);$$

3. *the functions $Z_{M_1}(\cdot,\tau), \dots, Z_{M_r}(\cdot,\tau)$ are linearly independent, so ρ is unique.*

(The chapter quotes no Lean declaration for this statement; see book p. 734.)

Theorem 15.16 (Convergence of one-point functions; Zhu 1996 §4.4, Dong–Li–Mason 2000 §§6–7). *Let V be C_2-cofinite and M an ordinary module with conformal weight h. Then for every $v \in V$ the coefficients of $Z_M(v;q)$ along $h - c/24 + \mathbb{N}$ grow subexponentially. Consequently every trace function $S_i(v,\tau)$ is given by an absolutely convergent series on $\mathbb{H}$.*

Lean. `VOA.onePointSubexp_of_c2`
File `VOA/Modularity/BlockGrowth.lean`.

```
theorem onePointSubexp_of_c2 {M : Type u} [AddCommGroup M] [Module ℂ M]
    (N : OrdVModule W M) {hw : ℂ} (hcw : HasConformalWeight N hw)
    (hzero : VacuumGradeZero W) (hC2 : C2Cofinite W) :
    OnePointSubexp N (hw - W.centralCharge / 24) := by
```

Hypotheses / status. C_2, conformal weight

Theorem 15.34 (Zhu's modularity theorem, pinned to a given family). *Let V be a vertex operator algebra of the library such that*

(a) V is C_2-cofinite;

(b) Zhu's algebra $A(V)$ is semisimple;

(c) $\Phi(V)$, the set of isomorphism classes of irreducible ordinary modules, is finite;

(d) the datum `Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))` *is given (Chapter 13).*

Let ι be a finite set with $\#\iota = \#\Phi(V)$ (hypothesis `hcard`*: one index per isomorphism class), and $(M_i, h_i)_{i\in\iota}$ irreducible ordinary modules with conformal weights h_i, whose bottoms $(M_i)_{h_i}$ are simple $A(V)$-modules, pairwise non-isomorphic whenever $h_j = h_k$, $j \neq k$. Then there are modular data D with $D.\mathrm{Mfam} = (M_i)$ and $D.h = (h_i)$ satisfying Zhu's analytic law: there is a homomorphism $\rho : SL_2(\mathbb{Z}) \to \mathrm{Mat}_\iota(\mathbb{C})$ with $\rho(T) = \mathrm{diag}(e^{2\pi i(h_i - c/24)})$ such that for all $\gamma \in SL_2(\mathbb{Z})$, $n \in \mathbb{N}$, $v \in V_{[n]}$, $i \in \iota$, $\tau \in \mathbb{H}$,*

$$\mathrm{tr}_{M_i}\, o(v) q^{L_0 - c/24}\big|_{\gamma\tau} = (c\tau + d)^n \sum_{j\in\iota} \rho(\gamma)_{ij}\ \mathrm{tr}_{M_j}\, o(v) q^{L_0 - c/24}\big|_{\tau}.$$

Since (d) is supplied for every V by the theorem `VOA.pbw` *(through* `voaPBWBottomProjection`*), the conclusion holds under (a)–(c) (this is* `zhuModularity_pinned_of_C2Cofinite_voaPBW`*).*

Lean. `VOA.zhuModularity_pinned_of_C2Cofinite`, `VOA.zhuModularity_pinned_of_C2Cofinite_voaPBW`
Files `VOA/Modularity/GNProp8Closure.lean` and `VOA/Modularity/GNProp8PBW.lean`.

```
theorem zhuModularity_pinned_of_C2Cofinite [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D :=
  zhuModularity_pinned_of_strictSpanning Mfam h hC2 hss Pr
    (strictSpanning_of_C2Cofinite W hC2) hcard hcw hsep

theorem zhuModularity_pinned_of_C2Cofinite_voaPBW [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D :=
  zhuModularity_pinned_of_C2Cofinite Mfam h hC2 hss
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

Theorem 15.35 (Finitely many irreducible modules; Dong–Li–Mason, Abe–Buhl–Dong). *If V is C_2-cofinite, then $\Phi(V)$ is finite.*

Lean. `VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw`
File `VOA/Classification/COne/McRaeO1FinitePhi.lean` (Chapter 22), in the namespace `VOA.Huang.ZhuFiniteIrr`, with section variable `{W : VertexOperatorAlgebra 𝒱}`.

```
theorem finite_Φ_of_C2Cofinite_pbw (hC2 : C2Cofinite W) : Finite (Φ W) :=
  finite_Φ_of_C2Cofinite' (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
    ↪ hC2
```

Hypotheses / status. C_2

Theorem 15.37 (Zhu's modularity theorem for one-point functions, canonical family, semisimple form). *Let V be a vertex operator algebra of the library (so of CFT type) such that*

(a) V is C_2-cofinite, and

(b) Zhu's algebra $A(V)$ is a semisimple ring.

Then $\Phi(V)$ is finite, and there are modular data D indexed by $\Phi(V)$, whose family is the canonical family $(M_c)_{c\in\Phi(V)}$ and whose weights are its conformal weights (h_c), satisfying Zhu's analytic law: there is a group homomorphism $\rho : SL_2(\mathbb{Z}) \to \mathrm{Mat}_{\Phi(V)}(\mathbb{C})$ with $\rho(T) = \mathrm{diag}(e^{2\pi i(h_c - c_V/24)})$ such that for all $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in SL_2(\mathbb{Z})$, $n \in \mathbb{N}$, $v \in V_{[n]}$, $c' \in \Phi(V)$ and $\tau \in \mathbb{H}$,

$$\mathrm{tr}_{M_{c'}}\, o(v) q^{L_0 - c_V/24}\big|_{\gamma\tau} = (c\tau + d)^n \sum_{c''\in\Phi(V)} \rho(\gamma)_{c'c''}\ \mathrm{tr}_{M_{c''}}\, o(v) q^{L_0 - c_V/24}\big|_{\tau},$$

where c_V is the central charge.

Lean. `VOA.zhuModularity_canonical_of_C2Cofinite_semisimple`
File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
theorem zhuModularity_canonical_of_C2Cofinite_semisimple (W : VertexOperatorAlgebra 𝒱)
    (hC2 : C2Cofinite W) (hss : letI := instRingA W; IsSemisimpleRing (A W)) :
    letI : Finite (Φ W) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw hC2
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ ZhuModularityAnalytic D
       ↪  := by
```

Theorem 15.38 (Zhu's semisimplicity theorem; Zhu 1996, Theorem 2.2.3; Dong–Li–Mason). *Let V be a vertex operator algebra of the library, and let the datum* `Pr` *of Chapter 13 be given. If V is rational in the sense of* `AdmissiblyRational` *(every admissible V-module is completely reducible; Chapter 12, Definition 12.35), then Zhu's algebra $A(V)$ is a semisimple ring. Since* `Pr` *is supplied for every V by the theorem* `VOA.pbw`*, the implication holds for every V; in particular the named predicate* `ZhuAdmissibleSemisimple W` *of Chapter 12 holds for every V.*

Lean. `VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational`, `VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr`
File `VOA/Huang/ZhuSemisimple.lean` (Chapter 21), namespace `VOA.Huang.ZhuSemisimple`, with section variables {`𝒱 : Type u`} [`AddCommGroup 𝒱`] [`Module ℂ 𝒱`] {`W : VertexOperatorAlgebra 𝒱`} and (`Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))`); each theorem is preceded by the line `include Pr in`.

```
theorem isSemisimpleRing_A_of_admissiblyRational (hrat : AdmissiblyRational W) :
    letI := instRingA W; IsSemisimpleRing (A W) := by

theorem zhuAdmissibleSemisimple_of_Pr : ZhuAdmissibleSemisimple W :=
  fun hrat => isSemisimpleRing_A_of_admissiblyRational Pr hrat
```

Theorem 15.39 (Zhu's modularity theorem for rational C_2-cofinite V; the chapter's headline). *Let V be a vertex operator algebra of the library (so of CFT type) such that*

(a) V is C_2-cofinite, and

(r) V is rational, in the sense of `AdmissiblyRational`*.*

Then the conclusion of Theorem 15.37 holds: $\Phi(V)$ is finite, and the one-point functions $\mathrm{tr}_{M_c}\, o(v) q^{L_0 - c_V/24}$ of the canonical irreducible modules M_c, $c \in \Phi(V)$, with their conformal weights h_c, transform under $SL_2(\mathbb{Z})$ by a matrix representation ρ with $\rho(T) = \mathrm{diag}(e^{2\pi i(h_c - c_V/24)})$, with weight n for $v \in V_{[n]}$. In particular this holds for every strongly rational V.

Lean. `VOA.zhuModularity_canonical_of_C2Cofinite_admissiblyRational`, `VOA.zhuModularity_canonical_of_stronglyRational`
File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
theorem zhuModularity_canonical_of_C2Cofinite_admissiblyRational (W : VertexOperatorAlgebra
 ↪  𝒱)
    (hC2 : C2Cofinite W) (hrat : AdmissiblyRational W) :
    letI : Finite (Φ W) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw hC2
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ ZhuModularityAnalytic D
       ↪  :=
  zhuModularity_canonical_of_C2Cofinite_semisimple W hC2
    (VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational
      (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))) hrat)

theorem zhuModularity_canonical_of_stronglyRational (W : VertexOperatorAlgebra 𝒱)
    (hSR : StronglyRational W) :
    letI : Finite (Φ W) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw hSR.c2Cofinite
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ ZhuModularityAnalytic D
       ↪  :=
  zhuModularity_canonical_of_C2Cofinite_admissiblyRational W hSR.c2Cofinite hSR.rational
```

Theorem 15.40 (Zhu's theorem for strongly rational V, the older packaging with two (now proved) literature hypotheses). *Let V be strongly rational (simple, self-dual, C_2-cofinite, rational; Definition 12.43), and assume the Dong–Li–Mason finiteness theorem (rational ⇒ finitely many irreducible admissible modules) and Zhu's semisimplicity theorem (rational ⇒ $A(V)$ semisimple) for V. Then, with $\iota = \Phi(V)$ and the canonical family of representatives with their conformal weights, the pinned conclusion of Theorem 15.34 holds. The proof uses the theorem* `VOA.pbw`.

Lean. `VOA.canonicalIrrModule`, `VOA.conformalWeightOf`, `VOA.zhuModularity_stronglyRational_voaPBW`
Files `VOA/Modularity/ZhuStronglyRational.lean`, `VOA/Modularity/StronglyRationalFamily.lean`, `VOA/Modularity/ConformalWeightChoice.lean`.

```
noncomputable def canonicalIrrModule (c : Φ W) : IrrModule W := Quotient.out c

noncomputable def conformalWeightOf (V : IrrModule W) : ℂ :=
  (OrdVModule.exists_hasConformalWeight_of_irreducible V.isMod V.irreducible).choose

theorem zhuModularity_stronglyRational_voaPBW
    (hSR : StronglyRational W)
    (hFin : DLMFinitelyManyIrredAdmissible W)
    (hZhu : ZhuAdmissibleSemisimple W) :
    letI : Finite (Φ W) := finite_Φ_of_finite_Φadm W (hFin hSR.rational)
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W),
      D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧
      ZhuModularityAnalytic D := by
```

Theorem 15.43 (Zhu's theorem for V_{2m}^+ and for the tetrahedral model, with no hypotheses). *(i) For every $m \geq 2$, the conclusion of Theorem 15.37 holds for $V = V_{2m}^+$: $\Phi(V_{2m}^+)$ is finite, and the one-point functions of its canonical irreducible modules, with their conformal weights, transform under $SL_2(\mathbb{Z})$ by a matrix representation with diagonal $\rho(T)$. (ii) The same holds for the tetrahedral model.*

Lean. `VOA.Classification.COne.zhuModularity_V2mPlus`, `VOA.Classification.COne.zhuModularity_cubicModel`, `VOA.Classification.COne.V2mPlus_c2Cofinite`, `VOA.Classification.COne.V2mPlus_zhuSemisimple`, `VOA.Classification.COne.V2mPlus_zhu_finrank`, `VOA.Classification.COne.ZhuV2mPlusSemisimple`, `VOA.Classification.COne.OrbA4.cubicModel_c2Cofinite_holds`, `VOA.Classification.COne.OrbTop.cubicModel_zhuSemisimple` File `VOA/Classification/COne/ZhuModularityV2mPlus.lean`, namespace `VOA.Classification.COne`.

```
theorem zhuModularity_V2mPlus (m : ℕ) (hm : 1 ≤ m) (hm2 : 2 ≤ m) :
    letI : Finite (Φ (V2mPlus m hm)) :=
      VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw (V2mPlus_c2Cofinite m hm)
    letI : Fintype (Φ (V2mPlus m hm)) := Fintype.ofFinite _
    letI : DecidableEq (Φ (V2mPlus m hm)) := Classical.decEq _
    ∃ D : VOA.ModularData (V2mPlus m hm) (Φ (V2mPlus m hm)), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_C2Cofinite_semisimple (V2mPlus m hm) (V2mPlus_c2Cofinite m hm)
    (V2mPlus_zhuSemisimple m hm hm2)

theorem zhuModularity_cubicModel :
    letI : Finite (Φ OrbRat2.cubicModel) :=
      VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw OrbA4.cubicModel_c2Cofinite_holds
    letI : Fintype (Φ OrbRat2.cubicModel) := Fintype.ofFinite _
    letI : DecidableEq (Φ OrbRat2.cubicModel) := Classical.decEq _
    ∃ D : VOA.ModularData OrbRat2.cubicModel (Φ OrbRat2.cubicModel),
      D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_C2Cofinite_semisimple OrbRat2.cubicModel
    OrbA4.cubicModel_c2Cofinite_holds OrbTop.cubicModel_zhuSemisimple
```

```
theorem V2mPlus_c2Cofinite (m : ℕ) (hm : 1 ≤ m) : C2Cofinite (V2mPlus m hm) :=
  v2mPlusC2CofiniteFact_holds m hm

theorem V2mPlus_zhuSemisimple : ZhuV2mPlusSemisimple :=
  zhuSemisimple_of_fourExtraCharacters
    (fourExtraCharacters_of_twistedOmegaECharacters V2mPlus_twistedOmegaECharacters)

theorem V2mPlus_zhu_finrank (m : ℕ) (hm : 1 ≤ m) (hm2 : 2 ≤ m) :
    Module.finrank ℂ (VOA.A (V2mPlus m hm)) = m + 7 :=
```

```
def ZhuV2mPlusSemisimple : Prop :=
  ∀ (m : ℕ) (hm : 1 ≤ m), 2 ≤ m →
    letI := instRingA (V2mPlus m hm)
    IsSemisimpleRing (VOA.A (V2mPlus m hm))
```

```
theorem cubicModel_c2Cofinite_holds : C2Cofinite cubicModel := by

theorem cubicModel_zhuSemisimple : IsSemisimpleRing (VOA.A cubicModel) :=
  OrbRat2.isSemisimpleRing_of_charTable OrbA4.cubicModel_c2Cofinite_holds
    OrbSgl.cubicZhuDimensionBound_holds OrbRat6.cubicZhuCharacterTable_holds
```

Theorem 15.44 (Zhu's theorem for the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$, and for every nice VOA). *(i) For every $m \geq 1$, the conclusion of Theorem 15.37 holds for the rank-one lattice VOA $V = V_{2m} = V_{\sqrt{2m}\mathbb{Z}}$,*

with no further hypotheses. (ii) It holds for every nice *vertex operator algebra V, i.e. every V that is strongly rational and pseudo-unitary (Gannon–Rayhaun, Definition 2.6).*

Lean. `VOA.Classification.COne.zhuModularity_V2m`, `VOA.Classification.COne.zhuModularity_of_nice`, `VOA.Classification.COne.Nice`, `VOA.Classification.COne.V2m`, `VOA.Classification.COne.V2m_stronglyRational`
File `VOA/Classification/COne/ZhuModularityV2mPlus.lean`, namespace `VOA.Classification.COne`.

```
theorem zhuModularity_V2m (m : ℕ) (hm : 1 ≤ m) :
    letI : Finite (Φ (V2m m hm)) :=
      VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw (V2m_stronglyRational m
       ↪ hm).c2Cofinite
    letI : Fintype (Φ (V2m m hm)) := Fintype.ofFinite _
    letI : DecidableEq (Φ (V2m m hm)) := Classical.decEq _
    ∃ D : VOA.ModularData (V2m m hm) (Φ (V2m m hm)), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_stronglyRational (V2m m hm) (V2m_stronglyRational m hm)

theorem zhuModularity_of_nice {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (V : VertexOperatorAlgebra 𝒱) (hV : Nice V) :
    letI : Finite (Φ V) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw
     ↪ hV.1.c2Cofinite
    letI : Fintype (Φ V) := Fintype.ofFinite _
    letI : DecidableEq (Φ V) := Classical.decEq _
    ∃ D : VOA.ModularData V (Φ V), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_stronglyRational V hV.1
```

```
def Nice (V : VertexOperatorAlgebra 𝒱) : Prop :=
  StronglyRational V ∧ PseudoUnitary V

noncomputable def V2m (m : ℕ) (hm : 1 ≤ m) : VertexOperatorAlgebra (LatModuleGen (latP m))
 ↪ :=
  latticeVOAGen (latP m) (latP_even m) (latP_posDef m hm)

theorem V2m_stronglyRational (m : ℕ) (hm : 1 ≤ m) : StronglyRational (V2m m hm) :=
  { V2m_stronglyFinite m hm with rational := V2m_admissiblyRational hm }
```

Chapter 16

Quasi-lisse vertex algebras and the Arakawa–Kawasetsu theorem

13 of the chapter's 43 numbered statements: its headline results and every definition they use.

Theorem 16.4 (Etingof–Schedler 2010). *If R is a finitely generated Poisson algebra whose variety* Specm R *has finitely many symplectic leaves, then* $\dim \mathrm{HP}_0(R) < \infty$.

(The chapter quotes no Lean declaration for this statement; see book p. 800.)

Definition 16.9 (Etingof-finite). V is *Etingof-finite* if $\dim_{\mathbb{C}} \mathrm{HP}_0(R_V) < \infty$.

Lean. `VOA.EtingofFinite`
File `VOA/Modules/PoissonTrace.lean`.

```
def EtingofFinite (W : VertexOperatorAlgebra 𝒱) : Prop := FiniteDimensional ℂ (HP₀ W)
```

Hypotheses / status. Definition; **not** quasi-lisseness (weaker, by Etingof–Schedler)

Theorem 16.16 (Growth of one-point functions from Etingof-finiteness). *If V is Etingof-finite and M is an ordinary module of conformal weight h (that is, $M_h \neq 0$ and every nonzero weight space M_α has $\alpha \in h + \mathbb{N}$, Chapter 12), then for every $v \in V$ the coefficients of the torus one-point function $Z_M(v;q) = \mathrm{tr}_M\, o(v) q^{L_0 - c/24}$ grow subexponentially: for each $r > 1$ there is C with $|[q^{h-c/24+n}]Z_M(v;q)| \le Cr^n$.*

Lean. `VOA.onePointSubexp_of_etingofFinite`, `VOA.OnePointSubexp`, `VOA.Subexponential`
File `VOA/Modules/QuasiLisseFinite.lean`.

```
theorem onePointSubexp_of_etingofFinite {M : Type u} [AddCommGroup M] [Module ℂ M]
    (N : OrdVModule W M) {hw : ℂ} (hcw : HasConformalWeight N hw)
    (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    OnePointSubexp N (hw - W.centralCharge / 24) := by
```

Files `VOA/Modularity/AnalyticBlock.lean`, `VOA/Modularity/ConvergenceBridge.lean`.

```
def OnePointSubexp (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ v : 𝒱, Subexponential (fun n : ℕ => ‖torusOnePoint N v (h + (n : ℂ))‖)

def Subexponential (a : ℕ → ℝ) : Prop :=
  ∀ r : ℝ, 1 < r → ∃ C : ℝ, ∀ n : ℕ, a n ≤ C * r ^ n
```

Hypotheses / status. `hcw`, `hE`

Definition 16.19 (Formal genus-one block; Zhu Def. 5.1.1, formal form). A linear map $S : V \to \{\text{formal } q\text{-series}\}$ is a *formal block* if for every homogeneous a of weight $k_a \geq 1$ and every $v \in V$: (i) $S(a[0]v) = 0$; (ii) $S(a[-2]v) + \sum_{m\geq 0}(2m+1)\,G_{2m+2} \star S(a[2m]v) = 0$; (iii) $S(L[-2]v) = \theta S(v) + \sum_{m\geq 0} G_{2m+2} \star S(L[2m]v)$, where $\star$ is the product of a power series with a formal q-series (`qConv`) and G_{2j} is Zhu's normalised Eisenstein series (below).

Lean. `VOA.IsFormalBlock`, `VOA.zhuG`
File `VOA/Modularity/ZhuBlockConditions.lean` (Chapter 15).

```
structure IsFormalBlock (W : VertexOperatorAlgebra 𝒱) (S : 𝒱 →ₗ[ℂ] FormalQSeries) : Prop
 ↪  where
  vanish : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : 𝒱, S (vSqZ W a ka 0 v) = 0
  negTwo : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : 𝒱,
    S (vSqZ W a ka (-2) v)
      + ∑ᶠ m : ℕ, ((2 * (m + 1) - 1 : ℕ) : ℂ)
          • qConv (zhuG (m + 1)) (S (vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v)) = 0
  omega : ∀ v : 𝒱, S (Lsq W (-2) v)
    = qDeriv (S v) + ∑ᶠ m : ℕ, qConv (zhuG (m + 1)) (S (Lsq W (2 * (m + 1) - 2 : ℤ) v))
```

File `VOA/Modularity/ZhuNormalization.lean` (Chapter 14).

```
noncomputable def zhuG (j : ℕ) : ℕ → ℂ :=
  fun m =>
    if m = 0 then -(((bernoulli (2 * j) : ℚ) : ℂ) / (Nat.factorial (2 * j) : ℂ))
    else (2 / (Nat.factorial (2 * j - 1) : ℂ)) * lambertCoeff (2 * j - 1) m
```

Theorem 16.20 (AK Lemma 5.5, for primary vectors). *Let S be a formal block supported on $h+\mathbb{N}$ and a a square-bracket primary. For each i there are $\eta_{i,0}, \dots, \eta_{i,i} \in \mathcal{S}$ with $\eta_{i,i} = 1$ and $S(L[-2]^i a) = \sum_{j=0}^{i} \eta_{i,j}\,\theta^j S(a)$.*
Lean. `VOA.huni_of_sqPrimary`
File `VOA/Modularity/AKModularODE.lean`.

```
theorem huni_of_sqPrimary (W : VertexOperatorAlgebra 𝒱)
    {S : 𝒱 →ₗ[ℂ] FormalQSeries} {hbase : ℂ} (hsupp : ∀ v, S v ∈ QSeriesAt hbase)
    (hS : IsFormalBlock W S) {a : 𝒱} {Δ : ℂ} (ha : IsSqPrimary W a Δ) (i : ℕ) :
    ∃ hc : ℕ → PowerSeries ℂ, (∀ j, hc j ∈ SubexpPS) ∧ hc i = 1 ∧ (∀ j, i < j → hc j = 0) ∧
      S ((↟(Lsq W (-2)))^[i] a)
        = ∑ j ∈ Finset.range (i + 1), psSMul (hc j) (qDeriv^[j] (S a)) := by
```

Hypotheses / status. Primary a

Proposition 16.24 (The formal equation is almost a growth statement). *Let $\chi = q^h F(q)$ with $F = \sum_{n\geq 0} F_n q^n$, $F_0 \neq 0$, and suppose F converges on the open unit disc D and its zeros in D have multiplicity at most m. Then for $s = m+1$ there are $g_0, \dots, g_{s-1} \in \mathcal{S}$ with $\theta^s\chi + \sum_{i<s} g_i \theta^i \chi = 0$.*
Lean. `VOA.character_formalODE_of_etingofFinite`, `VOA.character_formalODE_adjoint_of_etingofFinite`
File `VOA/Modularity/AKModularODE.lean`.

```
theorem character_formalODE_of_etingofFinite (W : VertexOperatorAlgebra 𝒱) (N : OrdVModule
 ↪  W M)
    {hw : ℂ} (hcw : HasConformalWeight N hw) (hzero : VacuumGradeZero W) (hE :
     ↪  EtingofFinite W) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
        + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 := by
```

```
theorem character_formalODE_adjoint_of_etingofFinite (W : VertexOperatorAlgebra 𝒱)
    (hne : W.vacuum ≠ 0) (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition (adjointModule W))) e
        + ∑ i ∈ Finset.range s,
            qConv (g i) (qDeriv^[i] (torusPartition (adjointModule W))) e = 0 :=
```

Hypotheses / status. `hcw`, `hE`

Definition 16.25 (Monic MLDE on $\mathbb{H}$). For $f : \mathbb{H} \to \mathbb{C}$, f satisfies a monic MLDE of order $s \geq 1$ and weight w with coefficients $p_0, \dots, p_{s-1}$ if each p_i is a modular form of weight $2(s-i)$ and $\vartheta^s_{(w)} f + \sum_{i<s} p_i \vartheta^i_{(w)} f = 0$ on $\mathbb{H}$.

Lean. `VOA.serreIterC`, `VOA.MonicMLDEOn`
File `VOA/Modularity/AKMLDE.lean`.

```
noncomputable def serreIterC (wt : ℤ) : ℕ → (ℂ → ℂ) → (ℂ → ℂ)
  | 0, f => f
  | i + 1, f => serreDerivC (wt + 2 * (i : ℤ)) (serreIterC wt i f)

def MonicMLDEOn (wt : ℤ) (f : ℂ → ℂ) (s : ℕ) (p : ℕ → E46) : Prop :=
  1 ≤ s ∧
    (∀ i < s, E46Weight (2 * ((s - i : ℕ) : ℤ)) (p i)) ∧
    ∀ z ∈ imGT 0, serreIterC wt s f z
      + ∑ i ∈ Finset.range s, e46C (p i) z * serreIterC wt i f z = 0
```

Hypotheses / status. Definitions; `AKWeightGrading` inhabited for every VOA (`akWeightGrading`); `E46SerreClosed` proved (`e46SerreClosed`)

Definition 16.26 (AK weight grading). Let $V_{\mathcal{A}}^E = \mathbb{C}[E_4, E_6] \otimes V$ and let its total-weight-n subspace be spanned by $f \otimes v$ with f of modular weight p, $L[0]v = qv$, $p+q = n$. An *AK weight grading* consists of: a direct-sum decomposition of $\mathbb{C}[E_4, E_6]$ into its weight spaces; a direct-sum decomposition of $V_{\mathcal{A}}^E$ into total-weight spaces; and the statement that the relation module $O_q^E(V)$ (`blockRelSpanE W`, the E_4, E_6-version of $O_q(V)$) is homogeneous.

Lean. `VOA.e46WeightSpace`, `VOA.akTotalWeightSpace`, `VOA.AKWeightGrading`
File `VOA/Modularity/AKGrading.lean`.

```
noncomputable def e46WeightSpace (k : ℤ) : Submodule ℂ E46 where
  carrier := {f | E46Weight k f}

noncomputable def akTotalWeightSpace (W : VertexOperatorAlgebra 𝒱) (n : ℤ) :
    Submodule ℂ (𝒱E 𝒱) :=
  Submodule.span ℂ {x | ∃ (p q : ℤ) (f : E46) (v : 𝒱), p + q = n ∧
    f ∈ e46WeightSpace p ∧ v ∈ sqWeightSpace W (q : ℂ) ∧ x = f ⊗ₜ[ℂ] v}

structure AKWeightGrading (W : VertexOperatorAlgebra 𝒱) where
  coeffDecomposition : DirectSum.Decomposition e46WeightSpace
  totalDecomposition : DirectSum.Decomposition (akTotalWeightSpace W)
  relationHomogeneous :
    letI := totalDecomposition
    DirectSum.SetLike.IsHomogeneous (akTotalWeightSpace W)
      ((blockRelSpanE W).restrictScalars ℂ)
```

Hypotheses / status. Definitions; `AKWeightGrading` inhabited for every VOA (`akWeightGrading`); `E46SerreClosed` proved (`e46SerreClosed`)

Theorem 16.29 (AK Theorem 5.1, conditional form). *Let V be Etingof-finite, $a \in V$ square-bracket primary of integer weight w, and assume an AK weight grading for V and* `E46SerreClosed`*. Then there are $s \geq 1$ and $p_0, \dots, p_{s-1}$ such that for* every *analytic genus-one block F the function $F(a)$ satisfies the monic MLDE of Definition 16.25 of weight w with these coefficients. Both side hypotheses are theorems:* `AKWeightGrading W` *is* `akWeightGrading W` *(Theorem 16.27) and* `E46SerreClosed` *is* `e46SerreClosed`*;* `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite'` *discharges the second and Theorem 16.30 below discharges both.*

Lean. `VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite`
File `VOA/Modularity/AKMLDE.lean`.

```
theorem IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite
    {W : VertexOperatorAlgebra 𝒱} {a : 𝒱} {wt : ℤ}
    (ha : IsSqPrimary W a (wt : ℂ)) (hgrade : AKWeightGrading W)
    (hclose : E46SerreClosed) (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    ∃ (s : ℕ) (p : ℕ → E46), 1 ≤ s ∧
      ∀ (F : 𝒱 → ℂ → ℂ), IsAnalyticBlock W F → MonicMLDEOn wt (F a) s p := by
```

Hypotheses / status. `hE`, `hgrade`, `hclose`; `hclose` discharged in the primed form

Theorem 16.30 (AK Theorem 5.1 under Etingof-finiteness; the $L_{-4/3}(\mathfrak{sl}_2)$ instance). *(i) Let V be Etingof-finite and $a \in V$ square-bracket primary of integer weight w. Then there are $s \geq 1$ and coefficients $p_i \in \mathbb{C}[E_4, E_6]$, modular of weight $2(s-i)$, such that for every analytic genus-one block F the function $F(a)$ satisfies $\vartheta^{(s)}F(a) + \sum_{i<s} p_i \vartheta^{(i)}F(a) = 0$ on $\mathbb{H}$ (iterated Serre derivatives). (ii) For $V = L_{-4/3}(\mathfrak{sl}_2)$ and $a = \mathbf{1}$ this holds with no hypothesis.*

Lean. `VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`, `VOA.exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL`
File `VOA/Modularity/AKMLDEUnconditional.lean`.

```
theorem IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond
    {W : VertexOperatorAlgebra 𝒱} {a : 𝒱} {wt : ℤ}
    (ha : IsSqPrimary W a (wt : ℂ)) (hE : EtingofFinite W) :
    ∃ (s : ℕ) (p : ℕ → E46), 1 ≤ s ∧
      ∀ (F : 𝒱 → ℂ → ℂ), IsAnalyticBlock W F → MonicMLDEOn wt (F a) s p :=
  ha.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite' (akWeightGrading W)
    (VOA.QuasiLisseSl2.vacuumGradeZero_of_vacuum_grade W) hE

theorem exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL :
    ∃ (s : ℕ) (p : ℕ → E46), 1 ≤ s ∧
      ∀ F, IsAnalyticBlock simpleSl2QL F →
        MonicMLDEOn 0 (F simpleSl2QL.vacuum) s p :=
  IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond
    (wt := 0) (by exact_mod_cast vacuum_isSqPrimary simpleSl2QL)
    ↪ VOA.Sl2Pres.etingofFinite_simpleSl2QL'
```

Hypotheses / status. `hE` only (despite the name); order unbounded

Theorem 16.32 (Zhu modularity under Etingof-finiteness, conditional). *Let V be Etingof-finite and $(M_i)_{i \in \iota}$ a finite family of irreducible modules. If the trace functions are independent on each T-eigenvalue fibre (*`hfib`*) and the restricted analytic blocks together with the S-slashes of the traces span a space of dimension at most $|\iota|$ (*`hrank`*), then there is an $SL_2(\mathbb{Z})$-representation ρ with $Z_{M_i}(v; A\tau) = (c\tau + d)^n \sum_j \rho(A)_{ij} Z_{M_j}(v; \tau)$ for all v of $L[0]$-weight n.*

Lean. `VOA.exists_zhuModularity_of_sqAnalyticRank_le_irr_etingofFinite`
File `VOA/Modularity/AKZhuModularity.lean`.

```
theorem exists_zhuModularity_of_sqAnalyticRank_le_irr_etingofFinite
    (hE : EtingofFinite W)
    (hfib : FibreIndependent Mfam (fun i => conformalWeightOf (Mfam i)))
    (hrank : Module.rank ℂ
        (sqAnalyticSlashAmbient Mfam (fun i => conformalWeightOf (Mfam i)))
      ≤ (Fintype.card ι : Cardinal)) :
    ∃ D : ModularData W ι, ZhuModularityAnalytic D :=
```

Hypotheses / status. `hE`, `hfib`, `hrank` (discharged only for the trivial VOA; `hrank` expected **false** at $L_{-4/3}$, paper argument)

Theorem 16.38 (The presentation of $R_{L_{-4/3}}$ is constructed). *(i) For every non-critical k, the algebra map* $\mathrm{Sym}(\mathfrak{sl}_2) \to R_{V^k(\mathfrak{sl}_2)}$, $x \mapsto [x_{-1}\mathbf{1}]$, *is surjective. (ii) In* $R_{L_{-4/3}}$, $[e] \cdot \mathrm{Cas} = 0$, *and the whole ideal* $\mathrm{Cas} \cdot \mathfrak{m}$ *maps to* 0. *(iii) These maps form an $\mathfrak{sl}_2$ symbol presentation of $L_{-4/3}(\mathfrak{sl}_2)$. (iv) Hence $L_{-4/3}(\mathfrak{sl}_2)$ is Etingof-finite.*

Lean. `VOA.Sl2Pres.symbolAlg`, `VOA.Sl2Pres.symbolAlg_surjective`, `VOA.Sl2Pres.symbolAlgL`, `VOA.Sl2Pres.eL_mul_casL`, `VOA.Sl2Pres.cas_vanishes_L`, `VOA.Sl2Pres.sl2PresentationSimpleSl2QL`, `VOA.Sl2Pres.etingofFinite_simpleSl2QL'`
File `VOA/Examples/AffineSl2Presentation.lean`.

```
def symbolAlg : SymSl2 →ₐ[ℂ] A₂ (affineSl2VOA_closed k hk) :=
  MvPolynomial.aeval ![eCl k hk, hCl k hk, fCl k hk]

theorem symbolAlg_surjective : Function.Surjective (symbolAlg k hk) := by

def symbolAlgL : SymSl2 →ₐ[ℂ] A₂ simpleSl2QL := MvPolynomial.aeval ![eL, hL, fL]

theorem eL_mul_casL : eL * casL = 0 := by

theorem cas_vanishes_L : ∀ p ∈ casIdeal, symbolAlgL p = 0 := by

def sl2PresentationSimpleSl2QL : Sl2Presentation simpleSl2QL where

theorem etingofFinite_simpleSl2QL' : EtingofFinite simpleSl2QL :=
  sl2PresentationSimpleSl2QL.etingofFinite
```

Hypotheses / status. unconditional, `pbw`-free

Theorem 16.40 (Non-lisseness of $L_{-4/3}(\mathfrak{sl}_2)$). *Let $E_r = e_{-1}^r\mathbf{1}$ and $F_r = f_{-1}^r\mathbf{1}$ in $V = V^{-4/3}(\mathfrak{sl}_2)$, and $\langle\cdot,\cdot\rangle$ the canonical invariant bilinear form. (i) $\langle F_r, x\rangle = 0$ for all $x \in C_2(V)$. (ii) $\langle F_r, E_r\rangle = (-1)^r\, r!\, \prod_{j<r}(k-j) \neq 0$ at $k = -4/3$. (iii) The class of E_r in R_L, $L = L_{-4/3}(\mathfrak{sl}_2)$, is nonzero, and these classes are linearly independent. (iv) L is not C_2-cofinite. (v) L is not C_2-cofinite and is Etingof-finite.*

Lean. `VOA.Sl2NonLisse.form_fHarmonic_O₂_zero`, `VOA.Sl2NonLisse.form_fHarmonic_eHarmonic_ne_zero`, `VOA.Sl2NonLisse.eHarmonicL_not_mem_O₂`, `VOA.Sl2NonLisse.eHarmonicL_C₂_linearIndependent`, `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL`, `VOA.Sl2NonLisse.strictEtingofFinite_simpleSl2QL`
File `VOA/Examples/AffineSl2NonLisse.lean`.

```
theorem form_fHarmonic_O₂_zero (r : ℕ) {x : WakModule kk} (hx : x ∈ O₂ W43) :
    LQ43.formAdm.form (fHarmonic kk r) x = 0 := by

theorem form_fHarmonic_eHarmonic_ne_zero (r : ℕ) :
    LQ43.formAdm.form (fHarmonic kk r) (eHarmonic kk r) ≠ 0 := by

theorem eHarmonicL_not_mem_O₂ (r : ℕ) : eHarmonicL r ∉ O₂ L43 := by
```

```
theorem eHarmonicL_C₂_linearIndependent :
    LinearIndependent ℂ (fun r : ℕ => mk₂ L43 (eHarmonicL r)) := by

theorem not_C2Cofinite_simpleSl2QL : ¬ C2Cofinite L43 := by

theorem strictEtingofFinite_simpleSl2QL :
    ¬ C2Cofinite L43 ∧ EtingofFinite L43 :=
  ⟨not_C2Cofinite_simpleSl2QL, Sl2Pres.etingofFinite_simpleSl2QL'⟩
```

Hypotheses / status. unconditional, pbw-free

Part V

Super, logarithmic and unitary theories

Chapter 17

The super generalization

9 of the chapter's 58 numbered statements: its headline results and every definition they use.

Theorem 17.4 (No translation covariance). *There is a three-dimensional super vertex algebra in the library's sense (purely even, weights 0 and 2) on which the super Jacobi identity fails at $(u, v, x, \ell, m, n) = (x_1, \mathbf{1}, \mathbf{1}, -2, 0, -2)$ and on which $Y(Tu, z) \neq \partial_z Y(u, z)$.*

(The chapter quotes no Lean declaration for this statement; see book p. 850.)

Theorem 17.14 (Super Borcherds / Jacobi identity). *Let V be a super near-VOA, $u \in V_{\tilde{p}}$, $v \in V_{\tilde{q}}$, $x \in V$. For all $\ell, m, n \in \mathbb{Z}$,*

$$\sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+m-i)}v_{(n+i)}x - (-1)^{pq}(-1)^{\ell}\, v_{(\ell+n-i)}u_{(m+i)}x\Big) = \sum_{i\geq 0}\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x.$$

Lean. `VOA.SuperNearVOA.superBorcherds`, `VOA.SuperNearVOA.superJacobi`
File `VOA/Core/SuperBorcherds.lean`.

```
theorem superBorcherds (V : SuperNearVOA 𝒱) {p q : ZMod 2} {u v : 𝒱} (hu : u ∈ V.parity p)
    (hv : v ∈ V.parity q) (x : 𝒱) (ℓ m n : ℤ) :
    BLε V (koszul p q) u v x ℓ m n = BRs V u v x ℓ m n := by
...
theorem superJacobi (V : SuperNearVOA 𝒱) {p q : ZMod 2} {u v : 𝒱} (hu : u ∈ V.parity p)
    (hv : v ∈ V.parity q) (x : 𝒱) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (V.smul u (ℓ + m - i) (V.smul v (n + i) x)
          - (koszul p q * negOnePow ℓ) • V.smul v (ℓ + n - i) (V.smul u (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • V.smul (V.smul u (ℓ + i) v) (m + n - i) x := by
```

Proposition 17.21 (The adjoint super module). *Every super near-VOA is a super weak module over itself, with $Y_M = Y$ and its own parity.*

Lean. `VOA.SuperNearVOA.adjointSuperWeakModule`
File `VOA/Modules/SuperAdjointModule.lean`.

```
def SuperNearVOA.adjointSuperWeakModule (V : SuperNearVOA 𝒱) :
    SuperWeakModule V.toSuperVertexAlgebra 𝒱 where
  YM := V.Y
```

Theorem 17.24 (Twisted super modules with $N \geq 2$ are zero). *Let W be any super VOA, g any even automorphism, $N \geq 2$, and T any* `SuperTwistedModule W g N M`*. Then every $x \in M$ is 0. In particular M is a subsingleton, every weight space of T is zero, and every parity-twisted (Ramond) carrier* `W.ParityTwistedModule M` *is zero.*

(The chapter quotes no Lean declaration for this statement; see book p. 864.)

Theorem 17.27 (ψ is not a field). $\neg$ IsField(ψ)*: there is no B with $\psi(n)\mathbf{1} = 0$ for all $n < B$.*
(The chapter quotes no Lean declaration for this statement; see book p. 867.)

Theorem 17.38 (The super Zhu ring of symplectic fermions is not semisimple). *The super Zhu algebra $A(SF)$ (Definition 17.42) is an associative ring of dimension at most 4, $[\chi^a] \neq 0$, and the two-sided ideal generated by $[\chi^0]$ is nonzero and square-zero; hence $A(SF)$ is not a semisimple ring.*

Lean. `VOA.SympFerm.finrank_ASF_le_four`, `VOA.SympFerm.chiClass_ne_zero`, `VOA.SympFerm.ASF_not_isSemisimpleRing`
File `VOA/Examples/SympFermSuperZhu.lean`.

```
theorem finrank_ASF_le_four : Module.finrank ℂ ASF ≤ 4 := by
...
theorem chiClass_ne_zero (a : Fin 2) : sfN.superMk hSF (chiStateSF a) ≠ 0 :=
...
theorem ASF_not_isSemisimpleRing :
    letI := instRingASF
    ¬ IsSemisimpleRing ASF :=
```

Hypotheses / status. Unconditional

Definition 17.42 (Super Zhu algebra). Let V be a super near-VOA with integer weights. For $u \in V_k$ put $u * v = \sum_{m=0}^{k} \binom{k}{m} u_{(m-1)}v$, extended linearly; $O(V) = \mathrm{span}\{(L_{-1}u + L_0 u) * v\}$; $A(V) = V/O(V)$.

Lean. `VOA.SuperNearVOA.superZhuFamily`, `VOA.SuperNearVOA.superO`, `VOA.SuperNearVOA.superA`
File `VOA/Modules/SuperZhuRing.lean`.

```
noncomputable def superZhuFamily (v : 𝒱) (k : ℤ) : (V.gradingZ k) →ₗ[ℂ] 𝒱 where
  toFun uk := ∑ m ∈ Finset.range (k.toNat + 1), cbz k m • V.smul (uk : 𝒱) ((m : ℤ) - 1) v
...
def superO (hV : V.IntegralWeights) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = V.superZhuMul hV (V.L (-1) u + V.L 0 u) v}
...
abbrev superA (hV : V.IntegralWeights) : Type u := 𝒱 ⧸ V.superO hV
```

Theorem 17.45 (Twisted Zhu finiteness). *If W is integrally graded and super C_2-cofinite, $g^N = 1$ with $N > 0$, and the eigenspace decomposition of g is internal and compatible with weight and parity, then $A_g(W)$ is finite-dimensional.*

Lean. `VOA.finiteDimensional_twistedA_of_superC2Cofinite`
File `VOA/Modules/SuperTwistedZhuFinite.lean`.

```
theorem finiteDimensional_twistedA_of_superC2Cofinite
    (W : SuperVertexOperatorAlgebra 𝒱) (hW : W.toSuperNearVOA.IntegralWeights)
    (g : SuperAut W) {N : ℕ} (hN : 0 < N) (_hpow : g ^ N = 1)
    (hE : TwistEigenspaceDecomposition W g N) (hC2 : SuperC2Cofinite W) :
    FiniteDimensional ℂ (W.twistedA hW g N) := by
```

Theorem 17.58 (Finite square-spin cusp quotient). *Let W be a half-integrally graded super VOA ($W_h = 0$ unless $h \in \frac{1}{2}\mathbb{Z}$), super C_2-cofinite, and (g, h) a commuting pair of involutive even automorphisms. Then the quotient of W by the square-spin cusp relation span of the $(+,+)$ inner eigensector of (g, h) is finite-dimensional.*

Lean. VOA.SuperNearVOA.BoundaryPair.finiteDimensional_superSpinSqCuspQuotient_of_superC2, VOA.BoundaryPair
File VOA/Modularity/SuperSpinDescentSqFinite.lean.

```
theorem BoundaryPair.finiteDimensional_superSpinSqCuspQuotient_of_superC2
    (b : BoundaryPair W) (hhalf : IsHalfIntegerGraded W)
    (hspatial : b.spatial ^ 2 = 1) (htemporal : b.temporal ^ 2 = 1)
    (hC2 : SuperC2Cofinite W) :
    FiniteDimensional Complex
      (VType / superSpinSqCuspRelSpan W hhalf b.spinEigenspace) :=
```

File VOA/Modularity/SuperSectorSystem.lean.

```
structure BoundaryPair (W : SuperVertexOperatorAlgebra VType) where
  spatial : SuperAut W
  temporal : SuperAut W
  commute : Commute spatial temporal
```

Chapter 18

Logarithmic vertex algebras: the triplet and doublet algebras

12 of the chapter's 43 numbered statements: its headline results and every definition they use.

Theorem 18.6 (Central charge; non-degeneracy; consistency with the Abe model). *$W(p)$ has central charge $c_{1,p}$; it contains $\mathbf{1}, \omega, E, H, F$; it is nonzero; and at $p = 2$ its central charge equals that of* `SympFerm.tripletW2`.

Lean. `VOA.LatGen.Wp.tripletW_centralCharge`, `VOA.LatGen.Wp.tripletW_carrier_ne_bot`, `VOA.LatGen.Wp.omegaTrip_ne_zero`, `VOA.LatGen.Wp.tripletW_centralCharge_eq_tripletW2`
File `VOA/Examples/TripletWp.lean`.

```
@[simp] theorem tripletW_centralCharge (p : ℕ) (hp : 1 ≤ p) :
    (tripletWVOA p hp).centralCharge = 1 - 6 * ((p : ℂ) - 1) ^ 2 / (p : ℂ) := rfl

theorem tripletW_carrier_ne_bot (p : ℕ) (hp : 1 ≤ p) :
    (tripletW p hp).carrier ≠ ⊥ := by

theorem omegaTrip_ne_zero (p : ℕ) (hp : 1 ≤ p) : omegaTrip p ≠ 0 := by

theorem tripletW_centralCharge_eq_tripletW2 (hp : 1 ≤ 2) :
    (tripletWVOA 2 hp).centralCharge = SympFerm.tripletW2.centralCharge := by
```

Theorem 18.7 ($E \neq 0$ and $W(2) \neq V_{L_2}$; Lean check outside the library). *At $p = 2$, $E \neq 0$, and $e^{\alpha} \notin W(2)$; hence $W(2)$ is a proper subalgebra of V_{L_2}.*

(The chapter quotes no Lean declaration for this statement; see book p. 912.)
Hypotheses / status. None

Theorem 18.10 (Adamović–Milas relations). *For every $p \geq 1$ and every $n \geq -2p$, in V_{L_p}:*

$$F_{(n)}F = 0, \qquad E_{(n)}E = 0, \qquad E_{(n)}F + F_{(n)}E + 2H_{(n)}H = 0,$$

and Q is a derivation of all products: $Q(u_{(n)}v) = (Qu)_{(n)}v + u_{(n)}(Qv)$. Consequently, in $R_{W(p)}$,

$$\bar{E}^2 = 0, \qquad \bar{F}^2 = 0, \qquad \bar{H}^2 = -\bar{E}\bar{F}, \qquad \bar{H}^4 = 0.$$

Lean. `VOA.LatGen.WpC2.smul_Ftrip_Ftrip`, `VOA.LatGen.WpC2.Qwp_leibniz`, `VOA.LatGen.WpC2.EF_add_FE_add_two_HH`, `VOA.LatGen.WpC2.smul_Etrip_Etrip`, `VOA.LatGen.WpC2.genWp`, `VOA.LatGen.WpC2.mk₂_genE_sq`, `VOA.LatGen.WpC2.mk₂_genF_sq`, `VOA.LatGen.WpC2.mk₂_genH_sq`, `VOA.LatGen.WpC2.mk₂_genH_pow_four`
File `VOA/Examples/TripletWpC2.lean`.

```
theorem smul_Ftrip_Ftrip (p : ℕ) (hp : 1 ≤ p) {n : ℤ} (hn : -(2 * (p : ℤ)) ≤ n) :
    (tripletLatticeVOA p hp).smul (Ftrip p) n (Ftrip p) = 0 := by

theorem Qwp_leibniz (p : ℕ) (hp : 1 ≤ p) (u : LatModuleGen (latP p)) (n : ℤ)
    (v : LatModuleGen (latP p)) :
    Qwp p hp ((tripletLatticeVOA p hp).smul u n v)
      = (tripletLatticeVOA p hp).smul (Qwp p hp u) n v
        + (tripletLatticeVOA p hp).smul u n (Qwp p hp v) :=

theorem EF_add_FE_add_two_HH (p : ℕ) (hp : 1 ≤ p) {n : ℤ} (hn : -(2 * (p : ℤ)) ≤ n) :
    (tripletLatticeVOA p hp).smul (Etrip p) n (Ftrip p)
      + (tripletLatticeVOA p hp).smul (Ftrip p) n (Etrip p)
      + (2 : ℂ) • (tripletLatticeVOA p hp).smul (Htrip p) n (Htrip p) = 0 := by

theorem smul_Etrip_Etrip (p : ℕ) (hp : 1 ≤ p) {n : ℤ} (hn : -(2 * (p : ℤ)) ≤ n) :
    (tripletLatticeVOA p hp).smul (Etrip p) n (Etrip p) = 0 := by
```

```
noncomputable def genWp (p : ℕ) (hp : 1 ≤ p) : Fin 4 → (tripletW p hp).carrier
  | 0 => genOmega p hp
  | 1 => genE p hp
  | 2 => genH p hp
  | 3 => genF p hp

theorem mk₂_genE_sq (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genE p hp)) ^ 2 = 0 := by

theorem mk₂_genF_sq (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genF p hp)) ^ 2 = 0 := by

theorem mk₂_genH_sq (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genH p hp)) ^ 2
      = -(mk₂ (tripletWVOA p hp) (genE p hp) * mk₂ (tripletWVOA p hp) (genF p hp)) := by

theorem mk₂_genH_pow_four (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genH p hp)) ^ 4 = 0 := by
```

Hypotheses / status. None

Theorem 18.12 (C_2-cofiniteness of $W(p)$, conditional). *For every $p \geq 1$ and $m \in \mathbb{N}$: if $R_{W(p)}$ is generated by $\bar{\omega}, \bar{E}, \bar{H}, \bar{F}$ and $L_{-2}^m \mathbf{1} \in C_2(W(p))$, then $W(p)$ is C_2-cofinite and $\dim R_{W(p)} \leq 16m$.*

Lean. `VOA.LatGen.WpC2.c2Cofinite_tripletWVOA`, `VOA.LatGen.WpC2.c2Cofinite_tripletWVOA_of_omegaPowMem`
File `VOA/Examples/TripletWpC2.lean`.

```
theorem c2Cofinite_tripletWVOA (p : ℕ) (hp : 1 ≤ p) (hgen : AdjoinGenWp p hp)
    (homega : OmegaNilpotentWp p hp) : C2Cofinite (tripletWVOA p hp) := by

theorem c2Cofinite_tripletWVOA_of_omegaPowMem (p : ℕ) (hp : 1 ≤ p) (m : ℕ)
    (hgen : AdjoinGenWp p hp) (homega : OmegaPowMem p hp m) :
    C2Cofinite (tripletWVOA p hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA p hp)) ≤ 16 * m :=
```

Hypotheses / status. `AdjoinGenWp`, `OmegaPowMem`

Theorem 18.14 (Generation from the generator OPE table). *Let T_p be the span of the strictly-negative-mode words in ω, E, H, F. If $g_{(j)}h \in T_p$ for all generators g, h and $0 \leq j < 4p + 2$, then $W(p)$ is strongly generated by ω, E, H, F. The ω-row of the table holds for every p, because E, H, F are Virasoro primary. The remaining mixed pairs reduce to the two "lowering rows" $F_{(j)}H$ and $F_{(j)}E$.*

Lean. VOA.LatGen.WpC2.GenPairClosedWp, VOA.LatGen.WpC2.strongGenWp_of_genPairClosed, VOA.LatGen.WpC2.LoweringPairsWp, VOA.LatGen.WpC2.screenGens_primary, VOA.LatGen.WpC2.c2Cofinite_tripletWVOA_of_lowering_AM
File VOA/Examples/TripletWpPairClosure.lean.

```
def GenPairClosedWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  ∀ g ∈ tripGens p, ∀ h ∈ tripGens p, ∀ j : ℤ, 0 ≤ j →
    (tripletLatticeVOA p hp).smul g j h ∈ iterProdSpan (tripletLatticeVOA p hp) (tripGens
    ↪ p)

theorem strongGenWp_of_genPairClosed (p : ℕ) (hp : 1 ≤ p) (h : GenPairClosedWp p hp) :
    StrongGenWp p hp :=

def LoweringPairsWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  (∀ j : ℤ, 0 ≤ j →
      (tripletLatticeVOA p hp).smul (Ftrip p) j (Htrip p)
        ∈ iterProdSpan (tripletLatticeVOA p hp) (tripGens p))
    ∧
  (∀ j : ℤ, 0 ≤ j →
      (tripletLatticeVOA p hp).smul (Ftrip p) j (Etrip p)
        ∈ iterProdSpan (tripletLatticeVOA p hp) (tripGens p))

theorem screenGens_primary (p : ℕ) (hp : 1 ≤ p) :
    ∀ g ∈ screenGens p, ∀ n : ℤ, 1 ≤ n → (tripletLatticeVOA p hp).L n g = 0 := by

theorem c2Cofinite_tripletWVOA_of_lowering_AM (p : ℕ) (hp : 1 ≤ p)
    (hlower : LoweringPairsWp p hp) (hAM : AMOmegaIdentity p hp (3 * p - 1)) :
    C2Cofinite (tripletWVOA p hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA p hp)) ≤ 16 * (3 * p - 1) :=
```

Hypotheses / status. LoweringPairsWp, AMOmegaIdentity

Theorem 18.15 (Mixed products at $p = 2$). *With $c = \varepsilon(\alpha, -\alpha)$, $d = \varepsilon(-\alpha, \alpha)$, in V_{L_2}:*

$$F_{(0)}H = \frac{48c}{5}\,L_{-2}F + \frac{4c}{5}\,T^2F, \qquad F_{(1)}E = -64cd\,L_{-2}\omega + 12cd\,T^2\omega + 10d\,TH, \qquad F_{(3)}E = -48cd\,\omega.$$

Consequently all mixed generator pairs close, and $W(2)$ is strongly generated by ω, E, H, F.

Lean. VOA.LatGen.WpC2.tripletTwoC, VOA.LatGen.WpC2.tripletTwoD, VOA.LatGen.WpC2.ThreeExplicitMixedIdentitiesWp, VOA.LatGen.WpC2.Ftrip_three_Etrip_two_explicit, VOA.LatGen.WpC2.threeCoordinateIdentitiesWp_two, VOA.LatGen.WpC2.mixedPairsWp_two, VOA.LatGen.Wp.strongGenWp_two
Files VOA/Examples/TripletWpTwoMixed.lean, TripletWpTwoMixedClose.lean, TripletWpTwoIrrational.lean.

```
noncomputable abbrev tripletTwoC : ℂ :=
  latticeCocycle (latP 2) (alphaP 2) (-alphaP 2)

noncomputable abbrev tripletTwoD : ℂ :=
  latticeCocycle (latP 2) (-alphaP 2) (alphaP 2)

def ThreeExplicitMixedIdentitiesWp (hp : 1 ≤ 2) : Prop :=
  let W := tripletLatticeVOA 2 hp
  W.smul (Ftrip 2) 0 (Htrip 2) =
      ((48 : ℂ) / 5 * tripletTwoC) • W.smul (omegaTrip 2) (-1) (Ftrip 2) +
        ((4 : ℂ) / 5 * tripletTwoC) • (W.toPreVertexAlgebra.T ^ 2) (Ftrip 2) ∧
  W.smul (Ftrip 2) 1 (Etrip 2) =
      (-64 * (tripletTwoC * tripletTwoD)) • W.smul (omegaTrip 2) (-1) (omegaTrip 2) +
```

```
        (12 * (tripletTwoC * tripletTwoD)) • (W.toPreVertexAlgebra.T ^ 2) (omegaTrip 2) +
          (10 * tripletTwoD) • W.toPreVertexAlgebra.T (Htrip 2) ∧
  W.smul (Ftrip 2) 3 (Etrip 2) =
      (-48 * (tripletTwoC * tripletTwoD)) • omegaTrip 2
```

File `VOA/Examples/TripletWpTwoMixedAssembly.lean`.

```
theorem Ftrip_three_Etrip_two_explicit (hp : 1 ≤ 2) :
    (tripletLatticeVOA 2 hp).smul (Ftrip 2) 3 (Etrip 2) =
      (-48 * (tripletTwoC * tripletTwoD)) • omegaTrip 2 := by

theorem threeCoordinateIdentitiesWp_two (hp : 1 ≤ 2) :
    ThreeCoordinateIdentitiesWp hp :=

theorem mixedPairsWp_two (hp : 1 ≤ 2) : MixedPairsWp 2 hp :=

theorem strongGenWp_two (hp : 1 ≤ 2) : StrongGenWp 2 hp :=
```

Hypotheses / status. None

Theorem 18.17 (The lattice triplet $W(2)$ is C_2-cofinite). *$W(2) =$ `tripletWVOA 2 hp` is C_2-cofinite, and $\dim W(2)/C_2(W(2)) \leq 80$.*

Lean. `VOA.LatGen.WpC2.tripletWVOA_two_c2Cofinite_and_finrank_le`, `VOA.LatGen.WpC2.tripletWVOA_two_c2Cofinite`
File `VOA/Examples/TripletWpTwoClose.lean`.

```
theorem tripletWVOA_two_c2Cofinite_and_finrank_le (hp : 1 ≤ 2) :
    C2Cofinite (tripletWVOA 2 hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA 2 hp)) ≤ 80 :=

theorem tripletWVOA_two_c2Cofinite (hp : 1 ≤ 2) : C2Cofinite (tripletWVOA 2 hp) :=
```

Hypotheses / status. None (lattice model only)

Theorem 18.18 (The lattice $W(2)$ is not rational). *$W(2)$ has no nonzero state of weight one. The ambient V_{L_2}, regarded as an ordinary $W(2)$-module by restriction, is not completely reducible: the submodule $W(2)$ has no complement. Consequently $W(2)$ is neither Gannon-rational nor admissibly rational.*

Lean. `VOA.LatGen.Wp.tripletAmbientOrdModuleTwo`, `VOA.LatGen.Wp.tripletCarrierWeakSubmoduleTwo`, `VOA.LatGen.Wp.eq_zero_of_mem_tripletW_two_and_grading_one`, `VOA.LatGen.Wp.Ftrip_three_latChargeGen_alpha`, `VOA.LatGen.Wp.tripletAmbientOrdModuleTwo_not_completelyReducible`, `VOA.LatGen.Wp.not_gannonRational_tripletWVOA_two`, `VOA.LatGen.Wp.not_admissiblyRational_tripletWVOA_two`, `VOA.LatGen.Wp.tripletWVOA_two_c2Cofinite_and_not_admissiblyRational`
Files `VOA/Examples/TripletWpTwoFelderStart.lean`, `TripletWpTwoIrrational.lean`.

```
noncomputable def tripletAmbientOrdModuleTwo (hp : 1 ≤ 2) :
    OrdVModule (tripletWVOA 2 hp) (LatModuleGen (latP 2)) :=
  (tripletW 2 hp).adjointRestrictOrd

noncomputable def tripletCarrierWeakSubmoduleTwo (hp : 1 ≤ 2) :
    WeakModule.Submodule' (tripletAmbientOrdModuleTwo hp).toWeakModule :=
  (tripletW 2 hp).carrierWeakSubmodule
```

```
theorem eq_zero_of_mem_tripletW_two_and_grading_one (hp : 1 ≤ 2)
    {x : LatModuleGen (latP 2)} (hxW : x ∈ (tripletW 2 hp).carrier)
    (hx1 : x ∈ (tripletLatticeVOA 2 hp).grading 1) : x = 0 := by

theorem Ftrip_three_latChargeGen_alpha (hp : 1 ≤ 2) :
    (tripletLatticeVOA 2 hp).smul (Ftrip 2) 3
        (latChargeGen (latP 2) (alphaP 2))
      = latticeCocycle (latP 2) (-alphaP 2) (alphaP 2) •
          latChargeGen (latP 2) (-alphaP 2 + alphaP 2) := by

theorem tripletAmbientOrdModuleTwo_not_completelyReducible (hp : 1 ≤ 2) :
    ¬ OrdVModule.IsCompletelyReducible (tripletAmbientOrdModuleTwo hp) := by

theorem not_gannonRational_tripletWVOA_two (hp : 1 ≤ 2) :
    ¬ GannonRational (tripletWVOA 2 hp) := by

theorem not_admissiblyRational_tripletWVOA_two (hp : 1 ≤ 2) :
    ¬ AdmissiblyRational (tripletWVOA 2 hp) := by

theorem tripletWVOA_two_c2Cofinite_and_not_admissiblyRational (hp : 1 ≤ 2) :
    C2Cofinite (tripletWVOA 2 hp) ∧ ¬ AdmissiblyRational (tripletWVOA 2 hp) :=
```

Hypotheses / status. None

Theorem 18.20 ($W(2)$ is simple and strongly finite). *The lattice $W(2)$ satisfies the f_2 certificate; hence it is simple, strongly finite, and neither admissibly nor Gannon rational (so not strongly rational).*

Lean. `VOA.LatGen.Wp.fTwoZhuCertificateWp`, `VOA.LatGen.Wp.tripletWVOA_two_isSimple`, `VOA.LatGen.Wp.tripletWVOA_two_stronglyFinite`, `VOA.LatGen.Wp.tripletWVOA_two_stronglyFinite_and_irrational`, `VOA.LatGen.Wp.tripletWVOA_two_stronglyFinite_of_isSimple`, `VOA.LatGen.Wp.not_stronglyRational_tripletWVOA_two`
Files `VOA/Examples/TripletWpTwoZhuFinal.lean`, `TripletWpStronglyFinite.lean`.

```
theorem fTwoZhuCertificateWp (hp : 1 ≤ 2) : FTwoZhuCertificateWp hp :=

theorem tripletWVOA_two_isSimple (hp : 1 ≤ 2) : (tripletWVOA 2 hp).IsSimple :=

theorem tripletWVOA_two_stronglyFinite (hp : 1 ≤ 2) :
    StronglyFinite (tripletWVOA 2 hp) :=

theorem tripletWVOA_two_stronglyFinite_and_irrational (hp : 1 ≤ 2) :
    StronglyFinite (tripletWVOA 2 hp) ∧
      ¬ AdmissiblyRational (tripletWVOA 2 hp) ∧
      ¬ GannonRational (tripletWVOA 2 hp) :=
```

```
theorem tripletWVOA_two_stronglyFinite_of_isSimple (hp : 1 ≤ 2)
    (hsimple : (tripletWVOA 2 hp).IsSimple) : StronglyFinite (tripletWVOA 2 hp) := by

theorem not_stronglyRational_tripletWVOA_two (hp : 1 ≤ 2) :
    ¬ StronglyRational (tripletWVOA 2 hp) := by
```

Hypotheses / status. None

Definition 18.22 (The Abe model of $W(2)$). SF^+ is the even part of the symplectic-fermion super VOA `sympFermSVOA` (Chapter 17, Theorem 17.36): the span of the word states $\chi^{a_1}_{-k_1-1}\cdots\chi^{a_n}_{-k_n-1}\mathbf{1}$ with n even, with the restricted vertex operators, vacuum and conformal vector $\omega_{SF} = -\chi^0_{-1}\chi^1_{-1}\mathbf{1}$.

Lean. `VOA.SympFerm.sympFermSVOA_integralWeights`, `VOA.SympFerm.tripletW2`, `VOA.SympFerm.tripletW2_centralCharge`, `VOA.SympFerm.tripletW2_grading`, `VOA.SympFerm.evenPartSF_eq_span`, `VOA.SympFerm.tripletW2_grading_one`
File `VOA/Examples/TripletW2.lean`.

```
theorem sympFermSVOA_integralWeights (h : ℂ) (hh : ¬ ∃ n : ℤ, (n : ℂ) = h) :
    ∀ x ∈ sympFermSVOA.parity 0, x ∈ sympFermSVOA.grading h → x = 0 := by

noncomputable def tripletW2 : VertexOperatorAlgebra (evenOdd QSF 0) :=
  sympFermSVOA.evenPartVOA sympFermSVOA_integralWeights

@[simp] theorem tripletW2_centralCharge : tripletW2.centralCharge = -2 := rfl

@[simp] theorem tripletW2_grading (n : ℤ) :
    tripletW2.grading n = (gradingSF (n : ℂ)).comap (evenOdd QSF 0).subtype := rfl

theorem evenPartSF_eq_span :
    (evenOdd QSF (0 : ZMod 2) : Submodule ℂ FockSF)
      = Submodule.span ℂ (stWordSF '' {l : SFWord | Even l.length}) := by

theorem tripletW2_grading_one : tripletW2.grading 1 = ⊥ := by
```

Hypotheses / status. None

Theorem 18.25 (Abe's C_2 relations in the Abe model). *In $R_{W(2)}$, with $\bar\omega, \bar E, \bar H, \bar F$ the classes of the generators:*

$$\bar E^2 = \bar F^2 = 0, \quad \bar E\bar H = \bar F\bar H = 0, \quad 2\bar H^2 + \bar E\bar F = 0, \quad 2\bar H^2 = \tfrac{16}{9}\bar\omega^3, \quad \bar H^4 = 0, \quad \bar\omega^6 = 0.$$

Abe's sharper $\bar\omega^5 = 0$ follows from his (unproved here) $\bar\omega^2\bar H = 0$.

Lean. `VOA.AbeRel.wbar`, `VOA.AbeRel.Ebar`, `VOA.AbeRel.Hbar`, `VOA.AbeRel.Fbar`, `VOA.AbeRel.Ebar_sq`, `VOA.AbeRel.Fbar_sq`, `VOA.AbeRel.Ebar_mul_Hbar`, `VOA.AbeRel.two_Hbar_sq_add_Ebar_Fbar`, `VOA.AbeRel.two_Hbar_sq_eq`, `VOA.AbeRel.Hbar_pow_four`, `VOA.AbeRel.wbar_pow_six`, `VOA.AbeRel.OmegaSqHbarZero`, `VOA.AbeRel.OmegaPowFiveZero`, `VOA.AbeRel.omegaPowFive_of_omegaSqHbar`
File `VOA/Examples/TripletW2AbeRelations.lean`.

```
noncomputable def wbar : A₂ tripletW2 := mk₂ tripletW2 abeOmega

noncomputable def Ebar : A₂ tripletW2 := mk₂ tripletW2 abeE

noncomputable def Hbar : A₂ tripletW2 := mk₂ tripletW2 abeH

noncomputable def Fbar : A₂ tripletW2 := mk₂ tripletW2 abeF

theorem Ebar_sq : Ebar * Ebar = 0 := by

theorem Fbar_sq : Fbar * Fbar = 0 := by

theorem Ebar_mul_Hbar : Ebar * Hbar = 0 := by

theorem two_Hbar_sq_add_Ebar_Fbar : (2 : ℂ) • (Hbar * Hbar) + Ebar * Fbar = 0 := by

theorem two_Hbar_sq_eq : (2 : ℂ) • (Hbar * Hbar) = ((16 : ℂ) / 9) • (wbar * (wbar * wbar))
 ↪  := by

theorem Hbar_pow_four : Hbar ^ 4 = 0 := by
```

```
theorem wbar_pow_six : wbar ^ 6 = 0 := by

def OmegaSqHbarZero : Prop := wbar * (wbar * Hbar) = 0

def OmegaPowFiveZero : Prop := wbar ^ 5 = 0

theorem omegaPowFive_of_omegaSqHbar (h : OmegaSqHbarZero) : OmegaPowFiveZero := by
```

Hypotheses / status. None

Theorem 18.29 (A logarithmic module from a Zhu-algebra Jordan block). *Let V be a VOA with $A(V)$ and all $A_{n,0}(V)$ finite-dimensional. For every h there is a logarithmic module $M_h = M(U_h)$ (the Dong–Jiang induced module on $U_h = A(V)/A(V)\omega_h^2$) on which $L_0 = \rho([\omega]) + n$ on level n. If moreover the Dong–Jiang gate $A_{0,0}(V) = A(V)$ holds and* `OmegaJordanAt V h`*, then $L_0^{\mathrm{nil}} \neq 0$ on M_h: there are x, y of generalized weight h with $y \neq 0$ and $L_0 x = hx + y$. For the Abe model the two finiteness hypotheses follow from C_2-cofiniteness.*

Lean. `VOA.djLogVModule_L0nil_ne_zero`, `VOA.omegaJordan_nil_sq`, `VOA.SympFerm.tripletW2Log`, `VOA.SympFerm.tripletW2Log_jordan_pair`, `VOA.SympFerm.tripletW2_zhuFiniteness_of_C2Cofinite`
Files `VOA/Examples/TripletW2Log.lean`, `TripletW2C2.lean`.

```
theorem djLogVModule_L0nil_ne_zero (W : VertexOperatorAlgebra 𝒱) (U : AmMod W 0) (h : ℂ) (k
↪  : ℕ)
    (hnil : amModOmegaNil U h ^ k = 0)
    (hfin : ∀ n : ℕ, FiniteDimensional ℂ (DJPiece W U n))
    (hgate : DJAnnEqAn W 0) (hne : amModOmegaNil U h ≠ 0) :
    (djLogVModule W U h k hnil hfin).L0nil ≠ 0 := by

theorem omegaJordan_nil_sq (h : ℂ) :
    amModOmegaNil (omegaJordanAMod W h).toAmModZero h ^ 2 = 0 := by

noncomputable def tripletW2Log (h : ℂ)
    (hAnm : ∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
    (hA : FiniteDimensional ℂ (A tripletW2)) :
    LogVModule tripletW2
      (DJCarrier tripletW2 (omegaJordanAMod tripletW2 h).toAmModZero) :=

theorem tripletW2Log_jordan_pair (h : ℂ)
    (hAnm : ∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
    (hA : FiniteDimensional ℂ (A tripletW2)) (hgate : DJAnnEqAn tripletW2 0)
    (hnd : OmegaJordanAt tripletW2 h) :
    ∃ x y : DJCarrier tripletW2 (omegaJordanAMod tripletW2 h).toAmModZero,
      x ∈ (tripletW2Log h hAnm hA).weight h ∧ y ∈ (tripletW2Log h hAnm hA).weight h ∧
        y ≠ 0 ∧ (tripletW2Log h hAnm hA).L0op x = h • x + y :=
```

```
theorem tripletW2_zhuFiniteness_of_C2Cofinite (hC2 : C2Cofinite tripletW2) :
    (∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
      ∧ FiniteDimensional ℂ (A tripletW2) :=
```

Hypotheses / status. `hAnm`, `hA`, `hgate`, `hnd`

Chapter 19

Graded unitarity and Lee–Yang

7 of the chapter's 52 numbered statements: its headline results and every definition they use.

Theorem 19.4 (A bilinear form is never positive on a nonzero complex subspace). *Let β be a $\mathbb{C}$-bilinear form on a complex vector space E and $S \le E$ a complex subspace. If $\beta(x,x)$ is a positive real number for every nonzero $x \in S$, then $S = 0$.*

Lean. `VOA.eq_bot_of_bilinear_pos`
File `VOA/Core/GradedUnitarity.lean`.

```
theorem eq_bot_of_bilinear_pos (β : BilinForm ℂ E) (S : Submodule ℂ E)
    (hpos : ∀ x ∈ S, x ≠ 0 → ∃ r : ℝ, 0 < r ∧ β x x = (r : ℂ)) : S = ⊥ := by
```

Hypotheses / status. None; unconditional

Theorem 19.12 (The two orthogonal complements agree on ρ-stable subspaces). *Let $S \le V_n$ be ρ-stable ($\rho(S) \subseteq S$) and suppose $s(n) \neq 0$. Then $\{y : h_n(x,y) = 0\ \forall x \in S\} = \{y : B(x,y) = 0\ \forall x \in S\}$.*

Lean. `VOA.RhoStableOn`, `VOA.orthogonal_gradedSesq_eq`
File `VOA/Core/GradedUnitarity.lean`.

```
def RhoStableOn (R : RealStructure W) {n : ℤ} (S : Submodule ℂ (W.grading n)) : Prop :=
  ∀ x ∈ S, ∃ y ∈ S, (y : 𝒱) = R.rho (x : 𝒱)

theorem orthogonal_gradedSesq_eq (B : InvariantForm W) (R : RealStructure W) {n : ℤ}
    {S : Submodule ℂ (W.grading n)} (hS : RhoStableOn R S) (hp : R.parity n ≠ 0) :
    (gradedSesq B R n).orthogonal S = (LiRadical.gradedForm B n).orthogonal S := by
```

Hypotheses / status. ρ-stable, $s(n) \neq 0$

Theorem 19.24 (Canonical real structures on the boundary models). *For every $n \in \mathbb{N}$, $c_{2,2n+1} = 1 - 12\lambda_n^2$ with the real number $\lambda_n = \sqrt{(2n-1)^2/(4(2n+1))}$, so $L(c_{2,2n+1}, 0)$ carries a canonical real structure.*

Lean. `VOA.VirGradedUnitarity.boundaryBackgroundChargeSq`, `VOA.VirGradedUnitarity.boundaryBackgroundCharge_spec`, `VOA.VirGradedUnitarity.boundaryVirasoroRealStructure`
File `VOA/Examples/VirasoroBoundaryRealStructure.lean`.

```
noncomputable def boundaryBackgroundChargeSq (n : ℕ) : ℝ :=
  (2 * (n : ℝ) - 1) ^ 2 / (4 * (2 * n + 1))

theorem boundaryBackgroundCharge_spec (n : ℕ) :
    1 - 12 * boundaryBackgroundCharge n ^ 2 = boundaryCharge n := by
```

```
noncomputable def boundaryVirasoroRealStructure (n : ℕ) :
    RealStructure (virasoroSimple (boundaryCharge n)) :=
  virasoroSimpleRealStructure (boundaryBackgroundCharge_spec n)
    (boundaryBackgroundCharge_fixed n)
```

Hypotheses / status. $c = 1 - 12\lambda^2$, $\lambda \in \mathbb{R}$; boundary: none

Definition 19.51 (Layer signs, step nondegeneracy, and the two analytic inputs). For a family R of real structures on V^x, $x < -2$, write $\mathrm{Sign}_R(x, E)$ for clause (vi) at weight E and $\mathrm{Nd}(x, E)$ for clause (iv) at weight E, both for the ordinary flag and the form B_x.

1. The *anchor* for R: for every E there is $K \in \mathbb{R}$ such that $\mathrm{Sign}_R(x, E)$ holds for every $x < -2$ with $x < K$.

2. *Rigidity* for R: for every E, if $\mathrm{Nd}(x, E)$ holds for every $x < -2$, then $\mathrm{Sign}_R(x_0, E)$ for one $x_0 < -2$ implies $\mathrm{Sign}_R(x_1, E)$ for every $x_1 < -2$.

Lean. `VOA.AffineSl2BelowCritical.LayerSigns`, `VOA.AffineSl2BelowCritical.LayerNondeg`, `VOA.AffineSl2BelowCritical.LayerSignAnchor`, `VOA.AffineSl2BelowCritical.LayerSignRigidity`
File `VOA/Examples/AffineSl2BelowCriticalGradedUnitarity.lean`.

```
def LayerSigns (R : RealFamily) (x : ℝ) (hx : x < -2) (E : ℤ) : Prop :=
  LayerPositive (ordinaryFlag (x : ℂ) (noncritical_of_lt hx)) E
    (LiRadical.gradedForm (universalForm (x : ℂ) (noncritical_of_lt hx)) E)
    (gradedSesq (universalForm (x : ℂ) (noncritical_of_lt hx)) (R x hx) E)

def LayerNondeg (x : ℝ) (hx : x < -2) (E : ℤ) : Prop :=
  LayerNondegenerate (ordinaryFlag (x : ℂ) (noncritical_of_lt hx)) E
    (LiRadical.gradedForm (universalForm (x : ℂ) (noncritical_of_lt hx)) E)

def LayerSignAnchor (R : RealFamily) : Prop :=
  ∀ E : ℤ, ∃ K : ℝ, ∀ (x : ℝ) (hx : x < -2), x < K → LayerSigns R x hx E

def LayerSignRigidity (R : RealFamily) : Prop :=
  ∀ E : ℤ, (∀ (x : ℝ) (hx : x < -2), LayerNondeg x hx E) →
    ∀ (x₀ : ℝ) (hx₀ : x₀ < -2) (x₁ : ℝ) (hx₁ : x₁ < -2),
      LayerSigns R x₀ hx₀ E → LayerSigns R x₁ hx₁ E
```

Hypotheses / status. Definitions; both proved for the compact family

Theorem 19.53 (Anchor: layer signs at large negative level). ***(a) Matrix version.*** *Let I be a finite set with a length $\ell : I \to \mathbb{N}$, P an $I \times I$ matrix of complex polynomials, and $d : I \to \mathbb{R}_{>0}$, such that*

1. $\deg P_{ij} \le \min(\ell(i), \ell(j))$;

2. *if $\ell(i) = \ell(j)$, the coefficient of $X^{\ell(i)}$ in P_{ij} is $\delta_{ij} d_i$;*

3. *$P(x)$ is Hermitian for all real $x < x_*$ (some $x_* \in \mathbb{R}$; `x₀` in the Lean statement).*

Then there is K such that for every real $x < K$ and every r, $\textsc{LayerSign}_r(P(x))$ holds.

(b) Affine version. *For the compact family, the anchor of Definition 19.51 holds: for every weight E there is K such that the weight-E layer signs hold at every real level $x < \min(K, -2)$.*

Lean. `VOA.MatrixRigidity.matrixLayerSign_eventually`, `VOA.AffineSl2BelowCritical.gramPoly_natDegree_le`, `VOA.AffineSl2BelowCritical.gramPoly_coeff_top`, `VOA.AffineSl2BelowCritical.layerSignAnchor_compact`
Files `VOA/Examples/AffineSl2BelowCriticalMatrixAnchor.lean` (a) and `VOA/Examples/AffineSl2BelowCriticalAnchor.lean` (b, and the two polynomial facts).

```
theorem matrixLayerSign_eventually (P : Matrix ι ι ℂ[X]) (d : ι → ℝ) (hd : ∀ i, 0 < d i)
    (hdeg : ∀ i j, (P i j).natDegree ≤ min (ℓ i) (ℓ j))
    (htop : ∀ i j, ℓ i = ℓ j → (P i j).coeff (ℓ i) = if i = j then (d i : ℂ) else 0)
    (x₀ : ℝ) (hherm : ∀ x : ℝ, x < x₀ → (P.map (Polynomial.eval (x : ℂ))).IsHermitian) :
    ∃ K : ℝ, ∀ x : ℝ, x < K → ∀ r, MatrixLayerSign ℓ (P.map (Polynomial.eval (x : ℂ))) r :=
     ↪ by

theorem gramPoly_natDegree_le {E : ℤ} (i j : EIdx E) :
    (gramPoly E i j).natDegree ≤ min (eLen E i) (eLen E j) := by
...
theorem gramPoly_coeff_top {E : ℤ} (i j : EIdx E) (hlen : eLen E i = eLen E j) :
    (gramPoly E i j).coeff (eLen E i) = if i = j then ((topWeight i : ℝ) : ℂ) else 0 := by
...
theorem layerSignAnchor_compact : LayerSignAnchor compactRealFamily := by
```

Hypotheses / status. Degree and top-coefficient bounds (proved for `gramPoly`); unconditional

Theorem 19.54 (Rigidity of layer signs). ***(a) Matrix version.*** *Let X be a preconnected topological space and $H : X \to \mathrm{Mat}_I(\mathbb{C})$ a continuous family of Hermitian matrices with* StepNondeg$(H(t))$ *for every t. If* LayerSign$_r(H(t_0))$ *holds for one t_0, it holds for every t_1.*

(b) Affine version. *Rigidity (Definition 19.51) holds for the compact family. Consequently, for every real $k < -2$, graded unitarity of $V^k(\mathfrak{sl}_2)$ with the compact structure and the ordinary flag follows from the anchor alone.*

Lean. `VOA.MatrixRigidity.matrixLayerSign_rigid`, `VOA.AffineSl2BelowCritical.gram_isHermitian`, `VOA.AffineSl2BelowCritical.gram_eq_eval`, `VOA.AffineSl2BelowCritical.layerSignRigidity_compact`, `VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical_of_anchor`
Files `VOA/Examples/AffineSl2BelowCriticalMatrixRigidity.lean` (a) and `VOA/Examples/AffineSl2BelowCriticalRigidity.lean` (b).

```
theorem matrixLayerSign_rigid {X : Type*} [TopologicalSpace X] [PreconnectedSpace X]
    (H : X → Matrix ι ι ℂ) (hc : Continuous H) (hH : ∀ t, (H t).IsHermitian)
    (hnd : ∀ t, MatrixStepNondeg ℓ (H t)) (r : ℕ) {t₀ t₁ : X}
    (h₀ : MatrixLayerSign ℓ (H t₀) r) : MatrixLayerSign ℓ (H t₁) r := by

theorem gram_isHermitian (E : ℤ) : (gram hk (compactRealStructure hreal hk) E).IsHermitian
 ↪ := by
...
theorem gram_eq_eval (hk0 : k ≠ 0) (E : ℤ) (i j : EIdx E) :
    gram hk (compactRealStructure hreal hk) E i j = (gramPoly E i j).eval k := by
...
theorem layerSignRigidity_compact : LayerSignRigidity compactRealFamily := by
...
theorem universal_gradedUnitary_belowCritical_of_anchor
    (hanchor : LayerSignAnchor compactRealFamily) (k : ℝ) (hk : k < -2) :
    GradedUnitary (universalForm (k : ℂ) (noncritical_of_lt hk)) (compactRealFamily k hk)
      (ordinaryFlag (k : ℂ) (noncritical_of_lt hk)) :=
  universal_gradedUnitary_belowCritical_compact hanchor layerSignRigidity_compact k hk
```

Hypotheses / status. Continuous Hermitian family, nondegenerate steps; unconditional

Theorem 19.55 (Nondegeneracy of the full form below critical level). *For every real $k < -2$ and every E, the form B_k is nondegenerate on V_E^k: if $v \in V_E^k$ and $B_k(y, v) = 0$ for all $y \in V_E^k$, then $v = 0$.*

Lean. `VOA.AffineSl2BelowCritical.universalForm_separating_of_lt`
File `VOA/Examples/AffineSl2BelowCriticalNondegenerate.lean`.

```
theorem universalForm_separating_of_lt {x : ℝ} (hx : x < -2) (E : ℤ) :
    ∀ v ∈ (universal (x : ℂ) (noncritical_of_lt hx)).grading E,
      (∀ y ∈ (universal (x : ℂ) (noncritical_of_lt hx)).grading E,
        (universalForm (x : ℂ) (noncritical_of_lt hx)).form y v = 0) → v = 0 := by
```

Hypotheses / status. None; unconditional

Part VI

Two recent theorems, formalized

Chapter 20

Classical freeness at the boundary admissible level: the Arakawa–Dai theorems

9 of the chapter's 56 numbered statements: its headline results and every definition they use.

Definition 20.8 (The fermionic chain and the Hilbert-bound series). For $\ell, n \geq 0$ define $K_\ell(n) \in \mathbb{Q}[[q]]$ by

$$K_0(n) = \frac{(q)_n^2}{(q)_{2n+1}}, \qquad K_{\ell+1}(n) = \sum_{j=0}^{n} \frac{q^{j(j+1)}}{(q)_{n-j}} K_\ell(j),$$

and the *fermionic trace* $T_m = \sum_{n\geq 0} q^{n(n+1)} K_{m-1}(n)$. Unwinding the recursion, $T_m = \sum_{n_1 \geq \cdots \geq n_m \geq 0} q^{\sum n_j(n_j+1)} (q)_{n_m}^2 /($ the sum in (20.3). The *Hilbert-bound series* S_m (equation (5.12) of the paper) is the same chain with K_0 replaced by the three-species residual sum

$$\mathrm{res}(n) = \sum_{s\geq 0} q^{(n+1)s} \frac{1}{(q)_s} E(n+s+1)\, E(n+1) \frac{1}{(q)_n}, \qquad E(A) = \sum_{j\geq 0} \frac{q^{Aj}}{(q)_j}.$$

Lean. `VOA.QSeries.fermionicKernel`, `VOA.QSeries.fermionicTrace`, `VOA.QSeries.residualSum`, `VOA.QSeries.eulerSum`, `VOA.QSeries.powerShiftSum`, `VOA.QSeries.hilbertBoundKernel`, `VOA.QSeries.hilbertBoundSeries`
File `VOA/Commutative/QChainTrace.lean`.

```
def fermionicKernel : ℕ → ℕ → PowerSeries K
  | 0, n => qFactorial PowerSeries.X n ^ 2 * reciprocalQFactorial (2 * n + 1)
  | ℓ + 1, n => ∑ j ∈ Finset.range (n + 1),
      PowerSeries.X ^ (j * (j + 1)) * reciprocalQFactorial (n - j) * fermionicKernel ℓ j

def fermionicTrace (m : ℕ) : PowerSeries K :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun n =>
    PowerSeries.X ^ (n * (n + 1)) * fermionicKernel (m - 1) n)
```

Files `VOA/Commutative/QBinomialResidual.lean`, `VOA/Commutative/QEuler.lean`, `VOA/Commutative/QSeriesSum.lean`.

```
def residualSum (n : ℕ) : PowerSeries R :=
  powerShiftSum (n + 1) fun s =>
    reciprocalQFactorial s * eulerSum (n + s + 1) *
      eulerSum (n + 1) * reciprocalQFactorial n

def eulerSum (A : ℕ) : PowerSeries R :=
```

```
  powerShiftSum A reciprocalQFactorial

def powerShiftSum (A : ℕ) (B : ℕ → PowerSeries R) : PowerSeries R :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun j => PowerSeries.X ^ (A * j) * B j)
```

File `VOA/Commutative/QHilbertProduct.lean`.

```
def hilbertBoundKernel : ℕ → ℕ → PowerSeries K
  | 0, n => residualSum n
  | ℓ + 1, n => ∑ j ∈ Finset.range (n + 1),
      PowerSeries.X ^ (j * (j + 1)) * reciprocalQFactorial (n - j) * hilbertBoundKernel ℓ j

def hilbertBoundSeries (m : ℕ) : PowerSeries K :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun n =>
    PowerSeries.X ^ (n * (n + 1)) * hilbertBoundKernel (m - 1) n)
```

Hypotheses / status. Definitions; finiteness proved

Theorem 20.11 (The arc universal property). *For every commutative R-algebra A, restriction along the universal arc (the tautological algebra map $R[x]/I \to (J_\infty(R[x]/I))[[t]]$, $x_i \mapsto \sum_n x_{i,n} t^n$) gives a bijection $\mathrm{Hom}_{R\text{-}alg}(J_\infty(R[x]/I), A) \cong \mathrm{Hom}_{R\text{-}alg}(R[x]/I, A[[t]])$. If $e : R[x]/I \cong B$ is an isomorphism of algebras, composing with e gives $\mathrm{Hom}(J_\infty(R[x]/I), A) \cong \mathrm{Hom}(B, A[[t]])$.*

Lean. `VOA.InfiniteJet.seriesJetEval`, `VOA.InfiniteJet.toArc`, `VOA.InfiniteJet.arcEquiv`, `VOA.InfiniteJet.existsUnique_arcLift`, `VOA.InfiniteJet.arcEquivOfBaseAlgebraEquiv`, `VOA.InfiniteJet.baseToJet_injective`
File `VOA/Commutative/InfiniteJetArc.lean`.

```
def seriesJetEval (x : σ → PowerSeries A) : JetPoly R σ →ₐ[R] A :=
  MvPolynomial.aeval fun v => PowerSeries.coeff v.2 (x v.1)

def toArc (I : Ideal (MvPolynomial σ R)) (f : PresentedJet I →ₐ[R] A) :
    (MvPolynomial σ R ⧸ I) →ₐ[R] PowerSeries A :=
  Ideal.Quotient.liftₐ I (MvPolynomial.aeval (pointSeries I f))
    (pointSeries_relations I f)

def arcEquiv (I : Ideal (MvPolynomial σ R)) :
    (PresentedJet I →ₐ[R] A) ≃ ((MvPolynomial σ R ⧸ I) →ₐ[R] PowerSeries A) where
  toFun := toArc I
  invFun := ofArc I
  left_inv := ofArc_toArc I
  right_inv := toArc_ofArc I

theorem existsUnique_arcLift (I : Ideal (MvPolynomial σ R))
    (u : (MvPolynomial σ R ⧸ I) →ₐ[R] PowerSeries A) :
    ∃! f : PresentedJet I →ₐ[R] A,
      (PowerSeries.mapAlgHom f).comp (universalArc I) = u := by
```

File `VOA/Commutative/InfiniteJetBaseEmbedding.lean`.

```
def arcEquivOfBaseAlgebraEquiv {A B : Type*} [CommRing A] [Algebra R A]
    [CommRing B] [Algebra R B] (I : Ideal (MvPolynomial σ R))
    (e : (MvPolynomial σ R ⧸ I) ≃ₐ[R] B) :
    (PresentedJet I →ₐ[R] A) ≃ (B →ₐ[R] PowerSeries A) where
  toFun f := (toArc I f).comp e.symm.toAlgHom
  invFun u := ofArc I (u.comp e.toAlgHom)
  left_inv f := by simp [AlgHom.comp_assoc]
  right_inv u := by simp [AlgHom.comp_assoc]
```

```
theorem baseToJet_injective (I : Ideal (MvPolynomial σ R)) : Function.Injective (baseToJet
 ↪  I) := by
```

Hypotheses / status. Definitions; arc property proved

Definition 20.12 (Poisson vertex algebra, in mode form). A Poisson vertex algebra over $\mathbb{C}$ is a commutative $\mathbb{C}$-algebra A with a derivation ∂ and bilinear products $a_{(n)}b$, $n \in \mathbb{N}$, such that for all a, b, c:

1. (locality) $a_{(n)}b = 0$ for $n \gg 0$;
2. (sesquilinearity) $(\partial a)_{(0)}b = 0$, $(\partial a)_{(n+1)}b = -(n+1)\,a_{(n)}b$, and $\partial(a_{(n)}b) = (\partial a)_{(n)}b + a_{(n)}(\partial b)$;
3. (Leibniz) $a_{(n)}(bc) = (a_{(n)}b)c + b(a_{(n)}c)$;
4. (skew-symmetry) $a_{(n)}b = \sum_{j\ge 0} \frac{(-1)^{n+j+1}}{j!}\partial^j(b_{(n+j)}a)$;
5. (Jacobi) $a_{(m)}(b_{(n)}c) - b_{(n)}(a_{(m)}c) = \sum_{j=0}^{m}\binom{m}{j}(a_{(j)}b)_{(m+n-j)}c$.

Equivalently, $\{a_\lambda b\} = \sum_n \frac{\lambda^n}{n!}a_{(n)}b$ is a λ-bracket satisfying the usual PVA axioms. A homomorphism is an algebra map commuting with ∂ and every (n)-product; an equivalence is a bijective homomorphism (whose inverse is then automatically a homomorphism).

Lean. `VOA.PoissonVertexAlgebra`, `VOA.PoissonVertexAlgebra.Hom`, `VOA.PoissonVertexAlgebra.Equiv`, `VOA.PoissonVertexAlgebra.Equiv.ofBijective`, `VOA.PoissonVertexAlgebra.Equiv.symm` File `VOA/Commutative/PoissonVertexAlgebra.lean`.

```
structure PoissonVertexAlgebra (A : Type*) [CommRing A] [Algebra ℂ A] where
  translation : Derivation ℂ A A
  mode : ℕ → A →ₗ[ℂ] A →ₗ[ℂ] A
  locality : ∀ x y, ∃ B : ℕ, ∀ n, B ≤ n → mode n x y = 0
  translation_left_zero : ∀ x y, mode 0 (translation x) y = 0
  translation_left_succ : ∀ n x y,
    mode (n + 1) (translation x) y = (-((n + 1 : ℕ) : ℂ)) • mode n x y
  translation_mode : ∀ n x y,
    translation (mode n x y) = mode n (translation x) y + mode n x (translation y)
  leibniz : ∀ n x y z, mode n x (y * z) = mode n x y * z + y * mode n x z
  skew : ∀ n x y, mode n x y = ∑ᶠ j : ℕ,
    (j.factorial : ℂ)⁻¹ • (-1 : ℂ) ^ (n + j + 1) •
      ((translation.toLinearMap ^ j) (mode (n + j) y x))
  jacobi : ∀ m n x y z,
    mode m x (mode n y z) - mode n y (mode m x z) =
      ∑ j : Fin (m + 1), (m.choose j.val : ℂ) • mode (m + n - j.val) (mode j.val x y) z
```

Files `VOA/Commutative/PoissonVertexHom.lean`, `VOA/Commutative/PoissonVertexEquiv.lean`.

```
structure Hom (P : VOA.PoissonVertexAlgebra A) (Q : VOA.PoissonVertexAlgebra B) where
  toAlgHom : A →ₐ[ℂ] B
  map_translation : ∀ x, toAlgHom (P.translation x) = Q.translation (toAlgHom x)
  map_mode : ∀ n x y, toAlgHom (P.mode n x y) = Q.mode n (toAlgHom x) (toAlgHom y)

structure Equiv (P : VOA.PoissonVertexAlgebra A) (Q : VOA.PoissonVertexAlgebra B) where
  toAlgEquiv : A ≃ₐ[ℂ] B
  map_translation : ∀ x, toAlgEquiv (P.translation x) = Q.translation (toAlgEquiv x)
  map_mode : ∀ n x y, toAlgEquiv (P.mode n x y) = Q.mode n (toAlgEquiv x) (toAlgEquiv y)

def Equiv.ofBijective {P : VOA.PoissonVertexAlgebra A} {Q : VOA.PoissonVertexAlgebra B}
```

```
    (f : Hom P Q) (hf : Function.Bijective f.toAlgHom) : Equiv P Q where
  toAlgEquiv := AlgEquiv.ofBijective f.toAlgHom hf
  map_translation := f.map_translation
  map_mode := f.map_mode

def Equiv.symm {P : VOA.PoissonVertexAlgebra A} {Q : VOA.PoissonVertexAlgebra B}
    (e : Equiv P Q) : Equiv Q P where
```

Hypotheses / status. $1 \le m$; unconditional

Theorem 20.18 (The vacuum character of $L_{k_m}(\mathfrak{sl}_2)$). *For every $m \ge 1$, $p = 2m+1$:*

$$\sum_{N \ge 0} \dim L_{k_m}(\mathfrak{sl}_2)_N \, q^N = \frac{(q^p;q^p)_\infty^3}{(q)_\infty^3} = S_m(q) = \frac{1}{(q)_\infty^2}\, T_m(q),$$

with S_m, T_m as in Definition 20.8; the last expression is the fermionic chain of (20.3).

Lean. `VOA.AffineSl2BoundaryCharacter.coeff_product_le_character`, `VOA.AffineSl2BoundaryCharacter.character_eq_product`, `VOA.AffineSl2BoundaryCharacter.character_eq_chain`, `VOA.AffineSl2BoundaryCharacter.character_eq_fermionic_chain`
File `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

```
theorem coeff_product_le_character (m : ℕ) (hm : 1 ≤ m) (N : ℕ) :
    PowerSeries.coeff N (product m) ≤ PowerSeries.coeff N (character m) := by

theorem character_eq_product (m : ℕ) (hm : 1 ≤ m) :
    character m = product m := by

theorem character_eq_chain (m : ℕ) (hm : 1 ≤ m) :
    character m = QSeries.hilbertBoundSeries m :=
  character_eq_chain_of_product m hm (character_eq_product m hm)

theorem character_eq_fermionic_chain (m : ℕ) (hm : 1 ≤ m) :
    character m = (QSeries.reciprocalEuler : PowerSeries ℚ)^2 * QSeries.fermionicTrace m :=
    ↪ by
```

Hypotheses / status. $1 \le m$; unconditional, `pbw`-free

Theorem 20.20 (Classical freeness of $L_{k_m}(\mathfrak{sl}_2)$). *For every $m \ge 1$ the canonical map $\pi : J_\infty A_m \to \mathrm{gr}^F L_{k_m}(\mathfrak{sl}_2)$ is bijective, and is an isomorphism of Poisson vertex algebras which, in both directions, maps the conformal weight N piece onto the conformal weight N piece.*

Lean. `VOA.AffineSl2BoundaryClassicalFreeness.canonicalMap_bijective`, `VOA.AffineSl2BoundaryClassicalFreeness.boundary_classically_free`, `VOA.AffineSl2BoundaryClassicalFreeness.arcPoissonEquiv`, `VOA.AffineSl2BoundaryClassicalFreeness.arcPoissonEquiv_conformal_iff`, `VOA.AffineSl2BoundaryClassicalFreeness.arcPoissonEquivOfCharacter`
File `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

```
theorem canonicalMap_bijective (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (canonicalMap m hm) :=
  canonicalMap_bijective_of_character m hm (character_eq_product m hm)

theorem boundary_classically_free (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (uniformCandidate m hm).arcLiPoissonHom.toAlgHom :=
  ⟨arcLiPoissonHom_injective_of_character m hm (character_eq_product m hm),
    (uniformCandidate m hm).arcLiPoissonHom_surjective⟩
```

```
def arcPoissonEquiv (m : ℕ) (hm : 1 ≤ m) :
    PoissonVertexAlgebra.Equiv (arcPVA m)
      (liPoissonVertexAlgebra (AffineSl2BoundaryCharacter.vacuum m)) :=
  arcPoissonEquivOfCharacter m hm (character_eq_product m hm)

theorem arcPoissonEquiv_conformal_iff (m : ℕ) (hm : 1 ≤ m) (N : ℕ)
    (x : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)) :
    (arcPoissonEquiv m hm).toAlgEquiv x ∈
      BoundaryArcConformalSqueeze.liConformalPiece (AffineSl2BoundaryCharacter.vacuum m) N
      ↔
      x ∈ arcConformalPiece m N :=
  arcPoissonEquivOfCharacter_conformal_iff m hm (character_eq_product m hm) N x
```

File `VOA/Examples/AffineSl2BoundaryClassicalFreeness.lean`.

```
def arcPoissonEquivOfCharacter (m : ℕ) (hm : 1 ≤ m)
    (hcharacter : character m = product m) :
    PoissonVertexAlgebra.Equiv (arcPVA m) (liPoissonVertexAlgebra
      (AffineSl2BoundaryCharacter.vacuum m)) :=
  PoissonVertexAlgebra.Equiv.ofBijective (uniformCandidate m hm).arcLiPoissonHom
    ⟨arcLiPoissonHom_injective_of_character m hm hcharacter,
      (uniformCandidate m hm).arcLiPoissonHom_surjective⟩
```

Hypotheses / status. $1 \le m$; unconditional

Theorem 20.21 (The C_2-algebra and the arc universal property). *For $m \ge 1$: (1) the map $\mathbb{C}[e,h,f] \to R_{L_{k_m}}$, $x_i \mapsto [\overline{x_i}]$, induces an isomorphism $A_m = \mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \cong R_{L_{k_m}(\mathfrak{sl}_2)}$; (2) for every commutative $\mathbb{C}$-algebra A, algebra maps $J_\infty A_m \to A$ correspond bijectively to algebra maps $R_{L_{k_m}} \to A[[t]]$, via the specific bijection obtained from Theorem 20.11 and (1); (3) this bijection is natural in A. So $J_\infty A_m$ represents the arc functor $A \mapsto \mathrm{Hom}(R_{L_{k_m}}, A[[t]])$, i.e. it is the arc algebra $J_\infty R_{L_{k_m}}$. (Naturality is not part of the type of* `actualC2ArcEquiv`*; it is the separate theorem* `actualC2ArcEquiv_natural`*; see the warning below.)*

Lean. `VOA.AffineSl2BoundaryClassicalFreeness.baseC2AlgEquiv`, `VOA.AffineSl2BoundaryClassicalFreeness.actualC2ArcEquiv`, `VOA.AffineSl2BoundaryClassicalFreeness.actualC2ArcEquiv_natural`, `VOA.AffineSl2BoundarySingularFamily.Candidate.baseC2AlgEquiv`, `VOA.AffineSl2BoundarySingularFamily.Candidate.actualC2ArcEquiv`, `VOA.AffineSl2BoundarySingularFamily.Candidate.actualC2ArcEquiv_natural`, `VOA.AffineSl2BoundarySingularFamily.Candidate.baseIdeal_eq_simpleSymbol_kernel`, `VOA.baseC2Map`

File `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

```
def baseC2AlgEquiv (m : ℕ) (hm : 1 ≤ m) :
    (BasePoly ℂ / baseIdeal (R := ℂ) m) ≃ₐ[ℂ] A₂ (AffineSl2BoundaryCharacter.vacuum m) :=
  baseC2AlgEquivOfCharacter m hm (character_eq_product m hm)

def actualC2ArcEquiv {A : Type*} [CommRing A] [Algebra ℂ A]
    (m : ℕ) (hm : 1 ≤ m) :
    (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A) ≃
      (A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ] PowerSeries A) :=
  actualC2ArcEquivOfCharacter m hm (character_eq_product m hm)

theorem actualC2ArcEquiv_natural {A B : Type*} [CommRing A] [Algebra ℂ A]
    [CommRing B] [Algebra ℂ B] (m : ℕ) (hm : 1 ≤ m)
    (f : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A) (g : A →ₐ[ℂ] B) :
    actualC2ArcEquiv m hm (g.comp f) = (PowerSeries.mapAlgHom g).comp (actualC2ArcEquiv m
      hm f) :=
```

File VOA/Filtered/BoundarySimpleArcPoisson.lean.

```
def Candidate.baseC2AlgEquiv
    (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom) :
    (BoundaryAuxiliary.BasePoly ℂ / BoundaryAuxiliary.baseIdeal (R := ℂ) m) ≃ₐ[ℂ]
      A₂ (BoundaryArcLiMap.simpleVOA C) :=
  baseC2AlgEquiv_of_arc_injective (BoundaryArcLiMap.simpleVOA C) (fun _ => 1)
    (fun i => ⟨BoundaryArcLiMap.gen C i,BoundaryArcLiMap.gen_mem C i⟩)
    (BoundaryAuxiliary.baseIdeal m) C.baseC2Relations (BoundaryArcLiMap.baseIdeal_le_kernel
     ↪ C)
    hinj C.baseC2Eval_surjective

def Candidate.actualC2ArcEquiv {A : Type*} [CommRing A] [Algebra ℂ A]
    (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom) :
    (InfiniteJet.PresentedJet (BoundaryAuxiliary.baseIdeal (R := ℂ) m) →ₐ[ℂ] A) ≃
      (A₂ (BoundaryArcLiMap.simpleVOA C) →ₐ[ℂ] PowerSeries A) :=
  InfiniteJet.arcEquivOfBaseAlgebraEquiv (BoundaryAuxiliary.baseIdeal m) (C.baseC2AlgEquiv
   ↪ hinj)

theorem Candidate.actualC2ArcEquiv_natural {A B : Type*} [CommRing A] [Algebra ℂ A]
    [CommRing B] [Algebra ℂ B] (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom)
    (f : InfiniteJet.PresentedJet (BoundaryAuxiliary.baseIdeal (R := ℂ) m) →ₐ[ℂ] A)
    (g : A →ₐ[ℂ] B) :
    C.actualC2ArcEquiv hinj (g.comp f) =
      (PowerSeries.mapAlgHom g).comp (C.actualC2ArcEquiv hinj f) :=

theorem Candidate.baseIdeal_eq_simpleSymbol_kernel
    (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom) :
    BoundaryAuxiliary.baseIdeal (R := ℂ) m = RingHom.ker (simpleSymbol C) := by
```

File VOA/Filtered/ArcBaseExactness.lean.

```
def baseC2Map : (MvPolynomial σ ℂ / I) →ₐ[ℂ] A₂ W :=
  Ideal.Quotient.liftₐ I (MvPolynomial.aeval (fun i => mk₂ W (gen i).1)) (fun _ hf => hI
   ↪ hf)
```

Hypotheses / status. $1 \le m$; unconditional

Definition 20.22 (The canonical map, the universal arc of R_V, and the zero-jet embedding). Let $\bar{e}, \bar{h}, \bar{f} \in V_1$ be the images in V of the three current states $e_{(-1)}\mathbf{1}$, $h_{(-1)}\mathbf{1}$, $f_{(-1)}\mathbf{1}$, and write $(x_0, x_1, x_2) = (e, h, f)$.

1. The *canonical map* $\kappa : A_m \to R_V$ is the algebra map with $\kappa(x_i) = [\bar{x}_i]$, the C_2-class of the i-th current. (It is well defined because the boundary relations $x_i Q^m$ vanish in R_V, Section 20.4.)

2. The *universal arc of* R_V is the algebra map $\gamma : R_V \to B_m[[t]]$ obtained by composing $\kappa^{-1} : R_V \to A_m$ (Theorem 20.23 below shows κ is bijective) with the tautological arc $A_m \to B_m[[t]]$, $x_i \mapsto \sum_{n \ge 0} x_{i,n} t^n$ of Theorem 20.11.

3. The *zero-jet embedding* $\iota_0 : R_V \to B_m$ is κ^{-1} followed by the constant-arc inclusion $A_m \to B_m$, $x_i \mapsto x_{i,0}$; equivalently, the constant coefficient of γ.

Lean. VOA.AffineSl2BoundaryClassicalFreeness.canonicalC2Map, VOA.AffineSl2BoundaryClassicalFreeness.c2UniversalArc, VOA.AffineSl2BoundaryClassicalFreeness.c2ToArc, VOA.AffineSl2BoundaryClassicalFreeness.c2ToArc_eq_constantCoeff
File VOA/Examples/AffineSl2BoundaryTheorem11.lean.

```
def canonicalC2Map (m : ℕ) (hm : 1 ≤ m) :
    (BasePoly ℂ / baseIdeal (R := ℂ) m) →ₐ[ℂ] A₂ (AffineSl2BoundaryCharacter.vacuum m) :=
  baseC2Map (BoundaryArcLiMap.simpleVOA (uniformCandidate m hm)) (fun _ => 1)
    (fun i => ⟨BoundaryArcLiMap.gen (uniformCandidate m hm) i,
      BoundaryArcLiMap.gen_mem (uniformCandidate m hm) i⟩)
    (baseIdeal m) (uniformCandidate m hm).baseC2Relations

def c2UniversalArc (m : ℕ) (hm : 1 ≤ m) :
    A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ]
      PowerSeries (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)) :=
  (InfiniteJet.universalArc (baseIdeal (R := ℂ) m)).comp (baseC2AlgEquiv m
   ↪ hm).symm.toAlgHom

def c2ToArc (m : ℕ) (hm : 1 ≤ m) :
    A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ]
      InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) :=
  (InfiniteJet.baseToJet (baseIdeal (R := ℂ) m)).comp (baseC2AlgEquiv m hm).symm.toAlgHom

theorem c2ToArc_eq_constantCoeff (m : ℕ) (hm : 1 ≤ m)
    (a : A₂ (AffineSl2BoundaryCharacter.vacuum m)) :
    c2ToArc m hm a = PowerSeries.constantCoeff (c2UniversalArc m hm a) := rfl
```

Hypotheses / status. $1 \le m$; unconditional

Theorem 20.26 (Arakawa–Dai, Theorem 1.1, in the paper's form). *Let $m \ge 1$, $k = -2 + 2/(2m+1)$, $V = L_k(\mathfrak{sl}_2)$ (the quotient of $V^k(\mathfrak{sl}_2)$ by its maximal proper graded ideal), $R_V = V/C_2(V)$, $Q = h^2 + 4ef$. Realise $J_\infty R_V$ as $B_m = J_\infty\big(\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)\big)$ with the arc $\gamma : R_V \to B_m[[t]]$ of Definition 20.22. Then:*

1. *the canonical map $\kappa : \mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \to R_V$ is bijective;*
2. *κ sends e, h, f to the C_2-classes of the currents $e_{(-1)}\mathbf{1}$, $h_{(-1)}\mathbf{1}$, $f_{(-1)}\mathbf{1}$;*
3. *(B_m, γ) has the universal property of the arc algebra of R_V: for every commutative $\mathbb{C}$-algebra A and every algebra map $u : R_V \to A[[t]]$ there is a unique $f : B_m \to A$ with $f_* \circ \gamma = u$;*
4. *the Poisson vertex algebra isomorphism $\pi : B_m \to \mathrm{gr}^F V$ restricts on R_V (via the zero-jet embedding ι_0) to the canonical inclusion $c : R_V \to \mathrm{gr}^F V$;*
5. *π is the unique algebra map $B_m \to \mathrm{gr}^F V$ that commutes with the translations and restricts to c; that is, π is the canonical map $J_\infty R_V \to \mathrm{gr}^F V$;*
6. *for every N, $\pi(x)$ has conformal weight N if and only if x has conformal weight N.*

Consequently, the canonical map $J_\infty R_V \to \mathrm{gr}^F V$ is an isomorphism of conformally graded Poisson vertex algebras.

Lean. `VOA.AffineSl2BoundaryClassicalFreeness.arakawaDai_theorem_1_1`
File `VOA/Examples/AffineSl2BoundaryTheorem11.lean`.

```
theorem arakawaDai_theorem_1_1 (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (canonicalC2Map m hm) ∧
    (∀ i : Fin 3,
      canonicalC2Map m hm (Ideal.Quotient.mk (baseIdeal (R := ℂ) m) (MvPolynomial.X i)) =
        mk₂ (AffineSl2BoundaryCharacter.vacuum m)
          ((QuotVOA.maxGradedIdeal (AffineVOA.affineSl2VOA_closed (level m)
            (level_noncritical m))).mkQ
            (AffineVOA.J (level m) (AffineSl2UniversalC2PBW.symbolToCurrentColor i) (-1)
              (AffineVOA.vacuumA (level m))))) ∧
```

```
    (∀ (A : Type) [CommRing A] [Algebra ℂ A]
        (u : A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ] PowerSeries A),
      ∃! f : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A,
        (PowerSeries.mapAlgHom f).comp (c2UniversalArc m hm) = u) ∧
    (∀ a, (arcPoissonEquiv m hm).toAlgEquiv (c2ToArc m hm a) =
      c2ToLiAlgebra (AffineSl2BoundaryCharacter.vacuum m) a) ∧
    (∀ F : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ]
        liAssociatedGraded (AffineSl2BoundaryCharacter.vacuum m),
      (∀ x, F ((arcPVA m).translation x) =
        (liPoissonVertexAlgebra (AffineSl2BoundaryCharacter.vacuum m)).translation (F x)) →
      (∀ a, F (c2ToArc m hm a) = c2ToLiAlgebra (AffineSl2BoundaryCharacter.vacuum m) a) →
      F = (arcPoissonEquiv m hm).toAlgEquiv.toAlgHom) ∧
    (∀ (N : ℕ) (x : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)),
      (arcPoissonEquiv m hm).toAlgEquiv x ∈
        BoundaryArcConformalSqueeze.liConformalPiece (AffineSl2BoundaryCharacter.vacuum m)
          ↪ N ↔
        x ∈ arcConformalPiece m N) :=
  ⟨canonicalC2Map_bijective m hm, canonicalC2Map_mk_X m hm,
    fun _ _ _ u => c2UniversalArc_universal m hm u, arcPoissonEquiv_c2ToArc m hm,
    eq_arcPoissonEquiv_of_c2ToArc m hm, arcPoissonEquiv_conformal_iff m hm⟩
```

Hypotheses / status. $1 \le m$; unconditional; $J_\infty R_V$ realised as presented arc algebra with universal arc

Theorem 20.47 (Li's filtration of $V(c,0)$ and of $L(c,0)$ is the word filtration). *For every $c \in \mathbb{C}$ and every $p \in \mathbb{N}$: $F^pV(c,0) = C_p$, and $F^pL(c,0)$ is the image of C_p under the quotient map $V(c,0) \to L(c,0)$.*

Lean. `VOA.VirasoroLi.liFiltration_eq_virWordPiece`, `VOA.VirasoroLi.simpleLiFiltration_eq_virWordPiece`, `VOA.VirasoroLi.virWordNegativeModeClosed`
File `VOA/Filtered/VirasoroWordLi.lean`.

```
theorem liFiltration_eq_virWordPiece (p : ℕ) :
    liFiltration (virasoroVOA c) p = virWordPiece c p :=

theorem simpleLiFiltration_eq_virWordPiece (p : ℕ) :
    liFiltration (VirSimple.virasoroSimple c) p =
      (virWordPiece c p).map (VirSimple.maxIdealVir c).mkQ := by

theorem virWordNegativeModeClosed (q n : ℕ) (u : VirUniv c) {v : VirUniv c}
    (hv : v ∈ virWordPiece c q) :
    (virasoroVOA c).smul u (-(n : ℤ) - 1) v ∈ virWordPiece c (q + n) := by
```

Hypotheses / status. Every c; unconditional

Chapter 21

Huang's modular tensor category and the congruence property

9 of the chapter's 30 numbered statements: its headline results and every definition they use.

Definition 21.1 (Nice VOA; Chapter 22, Definitions 22.2 and 22.3). A VOA V is *pseudo-unitary* if every irreducible ordinary module $M \ncong V$ has all its L_0-eigenvalues of strictly positive real part. V is *nice* if it is strongly rational (simple, of CFT type, self-dual, C_2-cofinite and rational; Chapter 12, Definition 12.43) and pseudo-unitary.

Lean. `VOA.Classification.COne.Nice`
File `VOA/Classification/COne/Statement.lean`.

```
def Nice (V : VertexOperatorAlgebra 𝒱) : Prop :=
  StronglyRational V ∧ PseudoUnitary V
```

Theorem 21.5 (Huang–Lepowsky: the tensor category). *For a strongly rational V and any choice F of fusion products, the category of ordinary V-modules carries a monoidal structure with tensor product $\boxtimes_F$, unit V, the analytically defined associator and the unitors.*

Lean. `VOA.Huang.TensorCat.monoidalCategoryOfStronglyRational_pbw`, `VOA.Huang.TensorCat.Ordinary`, `VOA.Huang.FusionProduct.IsUniversal`, `VOA.Huang.FusionProduct.Product`, `VOA.Huang.TensorCat.FusionChoice`, `VOA.Huang.TensorCat.tensorObj`
File `VOA/Huang/PentagonCloseMain.lean`.

```
abbrev monoidalCategoryOfStronglyRational_pbw (hV : StronglyRational W) :
    MonoidalCategory (Ordinary W) :=
  monoidalCategoryOfStronglyRationalSewing F hV (fivePointSewingInput_stronglyRational_pbw
   ↪  F hV)
    (vacuumCreationCompatibility_pbw F hV)
```

File `VOA/Huang/TensorCatOrdinary.lean`.

```
structure Ordinary where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : OrdVModule W carrier
```

File `VOA/Huang/FusionProductBasic.lean`.

```
def IsUniversal {A₁ : OrdVModule W M₁} {A₂ : OrdVModule W M₂}
    {AT : OrdVModule W T} (Y : IntertwiningOperator A₁ A₂ AT) : Prop :=
  ∀ (N : Type u) [AddCommGroup N] [Module ℂ N] (AN : OrdVModule W N), UniversalFor Y AN
```

File VOA/Huang/FusionProductBasic.lean.

```
structure Product (A₁ : OrdVModule W M₁) (A₂ : OrdVModule W M₂) where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : OrdVModule W carrier
  intertwiner : IntertwiningOperator A₁ A₂ isMod
  universal : IsUniversal intertwiner
```

File VOA/Huang/TensorCatBifunctor.lean.

```
abbrev FusionChoice (W : VertexOperatorAlgebra V) :=
  ∀ X Y : Ordinary W, Product X.isMod Y.isMod
```

File VOA/Huang/TensorCatBifunctor.lean.

```
def tensorObj (X Y : Ordinary W) : Ordinary W := Ordinary.ofProduct (F X Y)
```

Hypotheses / status. Construction; VOA.pbw

Theorem 21.6 (The braiding). *The braiding given by the skew-symmetry of intertwining operators makes this monoidal category braided.*

Lean. VOA.Huang.TensorCat.HexagonClose.braidedCategory_stronglyRational_pbw, VOA.Huang.TensorCat.BraidRibbon.braidedCategoryOfStronglyRational_pbw_braiding, VOA.Huang.TensorCat.BraidRibbon.braiding, VOA.Huang.TensorCat.BraidRibbon.braidingP_intertwiner
File VOA/Huang/HexagonCloseResidual.lean.

```
abbrev braidedCategory_stronglyRational_pbw :
    @BraidedCategory (Ordinary W) _ (monoidalCategoryOfStronglyRational_pbw F hV) :=
  braidedCategoryOfStronglyRational_pbw F hV (hexagonInput_pbw F hV)
```

File VOA/Huang/BraidRibbonHexagon.lean.

```
theorem braidedCategoryOfStronglyRational_pbw_braiding (hH : HexagonInput F) (X Y :
↪  Ordinary W) :
    @BraidedCategory.braiding (Ordinary W) _ (monoidalCategoryOfStronglyRational_pbw F hV)
      (braidedCategoryOfStronglyRational_pbw F hV hH) X Y = braiding F X Y := rfl
```

File VOA/Huang/BraidRibbonBraiding.lean.

```
abbrev braiding (X Y : Ordinary W) : tensorObj F X Y ≅ tensorObj F Y X :=
  braidingP Phase.upper F X Y
```

File VOA/Huang/BraidRibbonBraiding.lean.

```
@[simp] theorem braidingP_intertwiner (X Y : Ordinary W) (n : ℂ) (x : X.carrier)
    (y : Y.carrier) :
    (braidingP p F X Y).hom.val ((F X Y).intertwiner.val n x y) =
      (skewP p Y X (tensorObj F Y X) (F Y X).intertwiner).val n x y :=
```

Hypotheses / status. Construction; VOA.pbw

Theorem 21.7 (Huang's rigidity theorem, right-rigid form, for nice V). *For a nice (strongly rational and pseudo-unitary) V the monoidal category of ordinary modules is right rigid: every object has a right dual.*

Lean. `VOA.Huang.HuangRigid.rightRigid_ordinary`
File `VOA/Huang/HuangRigidOrdinary.lean`.

```
theorem rightRigid_ordinary (F : FusionChoice W) (hNice : Nice W) :
    letI := monoidalCategoryOfStronglyRational_pbw F hNice.1
    Nonempty (RightRigidCategory (Ordinary W)) := by
```

Hypotheses / status. Proved for *nice V*; `VOA.pbw`

Theorem 21.8 (Huang's rigidity theorem, two-sided, for nice V). *For a nice (strongly rational and pseudo-unitary) V and any choice F of fusion products, the monoidal category of ordinary V-modules of Theorem 21.5 is rigid: every object has both a right dual and a left dual.*

Lean. `VOA.Huang.HuangRigid.rigid_ordinary`, `VOA.Huang.HuangRigid.rigidCategoryOrdinary`
File `VOA/Huang/RigidOrdinary.lean`.

```
theorem rigid_ordinary (F : FusionChoice W) (hNice : Nice W) :
    letI := monoidalCategoryOfStronglyRational_pbw F hNice.1
    Nonempty (RigidCategory (Ordinary W)) := by

def rigidCategoryOrdinary (F : FusionChoice W) (hNice : Nice W) :
    letI := monoidalCategoryOfStronglyRational_pbw F hNice.1
    RigidCategory (Ordinary W) :=
  Classical.choice (rigid_ordinary F hNice)
```

Hypotheses / status. Proved for *nice V*; `VOA.pbw`

Theorem 21.10 ($V' \cong V$ and the vacuum-fusion chain, for strongly rational V). *Let V be strongly rational. (i) If A is a V-module with $A \cong V$, then $A' \cong V$; no pseudo-unitarity is used. (ii) For a modular-data family (M_a) with $M_{v_0} \cong V$: $N_{ab}^{v_0} \le [M_b \cong M_a']$; $N_{ab}^{v_0} = 1$ (in particular $\neq 0$, and ≥ 1) when $M_a' \cong M_b$; and $\dim I\binom{V}{M_b\,M_a} \le 1$. So $N_{ab}^0 = \delta_{b,a'}$ holds without pseudo-unitarity.*

Lean. `VOA.contra_vacuum_equiv_of_stronglyRational`, `VOA.Huang.VerlindeVacuum.famFusion_vacuum_le_of_stronglyRational`
File `VOA/Modules/SelfDualContragredient.lean`.

```
theorem contra_vacuum_equiv_of_stronglyRational (hSR : StronglyRational W) (A : OrdVModule
 ↪  W M)
    (_hirr : A.IsIrreducible) (e : ModuleEquiv W A (adjointModule W)) :
    Nonempty (ModuleEquiv W (contragredientModule A) (adjointModule W)) :=
  contragredient_equiv_of_selfDual hSR.toStronglyFinite.selfDual A e
```

File `VOA/Huang/VerlindeVacuumFusionSR.lean` (one of five copies).

```
theorem famFusion_vacuum_le_of_stronglyRational (hSR : StronglyRational W) (v0 : ι)
    (hv0 : Nonempty (ModuleEquiv W (D.Mfam v0).isMod (adjointModule W))) (a b : ι) :
    famFusion D a b v0
      ≤ if Nonempty (ModuleEquiv W (D.Mfam b).isMod (contragredientModule (D.Mfam
       ↪  a).isMod))
        then 1 else 0 := by
```

Definition 21.11 (Huang's Verlinde theorem, fusion form, for nice V). For a nice (strongly rational and pseudo-unitary) V and a complete irredundant family of irreducibles with Zhu's modular matrix S, the vacuum row is nonzero, $S_{0a} = S_{a0}$, and $S_{ax}S_{bx} = S_{0x}\sum_c N_{ab}^c S_{cx}$ with the true fusion rules N_{ab}^c.

Lean. `VOA.Huang.VerlindeFusion.HuangVerlindeFusionInput`, `VOA.Classification.COne.huangVerlindeFusionInput_holds`
File `VOA/Huang/VerlindeFusion.lean`.

```
def HuangVerlindeFusionInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (W : VertexOperatorAlgebra 𝒱), Nice W →
    ∀ (ι : Type w) [Fintype ι] [DecidableEq ι] (D : VOA.ModularData W ι),
      ZhuModularityAnalytic D → Irredundant D.Mfam → Complete D.Mfam →
      ∀ v0 : ι, Nonempty (ModuleEquiv W (D.Mfam v0).isMod (adjointModule W)) →
        (∀ x, D.repS v0 x ≠ 0) ∧ (∀ a, D.repS v0 a = D.repS a v0) ∧
        ∀ a b x, D.repS a x * D.repS b x
          = D.repS v0 x * ∑ c, (famFusion D a b c : ℂ) * D.repS c x
```

File `VOA/Classification/COne/MainVerlindeProved.lean`.

```
theorem huangVerlindeFusionInput_holds :
↪  VOA.Huang.VerlindeFusion.HuangVerlindeFusionInput.{u, u + 1} :=
  VOA.Huang.VerlindeVacuum.huangVerlindeFusionInput_of_eigen_rho
    VOA.Huang.GenusOneG1f.fusionEigenInput_holds VOA.Huang.VerlindeRho.rhoNegOneInput_pbw
```

Hypotheses / status. Proved for *nice* V; `VOA.pbw`

Theorem 21.16 (Ng–Schauenburg, kernel half: kernels of modular representations are congruence). *For every modular category there is $N > 0$ such that $\Gamma(N)$ acts by scalars in every lift of the categorical projective $\mathrm{SL}_2(\mathbb{Z})$-representation.*

Lean. `VOA.Huang.DLNBridge.NSProjectiveKernelAll`, `VOA.Huang.DLNBridge.nsProjectiveKernelAll_holds`, `VOA.Huang.NgSchauenburg.IsModularRepresentation`, `VOA.Huang.NgSchauenburg.InPrincipalCongruence`, `VOA.Huang.NgSchauenburg.ProjectiveKernelContainsGamma`
File `VOA/Huang/NSCongruenceBridge.lean`.

```
def NSProjectiveKernelAll : Prop :=
  ∀ (C : Type w) [Category.{v} C] [MonoidalCategory C] [RightRigidCategory C]
    [Preadditive C] [Linear ℂ C] [MonoidalPreadditive C] [MonoidalLinear ℂ C]
    [HasKernels C] [HasFiniteBiproducts C] [BraidedCategory C]
    (F : SphericalFusion C) (M : Modular F),
    ∃ N : ℕ, 0 < N ∧ ∀ ρ : SL(2, ℤ) →* ((F.Label → ℂ) ≃ₗ[ℂ] (F.Label → ℂ)),
      IsModularRepresentation F M ρ → ProjectiveKernelContainsGamma ρ N
```

File `VOA/Huang/NSSMoveC4Assembly.lean`.

```
theorem nsProjectiveKernelAll_holds : NSProjectiveKernelAll.{v, w} :=
```

File `VOA/Huang/NgSchauenburgCongruence.lean`.

```
def IsModularRepresentation (F : SphericalFusion C) (M : Modular F)
    (ρ : SL(2, ℤ) →* ((F.Label → ℂ) ≃ₗ[ℂ] (F.Label → ℂ))) : Prop :=
  ∃ a b : ℂˣ,
    (∀ z i, ρ ModularGroup.S z i = (a : ℂ) * ∑ j, categoricalS F i j * z j) ∧
    (∀ z i, ρ ModularGroup.T z i = (b : ℂ) * M.twistScalar i * z i)
```

File `VOA/Huang/NgSchauenburgCongruence.lean`.

```
def InPrincipalCongruence (N : ℕ) (γ : SL(2, ℤ)) : Prop :=
  ∀ i j : Fin 2, (γ i j : ZMod N) = (1 : Matrix (Fin 2) (Fin 2) (ZMod N)) i j
```

File `VOA/Huang/NgSchauenburgCongruence.lean`.

```
def ProjectiveKernelContainsGamma (ρ : SL(2, ℤ) →* (W ≃ₗ[ℂ] W)) (N : ℕ) : Prop :=
  ∀ γ, InPrincipalCongruence N γ → ∃ c : ℂˣ, ∀ w, ρ γ w = (c : ℂ) • w
```

Hypotheses / status. Proved

Theorem 21.20 (The Dong–Lin–Ng congruence property, named and hypothesis-free, for nice V). *Let V be a nice VOA, D its modular data, ρ_V Zhu's representation on the labels of D (any Zhu datum D' on them).*

1. *(*Character form.*) For every level N of D, every $\gamma \in \Gamma(N)$, every label a and every τ with $\operatorname{Im}\tau > 0$:* $\mathrm{ch}_a(\gamma\tau) = \mathrm{ch}_a(\tau)$.
2. *(*Galois symmetry, nontrivial case.*) For every level N, every ℓ invertible modulo N with $\ell \not\equiv \pm 1$, and every $\gamma \equiv \operatorname{diag}(\ell, \ell^{-1}) \pmod N$, γ acts on the characters by a signed permutation.*
3. *(*Kernel form.*) There is $L > 0$ with $\Gamma(L) \subseteq \ker\rho_V$.*
4. *(*Kernel form at every level.*) For every level N of D, $\Gamma(N) \subseteq \ker\rho_V$.*

Lean. `VOA.Huang.CongruenceNamed.dlnCongruence_and_galois_holds`, `VOA.Huang.CongruenceNamed.dlnCongruenceInput_holds`, `VOA.Huang.CongruenceNamed.galoisSymmetryNontrivialInput_holds`, `VOA.Huang.CongruenceNamed.zhuCongruenceInput_holds`, `VOA.Huang.CongruenceNamed.zhu_kernelContainsGamma_at_level` File `VOA/Huang/CongruenceNamed.lean`.

```
theorem dlnCongruence_and_galois_holds :
    CongruenceGaloisSplit.DLNCongruenceInput.{u} ∧
      CongruenceGaloisSplit.GaloisSymmetryNontrivialInput.{u} :=
  NSGaloisCore.dln_inputs_of_beyl
    (DLNSchur.oddPrimePowerSchur_iff_largePrimeGaussLowerRelations.mpr
      DLNSchur.largePrimeGaussLowerRelations)
    DLNSchur.twoPowerInflationFromTwo_holds

theorem dlnCongruenceInput_holds : CongruenceGaloisSplit.DLNCongruenceInput.{u} :=
  dlnCongruence_and_galois_holds.1

theorem galoisSymmetryNontrivialInput_holds :
    CongruenceGaloisSplit.GaloisSymmetryNontrivialInput.{u} :=
  dlnCongruence_and_galois_holds.2

theorem zhuCongruenceInput_holds : DLNBridge.ZhuCongruenceInput.{u} :=
  DLNBeyl.zhuCongruence_of_beyl
    (DLNSchur.beylInflationSL2_of_large DLNSchur.largePrimeGaussLowerRelations)
    (DLNBridge.zhuProjectiveCongruence_of_NSKernel DLNBridge.nsProjectiveKernelAll_holds
      ZhuRep.zhuModularRepresentationInput_holds)

theorem zhu_kernelContainsGamma_at_level {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    {V : VertexOperatorAlgebra 𝒱} (hV : VOA.Classification.COne.Nice V)
    (D : VOA.Classification.COne.ModularData V) (hD : D.IsModularData)
    (D' : VOA.ModularData V D.ι) (hZD : DLNBridge.IsZhuDatum D hD D') {N : ℕ}
    (hN : D.IsLevel N) : DLN.KernelContainsGamma D'.rep N :=
  DLNBridge.kernel_at_level zhuCongruenceInput_holds hV D hD D' hZD hN
```

Hypotheses / status. proved, hypothesis-free; kernel half of NS 6.8 only; `VOA.pbw`

Chapter 22

The classification of nice central-charge-one vertex operator algebras (Gannon–Rayhaun)

9 of the chapter's 57 numbered statements: its headline results and every definition they use.

Definition 22.2 (Pseudo-unitarity; Gannon–Rayhaun Def. 2.4). A VOA V is *pseudo-unitary* if for every irreducible ordinary V-module $M \not\cong V$ and every $\alpha \in \mathbb{C}$ with $M_\alpha \neq 0$, $\operatorname{Re}\alpha > 0$.

Lean. `VOA.Classification.COne.PseudoUnitary`
File `VOA/Classification/COne/Statement.lean`.

```
def PseudoUnitary (V : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ (M : Type u) [AddCommGroup M] [Module ℂ M] (N : OrdVModule V M),
    N.IsIrreducible → IsEmpty (ModuleEquiv V N (adjointModule V)) →
      ∀ α : ℂ, N.weight α ≠ ⊥ → 0 < α.re
```

Hypotheses / status. Definitions

Definition 22.3 (Nice; Gannon–Rayhaun Def. 2.6). V is *nice* if it is strongly rational and pseudo-unitary.

Lean. `VOA.Classification.COne.Nice`
File `VOA/Classification/COne/Statement.lean`.

```
def Nice (V : VertexOperatorAlgebra 𝒱) : Prop :=
  StronglyRational V ∧ PseudoUnitary V
```

Hypotheses / status. Definitions

Theorem 22.20 (GR Lemma 3.11: the theta weights are non-negative). *Suppose a power series F equals $\sum_h A_h \eta\chi_h$ with $A_h \in \mathbb{N}$, $A_0 = 1$, and also $F = \sum_{t\in T}\sum_{j \bmod r_t} a_t(j)\,\theta_{j,r_t,t}$ with all t square-free. Then for each $t \in T$, $w_t := (r_t\sqrt{2t})^{-1}\sum_j a_t(j)$ is a non-negative real number.*

Lean. `VOA.Classification.COne.theta_weight_nonneg`
File `VOA/Classification/COne/WeightPositivity.lean`.

```
theorem theta_weight_nonneg (F : PowerSeries ℂ) (A : ℕ → ℕ) (hA0 : A 0 = 1)
    (hVir : F = map (Int.castRingHom ℂ) (etaVirasoroSum A))
    (T : Finset ℕ) (hT : ∀ t ∈ T, Squarefree t) (r : ℕ → ℕ) [∀ t, NeZero (r t)]
    (a : (t : ℕ) → ZMod (r t) → ℂ)
    (hθ : F = ∑ t ∈ T, ∑ j : ZMod (r t), a t j • map (Int.castRingHom ℂ) (thetaSeries (r t)
     ↪ j t))
```

```
    {t : ℕ} (ht : t ∈ T) :
    ∃ x : ℝ, 0 ≤ x ∧
      (1 / ((r t : ℂ) * (Real.sqrt (2 * t) : ℂ))) * ∑ j : ZMod (r t), a t j = x :=
```

Hypotheses / status. None; Lean's 3

Theorem 22.28 (Gannon–Rayhaun Theorem 3.1, forward direction). *Every nice vertex operator algebra of central charge* 1 *is isomorphic to some V_{2m} ($m \geq 1$), some V_{2m}^{+} ($m \geq 2$), or a fixed-point algebra V_2^G with $G \cong A_4$, S_4 or A_5.*

Lean. `VOA.Classification.COne.gannonRayhaun_forward`
File `VOA/Classification/COne/MainUnconditional.lean`.

```
theorem gannonRayhaun_forward : GannonRayhaunStatement.{u} :=
  gannonRayhaun_beyl_large VOA.Huang.DLNSchur.largePrimeGaussLowerRelations
```

Hypotheses / status. none; Lean's 3, via thm `VOA.pbw`

Theorem 22.32 (Pairwise distinctness). *A VOA carries at most one label of the list.*

Lean. `VOA.Classification.COne.plusExceptionalResidual`, `VOA.Classification.COne.exceptionalResidual`, `VOA.Classification.COne.gannonRayhaunDistinct`
File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem plusExceptionalResidual : PlusExceptionalResidual.{u} := by
```

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem exceptionalResidual : ExceptionalResidual.{u} := by
```

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem gannonRayhaunDistinct : GannonRayhaunDistinct.{u} :=
  gannonRayhaunDistinct_of_residuals plusExceptionalResidual exceptionalResidual
```

Hypotheses / status. none; Lean's 3

Theorem 22.33 (V_{2m} is nice). *For every $m \geq 1$, V_{2m} is strongly rational and pseudo-unitary.*

Lean. `VOA.Classification.COne.V2m_stronglyRational`, `VOA.Classification.COne.V2m_pseudoUnitary`, `VOA.Classification.COne.V2m_nice`
File `VOA/Classification/COne/LatticeStronglyRational.lean`.

```
theorem V2m_stronglyRational (m : ℕ) (hm : 1 ≤ m) : StronglyRational (V2m m hm) :=
```

File `VOA/Classification/COne/LatticeNice.lean`.

```
theorem V2m_pseudoUnitary : PseudoUnitary (V2m m hm) :=
```

File `VOA/Classification/COne/LatticeNice.lean`.

```
theorem V2m_nice : Nice (V2m m hm) := ⟨V2m_stronglyRational m hm, V2m_pseudoUnitary m hm⟩
```

Hypotheses / status. None; Lean's 3

Theorem 22.48 (Step 5b: the Jacobi bridge holds). *For every VOA V, every subgroup $G \leq \mathrm{Aut}\, V$ (finite or not), every proof of finite generation and of C_2-cofiniteness of V^G, the twisted Jacobi bridge holds.*

Lean. `VOA.Classification.COne.McRaeO5b.twistedJacobiBridge_holds`, `VOA.Classification.COne.McRaeO5b.residue_of_glue`
File `VOA/Classification/COne/McRaeO5bMain.lean`.

```
theorem twistedJacobiBridge_holds (G : Subgroup (Aut W)) (hfg : FinGen (orbifold
 ↪  G).adjointRestrict)
    (hC2 : C2Cofinite (orbifoldVOA G)) : TwistedJacobiBridge G hfg hC2 := by
```

File `VOA/Classification/COne/McRaeO5bLaurent.lean`.

```
theorem residue_of_glue {s t q : ℤ → ℂ} {n₀ n₁ m₁ : ℤ}
    (hs0 : ∀ n, n₀ < n → s n = 0) (ht0 : ∀ n, n < n₁ → t n = 0) (hq0 : ∀ m, m < m₁ → q m =
     ↪  0)
    {Ψ : ℂ → ℂ} (hΨ : ∀ ζ : ℂ, ζ ≠ 0 → ζ ≠ 1 → DifferentiableAt ℂ Ψ ζ)
    (hS : ∀ ζ : ℂ, 1 < ‖ζ‖ → HasSum (fun n : ℤ => s n * ζ ^ n) (Ψ ζ))
    (hT : ∀ ζ : ℂ, ζ ≠ 0 → ‖ζ‖ < 1 → HasSum (fun n : ℤ => t n * ζ ^ n) (Ψ ζ))
    (hQ : ∀ r : ℂ, r ≠ 0 → ‖r‖ < 1 → HasSum (fun m : ℤ => q m * r ^ m) (Ψ (1 + r)))
    (a ℓ : ℤ) : resInf a ℓ s - resZero a ℓ t = resOne a ℓ q := by
```

Hypotheses / status. none; Lean's 3, via thm `VOA.pbw`

Theorem 22.54 (Rationality of the orbifolds of V_2: McRae's Corollary 4.23). *For every finite subgroup $G \leq \mathrm{Aut}\, V_2$ with V_2^G C_2-cofinite, V_2^G is admissibly rational.*

Lean. `VOA.Classification.COne.twistedJacobiBridgeAll_holds`, `VOA.Classification.COne.finiteV2OrbifoldRationalOfC2`
File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem twistedJacobiBridgeAll_holds : McRaeO7.TwistedJacobiBridgeAll := by
  intro G _ hC2
  exact McRaeO5b.twistedJacobiBridge_holds G _ hC2
```

File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem finiteV2OrbifoldRationalOfC2 : FiniteV2OrbifoldRationalOfC2Fact :=
  McRaeO7.finiteV2OrbifoldRationalOfC2_of_bridge twistedJacobiBridgeAll_holds
```

Hypotheses / status. none; Lean's 3, via thm `VOA.pbw`

Theorem 22.59 (Gannon–Rayhaun Theorem 3.1 and Proposition 3.2, with the library's rationality convention). *A vertex operator algebra (with carrier in universe 0) is nice ("rational" read as* `AdmissiblyRational`*, Remark 22.4) and has central charge 1 if and only if it is isomorphic to one of V_{2m} $(m \geq 1)$, V_{2m}^{+} $(m \geq 2)$, $V_2^{A_4}$, $V_2^{S_4}$, $V_2^{A_5}$; and it is isomorphic to exactly one of them.*

Lean. `VOA.Classification.COne.gannonRayhaunConverse_holds`, `VOA.Classification.COne.gannonRayhaun_classification`
File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem gannonRayhaunConverse_holds : GannonRayhaunConverse.{0} :=
  McRaeO7.gannonRayhaunConverse_of_bridge twistedJacobiBridgeAll_holds
```

File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem gannonRayhaun_classification :
    GannonRayhaunStatement.{0} ∧ GannonRayhaunDistinct.{0} ∧ GannonRayhaunConverse.{0} :=
  ⟨gannonRayhaun_forward, gannonRayhaunDistinct, gannonRayhaunConverse_holds⟩
```

Hypotheses / status. none; Lean's 3, via thm `VOA.pbw`; universe 0

Chapter 23

Affine vertex algebras for every simple Lie algebra, and the exceptional Lie algebras

7 of the chapter's 37 numbered statements: its headline results and every definition they use.

Proposition 23.3 (Finite identities). *For Lie data as above:*

1. *(Casimir in coordinates)* $\sum_{a,d}\big(\sum_b \kappa^{ab} f^{bc}{}_d\big) f^{ad}{}_e = 2h\,\delta_{ce}$;
2. *(trace of* ad *vanishes)* $\sum_a f^{ac}{}_a = 0$ *for every* c *(note* $\operatorname{tr}\operatorname{ad}(b_c) = \sum_a f^{ca}{}_a = -\sum_a f^{ac}{}_a$*);*
3. *(central charge numerator)* $\sum_{a,b} \kappa^{ab}\kappa_{ab} = |\iota| = \dim \mathfrak{g}$.

Lean. `VOA.AffineGeneral.LieData.struct_casimir_contract`, `VOA.AffineGeneral.LieData.structureConstant_diag_sum`, `VOA.AffineGeneral.LieData.trace_kappaInv_formTable`
File `VOA/AffineGeneral/LieData.lean`.

```
theorem struct_casimir_contract (c e : ι) :
    (∑ a, ∑ d, (∑ b, D.kappaInv a b * D.structureConstant b c d) * D.structureConstant a d
     ↪ e) =
      if c = e then 2 * D.h else 0 := by

theorem structureConstant_diag_sum (c : ι) : (∑ a, D.structureConstant a c a) = 0 := by

theorem trace_kappaInv_formTable :
    (∑ a, ∑ b, D.kappaInv a b * D.formTable a b) = Fintype.card ι := by
```

Hypotheses / status. Unconditional

Theorem 23.10 (Universal affine VOA of a simple Lie algebra). *Let* $\mathfrak{g}$ *be a finite-dimensional simple complex Lie algebra.*

1. *For every nondegenerate symmetric invariant* κ *there is Lie data indexed by* $\{0,\dots,\dim\mathfrak{g}-1\}$*, with* $h = \frac{1}{2}\mu$*,* μ *the Casimir eigenvalue; for every* k *with* $k + h \neq 0$ *the universal affine VOA exists, with central charge* $\dim\mathfrak{g}\cdot k/(k+h)$.
2. *The Killing form* $\operatorname{Kil}(x,y) = \operatorname{tr}(\operatorname{ad} x\ \operatorname{ad} y)$ *is symmetric, invariant and nondegenerate, and in its normalization* $h = \frac{1}{2}$.

3. *For every complex number $\nu \neq 0$ the form $\kappa = \mathrm{Kil}/(2\nu)$ gives Lie data with Casimir number $h = \nu$, and for $k + \nu \neq 0$ the VOA $V^k(\mathfrak{g})$ in this normalization has central charge* $\dim \mathfrak{g} \cdot k/(k+\nu)$. *(Here ν is a free parameter, Lean's* `hv`*; only for $\nu = h^\vee(\mathfrak{g})$ is $\mathrm{Kil}/(2\nu)$ the standard normalization, which must be proved separately for each type.)*

Lean. `VOA.AffineGeneral.LieData.ofSimple`, `VOA.AffineGeneral.simpleData`, `VOA.AffineGeneral.universalAffineVOA`, `VOA.AffineGeneral.universalAffineVOA_centralCharge`, `VOA.AffineGeneral.killingInvariant`, `VOA.AffineGeneral.killingAffineVOA`, `VOA.AffineGeneral.killing_h`, `VOA.AffineGeneral.standardData`, `VOA.AffineGeneral.standardData_h`, `VOA.AffineGeneral.standardAffineVOA`, `VOA.AffineGeneral.standardAffineVOA_centralCharge`
File `VOA/AffineGeneral/Simple.lean` (and `VOA/AffineGeneral/Casimir.lean` for `LieData.ofSimple`).

```
def ofSimple [FiniteDimensional ℂ 𝔤] [LieAlgebra.IsSimple ℂ 𝔤] (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (b : Module.Basis ι ℂ 𝔤) : LieData κ ι :=

def simpleData (κ : LinearMap.BilinForm ℂ 𝔤) (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) : LieData κ (Fin (Module.finrank ℂ 𝔤)) :=
  LieData.ofSimple hinv hnd (Module.finBasis ℂ 𝔤)

def universalAffineVOA (κ : LinearMap.BilinForm ℂ 𝔤) (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
     ↪ hnd).h ≠ 0) :
    VertexOperatorAlgebra (AffineLie.vacuumSpace κ hinv k) :=
  affineVOA (simpleData 𝔤 κ hinv hnd) k hk

theorem universalAffineVOA_centralCharge (κ : LinearMap.BilinForm ℂ 𝔤) (hinv :
 ↪ κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
     ↪ hnd).h ≠ 0) :
    (universalAffineVOA 𝔤 κ hinv hnd k hk).centralCharge =
      (Module.finrank ℂ 𝔤 : ℂ) * k / (k + (simpleData 𝔤 κ hinv hnd).h) := by

theorem killingInvariant : (killingForm ℂ 𝔤).IsInvariant where

def killingAffineVOA (k : ℂ)
    (hk : k + (simpleData 𝔤 (killingForm ℂ 𝔤) (killingInvariant 𝔤)
      (LieAlgebra.IsKilling.killingForm_nondegenerate ℂ 𝔤)).h ≠ 0) :
    VertexOperatorAlgebra (AffineLie.vacuumSpace (killingForm ℂ 𝔤) (killingInvariant 𝔤) k)
     ↪ :=

theorem killing_h :
    (simpleData 𝔤 (killingForm ℂ 𝔤) (killingInvariant 𝔤)
      (LieAlgebra.IsKilling.killingForm_nondegenerate ℂ 𝔤)).h = 1 / 2 := by

def standardData (hv : ℂ) (hhv : hv ≠ 0) :
    LieData ((2 * hv)⁻¹ • killingForm ℂ 𝔤) (Fin (Module.finrank ℂ 𝔤)) :=

theorem standardData_h (hv : ℂ) (hhv : hv ≠ 0) :
    (standardData 𝔤 hv hhv).h = hv := by

def standardAffineVOA (hv : ℂ) (hhv : hv ≠ 0) (k : ℂ) (hk : k + hv ≠ 0) :
    VertexOperatorAlgebra (Vacuum (standardData 𝔤 hv hhv) k) :=
  affineVOA (standardData 𝔤 hv hhv) k (by rw [standardData_h]; exact hk)

theorem standardAffineVOA_centralCharge (hv : ℂ) (hhv : hv ≠ 0) (k : ℂ)
    (hk : k + hv ≠ 0) :
    (standardAffineVOA 𝔤 hv hhv k hk).centralCharge = (Module.finrank ℂ 𝔤 : ℂ) * k / (k +
     ↪ hv) := by
```

Hypotheses / status. Simple; $h = h^\vee$ only per type (`*_standard`)

Theorem 23.19 (Singular vectors die in $L_k(\mathfrak{g})$). *A homogeneous positive-mode singular vector of positive weight lies in the maximal ideal. In particular a quadratic state $t = \sum_i c_i J^{a_i}_{-1} J^{b_i}_{-1}|0\rangle$ lies in it as soon as, for all x, g,*

$$\sum_i c_i\Big(\delta_{a_i g}\, k\kappa_{x b_i} + \delta_{b_i g}\, k\kappa_{x a_i} + \sum_c f^{x a_i}{}_c f^{c b_i}{}_g\Big) = 0, \qquad \sum_i c_i\, k \sum_c f^{x a_i}{}_c \kappa_{c b_i} = 0.$$

Lean. `VOA.AffineGeneral.mem_maxIdeal_of_SingP`, `VOA.AffineGeneral.quadState`, `VOA.AffineGeneral.quadCoef1`, `VOA.AffineGeneral.quadCoef2`, `VOA.AffineGeneral.quadState_mem_maxIdeal`
Files `VOA/AffineGeneral/Singular.lean`, `VOA/AffineGeneral/SingularQuad.lean`.

```
theorem mem_maxIdeal_of_SingP {t : Vacuum D k} (ht : SingP D k t)
    {w : ℤ} (hw : 0 < w) (hmem : t ∈ gradingZA D k w) : t ∈ maxIdeal D k hk := by

def quadState (terms : List (ℂ × ι × ι)) : Vacuum D k :=
  (terms.map fun t => t.1 • st2 D k t.2.1 t.2.2).sum

def quadCoef1 (terms : List (ℂ × ι × ι)) (x g : ι) : ℂ :=
  (terms.map fun t => t.1 * ((if t.2.1 = g then k * formTable D x t.2.2 else 0)
    + (if t.2.2 = g then k * formTable D x t.2.1 else 0)
    + ∑ c, structureConstant D x t.2.1 c * structureConstant D c t.2.2 g)).sum

def quadCoef2 (terms : List (ℂ × ι × ι)) (x : ι) : ℂ :=
  (terms.map fun t => t.1 * (k * ∑ c, structureConstant D x t.2.1 c * formTable D c
   ↪ t.2.2)).sum

theorem quadState_mem_maxIdeal (hk : k + D.h ≠ 0) (terms : List (ℂ × ι × ι))
    (h1 : ∀ x g, quadCoef1 D k terms x g = 0) (h2 : ∀ x, quadCoef2 D k terms x = 0) :
    quadState D k terms ∈ maxIdeal D k hk :=
  mem_maxIdeal_of_SingP D k hk (singP_quad D k terms h1 h2) (by norm_num) (quadState_mem D
   ↪ k terms)
```

Hypotheses / status. Weight > 0

Proposition 23.21 (Kernel facts for current coordinates). *Let* $\mathrm{cls} : \mathbb{C}[X_1, \dots, X_n] \to \mathrm{gr}^F L_k(\mathfrak{g})$ *send X_i to the Li-degree-zero class of the current $[J^i_{-1}|0\rangle]$. Then (i) the kernel is stable under the coadjoint derivations* $\mathrm{ad}^*(b)$, $X_i \mapsto \sum f^{bi}{}_c X_c$, *and (ii) if a quadratic state $\sum c_i J^{a_i}_{-1} J^{b_i}_{-1}|0\rangle$ lies in $I_{\max}$, then $\sum c_i X_{a_i} X_{b_i} \in \ker \mathrm{cls}$.*

Lean. `VOA.AffineGeneral.clsG`, `VOA.AffineGeneral.clsG_adPoly_eq_zero`, `VOA.AffineGeneral.clsG_quad_eq_zero`
File `VOA/AffineGeneral/LiPresentation.lean`.

```
abbrev clsG : MvPolynomial (Fin n) ℂ →ₐ[ℂ] liAssociatedGraded W' :=
  aeval fun i => liDegreeZeroClass W' 1 ⟨g i, hg i⟩

theorem clsG_adPoly_eq_zero (b : Fin n) {p : MvPolynomial (Fin n) ℂ}
    (hp : clsG (simpleVOA D k hk) (genS D k hk) (genS_weight_one D k hk) p = 0) :
    clsG (simpleVOA D k hk) (genS D k hk) (genS_weight_one D k hk)
      (adPolyT (structureConstant D) b p) = 0 :=

theorem clsG_quad_eq_zero (terms : List (ℂ × Fin n × Fin n)) (h : quadState D k terms ∈
 ↪ maxIdeal D k hk) :
    clsG (simpleVOA D k hk) (genS D k hk) (genS_weight_one D k hk) (quadPoly terms) = 0 :=
     ↪ by
```

Hypotheses / status. Unconditional

Theorem 23.23 (Root-space criterion). *Let L be a complex Lie algebra, R a finite nonempty set, $e, h : R \to L$, $p : R \times R \to \mathbb{C}$, and $- : R \to R$, such that*

1. *the e_β and h_α together span L, and $e_\beta \neq 0$;*
2. *$[h_\alpha, e_\beta] = p(\alpha, \beta) e_\beta$ and $[h_\alpha, h_\beta] = 0$;*
3. *$p(\alpha, \alpha) \neq 0$, and the weights separate: for $\alpha \neq \beta$ some γ has $p(\gamma, \alpha) \neq p(\gamma, \beta)$;*
4. *$[e_\alpha, e_{-\alpha}] = c_\alpha h_\alpha$ with $c_\alpha \neq 0$;*
5. *no nonzero element of $\mathrm{span}(h)$ commutes with all e_β;*
6. *the graph on R with an edge $\alpha \to \beta$ when $p(\alpha, \beta) \neq 0$ is connected (every β is reachable from every α).*

Then L is simple.

Lean. `VOA.AffineGeneral.RootSpace.isSimple_of_roots`
File `VOA/AffineGeneral/RootSpaceSimple.lean`.

```
theorem isSimple_of_roots [Fintype R] [DecidableEq R] [Nonempty R]
    (hspan : span ℂ (Set.range e ∪ Set.range hb) = ⊤)
    (he0 : ∀ β, e β ≠ 0)
    (hbe : ∀ α β, ⁅hb α, e β⁆ = pair α β • e β)
    (hhb : ∀ α β, ⁅hb α, hb β⁆ = 0)
    (hdiag : ∀ α, pair α α ≠ 0)
    (sep : ∀ α β, α ≠ β → ∃ γ, pair γ α ≠ pair γ β)
    (neg : R → R) (hneg : ∀ α, ∃ c : ℂ, c ≠ 0 ∧ ⁅e α, e (neg α)⁆ = c • hb α)
    (center : ∀ y ∈ span ℂ (Set.range hb), (∀ β, ⁅y, e β⁆ = 0) → y = 0)
    (conn : ∀ α β, Relation.ReflTransGen (fun a b => pair a b ≠ 0) α β) :
    LieAlgebra.IsSimple ℂ L where
```

Hypotheses / status. Listed

Theorem 23.27 (Frenkel–Kac brackets, spanning, dimension and simplicity). *Let L be even and positive definite.*

1. *$[h_\alpha, e_\beta] = \langle \alpha, \beta \rangle e_\beta$, $[h_\alpha, h_\beta] = 0$, $[e_\alpha, e_{-\alpha}] = \varepsilon(\alpha, -\alpha) h_\alpha$, $e_\beta \neq 0$.*
2. *If the roots generate L, the e_β and h_β span $(V_L)_1$.*
3. *$\dim (V_L)_1 = \operatorname{rank} L + \#\{\beta : \langle \beta, \beta \rangle = 2\}$.*
4. *If a family s of roots generates L and is connected (the graph $\langle s_i, s_j \rangle \neq 0$ is connected), $(V_L)_1$ is simple.*

Lean. `VOA.AffineGeneral.LatticeLie.hV_eV`, `VOA.AffineGeneral.LatticeLie.hV_hV`, `VOA.AffineGeneral.LatticeLie.eV_eV_neg`, `VOA.AffineGeneral.LatticeLie.eV_ne_zero`, `VOA.AffineGeneral.LatticeLie.span_eV_hV`, `VOA.AffineGeneral.LatticeLie.finrank_V1`, `VOA.AffineGeneral.LatticeLie.isSimple_of_simpleRoots`
File `VOA/AffineGeneral/LatticeV1Simple.lean`.

```
theorem hV_eV (α β : Root L hPos) :
    ⁅hV L hEven hPos α, eV L hEven hPos β⁆ = ((L.form α.1 β.1 : ℤ) : ℂ) • eV L hEven hPos β
    ↪ := by
```

```
theorem hV_hV (α β : Root L hPos) : ⁅hV L hEven hPos α, hV L hEven hPos β⁆ = 0 := by

theorem eV_eV_neg (α : Root L hPos) :
    ⁅eV L hEven hPos α, eV L hEven hPos (Root.neg L hPos α)⁆
      = latticeCocycle L α.1 (-α.1) • hV L hEven hPos α := by

theorem eV_ne_zero (β : Root L hPos) : eV L hEven hPos β ≠ 0 := by

theorem span_eV_hV (hgen : RootsGenerate L) :
    span ℂ (Set.range (eV L hEven hPos) ∪ Set.range (hV L hEven hPos)) = ⊤ := by

theorem finrank_V1 : Module.finrank ℂ (V1 L hEven hPos) = rank L + (normEq L hPos 2).card
 ↪  := by

theorem isSimple_of_simpleRoots {ι : Type*} [Nonempty ι] (s : ι → Root L hPos)
    (hsgen : AddSubgroup.closure (Set.range fun i => (s i).1) = ⊤)
    (hsconn : ∀ i j, Relation.ReflTransGen (fun i j => L.form (s i).1 (s j).1 ≠ 0) i j) :
    LieAlgebra.IsSimple ℂ (V1 L hEven hPos) := by
```

Hypotheses / status. Connected generating roots

Theorem 23.32 ($\mathrm{Kil}/(2h^\vee)$ **is the standard normalization).** *Let L be generated by $n = \operatorname{rank} L$ simple roots s. Then the root vectors together with the simple coroots form a basis of $(V_L)_1$, and $\mathrm{Kil}(h_\alpha, h_\alpha) = \sum_\beta \langle\alpha,\beta\rangle^2$ (sum over roots). For A_2, D_4, E_6, E_7, E_8 this sum is $12, 24, 48, 72, 120 = 4h^\vee$ for every root α, so $\big(\mathrm{Kil}/(2h^\vee)\big)(h_\alpha, h_\alpha) = 2$.*

Lean. `VOA.AffineGeneral.LatticeLie.v1Basis`, `VOA.AffineGeneral.LatticeLie.killing_hV_hV`, `VOA.AffineGeneral.Normalization.e8_killing`, `VOA.AffineGeneral.Normalization.e8_standard`
Files `VOA/AffineGeneral/LatticeKilling.lean`, `VOA/AffineGeneral/LatticeNormalization.lean`.

```
def v1Basis (hrank : rank L = n) : Module.Basis (Root L hPos ⊕ Fin n) ℂ (V1 L hEven hPos)
 ↪  :=
  basisOfTopLeSpanOfCardEqFinrank (Sum.elim (eV L hEven hPos) fun i => hV L hEven hPos (s
   ↪  i))
    (span_eV_hV_simple hEven s hsgen).ge
    (by rw [finrank_V1, Fintype.card_sum, Fintype.card_fin, hrank, Fintype.card_coe,
     ↪  add_comm])

theorem killing_hV_hV (hrank : rank L = n) (α : Root L hPos) :
    killingForm ℂ (V1 L hEven hPos) (hV L hEven hPos α) (hV L hEven hPos α)
      = ∑ β : Root L hPos, ((L.form α.1 β.1 : ℤ) : ℂ) ^ 2 := by

theorem e8_killing (β : Root e8 e8_posDef) :
    killingForm ℂ e8Lie (hV e8 e8_even e8_posDef β) (hV e8 e8_even e8_posDef β) = 120 := by

theorem e8_standard (β : Root e8 e8_posDef) :
    ((2 * 30 : ℂ)⁻¹ • killingForm ℂ e8Lie) (hV e8 e8_even e8_posDef β)
      (hV e8 e8_even e8_posDef β) = 2 := by
```

Hypotheses / status. Unconditional

Chapter 24

The Deligne series I: presentations, carriers and the divisor comparison

14 of the chapter's 38 numbered statements: its headline results and every definition they use.

Theorem 24.6 (Exact presentation from classical freeness). *Let* $I \subseteq \ker(\mathrm{aeval}\, x)$. *If* (x, I) *is classically free, then* $\ker(\mathrm{aeval}\, x) = I$.

Lean. `VOA.presentedJetOrderZero`, `VOA.presentedJetRetraction`, `VOA.ker_eq_of_liPresentationClassicallyFree`
File `VOA/DeligneAudit/C2PresentationOfFreeness.lean`.

```
noncomputable def presentedJetOrderZero (I : Ideal (MvPolynomial σ ℂ)) :
    MvPolynomial σ ℂ →ₐ[ℂ] PresentedJet (R := ℂ) I :=
  (presentedJetMk (R := ℂ) I).comp (MvPolynomial.rename fun i : σ => (i, 0))

noncomputable def presentedJetRetraction (I : Ideal (MvPolynomial σ ℂ)) :
    PresentedJet (R := ℂ) I →ₐ[ℂ] (MvPolynomial σ ℂ ⧸ I) :=

theorem ker_eq_of_liPresentationClassicallyFree (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom)
    (hfree : LiPresentationClassicallyFree W x I hI) :
    RingHom.ker (MvPolynomial.aeval x).toRingHom = I := by
```

Hypotheses / status. Unconditional

Definition 24.8 (The A_2 relation ideal). In the coordinates $X_0, \dots, X_7$ of $e_1, e_2, e_{12}, f_1, f_2, f_{12}, h_1, h_2 \in \mathfrak{sl}_3$, put $Z = 3\sum_a X_a g^a$ (with g^a the trace-dual basis) and let P_a be one third of the entry of Z^2 at the position of the a-th generator (and of the two diagonal differences for $a = 6, 7$). Then $I_2^{\mathrm{ns}} = \mathrm{span}(P_0, \dots, P_7) \cong V(\theta)$, and I_2^{ns} (the ideal) is generated by these eight quadrics.

Lean. `VOA.DeligneAudit.A2.q`, `VOA.DeligneAudit.A2.relationPoly`, `VOA.DeligneAudit.A2.I2ns`
File `VOA/DeligneAudit/A2Relations.lean`.

```
abbrev q (a b : Fin 8) : MvPolynomial (Fin 8) ℂ := X a * X b

def relationPoly : Fin 8 → MvPolynomial (Fin 8) ℂ :=
  ![ -- weight α₁:  e₁ (h₁ + 2h₂) + 3 e₁₂ f₂
     q 0 6 + 2 * q 0 7 + 3 * q 2 4,
     -- weight α₂:  -e₂ (2h₁ + h₂) + 3 e₁₂ f₁
     -(2 * q 1 6) - q 1 7 + 3 * q 2 3,
     -- weight θ (highest):  e₁₂ (h₁ - h₂) + 3 e₁ e₂
     q 2 6 - q 2 7 + 3 * q 0 1,
```

```
    -- weight -α₁:  f₁ (h₁ + 2h₂) + 3 e₂ f₁₂
    q 3 6 + 2 * q 3 7 + 3 * q 1 5,
    -- weight -α₂:  -f₂ (2h₁ + h₂) + 3 e₁ f₁₂
    -(2 * q 4 6) - q 4 7 + 3 * q 0 5,
    -- weight -θ:  f₁₂ (h₁ - h₂) + 3 f₁ f₂
    q 5 6 - q 5 7 + 3 * q 3 4,
    -- weight 0:  h₁² + 2 h₁h₂ + 3 e₁₂ f₁₂ - 3 e₂ f₂
    q 6 6 + 2 * q 6 7 + 3 * q 2 5 - 3 * q 1 4,
    -- weight 0:  -h₂² - 2 h₁h₂ - 3 e₁₂ f₁₂ + 3 e₁ f₁
    -q 7 7 - 2 * q 6 7 - 3 * q 2 5 + 3 * q 0 3 ]

def I2ns : Ideal (MvPolynomial (Fin 8) ℂ) := Ideal.span (Set.range relationPoly)
```

Theorem 24.12 (The explicit presentations for all seven types). *For each $X \in \{A_2, G_2, D_4, F_4, E_6, E_7, E_8\}$ and the simple affine VOA $W_X = L_k(\mathfrak{g}_X)$ at the Deligne level, the ideal generated by the explicit relations of I_2^{ns} is contained in the kernel of the current-coordinate map, so* (currents, (I_2^{ns})) *is a current C_2 presentation.*

Lean. `VOA.DeligneAudit.A2.I2ns_le_ker`, `VOA.DeligneAudit.A2.explicitPresentation`, `VOA.DeligneAudit.D4Lie.I2ns_le_ker4`, `VOA.DeligneAudit.D4Lie.explicitPresentation4`, `VOA.DeligneAudit.E7Lie.I2ns_le_ker7`, `VOA.DeligneAudit.E7Lie.explicitPresentation7`, `VOA.DeligneAudit.E8Lie.I2ns_le_ker8`, `VOA.DeligneAudit.E8Lie.explicitPresentation8`
File `VOA/DeligneAudit/A2RelationsPresentation.lean`.

```
theorem I2ns_le_ker : I2ns ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass W 1 ⟨gen i, gen_weight_one i⟩).toRingHom :=
  I2ns_le_ker_of_two hT_A2 clsG_relationPoly_two

def explicitPresentation : CurrentC2Presentation .A2 W where
  gen := gen
  gen_weight_one := gen_weight_one
  ideal := I2ns
  relations := I2ns_le_ker
```

File `VOA/DeligneAudit/G2LiePresentation.lean`.

```
theorem I2ns_le_ker : G2data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass g2W 1 ⟨genS g2CertData (-5/3) g2_level_ne i,
        genS_weight_one g2CertData (-5/3) g2_level_ne i⟩).toRingHom := by

def explicitPresentation : CurrentC2Presentation .G2 g2W where
  gen := genS g2CertData (-5/3) g2_level_ne
  gen_weight_one := genS_weight_one g2CertData (-5/3) g2_level_ne
  ideal := G2data.I
  relations := I2ns_le_ker
```

File `VOA/DeligneAudit/D4LiePresentation.lean`.

```
theorem I2ns_le_ker4 : DeligneCarrier.D4.D4data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i => liDegreeZeroClass W4 1 ⟨gen4 i, gen4_weight_one
    ↪ i⟩).toRingHom := by

def explicitPresentation4 : CurrentC2Presentation .D4 W4 where
  gen := gen4
  gen_weight_one := gen4_weight_one
  ideal := DeligneCarrier.D4.D4data.I
  relations := I2ns_le_ker4
```

File VOA/DeligneAudit/F4LiePresentation.lean.

```
theorem I2ns_le_ker : F4data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass f4W 1 ⟨genS f4CertData (-5/2) f4_level_ne i,
        genS_weight_one f4CertData (-5/2) f4_level_ne i⟩).toRingHom := by

def explicitPresentation : CurrentC2Presentation .F4 f4W where
  gen := genS f4CertData (-5/2) f4_level_ne
  gen_weight_one := genS_weight_one f4CertData (-5/2) f4_level_ne
  ideal := F4data.I
  relations := I2ns_le_ker
```

File VOA/DeligneAudit/E6Presentation.lean.

```
theorem I2ns_le_ker : E6data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass W6 1 ⟨genS e6LieData k6 hk6 i, genS_weight_one e6LieData k6 hk6
       ↪ i⟩).toRingHom := by

def explicitPresentation : CurrentC2Presentation .E6 W6 where
  gen := genS e6LieData k6 hk6
  gen_weight_one := genS_weight_one e6LieData k6 hk6
  ideal := E6data.I
  relations := I2ns_le_ker
```

File VOA/DeligneAudit/E7LiePresentation.lean.

```
theorem I2ns_le_ker7 : DeligneCarrier.E7.E7data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i => liDegreeZeroClass W7 1 ⟨gen7 i, gen7_weight_one
     ↪ i⟩).toRingHom := by

def explicitPresentation7 : CurrentC2Presentation .E7 W7 where
  gen := gen7
  gen_weight_one := gen7_weight_one
  ideal := DeligneCarrier.E7.E7data.I
  relations := I2ns_le_ker7
```

File VOA/DeligneAudit/E8Presentation.lean.

```
theorem I2ns_le_ker8 : E8data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i => liDegreeZeroClass W8 1 ⟨gen8 i, gen8_weight_one
     ↪ i⟩).toRingHom := by

def explicitPresentation8 : CurrentC2Presentation .E8 W8 where
  gen := gen8
  gen_weight_one := gen8_weight_one
  ideal := E8data.I
  relations := I2ns_le_ker8
```

Hypotheses / status. unconditional

Theorem 24.14 (Current-jet surjectivity for every Deligne type). *(i) For any case, VOA and presentation, if the currents span Li's filtration then* `CurrentJetSurjectivity` *holds. (ii) For every Lie data D and non-critical k, the currents of $L_k(\mathfrak{g})$ span Li's filtration. Hence current-jet surjectivity holds for every explicit presentation of Theorem 24.12.*

Lean. VOA.DeligneAudit.currentJetSurjectivity_of_genWordSpan, VOA.DeligneAudit.A2.genWordsSpan, VOA.GeneratorModeTracked, VOA.generatorWordsSpanLiFiltration_of_tracked, VOA.generatorModeTracked_of_current_comm, VOA.generatorWordsSpanLiFiltration_quotient
File VOA/DeligneAudit/A2CurrentJetSurjectivity.lean.

```
theorem currentJetSurjectivity_of_genWordSpan {V : Type*} [AddCommGroup V] [Module ℂ V]
    {case : HigherRankDeligneCase} {W : VertexOperatorAlgebra V}
    (P : CurrentC2Presentation case W)
    (h : GeneratorWordsSpanLiFiltration W P.gen (fun _ => 1)) :
    CurrentJetSurjectivity case W P :=
```

File VOA/AffineGeneral/DeligneInstance.lean.

```
theorem genWordsSpan :
    GeneratorWordsSpanLiFiltration (simpleVOA D k hk) (presentation case D k hk).gen (fun _
     ↪  => 1) :=
```

File VOA/Filtered/GeneratorCurrentLiSpanning.lean.

```
def GeneratorModeTracked : Prop :=
  ∀ (i : Fin g) (m : ℤ) (q : ℕ) (v : V), v ∈ genWordPiece W gen wt q →
    W.smul (gen i) m v ∈ genWordPiece W gen wt (currentModeTarget q m)

theorem generatorWordsSpanLiFiltration_of_tracked (htrack : GeneratorModeTracked W gen wt)
    (hspan : Submodule.span ℂ (Set.range (genWordState W gen wt)) = ⊤) :
    GeneratorWordsSpanLiFiltration W gen wt := by

theorem generatorModeTracked_of_current_comm (hJ : ∀ i m v, W.smul (gen i) m v = J i m v)
    (hcomm : ∀ i j m p, J i m * J j p - J j p * J i m =
      (∑ c, f i j c • J c (m + p)) +
        (if m + p = 0 then cen i j m • (1 : Module.End ℂ V) else 0))
    (hvac : ∀ i m, 0 ≤ m → J i m W.vacuum = 0) :
    GeneratorModeTracked W gen wt := by

theorem generatorWordsSpanLiFiltration_quotient (h : GeneratorWordsSpanLiFiltration W gen
 ↪  wt) :
    GeneratorWordsSpanLiFiltration (QuotVOA.quotientVOA W I hI hgr)
      (fun i => I.mkQ (gen i)) wt := by
```

Hypotheses / status. Unconditional

Definition 24.16 (The current carrier W_m and the auxiliary carrier). Fix a basis σ of $\mathfrak{g}^*$ and quadrics Q. A *tensor polynomial* with m current and d auxiliary particles is a function $t \mapsto p_t$ from labellings $t : \{1..m\} \to \sigma$ to polynomials in $z_1, \dots, z_{m+d}$. It is *symmetric* if simultaneous permutation of roots and labels fixes it. For a pair $i < j$ of current slots, a relation q and spectator labels t, the *collision readback* is $\sum_{x,y} q(x,y)\, p_{t[i\mapsto x, j\mapsto y]}|_{z_j = z_i}$. The *current carrier* $\mathrm{currentCarrier}(Q, m, d)$ consists of the symmetric tensor polynomials all of whose readbacks vanish. The *auxiliary carrier* additionally requires, for every pair involving an auxiliary particle, that the polynomial vanish on that collision. $\mathrm{degreePiece}(C, r)$ is the subspace of C of polynomials homogeneous of degree r. $W_m := \mathrm{currentCarrier}(I_2, m, 0)$.

Lean. VOA.QuadraticCurrentFunctionalCarrier.TensorPolynomial, VOA.QuadraticCurrentFunctionalCarrier.Symmetric, VOA.QuadraticCurrentFunctionalCarrier.currentReadback, VOA.QuadraticCurrentFunctionalCarrier.CurrentEquations, VOA.QuadraticCurrentFunctionalCarrier.currentCarrier, VOA.QuadraticCurrentFunctionalCarrier.auxiliaryCarrier, VOA.DeligneAuxiliaryCollision.

`Particle`, `VOA.DeligneAuxiliaryCollision.conductor`, `VOA.DeligneAuxiliaryCollision.conductorDegree`, `VOA.QuadraticCurrentFunctionalCarrier.degreePiece`
File `VOA/Commutative/QuadraticCurrentFunctionalCarrier.lean`.

```
abbrev TensorPolynomial (m d : ℕ) :=
  (Fin m → σ) → MvPolynomial (Particle m d) K

def Symmetric {m d : ℕ} (p : TensorPolynomial (K := K) (σ := σ) m d) : Prop :=
  ∀ (e : Equiv.Perm (Particle m d)) (g : Equiv.Perm (Fin m)),
    ExtendsCurrentPermutation e g → ∀ t,
      MvPolynomial.rename e (p t) = p (t ∘ g.symm)

def currentReadback {m d : ℕ} (q : σ × σ → K)
    (ij : IncreasingPair (Fin m)) (t : Fin m → σ)
    (p : TensorPolynomial (K := K) (σ := σ) m d) :=
  projectedDiagonal (ij.1.1.castAdd d) (ij.1.2.castAdd d)
    ((Fin.castAdd_injective m d).ne (ne_of_gt ij.2)) q
    (fun xy => p (insertPair ij t xy))

def CurrentEquations {m d : ℕ} (Q : ρ → σ × σ → K)
    (p : TensorPolynomial (K := K) (σ := σ) m d) : Prop :=
  ∀ r ij t, currentReadback (Q r) ij t p = 0

def currentCarrier (Q : ρ → σ × σ → K) (m d : ℕ) :
    Submodule K (TensorPolynomial (K := K) (σ := σ) m d) where
  carrier := {p | Symmetric p ∧ CurrentEquations Q p}

def auxiliaryCarrier (Q : ρ → σ × σ → K) (m d : ℕ) :
    Submodule K (TensorPolynomial (K := K) (σ := σ) m d) where
  carrier := {p | p ∈ currentCarrier Q m d ∧
    ∀ t (ij : Pair m d), m ≤ ij.1.2.val →
      collisionMap ij.1.2 ij.1.1 (ne_of_lt ij.2) (p t) = 0}
```

File `VOA/Commutative/DeligneAuxiliaryCollision.lean`.

```
abbrev Particle (m d : ℕ) := Fin (m + d)

def conductor (m d : ℕ) : MvPolynomial (Particle m d) K :=
  ∏ ij : Pair m d, factor (K := K) ij ^ order ij

def conductorDegree (m d : ℕ) : ℕ := ∑ ij : Pair m d, order ij
```

File `VOA/Commutative/QuadraticCurrentGradedCarrier.lean`.

```
def degreePiece {m d : ℕ}
    (C : Submodule K (TensorPolynomial (K := K) (σ := σ) m d)) (r : ℕ) :
    Submodule K C where
  carrier := {p | ∀ t, (p.1 t).IsWeightedHomogeneous (fun _ => 1) r}
```

Hypotheses / status. Definitions

Theorem 24.17 (The restricted dual of a jet piece is a carrier piece (B4)). *For any field K, quadrics Q, and $r \le N$: the dual of the piece of the order-N presented jet algebra of* quadIdeal(Q) *with m particles and r derivatives is linearly isomorphic to* degreePiece(currentCarrier$(Q, m, 0), r$).

Lean. `VOA.QuadraticJetRestrictedDual.restrictedDualEquivCurrentCarrier`
File `VOA/Commutative/QuadraticJetRestrictedDual.lean`.

```
def restrictedDualEquivCurrentCarrier (Q : ρ → σ × σ → K) (N m r : ℕ) (hr : r ≤ N) :
    (quotientPiece Q N m r →ₗ[K] K) ≃ₗ[K] degreePiece (currentCarrier Q m 0) r :=
  (pieceDualEquivCoefficients Q N m r).trans (fixedQuadraticReindexingEquiv Q N m r hr)
```

Hypotheses / status. Unconditional

Theorem 24.18 (Geometric carrier embedding for every quadratic presentation). *If the ideal of a current C_2 presentation is* quadIdeal(Q)*, then* $\mathrm{Carrier}_{p,N} := \mathrm{degreePiece}(\mathrm{currentCarrier}(Q, N-p, 0), p)$ *receives an injective (for $p \le N$ bijective) restricted-dual map in every bidegree; hence a* `GeometricCarrierEmbedding`*.*

Lean. `VOA.DeligneAudit.CarrierEmbedding.carrierPiece`, `VOA.DeligneAudit.CarrierEmbedding.Carrier`, `VOA.DeligneAudit.CarrierEmbedding.geometricCarrierEmbedding_of_quadraticIdeal`
File `VOA/DeligneAudit/A2CarrierEmbedding.lean`.

```
abbrev carrierPiece (Q : ρ → σ × σ → K) (p N : ℕ) : Submodule K (currentCarrier Q (N - p)
 ↪  0) :=

abbrev Carrier (Q : ρ → σ × σ → K) (p N : ℕ) : Type _ := carrierPiece Q p N

def geometricCarrierEmbedding_of_quadraticIdeal {ρ : Type}
    (P : CurrentC2Presentation case W)
    (Q : ρ → Fin case.currentDimension × Fin case.currentDimension → ℂ)
    (hQ : P.ideal = quadraticIdeal Q) :
    GeometricCarrierEmbedding case W P (Carrier Q) where
  toCarrier := (carrierMaps Q P.ideal hQ).1
  injective := (carrierMaps Q P.ideal hQ).2
```

Hypotheses / status. Unconditional (weak alone)

Theorem 24.19 (The character package is one inequality). *Given current-jet surjectivity and a geometric carrier embedding,* $\dim W_N \le \sum_{p \le N} \dim \mathrm{Carrier}_{p,N}$ *holds automatically; hence* `OrdinaryCarrierCharacter` *follows from the reverse inequality. For a quadratic presentation, classical freeness and* $\ker = \mathrm{quadIdeal}(Q)$ *follow from current-jet surjectivity and the named residual* $\sum_p \dim \mathrm{Carrier}(Q)_{p,N} \le \dim W_N$*.*

Lean. `VOA.DeligneAudit.CarrierEmbedding.QuadraticCarrierUpperBound`, `VOA.DeligneAudit.CarrierEmbedding.caseClassicallyFree_of_quadraticCarrierUpperBound`
File `VOA/DeligneAudit/A2CarrierUpperBound.lean`.

```
structure QuadraticCarrierUpperBound {ρ : Type} {n : ℕ} (Q : ρ → Fin n × Fin n → ℂ)
    (W : VertexOperatorAlgebra V) : Prop where
  le : ∀ N, (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (Carrier Q p N)) ≤
    Module.finrank ℂ (W.grading N)

theorem caseClassicallyFree_of_quadraticCarrierUpperBound {ρ : Type}
    (P : CurrentC2Presentation case W)
    (Q : ρ → Fin case.currentDimension × Fin case.currentDimension → ℂ)
    (hQ : P.ideal = quadraticIdeal Q)
    (hsurj : CurrentJetSurjectivity case W P)
    (hbound : QuadraticCarrierUpperBound Q W) :
    CaseClassicallyFree case W P ∧
      RingHom.ker
          (MvPolynomial.aeval fun i =>
            liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩).toRingHom =
        P.ideal :=
```

Hypotheses / status. `QuadraticCarrierUpperBound`

Theorem 24.25 (Carrier side of the chain, any type). *For carrier data $\mathcal{C}$, a table* hilb *of Demazure Hilbert series and a family F: if* `SectorFusionBound D F` *and* $\mathrm{hilbSeries}(\mathrm{hilb})_n = M_{n,0}$ *(U6), then* $\mathrm{carrierDim}(N) \le \sum_{n \le N} [q^N] q^n \,\mathrm{hilb}_n/(q)_n$ *for every N.*

Lean. `VOA.DeligneAudit.DeligneCarrier.hilbSeries`, `VOA.DeligneAudit.DeligneCarrier.carrierDim_le_demazureSum`
File `VOA/DeligneAudit/DeligneCarrierDemazure.lean`.

```
def hilbSeries (hilb : ℕ → ℕ → ℕ) (n : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (hilb n N : ℚ)

theorem carrierDim_le_demazureSum (D : DeligneCarrierData) (hilb : ℕ → ℕ → ℕ)
    (F : ℕ → PowerSeries ℚ) (hF : SectorFusionBound D F)
    (hU6 : MixedFusionRecursion F (hilbSeries hilb)) (N : ℕ) :
    carrierDim D N ≤ demazureSumCoeff hilb N :=
```

Hypotheses / status. Sector bound, U6

Definition 24.27 (Loop operators, M_m, B_m, the discriminant). For structure constants f of $\mathfrak{g}$ in the chosen basis, the loop operator $x \otimes t^r$ acts on tensor polynomials by the coadjoint action in each slot, weighted by z_i^r: $(x \otimes t^r \cdot p)_t = -\sum_i z_i^r \sum_c f(x, t_i, c)\, p_{t[i \mapsto c]}$. Let $\delta_{c_0}^{\otimes m}$ be the tensor polynomial equal to 1 at the labelling $t \equiv c_0$ and 0 elsewhere, M_m the span of all loop words applied to it, B_m the symmetric polynomials and $\Delta = \prod_{i \ne j}(z_i - z_j) \in B_m$.

Lean. `VOA.DeligneAudit.SectorLoop.loopOp`, `VOA.DeligneAudit.SectorLoop.deltaVec`, `VOA.DeligneAudit.SectorLoop.Mspace`, `VOA.DeligneAudit.SectorFiber.SymAlg`, `VOA.DeligneAudit.A2.discS`
File `VOA/DeligneAudit/A2SectorLoop.lean`.

```
def loopOp (m : ℕ) (x : σ) (r : ℕ) :
    TensorPolynomial (K := K) (σ := σ) m 0 →ₗ[K] TensorPolynomial (K := K) (σ := σ) m 0
    ↪ where
  toFun p t := -∑ i, zr i ^ r * ∑ c, C (f x (t i) c) * p (Function.update t i c)

def deltaVec (m : ℕ) (c₀ : σ) : TensorPolynomial (K := K) (σ := σ) m 0 :=
  fun t => if t = fun _ => c₀ then 1 else 0

def Mspace (m : ℕ) (c₀ : σ) : Submodule K (TensorPolynomial (K := K) (σ := σ) m 0) :=
  Submodule.span K (Set.range fun l => applyWord f m l (deltaVec m c₀))
```

File `VOA/DeligneAudit/A2SectorFiber.lean`.

```
abbrev SymAlg := MvPolynomial.symmetricSubalgebra (Fin (m + 0)) K
```

File `VOA/DeligneAudit/A2SectorB8.lean`.

```
def discS (m : ℕ) : SymAlg (K := ℂ) m :=
  ⟨∏ i : Fin (m + 0), ∏ j : Fin (m + 0), if i = j then 1 else (X i - X j), by
```

Definition 24.28 (Sector type data). Sector data extend carrier data by: structure constants f satisfying the Jacobi identity; ad-stability of $I_2 = I_2^{\mathrm{ns}} \oplus \mathbb{C}Q$; an index c_0 with no relation containing $X_{c_0}^2$; the local Weyl data for $W_{\mathrm{loc}}(m\theta)$ (raising vectors killing δ_{c_0}, Cartan vectors acting by $\theta(h)$, simple lowering vectors with exponents $\theta(h_i)$, a weight function compatible with f and a combinatorial no-weight fact, a fusion set); and a *depth* L such that every basis vector of $\mathfrak{g}^*$ is a combination of coadjoint words of length $\le L$ applied to δ_{c_0}.

Lean. VOA.DeligneAudit.DeligneSector.slotA, VOA.DeligneAudit.DeligneSector.slotWords, VOA.DeligneAudit.DeligneSector.DeligneSectorData, VOA.DeligneAudit.SectorLoop.JacobiCoord, VOA.DeligneAudit.SectorLoop.RelationsStable
File VOA/DeligneAudit/DeligneSectorData.lean.

```
def slotA {n : ℕ} (f : Fin n → Fin n → Fin n → ℂ) (x : Fin n) (v : Fin n → ℂ) : Fin n → ℂ
 ↪  :=
  fun y => -∑ d, f x y d * v d

def slotWords {n : ℕ} (f : Fin n → Fin n → Fin n → ℂ) (c₀ : Fin n) (L : ℕ) :
    Submodule ℂ (Fin n → ℂ) :=
  Submodule.span ℂ {v | ∃ l : List (Fin n), l.length ≤ L ∧ v = wordA f l (Pi.single c₀ 1)}

structure DeligneSectorData extends DeligneCarrierData where
  ρFin : Fintype ρ
  f : Fin n → Fin n → Fin n → ℂ
  jacobi : JacobiCoord f
  stable : RelationsStable f toDeligneCarrierData.fullMatrix
  c₀ : Fin n
  diag : ∀ r, toDeligneCarrierData.fullMatrix r (c₀, c₀) = 0
  raising : Fin n → Prop
  raising_table : ∀ e, raising e → ∀ y, f e y c₀ = 0
  cartan : Fin n → Prop
  thetaH : Fin n → ℕ
  cartan_table : ∀ h, cartan h → ∀ y, f h y c₀ = if y = c₀ then -(thetaH h : ℂ) else 0
  hθ : Fin n
  hθ_cartan : cartan hθ
  hθ_ne : thetaH hθ ≠ 0
  lowering : Fin n → Prop
  lowExp : Fin n → ℕ
  R : ℕ
  wt : Fin n → Fin R → ℤ
  table_wt : ∀ x y c, wt c ≠ wt x + wt y → f x y c = 0
  lowering_noweight : ∀ i, lowering i → ∀ (m : ℕ) (t : Fin m → Fin n),
    ∑ j, wt (t j) ≠ m • wt c₀ - (m * lowExp i + 1) • wt i
  fusion : Fin n → Prop
  depth : ℕ
  cyclic : ∀ c, (Pi.single c 1 : Fin n → ℂ) ∈ slotWords f c₀ depth
  cartanN : Fin n → Prop := fun _ => False
  thetaN : Fin n → ℕ := fun _ => 0
  cartanN_table : ∀ h, cartanN h → ∀ y, f h y c₀ = if y = c₀ then (thetaN h : ℂ) else 0 :=
   ↪  by
    intro h hh; exact False.elim hh
```

File VOA/DeligneAudit/A2SectorRep.lean.

```
def JacobiCoord : Prop :=
  ∀ a b x c, (∑ d, f a x d * f b d c) - (∑ d, f b x d * f a d c) = -∑ e, f a b e * f e x c
```

File VOA/DeligneAudit/A2SectorLoop.lean.

```
def RelationsStable (Q : ρ → σ × σ → K) : Prop :=
  ∀ x r, quadraticRelation (adMatrix f x (Q r)) ∈
    Submodule.span K (Set.range fun s => quadraticRelation (Q s))
```

Hypotheses / status. Definitions; U5 named input

Definition 24.29 (Current modules, local Weyl relations, input U5). A *current module* is a vector space with operators $\rho(x \otimes t^r)$, $r \geq 0$, satisfying $[x \otimes t^r, y \otimes t^s] = \sum_e f(x,y,e)\, e \otimes t^{r+s}$. For a vector w, $\mathrm{cycDeg}(w,b)$ is the span of loop words of total t-degree b applied to w. The *local Weyl and fusion relations* for $m\theta$ are: raising vectors $\otimes t^r$ kill w; Cartan $h \otimes t^r$ kills w for $r \geq 1$ and acts by $m\theta(h)$ for $r = 0$; $(f_i \otimes 1)^{m\theta(h_i)+1} w = 0$; $(x \otimes t^m)w = 0$ for x in the fusion set. *Input U5* for a family F: every such module has $\dim \mathrm{cycDeg}(w,b) \leq [q^b]F_m$, and $\sum_{b<B} \dim \mathrm{cycDeg}(w,b) \leq (\dim \mathfrak{g})^m$ for every B. The cyclic module generated by w subject only to the local Weyl relations (the first three) is the *local Weyl module* $W_{\mathrm{loc}}(m\theta)$ of Chari and Pressley, the largest graded cyclic $\mathfrak{g}[t]$-module of highest weight $m\theta$ that is finite-dimensional; adding the fusion relations gives the *fusion module* $\mathcal{F}_m$. Every module satisfying the relations, with w cyclic, is a quotient of $\mathcal{F}_m$, so U5 is a statement about $\mathcal{F}_m$.

Lean. `VOA.DeligneAudit.SectorWeyl.CurrentModule`, `VOA.DeligneAudit.SectorWeyl.cycDeg`, `VOA.DeligneAudit.DeligneSector.LocalWeylRelG`, `VOA.DeligneAudit.DeligneSector.FusionModuleBoundG`
File `VOA/DeligneAudit/A2SectorWeyl.lean`.

```
structure CurrentModule [Fintype σ] (V : Type*) [AddCommGroup V] [Module K V] where
  op : σ → ℕ → V →ₗ[K] V
  comm : ∀ a b r s v, op a r (op b s v) - op b s (op a r v) = ∑ e, f a b e • op e (r + s) v

def cycDeg (ρ : CurrentModule f V) (w : V) (b : ℕ) : Submodule K V :=
  Submodule.span K {v | ∃ l, SectorGraded.wordDeg l = b ∧ v = wordOp ρ l w}
```

File `VOA/DeligneAudit/DeligneSectorWeyl.lean`.

```
structure LocalWeylRelG {V : Type*} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule S.f V) (w : V) (m : ℕ) : Prop where
  raising : ∀ e, S.raising e → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ h, S.cartan h → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartan_zero : ∀ h, S.cartan h → ρ.op h 0 w = ((m * S.thetaH h : ℕ) : ℂ) • w
  lowering : ∀ i, S.lowering i → (ρ.op i 0)^[m * S.lowExp i + 1] w = 0
  fusion : ∀ x, S.fusion x → ρ.op x m w = 0
  cartanN_pos : ∀ h, S.cartanN h → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartanN_zero : ∀ h, S.cartanN h → ρ.op h 0 w = -((m * S.thetaN h : ℕ) : ℂ) • w

structure FusionModuleBoundG (F : ℕ → PowerSeries ℚ) : Prop where
  le : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w :
   ↪ V),
    LocalWeylRelG S ρ w m → ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ PowerSeries.coeff
     ↪ b (F m)
  total : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w :
   ↪ V),
    LocalWeylRelG S ρ w m → ∀ B : ℕ,
      ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ S.n ^ m
```

Hypotheses / status. Definitions; U5 named input

Theorem 24.33 (B6: freeness from U5). *For sector data S and a family F satisfying U5, M_m is a finite free B_m-module for every m.*

Lean. `VOA.DeligneAudit.DeligneSector.MFreenessG`, `VOA.DeligneAudit.DeligneSector.mFreeness`
File `VOA/DeligneAudit/DeligneSectorB6.lean`.

```
structure MFreenessG : Prop where
  free : ∀ m, Module.Free (SymAlg (K := ℂ) m) (MspaceR S m)
```

```
  finite : ∀ m, Module.Finite (SymAlg (K := ℂ) m) (MspaceR S m)

theorem mFreeness (hU5 : FusionModuleBoundG S F) : MFreenessG S := by
```

Hypotheses / status. U5

Theorem 24.41 (The divisor comparison and the sector fusion bound, all seven types). *For each $X \in \{A_2, G_2, D_4, F_4, E_6, E_7, E_8\}$ the divisor comparison holds for the type's sector data, and therefore, for every family F, U5 for F implies the sector fusion bound* `SectorFusionBound Xdata F`. *For A_2, U5 is taken in the A_2-specific form* `SectorWeyl.FusionModuleBound F` *below, and the Lean theorem takes the divisor comparison as a second hypothesis, discharged by the proved theorem* `A2.divisorComparison`.

Lean. `VOA.DeligneAudit.A2.divisorComparison`, `VOA.DeligneAudit.DeligneSector.A2_sectorFusionBound_generic`, `VOA.DeligneAudit.SectorWeyl.LocalWeylRel`, `VOA.DeligneAudit.SectorWeyl.FusionModuleBound`, `VOA.DeligneAudit.A2.DivisorComparison`, `VOA.DeligneAudit.DeligneSector.fusionModuleBoundG_of`, `VOA.DeligneAudit.DeligneSector.divisorComparisonG_of`, `VOA.DeligneAudit.DeligneDivisor.G2.g2_pairComparison`, `VOA.DeligneAudit.DeligneDivisor.G2.g2_divisorComparison`, `VOA.DeligneAudit.DeligneDivisor.G2.G2_sectorFusionBound_of_U5`, `VOA.DeligneAudit.DeligneDivisor.D4.d4_pairComparison`, `VOA.DeligneAudit.DeligneDivisor.D4.d4_divisorComparison`, `VOA.DeligneAudit.DeligneDivisor.D4.D4_sectorFusionBound_of_U5`, `VOA.DeligneAudit.DeligneDivisor.D4.d4_carrierDim_le_demazureSum_of_U5_U6`, `VOA.DeligneAudit.DeligneDivisor.F4.f4_pairComparison`, `VOA.DeligneAudit.DeligneDivisor.F4.f4_divisorComparison`, `VOA.DeligneAudit.DeligneDivisor.F4.F4_sectorFusionBound_of_U5`, `VOA.DeligneAudit.DeligneDivisor.E6.e6_pairComparison`, `VOA.DeligneAudit.DeligneDivisor.E6.e6_divisorComparison`, `VOA.DeligneAudit.DeligneDivisor.E6.E6_sectorFusionBound_of_U5`, `VOA.DeligneAudit.DeligneDivisor.E7.e7_pairComparison`, `VOA.DeligneAudit.DeligneDivisor.E7.e7_divisorComparison`, `VOA.DeligneAudit.DeligneDivisor.E7.E7_sectorFusionBound_of_U5`, `VOA.DeligneAudit.DeligneDivisor.E8.e8_pairComparison`, `VOA.DeligneAudit.DeligneDivisor.E8.e8_divisorComparison`, `VOA.DeligneAudit.DeligneDivisor.E8.E8_sectorFusionBound_of_U5` File `VOA/DeligneAudit/A2DivisorFinal.lean`.

```
theorem divisorComparison : DivisorComparison := divisorComparison_of_pair pairComparison
```

File `VOA/DeligneAudit/DeligneSectorA2.lean`.

```
theorem A2_sectorFusionBound_generic (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBound F)
    (hD : A2.DivisorComparison) : SectorFusionBound A2data F :=
  sectorFusionBoundG A2sector F (fusionModuleBoundG_of F hU5) (divisorComparisonG_of hD)
```

The two A_2-specific hypotheses. File `VOA/DeligneAudit/A2SectorWeyl.lean`.

```
structure LocalWeylRel {V : Type*} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule (bracketTable) V) (w : V) (m : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = (m : ℂ) • w
  lowering_f1 : (ρ.op 3 0)^[m + 1] w = 0
  lowering_f2 : (ρ.op 4 0)^[m + 1] w = 0
  fusion : ∀ (fα : Fin 8), 3 ≤ fα.val → fα.val < 6 → ρ.op fα m w = 0

structure FusionModuleBound (F : ℕ → PowerSeries ℚ) : Prop where
  le : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable)
   ↪  V)
```

```
    (w : V), LocalWeylRel ρ w m → ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ coeff b (F
     ↪  m)
  /-- The total-dimension form of U5 (Ravinder Prop. 1): `dim W_loc(mθ)/⟨(f_α ⊗ t^m) w⟩ ≤
   ↪  8^m`,
  read on the `t`-graded pieces of any module satisfying the relations. -/
  total : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule
   ↪  (bracketTable) V)
    (w : V), LocalWeylRel ρ w m → ∀ B : ℕ,
      ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8 ^ m
```

File VOA/DeligneAudit/A2SectorB8.lean.

```
structure DivisorComparison : Prop where
  le : ∀ m, ∀ x ∈ currentCarrier fullMatrix m 0, ∀ p : HeightOnePrime (SymAlg (K := ℂ) m),
    discS m ∈ p.asIdeal → ∃ s : SymAlg (K := ℂ) m, s ∉ p.asIdeal ∧
      s • x ∈ Mspace bracketTable m (5 : Fin 8)
```

File VOA/DeligneAudit/DeligneSectorA2.lean.

```
theorem fusionModuleBoundG_of (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBound F) :
    FusionModuleBoundG A2sector F where

theorem divisorComparisonG_of (hD : A2.DivisorComparison) : DivisorComparisonG A2sector
 ↪  where
```

File VOA/DeligneAudit/G2Divisor.lean.

```
theorem g2_pairComparison : PairComparisonG G2sector :=
  pairComparisonG_of_cert (S := G2sector) qI hqD 4 18 48384 (by norm_num) (by norm_num) UD
   ↪  hAD hUD hVD

theorem g2_divisorComparison : DivisorComparisonG G2sector :=
  divisorComparisonG_of_pair g2_pairComparison

theorem G2_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
 ↪  G2sector F) :
    SectorFusionBound G2data F :=
  sectorFusionBoundG G2sector F hU5 g2_divisorComparison
```

File VOA/DeligneAudit/DeligneDivisorD4.lean.

```
theorem d4_pairComparison : PairComparisonG D4sector :=
  pairComparisonG_of_cert (S := D4sector) qI hq4 4 8 384 (by norm_num) (by norm_num) U4 hA4
   ↪  hU4 hV4

theorem d4_divisorComparison : DivisorComparisonG D4sector :=
  divisorComparisonG_of_pair d4_pairComparison

theorem D4_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
 ↪  D4sector F) :
    SectorFusionBound D4data F :=
  D4_sectorFusionBound F hU5 d4_divisorComparison

theorem d4_carrierDim_le_demazureSum_of_U5_U6 (F : ℕ → PowerSeries ℚ)
    (hU5 : FusionModuleBoundG D4sector F)
    (hU6 : MixedFusionRecursion F (hilbSeries d4DemazureHilb)) (N : ℕ) :
    carrierDim D4data N ≤ demazureSumCoeff d4DemazureHilb N :=
  d4_carrierDim_le_demazureSum_of_sector F hU5 d4_divisorComparison hU6 N
```

File VOA/DeligneAudit/F4Divisor.lean.

```
theorem f4_pairComparison : PairComparisonG F4sector :=
  pairComparisonG_of_cert (S := F4sector) qI hqD 4 48 559104 (by norm_num) (by norm_num) UD
    ↪ hAD hUD hVD

theorem f4_divisorComparison : DivisorComparisonG F4sector :=
  divisorComparisonG_of_pair f4_pairComparison

theorem F4_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪ F4sector F) :
    SectorFusionBound F4data F :=
  sectorFusionBoundG F4sector F hU5 f4_divisorComparison
```

File VOA/DeligneAudit/E6Divisor.lean.

```
theorem e6_pairComparison : PairComparisonG E6sector :=
  pairComparisonG_of_cert (S := E6sector) qI hq6 4 12 216 (by norm_num) (by norm_num) U6
    ↪ hA6 hU6 hV6

theorem e6_divisorComparison : DivisorComparisonG E6sector :=
  divisorComparisonG_of_pair e6_pairComparison

theorem E6_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪ E6sector F) :
    SectorFusionBound E6data F :=
  sectorFusionBoundG E6sector F hU5 e6_divisorComparison
```

File VOA/DeligneAudit/E7Divisor.lean.

```
theorem e7_pairComparison : PairComparisonG E7sector :=
  pairComparisonG_of_cert (S := E7sector) qI hq7 4 24 672 (by norm_num) (by norm_num) U7
    ↪ hA7 hU7 hV7

theorem e7_divisorComparison : DivisorComparisonG E7sector :=
  divisorComparisonG_of_pair e7_pairComparison

theorem E7_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪ E7sector F) :
    SectorFusionBound E7data F :=
  sectorFusionBoundG E7sector F hU5 e7_divisorComparison
```

File VOA/DeligneAudit/E8Divisor.lean.

```
theorem e8_pairComparison : PairComparisonG E8sector :=
  pairComparisonG_of_cert (S := E8sector) qI hq8 4 120 96 (by norm_num) (by norm_num) U8
    ↪ hA8 hU8 hV8

theorem e8_divisorComparison : DivisorComparisonG E8sector :=
  divisorComparisonG_of_pair e8_pairComparison

theorem E8_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪ E8sector F) :
    SectorFusionBound E8data F :=
  sectorFusionBoundG E8sector F hU5 e8_divisorComparison
```

Hypotheses / status. unconditional

Chapter 25

The Deligne series II: characters, the Arakawa–Kawasetsu MLDE and q-series identities

17 of the chapter's 37 numbered statements: its headline results and every definition they use.

Theorem 25.2 (The lower bound suffices). *Given a presentation P, current-jet surjectivity and an injective geometric carrier embedding, the carrier character lower bound implies the ordinary carrier character identity.*

Lean. `VOA.DeligneAudit.ordinaryCarrierCharacter_of_le`
File `VOA/DeligneAudit/A2Character.lean`.

```
theorem ordinaryCarrierCharacter_of_le
    (P : CurrentC2Presentation case W)
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)]
    [∀ p N, FiniteDimensional ℂ (Carrier p N)]
    (hsurj : CurrentJetSurjectivity case W P)
    (hgeom : GeometricCarrierEmbedding case W P Carrier)
    (hlow : CarrierCharacterLowerBound W Carrier) :
    OrdinaryCarrierCharacter W Carrier
```

Hypotheses / status. Packages; unconditional impl.

Definition 25.3 (Coefficient arithmetic; the Demazure sum). For coefficient tables $f, g : \mathbb{N} \to \mathbb{N}$ let $(f * g)(N) = \sum_{i=0}^{N} f(i)g(N-i)$ (the Cauchy product), and let $p_{\le n}(N)$ be the number of partitions of N into parts of size at most n, i.e. $[q^N]1/(q;q)_n$. For a table $\mathrm{hilb}(n, N) = [q^N]\mathrm{Hilb}_q D(1, n\theta)$, the Demazure sum is

$$D_N = \sum_{n=0}^{N} \big(\mathrm{hilb}(n, \cdot) * p_{\le n}\big)(N - n),$$

which is the q^N coefficient of (25.2).

Lean. `VOA.DeligneAudit.A2.conv`, `VOA.DeligneAudit.A2.partitionsLe`, `VOA.DeligneAudit.DeligneCharacter.demazureSumCoeff`
Files `VOA/DeligneAudit/A2CharacterInterface.lean` and `VOA/DeligneAudit/DeligneCharacter.lean`.

```
def conv (f g : ℕ → ℕ) (N : ℕ) : ℕ :=
  ∑ i ∈ Finset.range (N + 1), f i * g (N - i)
```

```
def partitionsLe : ℕ → ℕ → ℕ
  | 0, N => if N = 0 then 1 else 0
  | n + 1, N => ∑ j ∈ Finset.range (N / (n + 1) + 1), partitionsLe n (N - j * (n + 1))

def demazureSumCoeff (hilb : ℕ → ℕ → ℕ) (N : ℕ) : ℕ :=
  ∑ n ∈ Finset.range (N + 1), conv (hilb n) (partitionsLe n) (N - n)
```

Hypotheses / status. Definitions

Definition 25.4 (Named inputs of the character side, integer form). For a table `hilb`, a target sequence $t : \mathbb{N} \to \mathbb{N}$ and a VOA W:

- the *star identity* $D_N = t_N$ for all N;
- the *target lower bound* $t_N \le \dim W_N$ for all N;
- the *Demazure bound* $D_N \le \dim W_N$ for all N.

Lean. `VOA.DeligneAudit.DeligneCharacter.StarIdentityFor`, `VOA.DeligneAudit.DeligneCharacter.TargetLowerBound`, `VOA.DeligneAudit.DeligneCharacter.DemazureBoundFor`, `VOA.DeligneAudit.DeligneCharacter.demazureBoundFor_of_inputs`
File `VOA/DeligneAudit/DeligneCharacter.lean`.

```
structure StarIdentityFor (hilb : ℕ → ℕ → ℕ) (target : ℕ → ℕ) : Prop where
  eq : ∀ N, demazureSumCoeff hilb N = target N

structure TargetLowerBound (W : VertexOperatorAlgebra V) (target : ℕ → ℕ) : Prop where
  le : ∀ N : ℕ, target N ≤ Module.finrank ℂ (W.grading N)

structure DemazureBoundFor (W : VertexOperatorAlgebra V) (hilb : ℕ → ℕ → ℕ) : Prop where
  le : ∀ N : ℕ, demazureSumCoeff hilb N ≤ Module.finrank ℂ (W.grading N)

theorem demazureBoundFor_of_inputs {W : VertexOperatorAlgebra V} {hilb : ℕ → ℕ → ℕ}
    {target : ℕ → ℕ} (hstar : StarIdentityFor hilb target) (hlow : TargetLowerBound W
    ↪ target) :
    DemazureBoundFor W hilb
```

Definition 25.7 (The AK coefficients). $\sigma_k(n) = \sum_{d \mid n} d^k$. For $h \in \mathbb{N}$, $c_N = \texttt{mldeCoeff}\ h\ N \in \mathbb{Q}$ is defined by $c_0 = 1$ and (25.4) solved for c_N.

Lean. `VOA.DeligneAudit.DeligneCharacter.sigmaK`, `VOA.DeligneAudit.DeligneCharacter.mldeList`, `VOA.DeligneAudit.DeligneCharacter.mldeCoeff`
File `VOA/DeligneAudit/DeligneCharacterTypes.lean`.

```
def sigmaK (k n : ℕ) : ℕ := ∑ d ∈ n.divisors, d ^ k

def mldeList (h : ℕ) : ℕ → List ℚ
  | 0 => [1]
  | n + 1 =>
      let l := mldeList h n
      let N := n + 1
      l ++ [(∑ r ∈ Finset.range N,
          ((10 * ((h : ℚ) ^ 2 - 1) * sigmaK 3 (r + 1) -
            (24 * ((N : ℚ) - (r + 1)) + 2 * ((h : ℚ) + 1)) * sigmaK 1 (r + 1)) *
              l.getD (N - (r + 1)) 0)) / ((N : ℚ) * (6 * N + h))]

def mldeCoeff (h N : ℕ) : ℚ := (mldeList h N).getD N 0
```

Hypotheses / status. Definition

Theorem 25.8 (The recurrence, and uniqueness). *Call $d : \mathbb{N} \to K$ (K a field of characteristic 0) a* normalised AK solution *with parameter h if $d_0 = 1$ and (25.4) holds for all $N \geq 1$. Then $c =$ `mldeCoeff` h is one, and every normalised AK solution equals c (cast into K).*

Lean. `VOA.DeligneAudit.DeligneMLDE.akKernel`, `VOA.DeligneAudit.DeligneMLDE.IsAKSolution`, `VOA.DeligneAudit.DeligneMLDE.mldeCoeff_isAKSolution`, `VOA.DeligneAudit.DeligneMLDE.eq_mldeCoeff_of_isAKSolution`
File `VOA/DeligneAudit/DeligneMLDE.lean`.

```
def akKernel (h N r : ℕ) : ℚ :=
  10 * ((h : ℚ) ^ 2 - 1) * sigmaK 3 r - (24 * ((N : ℚ) - r) + 2 * ((h : ℚ) + 1)) * sigmaK 1
   ↪ r

structure IsAKSolution {K : Type*} [Field K] [CharZero K] (h : ℕ) (d : ℕ → K) : Prop where
  zero : d 0 = 1
  step : ∀ N : ℕ, 1 ≤ N →
    (N : K) * (6 * N + h) * d N
      = ∑ r ∈ Finset.range N, (akKernel h N (r + 1) : K) * d (N - (r + 1))

theorem mldeCoeff_isAKSolution (h : ℕ) : IsAKSolution (K := ℚ) h (mldeCoeff h) where

theorem eq_mldeCoeff_of_isAKSolution {K : Type*} [Field K] [CharZero K] {h : ℕ} {d : ℕ → K}
    (hd : IsAKSolution h d) : ∀ N, d N = (mldeCoeff h N : K)
```

Hypotheses / status. Unconditional

Definition 25.13 (Vacuum quartic relation; formal MLDE). For a VOA W and $\kappa \in \mathbb{C}$, the *vacuum quartic relation* with constant κ is the trace identity $S_W(L[-2]^2|0\rangle) = \kappa\, G_4 \star f$ on the adjoint module. The *formal MLDE* with constant κ is $\delta_q^2 f + G_2 \star (2\delta_q f) + G_4 \star (\frac{c}{2} f) = \kappa\, G_4 \star f$.

Lean. `VOA.DeligneAudit.DeligneMLDE.VacuumQuarticRelation`, `VOA.DeligneAudit.DeligneMLDE.FormalMLDE`
File `VOA/DeligneAudit/DeligneMLDEBlock.lean`.

```
structure VacuumQuarticRelation (W : VertexOperatorAlgebra 𝒱) (κ : ℂ) : Prop where
  eq : torusOnePoint (adjointModule W) (Lsq W (-2) (Lsq W (-2) W.vacuum))
    = κ • qConv (zhuG 2) (torusPartition (adjointModule W))

def FormalMLDE (κ : ℂ) : Prop :=
  qDeriv (qDeriv (torusPartition (adjointModule W)))
      + (qConv (zhuG 1) ((2 : ℂ) • qDeriv (torusPartition (adjointModule W)))
        + qConv (zhuG 2) ((W.centralCharge / 2) • torusPartition (adjointModule W)))
    = κ • qConv (zhuG 2) (torusPartition (adjointModule W))
```

Hypotheses / status. Definitions

Theorem 25.14 (The constant is forced; the AK recurrence). *Let W have central charge $c = -2(h+1)$ and $|0\rangle \neq 0$. If the formal MLDE (or the vacuum quartic relation, which implies it) holds with* some *constant κ, then $\kappa = (h+1)(5h-6)$, and the graded dimensions $N \mapsto \dim W_N$ form a normalised AK solution with parameter h. Hence $\dim W_N = c_N$ and* `TargetLowerBoundQ W (mldeCoeff h)`.

Lean. `VOA.DeligneAudit.DeligneMLDE.kappa_eq_of_formalMLDE`, `VOA.DeligneAudit.DeligneMLDE.isAKSolution_of_vacuumQuarticRelation`, `VOA.DeligneAudit.DeligneMLDE.targetLowerBoundQ_of_vacuumQuarticRelation`
File `VOA/DeligneAudit/DeligneMLDEBlock.lean`.

```
theorem kappa_eq_of_formalMLDE (h : ℕ) (hc : W.centralCharge = -2 * ((h : ℂ) + 1))
    (hvac : W.vacuum ≠ 0) {κ : ℂ} (hE : FormalMLDE W κ) :
    κ = ((h : ℂ) + 1) * (5 * h - 6)

theorem isAKSolution_of_vacuumQuarticRelation (h : ℕ)
    (hc : W.centralCharge = -2 * ((h : ℂ) + 1)) (hvac : W.vacuum ≠ 0) {κ : ℂ}
    (hrel : VacuumQuarticRelation W κ) :
    IsAKSolution h (fun N : ℕ => (Module.finrank ℂ (W.grading N) : ℂ))

theorem targetLowerBoundQ_of_vacuumQuarticRelation (W : VertexOperatorAlgebra 𝒱) (h : ℕ)
    (hc : W.centralCharge = -2 * ((h : ℂ) + 1)) (hvac : W.vacuum ≠ 0)
    {κ : ℂ} (hrel : VacuumQuarticRelation W κ) :
    DeligneCharacter.TargetLowerBoundQ W (DeligneCharacter.mldeCoeff h)
```

Hypotheses / status. c, $|0\rangle \neq 0$, relation

Theorem 25.16 (Square-bracket current algebra). *Let W be any VOA. For $a, b \in W_1$ with $a_{(1)}b = \beta|0\rangle$, and all $m, n \in \mathbb{Z}$,*

$$[a[m], b[n]] = (a_{(0)}b)[m+n] + m\beta\,\delta_{m+n,0}.$$

If moreover a is primary ($L_1 a = 0$), then $[L[m], a[n]] = -n\,a[m+n]$. For $a \in W_{P+1}$, $(L_{-1}a)[n] = -n\,a[n-1] - (P+1)\,a[n]$ (the translation rule).

Lean. `VOA.ZhuCoordinate.sqCurrent_comm`, `VOA.ZhuCoordinate.Lsq_sqCurrent_comm`, `VOA.ZhuCoordinate.vSqZ_L_neg_one`
File `VOA/Modularity/ZhuCoordinateCurrent.lean`.

```
theorem sqCurrent_comm {a b : 𝒱} (ha : a ∈ W.grading 1) (hb : b ∈ W.grading 1)
    {β : ℂ} (hp : W.smul a 1 b = β • W.vacuum) (m n : ℤ) (x : 𝒱) :
    vSqZ W a 1 m (vSqZ W b 1 n x) - vSqZ W b 1 n (vSqZ W a 1 m x)
      = vSqZ W (W.smul a 0 b) 1 (m + n) x + (if m + n = 0 then ((m : ℂ) * β) • x else 0)

theorem Lsq_sqCurrent_comm {a : 𝒱} (ha : a ∈ W.grading 1) (ha1 : W.L 1 a = 0) (m n : ℤ)
    (x : 𝒱) :
    Lsq W m (vSqZ W a 1 n x) - vSqZ W a 1 n (Lsq W m x) = (-(n : ℂ)) • vSqZ W a 1 (m + n) x

theorem vSqZ_L_neg_one {a : 𝒱} (P : ℕ) (ha : a ∈ W.grading ((P + 1 : ℕ) : ℤ)) (n : ℤ) :
    vSqZ W (W.L (-1) a) (P + 2) n
      = (-(n : ℂ)) • vSqZ W a (P + 1) (n - 1) - ((P : ℂ) + 1) • vSqZ W a (P + 1) n
```

Hypotheses / status. Unconditional

Theorem 25.21 (From carrier data to the target bound). *Suppose given the following data and hypotheses.*

1. Carrier data *CD (Chapter 24): a number n of currents (for the intended use, $n = \dim \mathfrak{g}$); quadrics $\mathrm{rel}_r \in \mathrm{Sym}^2$ spanning I_2^{ns}; a Casimir quadric Q; and a certificate of U3, i.e. $X_a Q \in (\mathrm{rel}_r)_r$ for every a.*

2. *A Lie algebra record D (a `LieData`, Chapter 23) on the index set $\{0, \dots, n-1\}$, with dual Coxeter number $D.h$ in the normalisation of its invariant form.*

3. *A level $k \in \mathbb{C}$ with $k + D.h \neq 0$, so that the Sugawara construction and L_k =`simpleVOA D k hk` exist.*

4. `hcas`*: a number $c_0 \in \mathbb{C}$ with* Cas $= c_0\,Q$. *Here* Cas $= \sum K_{ij} X_i X_j$ *is the Sugawara quadric of D, and Q is the certificate's Casimir.*

5. `hrel`*: every relation* rel_r *has vanishing symbol in* L_k *(Theorem 25.20).*

Then the vacuum quartic relation holds for L_k *with* $\kappa = \frac{5}{4}c^2 + \frac{11}{2}c$*. If moreover*

6. `hc`*:* $h \in \mathbb{N}$ *satisfies* $nk/(k + D.h) = -2(h + 1)$*, i.e. the central charge is* $-2(h + 1)$*,*

then $\dim L_k(\mathfrak{g})_N \geq c_N =$ `mldeCoeff` $h\ N$ *for all* N*.*

Lean. `VOA.DeligneAudit.VacuumRelation.vacuumQuarticRelation_of_carrierData`, `VOA.DeligneAudit.VacuumRelation.targetLowerBound_of_carrierData`
File `VOA/DeligneAudit/DeligneVacuumRelationCarrier.lean`.

```
theorem vacuumQuarticRelation_of_carrierData (CD : DeligneCarrierData) (D : LieData κ (Fin
↪  CD.n))
    (k : ℂ) (hk : k + D.h ≠ 0) (c₀ : ℂ)
    (hcas : (affineCurrentData D k hk).casPoly = MvPolynomial.C c₀ * CD.Q)
    (hrel : ∀ r, (affineCurrentData D k hk).sym (quadraticRelation (CD.rel r)) = 0) :
    VacuumQuarticRelation (simpleVOA D k hk)
      (5 / 4 * (simpleVOA D k hk).centralCharge ^ 2 + 11 / 2 * (simpleVOA D k
       ↪  hk).centralCharge) :=

theorem targetLowerBound_of_carrierData (CD : DeligneCarrierData) (D : LieData κ (Fin
↪  CD.n))
    (k : ℂ) (hk : k + D.h ≠ 0) (h : ℕ)
    (hc : (CD.n : ℂ) * k / (k + D.h) = -2 * ((h : ℂ) + 1)) (c₀ : ℂ)
    (hcas : (affineCurrentData D k hk).casPoly = MvPolynomial.C c₀ * CD.Q)
    (hrel : ∀ r, (affineCurrentData D k hk).sym (quadraticRelation (CD.rel r)) = 0) :
    DeligneCharacter.TargetLowerBoundQ (simpleVOA D k hk) (DeligneCharacter.mldeCoeff h)
```

Hypotheses / status. `hc`, `hcas`, `hrel`

Theorem 25.22 (The character bound for every Deligne type). *For each row of the table, the simple affine VOA* $W = L_k(\mathfrak{g})$ *satisfies, with no hypothesis,* $\dim W_N \geq c_N =$ `mldeCoeff` $h^\vee\ N$ *for all* N *(*`TargetLowerBoundQ W (mldeCoeff hv)`*). The last column says what else is stated as a Lean theorem.*

type	k	$h^\vee$	κ	*bound* $\dim W_N \geq c_N$	*also stated*
A_2	$-3/2$	*3*	*36*	*via* `finrank_eq_mldeCoeff'`	*the equality* $\dim W_N = c_N$*;* `vacuumQuarticRelation` *(*$\kappa = 36$*)*
G_2	$-5/3$	*4*	*70*	`g2_targetLowerBound`	`g2_vacuumQuarticRelation` *(as* $\frac{5}{4}c^2+\frac{11}{2}c$*); c and* $\lvert 0\rangle \neq 0$
D_4	-2	*6*	*168*	`targetLowerBoundQ4`	`vacuumQuarticRelation4` *(*$\kappa = 168$*)*
F_4	$-5/2$	*9*	*390*	`f4_targetLowerBound`	`f4_vacuumQuarticRelation` *(as* $\frac{5}{4}c^2+\frac{11}{2}c$*); c and* $\lvert 0\rangle \neq 0$
E_6	-3	*12*	*702*	`e6_targetLowerBound`	*only the inputs:* `e6_hc`*,* `e6_casimir_match`*,* `e6_hrel`
E_7	-4	*18*	*1596*	`e7_targetLowerBound`	*only the inputs:* `e7_hc`*,* `e7_casimir_match`*,* `e7_hrel`
E_8	-6	*30*	*4464*	`targetLowerBoundQ8`	`vacuumQuarticRelation8` *(*$\kappa = 4464$*)*

In every row $\kappa = (h^\vee + 1)(5h^\vee - 6)$*, as Theorem 25.14 forces.*

Lean. `VOA.DeligneAudit.A2.vacuumQuarticRelation`, `VOA.DeligneAudit.A2.finrank_eq_mldeCoeff'`, `VOA.DeligneAudit.D4Lie.vacuumQuarticRelation4`, `VOA.DeligneAudit.D4Lie.targetLowerBoundQ4`, `VOA.DeligneAudit.DeligneCarrier.E6.e6_hc`, `VOA.DeligneAudit.DeligneCarrier.E6.e6_casimir_match`, `VOA.DeligneAudit.DeligneCarrier.E6.e6_hrel`, `VOA.DeligneAudit.DeligneCarrier.E6.e6_targetLowerBound`, `VOA.DeligneAudit.E7Lie.e7_hc`, `VOA.`

DeligneAudit.E7Lie.e7_casimir_match, VOA.DeligneAudit.E7Lie.e7_hrel, VOA.DeligneAudit.E7Lie.e7_targetLowerBound, VOA.DeligneAudit.E8Lie.targetLowerBoundQ8, VOA.DeligneAudit.E8Lie.vacuumQuarticRelation8, VOA.DeligneAudit.G2Lie.g2_vacuumQuarticRelation, VOA.DeligneAudit.G2Lie.g2W_centralCharge, VOA.DeligneAudit.G2Lie.g2W_vacuum_ne_zero, VOA.DeligneAudit.G2Lie.g2_targetLowerBound, VOA.DeligneAudit.F4Lie.f4_vacuumQuarticRelation, VOA.DeligneAudit.F4Lie.f4W_centralCharge, VOA.DeligneAudit.F4Lie.f4W_vacuum_ne_zero, VOA.DeligneAudit.F4Lie.f4_targetLowerBound
Files VOA/DeligneAudit/DeligneVacuumRelationA2.lean, DeligneVacuumRelationD4.lean, E6Vacuum.lean, E7LieVacuum.lean, E7LieZhu.lean, DeligneVacuumRelationE8.lean, G2LieVacuum.lean, F4LieVacuum.lean (all in VOA/DeligneAudit/). Proof bodies are omitted: each block ends at := or := by. The line above each block names the type and its W.

```
theorem vacuumQuarticRelation : VacuumQuarticRelation W 36 := by

theorem finrank_eq_mldeCoeff' (N : ℕ) :
    (Module.finrank ℂ (W.grading N) : ℚ) = DeligneCharacter.mldeCoeff 3 N :=
```

```
theorem vacuumQuarticRelation4 : VacuumQuarticRelation W4 168 := by

theorem targetLowerBoundQ4 : DeligneCharacter.TargetLowerBoundQ W4
 ↪  (DeligneCharacter.mldeCoeff 6) :=
```

```
theorem e6_hc : ((E6data.n : ℕ) : ℂ) * k6 / (k6 + e6LieData.h) = -2 * (((12 : ℕ) : ℂ) + 1)
 ↪  := by

theorem e6_casimir_match : E6cd.casPoly = MvPolynomial.C (2 / 3 : ℂ) * E6data.Q :=

theorem e6_hrel (r : Fin 650) : E6cd.sym (quadraticRelation (E6data.rel r)) = 0 := by

theorem e6_targetLowerBound : DeligneCharacter.TargetLowerBoundQ W6
 ↪  (DeligneCharacter.mldeCoeff 12) :=
```

```
theorem e7_hc : ((DeligneCarrier.E7.E7data.n : ℕ) : ℂ) * (-4) / (-4 + e7CertData.h) =
    -2 * (((18 : ℕ) : ℂ) + 1) := by

theorem e7_casimir_match :
    (VacuumRelation.affineCurrentData (ι := Fin DeligneCarrier.E7.E7data.n) e7CertData (-4)
      hk7).casPoly = MvPolynomial.C (1 / 2 : ℂ) * DeligneCarrier.E7.E7data.Q :=

theorem e7_hrel (r' : Fin 1539) :
    (affineCurrentData (ι := Fin DeligneCarrier.E7.E7data.n) e7CertData (-4) hk7).sym
      (quadraticRelation (DeligneCarrier.E7.E7data.rel r')) = 0 :=

theorem e7_targetLowerBound : DeligneCharacter.TargetLowerBoundQ W7
 ↪  (DeligneCharacter.mldeCoeff 18) :=
```

```
theorem targetLowerBoundQ8 :
    DeligneCharacter.TargetLowerBoundQ W8 (DeligneCharacter.mldeCoeff 30) :=

theorem vacuumQuarticRelation8 : VacuumQuarticRelation W8 4464 := by
```

```
theorem g2_vacuumQuarticRelation :
    DeligneMLDE.VacuumQuarticRelation g2W
      (5 / 4 * g2W.centralCharge ^ 2 + 11 / 2 * g2W.centralCharge) :=

theorem g2W_centralCharge : g2W.centralCharge = -2 * (((4 : ℕ) : ℂ) + 1) := by
```

```
theorem g2W_vacuum_ne_zero : g2W.vacuum ≠ 0 := simple_vacuum_ne_zero' _ _ _

theorem g2_targetLowerBound :
    DeligneCharacter.TargetLowerBoundQ g2W (DeligneCharacter.mldeCoeff 4) :=
```

```
theorem f4_vacuumQuarticRelation :
    DeligneMLDE.VacuumQuarticRelation f4W
      (5 / 4 * f4W.centralCharge ^ 2 + 11 / 2 * f4W.centralCharge) :=

theorem f4W_centralCharge : f4W.centralCharge = -2 * (((9 : ℕ) : ℂ) + 1) := by

theorem f4W_vacuum_ne_zero : f4W.vacuum ≠ 0 := simple_vacuum_ne_zero' _ _ _

theorem f4_targetLowerBound :
    DeligneCharacter.TargetLowerBoundQ f4W (DeligneCharacter.mldeCoeff 9) :=
```

Hypotheses / status. unconditional

Theorem 25.26 (Euler's pentagonal theorem; the sum side of the quintuple product). $(q;q)_\infty = \sum_{r\in\mathbb{Z}}(-1)^r q^{r(3r-1)/2}$. *With* $J_1(u) = \sum(-1)^n u^n p^{n(n-1)/2}$ *and* $J_2(u) = \sum u^m p^{m^2}$, $J_1(u)J_2(u) = (p;p)_\infty \sum_j \left(u^{3j}p^{j(3j-1)} - u^{3j+1}p^{j(3j+1)}\right)$ *(the sum side of the quintuple product identity).*

Lean. `VOA.QSeries.JTP.pentSeries`, `VOA.QSeries.JTP.euler_eq_pentagonal`, `VOA.QSeries.JTP.quintuple_sum`
Files `VOA/Commutative/JacobiTriplePentagonal.lean`, `VOA/Commutative/JacobiTripleQuintuple.lean`.

```
def pentSeries : PowerSeries R := zSeries (fun r => negSign r) pentagonal

theorem euler_eq_pentagonal : (euler : PowerSeries R) = pentSeries := by

theorem quintuple_sum (u : Rˣ) :
    fsr (fun n : ℤ => negSign n * ((u ^ n : Rˣ) : R)) pentExp *
        fsr (fun m : ℤ => ((u ^ m : Rˣ) : R)) sqExp =
      euler * (fsr (fun j : ℤ => ((u ^ (3 * j) : Rˣ) : R)) (fun j => 2 * pentagonal j) -
        fsr (fun j : ℤ => ((u ^ (3 * j + 1) : Rˣ) : R)) (fun j => 2 * pentagonal (-j)))
```

Theorem 25.27 (Legendre's four-triangular-numbers theorem). *In* $\mathbb{Z}[[q]]$: $\displaystyle\sum_{v\ge 0}\frac{(2v+1)q^v}{1-q^{2v+1}} = (q;q)_\infty^4 \prod_{k\ge 1}(1+q^k)^8$.

Lean. `VOA.QSeries.JTP.lambertS`, `VOA.QSeries.JTP.lambertS_eq`
File `VOA/Commutative/FourTriangular.lean`.

```
def lambertS : PowerSeries R :=
  powerShiftSum 1 fun v => ((2 * v + 1 : ℕ) : PowerSeries R) * geomInvSeries (2 * v + 1)

theorem lambertS_eq :
    (lambertS : PowerSeries ℤ) = euler ^ 4 * infProd fun k => (1 + X ^ (k + 1)) ^ 8
```

Hypotheses / status. Unconditional

Theorem 25.28 (Ramanujan's three identities). *With* $E_2 = 1 - 24\sum\sigma_1 q^n$, $E_4 = 1 + 240\sum\sigma_3 q^n$, $E_6 = 1 - 504\sum\sigma_5 q^n$ *in* $\mathbb{Q}[[q]]$:

$$12\delta_q E_2 = E_2^2 - E_4, \qquad 3\delta_q E_4 = E_2E_4 - E_6, \qquad 2\delta_q E_6 = E_2E_6 - E_4^2.$$

Lean. VOA.DeligneAudit.DeligneCharacter.D4MLDE.sigS, VOA.DeligneAudit.DeligneCharacter.D4MLDE.E2s, VOA.DeligneAudit.DeligneCharacter.D4MLDE.E4s, VOA.DeligneAudit.DeligneCharacter.D4MLDE.E6s, VOA.DeligneAudit.DeligneCharacter.D4MLDE.RamanujanIdentities, VOA.DeligneAudit.DeligneCharacter.D4MLDE.ramanujanIdentities, VOA.DeligneAudit.DeligneMLDE.Heat.ramanujan, VOA.DeligneAudit.DeligneMLDE.EightSq.eight_sq
Files VOA/DeligneAudit/DeligneCharacterD4MLDE.lean, DeligneMLDERamanujan.lean, DeligneMLDERamanujanFull.lean, DeligneMLDERamanujanAdapters.lean, DeligneMLDEEightSq.lean.

```
def sigS (k : ℕ) : PowerSeries ℚ := PowerSeries.mk fun n => if n = 0 then 0 else (sigmaK k
 ↪  n : ℚ)

def E2s : PowerSeries ℚ := 1 - 24 * sigS 1
def E4s : PowerSeries ℚ := 1 + 240 * sigS 3
def E6s : PowerSeries ℚ := 1 - 504 * sigS 5

structure RamanujanIdentities : Prop where
  e2 : 12 * eulerOp E2s = E2s ^ 2 - E4s
  e4 : 3 * eulerOp E4s = E2s * E4s - E6s
  e6 : 2 * eulerOp E6s = E2s * E6s - E4s ^ 2

theorem ramanujanIdentities : RamanujanIdentities := by
```

```
theorem ramanujan : C 12 * (P1 * P1) = C 5 * P3 + P1 - C 6 * eulerOp P1 := by

theorem eight_sq (n : ℕ) :
    ∑ p ∈ Finset.antidiagonal n, e4 p.1 * e4 p.2 = e8 n := by
```

Hypotheses / status. Unconditional (Mathlib Sturm)

Theorem 25.30 (Lorenz's formula; Borwein's cubic identity; the G_2 target). *Let $a = \theta_{A_2} = \sum_{x,y\in\mathbb{Z}} q^{x^2-xy+y^2}$, $b^3 = (q;q)_\infty^9/(q^3;q^3)_\infty^3$, $c^3 = 27q(q^3;q^3)_\infty^9/(q;q)_\infty^3$. Then $a = 1+6\sum_{n\ge1}\big(\sum_{d\mid n}\chi_{-3}(d)\big)q^n$ (Lorenz), $a^3 = b^3 + c^3$ (Borwein), $2a^2 = 3E_2(q^3) - E_2$, $10a^4 = E_4 + 9E_4(q^3)$. Consequently the Kac–Wakimoto form $T = \theta_{A_2}(q)(q^3;q^3)_\infty^6/(q;q)_\infty^8$ has $[q^N]T = \texttt{mldeCoeff}\ 4\ N$.*

Lean. VOA.DeligneAudit.G2Star.lambertA2, VOA.DeligneAudit.G2Star.LorenzA2, VOA.DeligneAudit.G2Star.lorenzA2, VOA.DeligneAudit.G2Star.G2Level3Core, VOA.DeligneAudit.G2Star.g2Level3Core, VOA.DeligneAudit.G2Star.g2KW, VOA.DeligneAudit.G2Star.G2KWEqMLDE, VOA.DeligneAudit.G2Star.g2KWEqMLDE
Files VOA/DeligneAudit/G2StarRiccati.lean, G2StarMLDE.lean, G2StarLorenz.lean, G2StarReduction.lean, G2StarUnconditional.lean.

```
def lambertA2 : PowerSeries ℚ :=
  PowerSeries.mk fun n => if n = 0 then 1 else 6 * ∑ d ∈ n.divisors, chi3 d

def LorenzA2 : Prop := AQ = lambertA2

theorem lorenzA2 : LorenzA2 := aq_eq_one_add_six_L0.trans lambertA2_eq_L0.symm

structure G2Level3Core : Prop where
  l1 : AQ ^ 3 = BQ + CQ
  l2 : 2 * AQ ^ 2 = 3 * E2q3 - E2q
  l3 : 10 * AQ ^ 4 = E4q + 9 * E4q3

theorem g2Level3Core : G2Level3Core := g2Level3Core_of_eisLambert g2Level3EisLambert

def g2KW : PowerSeries ℤ := thetaA2 * euler3 ^ 6 * reciprocalEuler ^ 8
```

```
structure G2KWEqMLDE : Prop where
  eq : ∀ N, ((coeff N g2KW : ℤ) : ℚ) = mldeCoeff 4 N

theorem g2KWEqMLDE : G2KWEqMLDE := g2KWEqMLDE_of_eisLambert g2Level3EisLambert
```

Hypotheses / status. Unconditional

Theorem 25.32 (D_4, E_6, E_7, E_8 closed forms). *With $H_{E_6} = 7E_6^2 + 5E_4^3 - 12E_2E_4E_6$, $H_{E_7} = 211E_2E_4E_6^2 + 77E_2E_4^4 - 91E_6^3 - 197E_4^3E_6$, $H_{E_8} = 318910E_2E_4^4E_6^2 + 145105E_2E_4E_6^4 + 33649E_2E_4^7 - 43225E_6^5 - 139129E_4^6E_6 - 315310E_4^3E_6^3$:*

$$\begin{aligned}
\texttt{mldeCoeff}\ 12\ N &= [q^{N+2}]\, H_{E_6}(q)_\infty^{-22}/3991680,\\
\texttt{mldeCoeff}\ 18\ N &= [q^{N+3}]\, H_{E_7}(q)_\infty^{-34}/42343741440,\\
\texttt{mldeCoeff}\ 30\ N &= [q^{N+5}]\, H_{E_8}(q)_\infty^{-58}/1390924833817559040 0,\\
\texttt{mldeCoeff}\ 6\ N &= [q^{N+1}]\, \tfrac{1}{240}(\delta_q E_4)(q)_\infty^{-10}.
\end{aligned}$$

The last is the coefficientwise form of $[q^N]\, q^{-7/12}\,\delta_q E_4/(240\,\eta^{10})$: since $\eta^{10} = q^{10/24}(q)_\infty^{10}$, the prefactor is $q^{-7/12-5/12} = q^{-1}$. Check: $\frac{1}{240}\delta_q E_4 = \sum n\sigma_3(n)q^n = q + 18q^2 + \dots$, so $c_0 = 1$ and $c_1 = 18 + 10 = 28$ (Example 25.11).

Lean. `VOA.DeligneAudit.DeligneCharacter.E6MLDE.H6`, `VOA.DeligneAudit.DeligneCharacter.E6MLDE.e6Target`, `VOA.DeligneAudit.DeligneCharacter.E6MLDE.e6Target_eq_mldeCoeff`, `VOA.DeligneAudit.DeligneCharacter.MLDEClosed.e7Target_eq_mldeCoeff`, `VOA.DeligneAudit.DeligneCharacter.MLDEClosed.e8Target_eq_mldeCoeff`, `VOA.DeligneAudit.DeligneCharacter.D4MLDE.d4E4Target`, `VOA.DeligneAudit.DeligneCharacter.D4MLDE.d4E4Target_eq_mldeCoeff`
Files `VOA/DeligneAudit/DeligneCharacterE6MLDE.lean`, `DeligneCharacterE78MLDE.lean`, `DeligneCharacterD4MLDE.lean`.

```
def H6 : PowerSeries ℚ := 7 * E6s ^ 2 + 5 * E4s ^ 3 - 12 * E2s * E4s * E6s

def e6Target (N : ℕ) : ℚ := coeff N g6 / 3991680

theorem e6Target_eq_mldeCoeff (N : ℕ) : e6Target N = mldeCoeff 12 N := by
```

```
theorem e7Target_eq_mldeCoeff (N : ℕ) :
    coeff (N + 3) (HE7 * Qk 34) / 42343741440 = mldeCoeff 18 N := by

theorem e8Target_eq_mldeCoeff (N : ℕ) :
    coeff (N + 5) (HE8 * Qk 58) / 1390924833817559040 0 = mldeCoeff 30 N := by
```

```
def d4E4Target (N : ℕ) : ℚ := coeff N gser / 240

theorem d4E4Target_eq_mldeCoeff (hR : RamanujanIdentities) (N : ℕ) :
    d4E4Target N = mldeCoeff 6 N := by
```

Hypotheses / status. Unconditional

Theorem 25.36 (The A_2 star identity). *With $\mathrm{Hilb}_q D(1, n\theta) = P_{(2n,n,0)}(1,1,1;q,0)$ (the q-Whittaker polynomial, by Gelfand–Tsetlin branching at $t = 0$),*

$$\sum_{n\ge 0} \frac{q^n P_{(2n,n,0)}(1,1,1;q,0)}{(q;q)_n} = \prod_{r\ge 1}(1+q^r)^8.$$

Lean. `VOA.DeligneAudit.A2.demazureHilb`, `VOA.DeligneAudit.A2.StarIdentity`, `VOA.DeligneAudit.A2.StarReduction`, `VOA.DeligneAudit.A2.ShiftedMehler`, `VOA.DeligneAudit.A2.FourTriangular`, `VOA.DeligneAudit.A2.starIdentity`
Files `VOA/DeligneAudit/A2CharacterInterface.lean`, `A2CharacterStarChain.lean`, `A2Headline.lean`.

```
def demazureHilb (n N : ℕ) : ℕ := gl3Hilb (2 * n) n 0 N

structure StarIdentity : Prop where
  eq : ∀ N, demazureCoeff N = productCoeff N

structure StarReduction : Prop where
  eq : (euler : PowerSeries ℤ) * starF = powerShiftSum 1 fun v => shiftedU (2 * v)

structure ShiftedMehler : Prop where
  eq : ∀ m : ℕ, shiftedU m * (euler : PowerSeries ℤ) ^ 3 * (1 - q ^ (m + 1)) = (m + 1 : ℕ)

structure FourTriangular : Prop where
  eq : (powerShiftSum 1 fun v => ((2 * v + 1 : ℕ) : PowerSeries ℤ) * geomInv (2 * v + 1)) =
    (euler : PowerSeries ℤ) ^ 4 * fermionInf

theorem starIdentity : StarIdentity := starIdentity_of_fourTriangular fourTriangular
```

Hypotheses / status. Unconditional (q-series)

Theorem 25.37 (D_4: the series `d4SchurQ` = MLDE solution; Demazure = `d4SchurQ` under positivity). *Work over $\Bbbk = \mathbb{Q}((r))$, $r = q^{1/2}$. Let*

$$\Theta(x) = \sum_{k\in\mathbb{Z}} (-1)^k \big(q^{\binom{k}{2}} + q^{\binom{k+1}{2}}\big) x^{2k}, \qquad M(s,x,z) = \frac{(s^2;q)_\infty (s^2x^2;q)_\infty (s^2x^{-2};q)_\infty}{\prod_{\epsilon=\pm1}\prod_{i=1}^{4}\prod_{\sigma=\pm1}(s\,x^{\epsilon}z_i^{\sigma};q)_\infty},$$

a power series in s with coefficients Laurent polynomials in $x, z_1, \dots, z_4$ over $\Bbbk$; let $I(s,z) = \mathrm{CT}_x\,\Theta(x)M(s,x,z)$ (constant term in x), and let

$$\texttt{d4SchurQ}_N = \sum_{m=0}^{N} \Big[r^{2(N-m)}\Big]\,\tfrac{1}{2}\,[s^{2m}]\,I(s,z)\big|_{z=1}.$$

Then, unconditionally, $\texttt{d4SchurQ}_N = [q^{N+1}]\,\frac{1}{240}(\delta_q E_4)\,(q)_\infty^{-10} = c_N$ for $h = 6$; in modular notation this is $[q^N]\,q^{-7/12}\,\delta_q E_4/(240\eta^{10})$ (Theorem 25.32). If moreover `D4DemazurePositivity` holds, the D_4 Demazure sum equals $\texttt{d4SchurQ}_N$.

Lean. `VOA.DeligneAudit.DeligneCharacter.D4MLDE.D4SchurEqE4`, `VOA.DeligneAudit.DeligneCharacter.D4Wp.d4SchurEqE4`, `VOA.DeligneAudit.DeligneCharacter.D4Wp.d4SchurQ_eq_mldeCoeff_holds`, `VOA.DeligneAudit.DeligneCharacter.D4K.Kq`, `VOA.DeligneAudit.DeligneCharacter.D4K.R`, `VOA.DeligneAudit.DeligneCharacter.D4K.S`, `VOA.DeligneAudit.DeligneCharacter.D4K.tFactors`, `VOA.DeligneAudit.DeligneCharacter.D4K.hyperFactors`, `VOA.DeligneAudit.DeligneCharacter.D4K.kernelM`, `VOA.DeligneAudit.DeligneCharacter.D4K.thK`, `VOA.DeligneAudit.DeligneCharacter.D4K.ctTheta`, `VOA.DeligneAudit.DeligneCharacter.D4K.Ikernel`, `VOA.DeligneAudit.DeligneCharacter.D4K.ev1`, `VOA.DeligneAudit.DeligneCharacter.D4K.d4SchurQ`, `VOA.DeligneAudit.DeligneCharacter.D4K.D4DemazurePositivity`, `VOA.DeligneAudit.DeligneCharacter.D4DemazureEqSchur`, `VOA.DeligneAudit.DeligneCharacter.d4DemazureEqSchur_of_positivity`, `VOA.DeligneAudit.DeligneCharacter.D4Wp.d4DemazurePositivity_of_U5Z`
Files `VOA/DeligneAudit/DeligneCharacterD4MLDE.lean`, `DeligneCharacterD4Wp12.lean`, `DeligneCharacterD4Schur.lean`, `DeligneCharacterD4Final.lean`, `D4CharacterTop.lean`.

```
def D4SchurEqE4 : Prop := ∀ N, D4K.d4SchurQ N = d4E4Target N

theorem d4SchurEqE4 : D4MLDE.D4SchurEqE4 := by

theorem d4SchurQ_eq_mldeCoeff_holds (N : ℕ) : D4K.d4SchurQ N = mldeCoeff 6 N :=
```

```
abbrev Kq := LaurentSeries ℚ

abbrev R := AddMonoidAlgebra Kq V

abbrev S := PowerSeries R

def tFactors : S :=
  EPos 1 0 2 two_ne_zero * EPos 1 (2 • xv) 2 two_ne_zero * EPos 1 (-2 • xv) 2 two_ne_zero

def hyperFactors : S := (hyperVecs.map fun v => EInv 1 v 1 one_ne_zero).prod

def kernelM : S := tFactors * hyperFactors

def thK (i : ℤ) : Kq := if Even i then sgn (i / 2) * (qC (i / 2) + qC (i / 2 + 1)) else 0

abbrev ctTheta : R →ₗ[Kq] R := ctFun thK

def Ikernel : S := smap ctTheta kernelM

def ev1 : R →ₗ[Kq] Kq := Finsupp.linearCombination Kq fun _ => (1 : Kq)

def d4SchurQ (N : ℕ) : ℚ :=
  ∑ m ∈ Finset.range (N + 1), ((2 : Kq)⁻¹ * ev1 (coeff (2 * m) Ikernel)).coeff (2 * ((N :
   ↪  ℤ) - m))
```

Files `DeligneCharacterD4Schur.lean`, `DeligneCharacterD4Final.lean`, `DeligneCharacterD4Positivity.lean`.

```
structure D4DemazurePositivity : Prop where
  energy : ∀ m : ℕ, ∀ t ∈ demazureChar cartanD4 thetaD4 wordD4 m, 0 ≤ t.1.2 + (m : ℤ) * m
  coeff : ∀ (m : ℕ) (N : ℕ), 0 ≤ hilbInt m N

def D4DemazureEqSchur : Prop := ∀ N, (demazureSumCoeff d4DemazureHilb N : ℚ) = d4SchurQ N

theorem d4DemazureEqSchur_of_positivity (hpos : D4DemazurePositivity) : D4DemazureEqSchur
 ↪  :=

theorem d4DemazurePositivity_of_U5Z
    (hU5 : DeligneSector.FusionModuleBoundG DeligneSector.D4.D4sector d4FusionFZ) :
    D4K.D4DemazurePositivity :=
```

Hypotheses / status. Unconditional; `D4DemazurePositivity` (`coeff` only from open U5)

Chapter 26

The Deligne series III: fusion modules, Demazure modules and Ravinder's kernel

19 of the chapter's 35 numbered statements: its headline results and every definition they use.

Definition 26.2 (Evaluation modules; fusion products). (a) For a $\mathfrak{g}$-module V and $z \in \mathbb{C}$ the **evaluation module** $V(z)$ is V with $x(r)$ acting by $z^r x$ (with $0^0 = 1$). At $z = 0$ every $x(r)$, $r \geq 1$, acts by 0, and $V(0)$ is a graded module concentrated in degree 0. We write $V(\lambda)$ for the irreducible $\mathfrak{g}$-module of highest weight λ; $V(\theta) = \mathfrak{g}$.

(b) More generally, for a graded cyclic $\mathfrak{g}[t]$-module V and $z \in \mathbb{C}$, the **translate** V^z is V with $x(r)$ acting as $x \otimes (t+z)^r = \sum_s \binom{r}{s} z^{r-s} x(s)$. For $V = V(\lambda)(0)$ this is $V(\lambda)(z)$.

(c) Let $V_1, \dots, V_m$ be graded cyclic $\mathfrak{g}[t]$-modules with cyclic vectors v_j, and $z_1, \dots, z_m$ distinct. Put $T = V_1^{z_1} \otimes \cdots \otimes V_m^{z_m}$ (with $x(r)$ acting by the Leibniz rule) and $v = v_1 \otimes \cdots \otimes v_m$. Feigin and Loktev show that T is cyclic on v. Its **degree filtration** is

$$\Phi_{\leq d} = \operatorname{span}\{x_1(r_1) \cdots x_\ell(r_\ell) v \,:\, r_1 + \cdots + r_\ell \leq d\}, \qquad \Phi_{<d} = \Phi_{\leq d-1}.$$

Then $\Phi_{\leq d} \subseteq \Phi_{\leq d+1}$, $x(r)\Phi_{\leq d} \subseteq \Phi_{\leq d+r}$, and $\bigcup_d \Phi_{\leq d} = T$. The **fusion product** is the associated graded space

$$V_1 * \cdots * V_m = \bigoplus_{d \geq 0} \Phi_{\leq d}/\Phi_{<d},$$

on which $x(r)$ acts by the map $\Phi_{\leq d}/\Phi_{<d} \to \Phi_{\leq d+r}/\Phi_{<d+r}$ that it induces. We write V^{*m} for m factors equal to V.

(The chapter quotes no Lean declaration for this statement; see book p. 1393.)

Definition 26.6 (The A_2 relations and two named statements). Let ρ be a current module and $w \in V$.

(a) w satisfies the *relations of* $M_{a,b}$ if, with $k = a + b$: $e_c(r)w = 0$ for $c = e_1, e_2, e_\theta$ and all r; $h(r)w = 0$ for $r \geq 1$ and $h(0)w = k\,w$ for $h = h_1, h_2$; $f_1(0)^{k+1}w = f_2(0)^{k+1}w = 0$; $f_1(k)w = f_2(k)w = 0$; and $f_\theta(2a+b)w = 0$.

(b) **(TB)**, the *total bound* `FusionTotalBound`: for every m and every module with the local Weyl and fusion relations for $m\theta$, every $\operatorname{cyc}(w)_d$ is finite-dimensional and $\sum_{d<B} \dim \operatorname{cyc}(w)_d \leq 8^m$ for every B.

(c) **(KL)**, the *kernel relation* `RavinderKernelRel`: for $a \geq 1$, if w satisfies the relations of $M_{a,b}$, then $f_\theta(2a+b-1)w$ satisfies those of $M_{a-1,b}$.

Lean. `VOA.DeligneAudit.SectorWeyl.MabRel`, `VOA.DeligneAudit.SectorWeyl.FusionTotalBound`, `VOA.DeligneAudit.SectorWeyl.RavinderKernelRel`
File `VOA/DeligneAudit/DeligneFusionRel.lean`.

```
structure MabRel {V : Type} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule (bracketTable) V) (w : V) (a b : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = ((a + b : ℕ) : ℂ) • w
  lowering_f1 : (ρ.op 3 0)^[a + b + 1] w = 0
  lowering_f2 : (ρ.op 4 0)^[a + b + 1] w = 0
  trunc_f1 : ρ.op 3 (a + b) w = 0
  trunc_f2 : ρ.op 4 (a + b) w = 0
  trunc_theta : ρ.op 5 (2 * a + b) w = 0

def FusionTotalBound : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    LocalWeylRel ρ w m → (∀ b, FiniteDimensional ℂ (cycDeg ρ w b)) ∧
      ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8 ^ m

def RavinderKernelRel : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    1 ≤ a → MabRel ρ w a b → MabRel ρ (ρ.op 5 (2 * a + b - 1) w) (a - 1) b
```

Hypotheses / status. Definition

Theorem 26.7 (Finiteness clause for $\hat{\mathfrak{sl}}_3$). *If w satisfies the local Weyl and fusion relations for $m\theta$, then every* $\mathrm{cyc}(w)_b$ *is finite-dimensional. Consequently* `FusionTotalBound` *is equivalent to its dimension part* `FusionTotalDim`*: for every module with these relations and every B, $\sum_{b<B} \dim \mathrm{cyc}(w)_b \le 8^m$.*

Lean. `VOA.DeligneAudit.SectorWeyl.cycDeg_finiteDimensional`, `VOA.DeligneAudit.SectorWeyl.FusionTotalDim`, `VOA.DeligneAudit.SectorWeyl.fusionTotalBound_of_dim`
File `VOA/DeligneAudit/DeligneFusionFinite.lean`.

```
theorem cycDeg_finiteDimensional {w : V} {m : ℕ} (hW : LocalWeylRel ρ w m) (b : ℕ) :
    FiniteDimensional ℂ (cycDeg ρ w b) := by

def FusionTotalDim : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    LocalWeylRel ρ w m → ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8
     ↪ ^ m

theorem fusionTotalBound_of_dim (h : FusionTotalDim) : FusionTotalBound :=
```

Hypotheses / status. None; proved

Definition 26.8 (The fusion characters as the inverse of the mixed recursion). For any sequence of power series Dem_n define recursively

$$\mathrm{fusionOf}(\mathrm{Dem})_n = \mathrm{Dem}_n - \sum_{d=1}^{n} q^{dn} \begin{bmatrix} n \\ d \end{bmatrix}_q \mathrm{fusionOf}(\mathrm{Dem})_{n-d}.$$

For A_2, fusionF $=$ fusionOf$(\mathcal{D})$ with $\mathcal{D}_n$ the Gelfand–Tsetlin polynomial `demazureHilbSeries n`.

Lean. `VOA.DeligneAudit.DeligneCarrier.fusionOf`, `VOA.DeligneAudit.DeligneCarrier.mixedFusionRecursion_fusionOf`, `VOA.DeligneAudit.DeligneCarrier.fusionOf_unique`, `VOA.DeligneAudit.A2.fusionF`, `VOA.DeligneAudit.A2.mixedFusionRecursion_fusionF`
File `VOA/DeligneAudit/D4FusionF.lean`.

```
def fusionOf (Dem : ℕ → PowerSeries ℚ) : ℕ → PowerSeries ℚ
  | n => Dem n -
      ∑ d : Fin n, X ^ ((d.1 + 1) * n) * qbinom (X : PowerSeries ℚ) n (d.1 + 1) *
        fusionOf Dem (n - (d.1 + 1))
decreasing_by omega

theorem mixedFusionRecursion_fusionOf (Dem : ℕ → PowerSeries ℚ) :
    MixedFusionRecursion (fusionOf Dem) Dem := by

theorem fusionOf_unique {F Dem : ℕ → PowerSeries ℚ} (h : MixedFusionRecursion F Dem) (n :
 ↪  ℕ) :
    F n = fusionOf Dem n := by
```

File `VOA/DeligneAudit/DeligneFusionInverse.lean`.

```
def fusionF : ℕ → PowerSeries ℚ
  | n => demazureHilbSeries n -
      ∑ d : Fin n, X ^ ((d.1 + 1) * n) * qbinom (X : PowerSeries ℚ) n (d.1 + 1) * fusionF
       ↪  (n - (d.1 + 1))
decreasing_by omega

theorem mixedFusionRecursion_fusionF : MixedFusionRecursion fusionF demazureHilbSeries :=
 ↪  by
```

Hypotheses / status. None; proved

Definition 26.11 (The squeeze inputs and U7). (a) For $f, g \in \mathbb{Q}[[q]]$ write $f \le g$ (*coefficient-wise order*) if $[q^N]f \le [q^N]g$ for all N.

(b) A family $M_{a,b} \in \mathbb{Q}[[q]]$ satisfies the *squeeze inputs* if (nonneg) its coefficients are ≥ 0; (R) $M_{a+1,b} \le M_{a,b+1} + q^{2a+b+1} M_{a,b}$; (D) $\mathcal{D}_n \le M_{n,0}$; (T) $\sum_{N<B} [q^N] M_{0,m} \le 8^m$ for all m, B. These are the three inequalities of Section 26.2.5.

(c) It satisfies the *universal bound* if every module whose generator satisfies the relations of $M_{a,b}$ has $\dim \mathrm{cyc}(w)_N \le [q^N] M_{a,b}$.

(d) **U7 at n** (`DemazureRealization n`): there is a current module and a vector w satisfying the relations of $M_{n,0}$ with $\dim \mathrm{cyc}(w)_N \ge [q^N]\mathcal{D}_n$ for every N.

Lean. `VOA.DeligneAudit.A2.CoeffLe`, `VOA.DeligneAudit.A2.FusionSqueezeInputs`, `VOA.DeligneAudit.A2.MabUniversalBound`, `VOA.DeligneAudit.A2.DemazureRealization`
File `VOA/DeligneAudit/DeligneFusionSqueeze.lean`.

```
def CoeffLe (f g : PowerSeries ℚ) : Prop := ∀ N, coeff N f ≤ coeff N g

structure FusionSqueezeInputs (M : ℕ → ℕ → PowerSeries ℚ) : Prop where
  /-- Graded characters have nonnegative coefficients. -/
  nonneg : ∀ a b N, 0 ≤ coeff N (M a b)
  /-- Spanning half of Ravinder's exact sequences. -/
  rec_le : ∀ a b, CoeffLe (M (a + 1) b) (M a (b + 1) + X ^ (2 * a + b + 1) * M a b)
  /-- `Hilb_q D(1,nθ) ≤ ch W_loc(nθ)`, with U7 (Demazure = Gelfand–Tsetlin). -/
  dem_le : ∀ n, CoeffLe (demazureHilbSeries n) (M n 0)
  /-- F4: `dim M_{0,m} ≤ 8^m`. -/
  total_le : ∀ m B, ∑ N ∈ Finset.range B, coeff N (M 0 m) ≤ 8 ^ m
```

File `VOA/DeligneAudit/DeligneFusionDemazure.lean`.

```
def MabUniversalBound (M : ℕ → ℕ → PowerSeries ℚ) : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
    (w : V), MabRel ρ w a b → ∀ N, (Module.finrank ℂ (cycDeg ρ w N) : ℚ) ≤ coeff N (M a b)

def DemazureRealization (n : ℕ) : Prop :=
  ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule (bracketTable) V)
   ↪  (w : V),
    MabRel ρ w n 0 ∧ ∀ N, demazureHilb n N ≤ Module.finrank ℂ (cycDeg ρ w N)
```

Hypotheses / status. Proved for all n

Theorem 26.12 (Ravinder's squeeze; U5 from the squeeze). *If M satisfies the squeeze inputs, then $M_{0,m} =$ fusionF$_m$ for every m, and* fusionF *has nonnegative coefficients. If moreover every module with the local Weyl and fusion relations for $m\theta$ is bounded degreewise by $M_{0,m}$, then U5 holds for* fusionF. *Finally U5 for* fusionF *follows from the universal bound, U7 for all n, (nonneg), (R) and (T).*

Lean. `VOA.DeligneAudit.A2.squeeze`, `VOA.DeligneAudit.A2.fusionF_coeff_nonneg`, `VOA.DeligneAudit.A2.fusionModuleBound_of_squeeze`, `VOA.DeligneAudit.A2.dem_le_of_realization`, `VOA.DeligneAudit.A2.fusionModuleBound_of_realizations`
File `VOA/DeligneAudit/DeligneFusionSqueeze.lean`.

```
theorem squeeze (h : FusionSqueezeInputs M) (m : ℕ) : M 0 m = fusionF m := by

theorem fusionF_coeff_nonneg (h : FusionSqueezeInputs M) (m N : ℕ) :
    0 ≤ coeff N (fusionF m) := by

theorem fusionModuleBound_of_squeeze (h : FusionSqueezeInputs M)
    (huniv : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V]
      (ρ : CurrentModule (bracketTable) V) (w : V), LocalWeylRel ρ w m →
        ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ coeff b (M 0 m)) :
    FusionModuleBound fusionF := by
```

File `VOA/DeligneAudit/DeligneFusionDemazure.lean`.

```
theorem dem_le_of_realization (hU : MabUniversalBound M) {n : ℕ} (hD : DemazureRealization
 ↪  n) :
    CoeffLe (demazureHilbSeries n) (M n 0) := by

theorem fusionModuleBound_of_realizations (hU : MabUniversalBound M)
    (hD : ∀ n, DemazureRealization n) (hnonneg : ∀ a b N, 0 ≤ coeff N (M a b))
    (hrec : ∀ a b, CoeffLe (M (a + 1) b) (M a (b + 1) + X ^ (2 * a + b + 1) * M a b))
    (htot : ∀ m B, ∑ N ∈ Finset.range B, coeff N (M 0 m) ≤ 8 ^ m) :
    FusionModuleBound fusionF :=
```

Hypotheses / status. Squeeze inputs; proved

Definition 26.13 (The universal modules $U_{a,b}$ and their characters). Let FreeW be the vector space with basis the loop words (lists of letters $(c, r) \in \{0, \dots, 7\} \times \mathbb{N}$). Let $J_{a,b} \subseteq$ FreeW be spanned by: all *commutation instances* $p\,xy\,s - p\,yx\,s - \sum_e f(x,y,e)\,p\,(e, r_x + r_y)\,s$ (for prefixes p, suffixes s); all words $p\,t$ with t a relation tail of $M_{a,b}$ (a single raising letter, a positive-grade Cartan letter, $f_i(0)^{a+b+1}$, $f_1(a+b)$, $f_2(a+b)$, $f_\theta(2a+b)$); and all $p\,h(0) - (a+b)\,p$. Then $U_{a,b} =$ FreeW$/J_{a,b}$, with $x_c(r)$ acting by prepending the letter, and generator $w_U =$ [empty word]. Its character is $M^{\mathrm{univ}}_{a,b}(q) = \sum_N \dim \mathrm{cyc}(w_U)_N\, q^N$ (`Mchar a b`).

Lean. VOA.DeligneAudit.SectorWeyl.Ltr, VOA.DeligneAudit.SectorWeyl.FreeW, VOA.DeligneAudit.SectorWeyl.commInst, VOA.DeligneAudit.SectorWeyl.relTail, VOA.DeligneAudit.SectorWeyl.relGen, VOA.DeligneAudit.SectorWeyl.Jrel, VOA.DeligneAudit.SectorWeyl.UMod, VOA.DeligneAudit.SectorWeyl.univRep, VOA.DeligneAudit.SectorWeyl.wU, VOA.DeligneAudit.SectorWeyl.Mchar, VOA.DeligneAudit.SectorWeyl.univ_mabRel
File VOA/DeligneAudit/DeligneFusionUniv.lean.

```
abbrev Ltr := Fin 8 × ℕ

abbrev FreeW := List Ltr →₀ ℂ

def commInst (p s : List Ltr) (x y : Ltr) : FreeW :=
  sw (p ++ x :: y :: s) - sw (p ++ y :: x :: s) -
    ∑ e, bracketTable x.1 y.1 e • sw (p ++ (e, x.2 + y.2) :: s)

def relTail (a b : ℕ) : Set (List Ltr) :=
  {t | (∃ e r, e.val < 3 ∧ t = [(e, r)]) ∨ (∃ h r, 6 ≤ h.val ∧ 1 ≤ r ∧ t = [(h, r)]) ∨
    t = List.replicate (a + b + 1) (3, 0) ∨ t = List.replicate (a + b + 1) (4, 0) ∨
    t = [(3, a + b)] ∨ t = [(4, a + b)] ∨ t = [(5, 2 * a + b)]}

def relGen (a b : ℕ) : Set FreeW :=
  {g | ∃ p : List Ltr, (∃ s x y, g = commInst p s x y) ∨ (∃ t ∈ relTail a b, g = sw (p ++
   ↪  t)) ∨
    (∃ h : Fin 8, 6 ≤ h.val ∧ g = sw (p ++ [(h, 0)]) - ((a + b : ℕ) : ℂ) • sw p)}

def Jrel (a b : ℕ) : Submodule ℂ FreeW := Submodule.span ℂ (relGen a b)

abbrev UMod (a b : ℕ) := FreeW ⧸ Jrel a b

def univRep (a b : ℕ) : CurrentModule (bracketTable) (UMod a b) where
  op c r := opU a b (c, r)
  comm c d r s v := by

def wU (a b : ℕ) : UMod a b := Submodule.Quotient.mk (sw [])

def Mchar (a b : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (Module.finrank ℂ (cycDeg (univRep a b) (wU a b) N) : ℚ)
```

```
theorem univ_mabRel (a b : ℕ) : MabRel (univRep a b) (wU a b) a b where
```

Hypotheses / status. None; proved

Theorem 26.14 (The exact-sequence step: quotient and rank–nullity). *(i) If w satisfies the relations of $M_{a+1,b}$ and S is an invariant subspace containing $f_\theta(2a+b+1)w$, then the image of w in V/S satisfies the relations of $M_{a,b+1}$. (ii) Assume the kernel relation (KL). Then for every degree N, with $s = 2a+b+1$,*

$$\dim \mathrm{cyc}_{U_{a+1,b}}(w_U)_N \;\le\; \dim \mathrm{cyc}_{U_{a,b+1}}(w_U)_N + [s \le N]\, \dim \mathrm{cyc}_{U_{a,b}}(w_U)_{N-s},$$

provided the two right-hand pieces are finite-dimensional, and then the left piece is finite-dimensional too.

Lean. VOA.DeligneAudit.SectorWeyl.mabRel_quot, VOA.DeligneAudit.SectorWeyl.vK, VOA.DeligneAudit.SectorWeyl.fd_step, VOA.DeligneAudit.SectorWeyl.finrank_step
File VOA/DeligneAudit/DeligneFusionGrade.lean.

```
theorem mabRel_quot {w : V} {a b : ℕ} (h : MabRel ρ w (a + 1) b)
    (hv : ρ.op 5 (2 * a + b + 1) w ∈ S) : MabRel (quotRep ρ S hS) (S.mkQ w) a (b + 1) where
```

```
def vK : UMod (a + 1) b := (univRep (a + 1) b).op 5 (2 * a + b + 1) (wU (a + 1) b)

theorem fd_step (hR : RavinderKernelRel) (N : ℕ)
    [FiniteDimensional ℂ (cycDeg (univRep a (b + 1)) (wU a (b + 1)) N)]
    [FiniteDimensional ℂ (cycDeg (univRep a b) (wU a b) (N - (2 * a + b + 1)))] :
    FiniteDimensional ℂ (cycDeg (univRep (a + 1) b) (wU (a + 1) b) N) := by

theorem finrank_step (hR : RavinderKernelRel) (N : ℕ)
    [FiniteDimensional ℂ (cycDeg (univRep a (b + 1)) (wU a (b + 1)) N)]
    [FiniteDimensional ℂ (cycDeg (univRep a b) (wU a b) (N - (2 * a + b + 1)))] :
    Module.finrank ℂ (cycDeg (univRep (a + 1) b) (wU (a + 1) b) N) ≤
      Module.finrank ℂ (cycDeg (univRep a (b + 1)) (wU a (b + 1)) N) +
        if 2 * a + b + 1 ≤ N then
          Module.finrank ℂ (cycDeg (univRep a b) (wU a b) (N - (2 * a + b + 1))) else 0 :=
           ↪ by
```

Hypotheses / status. (KL), which is proved

Theorem 26.15 (The A_2 assembly). *(1)* Structural statements. *Assume (TB) and (KL). Then all degree pieces of all $U_{a,b}$ are finite-dimensional; if moreover U7 holds for every n, the universal characters* `Mchar` *satisfy the squeeze inputs. (2)* Corollary. *If* `FusionTotalDim` *holds and U7 holds for every n, then $L_{-3/2}(\mathfrak{sl}_3)$ is classically free with kernel (I_2^{ns}). ((KL) is a theorem, Section 26.7, and (TB) follows from* `FusionTotalDim` *by Theorem 26.7, so neither appears as a hypothesis of (2).)*

Lean. `VOA.DeligneAudit.A2.univ_fd`, `VOA.DeligneAudit.A2.fusionSqueezeInputs_Mchar`, `VOA.DeligneAudit.A2.classicallyFree_of_totalBound`, `VOA.DeligneAudit.A2.classicallyFree_of_totalDim`
File `VOA/DeligneAudit/DeligneFusionAssembly.lean`.

```
theorem univ_fd (hF : FusionTotalBound) (hR : RavinderKernelRel) :
    ∀ a b N, FiniteDimensional ℂ (cycDeg (univRep a b) (SectorWeyl.wU a b) N)

theorem fusionSqueezeInputs_Mchar (hF : FusionTotalBound) (hR : RavinderKernelRel)
    (hD : ∀ n, DemazureRealization n) : FusionSqueezeInputs Mchar where

theorem classicallyFree_of_totalBound (hF : FusionTotalBound) (hD : ∀ n,
 ↪  DemazureRealization n) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=

theorem classicallyFree_of_totalDim (hF : FusionTotalDim) (hD : ∀ n, DemazureRealization n)
 ↪  :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
```

Hypotheses / status. `FusionTotalDim`, U7

Theorem 26.16 (Heisenberg nilpotency; Garland's identity with $P(u) = 1$). *(1) Let X, Y, Z be operators with $[X, Y] = Z$ and Z commuting with X and Y. If $X^{p+1}w = Y^{p+1}w = 0$ then $Z^{2p+1}w = 0$. (2) Let E_1, H_k $(k \geq 1)$, F_k $(k \geq 0)$ be operators with $[E_1, F_k] = H_{k+1}$, $[H_{m+1}, F_k] = -2F_{m+1+k}$ and $[F_j, F_k] = 0$, and let $E_1 w = 0$, $H_{m+1}w = 0$ for all m. If $F_0^N w = 0$, then for $r \geq 1$ and $r + s \geq N$,*

$$[u^s]\, F(u)^r\, w = \sum_{k_1 + \cdots + k_r = s} F_{k_1} \cdots F_{k_r}\, w = 0.$$

Lean. `VOA.DeligneAudit.Garland.heis_nilpotent`, `VOA.DeligneAudit.Garland.Fs`, `VOA.DeligneAudit.Garland.garland`
File `VOA/DeligneAudit/DeligneFusionGarland.lean`.

```
variable {X Y Z : Module.End ℂ V} (hXY : X * Y - Y * X = Z) (hXZ : X * Z = Z * X)
  (hYZ : Y * Z = Z * Y)

theorem heis_nilpotent (hXZ : X * Z = Z * X) {p : ℕ} {w : V} (hX : (X ^ (p + 1)) w = 0)
    (hY : (Y ^ (p + 1)) w = 0) : (Z ^ (2 * p + 1)) w = 0 := by

def Fs : PowerSeries (Module.End ℂ V) := PowerSeries.mk F


variable {E1 H F} (hEF : ∀ k, E1 * F k - F k * E1 = H (k + 1))
  (hHF : ∀ m k, H (m + 1) * F k - F k * H (m + 1) = (-2 : ℂ) • F (m + 1 + k))

theorem garland (r : ℕ) (hr : 1 ≤ r) {N : ℕ} (hN : (F 0 ^ N) w = 0) (s : ℕ) (hs : N ≤ r +
↪  s) :
    (coeff s (Fs F ^ r) : Module.End ℂ V) w = 0 := by
```

Hypotheses / status. Five `included` hyps

Theorem 26.17 (Ravinder's kernel lemma for $\mathfrak{sl}_3$). *Let w satisfy the relations of $M_{a,b}$. Then $f_\theta(0)^{2(a+b)+1}w = 0$ and $\sum_{i+j=s} f_\theta(i)f_\theta(j)w = 0$ for $s+1 \geq 2(a+b)$. If $a \geq 1$, the kernel vector $v = f_\theta(2a+b-1)w$ satisfies $f_1(a-1+b)v = f_2(a-1+b)v = 0$ and $f_\theta(2(a-1)+b)v = 0$, and therefore all relations of $M_{a-1,b}$.*

Lean. `VOA.DeligneAudit.SectorWeyl.RavinderKernelTrunc`, `VOA.DeligneAudit.SectorWeyl.ravinderKernelRel_of_trunc`, `VOA.DeligneAudit.SectorWeyl.ftheta_pow`, `VOA.DeligneAudit.SectorWeyl.theta_garland`, `VOA.DeligneAudit.SectorWeyl.ravinderKernelTrunc`, `VOA.DeligneAudit.SectorWeyl.ravinderKernelRel`
File `VOA/DeligneAudit/DeligneFusionKernel.lean`.

```
def RavinderKernelTrunc : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
  ↪  (w : V),
    1 ≤ a → MabRel ρ w a b →
      ρ.op 3 (a - 1 + b) (ρ.op 5 (2 * a + b - 1) w) = 0 ∧
      ρ.op 4 (a - 1 + b) (ρ.op 5 (2 * a + b - 1) w) = 0 ∧
      ρ.op 5 (2 * (a - 1) + b) (ρ.op 5 (2 * a + b - 1) w) = 0

theorem ravinderKernelRel_of_trunc (hT : RavinderKernelTrunc) : RavinderKernelRel := by
```

File `VOA/DeligneAudit/DeligneFusionGarlandSl3.lean`.

```
theorem ftheta_pow (h : MabRel ρ w a b) : ((ρ.op 5 0) ^ (2 * (a + b) + 1)) w = 0 := by

theorem theta_garland (h : MabRel ρ w a b) (s : ℕ) (hs : 2 * (a + b) ≤ s + 1) :
    ∑ x ∈ Finset.antidiagonal s, ρ.op 5 x.1 (ρ.op 5 x.2 w) = 0 := by

theorem ravinderKernelTrunc : RavinderKernelTrunc := by

theorem ravinderKernelRel : RavinderKernelRel := ravinderKernelRel_of_trunc
↪  ravinderKernelTrunc
```

Hypotheses / status. None; proved

Theorem 26.18 (Chari–Pressley, Prop. 6.1). *In $R = \mathbb{C}[z_0, z_1, \dots]$ let $Z(u) = \sum_i z_i u^i$, $x(r,s) = [u^s]Z(u)^r$ and $J_m = (x(r,s) : r \geq 1,\ r+s \geq m+1)$. Then every $p \in R$ is a combination of* box monomials $z_{i_1} \cdots z_{i_\ell}$ *with $i_j + \ell \leq m$, plus an element of J_m. Consequently, in every $\hat{\mathfrak{sl}}_2[t]$-module, if w is killed by $e \otimes t$ and by $h(k)$, $k \geq 1$, and $f(0)^{m+1}w = 0$, then every polynomial in the $f(k)$ applied to w is a combination of box monomials applied to w.*

Lean. `VOA.DeligneAudit.Sl2Weyl.Zs`, `VOA.DeligneAudit.Sl2Weyl.xx`, `VOA.DeligneAudit.Sl2Weyl.J`, `VOA.DeligneAudit.Sl2Weyl.BoxP`, `VOA.DeligneAudit.Sl2Weyl.Bk`, `VOA.DeligneAudit.Sl2Weyl.T`, `VOA.DeligneAudit.Sl2Weyl.box_span`, `VOA.DeligneAudit.Sl2Weyl.weyl_span_box`
File `VOA/DeligneAudit/DeligneFusionSl2.lean`.

```
def Zs : PowerSeries R := PowerSeries.mk fun i => z i

def xx (r s : ℕ) : R := PowerSeries.coeff s (Zs ^ r)

def J (m : ℕ) : Ideal R := Ideal.span {p | ∃ r s, 1 ≤ r ∧ m + 1 ≤ r + s ∧ p = xx r s}

def BoxP (m : ℕ) (f : ℕ →₀ ℕ) : Prop := ∀ i ∈ f.support, i + Finsupp.degree f ≤ m

def Bk (m k : ℕ) : Submodule ℂ R :=
  Submodule.span ℂ {p | ∃ f, BoxP m f ∧ f 0 ≤ k ∧ p = MvPolynomial.monomial f 1}

def T (m k : ℕ) : Submodule ℂ R := Bk m k ⊔ (J m).restrictScalars ℂ

theorem box_span : ∀ m (p : R), p ∈ T m m := by
```

File `VOA/DeligneAudit/DeligneFusionSl2Module.lean`.

```
theorem weyl_span_box {E1 : Module.End ℂ V} {H : ℕ → Module.End ℂ V}
    (hEF : ∀ k, E1 * F k - F k * E1 = H (k + 1))
    (hHF : ∀ m k, H (m + 1) * F k - F k * H (m + 1) = (-2 : ℂ) • F (m + 1 + k))
    {y : V} (hE : E1 y = 0) (hH : ∀ m, H (m + 1) y = 0) (m : ℕ) (hN : (F 0 ^ (m + 1)) y =
    ↪ 0)
    (p : R) :
    evAlg F hFF p y ∈
      Submodule.span ℂ {v | ∃ f, BoxP m f ∧ v = evAlg F hFF (MvPolynomial.monomial f 1) y}
      ↪ :=
```

Hypotheses / status. None; proved

Theorem 26.20 (Chari–Loktev spanning and count for $\hat{\mathfrak{sl}}_3$). *For every N,* `CLSpanning N` *and* `CLCount N` *hold. Consequently, in every module with the $M_{N,0}$ relations,* $\dim \mathrm{cyc}(w)_d \leq [q^d]\mathcal{D}_N$ *for every d.*

Lean. `VOA.DeligneAudit.CLAssembly.clSpanning`, `VOA.DeligneAudit.CLAssembly.finrank_cycDeg_le`, `VOA.DeligneAudit.CL.clCount`
File `VOA/DeligneAudit/DeligneFusionCLAssembly.lean`.

```
theorem clSpanning (N : ℕ) : CLSpanning N := by

theorem finrank_cycDeg_le {V : Type} [AddCommGroup V] [Module ℂ V] {ρ : CurrentModule
↪ (bracketTable) V}
    {w : V} {N : ℕ} (hR : MabRel ρ w N 0) (d : ℕ) :
    Module.finrank ℂ (cycDeg ρ w d) ≤ A2.demazureHilb N d := by
```

File `VOA/DeligneAudit/DeligneFusionCLCount.lean`.

```
theorem clCount (N : ℕ) : CLCount N := by
```

Hypotheses / status. None; proved

Definition 26.23 (Filtered relations; the evaluation model). Let T be a current module (no grading) and $w \in T$; let $\Phi_{<d}$ be the span of words of degree $< d$ applied to w and $\Phi_{\le d} = \Phi_{<d+1}$. Say w satisfies the *filtered $M_{N,0}$ relations* if the raising and lowering relations hold exactly, $h(r)w \in \mathbb{C}w$, $h(0)w = Nw$, and the truncations hold modulo lower filtration: $f_i(N)w \in \Phi_{<N}$, $f_\theta(2N)w \in \Phi_{<2N}$. The *evaluation model* is $T_N = \mathbb{C}^{\{1,2,3\}^{2N}}$, the tensor product of N copies of the standard representation $\mathbb{C}^3$ and N copies of its dual, at the distinct points $z = 0, 1, \dots, 2N-1$, with $x(k)$ acting by $\sum_i z_i^k\, x^{(i)}$ ($x^{(i)}$ the action on factor i).

Lean. `VOA.DeligneAudit.GrFilt.Fbelow`, `VOA.DeligneAudit.GrFilt.Fle`, `VOA.DeligneAudit.GrFilt.FiltRel`, `VOA.DeligneAudit.EvalModel.matT`, `VOA.DeligneAudit.EvalModel.Site`, `VOA.DeligneAudit.EvalModel.Cfg`, `VOA.DeligneAudit.EvalModel.TT`, `VOA.DeligneAudit.EvalModel.R`, `VOA.DeligneAudit.EvalModel.z`, `VOA.DeligneAudit.EvalModel.loc`, `VOA.DeligneAudit.EvalModel.op`
File `VOA/DeligneAudit/DeligneFusionGrFilt.lean`.

```
def Fbelow (d : ℕ) : Submodule ℂ T :=
  Submodule.span ℂ {v | ∃ l : List (Fin 8 × ℕ), SectorGraded.wordDeg l < d ∧ v = wordOp ρ l
    ↪ w}

abbrev Fle (d : ℕ) : Submodule ℂ T := Fbelow ρ w (d + 1)

structure FiltRel (N : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, ρ.op h r w ∈ ℂ • w
  cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = (N : ℂ) • w
  lowering_f1 : (ρ.op 3 0)^[N + 1] w = 0
  lowering_f2 : (ρ.op 4 0)^[N + 1] w = 0
  trunc_f1 : ρ.op 3 N w ∈ Fbelow ρ w N
  trunc_f2 : ρ.op 4 N w ∈ Fbelow ρ w N
  trunc_theta : ρ.op 5 (2 * N) w ∈ Fbelow ρ w (2 * N)
```

File `VOA/DeligneAudit/DeligneFusionEvalModel.lean`.

```
def matT : Fin 8 → Matrix (Fin 3) (Fin 3) ℂ :=
  ![Matrix.single 0 1 1, Matrix.single 1 2 1, Matrix.single 0 2 1, Matrix.single 1 0 1,
    Matrix.single 2 1 1, Matrix.single 2 0 1, Matrix.single 0 0 1 - Matrix.single 1 1 1,
    Matrix.single 1 1 1 - Matrix.single 2 2 1]

abbrev Site := Fin N ⊕ Fin N
abbrev Cfg := Site N → Fin 3
abbrev TT := Cfg N → ℂ

def R : Site N → Fin 8 → Matrix (Fin 3) (Fin 3) ℂ
  | .inl _, c => matT c
  | .inr _, c => -(Matrix.transpose (matT c))

def z : Site N → ℂ
  | .inl i => ((i : ℕ) : ℂ)
  | .inr i => ((N + i : ℕ) : ℂ)

def loc (i : Site N) (X : Matrix (Fin 3) (Fin 3) ℂ) : Module.End ℂ (TT N) where
  toFun f γ := ∑ k, X (γ i) k * f (Function.update γ i k)
  map_add' f g := by ext γ; simp [mul_add, Finset.sum_add_distrib]
  map_smul' c f := by ext γ; simp [Finset.mul_sum, mul_left_comm]

def op (c : Fin 8) (k : ℕ) : Module.End ℂ (TT N) := ∑ i, (z N i) ^ k • loc i (R N i c)
```

Theorem 26.24 (U7 for A_2). *(1) Let T be a* finite-dimensional *current module. If $w \in T$ satisfies the filtered $M_{N,0}$ relations and $\dim \Phi_{\le D} \ge 9^N$ for some D, then* `DemazureRealization N` *holds. (2) The evaluation model satisfies the hypotheses of (1). Hence* `DemazureRealization N` *holds for every N.*

Lean. `VOA.DeligneAudit.GrFilt.demazureRealization_of_filt`, `VOA.DeligneAudit.EvalModel.demazureRealization_all`
File `VOA/DeligneAudit/DeligneFusionGrFilt.lean`.

```
theorem demazureRealization_of_filt {N : ℕ} (h : FiltRel ρ w N) (D : ℕ)
    (hD : 9 ^ N ≤ Module.finrank ℂ (Fle ρ w D)) : A2.DemazureRealization N := by
```

File `VOA/DeligneAudit/DeligneFusionEvalModel.lean`.

```
theorem demazureRealization_all (N : ℕ) : A2.DemazureRealization N := by
```

Hypotheses / status. None; proved

Theorem 26.27 (Each refinement implies the total bound). *(1)* `RavinderKernelLower` *implies $\sum_{N<B} \dim \mathrm{cyc}_{U_{a,b}}(w_U)_N \le 9^a 8^b$ for all a, b, B, hence* `FusionTotalDim`*. (2) In a module with the relations of $M_{0,N}$, every mode $x(k)$ with $k \ge N$ kills the whole cyclic module. (3)* `IdealLower` *implies* `FusionTotalDim`*.*

Lean. `VOA.DeligneAudit.A2.uTot_le`, `VOA.DeligneAudit.A2.fusionTotalDim_of_kernelLower`, `VOA.DeligneAudit.A2.trunc_all`, `VOA.DeligneAudit.A2.fusionTotalDim_of_idealLower`
File `VOA/DeligneAudit/DeligneFusionKernelLower.lean`.

```
theorem uTot_le (hK : RavinderKernelLower) : ∀ b a B, uTot a b B ≤ 9 ^ a * 8 ^ b

theorem fusionTotalDim_of_kernelLower (hK : RavinderKernelLower) : FusionTotalDim := by
```

File `VOA/DeligneAudit/DeligneFusionIdeal.lean`.

```
theorem trunc_all {V : Type} [AddCommGroup V] [Module ℂ V] {ρ : CurrentModule
 ↪ (bracketTable) V}
    {w : V} {N : ℕ} (h : MabRel ρ w 0 N) :
    ∀ (l : List (Fin 8 × ℕ)) (c : Fin 8) (k : ℕ), N ≤ k → ρ.op c k (wordOp ρ l w) = 0

theorem fusionTotalDim_of_idealLower (hI : IdealLower) : SectorWeyl.FusionTotalDim := by
```

Hypotheses / status. The named `Prop`; proved

Theorem 26.28 (The A_2 case: final forms in this chapter). *$L_{-3/2}(\mathfrak{sl}_3)$ is classically free for its explicit presentation, with kernel (I_2^{ns}), assuming either* `RavinderKernelLower` *or* `FusionTotalDim`*.*

Lean. `VOA.DeligneAudit.A2.classicallyFree_of_kernelLower'`, `VOA.DeligneAudit.A2.classicallyFree_of_totalDim'`
File `VOA/DeligneAudit/DeligneFusionU7.lean`.

```
theorem classicallyFree_of_kernelLower' (hK : RavinderKernelLower) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=

theorem classicallyFree_of_totalDim' (hF : SectorWeyl.FusionTotalDim) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
```

Hypotheses / status. `FusionTotalDim` (proved, so unconditional in Ch. 27), resp. `RavinderKernelLower`

Theorem 26.30 (The A_2 total bound).

1. *The two base families hold:* `CVUniv.sharpRectangularBases` *and* `CVUniv.residualTheta3Bound`.
2. *If both base families hold, then* `FusionTotalDim`*: for every m, every current module and every vector w satisfying the local Weyl and fusion relations for $m\theta$, $\sum_{b<B} \dim \mathrm{cyc}(w)_b \le 8^m$ for all B.*
3. *Hence* `FusionTotalDim` *holds, with no hypothesis.*

Lean. `VOA.DeligneAudit.A2.CVUniv.sharpRectangularBases`, `VOA.DeligneAudit.A2.CVUniv.residualTheta3Bound`, `VOA.DeligneAudit.A2.CVUniv.fusionTotalDim_of_cvBases`, `VOA.DeligneAudit.A2.CVUniv.fusionTotalDim`
Files `VOA/DeligneAudit/DeligneFusionCVRectFilteredRectangular.lean` (1, first family), `VOA/DeligneAudit/DeligneFusionCVResidualThetaBoundFinal.lean` (1, second family), `VOA/DeligneAudit/DeligneFusionCVTotal.lean` (2) and `VOA/DeligneAudit/A2Unconditional.lean` (3).

```
theorem sharpRectangularBases : SharpRectangularBases := by

theorem residualTheta3Bound : ResidualTheta3Bound := by

theorem fusionTotalDim_of_cvBases (hR : SharpRectangularBases)
    (hθ : ResidualTheta3Bound) : FusionTotalDim :=

theorem CVUniv.fusionTotalDim : SectorWeyl.FusionTotalDim :=
  CVUniv.fusionTotalDim_of_cvBases CVUniv.sharpRectangularBases CVUniv.residualTheta3Bound
```

Hypotheses / status. none; proved (record)

Theorem 26.41 ($e^{N\theta}$ satisfies Ravinder's $M_{N,0}$ relations; lattice U7). *(1) For lattice realization data with $\langle\theta,\theta\rangle = 2$ and every N, $e^{N\theta}$ satisfies* `ravFam S N 0`*. (2) Hence* `LatDim Dem N` *implies* `DemazureRealizationG S (ravFam S) Dem N`*. (3) Realization data exist for D_4, E_6, E_7, E_8 (*`d4LatData`*,* `e6LatData`*,* `e7LatData`*,* `e8LatData`*, with $\langle\theta,\theta\rangle = 2$ proved as* `d4_hθ`*, …,* `e8_hθ`*); for these types the relations half of U7 is unconditional. (4) (Corollary for D_4.) If* `FusionTotalBoundG D4sector` *holds and* `D4LatticeDim N` *holds for every N, then D_4 is classically free for its explicit presentation, with the expected kernel (*`d4_classicallyFree_of_totalBound_latDim`*). The E_6, E_7 analogues take additional "star" inputs (Chapter 25). There are no lattice data for G_2, F_4: the Frenkel–Kac construction realizes level-one currents of* simply laced *algebras only, since it needs all roots to have norm 2 in an even lattice.*

Lean. `VOA.DeligneAudit.LatticeRealization.LatData.sat`, `VOA.DeligneAudit.LatticeRealization.LatData.demazureRealizationG`, `VOA.DeligneAudit.LatticeRealization.D4.d4_lattice_sat`, `VOA.DeligneAudit.LatticeRealization.D4.D4LatticeDim`, `VOA.DeligneAudit.LatticeRealization.D4.d4_classicallyFree_of_totalBound_latDim`, `VOA.DeligneAudit.LatticeRealization.E8.e8_lattice_sat`, `VOA.DeligneAudit.LatticeRealization.E8.E8LatticeDim`, `VOA.DeligneAudit.LatticeRealization.E8.e8_demazureRealization`
File `VOA/DeligneAudit/LatticeRealizationGeneric.lean`.

```
theorem sat (hθ : L.form D.θ D.θ = 2) (N : ℕ) : Sat S.f D.ρ (D.vN N) (ravFam S N 0) := by

theorem demazureRealizationG (hθ : L.form D.θ D.θ = 2) {Dem : ℕ → PowerSeries ℚ} {N : ℕ}
    (hdim : D.LatDim Dem N) : DemazureRealizationG S (ravFam S) Dem N :=
```

File `VOA/DeligneAudit/LatticeRealizationD4.lean`.

```
theorem d4_lattice_sat (N : ℕ) : Sat D4sector.f d4LatData.ρ (d4LatData.vN N) (ravFam
 ↪  D4sector N 0) :=

def D4LatticeDim (N : ℕ) : Prop :=
  d4LatData.LatDim DeligneCharacter.D4Wp.hilbSeriesZ N

theorem d4_classicallyFree_of_totalBound_latDim (hF : FusionTotalBoundG D4sector)
    (hdim : ∀ N, D4LatticeDim N) :
    VOA.DeligneHigherRankConditionalCapstone.CaseClassicallyFree .D4 D4Lie.W4
     ↪  D4Lie.explicitPresentation4 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass D4Lie.W4 1 ⟨D4Lie.explicitPresentation4.gen i,
          D4Lie.explicitPresentation4.gen_weight_one i⟩).toRingHom =
        D4Lie.explicitPresentation4.ideal :=
```

File `VOA/DeligneAudit/LatticeRealizationE8.lean`.

```
theorem e8_lattice_sat (N : ℕ) : Sat E8sector.f e8LatData.ρ (e8LatData.vN N) (ravFam
 ↪  E8sector N 0) :=

def E8LatticeDim (N : ℕ) : Prop :=
  e8LatData.LatDim (DeligneCarrier.hilbSeries DeligneCharacter.e8DemazureHilb) N

theorem e8_demazureRealization {N : ℕ} (h : E8LatticeDim N) :
    DemazureRealizationG E8sector (ravFam E8sector) (DeligneCarrier.hilbSeries
     ↪  DeligneCharacter.e8DemazureHilb) N :=
```

Hypotheses / status. `FusionTotalBoundG`, `D4LatticeDim`

Chapter 27

The Deligne series IV: the final theorems and the named inputs

7 of the chapter's 21 numbered statements: its headline results and every definition they use.

Lemma 27.6 (The fusion module is a truncated Weyl module). *Let $m \geq 0$ and let w be a vector of a $\mathfrak{g}[t]$-module satisfying the local Weyl relations of $W_{\mathrm{loc}}(m\theta)$, together with $x(m)w = 0$ for every x in a set $\mathcal{S}$ of negative root vectors with $\{f_\alpha : \langle \alpha, \theta^\vee \rangle = 1\} \cup \{f_\theta\} \subseteq \mathcal{S}$. Then $(\mathfrak{g} \otimes t^m\mathbb{C}[t])$ annihilates the cyclic module $U(\mathfrak{g}[t])w$. Consequently the universal module with these relations, $\mathcal{F}_m = M_{0,m}$, is the* truncated Weyl module *$W(m\theta, m)$ of Kus–Littelmann (Definition 4.1): the quotient of $W_{\mathrm{loc}}(m\theta)$ on which $\mathfrak{g} \otimes t^m\mathbb{C}[t]$ acts by zero. In particular $M_{0,m}$ does not depend on the choice of $\mathcal{S}$ within these bounds.*

(The chapter quotes no Lean declaration for this statement; see book p. 1463.)
Hypotheses / status. On paper; no Lean statement

Definition 27.7 (Input R). Let S be sector data, spec a family of relation specifications indexed by $\mathbb{N} \times \mathbb{N}$, and $\mathrm{Dem} : \mathbb{N} \to \mathbb{Q}[[q]]$ a family of power series. For $j \in \mathbb{N}$, a *Demazure realization at j* is a current module V and a vector $w \in V$ satisfying $\mathrm{spec}(j, 0)$ such that, for every degree N,

$$[q^N]\,\mathrm{Dem}_j \;\leq\; \dim\big(\text{degree-}N \text{ part of the cyclic span of } w\big).$$

In the final theorems spec $=$ `ravFam S`, so $\mathrm{spec}(j, 0)$ are the relations of $M_{j,0}$, and Dem_j is the type's Demazure table from Chapter 25. Input R is "a Demazure realization exists at every j".

Lean. `VOA.DeligneAudit.UnivG.DemazureRealizationG`, `VOA.DeligneAudit.DeligneCarrier.hilbSeries`, `VOA.DeligneAudit.DeligneCarrier.g2HilbInt`, `VOA.DeligneAudit.DeligneCarrier.g2HilbSeriesZ`
File `VOA/DeligneAudit/DeligneFusionUnivGSector.lean`.

```
def DemazureRealizationG (spec : ℕ → ℕ → RelSpec S.n) (Dem : ℕ → PowerSeries ℚ) (n : ℕ) :
 ↪  Prop :=
  ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule S.f V) (w : V),
    Sat S.f ρ w (spec n 0) ∧ ∀ N, coeff N (Dem n) ≤ (Module.finrank ℂ (cycDeg ρ w N) : ℚ)
```

File `VOA/DeligneAudit/DeligneCarrierDemazure.lean`.

```
def hilbSeries (hilb : ℕ → ℕ → ℕ) (n : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (hilb n N : ℚ)
```

File `VOA/DeligneAudit/DeligneCharacterG2FusionZ.lean`.

```
def g2HilbInt (m : ℕ) (N : ℤ) : ℤ := hilbIntG cartanG2 thetaG2 wordG2 m N
```

```
def g2HilbSeriesZ (n : ℕ) : PowerSeries ℚ := PowerSeries.mk fun N => (g2HilbInt n N : ℚ)
```

Hypotheses / status. A_2, D_4: proved; others: **hypothesis**

Theorem 27.8 (Input R for D_4). *For every n there is a current module for the D_4 structure constants with a vector satisfying the relations* $\mathrm{rav}(n, 2n)$ *whose degree-N piece has dimension at least* $[q^N]$ *of the unclamped D_4 Demazure series, for every N.*

Lean. `VOA.DeligneAudit.D4Char.d4_demazureRealization`
File `VOA/DeligneAudit/D4CharDemazureRealization.lean`.

```
theorem d4_demazureRealization :
    ∀ n, DemazureRealizationG D4sector (ravFam D4sector) D4Wp.hilbSeriesZ n :=
```

Hypotheses / status. Unconditional; aggregate build green 2026-11-03; axioms standard

Theorem 27.13 (A_2: classical freeness of $L_{-3/2}(\mathfrak{sl}_3)$). *Let $W = L_{-3/2}(\mathfrak{sl}_3)$ and let P be its explicit current C_2 presentation, with generators the eight currents and ideal (I_2^{ns}) generated by the eight quadrics $P_0, \dots, P_7$ of Definition 24.8. Then W is classically free for P, and* $\ker(\mathrm{Sym}\,\mathfrak{sl}_3 \to R_W) = (I_2^{\mathrm{ns}})$*; that is, $R_W = \mathrm{Sym}\,\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$ and $J_\infty R_W \to \mathrm{gr}^{\mathrm{Li}} W$ is an isomorphism. The same conclusion is also stated under the (unproved) hypothesis* `RavinderKernelLower`.

Lean. `VOA.DeligneAudit.A2.classicallyFree`, `VOA.DeligneAudit.DeligneSeries.A2_classicallyFree_final`, `VOA.DeligneAudit.DeligneSeries.A2_classicallyFree_final_S2`
File `VOA/DeligneAudit/A2Unconditional.lean`.

```
theorem classicallyFree :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
  classicallyFree_of_totalDim' CVUniv.fusionTotalDim
```

File `VOA/DeligneAudit/DeligneSeriesA2.lean`.

```
theorem A2_classicallyFree_final :
    CaseClassicallyFree .A2 VOA.DeligneAudit.A2.W VOA.DeligneAudit.A2.explicitPresentation
     ↪ ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.A2.W 1
         ↪ ⟨VOA.DeligneAudit.A2.explicitPresentation.gen i,
          VOA.DeligneAudit.A2.explicitPresentation.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.A2.I2ns :=
  VOA.DeligneAudit.A2.classicallyFree

/-- The older A₂ S2 interface, retained for reuse. -/
theorem A2_classicallyFree_final_S2 (hK : VOA.DeligneAudit.A2.RavinderKernelLower) :
    CaseClassicallyFree .A2 VOA.DeligneAudit.A2.W VOA.DeligneAudit.A2.explicitPresentation
     ↪ ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.A2.W 1
         ↪ ⟨VOA.DeligneAudit.A2.explicitPresentation.gen i,
          VOA.DeligneAudit.A2.explicitPresentation.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.A2.I2ns :=
  VOA.DeligneAudit.A2.classicallyFree_of_kernelLower' hK
```

Theorem 27.14 (D_4: the final theorem). *Let $W = L_{-2}(\mathfrak{so}_8)$, built from the certificate-basis Lie data, with its explicit presentation by* 105 *quadrics. Suppose Input T (*$\dim \mathcal{F}_m \le 28^m$*) and Input R for the unclamped D_4 Demazure series. Then W is classically free for its presentation and* ker *is the ideal generated by the* 105 *quadrics.*

Lean. `VOA.DeligneAudit.DeligneSeries.D4_classicallyFree_final`
File `VOA/DeligneAudit/DeligneSeries.lean`.

```
theorem D4_classicallyFree_final (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
 ↪  D4.D4sector)
    (hD : ∀ n, DemazureRealizationG D4.D4sector (ravFam D4.D4sector)
      VOA.DeligneAudit.DeligneCharacter.D4Wp.hilbSeriesZ n) :
    CaseClassicallyFree .D4 VOA.DeligneAudit.D4Lie.W4
     ↪  VOA.DeligneAudit.D4Lie.explicitPresentation4 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.D4Lie.W4 1
          ⟨VOA.DeligneAudit.D4Lie.explicitPresentation4.gen i,
            VOA.DeligneAudit.D4Lie.explicitPresentation4.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.D4Lie.explicitPresentation4.ideal :=
  d4_classicallyFree_of_totalBound (VOA.DeligneAudit.FiniteG.D4.d4_fusionTotalBoundG_of_dim
   ↪  hF) hD
```

Hypotheses / status. T, R

Theorem 27.15 (D_4: classical freeness from Input T alone). *Let $W = L_{-2}(\mathfrak{so}_8)$, built from the certificate-basis Lie data, with its explicit presentation by* 105 *quadrics. Suppose Input T (*$\dim \mathcal{F}_m \le 28^m$*). Then W is classically free for its presentation and* ker *is the ideal generated by the* 105 *quadrics.*

Lean. `VOA.DeligneAudit.DeligneSeries.D4_classicallyFree_of_inputT`
File `VOA/DeligneAudit/DeligneSeriesD4.lean`.

```
theorem D4_classicallyFree_of_inputT (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
 ↪  D4.D4sector) :
    CaseClassicallyFree .D4 VOA.DeligneAudit.D4Lie.W4
     ↪  VOA.DeligneAudit.D4Lie.explicitPresentation4 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.D4Lie.W4 1
          ⟨VOA.DeligneAudit.D4Lie.explicitPresentation4.gen i,
            VOA.DeligneAudit.D4Lie.explicitPresentation4.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.D4Lie.explicitPresentation4.ideal :=
```

Hypotheses / status. T only; aggregate build green 2026-11-03; axioms standard

Theorem 27.23 (G_2 and F_4 concrete headlines). *For $W = L_{-5/3}(\mathfrak{g}_2)$, classical freeness and* ker $=$ *the ideal of the listed quadrics follow from U5 for the clamped fusion series and Input S; or from U5 and the two halves of the star identity. For $W = L_{-5/2}(\mathfrak{f}_4)$ they follow from U5 and Input S.*

Lean. `VOA.DeligneAudit.G2Lie.g2_classicallyFree_concrete`, `VOA.DeligneAudit.G2Lie.g2_classicallyFree_concrete_of_fermionic`, `VOA.DeligneAudit.F4Lie.f4_classicallyFree_concrete'`, `VOA.DeligneAudit.G2Star.G2DemazureFermionic`, `VOA.DeligneAudit.G2Star.G2FermionicKW`
File `VOA/DeligneAudit/G2LieHeadline.lean`.

```
theorem g2_classicallyFree_concrete (hU5 : FusionModuleBoundG G2sector g2FusionF)
    (hB : G2StarBound) :
    CaseClassicallyFree .G2 g2W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass g2W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

```
theorem g2_classicallyFree_concrete_of_fermionic (hU5 : FusionModuleBoundG G2sector
 ↪  g2FusionF)
    (hF1 : G2Star.G2DemazureFermionic) (hF2 : G2Star.G2FermionicKW) :
    CaseClassicallyFree .G2 g2W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass g2W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

File `VOA/DeligneAudit/F4LieHeadline.lean`.

```
theorem f4_classicallyFree_concrete' (hU5 : FusionModuleBoundG F4sector f4FusionF)
    (hB : F4StarBound) :
    CaseClassicallyFree .F4 f4W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass f4W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

File `VOA/DeligneAudit/G2StarFermionic.lean`.

```
structure G2DemazureFermionic : Prop where
  eq : ∀ n N, g2DemazureHilb n N = g2FermHilb n N

/-- **Named (one `q`-series identity)**: the fermionic Demazure sum is the Kac–Wakimoto
 ↪  form. -/
structure G2FermionicKW : Prop where
  eq : ∀ N, ((demazureSumCoeff g2FermHilb N : ℕ) : ℤ) = coeff N g2KW
```

Hypotheses / status. U5, star input

Chapter 28

Graded unitarity through the BRST coset: from affine $\mathfrak{sl}_2$ at level $-4/3$ to Lee–Yang

17 of the chapter's 47 numbered statements: its headline results and every definition they use.

Definition 28.4 (The corrected filtration). A *corrected word* is a finite list of letters (g, m), where g is a current e, f, h or the Sugawara vector ω, and $m \in \mathbb{N}$ is a mode depth. The letter (g, m) acts by $g_{(-m-1)}$: for a current this is J^g_{-m-1}, for ω it is L_{-m-2}. The *corrected filtration* F_pV^k is the span of the states $g^1_{(-m_1-1)} \cdots g^\ell_{(-m_\ell-1)}\mathbf{1}$ with $\ell \le p$ letters, intersected with each conformal weight space. On a quotient V^k/I by a graded ideal the flag is the image. The flag on A is the image of the corrected flag of $V^{-4/3}$.

Lean. `VOA.AffineVOA.CorrectedR.Generator`, `VOA.AffineVOA.CorrectedR.generatorState`, `VOA.AffineVOA.CorrectedR.correctedWordState`, `VOA.AffineVOA.CorrectedR.correctedAffineRPiece`, `VOA.AffineVOA.CorrectedR.correctedAffineRQuotientFlag`, `VOA.LeeYangTransferAssembly.affineFlag`
File `VOA/Examples/AffineSl2GoodFiltration.lean` (a file of Chapter 9).

```
inductive Generator
  | current (a : Fin 3)
  | sugawara
  deriving DecidableEq

def generatorState (k : ℂ) (hk : k + 2 ≠ 0) : Generator → WakModule k
  | .current a => currentState k a
  | .sugawara => (affineSl2VOA_closed k hk).conformalVector

def correctedWordState (k : ℂ) (hk : k + 2 ≠ 0) :
    List (Generator × ℕ) → WakModule k
  | [] => (affineSl2VOA_closed k hk).vacuum
  | gr :: rest =>
      (affineSl2VOA_closed k hk).smul (generatorState k hk gr.1)
        (-(gr.2 : ℤ) - 1) (correctedWordState k hk rest)

def correctedAffineRPiece (k : ℂ) (hk : k + 2 ≠ 0) (p : ℕ) :
    Submodule ℂ (WakModule k) :=
  Submodule.span ℂ (Set.range (fun w : {w : List (Generator × ℕ) // w.length ≤ p} =>
    correctedWordState k hk w.1))

noncomputable def correctedAffineRQuotientFlag (k : ℂ) (hk : k + 2 ≠ 0)
    (I : Submodule ℂ (WakModule k))
```

```
    (hI : (affineSl2VOA_closed k hk).IsIdeal I)
    (hgr : QuotVOA.IsGradedIdeal (affineSl2VOA_closed k hk) I) :
    GradedFlag (QuotVOA.quotientVOA (affineSl2VOA_closed k hk) I hI hgr) :=
  (correctedAffineRFlag k hk).quotient I hI hgr
```

File `VOA/Examples/LeeYangTransferAssemblyAffine.lean`.

```
def affineFlag : GradedFlag AVOA :=
  CorrectedR.correctedAffineRQuotientFlag ALevel aNoncritical
    (QuotVOA.maxGradedIdeal (universal ALevel aNoncritical))
    (QuotVOA.maxGradedIdeal_isIdeal _)
    (QuotVOA.maxGradedIdeal_isGradedIdeal _)
```

Hypotheses / status. Definitions

Definition 28.5 (Compact conjugation). For real $k \neq -2$, the *compact conjugation* of $V^k(\mathfrak{sl}_2)$ is the unique anti-linear map ρ that fixes $\mathbf{1}$, commutes with vertex operators in the sense $\rho(u_{(n)}v) = \rho(u)_{(n)}\rho(v)$, and acts on the currents by $\rho(e) = -f$, $\rho(f) = -e$, $\rho(h) = -h$; that is, $J^a \mapsto -J^{a^*}$ with a^* the "adjoint colour". It is an involution, and it descends to every quotient by an ideal that it preserves; on A we use the descended map.

Lean. `VOA.AffineSl2BelowCritical.IsCompactRealStructure`, `VOA.AffineSl2BelowCritical.compactRealStructure`, `VOA.LeeYangTransferAssembly.affineRealStructure`, `VOA.LeeYangTransferAssembly.affineRealStructure_current`
File `VOA/Examples/AffineSl2BelowCriticalCompact.lean`.

```
structure IsCompactRealStructure (R : RealStructure (universal k hk)) : Prop where
  current : ∀ a : Fin 3, R.rho (currentState k a) = -currentState k (adjointColour a)
  parity_ne_zero : ∀ n : ℤ, R.parity n ≠ 0
```

File `VOA/Examples/AffineSl2BelowCriticalRealStructure.lean`.

```
def compactRealStructure : RealStructure (universal k hk) where
```

File `VOA/Examples/LeeYangTransferAssemblyAffine.lean`.

```
def affineRealStructure : RealStructure AVOA where

theorem affineRealStructure_current (a : Fin 3) :
    affineRealStructure.rho (simpleCurrent ALevel aNoncritical a) =
      -simpleCurrent ALevel aNoncritical (adjointColour a) := by
```

Hypotheses / status. Constructed

Definition 28.7 (The Lee–Yang datum). On the Lee–Yang side the form is the canonical normalized form `Bsimp` of $L(c, 0)$, the flag is the length flag, and the real structure is the partition-basis conjugation of Theorem 19.23, at $c = c_{2,5} = 1 - 12\lambda^2$ with the real background charge $\lambda = \lambda_2$ of Theorem 19.24.

Lean. `VOA.LeeYangTransferAssembly.affineForm`, `VOA.LeeYangTransferAssembly.leeYangRealStructure`
File `VOA/Examples/LeeYangTransferAssemblyAffine.lean`.

```
abbrev affineForm : InvariantForm AVOA := simpleForm ALevel aNoncritical

def leeYangRealStructure : RealStructure (VirSimple.virasoroSimple LeeYang.leeYangC) :=
  virasoroSimpleRealStructure
    ((VirGradedUnitarity.boundaryBackgroundCharge_spec 2).trans
      VirGradedUnitarity.boundaryCharge_two_eq_leeYangC)
    (VirGradedUnitarity.boundaryBackgroundCharge_fixed 2)
```

Hypotheses / status. Constructed

Theorem 28.8 (Affine graded unitarity implies Lee–Yang graded unitarity). *Suppose that $A = L_{-4/3}(\mathfrak{sl}_2)$, with its normalized canonical invariant form, the descended compact conjugation (Definition 28.5) and the corrected flag (Definition 28.4), is graded unitary. Then the Lee–Yang VOA $L(-\frac{22}{5}, 0)$, with its normalized canonical form, its partition-basis conjugation and its length flag (Definition 28.7), is graded unitary.*

Lean. `VOA.AdamovicSymbol.leeYang_gradedUnitary_of_affine`
File `VOA/Examples/AdamovicDescCyclic.lean`.

```
theorem leeYang_gradedUnitary_of_affine
    (hA : GradedUnitary affineForm affineRealStructure affineFlag) :
    VirGradedUnitarity.VirGradedUnitary LeeYang.leeYangC leeYangRealStructure :=
  leeYang_gradedUnitary_of_affine_wordSymbols_jet hA fun r => Desc.wordSymbolDescends (r +
   ↪ 2)
```

Hypotheses / status. conditional on `hA` (open)

Theorem 28.11 (Ghost level and nilpotency). *(1) The ghost currents form an affine $\mathfrak{sl}_2$ action of level $+4 = 2h^\vee$ on the ghost system. (2) For a translated affine action of level k on a matter super vertex algebra V with nonzero vacuum (and the parity and grading conditions needed to form the tensor product), $\mathbf{Q}^2 = 0$ if and only if $k = -4$.*

Lean. `VOA.Sl2SemiInfinite.ghostAffineAction`, `VOA.Sl2SemiInfinite.semiInfiniteQ_sq_zero_iff`
Files `VOA/Examples/Sl2SemiInfiniteGhostAffine.lean`, `Sl2SemiInfiniteAnomalyCriterion.lean`.

```
noncomputable def ghostAffineAction : AffineSl2Action sl2GhostSVA 4 :=

theorem semiInfiniteQ_sq_zero_iff (A : AffineSl2Action V k)
    (H : TranslationCovariant V) (hV : TensorVA.ParityCompatible V)
    (hg : TensorVA.TensorGradingHyp V sl2GhostSVA) (hvac : V.vacuum ≠ 0) :
    semiInfiniteQ A ∗ semiInfiniteQ A = 0 ↔ k = -4 := by
```

Hypotheses / status. Structural hypotheses; proved

Theorem 28.16 (Coset states are relative cocycles; the map to H^0). *For a commutant state x, the state $x \otimes \mathbf{1}_U \otimes \mathbf{1}_{\mathrm{gh}}$ is relative and $\mathbf{Q}$-closed, and $x \mapsto [x \otimes \mathbf{1} \otimes \mathbf{1}]$ is a linear map into the ghost-number-zero cohomology which intertwines the vertex products.*

Lean. `VOA.Sl2CosetConcrete.cosetToComplex_apply`, `VOA.Sl2CosetConcrete.cosetToComplex_closed`, `VOA.Sl2CosetConcrete.cosetToComplex_relative`, `VOA.Sl2CosetConcrete.cosetToComplex_mode`, `VOA.Sl2CosetConcrete.leeYangCosetToH0`
File `VOA/Examples/Sl2CosetConcreteCohomology.lean`.

```
theorem cosetToComplex_apply (x : CosetSpace) :
    cosetToComplex x = (x.val ⊗ₜ[ℂ] USVA.vacuum) ⊗ₜ[ℂ] sl2GhostSVA.vacuum := rfl

theorem cosetToComplex_closed (x : CosetSpace) : leeYangBRSTComplex.Q (cosetToComplex x) =
 ↪ 0 :=

theorem cosetToComplex_relative (x : CosetSpace) :
    cosetToComplex x ∈ relativeCarrier leeYangMatterAction.toAffineSl2Action :=

theorem cosetToComplex_mode (x y : CosetSpace) (n : ℤ) :
    cosetToComplex (cosetMode diagonalAction.toAffineSl2Action diagonalAction.translation x
     ↪ n y) =
```

```
      mode (brstSVA.Y (cosetToComplex x)) n (cosetToComplex y) :=

def leeYangCosetToH0 : CosetSpace →ₗ[ℂ] leeYangBRSTComplex.H0 :=
```

Hypotheses / status. Unconditional

Theorem 28.17 (The Virasoro image surjects onto Lee–Yang). *Let $\phi : V(-\frac{22}{5}, 0) \to H^0$ be the composite of the VOA map $V(c,0) \to$ (coset) determined by $\omega \mapsto \omega_{\mathrm{coset}}$ with the map of Theorem 28.16. Then* $\ker \phi$ *is contained in the maximal ideal of $V(c,0)$.*

Lean. `VOA.LeeYangShortcut.conformalEval_hom`, `VOA.LeeYangShortcut.virasoroToCoset`, `VOA.LeeYangShortcut.virasoroToH0`, `virasoroToH0_ker_le_maxIdeal`
Files `VOA/Examples/LeeYangShortcutVirasoro.lean`, `LeeYangShortcutImage.lean`.

```
theorem conformalEval_hom (W : VertexOperatorAlgebra E) {c : ℂ}
    (hc : W.centralCharge = c) :
    IsVOAHom (virasoroVOA c) W (conformalEval W hc) := by

def virasoroToCoset : VirUniv LeeYang.leeYangC →ₗ[ℂ] Sl2CosetConcrete.CosetSpace :=
  conformalEval Sl2CosetConcrete.cosetVOA rfl

def virasoroToH0 : VirUniv LeeYang.leeYangC →ₗ[ℂ] leeYangBRSTComplex.H0 :=
  leeYangCosetToH0.comp virasoroToCoset

theorem virasoroToH0_ker_le_maxIdeal :
    LinearMap.ker virasoroToH0 ≤ maxIdealVir LeeYang.leeYangC :=
```

Hypotheses / status. Unconditional

Theorem 28.21 (The harmonic bound). *Let $H(Q^-)_{(j,g)}$ be the cohomology of the leading piece Q^- alone in bidegree (j,g). Harmonic vectors of bidegree (j,g) inject into $H(Q^-)_{(j,g)}$. Consequently, for a finite set S of values of $2R$, the classes of* $\ker D/\operatorname{im} D$ *with a representative of ghost number g and $2R \in S$ span a space of dimension at most $\sum_{j\in S} \dim H(Q^-)_{(j,g)}$.*

Lean. `VOA.GradedUnitaryCohomology.KahlerComplex.leadCycles`, `VOA.GradedUnitaryCohomology.KahlerComplex.leadBoundaries`, `VOA.GradedUnitaryCohomology.KahlerComplex.leadCohom`, `VOA.GradedUnitaryCohomology.KahlerComplex.harmDeg`, `VOA.GradedUnitaryCohomology.KahlerComplex.harmToLead_injective`, `VOA.GradedUnitaryCohomology.KahlerComplex.cohomFilt`, `VOA.GradedUnitaryCohomology.KahlerComplex.rowSet`, `VOA.GradedUnitaryCohomology.KahlerComplex.finrank_cohomFilt_row_le_lead`
Files `VOA/Examples/GradedUnitaryCohomologyKahler.lean`, `GradedUnitaryCohomologyFiltered.lean`.

```
def leadCycles (a : ℤ × ℤ) : Submodule ℂ K := LinearMap.ker C.Qm ⊓ C.deg a

def leadBoundaries (a : ℤ × ℤ) : Submodule ℂ (C.leadCycles a) :=
  (Submodule.map C.Qm (C.deg (a - shQm))).comap (C.leadCycles a).subtype

abbrev leadCohom (a : ℤ × ℤ) := C.leadCycles a ⧸ C.leadBoundaries a

def harmDeg (a : ℤ × ℤ) : Submodule ℂ K := C.harm ⊓ C.deg a

theorem harmToLead_injective (a : ℤ × ℤ) : Function.Injective (C.harmToLead a) := by

def cohomFilt (A : Set (ℤ × ℤ)) : Submodule ℂ (cycleQuotient C.D) :=
  Submodule.map (boundariesInCycles C.D).mkQ
    (Submodule.comap (LinearMap.ker C.D).subtype (C.degOn A))
```

```
def rowSet (S : Finset ℤ) (g : ℤ) : Set (ℤ × ℤ) := {a | a.1 ∈ S ∧ a.2 = g}

theorem finrank_cohomFilt_row_le_lead (S : Finset ℤ) (g : ℤ) :
    finrank ℂ (C.cohomFilt (rowSet S g)) ≤ ∑ j ∈ S, finrank ℂ (C.leadCohom (j, g)) := by
```

Hypotheses / status. Unconditional

Theorem 28.27 (Graded unitarity at one weight from a family of cohomology classes). *Let C be a Kähler complex with a GU datum, W a VOA with a flag F at weight n and forms β, h. Let $f : U \to \ker D/\operatorname{im} D$ and $g : U \to W_n$ be linear, g surjective, $\ker f \subseteq \ker g$, and $P_p \le U$ an exhaustive filtration with $g(P_p) = F_pW_n$ and $f(P_p) \subseteq F_p(\ker D/\operatorname{im} D)$ (classes with representatives of $R \le p$ at ghost number zero). Suppose $\sum_{j=0}^{2p} \dim H(Q^-)_{(j,0)} = \dim F_pW_n$ for every p, that β and h pulled back along g agree with B and h on the harmonic lifts of f, and that harmonic vectors of odd $2R$ at ghost number zero vanish. Then W is graded unitary at weight n.*

Lean. `VOA.LeeYangShortcut.gradedUnitaryAt_of_class_family`, `VOA.LeeYangTransferAssembly.IntegralHarmonicSupport`
File `VOA/Examples/LeeYangShortcutHodge.lean`.

```
theorem gradedUnitaryAt_of_class_family {U : Type*} [AddCommGroup U] [Module ℂ U]
    (C : KahlerComplex K) (G : GUData C) (F : GradedFlag W) (n : ℤ)
    (β : LinearMap.BilinForm ℂ (W.grading n)) (h : SesqForm (W.grading n))
    (f : U →ₗ[ℂ] cycleQuotient C.D) (g : U →ₗ[ℂ] W.grading n)
    (hgs : Function.Surjective g) (hker : LinearMap.ker f ≤ LinearMap.ker g)
    (P : ℕ → Submodule ℂ U)
    (hquotient : ∀ p, (P p).map g = F.pieceIn p n)
    (hexhaustive : ∀ x : U, ∃ p, x ∈ P p)
    (hfiltered : ∀ p, (P p).map f ≤
      C.cohomFilt (KahlerComplex.rowSet (Finset.Icc 0 (2 * (p : ℤ))) 0))
    (hcharacter : ∀ p : ℕ,
      (∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), Module.finrank ℂ (C.leadCohom (j, 0))) =
        Module.finrank ℂ (F.pieceIn p n))
    (hB : ∀ x y, β (g x) (g y) = G.B (harmonicClassLift C f x) (harmonicClassLift C f y))
    (hh : ∀ x y, h (g x) (g y) = G.h (harmonicClassLift C f x) (harmonicClassLift C f y))
    (hintegral : IntegralHarmonicSupport C) : GradedUnitaryAt F n β h := by
```

File `VOA/Examples/LeeYangTransferAssemblyHodge.lean`.

```
def IntegralHarmonicSupport (C : KahlerComplex K) : Prop :=
  ∀ p : ℕ, C.harmDeg (2 * (p : ℤ) + 1, 0) = ⊥
```

Hypotheses / status. Explicit hypotheses

Theorem 28.29 ($L_1(\mathfrak{sl}_2)$ is unitary). *With its normalized canonical form and the descended compact conjugation, $B = L_1(\mathfrak{sl}_2)$ is unitary. Moreover its conformal vector is $\omega = \frac{1}{4}h_{-1}h_{-1}\mathbf{1}$.*

Lean. `VOA.ConcreteHodge.bRealStructure`, `VOA.ConcreteHodge.bForm`, `VOA.LevelOneUnitary.bUnitary`, `VOA.LevelOneHeis.hB`, `VOA.LevelOneHeis.bConformal_eq`
Files `VOA/Examples/BGPackageBFactor.lean`, `LevelOneUnitaryPositivity.lean`, `LevelOneHeisSingular.lean`.

```
def bRealStructure : RealStructure BVOA where

abbrev bForm : InvariantForm BVOA := simpleForm 1 bNoncritical

theorem bUnitary : ConcreteHodge.Unitary bForm bRealStructure := by
```

```
abbrev hB : BSpace := bAction.current 2

theorem bConformal_eq : BVOA.conformalVector = (1 / 4 : ℂ) • BVOA.smul hB (-1) hB := by
```

Hypotheses / status. unconditional

Definition 28.31 (Goodness of the actual differential). Give each factor its R-grading: on A the orthogonal layers of the corrected flag ($R = p$ on layer p), on B everything in $R = 0$, on U the layers of the ordinary flag, and on the ghosts a positive grading by weights. The differential is *good* on relative states if it maps bidegree $(2R, \mathrm{gh})$ into $(2R \pm 1, \mathrm{gh} + 1)$, with no R-preserving component.

Lean. `VOA.ConcreteHodge.aDeg`, `VOA.ConcreteHodge.bDeg`, `VOA.ConcreteHodge.uDeg`, `VOA.ConcreteHodge.matterDeg`, `VOA.ConcreteHodge.brstDeg`, `VOA.ConcreteHodge.RelativeDifferentialGood`
File `VOA/Examples/ConcreteGoodInterpreted.lean`.

```
def aDeg : (ℤ × ℤ) × ℂ → Submodule ℂ ASpace := layerDeg affineForm affineFlag

def bDeg : (ℤ × ℤ) × ℂ → Submodule ℂ BSpace := trivialDeg BVOA

def uDeg : (ℤ × ℤ) × ℂ → Submodule ℂ USpace := layerDeg auxiliaryForm auxiliaryFlag

def matterDeg : (ℤ × ℤ) × ℂ → Submodule ℂ MatterSpace := tensorDeg (tensorDeg aDeg bDeg)
 ↪  uDeg

def brstDeg {PG : SesqForm Sl2GhostSpace} (Gh : GhostRGrading PG) :
    (ℤ × ℤ) × ℂ → Submodule ℂ BRSTSpace :=
  tensorDeg matterDeg Gh.deg

def RelativeDifferentialGood {PG : SesqForm Sl2GhostSpace} (Gh : GhostRGrading PG) : Prop
 ↪  :=
  ∀ (n j g : ℤ) (x : BRSTSpace), x ∈ relativeWeight n → x ∈ brstDeg Gh ((j, g), (n : ℂ)) →
    leeYangBRSTComplex.Q x ∈
      brstDeg Gh ((j + 1, g + 1), (n : ℂ)) ⊔ brstDeg Gh ((j - 1, g + 1), (n : ℂ))
```

Hypotheses / status. `hA`, `hB`

Theorem 28.36 (The Virasoro classes are filtered). *Assume* $\mathrm{GU}(A)$. *On the actual complex, at every conformal weight n and for every p, the cohomology class of a universal Virasoro word of length* $\leq p$ *(through the coset stress tensor, Theorem 28.16) has a representative of* $2R \in [0, 2p]$ *at ghost number zero. This is step (b) of Remark 28.1.*

Lean. `VOA.LeeYangShortcut.UniversalClassFiltered`, `VOA.ConcreteHodge.endpointKahler`, `VOA.ConcreteHodge.endpointKahler_filtered`
Files `VOA/Examples/LeeYangShortcutAssembly.lean`, `InvariantSymbolBridge.lean`.

```
def UniversalClassFiltered (S : ConcreteKahlerGeometry) : Prop :=
  ∀ n (hp : IsPosOn (S.geometry n).P τ) p,
    (universalPrefix p n).map (modelUniversalClass S n hp) ≤
      (S.model n hp).cohomFilt (KahlerComplex.rowSet (Finset.Icc 0 (2 * (p : ℤ))) 0)

abbrev endpointKahler (hA : GradedUnitary affineForm affineRealStructure affineFlag) :
    ConcreteKahlerGeometry :=
  sp2Kahler hA LevelOneUnitary.bUnitary bgPG_pos bgGh bgGh_isStd
    ((bgGhostRemainderI hA ghostConjRhoD_proved ghostConjBE_proved).pkg
      LevelOneUnitary.bUnitary).sp2
```

```
theorem endpointKahler_filtered (hA : GradedUnitary affineForm affineRealStructure
 ↪  affineFlag) :
    UniversalClassFiltered (endpointKahler hA) :=
```

Hypotheses / status. `hA`

Definition 28.39 (The gap-two coefficient and the invariant-symbol statements). $G_r(n)$ is the coefficient of z^n in $z^{r(r+1)}/\prod_{j=1}^{r}(1-z^j)$, i.e. the number of partitions of n into exactly r parts, all ≥ 2, with consecutive parts differing by at least 2. In the Kähler coordinate $j = 2R$ put $\mathrm{lc}(n,j) = G_{j/2}(n)$ for even $j \geq 0$ and 0 otherwise. The *invariant-symbol bound* for a concrete Kähler geometry says $\dim H(Q^-)_{(j,0)} \leq \mathrm{lc}(n,j)$ at every weight n; the *dimension* statement says equality. The *Lee–Yang PBW character* says $\dim F_pL_n = \sum_{j=0}^{2p}\mathrm{lc}(n,j)$.

Lean. `VOA.LeeYangTransferAssembly.gapTwoCoefficient`, `VOA.LeeYangTransferAssembly.leadingCoefficient`, `VOA.LeeYangTransferAssembly.LeeYangPBWCharacter`, `VOA.LeeYangTransferAssembly.LeadingInvariantSymbolDimension`, `VOA.ConcreteHodge.LeadingInvariantSymbolBound`
File `VOA/Examples/LeeYangTransferAssemblyCharacter.lean`.

```
def gapTwoCoefficient (n : ℤ) (r : ℕ) : ℕ := by
  classical
  exact Fintype.card {a : Fin r → Fin (n.toNat + 1) //
    ((r * (r + 1) + ∑ j : Fin r, (j.val + 1) * (a j).val : ℕ) : ℤ) = n}

def leadingCoefficient (n j : ℤ) : ℕ :=
  if 0 ≤ j ∧ j % 2 = 0 then gapTwoCoefficient n (j.toNat / 2) else 0

def LeeYangPBWCharacter : Prop :=
  ∀ (n : ℤ) (p : ℕ),
    Module.finrank ℂ ((virGradedFlag leeYangC).pieceIn p n) =
      ∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), leadingCoefficient n j
```

Files `VOA/Examples/LeeYangTransferAssemblyInputs.lean`, `InvariantSymbolBridge.lean`.

```
def LeadingInvariantSymbolDimension (S : ConcreteKahlerGeometry) : Prop :=
  ∀ n (hp : IsPosOn (S.geometry n).P τ) j,
    Module.finrank ℂ ((S.model n hp).leadCohom (j, 0)) = leadingCoefficient n j

def LeadingInvariantSymbolBound (S : ConcreteKahlerGeometry) : Prop :=
  ∀ n (hp : IsPosOn (S.geometry n).P τ) j,
    Module.finrank ℂ ((S.model n hp).leadCohom (j, 0)) ≤ leadingCoefficient n j
```

Hypotheses / status. Definitions

Theorem 28.48 (The fermionic jet identity, and strictness). *(1)* $\sum_{r\geq 0} q^r \sum_{k=0}^{2r} \begin{bmatrix} 2r \\ k \end{bmatrix}_q / (q)_r = \left((q^3;q^3)_\infty/(q;q)_\infty\right)^3$. *(2)* $[q^N]$ *of the fermionic series is at most* $\sum_{r\leq N}\dim A_{r,N}$*; hence the jet character bound holds. (3) Given leading-symbol maps in every degree* $r \geq 1$ *and the jet character bound, strictness holds in every degree* $r \geq 1$*; with Theorem 28.45 this gives strictness outright.*

Lean. `VOA.JetFermionic.jetFermionicIdentity`, `VOA.JetProduct.coeff_jetSeries_le`, `VOA.SymbolModel.grJetStrict_of_leadingSymbol_product`
Files `VOA/Examples/JetFermionicIdentity.lean`, `JetProductCount.lean`, `JetStrictEndpoint.lean`.

```
theorem jetFermionicIdentity : JetFermionicIdentity := by
```

```
theorem coeff_jetSeries_le (N : ℕ) :
    PowerSeries.coeff N jetSeries ≤
      ∑ r ∈ Finset.range (N + 1), (Module.finrank ℂ (jetDeg r (N : ℤ)) : ℚ) := by

def grJetStrict_of_leadingSymbol_product (sym : ∀ r, JetLeadingSymbol (r + 1))
    (chi : JetProductBound) (r : ℕ) : GrJetStrict (r + 1) :=
  grJetStrict_succ_of_leadingSymbol sym (jetCharacterBound_of_product chi) r
```

Hypotheses / status. unconditional; (3) from inputs

Theorem 28.50 (Gap-two spanning and the upper bound). *In L, the length-$\leq$ p step of the PBW flag at weight N is spanned by the gap-two words of length $\leq$ p (words $L_{-n_1} \cdots L_{-n_q}\mathbf{1}$ with $n_1 \geq n_2 + 2 \geq \ldots$ and $n_q \geq 2$). Hence* $\dim F_p L_N \leq \sum_{j=0}^{2p} \mathrm{lc}(N, j)$.

Lean. `VOA.LeeYangCharacter.GapTwo`, `VOA.LeeYangCharacter.gapSpan`, `VOA.LeeYangCharacter.goodFil_leeYang_eq_gapSpan`, `VOA.LeeYangCharacter.finrank_pieceIn_le`
Files `VOA/Examples/LeeYangCharacterSpanning.lean`, `LeeYangCharacterReduction.lean`.

```
def GapTwo (w : List ℕ) : Prop := w.IsChain (fun a b => b + 2 ≤ a)

noncomputable def gapSpan (ℓ N : ℕ) : Submodule ℂ (VirUniv leeYangC) :=
  Submodule.span ℂ
    {x | ∃ u : List ℕ, GapTwo u ∧ energyVir u = N ∧ u.length ≤ ℓ ∧ negWordVir leeYangC u =
     ↪ x}

theorem goodFil_leeYang_eq_gapSpan (p N : ℕ) :
    goodFil leeYangC p (N : ℤ) = (gapSpan p N).map (maxIdealVir leeYangC).mkQ := by

theorem finrank_pieceIn_le (n : ℤ) (p : ℕ) :
    Module.finrank ℂ ((virGradedFlag leeYangC).pieceIn p n) ≤
      ∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), leadingCoefficient n j := by
```

Hypotheses / status. Unconditional

Theorem 28.53 (The Lee–Yang vacuum character). $\mathrm{rc}(N) \leq \operatorname{rank} G_N \leq \dim L_N$ *for every N; hence* $\dim L_N = \mathrm{rc}(N) = \sum_r G_r(N)$*, and the Rogers–Ramanujan lower bound holds.*

Lean. `VOA.RRLower.gramRank_le_finrank`, `VOA.RRLower.leeYangGramRankBound_holds`, `VOA.RRLower.leeYangRochaCaridiCharacter_holds`, `VOA.RRLower.leeYangRRLowerBound_holds`
Files `VOA/Examples/RRLowerGram.lean`, `LYGramRank.lean`.
File `VOA/Examples/LYGramRankCharacter.lean`.

```
theorem gramRank_le_finrank (N : ℕ) :
    (vermaGramMatrix leeYangC 0 N).rank ≤
      Module.finrank ℂ ((virasoroSimple leeYangC).grading (N : ℤ)) := by

theorem leeYangGramRankBound_holds : LeeYangGramRankBound := by

theorem leeYangRochaCaridiCharacter_holds : LeeYangRochaCaridiCharacter :=

theorem leeYangRRLowerBound_holds : LeeYangCharacter.LeeYangRRLowerBound :=
```

Hypotheses / status. unconditional

Theorem 28.54 (The Lee–Yang PBW character, filtration level by filtration level). *For every weight N and every p,* $\dim F_p L_N = \sum_{s \leq p} G_s(N)$*; that is, the statement* `LeeYangPBWCharacter` *of Definition 28.39 holds. It is equivalent to the Rogers–Ramanujan lower bound.*

Lean. `VOA.LeeYangCharacter.leeYangPBWCharacter_iff_lowerBound`
File `VOA/Examples/LeeYangCharacterReduction.lean`.

```
theorem leeYangPBWCharacter_iff_lowerBound : LeeYangPBWCharacter ↔ LeeYangRRLowerBound :=
```

Hypotheses / status. unconditional

Chapter 29

The $(2, 2n+3)$ series: coset transfer of graded unitarity at every boundary level

20 of the chapter's 55 numbered statements: its headline results and every definition they use.

Definition 29.3 (Coset level data). A *coset level datum* consists of complex numbers k_A (the level of the boundary factor), k_{act} (the index of its current action), k_U (the auxiliary level), a real number $k_U^{\mathbb{R}}$, a target central charge c, and a natural number m, such that: $k_{\mathrm{act}} = k_A$; $k_A + 2 \neq 0$; $\overline{k_A} = k_A$; $k_U = k_U^{\mathbb{R}}$ and $k_U^{\mathbb{R}} < -2$; $k_U + 2 \neq 0$; $k_{\mathrm{act}} + 3 \neq 0$; $k_{\mathrm{act}} + 1 + k_U = -4$; $\frac{3k_A}{k_A+2} + 1 - \frac{3(k_{\mathrm{act}}+1)}{k_{\mathrm{act}}+3} = c$; and $c_{2,2m+1} = c$.

Lean. `VOA.TwoSeries.CosetLevels`
File `VOA/Examples/TwoSeriesLevels.lean`.

```
structure CosetLevels where
  /-- Level of the boundary affine carrier. -/
  kA : ℂ
  /-- Index of the boundary current action (equal to `kA`). -/
  kAct : ℂ
  /-- Level of the auxiliary universal factor. -/
  kU : ℂ
  /-- The auxiliary level as a real number. -/
  kUr : ℝ
  /-- Target central charge. -/
  c : ℂ
  /-- Index of `c` in the boundary family `c_{2,2m+1}`. -/
  m : ℕ
  kAct_eq : kAct = kA
  aNoncritical : kA + 2 ≠ 0
  aReal : (starRingEnd ℂ) kA = kA
  kU_eq : kU = (kUr : ℂ)
  kUr_lt : kUr < -2
  uNoncritical : kU + 2 ≠ 0
  diagNoncritical : kAct + 1 + 2 ≠ 0
  /-- The total matter level cancels the ghost level `-4`. -/
  sum : kAct + 1 + kU = -4
  /-- Coset central charge `c(k) + c(1) - c(k+1)`. -/
  charge : 3 * kA / (kA + 2) + 1 - 3 * (kAct + 1) / (kAct + 1 + 2) = c
  /-- The target is the `m`th boundary minimal model. -/
  boundary : boundaryCharge m = c
```

Theorem 29.4 (The coset central charge). *For every $n \in \mathbb{N}$, with $k = k_n = -2 + \frac{2}{2n+1}$,*

$$\frac{3k}{k+2} + 1 - \frac{3(k+1)}{k+3} \;=\; c_{2,2n+3} \;=\; 1 - \frac{3(2n+1)^2}{2n+3}.$$

Lean. `VOA.TwoSeries.twoSeries_charge`
File `VOA/Examples/TwoSeriesLevels.lean`.

```
theorem twoSeries_charge (n : ℕ) :
    3 * level n / (level n + 2) + 1 - 3 * (level n + 1) / (level n + 1 + 2) =
      boundaryCharge (n + 1) := by
```

Hypotheses / status. None

Definition 29.6 (The data on the boundary factor). For a level datum L, let $A = L_{k_A}(\mathfrak{sl}_2)$ be the simple quotient of $V^{k_A}(\mathfrak{sl}_2)$ by its maximal graded ideal. Its data are: the canonical invariant form normalised by $(|0\rangle||0\rangle) = 1$; the *corrected current-and-stress flag* (the PBW-type filtration of Chapter 28 in which a current mode J^a_{-j} and a Sugawara mode L_{-j} each raise the degree by one, descended to the quotient); and the compact conjugation (the antilinear involution fixing the compact real form $\mathfrak{su}(2)$), descended from V^{k_A} to A.

Lean. `VOA.TwoSeries.affineForm`, `VOA.TwoSeries.affineFlag`, `VOA.TwoSeries.affineConj`
File `VOA/Examples/TwoSeriesAffine.lean`.

```
abbrev affineForm : InvariantForm (AVOA L) := simpleForm L.kA L.aNoncritical

def affineFlag : GradedFlag (AVOA L) :=
  CorrectedR.correctedAffineRQuotientFlag L.kA L.aNoncritical
    (QuotVOA.maxGradedIdeal (universal L.kA L.aNoncritical))
    (QuotVOA.maxGradedIdeal_isIdeal _)
    (QuotVOA.maxGradedIdeal_isGradedIdeal _)

def affineConj : ASpace L →ₛₗ[starRingEnd ℂ] ASpace L :=
  (QuotVOA.maxGradedIdeal (universal L.kA L.aNoncritical)).mapQ
    (QuotVOA.maxGradedIdeal (universal L.kA L.aNoncritical))
    (affineUniversalConj L) (affineUniversalConj_maxIdeal_stable L)
```

Theorem 29.11 (Coset transfer for any level datum). *Let L be a level datum. If $A = L_{k_A}(\mathfrak{sl}_2)$ is graded unitary, and a Hodge package and a squeeze input are given, then $L(c,0)$ is graded unitary for its partition-basis real structure. In particular, for the n-th member of the series:* $\mathrm{GU}(L_{k_n}) \wedge \mathrm{Hodge} \wedge \mathrm{Squeeze} \Rightarrow \mathrm{GU}(M(2, 2n+3))$.

Lean. `VOA.TwoSeries.cosetLevels_gradedUnitary_of_affine`, `VOA.TwoSeries.twoSeries_gradedUnitary_of_affine`
File `VOA/Examples/TwoSeriesAssembly.lean`.

```
theorem cosetLevels_gradedUnitary_of_affine
    (hA : GradedUnitary (affineForm L) (affineRealStructure L) (affineFlag L))
    (hodge : BRSTHodgeInputs L) (hsq : SqueezeInput L hodge.kahler) :
    VirGradedUnitary L.c (targetRealStructure L) := by

theorem twoSeries_gradedUnitary_of_affine (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪ (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (hodge : BRSTHodgeInputs (twoSeriesLevels n))
    (hsq : SqueezeInput (twoSeriesLevels n) hodge.kahler) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  cosetLevels_gradedUnitary_of_affine (twoSeriesLevels n) hA hodge hsq
```

Hypotheses / status. `hA`, Hodge, squeeze

Theorem 29.12 (The Hodge package exists at every level). *For every level datum L and every proof h_A of* $\mathrm{GU}(A)$ *there is a BRST/Hodge package. Consequently* $\mathrm{GU}(A) \wedge \mathrm{Squeeze} \Rightarrow \mathrm{GU}(L(c,0))$.

Lean. `VOA.TwoSeries.brstHodgeInputs`, `VOA.TwoSeries.twoSeries_gradedUnitary_of_affine_hodgeFree`
File `VOA/Examples/TwoSeriesHodgeInputs.lean`.

```
def brstHodgeInputs
    (hA : GradedUnitary (affineForm L) (affineRealStructure L) (affineFlag L)) :
    BRSTHodgeInputs L where

theorem twoSeries_gradedUnitary_of_affine_hodgeFree (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪ (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (hsq : SqueezeInput (twoSeriesLevels n) (brstHodgeInputs (twoSeriesLevels n)
     ↪ hA).kahler) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  cosetLevels_gradedUnitary_of_affine_hodgeFree (twoSeriesLevels n) hA hsq
```

Hypotheses / status. `hA`

Definition 29.13 (The affine invariant spaces). For $r \in \mathbb{N}$ and $N \in \mathbb{Z}$, $\mathcal{I}_{r,N}$ is the space of tensors in $\mathrm{gr}_r A \otimes B$ of total conformal weight N annihilated by every diagonal non-negative current mode $J^a_m \otimes 1 + 1 \otimes J^a_m$ ($a \in \{e,f,h\}$, $m \geq 0$). Here $\mathrm{gr}_r A$ is the r-th layer of the corrected flag of A with its induced mode action, and $B = L_1(\mathfrak{sl}_2)$.

Lean. `VOA.TwoSeries.affineInvariants`
File `VOA/Examples/TwoSeriesKoszulInvariants.lean`.

```
def affineInvariants (r : ℕ) (N : ℤ) : Submodule ℂ (GrA L r ⊗[ℂ] BSpace) :=
  VOA.CyclicEval.tensorWeight (grDeg L r) VOA.CyclicEval.bWeight N ⊓ jointKer (grDiag L r)
```

Theorem 29.15 (Transfer with an invariant count). *For every n: if* $\mathrm{GU}(L_{k_n})$ *holds and a function* coeff *satisfies* $\dim \mathcal{I}_{r,N} \leq \mathrm{coeff}(N,r)$ *for all N, r and* $\sum_{r \leq p} \mathrm{coeff}(N,r) \leq \dim F_p T_N$ *for all N, p, then $M(2, 2n+3)$ is graded unitary.*

Lean. `VOA.TwoSeries.twoSeries_gradedUnitary_of_affine_invariants`
File `VOA/Examples/TwoSeriesKoszulInvariants.lean`.

```
theorem twoSeries_gradedUnitary_of_affine_invariants (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪ (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (coeff : ℤ → ℕ → ℕ)
    (hinv : ∀ (N : ℤ) (r : ℕ),
      Module.rank ℂ (affineInvariants (twoSeriesLevels n) r N) ≤ coeff N r)
    (hpbw : ∀ (N : ℤ) (p : ℕ), ∑ r ∈ Finset.range (p + 1), coeff N r ≤
      Module.finrank ℂ ((virGradedFlag (twoSeriesLevels n).c).pieceIn p N)) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
```

Hypotheses / status. `hA`, `hinv`, `hpbw`

Definition 29.16 (Gordon-good words; the Andrews–Gordon count). A list $w = (w_0, \dots, w_{\ell-1})$ is *Gordon-good of order n* if $w_i \geq w_{i+n} + 2$ whenever both entries exist. The *Andrews–Gordon count* $g_n(N, r)$ is the number of sorted Gordon-good words of length exactly r and energy N (and 0 for $N < 0$). Equivalently, it is the number of partitions of N into exactly r parts ≥ 2 in which, writing f_j for the multiplicity of the part j, $f_j + f_{j+1} \leq n$ for every j.

Lean. `VOA.TwoSeriesCharacter.GordonGood`, `VOA.TwoSeriesCharacter.tupleList`, `VOA.TwoSeriesCharacter.gordonWords`, `VOA.TwoSeriesCharacter.gordonCoeff`
Files `VOA/Examples/TwoSeriesCharacterSpanning.lean` (Gordon-good) and `VOA/Examples/TwoSeriesCharacterReduction.lean` (the count).

```
def GordonGood (n : ℕ) (w : List ℕ) : Prop :=
  ∀ i < w.length - n, w.getD (i + n) 0 + 2 ≤ w.getD i 0

def tupleList {N r : ℕ} (a : Fin r → Fin (N + 1)) : List ℕ := List.ofFn fun j => (a j : ℕ)

def gordonWords (n N r : ℕ) : Finset (List ℕ) :=
  ((Finset.univ : Finset (Fin r → Fin (N + 1))).image tupleList).filter
    (fun u => SortedWord u ∧ GordonGood n u ∧ energyVir u = N)

def gordonCoeff (n : ℕ) (N : ℤ) (r : ℕ) : ℕ :=
  if 0 ≤ N then (gordonWords n N.toNat r).card else 0
```

Theorem 29.19 (Jet relations of the series). *For every $n \in \mathbb{N}$, $V(c_{2,2n+3}, 0)$ has jet relations of order $n + 1$.*

Lean. `VOA.TwoSeries.twoSeries_hasJetRelations_all`
File `VOA/Examples/TwoSeriesCharacterEndpoint.lean`.

```
theorem twoSeries_hasJetRelations_all (n : ℕ) :
    VOA.TwoSeriesCharacter.HasJetRelations (twoSeriesLevels n).c n := by
```

Hypotheses / status. None

Theorem 29.26 (The vacuum character of $M(2, 2n + 3)$). *For all $n, N \in \mathbb{N}$,*

$$\dim L(c_{2,2n+3}, 0)_N \;=\; \sum_{r \leq N} g_n(N, r) \;=\; \mathrm{rcq}_n(N).$$

In particular the total lower bound GordonLowerBound$(c_{2,2n+3}, n)$ *holds.*

Lean. `VOA.TwoSeries.gordonLowerBound_twoSeries`, `VOA.TwoSeries.twoSeries_character`
File `VOA/Examples/TwoSeriesCharacterAGIdentity.lean`.

```
theorem gordonLowerBound_twoSeries (n : ℕ) :
    VOA.TwoSeriesCharacter.GordonLowerBound (twoSeriesLevels n).c n := by

theorem twoSeries_character (n N : ℕ) :
    Module.finrank ℂ ((VOA.VirSimple.virasoroSimple (twoSeriesLevels n).c).grading (N : ℤ))
     ↪ =
      ∑ r ∈ Finset.range (N + 1), VOA.TwoSeriesCharacter.gordonCoeff n (N : ℤ) r ∧
    ((∑ r ∈ Finset.range (N + 1), VOA.TwoSeriesCharacter.gordonCoeff n (N : ℤ) r : ℕ) : ℤ)
     ↪ =
      VOA.TwoSeriesCharacter.KacGram.rcq n N :=
```

Hypotheses / status. None; **unconditional**

Theorem 29.27 (The refined PBW character of $M(2,2n+3)$). *For every $n \in \mathbb{N}$, $N \in \mathbb{Z}$, $p \in \mathbb{N}$: $\sum_{r \le p} g_n(N,r) \le \dim F_p L(c_{2,2n+3},0)_N$. Combined with Theorem 29.20, the inequality is an equality.*

Lean. `VOA.TwoSeries.twoSeries_hpbw`
File `VOA/Examples/TwoSeriesCharacterAGIdentity.lean`.

```
theorem twoSeries_hpbw (n : ℕ) :
    ∀ (N : ℤ) (p : ℕ), ∑ r ∈ Finset.range (p + 1), VOA.TwoSeriesCharacter.gordonCoeff n N r
     ↪ ≤
      Module.finrank ℂ ((virGradedFlag (twoSeriesLevels n).c).pieceIn p N) :=
```

Hypotheses / status. None

Theorem 29.31 (The socle-word count is the Andrews–Gordon count). *For all $n, N, r \in \mathbb{N}$, $\mathrm{swc}_n(N,r) = g_n(N,r)$.*

Lean. `VOA.SocleCount.socleWordCount_eq`
File `VOA/Examples/TwoSeriesSocleCountGordon.lean`.

```
theorem socleWordCount_eq (n N r : ℕ) :
    socleWordCount n (N : ℤ) r = gordonCoeff n (N : ℤ) r := by
```

Hypotheses / status. None

Theorem 29.33 (Transfer from the socle data of $\mathrm{gr}_r A$). *For every n: if $L_{k_n}(\hat{\mathfrak{sl}}_2)$ is graded unitary and $\mathrm{gr}_r A$ (for every r, with the socle words of size r as labels and the Cartan image $I_N \cdot \mathrm{gr}_r A$) has socle data, then $M(2,2n+3)$ is graded unitary.*

Lean. `VOA.TwoSeries.TwoSeriesSocleSpectrum`, `VOA.TwoSeries.twoSeries_gradedUnitary_of_socleSpectrum`
File `VOA/Examples/TwoSeriesSocleDual.lean`.

```
structure TwoSeriesSocleSpectrum (n : ℕ) : Prop where
  rank_le : ∀ (r : ℕ) (d : ℤ) (c : ℂ),
    finrank ℂ (sing (grRho (twoSeriesLevels n) r) (grDeg (twoSeriesLevels n) r) d c) ≤
      Nat.card {i : SocleIndex n r //
        (socleDegree i.2.1 : ℤ) = d ∧ c = -(2 * (((i.1 : ℕ) : ℕ) : ℂ))}
  transverse : ∀ (r : ℕ) (d : ℤ) (N : ℕ),
    Disjoint (sing (grRho (twoSeriesLevels n) r) (grDeg (twoSeriesLevels n) r) d
        (-(2 * (N : ℂ))))
      (avoid (grRho (twoSeriesLevels n) r) (grDeg (twoSeriesLevels n) r)
        (grCart (twoSeriesLevels n) r) d N)

theorem twoSeries_gradedUnitary_of_socleSpectrum (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪ (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (S : TwoSeriesSocleSpectrum n) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  twoSeries_gradedUnitary_of_socleInput n hA S.toSocleInput
```

Hypotheses / status. `hA`, socle data

Definition 29.34 (The coefficient ring R_n). Let $\mathbb{C}[T] = \mathbb{C}[T_0, T_1, \dots]$ and $T(s) = \sum_j T_j s^j$. The *jet ideal* of T^n is the ideal generated by the coefficients of $T(s)^n$, and $R_n = \mathbb{C}[T]/(\text{jet ideal of } T^n)$, with t_j the image of T_j.

Lean. `VOA.TwoSeries.StrictModel.TPoly`, `VOA.TwoSeries.StrictModel.tSer`, `VOA.TwoSeries.StrictModel.jetIdealT`, `VOA.TwoSeries.StrictModel.Rn`, `VOA.TwoSeries.StrictModel.tn`
File `VOA/Examples/TwoSeriesStrictEndpoint.lean`.
File `VOA/Examples/TwoSeriesJetRing.lean`.

```
abbrev TPoly : Type := MvPolynomial ℕ ℂ

def tSer : PowerSeries TPoly := PowerSeries.mk X

def jetIdealT (n : ℕ) : Ideal TPoly :=
  Ideal.span (Set.range fun j => PowerSeries.coeff j (tSer ^ n))

abbrev Rn (n : ℕ) : Type := TPoly ⧸ jetIdealT n

def tn (n : ℕ) (j : ℕ) : Rn n := Ideal.Quotient.mk (jetIdealT n) (X j)
```

Definition 29.36 (The model $S_{n,r}$). Let $R_n[u_j, v_j]$ be the polynomial ring in two sequences of jet variables over R_n. Each generator letter of the corrected PBW basis of $V^k(\mathfrak{sl}_2)$ gets a *model symbol*: a current letter (J^a, j) gets the j-th jet of u^2, uv or v^2 (the Adamović symbols of Chapter 28); the Sugawara letter (L, j) gets the j-th jet of $-(u'v - uv')$ plus t_j. A word's symbol is the product of its letters' symbols. The model $S_{n,r}$ is the span of the symbols of words of length r; its degree-d piece is the span of symbols of words of length r and weight d.

Lean. `VOA.TwoSeries.StrictModel.MPoly`, `VOA.TwoSeries.StrictModel.mletter`, `VOA.TwoSeries.StrictModel.mword`, `VOA.TwoSeries.StrictModel.modelImage`, `VOA.TwoSeries.StrictModel.modelDeg`
Files `VOA/Examples/TwoSeriesStrictRing.lean` and `VOA/Examples/TwoSeriesStrictWords.lean`.

```
abbrev MPoly : Type := MvPolynomial (Fin 2 × ℕ) R

def mletter : Generator × ℕ → MPoly R
  | (.current a, n) => ι R (letterSym (.current a, n))
  | (.sugawara, n) => ι R (letterSym (.sugawara, n)) + MvPolynomial.C (t n)

def mword (w : List (Generator × ℕ)) : MPoly R := (w.map (mletter R t)).prod

def modelImage (r : ℕ) : Submodule ℂ (MPoly R) :=
  Submodule.span ℂ {Q | ∃ w : List (Generator × ℕ), w.length = r ∧ mword R t w = Q}

def modelDeg (r : ℕ) (d : ℤ) : Submodule ℂ (MPoly R) :=
  Submodule.span ℂ
    {Q | ∃ w : List (Generator × ℕ), w.length = r ∧ wordWt w = d ∧ mword R t w = Q}
```

Theorem 29.39 (Transfer from the strict model). *For every $n \geq 1$: graded unitarity of L_{k_n}, detection, the model character bound and model socle data for every r imply graded unitarity of $M(2, 2n+3)$.*

Lean. `VOA.TwoSeries.StrictModel.twoSeries_gradedUnitary_of_strictModel`
File `VOA/Examples/TwoSeriesStrictEndpoint.lean`.

```
theorem twoSeries_gradedUnitary_of_strictModel (n : ℕ) (hn : 1 ≤ n)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
    ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (det : TwoSeriesModelDetects n) (chi : TwoSeriesModelCharacter n)
    (spec : ∀ r, ModelSocleSpectrum n r) :
```

```
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  twoSeries_gradedUnitary_of_socleModels n hA
    fun r => twoSeriesSocleModel_of_strict hn det chi r (spec r)
```

Hypotheses / status. hA, det, chi, spec

Theorem 29.41 (The fermionic series is the product; the model character bound). *For every* $n \in \mathbb{N}$, $\mathrm{ferm}_n = \big((q^{2n+1};q^{2n+1})_\infty/(q;q)_\infty\big)^3$. *For every* $n \geq 1$ *the model character bound holds.*

Lean. `VOA.TwoSeries.ModelChar.fermSeries_eq_product`, `VOA.TwoSeries.ModelChar.twoSeriesModelCharacter_all`
Files `VOA/Examples/TwoSeriesModelCharFermionic.lean` and `VOA/Examples/TwoSeriesModelCharEndpoint.lean`.

```
theorem fermSeries_eq_product (n : ℕ) :
    fermSeries n = AffineSl2BoundaryCharacter.product n := by

theorem twoSeriesModelCharacter_all (n : ℕ) (hn : 1 ≤ n) : TwoSeriesModelCharacter n := by
  intro N
  rw [← fermSeries_eq_product n]
  exact fermSeries_le_model hn N
```

Hypotheses / status. None ($n \geq 1$)

Theorem 29.51 (The slice form of Arakawa's theorem, unconditionally). *For every level datum L (Definition 29.3) whose affine level* $k = k_A$ *is not a natural number, the slice map at* $c = c_{\mathrm{DS}}(k)$ *sends the maximal ideal of* $V^k(\mathfrak{sl}_2)$ *into the maximal ideal of* $V(c_{\mathrm{DS}}(k), 0)$*, in every Rees degree. For an arbitrary non-critical* $k \notin \mathbb{N}$ *(no level datum) the same holds for the ungraded map* W_1*. In particular, for every m, the slice map of the member* $n = m + 1$ *at* $c = c_{2,2m+3}$ *has the slice Arakawa property.*

Lean. `VOA.TwoSeries.SliceMap.W1_maxIdeal`, `VOA.TwoSeries.SliceMap.sliceArakawa_unconditional`, `VOA.TwoSeries.SliceMap.twoSeries_kA_not_nat`, `VOA.TwoSeries.SliceMap.twoSeries_sliceArakawa`
File `VOA/Examples/TwoSeriesArakawaUnconditional.lean`.

```
theorem W1_maxIdeal (hkN : ∀ i : ℕ, k ≠ i) {y : WakModule k} (hy : y ∈ Imax k hk) :
    W1 (cDS k) k hk y ∈ maxIdealVir (cDS k) := by

theorem sliceArakawa_unconditional (L : CosetLevels) (hkN : ∀ i : ℕ, L.kA ≠ i) :
    SliceArakawa (dsSliceMap (cDS L.kA) L) := by

theorem twoSeries_kA_not_nat (m : ℕ) (i : ℕ) : (twoSeriesLevels (m + 1)).kA ≠ i := by

theorem twoSeries_sliceArakawa (m : ℕ) :
    SliceArakawa (dsSliceMap (twoSeriesLevels m).c (twoSeriesLevels (m + 1))) := by
```

Hypotheses / status. None

Theorem 29.58 (The generation input for every $n \geq 2$**; the arrangement inclusions for every** $n \geq 1$**).** *For every* $n \geq 2$, GI_n *holds; hence* AI_n *holds. For every* $n \geq 1$, SAI_n *holds.*

Lean. `VOA.TwoSeries.GenIncl.genInclusion`, `VOA.TwoSeries.GenIncl.arrangementInclusion`, `VOA.TwoSeries.GenIncl.symArrangementInclusion`
Files `VOA/Examples/TwoSeriesGenIncl.lean` (and `VOA/Examples/TwoSeriesArrangementGen.lean` for the passage GI $\Rightarrow$ AI).

```
theorem genInclusion {n : ℕ} (hn : 2 ≤ n) : GenInclusion n := by

theorem arrangementInclusion {n : ℕ} (hn : 2 ≤ n) : ArrangementInclusion n :=
  arrangementInclusion_of_genInclusion hn (genInclusion hn)

theorem symArrangementInclusion {n : ℕ} (hn : 1 ≤ n) :
    ArrangementSym.SymArrangementInclusion n := by
```

Hypotheses / status. $2 \le n$ (resp. $1 \le n$)

Theorem 29.59 (Graded unitarity transfers along the coset, at every boundary level). *For every $m \in \mathbb{N}$: if $L_{-2+2/(2m+3)}(\mathfrak{sl}_2)$, with its canonical form, compact conjugation and corrected current-and-stress flag, is graded unitary, then the minimal model $M(2, 2m+5) = L(c_{2,2m+5}, 0)$, with its canonical form, partition-basis real structure and length flag, is graded unitary.*

Lean. `VOA.TwoSeries.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final`
File `VOA/Examples/TwoSeriesGenIncl.lean`.

```
theorem twoSeries_gradedUnitary_of_affine_final (m : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels (m + 1)))
      (affineRealStructure (twoSeriesLevels (m + 1))) (affineFlag (twoSeriesLevels (m +
      ↪ 1)))) :
    VirGradedUnitary (twoSeriesLevels (m + 1)).c (targetRealStructure (twoSeriesLevels (m +
    ↪ 1))) :=
  twoSeries_gradedUnitary_of_affine_symArrangement m hA
    (GenIncl.symArrangementInclusion (by omega))
```

Hypotheses / status. `hA` only; conditional

Part VII

The Moonshine module

Chapter 30

Lattices, codes and the Leech lattice

6 of the chapter's 42 numbered statements: its headline results and every definition they use.

Theorem 30.17 (Index–determinant relation). *Let $f : M \to L$ be an injective isometric map of integral lattices whose image has the same rank as L, and let $k = [L : f(M)]$. Then* $\det M = k^2 \det L$.

Lean. `IntegralLattice.det_of_injective_index`

```
theorem det_of_injective_index (f : Γ →ₗ[ℤ] Λ) (hf : Function.Injective f)
    (hform : ∀ x y : Γ, Λ.bilin (f x) (f y) = Γ.bilin x y)
    (hrank : finrank ℤ (LinearMap.range f) = Λ.rank)
    {k : ℕ} (hk : Nat.card (Λ ⧸ LinearMap.range f) = k) :
    Γ.det = (k : ℤ) ^ 2 * Λ.det
```

File `VOA/LatticesAndCodes/Lattice/Index.lean`. **Hypotheses / status.** Isometric, full rank

Theorem 30.21 (MacWilliams identity). *For a linear code $C \subseteq (\mathbb{Z}/q)^n$, $|C|\, W_{C^\perp}(X, Y) = W_C\big(X + (q-1)Y,\ X - Y\big)$.*

Lean. `Code.macWilliams_eval`, `Code.macWilliams`

```
theorem macWilliams_eval (C : Code q n) (A B : ℂ) :
    (C.toFinset.card : ℂ) * eval ![A, B] C.dual.hammingEnumerator
      = eval ![A + ((q : ℂ) - 1) * B, A - B] C.hammingEnumerator

theorem macWilliams (C : Code q n) :
    (C.toFinset.card : ℂ) • C.dual.hammingEnumerator
      = bind₁ ![X 0 + MvPolynomial.C ((q : ℂ) - 1) * X 1, X 0 - X 1] C.hammingEnumerator
```

File `VOA/LatticesAndCodes/Codes/MacWilliams.lean`. **Hypotheses / status.** $q \geq 1$

Theorem 30.24 ($\mathcal{G}$ is a doubly even self-dual $[24, 12, 8]$ code). $\dim \mathcal{G} = 12$, *$\mathcal{G}$ is self-dual and doubly even, and its minimum distance is* 8.

Lean. `Code.golay_rank`, `Code.golay_isSelfDual`, `Code.golay_isDoublyEven`, `Code.golay_minDist`

```
theorem golay_rank : golay.rank = 12

theorem golay_isSelfDual : golay.IsSelfDual

theorem golay_isDoublyEven : golay.IsDoublyEven

theorem golay_minDist : golay.minDist = 8
```

File `VOA/LatticesAndCodes/Codes/Golay.lean`. **Hypotheses / status.** None

Theorem 30.25 (Weight enumerator of $\mathcal{G}$). $W_{\mathcal{G}}(X,Y) = X^{24} + 759X^{16}Y^8 + 2576X^{12}Y^{12} + 759X^8Y^{16} + Y^{24}$. *In particular $\mathcal{G}$ has* 759 *codewords of weight* 8, 2576 *of weight* 12 *and* 759 *of weight* 16.

Lean. `Code.golay_weightCount_eight`, `Code.golay_weightCount_twelve`, `Code.golay_weightCount_sixteen_eq`, `Code.golay_hammingEnumerator`

```
theorem golay_weightCount_eight : golay.weightCount 8 = 759

theorem golay_weightCount_twelve : golay.weightCount 12 = 2576

theorem golay_weightCount_sixteen_eq : golay.weightCount 16 = 759

theorem golay_hammingEnumerator :
    golay.hammingEnumerator
      = X 0 ^ 24 + MvPolynomial.C 759 * (X 0 ^ 16 * X 1 ^ 8)
        + MvPolynomial.C 2576 * (X 0 ^ 12 * X 1 ^ 12)
        + MvPolynomial.C 759 * (X 0 ^ 8 * X 1 ^ 16) + X 1 ^ 24
```

File `VOA/LatticesAndCodes/Codes/GolayEnumerator.lean`. **Hypotheses / status.** None

Theorem 30.29 (Properties of Γ_C). *Let $C \subseteq \mathbb{F}_2^n$ be self-orthogonal of dimension k. Then Γ_C is positive definite of rank n;* $\det \Gamma_C = 2^{n-2k}$*; Γ_C is even iff C is doubly even; and Γ_C is unimodular iff C is self-dual.*

Lean. `latticeOfCode_isPosDef`, `latticeOfCode_rank`, `latticeOfCode_isEven`, `latticeOfCode_det`, `latticeOfCode_isUnimodular`

```
theorem latticeOfCode_isPosDef (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).IsPosDef

@[simp] theorem latticeOfCode_rank (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).rank = n

theorem latticeOfCode_isEven (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).IsEven ↔ C.IsDoublyEven

theorem latticeOfCode_det (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).det = 2 ^ (n - 2 * C.rank)

theorem latticeOfCode_isUnimodular (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).IsUnimodular ↔ C.IsSelfDual
```

File `VOA/LatticesAndCodes/Construction/CodeToLattice.lean`. **Hypotheses / status.** $C \subseteq C^\perp$

Theorem 30.42 (Rank divisible by 8; Ebeling, Thm. 2.1). *If L is even, unimodular and positive definite, then* $8 \mid \operatorname{rank} L$*, and ϑ_L is a modular form of weight* $\operatorname{rank} L/2$ *for $SL_2(\mathbb{Z})$.*

Lean. `IntegralLattice.eight_dvd_rank`, `IntegralLattice.thetaModularForm`, `IntegralLattice.theta_isModularForm`
File `VOA/LatticesAndCodes/Theta/ModularForm.lean`.

```
theorem eight_dvd_rank (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular) : 8 ∣ Γ.rank

def thetaModularForm (Γ : IntegralLattice) (h : Γ.IsPosDef) (he : Γ.IsEven)
    (hu : Γ.IsUnimodular) : ModularForm 𝒮ℒ ((Γ.rank / 2 : ℕ) : ℤ) where
  toFun := Γ.thetaFun
```

```
theorem theta_isModularForm (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular) :
    8 ∣ Γ.rank ∧
      ∃ f : ModularForm 𝒮ℒ ((Γ.rank / 2 : ℕ) : ℤ), ∀ z : ℍ, f z = Γ.theta (z : ℂ)
```

Hypotheses / status. Even, unimod., pos. definition

Chapter 31

Heisenberg and lattice vertex algebras

7 of the chapter's 35 numbered statements: its headline results and every definition they use.

Theorem 31.3 (Heisenberg relations). *For all colours a, b and all $m, n \in \mathbb{Z}$, $[h_a(m), h_b(n)] = m\,k\,\delta_{ab}\,\delta_{m+n,0}\cdot \mathrm{id}$ on $\mathcal{F}_r$.*

Lean. `VOA.HeisenbergN.oscN_comm`
File `VOA/Examples/HeisenbergNFock.lean`.

```
theorem oscN_comm (r : ℕ) (k : ℂ) (a b : Fin r) (m n : ℤ) :
    oscN r k a m * oscN r k b n - oscN r k b n * oscN r k a m
      = (if a = b ∧ m + n = 0 then (m : ℂ) * k else 0) • LinearMap.id
```

Hypotheses / status. Unconditional

Theorem 31.11 (Properties of $\varepsilon_{\mathcal{B}}$). *For every basis $\mathcal{B}$ and all $\alpha, \alpha', \beta, \beta', \gamma \in L$:*

(i) $\varepsilon(\alpha+\alpha',\beta) = \varepsilon(\alpha,\beta)\varepsilon(\alpha',\beta)$ *and* $\varepsilon(\alpha,\beta+\beta') = \varepsilon(\alpha,\beta)\varepsilon(\alpha,\beta')$*;*

(ii) $\varepsilon(0,\beta) = \varepsilon(\alpha,0) = 1$ *and* $\varepsilon(\alpha,\beta)^2 = 1$*;*

(iii) the 2-cocycle law (31.8);

(iv) $\mathrm{coc}(\alpha,\beta) + \mathrm{coc}(\beta,\alpha) = \langle\alpha,\beta\rangle - \sum_i \langle B_i, B_i\rangle \alpha_i\beta_i$*;*

(v) if L is even, the commutator property (31.9).

Lean. `VOA.IntegralLattice.cocycle_add_left`, `VOA.IntegralLattice.cocycle_add_right`, `VOA.IntegralLattice.cocycle_zero_left`, `VOA.IntegralLattice.cocycle_zero_right`, `VOA.IntegralLattice.cocycle_sq`, `VOA.IntegralLattice.cocycle_cocycleLaw`, `VOA.IntegralLattice.cocExp_add_swap`, `VOA.IntegralLattice.cocycle_comm`

```
theorem cocycle_add_left (α α' β : L.carrier) :
    cocycle L B (α + α') β = cocycle L B α β * cocycle L B α' β
theorem cocycle_add_right (α β β' : L.carrier) :
    cocycle L B α (β + β') = cocycle L B α β * cocycle L B α β'
theorem cocycle_zero_left (β : L.carrier) : cocycle L B 0 β = 1
theorem cocycle_zero_right (α : L.carrier) : cocycle L B α 0 = 1
theorem cocycle_sq (α β : L.carrier) : cocycle L B α β * cocycle L B α β = 1
theorem cocycle_cocycleLaw (α β γ : L.carrier) :
    cocycle L B α β * cocycle L B (α + β) γ = cocycle L B β γ * cocycle L B α (β + γ)
theorem cocExp_add_swap (α β : L.carrier) :
    cocExp L B α β + cocExp L B β α
      = L.form α β - ∑ i : Fin n, L.form (B i) (B i) * B.repr α i * B.repr β i
theorem cocycle_comm (hEven : L.Even) (α β : L.carrier) :
    cocycle L B α β * cocycle L B β α = (-1 : ℂ) ^ (L.form α β)
```

File `VOA/Foundations/LatticeCocycle.lean`. **Hypotheses / status.** Unconditional

Definition 31.18 (Grading of V_L). For $\alpha \in L$ let $|\alpha|^2/2 := \langle \alpha, \alpha \rangle/2$ (an integer when L is even). For $(w, \alpha) \in \mathbb{Z} \times L$ let $V_{(w,\alpha)} = (\mathcal{F})_w \otimes e^{\alpha}$ (zero if $w < 0$), and for $n \in \mathbb{Z}$ let $(V_L)_n = \sum_{\alpha} V_{(n-|\alpha|^2/2,\alpha)}$.

Lean. `VOA.LatGen.halfNorm`, `VOA.LatGen.bigradα`, `VOA.LatGen.gradingLatGen`

```
noncomputable def halfNorm (α : L.carrier) : ℤ := L.form α α / 2

noncomputable def bigradα (wα : ℤ × L.carrier) : Submodule ℂ (LatModuleGen L) :=
  (HeisenbergN.gradingZN (rank L) wα.1).map (mkEα L wα.2)

noncomputable def gradingLatGen (k : ℤ) : Submodule ℂ (LatModuleGen L) :=
  ⨆ α : L.carrier, bigradα L (k - halfNorm L α, α)
```

File `VOA/Examples/LatticeGenGrading.lean`.

Theorem 31.26 (The reconstructed vertex map). *Let L be even and positive-definite. There is a linear map $Y : V_L \to \mathrm{End}(V_L)[[z^{\pm 1}]]$ which equals (31.13) on every word state and is a local, T-covariant field map with the creation property.*

Lean. `VOA.LatGen.latYfullGen_of_locality`, `VOA.LatGen.YfullGen`

```
theorem latYfullGen_of_locality (hEven : L.Even) (hPos : L.PosDef) :
    ∃ Yfull : LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)),
      (∀ (lα : List (ℕ × Fin (rank L)) × L.carrier),
        Yfull (Function.uncurry (wordStateLatGen L) lα)
          = Function.uncurry (fun l α => YwordLatGen L l α) lα) ∧
      (∀ u : LatModuleGen L, IsField (Yfull u)) ∧
      (∀ u v : LatModuleGen L, Local (Yfull u) (Yfull v)) ∧
      (∀ u : LatModuleGen L, (Yfull u) 0 (latVacuumGen L) = u) ∧
      (∀ u : LatModuleGen L, ∀ q : ℤ, q < 0 → (Yfull u) q (latVacuumGen L) = 0) ∧
      (∀ u : LatModuleGen L, Reconstruction.DCovariant (TrecGen L) (Yfull u))

noncomputable def YfullGen (hEven : L.Even) (hPos : L.PosDef) :
    LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)) :=
  (latYfullGen_of_locality L hEven hPos).choose
```

Files `VOA/Examples/LatticeGenWordFields.lean`, `VOA/Examples/LatticeGenVOAInstance.lean`. **Hypotheses / status.** E, P

Theorem 31.27 (The lattice vertex operator algebra). *For every even positive-definite lattice L of rank r, V_L with the vertex map of Theorem 31.26, vacuum $1 \otimes e^0$, conformal vector $\omega = \frac{1}{2}\sum_a h_a(-1)^2\mathbb{1}$ and the grading of Definition 31.18 is a vertex operator algebra of CFT type with central charge r.*

Lean. `VOA.LatGen.latticeVOAGen`, `VOA.LatGen.latticeVOAGen_centralCharge`

```
noncomputable def latticeVOAGen (hEven : L.Even) (hPos : L.PosDef) :
    VertexOperatorAlgebra (LatModuleGen L) where
  grading := gradingLatGen L
  isInternal := gradingLatGen_isInternal_of_posDef L hEven hPos
  finrank := gradingLatGen_finrank_of_posDef L hEven hPos
  Y := YfullGen L hEven hPos
  vacuum := latVacuumGen L
  vacuum_mem := by
    rw [gradingLatGen_zero_eq_span_vacuum L hEven hPos]
    exact Submodule.mem_span_singleton_self (latVacuumGen L)
  grading_mode := fun u hu v hv j => YfullGen_grading_mode L hEven hPos u hu v hv j
  vacuum_Y := YfullGen_vacuum L hEven hPos
```

```
  creation := YfullGen_creation L hEven hPos
  locality := YfullGen_local L hEven hPos
  regularity := YfullGen_regularity L hEven hPos
  conformalVector := omegaLatGen L
  conformal_mem := omegaLatGen_mem L
  centralCharge := (rank L : ℂ)
  virasoro := YfullGen_virasoro L hEven hPos
  L0_grading := fun v hv => YfullGen_L0_grading L hEven hPos v hv
  translation := YfullGen_translation_omega L hEven hPos
  vacuum_grade := gradingLatGen_zero_eq_span_vacuum L hEven hPos
  positivity := fun n hn => gradingLatGen_positivity L hEven hPos n hn

theorem latticeVOAGen_centralCharge (hEven : L.Even) (hPos : L.PosDef) :
    (latticeVOAGen L hEven hPos).centralCharge = (rank L : ℂ) := rfl
```

File `VOA/Examples/LatticeGenInstance.lean`. **Hypotheses / status.** E, P

Theorem 31.39 (Functoriality). *Let K, L be even positive-definite lattices and $\iota\colon K \to L$ an isometry. Then*

(i) *Φ_ι is a homomorphism of vertex algebras $V_K \to V_L$ (preserves $\mathbb{1}$ and all products $u_{(n)}v$);*

(ii) *if $\operatorname{rank} K = \operatorname{rank} L$, $\Phi_\iota(\omega_K) = \omega_L$ and Φ_ι is injective, with image the set of $x \in V_L$ all of whose momenta lie in $\iota(K)$;*

(iii) *$\Phi_\iota \theta_K = \theta_L \Phi_\iota$;*

(iv) *$\Phi_\iota g_{\chi\circ\iota} = g_\chi \Phi_\iota$ for every $\chi\colon L \to \mathbb{Z}/2$.*

Lean. `VOA.LatGen.latMorph_isVOAHom`, `VOA.LatGen.latMorph_mode`, `VOA.LatGen.latMorph_omega`, `VOA.LatGen.latMorph_injective`, `VOA.LatGen.mem_range_latMorph_iff`, `VOA.LatGen.latMorph_thetaLat`, `VOA.LatGen.latMorph_chiLat`

```
theorem latMorph_isVOAHom :
    IsVOAHom (latticeVOAGen K hKe hK) (latticeVOAGen L hLe hL) (latMorph K L ι)
theorem latMorph_mode (u : LatModuleGen K) (n : ℤ) (v : LatModuleGen K) :
    latMorph K L ι (mode (YfullGen K hKe hK u) n v)
      = mode (YfullGen L hLe hL (latMorph K L ι u)) n (latMorph K L ι v)
theorem latMorph_omega : latMorph K L ι (omegaLatGen K) = omegaLatGen L
theorem latMorph_injective : Function.Injective (latMorph K L ι)
theorem mem_range_latMorph_iff (x : LatModuleGen L) :
    x ∈ LinearMap.range (latMorph K L ι) ↔ ∀ γ, γ ∉ Set.range ι → coordEα L γ x = 0
theorem latMorph_thetaLat (x : LatModuleGen K) :
    latMorph K L ι (thetaLat K x) = thetaLat L (latMorph K L ι x)
theorem latMorph_chiLat (χ : L.carrier →+ ZMod 2) (x : LatModuleGen K) :
    latMorph K L ι (chiLat K (χ.comp ι.toAddMonoidHom) x) = chiLat L χ (latMorph K L ι x)
```

File `VOA/Moonshine/LatticeMorph.lean`. **Hypotheses / status.** E, P (both), I

Theorem 31.42 (Rigidity). *Let L be even positive-definite and $\psi \in \operatorname{Aut}(V_L)$ such that $\psi(\alpha(-1)\mathbb{1}) = -\alpha(-1)\mathbb{1}$ for all $\alpha \in L$ (equivalently $\psi(h_a(-1)\mathbb{1}) = -h_a(-1)\mathbb{1}$ for all a) and $\psi(\mathcal{F} \otimes e^{\gamma}) \subseteq \mathcal{F} \otimes e^{-\gamma}$ for all γ. Then $\psi = g_d\theta$ for a character $d\colon L \to \mathbb{C}^\times$, and $\psi = g_e \theta g_e^{-1}$ for a character e; in particular ψ is conjugate to θ in $\operatorname{Aut}(V_L)$.*

Lean. `VOA.LatGen.exists_char_of_lift`, `VOA.LatGen.lift_eq_conj_theta`, `VOA.LatGen.lift_eq_conj_theta'`, `VOA.Aut.pullback`, `VOA.Aut.pullback_apply`

```
theorem exists_char_of_lift (ψ : Aut (latticeVOAGen L hEven hPos))
    (hh : ∀ a, ψ (heisSt L a) = -heisSt L a)
    (hsec : ∀ (γ : L.carrier) (x : LatModuleGen L), x ∈ chargeSec L γ → ψ x ∈ chargeSec L
     ↪ (-γ)) :
    ∃ d : L.carrier →+ Additive ℂˣ, ∀ x, ψ x = charLat L d (thetaLat L x)

theorem lift_eq_conj_theta (ψ : Aut (latticeVOAGen L hEven hPos))
    (hh : ∀ a, ψ (heisSt L a) = -heisSt L a)
    (hsec : ∀ (γ : L.carrier) (x : LatModuleGen L), x ∈ chargeSec L γ → ψ x ∈ chargeSec L
     ↪ (-γ)) :
    ∃ e : L.carrier →+ Additive ℂˣ, ∀ x, ψ x = charLat L e (thetaLat L (charLat L (-e) x))

theorem lift_eq_conj_theta' (ψ : Aut (latticeVOAGen L hEven hPos))
    (hh : ∀ α, ψ (hSt L α) = -hSt L α)
    (hsec : ∀ (γ : L.carrier) (x : LatModuleGen L), x ∈ chargeSec L γ → ψ x ∈ chargeSec L
     ↪ (-γ)) :
    ∃ e : L.carrier →+ Additive ℂˣ, ∀ x, ψ x = charLat L e (thetaLat L (charLat L (-e) x))
```

```
noncomputable def Aut.pullback (hΦ : IsVOAHom V W Φ)
    (hω : Φ V.conformalVector = W.conformalVector) : Aut V
theorem Aut.pullback_apply (hΦ : IsVOAHom V W Φ)
    (hω : Φ V.conformalVector = W.conformalVector) (x : 𝒱) :
    Φ (Aut.pullback Φ hinj g hg hΦ hω x) = g (Φ x)
```

File `VOA/Moonshine/LatticeMorphRigid.lean`. **Hypotheses / status.** E, P, two hyps on ψ

Chapter 32

The involution θ and the fixed-point algebra V_Λ^+

6 of the chapter's 37 numbered statements: its headline results and every definition they use.

Proposition 32.9. *For every integral lattice L and all $\alpha, \beta \in L$: $\varepsilon(-\alpha,\beta) = \varepsilon(\alpha,\beta) = \varepsilon(\alpha,-\beta)$; hence $\varepsilon(-\alpha,-\beta) = \varepsilon(\alpha,\beta)$.*

Lean. `VOA.LatGen.latticeCocycle_neg_left`, `VOA.LatGen.latticeCocycle_neg_right`, `VOA.LatGen.latticeCocycle_neg_neg`

```
-- VOA/Moonshine/LatticeInvolution.lean
theorem latticeCocycle_neg_left (α β : L.carrier) :
    latticeCocycle L (-α) β = latticeCocycle L α β
theorem latticeCocycle_neg_right (α β : L.carrier) :
    latticeCocycle L α (-β) = latticeCocycle L α β
theorem latticeCocycle_neg_neg (α β : L.carrier) :
    latticeCocycle L (-α) (-β) = latticeCocycle L α β
```

File `VOA/Moonshine/LatticeInvolution.lean`. **Hypotheses / status.** None

Theorem 32.13 (θ intertwines the vertex operators). *Let L be even and positive-definite. Then for all $u, v \in V_L$ and $n \in \mathbb{Z}$,*

$$\theta\big(u_{(n)}v\big) = (\theta u)_{(n)}(\theta v),$$

equivalently (32.2).

Lean. `VOA.LatGen.thetaLat_YfullGen`, `VOA.LatGen.thetaLat_mode`

```
-- VOA/Moonshine/LatticeInvolution.lean
theorem thetaLat_YfullGen (u : LatModuleGen L) (t : ℤ) (x : LatModuleGen L) :
    thetaLat L (YfullGen L hEven hPos u t x)
      = YfullGen L hEven hPos (thetaLat L u) t (thetaLat L x)

theorem thetaLat_mode (u : LatModuleGen L) (n : ℤ) (v : LatModuleGen L) :
    thetaLat L (mode (YfullGen L hEven hPos u) n v)
      = mode (YfullGen L hEven hPos (thetaLat L u)) n (thetaLat L v)
```

File `VOA/Moonshine/LatticeInvolution.lean`. **Hypotheses / status.** L even, pos.-definition

Theorem 32.29 (Fock generating functions). *In the ring $\mathbb{C}[[q]]$ of formal power series,*

$$\sum_{w\ge 0} P(r,w)q^w = \prod_{n\ge 1}(1-q^n)^{-r}, \qquad \sum_{w\ge 0}\big(P^+(r,w)-P^-(r,w)\big)q^w = \prod_{n\ge 1}(1+q^n)^{-r},$$

the infinite products converging in the q-adic (coefficientwise) topology.

Lean. `VOA.LatGen.fockGenFun`, `VOA.LatGen.fockSignedGenFun`, `VOA.LatGen.hasProd_fockGenFun`, `VOA.LatGen.hasProd_fockSignedGenFun`

```
-- VOA/Moonshine/LatticeGradedDim.lean
noncomputable def fockGenFun (r : ℕ) : PowerSeries ℂ := PowerSeries.mk fun w => (fockCount
 ↪  r w : ℂ)
noncomputable def fockSignedGenFun (r : ℕ) : PowerSeries ℂ :=
  PowerSeries.mk fun w => (fockSigned r w : ℂ)

open scoped PowerSeries.WithPiTopology in
theorem hasProd_fockGenFun (r : ℕ) :
    HasProd (fun i : ℕ => ((1 - (PowerSeries.X : ℂ⟦X⟧)) ^ (i + 1))⁻¹) ^ r) (fockGenFun r)

open scoped PowerSeries.WithPiTopology in
theorem hasProd_fockSignedGenFun (r : ℕ) :
    HasProd (fun i : ℕ => ((1 + (PowerSeries.X : ℂ⟦X⟧)) ^ (i + 1))⁻¹) ^ r)
      (fockSignedGenFun r)
```

File `VOA/Moonshine/LatticeGradedDim.lean`. **Hypotheses / status.** None

Theorem 32.31 (Graded dimension of V_L). *Let L be even and positive-definite of rank r. For every $k \in \mathbb{N}$,*

$$\dim(V_L)_k = \sum_{n=0}^{k} \#\{\alpha \in L : \langle \alpha, \alpha \rangle = 2n\} \cdot P(r, k-n),$$

the coefficient of q^k in $\Theta_L(q) \prod_{n \geq 1} (1-q^n)^{-r}$, where $\Theta_L(q) = \sum_{\alpha \in L} q^{\langle \alpha, \alpha \rangle / 2}$ is the theta series of L.

Lean. `VOA.LatGen.normEq`, `VOA.LatGen.mem_normEq`, `VOA.LatGen.finrank_gradingLatGen_eq_sum_theta`

```
-- VOA/Moonshine/LatticeGradedDim.lean
noncomputable def normEq (m : ℤ) : Finset L.carrier := (normEq_finite L hPos m).toFinset
theorem mem_normEq {m : ℤ} {α : L.carrier} : α ∈ normEq L hPos m ↔ L.form α α = m

theorem finrank_gradingLatGen_eq_sum_theta (hEven : L.Even) (k : ℕ) :
    finrank ℂ (gradingLatGen L k)
      = ∑ n ∈ Finset.range (k + 1), (normEq L hPos (2 * n)).card * fockCount (rank L) (k -
       ↪  n)
```

File `VOA/Moonshine/LatticeGradedDim.lean`. **Hypotheses / status.** Even, pos.-definition

Theorem 32.36 (Generation). *Let $S \subseteq V_\Lambda^+$ be a linear subspace such that $\mathbf{1} \in S$, $u_{(n)}v \in S$ for all $u, v \in S$ and $n \in \mathbb{Z}$, and $x_\beta \in S$ for every $\beta \in \Lambda$. Then $S = V_\Lambda^+$.*

Lean. `VOA.Moonshine.leechPlus_generated`

```
-- VOA/Moonshine/Generation.lean
theorem leechPlus_generated : ∀ S : Submodule ℂ VPlus, leechVOAPlus.vacuum ∈ S →
    (∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ, leechVOAPlus.smul u n v ∈ S) →
    (∀ β : leech.carrier, frameX0P β ∈ S) → S = ⊤
```

File `VOA/Moonshine/Generation.lean`. **Hypotheses / status.** None

Theorem 32.45 (Properties of the lattice form). *Let L be even. Then:*

(a) (invariance at e^γ) for all $\gamma \in L$, $n \in \mathbb{Z}$ and $x, y \in V_L$, with $N = \frac{1}{2}\langle \gamma, \gamma \rangle$,

$$B(e^\gamma_{(n)}x, y) = (-1)^N B(x, e^\gamma_{(2N-n-2)}y);$$

(b) (nondegeneracy) if $x \neq 0$ there is y with $B(x,y) \neq 0$;

(c) (θ-invariance) $B(\theta x, \theta y) = B(x,y)$;

(d) (orthogonality) distinct weight spaces of V_L are B-orthogonal.

Lean. `VOA.Moonshine.Nondeg.latForm_Yeαvec`, `VOA.Moonshine.Nondeg.latForm_nondeg`, `VOA.Moonshine.Nondeg.latForm_thetaLat`, `VOA.Moonshine.Nondeg.latForm_orth_grading`

```
-- VOA/Moonshine/NondegLattice.lean
theorem latForm_Yeαvec (hEven : L.Even) (γ : L.carrier) (n : ℤ) (x y : LatModuleGen L) :
    latForm L (mode (Yeαvec L γ) n x) y
      = (-1 : ℂ) ^ (LatGen.halfNorm L γ) *
          latForm L x (mode (Yeαvec L γ) (2 * LatGen.halfNorm L γ - n - 2) y)
theorem latForm_nondeg (x : LatModuleGen L) (hx : x ≠ 0) : ∃ y, latForm L x y ≠ 0
theorem latForm_thetaLat (x y : LatModuleGen L) :
    latForm L (thetaLat L x) (thetaLat L y) = latForm L x y
theorem latForm_orth_grading {m n : ℤ} (hmn : m ≠ n) {x y : LatModuleGen L}
    (hx : x ∈ gradingLatGen L m) (hy : y ∈ gradingLatGen L n) : latForm L x y = 0
```

File `VOA/Moonshine/NondegLattice.lean`. **Hypotheses / status.** L even

Chapter 33

Graded dimensions and the modular function J

8 of the chapter's 44 numbered statements: its headline results and every definition they use.

Theorem 33.5 (j is a Hauptmodul; argued on paper only). *Every $\mathrm{SL}_2(\mathbb{Z})$-invariant holomorphic function on $\mathfrak{H}$ with at most a pole of order m at the cusp is a polynomial of degree $\le m$ in j. In particular J is the unique invariant function of the form $q^{-1} + O(q)$.*

(The chapter quotes no Lean declaration for this statement; see book p. 1743.)

Definition 33.9 (Formal Δ). $\Delta = (E_4^3 - E_6^2)/1728 \in \mathbb{C}[[q]]$.

Lean. `VOA.deltaSeries`

```
noncomputable def deltaSeries : PowerSeries ℂ :=
  (1728 : ℂ)⁻¹ • (E4Series ^ 3 - E6Series ^ 2)
```

File `VOA/Modularity/JFunction.lean`.

Theorem 33.18 (Generating function of the twisted Fock space). *With $x = q^{1/2}$, in $\mathbb{C}[[x]]$,*

$$\sum_{d\ge 0} P_T(r,d)\,x^d = \prod_{i\ge 0}\bigl(1-x^{2i+1}\bigr)^{-r} \quad \Bigl(= \prod_{n\ge 1}\bigl(1-q^{n-\frac{1}{2}}\bigr)^{-r}\Bigr).$$

Lean. `VOA.TwistedSector.twGenFun`, `VOA.TwistedSector.hasProd_twGenFun`

```
noncomputable def twGenFun (r : ℕ) : PowerSeries ℂ := PowerSeries.mk fun d => (twCount r d
 ↪  : ℂ)

theorem hasProd_twGenFun (r : ℕ) :
    HasProd (fun i : ℕ => ((1 - (PowerSeries.X : ℂ⟦X⟧)) ^ (2 * i + 1))⁻¹) ^ r) (twGenFun r)
```

File `VOA/Moonshine/TwistedSectorDim.lean`. **Hypotheses / status.** None

Theorem 33.21 (Rank 24: fixed = integral weight, nothing below 2). *For $r = 24$ and any T: the fixed part is exactly the sum of the integer-weight eigenspaces; there are no fixed vectors of weight < 2; and $\dim(S[\theta]\otimes T)^+_{n+2} = P_T(24, 2n+1)\dim T$ for all $n \ge 0$.*

Lean. `VOA.TwistedSector.twFixed_eq_iSup_int`, `VOA.TwistedSector.twFixed_inf_twWeight_of_lt_two`, `VOA.TwistedSector.finrank_twFixed_inf_twWeight_24`
File `VOA/Moonshine/TwistedSectorDim.lean`.

```
theorem twFixed_eq_iSup_int : twFixed 24 T = ⨆ n : ℤ, twWeight 24 T n

theorem twFixed_inf_twWeight_of_lt_two {q : ℚ} (hq : q < 2) :
    twFixed 24 T ⊓ twWeight 24 T q = ⊥

theorem finrank_twFixed_inf_twWeight_24 (n : ℕ) :
    finrank ℂ (twFixed 24 T ⊓ twWeight 24 T (((n : ℤ) + 2 : ℤ) : ℚ) :
        Submodule ℂ (FockT 24 ⊗[ℂ] T))
      = twCount 24 (2 * n + 1) * finrank ℂ T
```

Hypotheses / status. None

Theorem 33.23 (Twisted dimensions in weights ≤ 3). $(V_\Lambda^T)_0^+ = (V_\Lambda^T)_1^+ = 0$, $\dim(V_\Lambda^T)_2^+ = 24 \cdot 4096 = 98304$, $\dim(V_\Lambda^T)_3^+ = 2624 \cdot 4096 = 10747904$. *Consequently*

$$\dim(V_\Lambda^+)_2 + \dim(V_\Lambda^T)_2^+ = 98580 + 98304 = 196884,$$
$$\dim(V_\Lambda^+)_3 + \dim(V_\Lambda^T)_3^+ = 10745856 + 10747904 = 21493760.$$

Lean. `VOA.Moonshine.leechTwisted_finrank_fixed_two`, `VOA.Moonshine.leechTwisted_finrank_fixed_three`, `VOA.Moonshine.leechTwisted_fixed_weight_zero_one`, `VOA.Moonshine.finrank_sectors_two`, `VOA.Moonshine.finrank_sectors_three`

```
theorem leechTwisted_finrank_fixed_two :
    finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 2 : Submodule ℂ leechTwistedSpace)
      = 98304

theorem leechTwisted_finrank_fixed_three :
    finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 3 : Submodule ℂ leechTwistedSpace)
      = 10747904

theorem leechTwisted_fixed_weight_zero_one :
    twFixed 24 leechT ⊓ twWeight 24 leechT 0 = ⊥ ∧ twFixed 24 leechT ⊓ twWeight 24 leechT 1
     ↪  = ⊥

theorem finrank_sectors_two :
    finrank ℂ (leechVOAPlus.grading 2)
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 2 : Submodule ℂ
       ↪  leechTwistedSpace)
      = 196884

theorem finrank_sectors_three :
    finrank ℂ (leechVOAPlus.grading 3)
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 3 : Submodule ℂ
       ↪  leechTwistedSpace)
      = 21493760
```

File `VOA/Moonshine/TwistedSectorDim.lean`. **Hypotheses / status.** None

Theorem 33.35 (Jacobi's product, formally). $\Delta = q\,D$ *in* $\mathbb{C}[[q]]$, *for the formal* $\Delta = (E_4^3 - E_6^2)/1728$; *equivalently* $\Delta/q = D$, *and (A):* $(\Delta/q) \cdot \prod (1-q^n)^{-24} = 1$.

Lean. `VOA.Moonshine.deltaSeries_eq_X_mul_deltaProd`, `VOA.Moonshine.deltaOverQ_eq_deltaProd`, `VOA.Moonshine.deltaProductIdentity`

```
theorem deltaSeries_eq_X_mul_deltaProd : deltaSeries = X * deltaProd

theorem deltaOverQ_eq_deltaProd : deltaOverQ = deltaProd

theorem deltaProductIdentity : DeltaProductIdentity
```

File `VOA/Moonshine/CharacterDelta.lean`. **Hypotheses / status.** None

Theorem 33.36 (Weight-12 forms are determined by two coefficients). *For every $f \in M_{12}$ with q-expansion $\sum a_n q^n$: $\sum a_n q^n = a_0 E_4^3 + \frac{a_1 - 720 a_0}{1728}(E_4^3 - E_6^2)$ in $\mathbb{C}[[q]]$.*

Lean. `VOA.Moonshine.qExpansion_weight_twelve`

```
theorem qExpansion_weight_twelve (f : ModularForm 𝒮ℒ ((12 : ℕ) : ℤ)) :
    qExpansion 1 (f : ℍ → ℂ)
      = (qExpansion 1 (f : ℍ → ℂ)).coeff 0 • E4Series ^ 3
        + (((qExpansion 1 (f : ℍ → ℂ)).coeff 1 - 720 * (qExpansion 1 (f : ℍ → ℂ)).coeff 0)
        ↪  / 1728)
          • (E4Series ^ 3 - E6Series ^ 2)
```

File `VOA/Moonshine/CharacterDelta.lean`. **Hypotheses / status.** $f \in M_{12}$

Theorem 33.47 ($\chi = qJ$). *In $\mathbb{C}[[q]]$,*

$$\sum_{n \geq 0} \Big(\dim (V_\Lambda^+)_n + \dim (V_\Lambda^T)_n^+ \Big) q^n = q\, J(q),$$

equivalently, for every $n \in \mathbb{N}$, $\dim (V_\Lambda^+)_n + \dim (V_\Lambda^T)_n^+ = [q^n]\, qJ$.

Lean. `VOA.Moonshine.sectorSeries_eq_JShift`, `VOA.Moonshine.finrank_sectors_eq_coeff_JShift`

```
theorem sectorSeries_eq_JShift : sectorSeries = JShift :=
  sectorSeries_eq_JShift_of_levelTwo levelTwoIdentity

theorem finrank_sectors_eq_coeff_JShift (n : ℕ) :
    ((finrank ℂ (leechVOAPlus.grading (n : ℤ))
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT ((n : ℤ) : ℚ) :
          Submodule ℂ leechTwistedSpace) : ℕ) : ℂ)
      = coeff n JShift
```

File `VOA/Moonshine/Character.lean`. **Hypotheses / status.** None

Chapter 34

Twisted modules, the twisted Fock space and the module T

6 of the chapter's 27 numbered statements: its headline results and every definition they use.

Proposition 34.6. *Every ordinary V-module is a 1-twisted module with $N = 1$, with $u_{(k/1)} = u_{(k)}$ and the same grading.*

Lean. `VOA.OrdVModule.toTwisted`
File `VOA/Modularity/Twisted.lean`.

```
noncomputable def OrdVModule.toTwisted {M : Type u} [AddCommGroup M] [Module ℂ M]
    (Nm : OrdVModule W M) : TwistedModule (1 : Aut W) 1 M where
```

Hypotheses / status. Unconditional

Theorem 34.8 (Dead-end certificate). *Let $N \geq 2$, and let $Y_M : V \to (\mathbb{Z} \to \mathrm{End}\, M)$ be linear with $\mathbf{1}_{(k/N)} = \delta_{k,-N}$. If* JacobiAllIndices$(g, N, Y_M)$ *holds, then $M = 0$: every $x \in M$ is zero.*

Lean. `VOA.jacobiAllIndices_eq_zero`
File `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`.

```
theorem jacobiAllIndices_eq_zero {g : Aut W} {N : ℕ} (hN : 2 ≤ N) {M : Type*}
    [AddCommGroup M] [Module ℂ M] (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M))
    (hvac : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0)
    (hJ : JacobiAllIndices g N YM) (x : M) : x = 0
```

Hypotheses / status. $N \geq 2$, vacuum

Theorem 34.23 (Dimension of the simple module). *Under the same hypothesis, $\mathbb{C}^c[V] \cong M_k(\mathbb{C})$ with $k^2 = 2^n$; hence n is even and $k = 2^{n/2}$. Put $T = \mathbb{C}^k$ with $\mathbb{C}^c[V]$ acting through this isomorphism. Then T is a simple module of dimension $2^{n/2}$, and every simple $\mathbb{C}^c[V]$-module is isomorphic to T.*

Lean. `VOA.Moonshine.Extraspecial.repDim`, `VOA.Moonshine.Extraspecial.repDim_eq`, `VOA.Moonshine.Extraspecial.Rep`, `VOA.Moonshine.Extraspecial.repEquiv`, `VOA.Moonshine.Extraspecial.finrank_rep`, `VOA.Moonshine.Extraspecial.isSimpleModule_rep`, `VOA.Moonshine.Extraspecial.nonempty_linearEquiv_rep`, `VOA.Moonshine.Extraspecial.even_of_isUnit`

```
noncomputable def repDim (hN : IsUnit (M + Mᵀ).det) : ℕ := (exists_matrix M hN).choose
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem repDim_eq (hN : IsUnit (M + Mᵀ).det) : repDim M hN = 2 ^ (n / 2)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
def Rep (hN : IsUnit (M + Mᵀ).det) : Type := Fin (repDim M hN) → ℂ
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
noncomputable def repEquiv (hN : IsUnit (M + Mᵀ).det) : alg M ≃ₐ[ℂ] Module.End ℂ (Rep M hN)
 ↪  :=
  (exists_matrix M hN).choose_spec.2.some.trans Matrix.toLinAlgEquiv'
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem finrank_rep (hN : IsUnit (M + Mᵀ).det) :
    Module.finrank ℂ (Rep M hN) = 2 ^ (n / 2)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
instance isSimpleModule_rep (hN : IsUnit (M + Mᵀ).det) : IsSimpleModule (alg M) (Rep M hN)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem nonempty_linearEquiv_rep (hN : IsUnit (M + Mᵀ).det) (S : Type*) [AddCommGroup S]
    [Module (alg M) S] [IsSimpleModule (alg M) S] :
    Nonempty (S ≃ₗ[alg M] Rep M hN)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem even_of_isUnit (hN : IsUnit (M + Mᵀ).det) : Even n
```

Hypotheses / status. N

Theorem 34.24. *(a)* red $:\ L \to \mathbb{F}_2^n$ *is surjective with kernel* $2L$*, so it identifies* $L/2L$ *with* $\mathbb{F}_2^n$*. (b)* $\varepsilon(\alpha,\beta) = c(\bar\alpha,\bar\beta)$ *with* c *built from* `cocMat`*; in particular* ε *depends only on classes mod* $2L$*. (c) If* L *is even and unimodular, the commutator form on* $L/2L$ *is nondegenerate.*

Lean. `VOA.IntegralLattice.red_surjective`, `VOA.IntegralLattice.red_eq_zero_iff`, `VOA.IntegralLattice.latticeCocycle_eq_coc`, `VOA.IntegralLattice.latticeCocycle_add_two_nsmul`, `VOA.IntegralLattice.isUnit_det_cocMat`
File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem red_surjective : Function.Surjective (red L)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem red_eq_zero_iff (α : L.carrier) : red L α = 0 ↔ ∃ γ, α = 2 • γ
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem latticeCocycle_eq_coc (α β : L.carrier) :
    latticeCocycle L α β = coc (cocMat L) (red L α) (red L β)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem latticeCocycle_add_two_nsmul (α β γ : L.carrier) :
    latticeCocycle L (α + 2 • γ) β = latticeCocycle L α β ∧
      latticeCocycle L α (β + 2 • γ) = latticeCocycle L α β
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem isUnit_det_cocMat (hEven : L.Even) (hU : IsUnit (gramMat L).det) :
    IsUnit (cocMat L + (cocMat L)ᵀ).det
```

Hypotheses / status. (E, U)

Theorem 34.25. *Let L be even unimodular and T_L the simple $\mathbb{C}^\varepsilon[L/2L]$-module. Then for all $\alpha, \beta, \gamma \in L$: $e_\alpha e_\beta = \varepsilon(\alpha,\beta) e_{\alpha+\beta}$, $e_0 = 1$, $e_{-\alpha} = e_\alpha$, $e_{\alpha+2\gamma} = e_\alpha$ on T_L; and* $\dim T_L = 2^{\operatorname{rank} L/2}$. *For the Leech lattice,* $\dim T = 4096$.

Lean. `VOA.IntegralLattice.eT_mul`, `VOA.IntegralLattice.eT_zero`, `VOA.IntegralLattice.eT_neg`, `VOA.IntegralLattice.eT_add_two_nsmul`, `VOA.IntegralLattice.finrank_extraT`, `VOA.Moonshine.leechT`, `VOA.Moonshine.finrank_leechT`
File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_mul (α β : L.carrier) :
    eT L hEven hU α * eT L hEven hU β = latticeCocycle L α β • eT L hEven hU (α + β)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_zero : eT L hEven hU 0 = 1
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_neg (α : L.carrier) : eT L hEven hU (-α) = eT L hEven hU α
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_add_two_nsmul (α γ : L.carrier) : eT L hEven hU (α + 2 • γ) = eT L hEven hU α
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem finrank_extraT : Module.finrank ℂ (ExtraT L hEven hU) = 2 ^ (L.rank / 2)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable abbrev leechT : Type := ExtraT leech leech_even leech_isUnit_det_gramMat
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem finrank_leechT : Module.finrank ℂ leechT = 4096
```

Hypotheses / status. E, U

Theorem 34.30. *For a quadratic sign s and $u \in T^*$: B_u is symmetric; $B_u(Xt, t') = B_u(t, \iota(X)t')$ for all $X \in \operatorname{End}(T)$; and if $B_u \neq 0$ then B_u is nondegenerate. Consequently, if there is an idempotent $P \in \operatorname{End}(T)$ with $\iota(P) = P$ and $\operatorname{tr} P = 1$, then T carries a nondegenerate symmetric bilinear form B with $B(Xt, t') = B(t, \iota(X)t')$ for all X.*

Lean. `VOA.Moonshine.Extraspecial.formOf_symm`, `VOA.Moonshine.Extraspecial.formOf_invariant`, `VOA.Moonshine.Extraspecial.formOf_nondegenerate`, `VOA.Moonshine.Extraspecial.exists_form_of_idempotent`
File `VOA/Moonshine/TFormAlg.lean`.

```
theorem formOf_symm {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (u : Module.Dual ℂ (Rep M hN)) (t t' : Rep M hN) :
    formOf M hN s u t t' = formOf M hN s u t' t
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem formOf_invariant {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (u : Module.Dual ℂ (Rep M hN)) (X : Module.End ℂ (Rep M hN)) (t t' : Rep M hN) :
    formOf M hN s u (X t) t' = formOf M hN s u t (iotaT M hN s X t')
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem formOf_nondegenerate {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (u : Module.Dual ℂ (Rep M hN)) {v w : Rep M hN} (hvw : formOf M hN s u v w ≠ 0) :
    (formOf M hN s u).Nondegenerate
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem exists_form_of_idempotent {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    {P : Module.End ℂ (Rep M hN)} (hP : IsIdempotentElem P) (hιP : iotaT M hN s P = P)
    (htr : trace ℂ (Rep M hN) P = 1) :
    ∃ B : LinearMap.BilinForm ℂ (Rep M hN), (∀ t t', B t t' = B t' t) ∧ B.Nondegenerate ∧
      ∀ X t t', B (X t) t' = B t (iotaT M hN s X t')
```

Hypotheses / status. N, Q, *P*

Chapter 35

The twisted Heisenberg module

9 of the chapter's 41 numbered statements: its headline results and every definition they use.

Definition 35.4 (Twisted eigenspaces; twisted module). Let W be a VOA on $\mathcal{V}$ and $g \in \mathrm{Aut}(W)$. For $N \geq 1$ and $j \in \mathbb{N}$ put $\mathcal{V}^j = \{v : gv = e^{-2\pi i j/N} v\}$. A g-twisted module of order N on M is a linear map $u \mapsto (u_{(k/N)})_{k\in\mathbb{Z}}$, $\mathcal{V} \to (\mathbb{Z} \to \mathrm{End}\, M)$, with: (i) $\mathbf{1}_{(k/N)} = \delta_{k,-N}\,\mathrm{id}$; (ii) lower truncation; (iii) for $u \in \mathcal{V}^j$, $u_{(k/N)} = 0$ unless $k \equiv -j \pmod N$; (iv) for $u \in \mathcal{V}^j$, $v \in \mathcal{V}$, $x \in M$, $\ell, m \in \mathbb{Z}$ and $k \equiv -j \pmod N$,

$$\sum_{i\in\mathbb{Z}} (-1)^i \binom{\ell}{i} \Big(u_{(\ell+\frac{k}{N}-i)}\, v_{(\frac{m}{N}+i)} x - (-1)^{\ell} v_{(\ell+\frac{m}{N}-i)}\, u_{(\frac{k}{N}+i)} x \Big) = \sum_{i\geq 0} \binom{k/N}{i} (u_{(\ell+i)}v)_{(\frac{k+m}{N}-i)} x; \quad (35.1)$$

(v) a direct-sum decomposition $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$ with $\omega_{(1)} = L_0$ acting on M_α by α, each M_α finite-dimensional, and $\mathrm{Re}\,\alpha$ bounded below on the nonzero M_α.

Lean. `VOA.Aut.twistedEigenspace`, `VOA.TwistedModule`, `VOA.TwistedJacobi`
File `VOA/Modularity/Twisted.lean`.

```
noncomputable def Aut.twistedEigenspace (g : Aut W) (N : ℕ) (j : ℕ) : Submodule ℂ 𝒱 :=
  Module.End.eigenspace (g.toLinearEquiv : Module.End ℂ 𝒱)
    (Complex.exp (-(2 * (Real.pi : ℂ) * Complex.I * j) / N))

structure TwistedModule (g : Aut W) (N : ℕ) (M : Type u) [AddCommGroup M] [Module ℂ M]
 ↪  where
  YM : 𝒱 →ₗ[ℂ] (ℤ → Module.End ℂ M)
  vacuum_YM : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0
  truncation : ∀ (u : 𝒱) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0
  support : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j →
    ∀ k : ℤ, (k + j) % (N : ℤ) ≠ 0 → YM u k = 0
  jacobi : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (x : M) (ℓ k m : ℤ),
    (k + j) % (N : ℤ) = 0 →
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → YM W.conformalVector N x = α • x
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
```

```
def TwistedJacobi (g : Aut W) (N : ℕ) {M : Type*} [AddCommGroup M] [Module ℂ M]
```

```
    (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M)) : Prop :=
  ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (x : M) (ℓ k m : ℤ),
    (k + j) % (N : ℤ) = 0 →
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
```

Theorem 35.5. *For every $r \in \mathbb{N}$ and level $k \neq 0$, σ is an automorphism of the Heisenberg VOA* `heisenbergNVOA r k hk`*: it is a linear bijection with $\sigma(u_{(n)}v) = (\sigma u)_{(n)}(\sigma v)$ for all u, v, n, $\sigma\mathbf{1} = \mathbf{1}$ and $\sigma\omega = \omega$. Moreover $\sigma^2 = 1$.*

Lean. `VOA.Aut`, `VOA.HeisenbergN.sigmaHeisAut`, `VOA.HeisenbergN.sigmaHeisAut_sq`, `VOA.HeisenbergN.heisVOA1`, `VOA.HeisenbergN.sigma1`
Files `VOA/Core/Automorphism.lean`, `VOA/Moonshine/HeisenbergTwistedModuleSigma.lean`.

```
structure Aut (W : VertexOperatorAlgebra 𝒱) where
  toLinearEquiv : 𝒱 ≃ₗ[ℂ] 𝒱
  map_smul' : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱),
    toLinearEquiv (W.smul u n v) = W.smul (toLinearEquiv u) n (toLinearEquiv v)
  map_vacuum : toLinearEquiv W.vacuum = W.vacuum
  map_omega : toLinearEquiv W.conformalVector = W.conformalVector

noncomputable def sigmaHeisAut (k : ℂ) (hk : k ≠ 0) : Aut (heisenbergNVOA r k hk) where
  toLinearEquiv := sigmaN r
  -- (the three proof fields map_smul', map_vacuum, map_omega are elided)
theorem sigmaHeisAut_sq (k : ℂ) (hk : k ≠ 0) : sigmaHeisAut r k hk * sigmaHeisAut r k hk =
 ↪ 1

noncomputable abbrev heisVOA1 : VertexOperatorAlgebra (FockN r) :=
  heisenbergNVOA r 1 one_ne_zero
noncomputable abbrev sigma1 : Aut (heisVOA1 r) := sigmaHeisAut r 1 one_ne_zero
```

Lemma 35.8 (Complex Chu–Vandermonde). *For $x, y \in \mathbb{C}$ and $s \in \mathbb{N}$, $\binom{x+y}{s} = \sum_{i+t=s} \binom{x}{i}\binom{y}{t}$.*

Lean. `VOA.TwistedReduction.cbinom_add`

```
theorem cbinom_add (x y : ℂ) (s : ℕ) :
    cbinom (x + y) s = ∑ ij ∈ Finset.antidiagonal s, cbinom x ij.1 * cbinom y ij.2
```

File `VOA/Moonshine/HeisenbergTwistedModuleReduction.lean`.

Theorem 35.10 (Twisted Jacobi from locality and the product formula). *Let Y_M be lower truncated, $u, v \in \mathcal{V}$, $e, e' \in \mathbb{Z}$. Suppose $\widetilde{F}_e(u)$ and $\widetilde{F}_{e'}(v)$ are mutually local and satisfy the twisted product formula (35.7) at every $\ell' \geq \ell$ (resp. every ℓ'). Then for all x and all $m, n \in \mathbb{Z}$ the twisted Jacobi identity (35.4) holds for (u, v) at ℓ (resp. every ℓ), with $k = Nm + e$ and the mode index of v equal to $Nn + e'$.*

Lean. `VOA.TwistedReduction.twisted_jacobi_of_local_pf_ge`, `VOA.TwistedReduction.twisted_jacobi_of_local_pf`

```
theorem twisted_jacobi_of_local_pf_ge (W : VertexOperatorAlgebra 𝒱)
    (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M))
    {N : ℕ} (hN : 0 < N) (htr : ∀ (u : 𝒱) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0)
    (u v : 𝒱) (e e' : ℤ)
    (hloc : Local (resc (YM u) N e) (resc (YM v) N e')) (ℓ : ℤ)
    (hpf : ∀ (ℓ' s : ℤ) (x : M), ℓ ≤ ℓ' →
```

```
      mode (Fld.prod ℓ' (rescFld hN (htr u) e) (rescFld hN (htr v) e')).1 s x
        = ∑ᶠ t : ℕ, cbinom ((e : ℂ) / N) t •
            YM (W.smul u (ℓ' + t) v) (N * (s - t) + e + e') x)
    (x : M) (m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + (N * m + e) - i * N) (YM v ((N * n + e') + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + (N * n + e') - i * N) (YM u ((N * m + e) + i * N)
            ↪ x)))
      = ∑ᶠ i : ℕ, cbinom (((N * m + e : ℤ) : ℂ) / N) i •
          YM (W.smul u (ℓ + i) v) ((N * m + e) + (N * n + e') - i * N) x

theorem twisted_jacobi_of_local_pf (W : VertexOperatorAlgebra V) (YM : V →ₗ[ℂ] (ℤ → End ℂ
↪  M))
    {N : ℕ} (hN : 0 < N) (htr : ∀ (u : V) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0)
    (u v : V) (e e' : ℤ)
    (hloc : Local (resc (YM u) N e) (resc (YM v) N e'))
    (hpf : ∀ (ℓ s : ℤ) (x : M),
      mode (Fld.prod ℓ (rescFld hN (htr u) e) (rescFld hN (htr v) e')).1 s x
        = ∑ᶠ t : ℕ, cbinom ((e : ℂ) / N) t •
            YM (W.smul u (ℓ + t) v) (N * (s - t) + e + e') x)
    (x : M) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + (N * m + e) - i * N) (YM v ((N * n + e') + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + (N * n + e') - i * N) (YM u ((N * m + e) + i * N)
            ↪ x)))
      = ∑ᶠ i : ℕ, cbinom (((N * m + e : ℤ) : ℂ) / N) i •
          YM (W.smul u (ℓ + i) v) ((N * m + e) + (N * n + e') - i * N) x
```

File `VOA/Moonshine/HeisenbergTwistedModuleReduction.lean`.

Theorem 35.15 (The generator commutator).

$$[F_a(z_1), F_b(z_2)] = \delta_{ab}\Big(z_2\,\partial_{z_2}\delta(z_1,z_2) + \tfrac{1}{2}\,\delta(z_1,z_2)\Big).$$

Consequently F_a, F_b are mutually local (of order 2), and $F_{a(0)}F_b = \frac{1}{2}\delta_{ab}\,1$, $F_{a(1)}F_b = \delta_{ab}\,z$, $F_{a(\ell)}F_b = 0$ for $\ell \geq 2$.

Lean. `VOA.Moonshine.opeOne`, `VOA.Moonshine.opeZero`, `VOA.Moonshine.bracketDist_Fgen`, `VOA.Moonshine.Fgen_local`, `VOA.Moonshine.prod_Fgen_zero`, `VOA.Moonshine.prod_Fgen_one`, `VOA.Moonshine.prod_Fgen_of_two_le`

```
noncomputable def opeOne (a b : Fin r) : FormalDist (End ℂ (FockT r)) :=
  fun t => if t = 1 then (if a = b then (1 : ℂ) else 0) • LinearMap.id else 0
noncomputable def opeZero (a b : Fin r) : FormalDist (End ℂ (FockT r)) :=
  fun t => if t = 0 then (if a = b then (1 / 2 : ℂ) else 0) • LinearMap.id else 0

theorem bracketDist_Fgen (a b : Fin r) :
    bracketDist (Fgen r a) (Fgen r b)
      = deltaDeriv 1 (opeOne r a b) + deltaDeriv 0 (opeZero r a b)
theorem Fgen_local (a b : Fin r) : Local (Fgen r a) (Fgen r b)
theorem prod_Fgen_zero (a b : Fin r) :
    (Fld.prod 0 (FgenFld r a) (FgenFld r b)).1 = opeZero r a b
theorem prod_Fgen_one (a b : Fin r) :
    (Fld.prod 1 (FgenFld r a) (FgenFld r b)).1 = opeOne r a b
theorem prod_Fgen_of_two_le (a b : Fin r) {ℓ : ℤ} (hℓ : 2 ≤ ℓ) :
    (Fld.prod ℓ (FgenFld r a) (FgenFld r b)).1 = 0
```

File `VOA/Moonshine/HeisenbergTwistedModuleGenerators.lean`.

Theorem 35.20 (Locality is inherited). *If the fields of S are pairwise mutually local, then so are all fields of $\langle S\rangle$. In particular, for $S = \{F_a : a < r\}$, every two fields in $\langle S\rangle$ are mutually local.*

Lean. `VOA.TwistedReduction.tgen_local`, `VOA.TwistedReduction.tgenerated_local`, `VOA.Moonshine.genSet`, `VOA.Moonshine.twistedSystem_local`

```
theorem tgen_local (S : Set (Fld M)) (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1) :
    ∀ (N i j : ℕ) (a b : Fld M), i + j ≤ N → TGen S i a → TGen S j b → Local a.1 b.1
theorem tgenerated_local {S : Set (Fld M)} (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1)
    {a b : Fld M} (ha : a ∈ TGenerated S) (hb : b ∈ TGenerated S) : Local a.1 b.1

def genSet : Set (Fld (FockT r)) := Set.range (FgenFld r)
theorem twistedSystem_local {a b : Fld (FockT r)} (ha : a ∈ TGenerated (genSet r))
    (hb : b ∈ TGenerated (genSet r)) : Local a.1 b.1
```

File `VOA/Moonshine/HeisenbergTwistedModuleLocality.lean`.

Definition 35.22 (The twisted vertex operator). For a monomial x^m of degree d with canonical word l_m, $Y^T(x^m)_{(k/2)} = \widetilde{Y}(l_m)_{((k-d)/2)}$ if $k \equiv d \pmod 2$, and 0 otherwise. Y^T is extended linearly to `FockN r`.

Lean. `VOA.HeisenbergN.monoWordN`, `VOA.Moonshine.YTmon`, `VOA.Moonshine.YT`
Files `VOA/Examples/HeisenbergNVertex.lean`, `VOA/Moonshine/HeisenbergTwistedModuleVertex.lean`.

```
noncomputable def monoWordN (a : (ℕ × Fin r) →₀ ℕ) : List (ℕ × Fin r) :=
  a.support.toList.flatMap (fun ia => List.replicate (a ia) ia)

noncomputable def YTmon (m : (ℕ × Fin r) →₀ ℕ) : ℤ → End ℂ (FockT r) := fun k =>
  if (k - (monoWordN m).length) % 2 = 0 then
    mode (Ywt r (monoWordN m)).1 ((k - (monoWordN m).length) / 2)
  else 0

noncomputable def YT : FockN r →ₗ[ℂ] (ℤ → End ℂ (FockT r)) :=
  Finsupp.linearCombination ℂ (YTmon r)
```

Theorem 35.23. *(i) $Y^T(\mathbf{1})_{(k/2)} = \delta_{k,-2}\,\mathrm{id}$; (ii) $Y^T(h_a(-1)\mathbf{1})_{(k/2)} = h_a(k/2)$; (iii) lower truncation; (iv) the support condition: for $u \in \mathcal{V}^j$, $Y^T(u)_{(k/2)} = 0$ unless $k \equiv -j \pmod 2$.*

Lean. `VOA.Moonshine.YT_vacuum`, `VOA.Moonshine.YT_X`, `VOA.Moonshine.YT_truncation`, `VOA.Moonshine.YT_support`

```
theorem YT_vacuum (k : ℤ) : YT r (vacuumN r) k = if k = -2 then 1 else 0
theorem YT_X (a : Fin r) : YT r (X (0, a)) = modeT r a
theorem YT_truncation (u : FockN r) (x : FockT r) :
    ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YT r u k x = 0
theorem YT_support (j : ℕ) (u : FockN r) (hu : u ∈ (sigma1 r).twistedEigenspace 2 j)
    (k : ℤ) (hk : (k + j) % ((2 : ℕ) : ℤ) ≠ 0) : YT r u k = 0
```

File `VOA/Moonshine/HeisenbergTwistedModuleVertex.lean`.

Theorem 35.43 (The twisted Heisenberg module). *For every $r \in \mathbb{N}$, the twisted Fock space $S[\theta]$, with the twisted vertex operator Y^T of Definition 35.22 and the grading M_α, is a σ-twisted module of order 2 for the level-one rank-r Heisenberg VOA, in the sense of Definition 35.4. Its generators act by the twisted bosons, and its L_0 is $\sum h_a(-n-\frac{1}{2})h_a(n+\frac{1}{2}) + \frac{r}{16}$.*

Lean. `VOA.Moonshine.heisTwistedModule`, `VOA.Moonshine.heisTwistedModule_YM`

```
noncomputable def heisTwistedModule : TwistedModule (sigma1 r) 2 (FockT r) where
  YM := YT r
  vacuum_YM k := by
    change YT r (vacuumN r) k = _
    rw [YT_vacuum]
    push_cast
    rfl
  truncation := YT_truncation r
  support := fun j u hu k hk => YT_support r j u hu k hk
  jacobi := twistedJacobi_YT r
  weight := weightC r
  isInternal := weightC_isInternal r
  L0_weight := fun α x hx => YT_omega_two_weightC r hx
  finrank := weightC_finiteDimensional r
  boundedBelow := weightC_boundedBelow r

@[simp] theorem heisTwistedModule_YM : (heisTwistedModule r).YM = YT r := rfl
```

File `VOA/Moonshine/HeisenbergTwistedModuleInstance.lean`.

Chapter 36

The twisted Leech lattice module and $(V_\Lambda^T)^+$

6 of the chapter's 46 numbered statements: its headline results and every definition they use.

Lemma 36.6 (Characterisation and the exponential law). *Let θ be the Euler operator on $R[[X]]$ and $D(d) = \sum_{k\ge1} d_k X^k$.*

1. *If $F \in R[[X]]$ satisfies $\theta F = D(d)F$ and $F_0 = 1$, then $F = \sum_p \operatorname{nexp}(d)_p X^p$.*
2. *$\operatorname{nexp}(d+e) = \operatorname{nexp}(d)\cdot\operatorname{nexp}(e)$ as power series.*
3. *If $\varphi : R \to S$ is a $\mathbb{C}$-algebra map, $\varphi(\operatorname{nexp}(d)_p) = \operatorname{nexp}(\varphi\circ d)_p$.*
4. *$\operatorname{nexp}(k \mapsto t^k d_k)_p = t^p \operatorname{nexp}(d)_p$.*
5. *If δ is a $\mathbb{C}$-derivation of R with $\delta(d_k) = \delta_{k,K}\,K\,c$ for some $K \ge 1$, then $\delta(E_p) = c\,E_{p-K}$ for $p \ge K$ and $\delta(E_p) = 0$ for $p < K$.*

Lean. `VOA.Moonshine.LatticeTwisted.eq_nexpPS`, `VOA.Moonshine.LatticeTwisted.nexpPS_add`, `VOA.Moonshine.LatticeTwisted.nexp_map`, `VOA.Moonshine.LatticeTwisted.nexp_scale`, `VOA.Moonshine.LatticeTwisted.nexp_derivation`

```
theorem eq_nexpPS (d : ℕ → R) (F : R⟦X⟧) (h : theta F = Dser d * F) (h0 : coeff 0 F = 1) :
    F = nexpPS d
theorem nexpPS_add (d e : ℕ → R) : nexpPS (d + e) = nexpPS d * nexpPS e
theorem nexp_map {S : Type*} [CommRing S] [Algebra ℂ S] (φ : R →ₐ[ℂ] S) (d : ℕ → R) (p : ℕ)
 ↪  :
    φ (nexp d p) = nexp (fun k => φ (d k)) p
theorem nexp_scale (t : R) (d : ℕ → R) (p : ℕ) :
    nexp (fun k => t ^ k * d k) p = t ^ p * nexp d p
theorem nexp_derivation (δ : Derivation ℂ R R) (d : ℕ → R) (K : ℕ) (hK : 0 < K) (c : R)
    (hd : ∀ k, δ (d k) = if k = K then (K : ℂ) • c else 0) (p : ℕ) :
    δ (nexp d p) = if K ≤ p then c * nexp d (p - K) else 0
```

File `VOA/Moonshine/LatticeTwistedExp.lean`. **Hypotheses / status.** None

Theorem 36.17 (Locality of the dressings). *If $\langle v,u\rangle = n \in \mathbb{Z}$ and $N \ge |n|$, then for all $A, B \in \mathbb{Z}$ and $x \in S[\theta]$*

$$\sum_{r=0}^{N}(-1)^{N-r}\binom{N}{r}\Big(\mathrm{DD}_v(A-2r)\mathrm{DD}_u(B-2(N-r)) - (-1)^{|n|}\mathrm{DD}_u(B-2(N-r))\mathrm{DD}_v(A-2r)\Big)x = 0,$$

that is, $(w_1^2 - w_2^2)^N\big(Y_v(w_1)Y_u(w_2) - (-1)^n Y_u(w_2)Y_v(w_1)\big) = 0$ with $Y_v(w) = E^-E^+(-v,w)$.

Lean. `VOA.Moonshine.LatticeTwisted.DD_bracket_local`

```
theorem DD_bracket_local (v u : Fin r → ℂ) (n : ℤ) (hn : pair v u = n) (N : ℕ)
    (h1 : -(N : ℤ) ≤ n) (h2 : n ≤ N) (A B : ℤ) (x : FockT r) :
    ∑ r ∈ Finset.range (N + 1), ((-1 : ℂ) ^ (N - r) * (N.choose r : ℂ)) •
      (DD v (A - 2 * r) (DD u (B - 2 * (N - r : ℕ)) x)
        - (-1 : ℂ) ^ n.natAbs • DD u (B - 2 * (N - r : ℕ)) (DD v (A - 2 * r) x)) = 0
```

File `VOA/Moonshine/LatticeTwistedTwoPoint.lean`. **Hypotheses / status.** $\langle v, u\rangle \in \mathbb{Z}$, $|n| \le N$

Theorem 36.21 (Properties of $Y^T(e^\alpha)$). *Let L be even, positive definite, unimodular.*

1. $e_\beta e_\alpha = (-1)^{\langle\alpha,\beta\rangle} e_\alpha e_\beta$ *on* $\mathcal{T}_L$.
2. *For all $\alpha, \beta \in L$ and residues e, e', the rescaled fields of $Y^T(e^\alpha)$ and $Y^T(e^\beta)$ are mutually local.*
3. $Y^T(e^{-\alpha})_{(k/2)} = (-1)^k Y^T(e^\alpha)_{(k/2)}$. *Consequently $e^\alpha + e^{-\alpha}$ has no odd modes and $e^\alpha - e^{-\alpha}$ no even modes.*
4. $Y^T(e^0)_{(k/2)} = \delta_{k,-2}$.

Lean. `VOA.Moonshine.LatticeTwisted.eT_swap`, `VOA.Moonshine.LatticeTwisted.YTL_local`, `VOA.Moonshine.LatticeTwisted.YTL_neg`, `VOA.Moonshine.LatticeTwisted.YTL_zero`
File `VOA/Moonshine/LatticeTwistedVertex.lean`.

```
theorem eT_swap (α β : L.carrier) :
    eT L hEven hU β * eT L hEven hU α
      = (-1 : ℂ) ^ (L.form α β).natAbs • (eT L hEven hU α * eT L hEven hU β)

theorem YTL_local (hPos : L.PosDef) (α β : L.carrier) (e e' : ℤ) :
    Local (resc (YTL L hEven hU α) 2 e) (resc (YTL L hEven hU β) 2 e') :=
  YTg_local (chargeCoord L α) (chargeCoord L β) (L.form α β) (sum_chargeCoord L hPos α β) _
   ↪ _ _ _
    _ _ (eT_swap L hEven hU α β) e e'

theorem YTL_neg (α : L.carrier) (k : ℤ) :
    YTL L hEven hU (-α) k = (-1 : ℂ) ^ k • YTL L hEven hU α k

theorem YTL_zero (k : ℤ) : YTL L hEven hU 0 k = if k = -2 then 1 else 0
```

Hypotheses / status. Lattice hypothesis

Theorem 36.23 (Locality of the twisted lattice system). *For L even, positive definite, unimodular, any two fields in the generated twisted system* TGenerated(*latGenSet*) *are mutually local.*

Lean. `VOA.Moonshine.LatticeTwisted.latticeTwistedSystem_local`
File `VOA/Moonshine/LatticeTwistedVertex.lean`.

```
theorem latticeTwistedSystem_local (hPos : L.PosDef)
    {a b : Fld (FockT (rank L) ⊗[ℂ] Rep (cocMat L) (isUnit_det_cocMat L hEven hU))}
    (ha : a ∈ TGenerated (latGenSet L hEven hU)) (hb : b ∈ TGenerated (latGenSet L hEven
     ↪ hU)) :
    Local a.1 b.1 :=
  tgenerated_local (latGenSet_local L hEven hU hPos) ha hb
```

Hypotheses / status. Lattice hypothesis

Theorem 36.27 (Generators and the reduction to exponentials). *Let L be even, positive definite, unimodular.*

1. *For every Heisenberg generator, every state w and every ℓ:* $\mathrm{star}_{1/2}(F_b)_\ell F_e(w) = F_{e+1}(h_{b(\ell)}w)$.

2. *If* $\mathrm{PF}(e^\alpha, e^\beta)$ *holds for all* $\alpha, \beta \in L$, *then* $\mathrm{PF}(u, v)$ *holds for all* $u, v \in V_L$.

Lean. `VOA.Moonshine.LatticeTwisted.genPF`, `VOA.Moonshine.LatticeTwisted.PF_of_base` File `VOA/Moonshine/LatticeTwistedProduct.lean`.

```
theorem genPF (b : Fin (rank L)) (e' ℓ : ℤ) (w : LatModuleGen L) :
    star (1 / 2) (FgT b) ℓ (FL L hEven hU e' w)
      = FL L hEven hU (e' + 1) ((latticeVOAGen L hEven hPos).smul (hgen L b) ℓ w)

theorem PF_of_base (hbase : ∀ α β : L.carrier, PF L hEven hU hPos (echg L α) (echg L β))
    (u v : LatModuleGen L) : PF L hEven hU hPos u v :=
```

Hypotheses / status. Lattice hypothesis (+ `hbase`)

Theorem 36.48 (Weights and dimensions). 1. *For* $q \in \mathbb{Q}$: $\mathrm{TP}_q = \mathrm{twFixed} \cap \mathrm{twWeight}_q$, *the graded piece used in Chapter 33.*

2. *If* $\mathrm{TP}_\alpha \neq 0$ *then* $\alpha = n \in \mathbb{Z}$ *with* $n \geq 2$.

3. $\dim \mathrm{TP}_2 = 98304 = 24 \cdot 4096$.

4. *For every* $n \in \mathbb{N}$: $\dim (V_\Lambda^+)_n + \dim \mathrm{TP}_n = [q^n]\big(qJ(q)\big)$, $J = j - 744$.

Lean. `VOA.Moonshine.map_plusModule_weight`, `VOA.Moonshine.plusModule_weight_ne_bot`, `VOA.Moonshine.finrank_plusModule_weight_two`, `VOA.Moonshine.finrank_plusModule_eq_coeff_JShift`

```
theorem map_plusModule_weight (q : ℚ) :
    (plusModule.weight (q : ℂ)).map TP.subtype = twFixed 24 leechT ⊓ twWeight 24 leechT q

theorem plusModule_weight_ne_bot {α : ℂ} (h : plusModule.weight α ≠ ⊥) :
    ∃ n : ℤ, α = n ∧ 2 ≤ n

theorem finrank_plusModule_weight_two : finrank ℂ (plusModule.weight 2) = 98304

theorem finrank_plusModule_eq_coeff_JShift (n : ℕ) :
    ((finrank ℂ (leechVOAPlus.grading (n : ℤ))
      + finrank ℂ (plusModule.weight ((n : ℤ) : ℂ)) : ℕ) : ℂ)
      = PowerSeries.coeff n JShift
```

File `VOA/Moonshine/TwistedPlus.lean`. **Hypotheses / status.** None

Chapter 37

Intertwiners and invariant forms: the operators W and $\overline{W}$

7 of the chapter's 52 numbered statements: its headline results and every definition they use.

Theorem 37.13 (L_{-1}-derivative property). *$W(L_{-1}t)_{(n)} = -n\,W(t)_{(n-1)}$ for all n; equivalently $W(L_{-1}t, z) = \partial_z W(t, z)$.*

Lean. `VOA.SkewIntertwiner.wMode_Lm1`, `VOA.SkewIntertwiner.Wsk_deriv`

```
theorem wMode_Lm1 (A : WeakModule W M) (t : M) (n : ℤ) (φ : 𝒱) :
    wMode A (Lm1 A t) n φ = (-(n : ℂ)) • wMode A t (n - 1) φ

theorem Wsk_deriv (A : WeakModule W M) (t : M) :
    Wsk A (Lm1 A t) = deriv (Wsk A t)
```

File `VOA/Moonshine/SkewIntertwiner.lean`. **Hypotheses / status.** None

Theorem 37.15 (Intertwining commutator formula for W). *Let M be a weak $\mathcal{V}$-module, $u \in \mathcal{V}$, $t \in M$, and let $J \in \mathbb{N}$ be such that $u_{(j)}t = 0$ for all $j \geq J$. Then for* all *$m, n \in \mathbb{Z}$ and $\varphi \in \mathcal{V}$,*

$$u^M_{(m)}\,W(t)_{(n)}\varphi - W(t)_{(n)}\big(u_{(m)}\varphi\big) = \sum_{r=0}^{J-1} \binom{m}{r} W\big(u^M_{(r)}t\big)_{(m+n-r)}\varphi. \tag{37.1}$$

Lean. `VOA.SkewIntertwiner.wMode_comm`
File `VOA/Moonshine/SkewIntertwiner.lean`.

```
theorem wMode_comm (A : WeakModule W M) (u : 𝒱) (m : ℤ) (t : M) (n : ℤ) (φ : 𝒱) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → A.act u j t = 0) :
    A.act u m (wMode A t n φ) - wMode A t n (W.smul u m φ)
      = ∑ r ∈ Finset.range J, gBinom m r • wMode A (A.act u r t) (m + n - r) φ
```

Hypotheses / status. J past truncation

Theorem 37.17 (W is local with every module vertex operator). *For $u \in \mathcal{V}$ and $t \in M$, let J be past the truncation of u on t. Then*

$$(z_1 - z_2)^J\Big(Y_M(u, z_1)\,W(t, z_2) - W(t, z_2)\,Y(u, z_1)\Big) = 0$$

as a two-variable distribution with values in $\mathrm{Hom}(\mathcal{V}, M)$*. In particular such an exponent exists.*

Lean. `VOA.SkewIntertwiner.wBracket`, `VOA.SkewIntertwiner.Wsk_local_of_bound`, `VOA.SkewIntertwiner.Wsk_local`
File `VOA/Moonshine/SkewIntertwiner.lean`.

```
noncomputable def wBracket (A : WeakModule W M) (u : 𝒱) (t : M) : FormalDist₂ (𝒱 →ₗ[ℂ] M)
 ↪  :=
  fun a b => (A.YM u a).comp (Wsk A t b) - (Wsk A t b).comp (W.Y u a)

theorem Wsk_local_of_bound (A : WeakModule W M) (u : 𝒱) (t : M) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → A.act u j t = 0) :
    mulBySubPow J (wBracket A u t) = 0

theorem Wsk_local (A : WeakModule W M) (u : 𝒱) (t : M) :
    ∃ N : ℕ, mulBySubPow N (wBracket A u t) = 0
```

Hypotheses / status. None

Theorem 37.51 (Intertwining commutator formula for $\overline{W}$). *Assume: B_V graded orthogonal, graded nondegenerate and invariant; B_M invariant at every homogeneous state; and $L_1\mathcal{V}_1 = 0$. Then for* every *$u \in \mathcal{V}$, $m, n \in \mathbb{Z}$, $t, a \in M$ and J past the truncation of u on t,*

$$u_{(m)}\overline{W}(t)_{(n)}a - \overline{W}(t)_{(n)}\big(u_{(m)}a\big) = \sum_{r<J} \binom{m}{r} \overline{W}\big(u_{(r)}t\big)_{(m+n-r)}a. \tag{37.2}$$

Lean. `VOA.Moonshine.SkewAdjoint.WbarMode_comm`
File `VOA/Moonshine/WbarComm.lean`.

```
theorem WbarMode_comm (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV) (hVinv : VInv W
 ↪  BV)
    (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    (hqpV : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (u : 𝒱) (m : ℤ) (t : M) (n : ℤ) (a : M)
    (J : ℕ) (hJ : ∀ j : ℕ, J ≤ j → N.act u j t = 0) :
    W.smul u m (WbarMode BV N BM hO n t a) - WbarMode BV N BM hO n t (N.act u m a)
      = ∑ r ∈ Finset.range J, gBinom m r • WbarMode BV N BM hO (m + n - r) (N.act u r t) a
```

Hypotheses / status. `hVo`,`hVnd`,`hVinv`,`hMinv`,`hqpV`

Theorem 37.52 (Locality of $Y(u,z_1)$ and $\overline{W}(t,z_2)$). *Under the hypotheses of Theorem 37.51, for every $u \in \mathcal{V}$, $t \in M$ there is N with $(z_1 - z_2)^N\big(Y(u,z_1)\overline{W}(t,z_2) - \overline{W}(t,z_2)Y_M(u,z_1)\big) = 0$ in* $\mathrm{Hom}(M, \mathcal{V})[[z_1^{\pm 1}, z_2^{\pm 1}]]$. *Moreover $[L_{-1}, \overline{W}(t,z)] = \partial_z \overline{W}(t,z)$, in coefficients $L_{-1} \circ \overline{W}(t)_k - \overline{W}(t)_k \circ L_{-1}^M = (k+1)\overline{W}(t)_{k+1}$.*

Lean. `VOA.Moonshine.SkewAdjoint.Wbar_local`, `VOA.Moonshine.SkewAdjoint.Wbar_homCov`
File `VOA/Moonshine/WbarComm.lean`.

```
theorem Wbar_local (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV)
    (hVinv : VInv W BV) (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    (hqpV : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (u : 𝒱) (t : M) :
    ∃ N₀ : ℕ, mulBySubPow N₀
      (fun a b => (W.Y u a).comp (Wbar BV N BM hO t b) - (Wbar BV N BM hO t b).comp (N.YM u
       ↪  a) :
        FormalDist₂ (M →ₗ[ℂ] 𝒱)) = 0

theorem Wbar_homCov (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV)
    (hVinv : VInv W BV) (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    (hqpV : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (t : M) (k : ℤ) :
    W.L (-1) ∘ₗ Wbar BV N BM hO t k - Wbar BV N BM hO t k ∘ₗ Lm1 N.toWeakModule
      = (k + 1 : ℤ) • Wbar BV N BM hO t (k + 1)
```

Hypotheses / status. Same

Theorem 37.57 (The vacuum pairing). (i) *For $t, t' \in ((V_\Lambda^T)^+)_2$ (the lowest weight space), $\overline{W}(t)_{(3)}t' = B_T(t', t)\,\mathbf{1}$.* (ii) *For every nonzero $t \in ((V_\Lambda^T)^+)_2$ there are $t' \in ((V_\Lambda^T)^+)_2$ and $c \neq 0$ with $\overline{W}(t)_{(3)}t' = c\,\mathbf{1}$.* (iii) $\overline{W} \neq 0$.

Lean. `VOA.Moonshine.leechWbarMode_three_weight_two`, `VOA.Moonshine.leechWbar_vacuum_pairing_ne_zero`, `VOA.Moonshine.leechWbar_ne_zero`

```
theorem leechWbarMode_three_weight_two {t t' : TP} (ht : t ∈ plusModule.weight ((2 : ℤ) :
↪ ℂ))
    (ht' : t' ∈ plusModule.weight ((2 : ℤ) : ℂ)) :
    leechWbarMode 3 t t' = TPForm t' t • leechVOAPlus.vacuum

theorem leechWbar_vacuum_pairing_ne_zero {t : TP} (ht : t ∈ plusModule.weight ((2 : ℤ) :
↪ ℂ))
    (ht0 : t ≠ 0) :
    ∃ t' ∈ plusModule.weight ((2 : ℤ) : ℂ), ∃ c : ℂ, c ≠ 0 ∧
      leechWbarMode 3 t t' = c • leechVOAPlus.vacuum

theorem leechWbar_ne_zero : leechWbar ≠ 0
```

File `VOA/Moonshine/VnatGenuine.lean`. **Hypotheses / status.** t, t' of weight 2

Theorem 37.58 (Skew-symmetry of $\overline{W}$). *For homogeneous $t_1 \in ((V_\Lambda^T)^+)_a$, $t_2 \in ((V_\Lambda^T)^+)_b$ and every $n \in \mathbb{Z}$,*

$$\overline{W}(t_1)_{(n)}t_2 = \sum_{i \geq 0} \frac{(-1)^{n+i+1}}{i!}\, L_{-1}^{i}\Big(\overline{W}(t_2)_{(n+i)}t_1\Big).$$

Lean. `VOA.Moonshine.M8h0.TPForm_Lm1`, `VOA.Moonshine.M8h0.leechBV_Lm1_pow`, `VOA.Moonshine.M8h0.leechWbar_skew`

```
theorem TPForm_Lm1 (x y : TP) : TPForm (leechLTP x) y = TPForm x (Lp1 plusModule y)

theorem leechBV_Lm1_pow (i : ℕ) (x y : SkewVP) :
    leechBV (((leechVOAPlus.L (-1)) ^ i) x) y = leechBV x (((leechVOAPlus.L 1) ^ i) y)

theorem leechWbar_skew {a b : ℤ} {t₁ t₂ : TP} (ht₁ : t₁ ∈ plusModule.weight (a : ℂ))
    (ht₂ : t₂ ∈ plusModule.weight (b : ℂ)) (n : ℤ) :
    leechWbarMode n t₁ t₂ = ∑ᶠ i : ℕ, (((i.factorial : ℂ)⁻¹) * negOnePow (n + i + 1)) •
      ((leechVOAPlus.L (-1)) ^ i) (leechWbarMode (n + i) t₂ t₁)
```

File `VOA/Moonshine/M8h0Skew.lean`. **Hypotheses / status.** t_1, t_2 homogeneous

Chapter 38

The vertex algebra on $V^\natural$ modulo one relation

8 of the chapter's 65 numbered statements: its headline results and every definition they use.

Theorem 38.16 (Diagonal–off-diagonal locality). $\mathrm{diag}(a(z_1), b(z_1))$ *and* $\mathrm{offdiag}(c'(z_2), d'(z_2))$ *are mutually local provided that, for some* N*,*

$$(z_1 - z_2)^N\big(a(z_1)c'(z_2) - c'(z_2)b(z_1)\big) = 0 \quad \textit{and} \quad (z_1 - z_2)^N\big(b(z_1)d'(z_2) - d'(z_2)a(z_1)\big) = 0.$$

Lean. `VOA.Moonshine.VnatBlock.local_diag_off`
File `VOA/Moonshine/VnatBlock.lean`.

```
theorem local_diag_off {a : FormalDist (End ℂ V)} {b : FormalDist (End ℂ M)}
    {c' : FormalDist (M →ₗ[ℂ] V)} {d' : FormalDist (V →ₗ[ℂ] M)}
    (hc : ∃ N : ℕ, mulBySubPow N
      ((fun m n => a m ∘ₗ c' n - c' n ∘ₗ b m) : FormalDist₂ (M →ₗ[ℂ] V)) = 0)
    (hd : ∃ N : ℕ, mulBySubPow N
      ((fun m n => b m ∘ₗ d' n - d' n ∘ₗ a m) : FormalDist₂ (V →ₗ[ℂ] M)) = 0) :
    Local (diagF a b) (offF c' d') := by
```

Hypotheses / status. Two block localities

Theorem 38.19 (Locality of module fields). *For every weak module* M *of* W *and all* $u, v \in W$*, the fields* $Y_M(u,z)$ *and* $Y_M(v,z)$ *are mutually local.*

Lean. `VOA.Moonshine.VnatBlock.local_YM`
File `VOA/Moonshine/VnatBlock.lean`.

```
theorem local_YM (A : WeakModule W M) (u v : 𝒱) : Local (A.YM u) (A.YM v) := by
```

Hypotheses / status. None

Theorem 38.41 (The VOA $V^\natural$, conditional on twisted–twisted locality). *Assume* $Y_T^\natural(t) \sim Y_T^\natural(t')$ *for all* $t, t' \in T^+$*. Then* $(V^\natural,\ (V^\natural{}_n)_n,\ Y^\natural,\ \mathbf{1}^\natural,\ \omega^\natural,\ c = 24)$ *is a vertex operator algebra in the sense of the library.*

Lean. `VOA.Moonshine.VnatAxioms.vnatVOA`, `VOA.Moonshine.VnatAxioms.vnat_VOA_of_htt`
File `VOA/Moonshine/VnatAxioms.lean`.

```
noncomputable def vnatVOA (htt : ∀ t t' : TP, Local (YnatT leechWbar t) (YnatT leechWbar
 ↪  t')) :
    VertexOperatorAlgebra Vnat where
  grading := gradingNat
```

```
  isInternal := gradingNat_isInternal
  finrank := finiteDimensional_gradingNat
  Y := YN
  vacuum := vacNat
  vacuum_mem := vacNat_mem
  grading_mode := fun x hx y hy n => YN_grading_mode x hx y hy n
  vacuum_Y := YN_vacuum
  creation := YN_creation
  locality := YN_local_of_htt htt
  regularity := YN_regularity
  conformalVector := omegaNat
  conformal_mem := omegaNat_mem
  centralCharge := 24
  virasoro := YN_virasoro
  L0_grading := fun v hv => YN_L0_grading v hv
  translation := YN_translation
  vacuum_grade := gradingNat_zero
  positivity := fun _ hn => gradingNat_neg hn

theorem vnat_VOA_of_htt (htt : ∀ t t' : TP, Local (YnatT leechWbar t) (YnatT leechWbar t'))
 ↪  :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
        V.centralCharge = 24 ∧ V.grading = gradingNat :=
  ⟨vnatVOA htt, rfl, rfl, rfl, rfl, rfl⟩
```

Hypotheses / status. Htt

Theorem 38.44. *$Y_T^\natural[\overline{W}'](t_1) \sim Y_T^\natural[\overline{W}'](t_2)$ if and only if $\mathrm{VP}(t_1,t_2)$ and $\mathrm{TP}(t_1,t_2)$ hold.*

Lean. `VOA.Moonshine.YnatT_local_iff`

```
theorem YnatT_local_iff (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t₁ t₂ : TP) :
    Local (YnatT Wbar t₁) (YnatT Wbar t₂) ↔
      VPBlockLocal Wbar t₁ t₂ ∧ TPBlockLocal Wbar t₁ t₂ :=
  local_off_off_iff _ _ _ _
```

File `VOA/Moonshine/TTBlock.lean`. **Hypotheses / status.** None

Theorem 38.45 (Two-variable Goddard lemma). *Let V be a vector space, $\mathrm{vac} \in V$, $A(w) \in \mathrm{End}(V)[[w^{\pm}]]$ with $A_p\,\mathrm{vac} = 0$ for $p < 0$ (raw indexing: no negative powers of w on the vacuum), and $B(z_1,z_2) \in \mathrm{End}(V)[[z_1^{\pm},z_2^{\pm}]]$ with $B_{m,n}\mathrm{vac} = 0$ for all m,n. If $(w-z_1)^{M_1}(w-z_2)^{M_2}[A(w),B(z_1,z_2)] = 0$, then $B_{m,n}(A_p\,\mathrm{vac}) = 0$ for all p,m,n.*

Lean. `VOA.Moonshine.M8h0.goddard₂`

```
theorem goddard₂ (vac : V) (A : FormalDist (End ℂ V)) (B : FormalDist₂ (End ℂ V)) (M₁ M₂ :
 ↪  ℕ)
    (hloc : mulBySub₁₂Pow M₁ (mulBySub₁₃Pow M₂ (brk3 A B)) = 0)
    (hneg : ∀ p : ℤ, p < 0 → A p vac = 0) (hvac : ∀ m n, B m n vac = 0) (p m n : ℤ) :
    B m n (A p vac) = 0 := by
```

File `VOA/Moonshine/M8h0Goddard.lean`. **Hypotheses / status.** Displayed

Theorem 38.55 (Four explicit pairs). *There exist $x_0 \in T_0^+$, $x_1 \in T_1^+$ (the two frame sectors of T^+) such that (4.23) on the four pairs (x_i,x_j) implies the conclusion of Theorem 38.41.*

Lean. `VOA.Moonshine.M8h0.vnat_VOA_of_TPBlockLocal_frame`
File `VOA/Moonshine/M8h0Gap.lean`.

```
theorem vnat_VOA_of_TPBlockLocal_frame :
    ∃ x : ZMod 2 → TP, (∀ s, x s ∈ TPframe s) ∧
      ((∀ i j, TPBlockLocal leechWbar (x i) (x j)) →
        ∃ V : VertexOperatorAlgebra Vnat,
          V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
            V.centralCharge = 24 ∧ V.grading = gradingNat) := by
```

Hypotheses / status. (4.23) on 4 chosen pairs

Theorem 38.66 ($V^\natural$ is simple). *Assume htt. Then* `vnatVOA`(htt) *is simple, and the canonical invariant form of the library on it coincides with $B^\natural$ and is nondegenerate on every weight space.*

Lean. `VOA.Moonshine.SimpleNat.vnatVOA_isSimple`, `VOA.Moonshine.SimpleNat.vnatVOA_invariantForm_nondegenerate`, `VOA.Moonshine.SimpleNat.vnatVOA_canonicalForm_eq`

```
theorem vnatVOA_isSimple : (vnatVOA htt).IsSimple :=
  (LiRadical.isSimple_iff_leftRadical_eq_bot (natInvForm_isFullyInvariant htt)
    (natInvForm_normalized htt)).2 (natInvForm_leftRadical htt)

theorem vnatVOA_invariantForm_nondegenerate :
    (natInvForm htt).IsFullyInvariant ∧ (natInvForm htt).Normalized ∧
      InvariantForm.leftRadical (natInvForm htt) = ⊥ ∧
        GradedNondeg (vnatVOA htt) (natInvForm htt).form :=

theorem vnatVOA_canonicalForm_eq :
    (canonicalInvariantForm (vnatVOA_vacuum_ne_zero htt) (vnatVOA_L1_one htt)).form =
    ↪ natForm :=
```

File `VOA/Moonshine/SimpleVnat.lean`. **Hypotheses / status.** Htt

Theorem 38.68 ($V^\natural$ modulo (4.23) on four pairs). *There exist $x_0 \in T_0^+$, $x_1 \in T_1^+$ such that, if DGM (4.23) holds on the four pairs (x_i, x_j), then there is a simple vertex operator algebra V on $V^\natural$ whose vertex map is the block map $Y^\natural[\overline{W}]$ of (38.1), with vacuum $(\mathbf{1}, 0)$, conformal vector $(\omega, 0)$, central charge 24, $V_1 = 0$, $\dim V_2 = 196884$, and $\dim V_n = [q^n]\, qJ(q)$ for all $n \geq 0$.*

Lean. `VOA.Moonshine.moonshine_of_frameLocality`

```
theorem moonshine_of_frameLocality :
    ∃ x : ZMod 2 → TP, (∀ s, x s ∈ TPframe s) ∧
      ((∀ i j, TPBlockLocal leechWbar (x i) (x j)) →
        ∃ V : VertexOperatorAlgebra Vnat,
          V.Y = Ynat wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
          V.centralCharge = 24 ∧
          V.grading 1 = ⊥ ∧
          finrank ℂ (V.grading 2) = 196884 ∧
          (∀ n : ℕ, (finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
          V.IsSimple) := by
```

File `VOA/Moonshine/Main.lean`. **Hypotheses / status.** (4.23) on 4 chosen pairs

Chapter 39

The Golay frame and the untwisted triality

6 of the chapter's 42 numbered statements: its headline results and every definition they use.

Theorem 39.6 ($V_\Lambda^+ = V_0^+ \oplus V_3^+$, and the $\mathbb{Z}/2$-grading of products). *(i) $V_\Lambda^+ = V_{(0)} \oplus V_{(1)}$ (internal direct sum). (ii) For $u \in V_{(i)}$, $v \in V_{(j)}$ and all $n \in \mathbb{Z}$: $u_{(n)}v \in V_{(i+j)}$. (iii) Each $V_{(i)}$ is stable under the projections onto the L_0-weight spaces.*

Lean. `VOA.Moonshine.frameSector_isInternal`, `VOA.Moonshine.frameSector_smul_mem`, `VOA.Moonshine.frameSector_graded`

File `VOA/Moonshine/FrameSectors.lean`.

```
/-- **`V_Λ⁺ = V₀⁺ ⊕ V₃⁺`** (internal direct sum). -/
theorem frameSector_isInternal : DirectSum.IsInternal frameSector

/-- **The `ℤ/2`-grading of products**: `u ∈ V_i`, `v ∈ V_j ⇒ u₍ₙ₎v ∈ V_{i+j}`. -/
theorem frameSector_smul_mem {i j : ZMod 2} {u v : VPlus} (hu : u ∈ frameSector i)
    (hv : v ∈ frameSector j) (n : ℤ) : leechVOAPlus.smul u n v ∈ frameSector (i + j)

/-- The sectors are graded: each is stable under the weight projections of `V_Λ⁺`. -/
theorem frameSector_graded (i : ZMod 2) (n : ℤ) {v : VPlus} (hv : v ∈ frameSector i) :
    piHom leechVOAPlus n v ∈ frameSector i
```

Theorem 39.10 (98304 norm-four vectors outside Λ_0). $\#\{\alpha \in \Lambda \setminus \Lambda_0 : \langle \alpha, \alpha \rangle = 4\} = 24 \cdot 2^{12} = 98304$.

Lean. `VOA.Moonshine.oddFour`, `VOA.Moonshine.card_normFour_frameChar_one`

```
/-- The norm-`4` vectors outside `Λ₀`. -/
noncomputable def oddFour : Finset leech.carrier :=
  (normFour leech leech_posDef).filter fun α => frameChar α = 1

/-- **`#{α ∈ Λ \ Λ₀ | ⟨α,α⟩ = 4} = 98304`.** -/
theorem card_normFour_frameChar_one : oddFour.card = 98304
```

File `VOA/Moonshine/FrameCount.lean`.

Theorem 39.15 (The Leech frame split of T). *For Λ, $w = 8e_0$: $T = T_+ \oplus T_-$ with $\dim T_+ = \dim T_- = 2048$; e_α preserves $T_\pm$ for $\alpha \in \Lambda_0$ and swaps them for $\alpha \notin \Lambda_0$; and on the twisted space $S[\theta] \otimes T$ the operator $\mathrm{id} \otimes J$ gives $S[\theta] \otimes T = (S[\theta] \otimes T_+) \oplus (S[\theta] \otimes T_-)$.*

Lean. VOA.Moonshine.frameW, VOA.Moonshine.frameW_sign, VOA.Moonshine.finrank_leechFrameT_plus, VOA.Moonshine.finrank_leechFrameT_minus, VOA.Moonshine.eT_mem_leechFrameT, VOA.Moonshine.leechTw_frame_isCompl

```
/-- The frame vector `w = 8e₀ ∈ Λ₀`. -/
noncomputable def frameW : leech.carrier := (⟨_, eightE0_mem⟩ : ₁Leech.carrier)

theorem frameW_sign (α : leech.carrier) :
    (-1 : ℂ) ^ (leech.form frameW α) = chiSign leech frameChar α

/-- **`dim T₊ = 2048`.** -/
theorem finrank_leechFrameT_plus : finrank ℂ (leechFrameT 0) = 2048

/-- **`dim T₋ = 2048`.** -/
theorem finrank_leechFrameT_minus : finrank ℂ (leechFrameT 1) = 2048

/-- `e_α` preserves `T_±` for `α ∈ Λ₀` and swaps them for `α ∉ Λ₀`. -/
theorem eT_mem_leechFrameT (α : leech.carrier) {s : ZMod 2}
    {x : TT leech leech_even leech_isUnit_det_gramMat} (hx : x ∈ leechFrameT s) :
    eT leech leech_even leech_isUnit_det_gramMat α x ∈ leechFrameT (s + frameChar α)

/-- The split of the twisted carrier `S[0] ⊗ T = (S[0] ⊗ T₊) ⊕ (S[0] ⊗ T₋)`. -/
theorem leechTw_frame_isCompl :
    IsCompl (twFrame leech leech_even leech_isUnit_det_gramMat (TwistedHeisenberg.FockT 24)
     ↪ frameW 0)
      (twFrame leech leech_even leech_isUnit_det_gramMat (TwistedHeisenberg.FockT 24)
       ↪ frameW 1)
```

File VOA/Moonshine/FrameTwisted.lean.

Theorem 39.19 (V_0^+ is cyclic over itself). *V_0^+ is cyclic with cyclic vector* $\mathbf{1}$.

Lean. VOA.Moonshine.cyclic_V0
File VOA/Moonshine/CyclicFrameSectors.lean.

```
/-- **`V₀⁺` is cyclic over itself**, generated by `𝟙`. -/
theorem cyclic_V0 : ∃ x₀ ∈ frameSector 0,
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ, y = leechVOAPlus.smul u n x₀}
      = frameSector 0
```

Theorem 39.28 (Each twisted frame sector is cyclic). *Let $0 \neq t_0 \in T_{(s)}$ and $x_0 = h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$ (weight $\frac{3}{2} + \frac{1}{2} = 2$). Then $x_0 \in T_{(s)}^+$ and $\mathrm{span}\{u_{(n)}x_0 : u \in V_0^+,\ n \in \mathbb{Z}\} = T_{(s)}^+$, where $u_{(n)}$ is the action of V_Λ^+ on $(V_\Lambda^T)^+$. In particular each $T_{(s)}^+$ is a cyclic V_0^+-module.*

Lean. VOA.Moonshine.frameX0T, VOA.Moonshine.span_smul_frameX0T, VOA.Moonshine.cyclic_TPframe

```
/-- The cyclic vector `h₀(-½)𝟙 ⊗ t₀` (weight `2`). -/
noncomputable def frameX0T (t₀ : TLeechT) : Tw := (X (0, 0) : FockT 24) ⊗ₜ t₀

/-- **The explicit form**: for `0 ≠ t₀ ∈ T_s`, `x₀ = h₀(-½)𝟙 ⊗ t₀` generates `T_s⁺` over
 ↪ `V₀⁺`. -/
theorem span_smul_frameX0T {s : ZMod 2} {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀
 ↪ ≠ 0) :
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ,
        y = mode (Moonshine.plusModule.YM u) n (⟨frameX0T t₀, frameX0T_mem_TP t₀⟩ : TP)}
      = TPframe s
```

```
/-- **M8f, twisted half: each twisted frame sector `T_s⁺` is a cyclic `V₀⁺`-module.** -/
theorem cyclic_TPframe (s : ZMod 2) : ∃ x₀ ∈ TPframe s,
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ, y = mode (Moonshine.plusModule.YM
     ↪ u) n x₀}
      = TPframe s
```

File `VOA/Moonshine/CyclicTwisted.lean`.

Theorem 39.41 ($V_{\Lambda_0}^+ \cong V_0^+$). `bridgeΛ` *induces a linear bijection* $V_{\Lambda_0}^+ = V_{\Lambda_0}^{\langle\theta\rangle} \to V_0^+$ *which is a VOA homomorphism preserving* ω.

Lean. `VOA.Moonshine.lam0PlusEquiv`, `VOA.Moonshine.lam0PlusEquiv_isVOAHom`, `VOA.Moonshine.lam0PlusEquiv_omega`

```
/-- **`V_{Λ₀}⁺` ≃ `V₀⁺`** as vector spaces (a VOA isomorphism by `lam0PlusEquiv_isVOAHom`).
 ↪ -/
noncomputable def lam0PlusEquiv : Lam0PlusCarrier ≃ₗ[ℂ] V0Sub.carrier :=
  LinearEquiv.ofBijective lam0PlusToV0 ⟨lam0PlusToV0_injective, lam0PlusToV0_surjective⟩

/-- `lam0PlusEquiv` is a homomorphism of VOAs `V_{Λ₀}⁺ → V₀⁺`. -/
theorem lam0PlusEquiv_isVOAHom : IsVOAHom lam0Plus V0P lam0PlusEquiv.toLinearMap

theorem lam0PlusEquiv_omega : lam0PlusEquiv lam0Plus.conformalVector = V0P.conformalVector
```

File `VOA/Moonshine/FrameBridge.lean`.

Chapter 40

Existence of the twisted triality I: reduction to Zhu generation

6 of the chapter's 61 numbered statements: its headline results and every definition they use.

Lemma 40.3 (What $\mathcal{O}(V)$ contains). *Let $a \in V_p$ with $p \in \mathbb{N}$, $b \in V$, and $y \in V_w$.*

(a) $L_{-1}y + L_0 y \in \mathcal{O}(V)$; hence $L_{-1}y + w\,y \in \mathcal{O}(V)$.

(b) (Frenkel–Zhu higher residues) For every $k \in \mathbb{N}$,

$$\operatorname{Pres}(a,p,b,k) \;:=\; \sum_{i=0}^{p+k} \binom{p+k}{i} a_{(i-2-k)}b \;=\; \operatorname{Res}_z\, Y(a,z)b\,\frac{(1+z)^{p+k}}{z^{2+k}} \;\in\; \mathcal{O}(V).$$

Lean. `VOA.Moonshine.ZhuGen.L_neg_one_add_mem_O`, `VOA.Moonshine.ZhuGen.L_neg_one_add_wt_mem_O`, `VOA.Pres`, `VOA.Pres_mem_O`
File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
theorem L_neg_one_add_mem_O (y : V) : W.L (-1) y + W.L 0 y ∈ O W

theorem L_neg_one_add_wt_mem_O {w : ℤ} {y : V} (hy : y ∈ W.grading w) :
    W.L (-1) y + (w : ℂ) • y ∈ O W
```

File `VOA/Modules/ZhuModule.lean`.

```
noncomputable def Pres (W : VertexOperatorAlgebra V) (M : ℕ) (a b : V) (n : ℕ) : V :=
  ∑ i ∈ Finset.range (M + n + 1), cbz ((M : ℤ) + n) (i : ℤ) • W.smul a ((i : ℤ) - 2 - n) b

theorem Pres_mem_O (W : VertexOperatorAlgebra V) {M : ℕ} {a : V}
    (ha : a ∈ W.grading (M : ℤ)) (b : V) (n : ℕ) : Pres W M a b n ∈ O W
```

Hypotheses / status. a homogeneous

Theorem 40.5 (Zhu's easy direction). *Let N have lowest weight h and $x \in N_h$. For all $u, v \in V$: $o(u * v)x = o(u)o(v)x$; $o(a)x = 0$ for $a \in \mathcal{O}(V)$; $o(u)x \in N_h$; and $o(\mathbf{1}) = \mathrm{id}$.*

Lean. `VOA.oMod_zhuMul_lowest_all`, `VOA.O_le_ker_rhoLW`
File `VOA/Modules/ZhuModule.lean`.

```
theorem oMod_zhuMul_lowest_all (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h)
    (u v : V) {x : Mod} (hx : x ∈ N.weight h) :
    oMod N (zhuMul W u v) x = oMod N u (oMod N v x)

theorem O_le_ker_rhoLW (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h) :
    O W ≤ LinearMap.ker (rhoLW N h)
```

Hypotheses / status. Lowest weight h

Theorem 40.19 (Existence criterion on the sectors). *For $\beta \notin \Lambda_0$ and $y_0 \in (V_\Lambda^T)^+$: a σ-twisted Φ on V_3^+ with $\Phi(x_\beta) = y_0$ exists iff (x_β, y_0) is annihilator-compatible for $(\mathrm{act}_V, \mathrm{act}_T, \sigma)$ with all letters. Likewise on T_s^+ with x_{t_0}.*

Lean. `VOA.Moonshine.TrialityTw.sigmaV3T_exists_iff`, `VOA.Moonshine.TrialityTw.sigmaTT_exists_iff`
File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem sigmaV3T_exists_iff {β : leech.carrier} (hβ : β ∉ lambda0) (y₀ : TP) :
    (∃ Φ : VPlus →ₗ[ℂ] TP, IsSigmaV3T Φ ∧ Φ (frameX0P β) = y₀)
      ↔ AnnCompat actV actT sig Set.univ (frameX0P β) y₀

theorem sigmaTT_exists_iff {s : ZMod 2} {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀
 ↪  ≠ 0)
    (y₀ : TP) :
    (∃ Φ : TP →ₗ[ℂ] TP, IsSigmaTT s Φ ∧ Φ (x0T t₀) = y₀)
      ↔ AnnCompat actT actT sig Set.univ (x0T t₀) y₀
```

Hypotheses / status. Cyclic vector

Theorem 40.25 (Annihilator compatibility from the top level). *Let A, B be weak W-modules on M, N; β a bilinear form on N; $\Omega \le M$; $\varphi : M \to N$ and $P : M \to M$ linear; $G \subseteq W$. Assume:*

(i) (top-level compatibility) for every k, every $a \in W_k$, every $q \in \mathbb{Z}$ and all $x, x' \in \Omega$: $\beta(a_{(q)}\varphi x, \varphi x') = \beta(\varphi(P(a_{(q)}x)), \varphi x')$;

(ii) (invariance on G) for every $c \in G$ and $r \in \mathbb{Z}$ there are finitely many $a_i \in W$, $q_i \in \mathbb{Z}$, $t_i \in \mathbb{C}$ with $\beta(y, c_{(r)}y') = \beta(\sum_i t_i\, a_{i\,(q_i)}y,\ y')$ for all $y, y' \in N$;

(iii) (nondegeneracy) if y lies in the submodule of B generated by $\varphi(\Omega)$ and $\beta(y, z) = 0$ for all $z \in Y_G$, then $y = 0$.

Then for every $x_0 \in \Omega$, $(x_0, \varphi x_0)$ is annihilator-compatible for the untwisted actions of all of W on A and on B.

Lean. `VOA.Moonshine.SigmaExist.annCompat_of_top`
File `VOA/Moonshine/SigmaExistGeneric.lean`.

```
theorem annCompat_of_top (A : WeakModule W M) (B : WeakModule W N)
    (β : LinearMap.BilinForm ℂ N) (Ω : Submodule ℂ M) (φ : M →ₗ[ℂ] N) (P : M →ₗ[ℂ] M)
    (G : Set 𝒱)
    (hZO : ∀ (k : ℤ) (a : 𝒱), a ∈ W.grading k → ∀ (q : ℤ), ∀ x ∈ Ω, ∀ x' ∈ Ω,
      β (B.act a q (φ x)) (φ x') = β (φ (P (A.act a q x))) (φ x'))
    (hadj : ∀ c ∈ G, ∀ r : ℤ, ∃ (n : ℕ) (a : ℕ → 𝒱) (q : ℕ → ℤ) (t : ℕ → ℂ), ∀ y y' : N,
      β y (B.act c r y') = β (∑ i ∈ Finset.range n, t i • B.act (a i) (q i) y) y')
    (hnd : ∀ y ∈ modeSpan B (φ '' Ω), (∀ z ∈ testSpan B G Ω φ, β y z = 0) → y = 0)
    {x₀ : M} (hx₀ : x₀ ∈ Ω) :
    AnnCompat (fun a q => mode (A.YM a) q) (fun a q => mode (B.YM a) q) id Set.univ x₀ (φ
     ↪  x₀)
```

Hypotheses / status. (i)–(iii)

Theorem 40.54 (Leading term, commutators and vacuum values). *On the charge-γ sector:*

(a) (leading term) if p has degree $\le d$ and weight w, then $\mathrm{quadF}(0)p - X(i,a)X(j,b)p$ has degree $\le d+1$ and weight $w+i+j+2$;

(b) (commutator, weight-one legs) for $s \geq -1$, $[\mathrm{quadF}(a,0;b,0;s), X(t,c)] = (t+1)\big(\delta_{cb}X(t+s+2,a) + \delta_{ca}X(t+s+2,b)\big)$;

(c) $(h_a h_b)_{(0)}e^{\gamma} = \gamma_b\, h_a(-1)e^{\gamma} + \gamma_a\, h_b(-1)e^{\gamma}$;

(d) $(h_a(-1)h_c(-j-1)\mathbf{1})_{(-1)}e^{\gamma} = \big(X(0,a)X(j,c) + (-1)^j\gamma_c X(j+1,a) + (j+1)\gamma_a X(j+1,c)\big)e^{\gamma}$.

Lean. `VOA.Moonshine.FockSpan.Fdw`, `VOA.Moonshine.FockSpan.quadF_lead`, `VOA.Moonshine.FockSpan.quadF_X_mul`, `VOA.Moonshine.FockSpan.quadF_D_one`, `VOA.Moonshine.FockSpan.quadF_lead_one`
File `VOA/Moonshine/FockSpanOps.lean`.

```
noncomputable def Fdw (d : ℕ) (w : ℤ) : Submodule ℂ Fk :=
  restrictTotalDegree (ℕ × Fin (rank leech)) ℂ d ⊓ gradingZN (rank leech) w

theorem quadF_lead (γ : leech.carrier) (a : Fin (rank leech)) (i : ℕ) (b : Fin (rank
 ↪ leech))
    (j : ℕ) {d : ℕ} {w : ℤ} {p : Fk} (hp : p ∈ Fdw d w) :
    quadF γ a i b j 0 p - X (i, a) * X (j, b) * p ∈ Fdw (d + 1) (w + i + j + 2)

theorem quadF_X_mul (γ : leech.carrier) (a b : Fin (rank leech)) {s : ℤ} (hs : -1 ≤ s) (t :
 ↪ ℕ)
    (c : Fin (rank leech)) (p : Fk) :
    quadF γ a 0 b 0 s (X (t, c) * p) = X (t, c) * quadF γ a 0 b 0 s p
      + (if c = b then ((t : ℂ) + 1) • (X (((t : ℤ) + s + 2).toNat, a) * p) else 0)
      + (if c = a then ((t : ℂ) + 1) • (X (((t : ℤ) + s + 2).toNat, b) * p) else 0)

theorem quadF_D_one (γ : leech.carrier) (a b : Fin (rank leech)) :
    quadF γ a 0 b 0 (-1) 1
      = chargeCoord leech γ b • (X (0, a) : Fk) + chargeCoord leech γ a • (X (0, b) : Fk)

theorem quadF_lead_one (γ : leech.carrier) (a c : Fin (rank leech)) (j : ℕ) :
    quadF γ a 0 c j 0 1
      = X (0, a) * X (j, c) + ((-1 : ℂ) ^ j * chargeCoord leech γ c) • (X (j + 1, a) : Fk)
        + (((j : ℂ) + 1) * chargeCoord leech γ a) • (X (j + 1, c) : Fk)
```

Hypotheses / status. None

Theorem 40.61 (Golay facts). *(a) Every codeword has weight* $0, 8, 12, 16$ *or* 24.

(b) Two distinct octads meet in at most 4 *points.*

(c) ***(Steiner system*** $S(5,8,24)$***, existence)*** *Every* 5*-subset of the coordinates lies in an octad.*

(d) A word of even weight not in $\mathcal{G}$ *has odd intersection with some octad.*

(e) A codeword of weight 12 *or* 16 *meets some octad in exactly* 6 *points.*

(f) Every 4*-set meets some octad in exactly* 3 *points.*

Lean. `VOA.Moonshine.GroundLat.golay_weight_cases`, `VOA.Moonshine.GroundLat.octad_inter_le`, `VOA.Moonshine.GroundLat.steiner`, `VOA.Moonshine.GroundLat.exists_octad_dot_ne_zero`, `VOA.Moonshine.GroundLat.exists_octad_inter_six`, `VOA.Moonshine.GroundLat.exists_octad_inter_three`
File `VOA/Moonshine/GroundLatticeGolay.lean`.

```
theorem golay_weight_cases {c : Fin 24 → ZMod 2} (hc : c ∈ golay) :
    weight c = 0 ∨ weight c = 8 ∨ weight c = 12 ∨ weight c = 16 ∨ weight c = 24
```

```
theorem octad_inter_le {O₁ O₂ : Finset (Fin 24)} (h₁ : IsOct O₁) (h₂ : IsOct O₂) (hne : O₁
 ↪  ≠ O₂) :
    (O₁ ∩ O₂).card ≤ 4

theorem steiner {T : Finset (Fin 24)} (hT : T.card = 5) : ∃ O, IsOct O ∧ T ⊆ O

theorem exists_octad_dot_ne_zero {x : Fin 24 → ZMod 2} (hx : x ∉ golay) (hw : Even (weight
 ↪  x)) :
    ∃ O, IsOct O ∧ ind O ·ᵥ x ≠ 0

theorem exists_octad_inter_six {C : Finset (Fin 24)} (hC : ind C ∈ golay)
    (h : C.card = 12 ∨ C.card = 16) : ∃ O, IsOct O ∧ (O ∩ C).card = 6

theorem exists_octad_inter_three {S : Finset (Fin 24)} (hS : S.card = 4) :
    ∃ O, IsOct O ∧ (O ∩ S).card = 3
```

Hypotheses / status. As stated

Chapter 41

Existence of the twisted triality II: the explicit construction

6 of the chapter's 54 numbered statements: its headline results and every definition they use.

Proposition 41.3 (source zero modes). *Let $n \in \Lambda$.*

(a) $\mathrm{o}(q_{ab})\, x_n = n_a n_b\, x_n$.

(b) If $\gamma \in \Lambda_0$, $n \notin \Lambda_0$ and both have norm 4, then $|\langle \gamma, n\rangle| \le 2$ and

$$\mathrm{o}(u_\gamma)\, x_n = [\langle \gamma, n\rangle = -2]\, \varepsilon_\Lambda(\gamma, n)\, x_{\gamma+n} + [\langle \gamma, n\rangle = 2]\, \varepsilon_\Lambda(\gamma, -n)\, x_{\gamma-n}.$$

Lean. `VOA.Moonshine.SigmaTop.actV_qV`, `VOA.Moonshine.SigmaTop.form_bound`, `VOA.Moonshine.SigmaTop.actV_lV`

```
theorem actV_qV (a b : Fin (rank leech)) (n : leech.carrier) :
    actV (qV a b) 1 (frameX0P n)
      = (chargeCoord leech n a * chargeCoord leech n b) • frameX0P n

theorem form_bound {γ n : leech.carrier} (hγ : γ ∈ lambda0) (hn : n ∉ lambda0)
    (hγ4 : leech.form γ γ = 4) (hn4 : leech.form n n = 4) :
    -2 ≤ leech.form γ n ∧ leech.form γ n ≤ 2

theorem actV_lV {γ n : leech.carrier} (hγ : γ ∈ lambda0) (hn : n ∉ lambda0)
    (hγ4 : leech.form γ γ = 4) (hn4 : leech.form n n = 4) :
    actV (lV γ hγ) 1 (frameX0P n)
      = (if leech.form γ n = -2 then latticeCocycle leech γ n • frameX0P (γ + n) else 0)
        + (if leech.form γ n = 2 then latticeCocycle leech γ (-n) • frameX0P (γ - n) else
          ↪ 0)
```

File `VOA/Moonshine/SigmaTopSource.lean`. **Hypotheses / status.** Norms, $\gamma \in \Lambda_0$, $n \notin \Lambda_0$

Proposition 41.7 (twisted zero modes). *On $h_c(-\frac{1}{2}) \otimes t$, $t \in T$:*

(a) $\mathrm{o}(h_a(-1)h_b(-1)\mathbf{1})\,(h_c \otimes t) = \big(\frac{1}{8}\delta_{ab}h_c + \frac{1}{2}\delta_{bc}h_a + \frac{1}{2}\delta_{ac}h_b\big) \otimes t$;

(b) for $\|\alpha\|^2 = 4$: $\mathrm{o}(e^\alpha)\,(h_c \otimes t) = 2^{-4}\big(h_c - 2\alpha_c\, \alpha(-\frac{1}{2})\big) \otimes e_\alpha t$;

(c) $\mathrm{o}(e^{-\alpha}) = \mathrm{o}(e^\alpha)$ *on the twisted module (more generally on all even modes).*

Lean. `VOA.Moonshine.SigmaTop.YML_quadSt`, `VOA.Moonshine.SigmaTop.YML_echg_X`, `VOA.Moonshine.SigmaTop.YML_echg_neg_two`
File `VOA/Moonshine/SigmaTopTarget.lean`.

```
theorem YML_quadSt (a b c : Fin (rank L)) (t : Rep (cocMat L) (isUnit_det_cocMat L hEven
 ↪  hU)) :
    YML L hEven hU (quadSt L hEven hPos a b) 2 ((X (0, c) : FockT (rank L)) ⊗ₜ t)
      = ((if a = b then (1 / 8 : ℂ) else 0) • (X (0, c) : FockT (rank L))
          + (if b = c then (1 / 2 : ℂ) else 0) • X (0, a)
          + (if a = c then (1 / 2 : ℂ) else 0) • X (0, b)) ⊗ₜ t

theorem YML_echg_X (α : L.carrier) (hα : L.form α α = 4) (c : Fin (rank L))
    (t : Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :
    YML L hEven hU (echg L α) 2 ((X (0, c) : FockT (rank L)) ⊗ₜ t)
      = (2 : ℂ) ^ (-(4 : ℤ)) • ((X (0, c) - (2 * chargeCoord L α c) • vVarT (chargeCoord L
       ↪  α) 0)
          ⊗ₜ eT L hEven hU α t)

theorem YML_echg_neg_two (α : L.carrier) :
    YML L hEven hU (echg L (-α)) 2 = YML L hEven hU (echg L α) 2
```

Hypotheses / status. Norm 4 (for e^{α})

Proposition 41.11 (zero modes in the frame). *For $v \in \mathbb{C}^{24}$, $t \in T$, $\alpha, \beta \in \Lambda$, $\gamma \in \Lambda_0$ of norm 4:*

(a) $\mathrm{o}(d(\alpha,\beta))\, x_n = \langle \alpha, n\rangle\langle \beta, n\rangle\, x_n$;

(b) $\mathrm{o}(d(\alpha,\beta))\,(h_v \otimes t) = h_{\frac{\langle\alpha,\beta\rangle}{8} v + \frac{1}{2}\langle v,\beta\rangle\alpha + \frac{1}{2}\langle v,\alpha\rangle\beta} \otimes t$;

(c) $\mathrm{o}(u_\gamma)\,(h_v \otimes t) = 2^{-3}\, h_{v - 2\langle v,\gamma\rangle\gamma} \otimes e_\gamma t$;

(d) (diagonal letters) $\mathrm{o}(d(w_j,w_j))$ *acts by* 9 *on block j and by* 1 *on every other block, on* both $(V_3^+)_2$ *and* $(T^+)_2$: $\mathrm{o}(d(w_j,w_j))x_n = (9 \text{ if } j = j(n) \text{ else } 1)\, x_n$ *and* $\mathrm{o}(d(w_j,w_j))(h_{c_m} \otimes t) = (9 \text{ if } j = m \text{ else } 1)\, h_{c_m} \otimes t$.

Lean. `VOA.Moonshine.SigmaTop.actV_dV`, `VOA.Moonshine.SigmaTop.actT_dV_hP`, `VOA.Moonshine.SigmaTop.actT_lV_hP`, `VOA.Moonshine.SigmaTop.actT_dV_fw_hP`, `VOA.Moonshine.SigmaTop.actV_dV_fw`

```
theorem actV_dV (α β n : leech.carrier) :
    actV (dV α β) 1 (frameX0P n)
      = (((leech.form α n : ℤ) : ℂ) * ((leech.form β n : ℤ) : ℂ)) • frameX0P n

theorem actT_dV_hP (α β : leech.carrier) (v : Fin (rank leech) → ℂ) (t : TLeechT) :
    actT (dV α β) 1 (hP v t) = hP (dL α β v) t

theorem actT_lV_hP {γ : leech.carrier} (hγ : γ ∈ lambda0) (hγ4 : leech.form γ γ = 4)
    (v : Fin (rank leech) → ℂ) (t : TLeechT) :
    actT (lV γ hγ) 1 (hP v t)
      = ((2 : ℂ) ^ (-(3 : ℤ))) • hP (latL γ v) (eT leech leech_even
       ↪  leech_isUnit_det_gramMat γ t)

theorem actT_dV_fw_hP (j m : Fin 24) (t : TLeechT) :
    actT (dV (fw j) (fw j)) 1 (hP (cw m) t) = (if j = m then (9 : ℂ) else 1) • hP (cw m) t

theorem actV_dV_fw {n : leech.carrier} (hn : n ∉ lambda0) (h4 : leech.form n n = 4) (j :
 ↪  Fin 24) :
    actV (dV (fw j) (fw j)) 1 (frameX0P n) = (if j = blk n then (9 : ℂ) else 1) • frameX0P
     ↪  n
```

File `VOA/Moonshine/SigmaTopFrame.lean`. **Hypotheses / status.** As stated

Proposition 41.23. *For $j \neq k$:*

(a) (θ-invariance) the coefficients of e^{γ_+} and $e^{-\gamma_+}$ in $\sigma(d(w_j, w_k))$ are equal, and so are those of $e^{\pm\gamma_-}$; consequently $\kappa_+ = \frac{1}{2}\varepsilon_N(\alpha_j,\alpha_k)\,t_c(a_+)$, $\kappa_- = \frac{1}{2}\varepsilon_k\varepsilon_N(\alpha_j,\alpha_k)\,t_c(a_-)$, and $g_e(a_\pm)^2 = \varepsilon_j\varepsilon_k$;

(b) $\kappa_\pm^2\,\varepsilon_\Lambda(\gamma_\pm,\gamma_\pm) = \frac{1}{4}$;

(c) $A_\pm^2 = \frac{1}{4}\cdot \mathrm{id}_T$. In particular $A_\pm$ is diagonalisable with eigenvalues in $\{\pm\frac{1}{2}\}$.

Lean. `VOA.Moonshine.SigmaTop.dQ_symm`, `VOA.Moonshine.SigmaTop.kP_eq`, `VOA.Moonshine.SigmaTop.eV_aP_sq`, `VOA.Moonshine.SigmaTop.kP_sq`, `VOA.Moonshine.SigmaTop.AP_mul_self`, `VOA.Moonshine.SigmaTop.AM_mul_self`
File `VOA/Moonshine/SigmaTop2Eigen.lean`.

```
theorem dQ_symm {j k : Fin 24} (hjk : j ≠ k) : dQ j k 0 = dQ j k 3 ∧ dQ j k 1 = dQ j k 2

theorem kP_eq {j k : Fin 24} (hjk : j ≠ k) : kP j k = (1 / 2 : ℂ) * cQ j k 0 * tc (aP j k)

theorem eV_aP_sq {j k : Fin 24} (hjk : j ≠ k) : eV (aP j k) * eV (aP j k) = golayEps j *
↪  golayEps k

theorem kP_sq {j k : Fin 24} (hjk : j ≠ k) : kP j k * kP j k * εΛ (gP j k) (gP j k) = 1 / 4

theorem AP_mul_self {j k : Fin 24} (hjk : j ≠ k) : AP j k * AP j k = (1 / 4 : ℂ) • 1

theorem AM_mul_self {j k : Fin 24} (hjk : j ≠ k) : AM j k * AM j k = (1 / 4 : ℂ) • 1
```

Hypotheses / status. $j \neq k$

Theorem 41.37 (the seed exists). *There is a nonzero $t \in T$ with $A_+(j,k)t = -\frac{1}{2}t$ and $A_-(j,k)t = \frac{1}{2}t$ for all $j \neq k$. Moreover one can take $t \in T_s$ for some s.*

Lean. `VOA.Moonshine.SigmaTop.seedExists`, `VOA.Moonshine.SigmaTop.seed_in_frame`
File `VOA/Moonshine/SigmaTop2Eigen.lean`.

```
theorem seedExists : SeedExists

theorem seed_in_frame (h : SeedExists) :
    ∃ s : ZMod 2, ∃ t : TLeechT, t ∈ leechFrameT s ∧ t ≠ 0 ∧ IsSeed t
```

Hypotheses / status. none

Theorem 41.47 (`classify_S0`). *Every $\gamma \in S_0$ (norm 4, in Λ_0) is either $\pm\gamma_+(j,k)$ or $\pm\gamma_-(j,k)$ for some $j \neq k$ (shape $(\pm4, \pm4, 0^{22})$), or $\gamma = \gamma_S$ for an octad D and an even $S \subseteq D$ (shape $(\pm2^8, 0^{16})$).*

Lean. `VOA.Moonshine.SigmaTop.classify_S0`

```
theorem classify_S0 {γ : leech.carrier} (hγ : γ ∈ halfFourChi 0) :
    (∃ j k : Fin 24, j ≠ k ∧ (γ = gP j k ∨ γ = -gP j k ∨ γ = gM j k ∨ γ = -gM j k)) ∨
    (∃ (D : Finset (Fin 24)) (hDA : ind D ∈ Leech.subA), D.card = 8 ∧
      ∃ S : Finset (Fin 24), _root_.Even S.card ∧ S ⊆ D ∧ γ = inclΛ (aSA hDA S))
```

File `VOA/Moonshine/SigmaTop2Class.lean`. **Hypotheses / status.** $\gamma \in S_0$

Chapter 42

The key identity and the completion of the proof

7 of the chapter's 56 numbered statements: its headline results and every definition they use.

Theorem 42.3 (One vector generates T^+). *For every nonzero $t_0 \in T$, the single vector $x_0(t_0) = h_0(-\frac{1}{2})\mathbb{1} \otimes t_0$ mode-spans $(V_\Lambda^T)^+$.*

Lean. `VOA.Moonshine.M8hKey.modeSpan_x0T`

```
theorem modeSpan_x0T {t₀ : TLeechT} (h0 : t₀ ≠ 0) : ModeSpan (fun _ : Unit => x0T t₀)
```

File `VOA/Moonshine/M8hKeySpan.lean`. **Hypotheses / status.** $t_0 \neq 0$

Theorem 42.9 ($\sigma^\natural$ is an involution compatible with V_0^+). *For triality data D:*

(a) $(\sigma^\natural)^2 = 1$*;*

(b) for every $u \in V_0^+$ and $k \in \mathbb{Z}$: $\sigma^\natural Y^\natural(u)_k = Y^\natural(\sigma u)_k \sigma^\natural$, where $Y^\natural(u)$ is the diagonal field $\mathrm{diag}(Y(u,z), Y^T(u,z))$*; consequently $\sigma^\natural Y^\natural(u)\sigma^\natural = Y^\natural(\sigma u)$;*

(c) $\sigma^\natural$ commutes with the translation operator L_{-1} of $V^\natural$, and fixes the vacuum.

Lean. `VOA.Moonshine.M8hKey.sigNat_sq`, `VOA.Moonshine.M8hKey.sigNat_YnatU_V0`, `VOA.Moonshine.M8hKey.conjF_YnatU_V0`, `VOA.Moonshine.M8hKey.sigNat_Tnat`, `VOA.Moonshine.M8hKey.sigNat_vac`

```
theorem sigNat_sq : sigNat D * sigNat D = 1

theorem sigNat_YnatU_V0 (u : V0Sub.carrier) (k : ℤ) :
    sigNat D * YnatU (u : VPlus) k = YnatU ((sig u : V0Sub.carrier) : VPlus) k * sigNat D

theorem conjF_YnatU_V0 (u : V0Sub.carrier) :
    conjF (sigNat D) (YnatU (u : VPlus)) = YnatU ((sig u : V0Sub.carrier) : VPlus)

theorem sigNat_Tnat : sigNat D * Tnat = Tnat * sigNat D

theorem sigNat_vac : sigNat D vacNat = vacNat
```

File `VOA/Moonshine/M8hKeySigma.lean`. **Hypotheses / status.** D

Theorem 42.10 (Circularity-free uniqueness). *Let F be a formal distribution with coefficients in* $\mathrm{End}\, V^\natural$ *that is local with $Y^\natural(u)$ for every $u \in V_0^+$. Fix $\beta \in \Lambda \setminus \Lambda_0$ and nonzero $t_0 \in T_0$, $t_1 \in T_1$. If every coefficient of F kills the four vectors*

$$\mathbb{1}, \qquad (x_\beta, 0), \qquad (0, x_0(t_0)), \qquad (0, x_0(t_1)),$$

then $F = 0$.

Lean. `VOA.Moonshine.M8hKey.eq_zero_of_V0local`

```
theorem eq_zero_of_V0local (F : FormalDist (End ℂ Vnat))
    (hloc : ∀ u : V0Sub.carrier, Local F (YnatU (u : VPlus)))
    {β : leech.carrier} (hβ : β ∉ lambda0) (t : ZMod 2 → TLeechT)
    (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (hvac : ∀ p, F p vacNat = 0) (h3 : ∀ p, F p (frameX0P β, 0) = 0)
    (hT : ∀ s p, F p (0, x0T (t s)) = 0) : F = 0
```

File `VOA/Moonshine/M8hKeySigma.lean`. **Hypotheses / status.** Cyclic data

Theorem 42.13 ((4.23) from the key identities). *If* `SigmaKey D β t` *holds, then (i) $\sigma^\natural Y^\natural(\Psi x)\sigma^\natural = Y^\natural((0,x))$ for $x = x_0(t_s)$; (ii) (4.23) holds for* all *pairs $t_1, t_2 \in T^+$; (iii) $V^\natural$ carries all the structure listed in the final theorem.*

Lean. `VOA.Moonshine.M8hKey.SigmaKey.conj_eq`, `VOA.Moonshine.M8hKey.tpBlockLocal_of_sigma`, `VOA.Moonshine.M8hKey.moonshine_of_sigma`

```
theorem SigmaKey.conj_eq {β : leech.carrier} {t : ZMod 2 → TLeechT} (h : SigmaKey D β t) :
    conjF (sigNat D) (YnatU (D.Ψ (x0T (t D.s)))) = YnatT leechWbar (x0T (t D.s))

theorem tpBlockLocal_of_sigma {β : leech.carrier} {t : ZMod 2 → TLeechT} (h : SigmaKey D β
↪  t)
    (t₁ t₂ : TP) : TPBlockLocal leechWbar t₁ t₂

theorem moonshine_of_sigma {β : leech.carrier} {t : ZMod 2 → TLeechT} (h : SigmaKey D β t)
↪  :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple
```

File `VOA/Moonshine/M8hKey.lean`. **Hypotheses / status.** `SigmaKey`

Theorem 42.14 (Automorphisms are isometries). *Let W be a VOA with $\mathbb{1} \neq 0$ and $L_1 W_1 = 0$, B a fully invariant normalised form, and $\alpha \in \mathrm{Aut}\, W$. Then $B(\alpha a, \alpha b) = B(a,b)$ for all a, b.*

Lean. `VOA.Moonshine.SigmaKey.aut_isometry`

```
theorem aut_isometry {W : VertexOperatorAlgebra 𝒱} (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {B : InvariantForm W}
    (hB : B.IsFullyInvariant) (hn : B.Normalized) (α : Aut W) (a b : 𝒱) :
    B.form (α a) (α b) = B.form a b
```

File `VOA/Moonshine/SigmaKeyForm.lean`. **Hypotheses / status.** CFT type, $L_1 V_1 = 0$

Theorem 42.45 (The Frenkel–Zhu reduction and its converse). *(a)* `BimodGenV3` *and* `TopPairs D` *imply* `KeyTopAll D`.

(b) `KeyTopAll D` *implies* `TopPairs D` *(so* `TopPairs` *is also necessary).*

(c) Hence `BimodGenV3` *and* `TopPairs D` *imply* `KeySkew D t`*, and with* `PhiTop`*,* `ThetaTop`*, all targets.*

Lean. `VOA.Moonshine.FZKey.keyTopAll_of_FZ`, `VOA.Moonshine.FZKey.topPairs_of_keyTopAll`, `VOA.Moonshine.FZKey.keySkew_of_FZ`, `VOA.Moonshine.FZKey.moonshine_of_FZ`

```
theorem keyTopAll_of_FZ (hgen : BimodGenV3) (htop : TopPairs D) : KeyTopAll D

theorem topPairs_of_keyTopAll (h : KeyTopAll D) : TopPairs D

theorem keySkew_of_FZ {t : ZMod 2 → TLeechT} (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t
 ↪  s ≠ 0)
    (hgen : BimodGenV3) (htop : TopPairs D) : KeySkew D t

theorem moonshine_of_FZ {β : leech.carrier} (hβ : β ∉ lambda0) {t : ZMod 2 → TLeechT}
    (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (hΦ : PhiTop D t) (hΘ : ThetaTop D t) (hgen : BimodGenV3) (htop : TopPairs D) :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple
```

File `VOA/Moonshine/FZBimoduleKey.lean`.

Hypotheses / status. `BimodGenV3`, `TopPairs`

Theorem 42.58 (The Moonshine module). *There is a vertex operator algebra structure on $V^\natural$ whose vertex operator is the block operator (42.1), with vacuum $(\mathbb{1}, 0)$, conformal vector $(\omega, 0)$, central charge 24, $V^\natural_1 = 0$, $\dim V^\natural_n =$ the coefficient of q^n in the formal q-expansion $qJ(q)$ for all $n \geq 0$ (so $\dim V^\natural_2 = 196884$), and which is simple. These data determine the structure uniquely, so the theorem constructs a specific object. The Lean statement and its full fidelity check are in §42.8.*

Lean. `VOA.Moonshine.moonshine`

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple :=
  moonshine_of_topPairs (by norm_num) topPairs_one
```

File `VOA/Moonshine/TopPairs.lean`.

Part VIII

Monstrous moonshine

Chapter 43

Monstrous moonshine: McKay–Thompson series, Hauptmoduln and the statement

7 of the chapter's 31 numbered statements: its headline results and every definition they use.

Proposition 43.3 (Basic properties of graded traces). *For every vertex operator algebra V, $n \in \mathbb{Z}$ and $g, h \in \mathrm{Aut}(V)$:*

(a) $\mathrm{tr}(1 \mid V_n) = \dim V_n$*;*

(b) $\mathrm{tr}(hgh^{-1} \mid V_n) = \mathrm{tr}(g \mid V_n)$ *(traces are class functions);*

(c) for $V = V^\natural$: every $g \in \mathrm{Aut}(V^\natural)$ maps ${V^\natural}_n$ onto ${V^\natural}_n$.

Lean. `VOA.Moonshine.ConwayNorton.gradedAutTrace_one`, `VOA.Moonshine.ConwayNorton.gradedAutTrace_conjugate`, `VOA.Moonshine.ConwayNorton.vnatAut_preserves_grading`

```
theorem gradedAutTrace_one (V : VertexOperatorAlgebra M) (n : ℤ) :
    gradedAutTrace V 1 n = (Module.finrank ℂ (V.grading n) : ℂ)

theorem gradedAutTrace_conjugate (V : VertexOperatorAlgebra M) (g h : Aut V) (n : ℤ) :
    gradedAutTrace V (h * g * h⁻¹) n = gradedAutTrace V g n

theorem vnatAut_preserves_grading (g : AutVnat) (n : ℤ) :
    (VnatVOA.grading n).map g.toLinearEquiv.toLinearMap = VnatVOA.grading n
```

File `VOA/Moonshine/MoonshineCNTrace.lean`.

Proposition 43.5 (First properties of MT_g). *For all $g, h \in \mathrm{Aut}(V^\natural)$ and $n \in \mathbb{N}$:*

(a) the n-th coefficient of MT_g is $\mathrm{tr}(g \mid {V^\natural}_{n+1})$;

(b) the constant coefficient vanishes: $\mathrm{tr}(g \mid {V^\natural}_1) = 0$;

(c) $\mathrm{MT}_{hgh^{-1}} = \mathrm{MT}_g$*;*

(d) $\mathrm{MT}_1 = \sum_{n \geq 0} [q^{n+1}](qJ)\, X^n$*, i.e. the coefficients of MT_1 are those of J; equivalently $q\, T_1 = qJ$;*

(e) $\mathrm{MT}_1 \neq qJ$ *(a reminder that the two series live in different normalisations).*

Lean. `VOA.Moonshine.ConwayNorton.mckayThompson_coeff`, `VOA.Moonshine.ConwayNorton.mckayThompson_coeff_zero`, `VOA.Moonshine.ConwayNorton.mckayThompson_conjugate`, `VOA.Moonshine.ConwayNorton.mckayThompson_one`, `VOA.Moonshine.ConwayNorton.mckayThompsonShift_one`, `VOA.Moonshine.ConwayNorton.mckayThompson_one_ne_JShift`

```
@[simp] theorem mckayThompson_coeff (g : AutVnat) (n : ℕ) :
    PowerSeries.coeff n (mckayThompson g) =
      gradedAutTrace VnatVOA g ((n : ℤ) + 1)

theorem mckayThompson_coeff_zero (g : AutVnat) :
    PowerSeries.coeff 0 (mckayThompson g) = 0

theorem mckayThompson_conjugate (g h : AutVnat) :
    mckayThompson (h * g * h⁻¹) = mckayThompson g

theorem mckayThompson_one :
    mckayThompson 1 = PowerSeries.mk (fun n => PowerSeries.coeff (n + 1) JShift)

theorem mckayThompsonShift_one : mckayThompsonShift 1 = JShift

theorem mckayThompson_one_ne_JShift : mckayThompson 1 ≠ JShift
```

File `VOA/Moonshine/MoonshineCNTrace.lean`.

Theorem 43.6 (Trace bounds). *(a) If f is a linear endomorphism of finite order of a finite-dimensional complex vector space M, then $|\operatorname{tr} f| \le \dim M$.*

(b) If g is an automorphism of finite order of a vertex operator algebra V, then $|\operatorname{tr}(g|V_n)| \le \dim V_n$ for all n.

(c) If $g \in \mathrm{Aut}(V^\natural)$ has finite order, then $|[X^n]\,\mathrm{MT}_g| \le |[X^n]\,\mathrm{MT}_1| = c(n)$ for all $n \ge 0$.

Lean. `VOA.Moonshine.ConwayNorton.finiteOrderTrace_norm_le`, `VOA.Moonshine.ConwayNorton.gradedAutTrace_norm_le`, `VOA.Moonshine.ConwayNorton.mckayThompson_coeff_norm_le`

```
theorem finiteOrderTrace_norm_le {M : Type*} [AddCommGroup M] [Module ℂ M]
    [FiniteDimensional ℂ M] (f : Module.End ℂ M) (hf : IsOfFinOrder f) :
    ‖LinearMap.trace ℂ M f‖ ≤ Module.finrank ℂ M

theorem gradedAutTrace_norm_le {M : Type*} [AddCommGroup M] [Module ℂ M]
    (V : VertexOperatorAlgebra M) (g : Aut V) (hg : IsOfFinOrder g) (n : ℤ) :
    ‖gradedAutTrace V g n‖ ≤ Module.finrank ℂ (V.grading n)

theorem mckayThompson_coeff_norm_le (g : AutVnat) (hg : IsOfFinOrder g) (n : ℕ) :
    ‖PowerSeries.coeff n (mckayThompson g)‖ ≤
      ‖PowerSeries.coeff n (mckayThompson 1)‖
```

File `VOA/Moonshine/MoonshineCNTraceBounds.lean`.

Proposition 43.13 (What the eight fields mean; on paper). *Let $\Gamma \le \mathrm{SL}_2(\mathbb{R})$ be commensurable with $\mathrm{SL}_2(\mathbb{Z})$ and contain $\left(\begin{smallmatrix}1 & 1\\ 0 & 1\end{smallmatrix}\right)$.*

(a) If f satisfies ??–??, then the width of ∞ is 1, the map $F : X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$ is an isomorphism of Riemann surfaces, $X(\Gamma)$ has genus zero, and f is the normalised Hauptmodul of Γ.

(b) Conversely, if $X(\Gamma)$ has genus zero and ∞ has width 1, the normalised Hauptmodul satisfies ??–??.

(The chapter quotes no Lean declaration for this statement; see book p. 2224.)

Definition 43.16 (The Conway–Norton property and the Conway–Norton conjecture). $g \in \mathrm{Aut}(V^\natural)$ has the *Conway–Norton property* if there are a subgroup $\Gamma \leq \mathrm{GL}_2(\mathbb{R})$ and $N \in \mathbb{N}$ such that Γ is a moonshine group of level N and T_g is a normalised Hauptmodul for Γ. The *Conway–Norton conjecture* (for $\mathrm{Aut}(V^\natural)$) is: every $g \in \mathrm{Aut}(V^\natural)$ has the Conway–Norton property. For a group homomorphism $\iota : G \to \mathrm{Aut}(V^\natural)$, *Conway–Norton via ι* means: $\iota(g)$ has the Conway–Norton property for every $g \in G$.

Lean. `VOA.Moonshine.ConwayNorton.ConwayNortonFor`, `VOA.Moonshine.ConwayNorton.ConwayNortonConjecture`, `VOA.Moonshine.ConwayNorton.ConwayNortonVia`

```
def ConwayNortonFor (g : AutVnat) : Prop :=
  ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
    IsNormalizedHauptmodul Γ (mckayThompsonFun g)

def ConwayNortonConjecture : Prop :=
  ∀ g : AutVnat, ConwayNortonFor g

def ConwayNortonVia {𝕄 : Type*} [Group 𝕄] (ι : 𝕄 →* AutVnat) : Prop :=
  ∀ g : 𝕄, ConwayNortonFor (ι g)
```

File `VOA/Moonshine/MoonshineCNStatement.lean`.

Theorem 43.24 (Constancy under a non-discrete set; Shimizu). *Let $\psi : \mathbb{H} \to \mathbb{C}$ be holomorphic.*

(a) If $C_n \in \mathrm{GL}_2(\mathbb{R})$ converge entrywise to the identity, each C_n moves some point of $\mathbb{H}$, and $\psi(C_n\tau) = \psi(\tau)$ for all n, τ, then ψ is constant.

(b) If $\psi(\tau+1) = \psi(\tau)$ and $\psi(L_y\tau) = \psi(\tau)$ for all τ, for some $0 < y < 1$, then ψ is constant.

Lean. `VOA.Moonshine.ConwayNorton.eq_of_invariant_of_entries_tendsto_one`, `VOA.Moonshine.ConwayNorton.eq_of_invariant_upper_lower_unipotent`

```
theorem eq_of_invariant_of_entries_tendsto_one {ψ : ℍ → ℂ}
    (hψ : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) ψ) {C : ℕ → GL (Fin 2) ℝ}
    (hC : ∀ i j, Tendsto (fun n => (C n : Matrix (Fin 2) (Fin 2) ℝ) i j) atTop
      (𝓝 ((1 : Matrix (Fin 2) (Fin 2) ℝ) i j)))
    (hmove : ∀ n, ∃ τ : ℍ, C n • τ ≠ τ) (hinv : ∀ n τ, ψ (C n • τ) = ψ τ) (τ σ : ℍ) :
    ψ τ = ψ σ

theorem eq_of_invariant_upper_lower_unipotent {ψ : ℍ → ℂ} (hψ : MDifferentiable 𝓘(ℂ) 𝓘(ℂ)
 ↪  ψ)
    {y : ℝ} (hy0 : 0 < y) (hy1 : y < 1) (hT : ∀ τ, ψ (glUpperUnipotent 1 • τ) = ψ τ)
    (hL : ∀ τ, ψ (glLowerUnipotent y • τ) = ψ τ) (τ σ : ℍ) : ψ τ = ψ σ
```

File `VOA/Moonshine/MoonshineCNGammaZeroShimizu.lean`.

Theorem 43.29 (Replicate equals Hauptmodul; diamond in $\pm\Gamma$). *Let f be a normalised Hauptmodul for $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ with $\Gamma_1(M) \leq \Gamma$, $M \geq 1$. Let $p > M$ be prime, $mp - nM = 1$, and f' holomorphic, 1-periodic and normalised, with $\Phi(f(\tau)) = \sum_{j<p} f((\tau+j)/p) + f'(p\tau)$ for some $\Phi : \mathbb{C} \to \mathbb{C}$. Then $f' = f$, and $\sigma \in \Gamma$ or $-\sigma \in \Gamma$.*

Lean. `VOA.Moonshine.ConwayNorton.replicate_eq_and_diamond_mem`

```
theorem replicate_eq_and_diamond_mem {Γ : Subgroup (GL (Fin 2) ℝ)} {f f' : ℍ → ℂ}
    {Φ : ℂ → ℂ} {M p : ℕ} (hH : IsNormalizedHauptmodul Γ f) (hdet : Γ.HasDetOne)
    (hΓ1 : gamma1GL M ≤ Γ) (hM : 0 < M) (hp : p.Prime) (hMp : M < p) {m n : ℤ}
```

```
    (hmn : m * p - n * M = 1) (hf'hol : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f')
    (hf'per : ∀ τ : ℍ, f' (glUpperUnipotent 1 • τ) = f' τ)
    (hf'norm : Tendsto (fun τ : ℍ => f' τ - (qPar τ)⁻¹) atImInfty (𝓝 0))
    (hrep : ∀ τ : ℍ, Φ (f τ) = ∑ j ∈ Finset.range p, f (heckePoint 1 p j one_pos hp.pos τ)
     ↪ +
      f' (heckePoint p 1 0 hp.pos one_pos τ)) :
    f' = f ∧ (mapGL ℝ (diamondLift M p m n hmn) ∈ Γ ∨ -mapGL ℝ (diamondLift M p m n hmn) ∈
     ↪ Γ)
```

File `VOA/Moonshine/MoonshineCNGammaZeroBridge.lean`.

Chapter 44

The vertex algebra of the hyperbolic plane and the tensor theory of central charge 26

9 of the chapter's 60 numbered statements: its headline results and every definition they use.

Definition 44.5 (Complex lattice frame). A *complex lattice frame* for an integral lattice L of rank r is an additive map $\alpha \mapsto \alpha^{\perp}, L \to \mathbb{C}^r$, satisfying (44.3). The *frame oscillator* of α is $\alpha(m) = \sum_a \alpha_a^{\perp} h_a(m)$ on $\mathcal{F}_r$. The Lorentzian frame is (44.4), reindexed along rank $\mathrm{II}_{1,1} = 2$.

Lean. `VOA.Moonshine.ConwayNorton.ComplexLatticeFrame`, `VOA.Moonshine.ConwayNorton.complexHyperbolicCoordinate`, `VOA.Moonshine.ConwayNorton.lorentzianComplexFrame`, `VOA.Moonshine.ConwayNorton.frameOscillator`
Files `VOA/Moonshine/MoonshineCNComplexFrame.lean`, `VOA/Moonshine/MoonshineCNOscillators.lean`.

```
/-- Complex bilinear coordinates for the integral lattice pairing. -/
structure ComplexLatticeFrame (L : IntegralLattice) where
  coordinate : L.carrier →+ (Fin (rank L) → ℂ)
  pairing : ∀ x y, ∑ a, coordinate x a * coordinate y a = (L.form x y : ℂ)

/-- An explicit complex frame: `(x+y/2, i(x-y/2))` has bilinear norm `2xy`. -/
noncomputable def complexHyperbolicCoordinate (x : lorentzianLattice.carrier) : Fin 2 → ℂ
 ↪  :=
  ![(x 0 : ℂ) + (x 1 : ℂ) / 2, Complex.I * ((x 0 : ℂ) - (x 1 : ℂ) / 2)]

/-- The explicit hyperbolic frame, reindexed to the existing Fock carrier. -/
noncomputable def lorentzianComplexFrame : ComplexLatticeFrame lorentzianLattice where
  coordinate :=
    { toFun := fun x a => complexHyperbolicCoordinate x (finCongr lorentzian_rank a)

/-- The oscillator defined by an arbitrary complex lattice frame. -/
noncomputable def frameOscillator {L : IntegralLattice} (F : ComplexLatticeFrame L)
    (α : L.carrier) (m : ℤ) : Module.End ℂ (FockN (rank L)) :=
  ∑ a, F.coordinate α a • oscN (rank L) 1 a m
```

Definition 44.8 (Near-VOA, following Gannon). A *near-VOA* on a complex vector space $\mathcal{V}$ consists of: a family of subspaces $\mathcal{V}_n$, $n \in \mathbb{Z}$, with $\mathcal{V} = \bigoplus_n \mathcal{V}_n$ (internal direct sum); a linear map $Y : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$; a vacuum $\mathbb{1} \in \mathcal{V}_0$; a conformal vector $\omega \in \mathcal{V}_2$; and a central charge $c \in \mathbb{C}$; such that, writing $L_n = \omega_{(n+1)}$: (VA1) $u_{(n)}\mathcal{V}_j \subseteq \mathcal{V}_{k+j-n-1}$ for $u \in \mathcal{V}_k$; (VA2) $Y(\mathbb{1}, z) = \mathrm{id}$; (VA3) $u_{(-1)}\mathbb{1} = u$ and $u_{(n)}\mathbb{1} = 0$ for $n \geq 0$; (VA4) all $Y(u,z), Y(v,z)$ are mutually local; (VA5) for all u, v,

$u_{(n)}v = 0$ for $n \gg 0$; (VOA1) $[L_m, L_n] = (m-n)L_{m+n} + \frac{c}{12}(m^3 - m)\delta_{m+n,0}$; (VOA2) $L_0 = n$ on $\mathcal{V}_n$; (VOA3) $Y(L_{-1}v, z) = \partial_z Y(v, z)$. No finiteness of dim $\mathcal{V}_n$ and no lower bound on n is required.

Lean. `VOA.NearVOA`
File `VOA/Core/VertexOperatorAlgebra.lean`.

```
structure NearVOA (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  /-- The internal `ℤ`-grading (pieces possibly infinite-dimensional). -/
  grading : ℤ → Submodule ℂ 𝒱
  /-- The grading is an internal direct-sum decomposition of `𝒱`. -/
  isInternal : DirectSum.IsInternal grading
  /-- The state-field correspondence. -/
  Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  /-- The vacuum vector. -/
  vacuum : 𝒱
  /-- `𝟙 ∈ 𝒱₀`. -/
  vacuum_mem : vacuum ∈ grading 0
  /-- **VA1.** -/
  grading_mode : ∀ {k ℓ : ℤ} (u : 𝒱), u ∈ grading k → ∀ (v : 𝒱), v ∈ grading ℓ →
    ∀ (n : ℤ), mode (Y u) n v ∈ grading (k + ℓ - n - 1)
  /-- **VA2.** -/
  vacuum_Y : Y vacuum = (1 : (End ℂ 𝒱)⟦z^±⟧)
  /-- **VA3.** -/
  creation : ∀ u : 𝒱, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  /-- **VA4 (locality).** -/
  locality : ∀ u v : 𝒱, Local (Y u) (Y v)
  /-- **VA5 (regularity).** -/
  regularity : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0
  /-- The conformal vector `ω`. -/
  conformalVector : 𝒱
  /-- `ω ∈ 𝒱₂`. -/
  conformal_mem : conformalVector ∈ grading 2
  /-- The central charge `c`. -/
  centralCharge : ℂ
  /-- **VOA1.** -/
  virasoro : ∀ m n : ℤ,
      ⁅mode (Y conformalVector) (m + 1), mode (Y conformalVector) (n + 1)⁆
        = ((m : ℂ) - n) • mode (Y conformalVector) (m + n + 1)
          + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * centralCharge) • (1 : End ℂ 𝒱)
             else 0)
  /-- **VOA2.** -/
  L0_grading : ∀ {n : ℤ} (v : 𝒱), v ∈ grading n → mode (Y conformalVector) 1 v = (n : ℂ) •
   ↪ v
  /-- **VOA3.** -/
  translation : ∀ v : 𝒱, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
```

Hypotheses / status. — (structure)

Theorem 44.10 (No lower bound is possible). *Let $(\mathcal{V}_n)_{n\in\mathbb{Z}}$ be* any *family of subspaces of $V_{\mathrm{II}_{1,1}}$'s state space such that each charged vacuum e^x lies in $\mathcal{V}_{\Delta(x)}$. Then there is no N with $\mathcal{V}_n = 0$ for all $n < N$. In particular the weight grading of $V_{\mathrm{II}_{1,1}}$ is not bounded below.*

Lean. `VOA.Moonshine.ConwayNorton.lorentzian_grading_not_bounded_below`, `VOA.Moonshine.ConwayNorton.lorentzianWeightGrading_not_bounded_below`
Files `VOA/Moonshine/MoonshineCNVertexTarget.lean`, `VOA/Moonshine/MoonshineCNGrading.lean`.

```
theorem lorentzian_grading_not_bounded_below
    (grading : ℤ → Submodule ℂ LorentzianFockSpace)
    (charge_mem : ∀ x : lorentzianLattice.carrier,
```

```
      lorentzianChargeState x ∈ grading (lorentzianEnergy x)) :
    ¬ ∃ N : ℤ, ∀ n : ℤ, n < N → grading n = ⊥ := by

theorem lorentzianWeightGrading_not_bounded_below :
    ¬ ∃ N : ℤ, ∀ n : ℤ, n < N → lorentzianWeightGrading n = ⊥ :=
```

Hypotheses / status. Any grading

Theorem 44.12 (Internality without positivity; infinite weight spaces). *(i) For* every *integral lattice L, $V_L = \bigoplus_{n\in\mathbb{Z}} (V_L)_n$ is an internal direct sum. (ii) For* $\mathrm{II}_{1,1}$, `halfNorm`$(\alpha) = \Delta(\alpha)$ *and* $e^\alpha \in (V)_{\Delta(\alpha)}$. *(iii) The states $e^{(i,0)}$, $i \in \mathbb{Z}$, are linearly independent; hence $(V_{\mathrm{II}_{1,1}})_0$ is infinite-dimensional.*

Lean. `VOA.Moonshine.ConwayNorton.latticeWeightGrading_isInternal`, `VOA.Moonshine.ConwayNorton.lorentzian_halfNorm`, `VOA.Moonshine.ConwayNorton.lorentzianChargeState_mem_weight`, `VOA.Moonshine.ConwayNorton.lorentzianIsotropicChargeStates_independent`, `VOA.Moonshine.ConwayNorton.lorentzianWeightZero_not_finiteDimensional`
File `VOA/Moonshine/MoonshineCNGrading.lean`.

```
/-- Internal integer grading, including indefinite lattices and infinite fibers. -/
theorem latticeWeightGrading_isInternal (L : IntegralLattice) :
    DirectSum.IsInternal (gradingLatGen L) :=

theorem lorentzian_halfNorm (α : lorentzianLattice.carrier) :
    halfNorm lorentzianLattice α = lorentzianEnergy α := by

theorem lorentzianChargeState_mem_weight (α : lorentzianLattice.carrier) :
    lorentzianChargeState α ∈ lorentzianWeightGrading (lorentzianEnergy α) := by

/-- Infinitely many linearly independent isotropic charged ground states. -/
theorem lorentzianIsotropicChargeStates_independent :
    LinearIndependent ℂ (fun i : ℤ => lorentzianChargeState ![i, 0]) := by

/-- This is an actual obstruction to the finite-weight-space `VertexAlgebra`. -/
theorem lorentzianWeightZero_not_finiteDimensional :
    ¬ FiniteDimensional ℂ (lorentzianWeightGrading 0) := by
```

Hypotheses / status. None

Theorem 44.17 (Reconstruction on the lattice state space). *Let L be even, with a complex frame F. Define the* word states $h_{a_1}(-i_1-1)\cdots h_{a_k}(-i_k-1)e^\alpha$ *and the* word fields $:\frac{\partial^{i_1}h_{a_1}}{i_1!}\cdots\frac{\partial^{i_k}h_{a_k}}{i_k!}Y(e^\alpha,z):$ *(with the FLM-cocycle charged field). There is a linear map $Y : V_L \to \mathrm{End}(V_L)[[z^{\pm1}]]$ sending every word state to its word field, such that every $Y(u,z)$ is a field, all are mutually local, $Y(u,z)\mathbb{1} = u + O(z)$, and every $Y(u,z)$ is T-covariant. The library's* `frameStateField F hEven` *is such a map, chosen once and for all.*

Lean. `VOA.Moonshine.ConwayNorton.frameWordState`, `VOA.Moonshine.ConwayNorton.frameStateField_exists`, `VOA.Moonshine.ConwayNorton.frameStateField`
Files `VOA/Moonshine/MoonshineCNReconstruction.lean`, `VOA/Moonshine/MoonshineCNStateFields.lean`.

```
noncomputable def frameWordState (L : IntegralLattice) : List (ℕ × Fin (rank L)) →
 ↪  L.carrier → LatModuleGen L
  | [], α => latChargeGen L α
  | (ia :: rest), α =>
      LinearMap.rTensor (L.carrier →₀ ℂ) (currentN (rank L) 1 ia.2 ia.1) (frameWordState L
       ↪  rest α)
```

```
theorem frameStateField_exists (hEven : L.Even)  :
    ∃ Yfull : LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)),
      (∀ (lα : List (ℕ × Fin (rank L)) × L.carrier),
        Yfull (Function.uncurry (frameWordState L) lα)
          = Function.uncurry (fun l α => frameWordField F l α) lα) ∧
      (∀ u : LatModuleGen L, IsField (Yfull u)) ∧
      (∀ u v : LatModuleGen L, Local (Yfull u) (Yfull v)) ∧
      (∀ u : LatModuleGen L, (Yfull u) 0 (latVacuumGen L) = u) ∧
      (∀ u : LatModuleGen L, ∀ q : ℤ, q < 0 → (Yfull u) q (latVacuumGen L) = 0) ∧
      (∀ u : LatModuleGen L, Reconstruction.DCovariant (frameTranslation F) (Yfull u)) :=

noncomputable def frameStateField (hEven : L.Even)  :
    LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)) :=
  (frameStateField_exists F hEven).choose
```

Hypotheses / status. Even

Theorem 44.19 (Virasoro, L_0, translation, grading). *Let L be even with a complex frame. Then, for $Y =$* `frameStateField F hEven` *and $\omega = \omega_L$: $[L_m, L_n] = (m-n)L_{m+n} + \frac{\operatorname{rank} L}{12}(m^3 - m)\delta_{m+n,0}$; $L_0 = n$ on $(V_L)_n$; $Y(L_{-1}v, z) = \partial_z Y(v, z)$; and $u_{(j)}(V_L)_q \subseteq (V_L)_{p+q-j-1}$ for $u \in (V_L)_p$.*

Lean. `VOA.Moonshine.ConwayNorton.frameStateField_virasoro`, `VOA.Moonshine.ConwayNorton.frameStateField_L0_grading`, `VOA.Moonshine.ConwayNorton.frameStateField_translation_omega`, `VOA.Moonshine.ConwayNorton.frameStateField_grading_mode`
Files `VOA/Moonshine/MoonshineCNVirasoro.lean`, `VOA/Moonshine/MoonshineCNConformalTranslation.lean`, `VOA/Moonshine/MoonshineCNGradedFields.lean`.

```
theorem frameStateField_virasoro (hEven : L.Even)  (m n : ℤ) :
    ⁅mode (frameStateField F hEven (latticeConformalVector L)) (m + 1),
      mode (frameStateField F hEven (latticeConformalVector L)) (n + 1)⁆
      = ((m : ℂ) - n) • mode (frameStateField F hEven (latticeConformalVector L)) (m + n +
       ↪ 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * (rank L : ℂ)) • (1 : End ℂ
         ↪ (LatModuleGen L))
           else 0) := by

theorem frameStateField_L0_grading (hEven : L.Even)  {n : ℤ} (v : LatModuleGen L)
    (hv : v ∈ gradingLatGen L n) :
    mode (frameStateField F hEven (latticeConformalVector L)) 1 v = (n : ℂ) • v := by

theorem frameStateField_translation_omega (hEven : L.Even)  (v : LatModuleGen L) :
    frameStateField F hEven (mode (frameStateField F hEven (latticeConformalVector L)) 0 v)
      = deriv (frameStateField F hEven v) := by

theorem frameStateField_grading_mode (hEven : L.Even)  {p q : ℤ} (u : LatModuleGen L)
    (hu : u ∈ gradingLatGen L p) (v : LatModuleGen L) (hv : v ∈ gradingLatGen L q) (j : ℤ)
     ↪ :
    mode (frameStateField F hEven u) j v ∈ gradingLatGen L (p + q - j - 1) := by
```

Hypotheses / status. Even

Theorem 44.34 (Leading term, PBW map, light-cone splitting). *Fix κ, λ. (i) For p homogeneous of b-count d and b-level w, $L_{-(j+1)}p - \kappa\, b_{j+1}p$ is strictly lower than $(d+1, w+j+1)$ in the lexicographic order. (ii) Let Φ send a monomial $a^\alpha b_{\lambda_1}\cdots b_{\lambda_k}$ ($\lambda_1 \geq \ldots \geq \lambda_k$) to $L_{-\lambda_1}\cdots L_{-\lambda_k}a^\alpha$. Then $\Phi(m) - \kappa^k m$ is strictly lower than m; hence if $\kappa \neq 0$, Φ is a linear automorphism of $\mathcal{F}^{\mathrm{iso}}$. (iii) If $\kappa \neq 0$, then*

$\mathcal{F}^{\mathrm{iso}} = \left(\sum_{n\ge 1} \operatorname{im} L_{-n}\right) \oplus \mathcal{F}^{\mathrm{iso}}(a)$*; if* $\lambda \neq 0$*, the same with* $\mathcal{F}^{\mathrm{iso}}(b)$*; so for every nonzero momentum one of the two common kernels* $\bigcap_j \ker \varphi(j)$ *is a complement of the spurious space.*

Lean. `VOA.Moonshine.ConwayNorton.hypSugawara_negative_leading_filtered`, `VOA.Moonshine.ConwayNorton.hypPBWMap`, `VOA.Moonshine.ConwayNorton.hypPBWMap_leading`, `VOA.Moonshine.ConwayNorton.hypPBWEquiv`, `VOA.Moonshine.ConwayNorton.hypNegativeSpurious`, `VOA.Moonshine.ConwayNorton.hypFock_lightConeSplit`, `VOA.Moonshine.ConwayNorton.hypFock_lightConeSplit_swapped`, `VOA.Moonshine.ConwayNorton.hypFock_lightConeSplit_nonzero`
Files `VOA/Moonshine/MoonshineCNHyperbolicFiltration.lean`, `VOA/Moonshine/MoonshineCNHyperbolicPBWMap.lean`, `VOA/Moonshine/MoonshineCNHyperbolicPBWEquiv.lean`, `VOA/Moonshine/MoonshineCNHyperbolicLightCone.lean`, `VOA/Moonshine/MoonshineCNHyperbolicSwap.lean`.

```
theorem hypSugawara_negative_leading_filtered (κ ell : ℂ) (j d w : ℕ)
    (p : hypFock) (hp : p ∈ hypBHomogeneous d w) :
    hypSugawara κ ell (-(j + 1 : ℤ)) p - κ • (X (j, 1) * p) ∈
      hypBFiltered (d + 1) (w + j + 1) := by

def hypPBWMap (κ ell : ℂ) : Module.End ℂ hypFock :=
  Finsupp.lsum ℂ fun e => LinearMap.smulRight LinearMap.id
    (stringNegativeWord (fun j => hypSugawara κ ell (-(j + 1 : ℤ)))
      (monomial (hypAExponent e) 1) (hypCanonicalBWord e))

/-- The sorted-word map is triangular with diagonal `κ` to the b-count. -/
theorem hypPBWMap_leading (κ ell : ℂ) (e : (ℕ × Fin 2) →₀ ℕ) :
    hypPBWMap κ ell (monomial e 1) -
      κ ^ (Finsupp.weight hypBCountWeight e) • monomial e 1 ∈
      hypBFiltered (Finsupp.weight hypBCountWeight e) (Finsupp.weight hypBLevelWeight e) :=
       ↪ by

/-- A linear automorphism of the polynomial space supplied by negative Sugawara words. -/
def hypPBWEquiv (κ ell : ℂ) (hκ : κ ≠ 0) : hypFock ≃ₗ[ℂ] hypFock :=

/-- The actual sum of negative Sugawara ranges. -/
def hypNegativeSpurious (κ ell : ℂ) : Submodule ℂ hypFock :=
  ⨆ j : ℕ, LinearMap.range (hypSugawara κ ell (-(j + 1 : ℤ)))

/-- Concrete Fock-space light-cone splitting with only the nonzero-momentum hypothesis. -/
theorem hypFock_lightConeSplit (κ ell : ℂ) (hκ : κ ≠ 0) :
    IsCompl (hypNegativeSpurious κ ell) (hypOneFamily 0) := by

/-- The complementary light-cone choice when the other momentum is nonzero. -/
theorem hypFock_lightConeSplit_swapped (κ ell : ℂ) (hell : ell ≠ 0) :
    IsCompl (hypNegativeSpurious κ ell) (hypOneFamily 1) := by

/-- Every nonzero momentum has a concrete choice of longitudinal common kernel. -/
theorem hypFock_lightConeSplit_nonzero (κ ell : ℂ) (h : κ ≠ 0 ∨ ell ≠ 0) :
    ∃ a : Fin 2, IsCompl (hypNegativeSpurious κ ell)
      (⨅ j : ℕ, LinearMap.ker (hypOscillator a (if a = 0 then κ else ell) (j + 1 : ℤ))) :=
       ↪ by
```

Hypotheses / status. $\kappa \neq 0$ / $(\kappa, \lambda) \neq 0$

Theorem 44.54 (Opposite Borcherds identity; adjoint closure). *Let* V *be a near-VOA with* L_1 *locally nilpotent. (i) For* $u \in \mathcal{V}_p$*, any* v*, any* q, ℓ, m, n*:*

$$\sum_i (-1)^i \binom{\ell}{i} \left(v^{\mathrm{op},q}_{(n+i)} u^{\mathrm{op},p}_{(\ell+m-i)} y \;-\; (-1)^{\ell} u^{\mathrm{op},p}_{(m+i)} v^{\mathrm{op},q}_{(\ell+n-i)} y \right) \;=\; \sum_i \binom{m}{i} (u_{(\ell+i)} v)^{\mathrm{op},\,p+q-\ell-i-1}_{(m+n-i)} y,$$

where $w^{\mathrm{op},N}$ denotes `oppositeModeAtWeight N`. *(ii) V satisfies the opposite iterate identity. (iii) Let B be any bilinear form and G a set of states generating V under all products $u_{(\ell)}v$. If every $u \in G$ satisfies $B(u_{(n)}x, y) = B(x, u^{\mathrm{op}}_{(n)}y)$, then every $u \in \mathcal{V}$ does.*

Lean. `VOA.NearVOA.oppositeModeAtWeight_borcherds`, `VOA.NearVOA.oppositeIterateIdentity_of_locallyNilpotent`, `VOA.NearVOA.adjoint_of_locallyNilpotent_generators`
Files `VOA/Moonshine/MoonshineCNNearOppositeIterate.lean`, `VOA/Moonshine/MoonshineCNNearAdjointClosure.lean`.

```
theorem oppositeModeAtWeight_borcherds (hNil : ∀ w : H, ∃ k : ℕ, (V.L 1 ^ k) w = 0)
    {p q : ℤ} {u : H} (hu : u ∈ V.grading p) (v : H) (ℓ m n : ℤ) (y : H) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (V.oppositeModeAtWeight q (n + i) v (V.oppositeModeAtWeight p (ℓ + m - i) u y)
          - negOnePow ℓ • V.oppositeModeAtWeight p (m + i) u
              (V.oppositeModeAtWeight q (ℓ + n - i) v y)))
      = ∑ᶠ i : ℤ, cbz m i •
          V.oppositeModeAtWeight (p + q - (ℓ + i) - 1) (m + n - i) (V.smul u (ℓ + i) v) y
          ↪  := by

/-- **The opposite iterate identity holds** for every near-VOA with locally nilpotent `L₁`.
 ↪ -/
theorem oppositeIterateIdentity_of_locallyNilpotent
    (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0) : V.OppositeIterateIdentity hNil := by

/-- Adjointness of all fields from adjointness of generators, for every near-VOA with
locally nilpotent `L₁`. The opposite iterate law is now proved, not assumed. -/
theorem adjoint_of_locallyNilpotent_generators
    (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0) (B : LinearMap.BilinForm ℂ H) (G : Set H)
    (hGen : ∀ S : Submodule ℂ H, G ⊆ S →
      (∀ u ∈ S, ∀ v ∈ S, ∀ ℓ : ℤ, V.smul u ℓ v ∈ S) → S = ⊤)
    (hAdj : ∀ u ∈ G, ∀ (n : ℤ) (x y : H),
      B (mode (V.Y u) n x) y = B x (V.oppositeModes hNil n u y))
    (u : H) (n : ℤ) (x y : H) :
    B (mode (V.Y u) n x) y = B x (V.oppositeModes hNil n u y) :=
```

Hypotheses / status. L_1 loc. nilpotent

Theorem 44.59 (Momentum reversal and the cross-momentum pairing). *Θ is an involution, maps $\mathcal{M}^{\alpha}$ to $\mathcal{M}^{-\alpha}$ and commutes with every L_n. For polynomials p, q: $B_L(\theta(pe^{\alpha}), qe^{\alpha}) = \chi(\alpha)\, B_{\mathcal{F}}(\sigma p, q)$, and for the sector states $B(\Theta\Psi_{\alpha}(a \otimes p), \Psi_{\alpha}(b \otimes q)) = \chi(\alpha)\, B_{\mathcal{F}}(\sigma\Phi p, \Phi q)\, B_{\natural}(a, b)$.*

Lean. `VOA.Moonshine.ConwayNorton.moonshineMatterReversal_involutive`, `VOA.Moonshine.ConwayNorton.moonshineMatterReversal_momentum`, `VOA.Moonshine.ConwayNorton.moonshineMatterReversal_virasoro`, `VOA.Moonshine.ConwayNorton.lorentzianReversal_form`, `VOA.Moonshine.ConwayNorton.moonshineMatterReversal_sector_pairing`
File `VOA/Moonshine/MoonshineCNMomentumReversal.lean`.

```
theorem moonshineMatterReversal_involutive : Function.Involutive moonshineMatterReversal :=
 ↪  by

theorem moonshineMatterReversal_momentum (α : lorentzianLattice.carrier)
    (u : moonshineMatterSpace) (hu : u ∈ moonshineMatterMomentum α) :
    moonshineMatterReversal u ∈ moonshineMatterMomentum (-α) := by

theorem moonshineMatterReversal_virasoro (n : ℤ) (u : moonshineMatterSpace) :
    moonshineMatterReversal (moonshineMatterNearVOA.L n u) =
      moonshineMatterNearVOA.L n (moonshineMatterReversal u) := by
```

```
/-- Exact cross-momentum normalization for arbitrary oscillator polynomials. -/
theorem lorentzianReversal_form (α : lorentzianLattice.carrier)
    (p q : FockN (rank lorentzianLattice)) :
    lorentzianBilinearForm (thetaLat lorentzianLattice (latStateGen lorentzianLattice p α))
      (latStateGen lorentzianLattice q α) =
      chSign lorentzianLattice α * uForm (rank lorentzianLattice)
        (sigmaN (rank lorentzianLattice) p) q := by

/-- The tensor-sector pairing after momentum reversal, before identifying its polynomial
 ↪  part. -/
theorem moonshineMatterReversal_sector_pairing (α : lorentzianLattice.carrier)
    (a b : Vnat) (p q : hypFock) :
    moonshineMatterForm
      (moonshineMatterReversal (moonshineMatterSectorEquiv α (a ⊗ₜ p)))
      (moonshineMatterSectorEquiv α (b ⊗ₜ q)) =
      chSign lorentzianLattice α *
        uForm (rank lorentzianLattice)
          (sigmaN (rank lorentzianLattice) (hypLorentzianPolynomialEquiv p))
          (hypLorentzianPolynomialEquiv q) * natForm a b := by
```

Hypotheses / status. None

Chapter 45

The Monster Lie algebra: string states, the no-ghost theorem and the Borcherds–Kac–Moody structure

6 of the chapter's 58 numbered statements: its headline results and every definition they use.

Theorem 45.5 (Orthogonals inside a weight space). *For every $N \in \mathbb{Z}$, inside the finite-dimensional space H^N with the nondegenerate form B^N:*

$$(\mathcal{S}^N)^\perp = \mathcal{P}^N, \quad (\mathcal{P}^N)^\perp = \mathcal{S}^N, \quad (\mathcal{S}_K^N)^\perp = \mathcal{K}^N, \quad (\mathcal{K}^N)^\perp = \mathcal{S}_K^N,$$

and consequently $\dim \mathcal{S}^N + \dim \mathcal{P}^N = \dim H^N = \dim \mathcal{S}_K^N + \dim \mathcal{K}^N$.

Lean. `VOA.Moonshine.ConwayNorton.StringModule.orthogonal_spurious_eq`, `VOA.Moonshine.ConwayNorton.StringModule.orthogonal_kSpurious_eq`, `VOA.Moonshine.ConwayNorton.StringModule.orthogonal_physical_eq`, `VOA.Moonshine.ConwayNorton.StringModule.orthogonal_kVacuum_eq`, `VOA.Moonshine.ConwayNorton.StringModule.physical_spurious_finrank`

```
theorem orthogonal_spurious_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.spurious N) =
      M.weightSlice M.physical N := by
theorem orthogonal_kSpurious_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.kSpurious N) =
      M.weightSlice M.kVacuum N := by
theorem orthogonal_physical_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.physical N) =
      M.weightSlice M.spurious N := by
theorem orthogonal_kVacuum_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.kVacuum N) =
      M.weightSlice M.kSpurious N := by
theorem physical_spurious_finrank (N : ℤ) :
    Module.finrank ℂ (M.weightSlice M.spurious N) +
      Module.finrank ℂ (M.weightSlice M.physical N) = Module.finrank ℂ (M.wt N) := by
```

File `VOA/Moonshine/MoonshineCNStringOrthogonals.lean`. **Hypotheses / status.** None

Theorem 45.6 (Radicals of the physical and longitudinal spaces). *In every string module, at every central charge: the radical of B on $\mathcal{P}^1$ is $\mathcal{P}^1 \cap \mathcal{S}$, and for every N the radical of B on $\mathcal{K}^N$ is $\mathcal{K}^N \cap \mathcal{S}_K$.*

Lean. `VOA.Moonshine.ConwayNorton.StringModule.physicalOne_radical_eq`, `VOA.Moonshine.ConwayNorton.StringModule.kVacuum_radical_eq`

```
theorem physicalOne_radical_eq :
    M.formRadicalIn M.physical 1 = M.weightSlice (M.physical ⊓ M.spurious) 1 := by
theorem kVacuum_radical_eq (N : ℤ) :
    M.formRadicalIn M.kVacuum N = M.weightSlice (M.kVacuum ⊓ M.kSpurious) N := by
```

File `VOA/Moonshine/MoonshineCNStringPhysical.lean`. **Hypotheses / status.** None

Theorem 45.7 (Virasoro generation). *(a) $\mathcal{S} = L_{-1}H + L_{-2}H$. (b) If $L_1 x = L_2 x = 0$ then $x \in \mathcal{P}$. (c) $L_n\mathcal{K} \subseteq \mathcal{K}$ for $n \geq 1$ and $K_n\mathcal{K} \subseteq \mathcal{K}$ for all n.*

Lean. `VOA.Moonshine.ConwayNorton.StringModule.spurious_eq_sup_two`, `VOA.Moonshine.ConwayNorton.StringModule.physical_of_L_one_two`, `VOA.Moonshine.ConwayNorton.StringModule.kVacuum_L_stable`, `VOA.Moonshine.ConwayNorton.StringModule.kVacuum_K_stable`

```
theorem spurious_eq_sup_two :
    M.spurious = LinearMap.range (M.L (-1)) ⊔ LinearMap.range (M.L (-2)) := by
theorem physical_of_L_one_two (x : H) (h₁ : M.L 1 x = 0)
    (h₂ : M.L 2 x = 0) : x ∈ M.physical :=
theorem kVacuum_L_stable (n : ℕ) (x : H) (hx : x ∈ M.kVacuum) :
    M.L (n + 1 : ℤ) x ∈ M.kVacuum := by
theorem kVacuum_K_stable (n : ℤ) (x : H) (hx : x ∈ M.kVacuum) :
    M.K n x ∈ M.kVacuum := by
```

File `VOA/Moonshine/MoonshineCNStringGenerators.lean`. **Hypotheses / status.** None

Theorem 45.22 (No-ghost theorem for $V^\natural$, nonzero momentum). *For every $(x, y) \in \mathbb{Z}^2 \setminus \{0\}$ there is a linear isomorphism*

$$\Theta_{(x,y)} : \mathcal{P}^1_{(x,y)} \big/ \mathrm{rad}\big(B|_{\mathcal{P}^1_{(x,y)}}\big) \xrightarrow{\sim} {V^\natural}_{1-xy}$$

which is an isometry for the descended form and $B^\natural$, and intertwines the action of every $g \in \mathrm{Aut}(V^\natural)$. Here $\mathcal{P}^1_{(x,y)}$ is the weight-one physical space of the string sector $H_{(x,y)}$, and $\mathrm{rad} = \mathcal{P}^1_{(x,y)} \cap \mathcal{S}$. In particular $\dim \mathcal{P}^1_{(x,y)}/\mathrm{rad} = \dim {V^\natural}_{1-xy}$.

Lean. `VOA.Moonshine.ConwayNorton.stringSectorNoGhostEquiv`, `VOA.Moonshine.ConwayNorton.stringSectorNoGhostEquiv_isometry`, `VOA.Moonshine.ConwayNorton.stringSectorNoGhostEquiv_equivariant`, `VOA.Moonshine.ConwayNorton.stringSectorNoGhost_finrank`

```
def stringSectorNoGhostEquiv (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    (stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1 ≃ₗ[ℂ]
      VnatVOA.grading (1 - x * y) :=
  ((stringSector x y h).physicalKQuotientEquiv (stringSector_charge x y h)
    (stringSector_lightConeSplit x y h)).trans
      (vnatKVacuumQuotientEquiv x y (stringSectorFamily x y h) 1)
theorem stringSectorNoGhostEquiv_isometry (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0)
    (u v : (stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1) :
    natForm (stringSectorNoGhostEquiv x y h u) (stringSectorNoGhostEquiv x y h v) =
      (stringSector x y h).radicalQuotientForm (stringSector x y h).physical 1 u v := by
theorem stringSectorNoGhostEquiv_equivariant (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0)
    (g : AutVnat)
    (u : (stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1) :
    stringSectorNoGhostEquiv x y h
      ((stringSectorAction x y h).stableWeightQuotientAction _
        (stringSectorAction x y h).action_preserves_physical 1 g u) =
      GriessAut.autRestrict g (1 - x * y) (stringSectorNoGhostEquiv x y h u) := by
```

```
theorem stringSectorNoGhost_finrank (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    Module.finrank ℂ
      ((stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1) =
      Module.finrank ℂ (VnatVOA.grading (1 - x * y)) :=
```

Files VOA/Moonshine/MoonshineCNVnatNoGhost.lean, VOA/Moonshine/MoonshineCNVnatNoGhostConsequences.lean.
Hypotheses / status. $(x, y) \neq 0$

Theorem 45.29 (Sector comparison). *For all $x, y, n, N \in \mathbb{Z}$, writing $\Psi =$* `moonshineMatterSectorMap ![x, y]`*: (a) $\Psi \circ L_n^{sector} = L_n^{V^\natural \otimes V_{\mathrm{II}_{1,1}}} \circ \Psi$; (b) Ψu has weight N iff $u \in H^N_{(x,y)}$; (c) Ψu is spurious in $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ iff u is spurious in the sector; (d) Ψu is a weight-one primary iff $u \in H^1_{(x,y)} \cap \mathcal{P}$.*

Lean. VOA.Moonshine.ConwayNorton.moonshineMatterSectorMap_virasoro, VOA.Moonshine.ConwayNorton.moonshineMatterSector_weight_iff, VOA.Moonshine.ConwayNorton.moonshineMatterSector_spurious_iff, VOA.Moonshine.ConwayNorton.moonshineMatterSector_primary_iff

```
theorem moonshineMatterSectorMap_virasoro (x y n : ℤ) (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] (vnatHypMode x y n u) =
      moonshineMatterNearVOA.L n (moonshineMatterSectorMap ![x, y] u) := by
theorem moonshineMatterSector_weight_iff (x y N : ℤ) (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] u ∈ moonshineMatterNearVOA.grading N ↔
      u ∈ vnatSectorWeight x y N := by
```

```
theorem moonshineMatterSector_spurious_iff (x y : ℤ) (a : Fin 2)
    (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] u ∈ moonshineMatterNearVOA.spurious ↔
      u ∈ (vnatStringModule x y a).spurious := by
theorem moonshineMatterSector_primary_iff (x y : ℤ) (a : Fin 2)
    (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] u ∈ moonshineMatterNearVOA.primaryWeightOne ↔
      u ∈ vnatSectorWeight x y 1 ∧ u ∈ (vnatStringModule x y a).physical := by
```

Files VOA/Moonshine/MoonshineCNSectorVirasoroComparison.lean, VOA/Moonshine/MoonshineCNSectorPhysicalTransfer.le
Hypotheses / status. None

Theorem 45.31 (The zero root space). *$\mathfrak{m}_0 \cong \mathbb{C}^2$, the isomorphism sending the class of $c_0 \mathbb{1} \otimes a_1 + c_1 \mathbb{1} \otimes b_1$ (transported to the vertex algebra) to (c_0, c_1).*

Lean. VOA.Moonshine.ConwayNorton.monsterLieZeroMomentumEquiv

```
def monsterLieZeroMomentumEquiv : monsterLieMomentum 0 ≃ₗ[ℂ] (Fin 2 → ℂ) :=
  monsterLieZeroMomentumTransport.symm.trans
    ((monsterLieSectorQuotientEquiv 0 0 0).symm.trans vnatZeroNoGhostEquiv)
```

File VOA/Moonshine/MoonshineCNRootSpaces.lean. **Hypotheses / status.** None

Chapter 46

Denominator identities, replication and genus zero

12 of the chapter's 61 numbered statements: its headline results and every definition they use.

Definition 46.2 (Unit part, Laurent series, Faber polynomials). For $c : \mathbb{N} \to \mathbb{C}$ let $u_c = 1 + \sum_{n\ge 1} c(n) q^{n+1} \in \mathbb{C}[[q]]$ (no q^1 term) and $f_c = q^{-1} u_c = q^{-1} + \sum_{n\ge 1} c(n) q^n \in \mathbb{C}((q))$. The Faber polynomials $F_m = F_{c,m} \in \mathbb{C}[X]$ are defined by strong recursion on m:

$$F_m = X^m - \sum_{j=0}^{m-1} \left([q^{-j}] f_c^{\,m}\right) F_j. \tag{46.1}$$

Lean. `VOA.Moonshine.ConwayNorton.M8.qSeriesUnit`, `VOA.Moonshine.ConwayNorton.M8.qSeriesLaurent`, `VOA.Moonshine.ConwayNorton.M8.faberPoly`

```
noncomputable def qSeriesUnit (c : ℕ → ℂ) : PowerSeries ℂ :=
  PowerSeries.mk fun k => if k = 0 then 1 else if k = 1 then 0 else c (k - 1)
noncomputable def qSeriesLaurent (c : ℕ → ℂ) : LaurentSeries ℂ :=
  HahnSeries.single (-1 : ℤ) (1 : ℂ) * HahnSeries.ofPowerSeries ℤ ℂ (qSeriesUnit c)
noncomputable def faberPoly (c : ℕ → ℂ) (m : ℕ) : ℂ[X] :=
  X ^ m - ∑ j : Fin m, C ((qSeriesLaurent c ^ m).coeff (-((j : ℕ) : ℤ))) * faberPoly c j
termination_by m
decreasing_by exact j.isLt
```

File `VOA/Moonshine/MoonshineLRFaber.lean`.

Theorem 46.4 (Faber polynomials: characterisation). *For every c and m:*

(a) F_m is monic of degree m;

(b) for every $k \le 0$, $[q^k] F_m(f_c) = 1$ if $k = -m$ and 0 otherwise; that is, $F_m(f_c) - q^{-m} \in q\,\mathbb{C}[[q]]$;

(c) if $P \in \mathbb{C}[X]$ satisfies the conclusion of (b), then $P = F_m$.

Lean. `VOA.Moonshine.ConwayNorton.M8.faberPoly_monic`, `VOA.Moonshine.ConwayNorton.M8.faberPoly_natDegree`, `VOA.Moonshine.ConwayNorton.M8.coeff_aeval_faberPoly`, `VOA.Moonshine.ConwayNorton.M8.eq_faberPoly_of_coeff`

```
theorem faberPoly_monic (c : ℕ → ℂ) (m : ℕ) : (faberPoly c m).Monic
theorem faberPoly_natDegree (c : ℕ → ℂ) (m : ℕ) : (faberPoly c m).natDegree = m
theorem coeff_aeval_faberPoly (c : ℕ → ℂ) (m : ℕ) {k : ℤ} (hk : k ≤ 0) :
    (aeval (qSeriesLaurent c) (faberPoly c m)).coeff k = if k = -(m : ℤ) then 1 else 0
theorem eq_faberPoly_of_coeff (c : ℕ → ℂ) (m : ℕ) (P : ℂ[X])
    (h : ∀ k : ℤ, k ≤ 0 → (aeval (qSeriesLaurent c) P).coeff k = if k = -(m : ℤ) then 1
    ↪  else 0) :
    P = faberPoly c m
```

File `VOA/Moonshine/MoonshineLRFaber.lean`.

Theorem 46.7 (The q-expansion of a Hecke sum). *If c is J-dominated and $M \geq 1$, then for every $\tau \in \mathbb{H}$*

$$R_M(\tau) - q^{-M} = \sum_{r \geq 1} h_c(M, r)\, q^r,$$

the series converging (in Mathlib's `HasSum` *sense) at every point.*

Lean. `VOA.Moonshine.ConwayNorton.M8.hasSum_qHeckeBlock`, `VOA.Moonshine.ConwayNorton.M8.hasSum_qHeckeSum`

```
theorem hasSum_qHeckeBlock {c : ℕ → ℂ} (hc : IsJDominated c) (a d : ℕ) (ha : 0 < a) (hd : 0
 ↪  < d)
    (τ : ℍ) :
    HasSum (fun r : ℕ => (if r ≠ 0 ∧ a ∣ r then (d : ℂ) * c (d * (r / a)) else 0) * qPar τ
     ↪  ^ r)
      (qHeckeBlock c a d τ - if d = 1 then (qPar τ ^ a)⁻¹ else 0)
theorem hasSum_qHeckeSum {c : ℕ → ℂ} (hc : IsJDominated c) {m : ℕ} (hm : 0 < m) (τ : ℍ) :
    HasSum (fun r : ℕ => (if r = 0 then 0 else qHeckeCoeff c m r) * qPar τ ^ r)
      (qHeckeSum c m τ - (qPar τ ^ m)⁻¹)
```

File `VOA/Moonshine/MoonshineLRM5Hecke.lean`.

Theorem 46.16 (Existence, uniqueness, multiplicativity). *Let $L, L_1, L_2 \in K[[p]]$ have constant term 0.*

(a) $A = \mathrm{odeSol}(L)$ *satisfies* $pA' = AL$ *and* $A(0) = 1$*;*

(b) two solutions of $pA' = AL$ *with the same constant term are equal;*

(c) $\mathrm{odeSol}(L_1 + L_2) = \mathrm{odeSol}(L_1)\,\mathrm{odeSol}(L_2)$.

Lean. `VOA.Moonshine.ConwayNorton.M8.odeSol_spec`, `VOA.Moonshine.ConwayNorton.M8.eq_of_ode`, `VOA.Moonshine.ConwayNorton.M8.odeSol_add`

```
theorem odeSol_spec (L : PowerSeries K) (hL : PowerSeries.constantCoeff L = 0) :
    PowerSeries.X * PowerSeries.derivative K (odeSol L) = odeSol L * L
theorem eq_of_ode {A B L : PowerSeries K} (hL : PowerSeries.constantCoeff L = 0)
    (hA : PowerSeries.X * PowerSeries.derivative K A = A * L)
    (hB : PowerSeries.X * PowerSeries.derivative K B = B * L)
    (h0 : PowerSeries.constantCoeff A = PowerSeries.constantCoeff B) : A = B
theorem odeSol_add (L₁ L₂ : PowerSeries K) (h₁ : PowerSeries.constantCoeff L₁ = 0)
    (h₂ : PowerSeries.constantCoeff L₂ = 0) :
    odeSol (L₁ + L₂) = odeSol L₁ * odeSol L₂
```

File `VOA/Moonshine/MoonshineLRM5Ode.lean`.

Definition 46.17 (Cone products). For $e : \mathbb{N} \times \mathbb{N} \to \mathbb{C}$ let

$$\mathrm{coneLog}(e) = -\sum_{k,m,n \geq 1} m\, e(m,n)\, p^{mk} q^{nk} \in \mathbb{C}((q))[[p]], \qquad \mathrm{coneProd}(e) = \mathrm{odeSol}\big(\mathrm{coneLog}(e)\big),$$

so that $\mathrm{coneProd}(e)$ "is" $\prod_{m,n \geq 1} (1 - p^m q^n)^{e(m,n)}$.

Lean. `VOA.Moonshine.ConwayNorton.M8.coneLogCoeff`, `VOA.Moonshine.ConwayNorton.M8.coneLog`, `VOA.Moonshine.ConwayNorton.M8.coneProd`

```
noncomputable def coneLogCoeff (e : ℕ → ℕ → ℂ) (a : ℕ) : LaurentSeries ℂ :=
  -HahnSeries.ofPowerSeries ℤ ℂ (PowerSeries.mk fun b =>
    ∑ m ∈ a.divisors, if a / m ∣ b ∧ b ≠ 0 then (m : ℂ) * e m (b / (a / m)) else 0)
noncomputable def coneLog (e : ℕ → ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  PowerSeries.mk (coneLogCoeff e)
noncomputable def coneProd (e : ℕ → ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  odeSol (coneLog e)
```

File `VOA/Moonshine/MoonshineLRM5Product.lean`.

Theorem 46.18 (The product identity (KNZ)). *(a)* $\prod_{m,n\ge 1}(1-p^m q^n)^{c(mn)} = 1-\sum_{a,b\ge 1} c(a+b-1)\,p^a q^b$ *in* $\mathbb{C}((q))[[p]]$*, where* $c(n)$ *are the coefficients of* J *and the product is* coneProd*;*

(b) $\mathrm{coneProd}(e_1+e_2) = \mathrm{coneProd}(e_1)\,\mathrm{coneProd}(e_2)$*;*

(c) if $\mathrm{coneProd}(e_1) = \mathrm{coneProd}(e_2)$ *then* $e_1(m,n) = e_2(m,n)$ *for all* $m,n \ge 1$.

Lean. `VOA.Moonshine.ConwayNorton.M8.knz_product`, `VOA.Moonshine.ConwayNorton.M8.coneProd_add`, `VOA.Moonshine.ConwayNorton.M8.coneProd_exponent_unique`

```
theorem knz_product : coneProd (fun m n => cJ (m * n)) = knzSeries cJ
theorem coneProd_add (e₁ e₂ : ℕ → ℕ → ℂ) : coneProd (e₁ + e₂) = coneProd e₁ * coneProd e₂
theorem coneProd_exponent_unique {e₁ e₂ : ℕ → ℕ → ℂ} (h : coneProd e₁ = coneProd e₂) :
    ∀ m n : ℕ, 0 < m → 0 < n → e₁ m n = e₂ m n
```

File `VOA/Moonshine/MoonshineLRM5Product.lean`.

Definition 46.19 (Euler derivation; the twisted series; the twisted denominator identity). In $\mathbb{C}[[q]][[p]]$ let $D = p\,\partial_p$. For $c : \mathbb{N}\to\mathbb{N}\to\mathbb{C}$ (with $c(k,\cdot)$ the coefficient sequence of the k-th replicate) define

$$\tilde u = \sum_{a,b\ge 1} c(1,a+b-1)\,p^a q^{a+b}, \qquad \tilde P = \sum_{k\ge 1}\sum_{a,b\ge 1} a\,c(k,ab)\,p^{ak}q^{(a+b)k},$$

and say that c satisfies the *twisted denominator identity* if $D\tilde u = (1-\tilde u)\tilde P$.

Lean. `VOA.Moonshine.ConwayNorton.eulerDeriv`, `VOA.Moonshine.ConwayNorton.twistedUnitSeries`, `VOA.Moonshine.ConwayNorton.twistedPowerSum`, `VOA.Moonshine.ConwayNorton.TwistedDenominatorIdentity`

```
def eulerDeriv (F : PowerSeries R) : PowerSeries R :=
  PowerSeries.mk fun n => (n : R) * PowerSeries.coeff n F
def twistedUnitSeries (c : ℕ → ℂ) : PowerSeries (PowerSeries ℂ) :=
  PowerSeries.mk fun a => PowerSeries.mk fun N => if 1 ≤ a ∧ a < N then c (N - 1) else 0
def twistedPowerSum (c : ℕ → ℕ → ℂ) : PowerSeries (PowerSeries ℂ) :=
  PowerSeries.mk fun A => PowerSeries.mk fun N =>
    ∑ k ∈ A.divisors,
      if k ∣ N ∧ A < N then ((A / k : ℕ) : ℂ) * c k (A / k * ((N - A) / k)) else 0
def TwistedDenominatorIdentity (c : ℕ → ℕ → ℂ) : Prop :=
  eulerDeriv (twistedUnitSeries (c 1)) = (1 - twistedUnitSeries (c 1)) * twistedPowerSum c
```

File `VOA/Moonshine/MoonshineCNReplicationFormal.lean`. **Hypotheses / status.** Definition

Theorem 46.40 (The tower down to J). *(a) For any* $c : \mathbb{N}\to\mathbb{N}\to\mathbb{C}$ *and* k*: if for every* $m\ge 1$ *the numbers* $\rho_k(m,r) = \sum_{a\mid(m,r)} \frac{m}{a}\,c(ka, \frac{mr}{a^2})$*,* $r\ge 1$*, are bounded, then for every* $m\ge 1$ *the sequence* $\ell \mapsto c(km,\ell)$*,* $\ell\ge 1$*, is bounded.*

(b) For a replicable family of period $N\ge 1$ *and every* k*, the numbers* $\rho_k(m,r)$ *are* not *bounded for every* m*.*

Lean. `VOA.Moonshine.ConwayNorton.M8.bounded_replicate_of_bounded_faberRemainderCoeff`, `VOA.Moonshine.ConwayNorton.M8.ReplicableFamily_c_mul_period`, `VOA.Moonshine.ConwayNorton.M8.not_bounded_faberRemainderCoeff`

```
theorem bounded_replicate_of_bounded_faberRemainderCoeff (c : ℕ → ℕ → ℂ) (k : ℕ)
    (h : ∀ m : ℕ, 1 ≤ m → ∃ M : ℝ, ∀ r : ℕ, 1 ≤ r → ‖faberRemainderCoeff c k m r‖ ≤ M) :
    ∀ m : ℕ, 1 ≤ m → ∃ M : ℝ, ∀ ℓ : ℕ, 1 ≤ ℓ → ‖c (k * m) ℓ‖ ≤ M
theorem ReplicableFamily_c_mul_period {N : ℕ} (F : ReplicableFamily N) (k : ℕ) :
    F.c (k * N) = F.c 0
theorem not_bounded_faberRemainderCoeff {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N) (k :
 ↪ ℕ) :
    ¬ ∀ m : ℕ, 1 ≤ m → ∃ M : ℝ, ∀ r : ℕ, 1 ≤ r → ‖faberRemainderCoeff F.c k m r‖ ≤ M
```

File `VOA/Moonshine/MoonshineLRTower.lean`.

Theorem 46.51 (Which explicit solutions are global symmetries). *Let $\mathfrak{c}$ be J-dominated and $f = \mathtt{qSeriesFun}\,\mathfrak{c}$.*

(a) No $Ke^{Az} + M_0$ $(A, K \neq 0)$ maps $\mathbb{H}$ into $\mathbb{H}$.

(b) No $B\log(z - c_0) + K$ maps $\mathbb{H}$ into $\mathbb{H}$ and satisfies $f \circ \beta = f$ on $\mathbb{H}$.

(c) If $Bz + K$ does, then $B = 1$ and $K \in \mathbb{Z}$.

(d) If $K(z - c_0)^A + M_0$ $(A, K \neq 0)$ does, then either $A = 1$ and $K > 0$ real, or $A = -1$ and $K < 0$ real.

Lean. `VOA.Moonshine.ConwayNorton.false_of_exp_family`, `VOA.Moonshine.ConwayNorton.false_of_log_family`, `VOA.Moonshine.ConwayNorton.exists_int_of_affine_family`, `VOA.Moonshine.ConwayNorton.power_family_cases`

```
theorem false_of_exp_family {A M₀ : ℝ} {K : ℂ} (hA : A ≠ 0) (hK : K ≠ 0)
    (hinto : ∀ z : ℂ, 0 < z.im → 0 < (K * Complex.exp (A * z) + M₀).im) : False
theorem false_of_log_family {c : ℕ → ℂ} (hc : IsJDominated c) {B c₀ : ℝ} {K : ℂ}
    (hglob : ∀ z : ℂ, 0 < z.im → 0 < ((B : ℂ) * Complex.log (z - c₀) + K).im ∧
      qSeriesExt c ((B : ℂ) * Complex.log (z - c₀) + K) = qSeriesExt c z) : False
theorem exists_int_of_affine_family {c : ℕ → ℂ} (hc : IsJDominated c) {B : ℝ} {K : ℂ}
    (hglob : ∀ z : ℂ, 0 < z.im → 0 < ((B : ℂ) * z + K).im ∧
      qSeriesExt c ((B : ℂ) * z + K) = qSeriesExt c z) :
    B = 1 ∧ ∃ n : ℤ, K = n
theorem power_family_cases {c : ℕ → ℂ} (hc : IsJDominated c) {c₀ A M₀ : ℝ} {K : ℂ}
    (hA : A ≠ 0) (hK : K ≠ 0)
    (hglob : ∀ z : ℂ, 0 < z.im → 0 < (K * (z - c₀) ^ (A : ℂ) + M₀).im ∧
      qSeriesExt c (K * (z - c₀) ^ (A : ℂ) + M₀) = qSeriesExt c z) :
    (A = 1 ∧ ∃ a : ℝ, 0 < a ∧ K = a) ∨ (A = -1 ∧ ∃ a : ℝ, a < 0 ∧ K = a)
```

File `VOA/Moonshine/MoonshineCNGenusZeroClassification.lean`.

Theorem 46.58 (Level). *Let F have period $N \geq 1$, satisfy the prime Hecke identity for k, and let $f = f_k$ not be injective modulo translations. Then:*

(a) (rationality) for $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in G$ with $c \neq 0$: $a/c \in \mathbb{Q}$ and $c^2 \in \mathbb{Q}$;

(b) (an integral element) G contains an integral matrix $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$ with $C \neq 0$, $N \mid C$, $D \equiv 1 \pmod{N}$;

(c) (integrality of the Hecke step) if $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right) \in G$ is integral, $C \neq 0$, $p > |C|$ is a prime with the prime Hecke identity and $p \mid D$, then $\left(\begin{smallmatrix} pA & B \\ C & D/p \end{smallmatrix}\right) \in G$;

(d) (a lower unipotent) $\left(\begin{smallmatrix} 1 & 0 \\ L & 1 \end{smallmatrix}\right) \in G$ *for some integer* $L \neq 0$ *with* $N \mid L$*;*

(e) (all first columns) if moreover $L \mid c_0$*,* $a_0 \equiv 1 \pmod{L}$ *and* $\gcd(a_0, c_0) = 1$*, some integral element of* G *has first column* (a_0, c_0)*;*

(f) $\Gamma_1(|L|) \leq G$*; hence* $\Gamma_1(N') \leq G$ *for some* $N' \geq 1$ *with* $N \mid N'$*.*

Lean. `VOA.Moonshine.ConwayNorton.exists_rat_upperLeft_div_lowerLeft`, `VOA.Moonshine.ConwayNorton.exists_rat_lowerLeft_sq`, `VOA.Moonshine.ConwayNorton.exists_integral_mem_symmetryGroup`, `VOA.Moonshine.ConwayNorton.exists_mem_symmetryGroup_heckeStep_of_integral`, `VOA.Moonshine.ConwayNorton.exists_glLowerUnipotent_mem_symmetryGroup`, `VOA.Moonshine.ConwayNorton.exists_mem_symmetryGroup_firstColumn`, `VOA.Moonshine.ConwayNorton.gamma1GL_le_symmetryGroup_of_lowerUnipotent`, `VOA.Moonshine.ConwayNorton.ReplicableFamily.exists_gamma1GL_le_symmetryGroup`

```
theorem exists_rat_upperLeft_div_lowerLeft (hN : 0 < N) (hB1 : PrimeHeckeIdentity F k)
    {g : GL (Fin 2) ℝ} (hg : g ∈ symmetryGroup (qSeriesFun (F.c k))) (hc0 : g 1 0 ≠ 0) :
    ∃ q : ℚ, g 0 0 / g 1 0 = q
theorem exists_rat_lowerLeft_sq (hN : 0 < N) (hB1 : PrimeHeckeIdentity F k)
    {g : GL (Fin 2) ℝ} (hg : g ∈ symmetryGroup (qSeriesFun (F.c k))) (hc0 : g 1 0 ≠ 0) :
    ∃ q : ℚ, g 1 0 ^ 2 = q
theorem exists_integral_mem_symmetryGroup (hN : 0 < N) (hB1 : PrimeHeckeIdentity F k)
    (hD7 : ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))) :
    ∃ m ∈ symmetryGroup (qSeriesFun (F.c k)), ∃ A B C D : ℤ, m 0 0 = A ∧ m 0 1 = B ∧
      m 1 0 = C ∧ m 1 1 = D ∧ C ≠ 0 ∧ (N : ℤ) ∣ C ∧ D ≡ 1 [ZMOD N]
theorem exists_mem_symmetryGroup_heckeStep_of_integral {p : ℕ} (hp : p.Prime)
    (hB1p : PrimeHeckeIdentityAt F k p hp.pos) {m : GL (Fin 2) ℝ}
    (hm : m ∈ symmetryGroup (qSeriesFun (F.c k))) {A B C D : ℤ} (hA : m 0 0 = A)
    (hB : m 0 1 = B) (hC : m 1 0 = C) (hD : m 1 1 = D) (hC0 : C ≠ 0) (hCp : |C| < p)
    (hpD : (p : ℤ) ∣ D) :
    ∃ m' ∈ symmetryGroup (qSeriesFun (F.c k)), m' 0 0 = p * A ∧ m' 0 1 = B ∧ m' 1 0 = C ∧
      m' 1 1 = D / p
theorem exists_glLowerUnipotent_mem_symmetryGroup (hN : 0 < N) (hB1 : PrimeHeckeIdentity F
↪  k)
    (hD7 : ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))) :
    ∃ L : ℤ, L ≠ 0 ∧ (N : ℤ) ∣ L ∧ glLowerUnipotent L ∈ symmetryGroup (qSeriesFun (F.c k))
theorem exists_mem_symmetryGroup_firstColumn (hB1 : PrimeHeckeIdentity F k)
    {L : ℤ} (hNL : (N : ℤ) ∣ L)
    (hLmem : glLowerUnipotent L ∈ symmetryGroup (qSeriesFun (F.c k))) {a₀ c₀ : ℤ}
    (hc₀ : L ∣ c₀) (ha₀ : a₀ ≡ 1 [ZMOD L]) (hcop : IsCoprime a₀ c₀) :
    ∃ m ∈ symmetryGroup (qSeriesFun (F.c k)), ∃ B D : ℤ, m 0 0 = a₀ ∧ m 0 1 = B ∧
      m 1 0 = c₀ ∧ m 1 1 = D
theorem gamma1GL_le_symmetryGroup_of_lowerUnipotent (hB1 : PrimeHeckeIdentity F k)
    {L : ℤ} (hNL : (N : ℤ) ∣ L)
    (hLmem : glLowerUnipotent L ∈ symmetryGroup (qSeriesFun (F.c k))) :
    gamma1GL L.natAbs ≤ symmetryGroup (qSeriesFun (F.c k))
theorem ReplicableFamily.exists_gamma1GL_le_symmetryGroup {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) (k : ℕ) (hB1 : PrimeHeckeIdentity F k)
    (hD7 : ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))) :
    ∃ N' : ℕ, 0 < N' ∧ N ∣ N' ∧ gamma1GL N' ≤ symmetryGroup (qSeriesFun (F.c k))
```

File `VOA/Moonshine/MoonshineCNGenusZeroLevel.lean`.

Theorem 46.60 (Commensurability). *If* $\mathfrak{c}$ *is* J*-dominated,* $f = \mathtt{qSeriesFun}\ \mathfrak{c}$ *has property* P *and* $\Gamma_1(M) \leq G$ *(*$M \geq 1$*), then* G *is commensurable with* $\mathrm{SL}_2(\mathbb{Z})$*.*

Lean. `VOA.Moonshine.ConwayNorton.exists_smul_eq_smul_fd`, `VOA.Moonshine.ConwayNorton.exists_im_bound_norm_sub_ge`, `VOA.Moonshine.ConwayNorton.commensurable_symmetryGroup_of_gamma1GL_le`

```
theorem exists_smul_eq_smul_fd (G : Subgroup (GL (Fin 2) ℝ)) (τ : ℍ) :
    ∃ r : SL(2, ℤ) ⧸ G.comap (mapGL ℝ), ∃ g : SL(2, ℤ), mapGL ℝ g ∈ G ∧
      ∃ σ ∈ 𝒟, mapGL ℝ g • τ = mapGL ℝ (Quotient.out r)⁻¹ • σ
theorem exists_im_bound_norm_sub_ge (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {δ : GL (Fin 2) ℝ} (hδ : δ.det.val = 1) {h : ℝ} (hh : 0 < h)
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c)) {v : ℂ}
    (hv : (∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ) 1 0 ≠ 0) →
      ∀ w, Tendsto (fun τ : ℍ => qSeriesFun c (δ • τ)) atImInfty (𝓝 w) → v ≠ w) :
    ∃ Υ ε : ℝ, 0 < ε ∧ ∀ σ : ℍ, Υ ≤ σ.im → ε ≤ ‖qSeriesFun c (δ • σ) - v‖
theorem commensurable_symmetryGroup_of_gamma1GL_le (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {M : ℕ} (hM : 0 < M) (hΓ1 : gamma1GL M ≤ symmetryGroup (qSeriesFun c)) :
    Subgroup.Commensurable (symmetryGroup (qSeriesFun c)) 𝒮ℒ
```

File `VOA/Moonshine/MoonshineCNGenusZeroIndex.lean`.

Theorem 46.63 (The genus-zero theorem for replicable families). *Let $N \geq 1$ and F a replicable family of period N satisfying the prime Hecke identity for every member, with no member injective modulo translations. For every k there are $\Gamma \leq \mathrm{GL}_2(\mathbb{R})$ and $N' \geq 1$ with $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$, $\Gamma_1(N') \leq \Gamma$, Γ commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and f_k a normalised Hauptmodul of Γ. (The proof takes $\Gamma = G(f_k)$ and also obtains $N \mid N'$; neither fact is part of the Lean type.)*

Lean. `VOA.Moonshine.ConwayNorton.ReplicableFamily.exists_hauptmodul_gammaOne`

```
theorem ReplicableFamily.exists_hauptmodul_gammaOne {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) (hB1 : ∀ k, PrimeHeckeIdentity F k)
    (hD7 : F.NotInjectiveModTranslations) (k : ℕ) :
    ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N' : ℕ), 0 < N' ∧ Γ.HasDetOne ∧ gamma1GL N' ≤ Γ ∧
      Subgroup.Commensurable Γ 𝒮ℒ ∧ IsNormalizedHauptmodul Γ (qSeriesFun (F.c k))
```

File `VOA/Moonshine/MoonshineCNGenusZeroMain.lean`.

Chapter 47

Conway–Norton for the automorphism group of the moonshine module

13 of the chapter's 27 numbered statements: its headline results and every definition they use.

Definition 47.2 (Griess algebra, its automorphism group, restriction). Let W be a vertex operator algebra with $\mathbf{1} \neq 0$. On W_2 put the product $u \times v = u_{(1)}v$ and the *intrinsic form* $\langle u, v\rangle_3$ defined by $u_{(3)}v = \langle u, v\rangle_3\,\mathbf{1}$ (note $u_{(3)}v \in W_0$). GriessAut(W) is the group of linear automorphisms of W_2 that preserve $\times$ and $\langle\cdot,\cdot\rangle_3$. Restriction $g \mapsto g|_{W_2}$ is a group homomorphism $\mathrm{Aut}(W) \to \mathrm{GriessAut}(W)$. W is *generated by* W_2 if every subspace containing W_2 and closed under all n-th products is W. For $W = V^\natural$ these are `GriessAutVnat` and `vnatGriessRestrict`.

Lean. `VOA.GriessAut.griessAut`, `VOA.GriessAut.griessRestrict`, `VOA.GriessAut.GeneratedByWeightTwo`, `VOA.GriessAut.griessForm`, `VOA.GriessAut.griessForm_spec`, `VOA.Moonshine.AutMonster.vnat_griessMul_comm`, `VOA.Moonshine.AutMonster.GriessAutVnat`, `VOA.Moonshine.AutMonster.vnatGriessRestrict`

```
noncomputable def griessAut (hvac : W.vacuum ≠ 0) : Subgroup (W.grading 2 ≃ₗ[ℂ] W.grading
↪  2) where
  carrier := {g | (∀ u v, g (W.griessMul u v) = W.griessMul (g u) (g v)) ∧
    ∀ u v, griessForm hvac (g u) (g v) = griessForm hvac u v}

noncomputable def griessRestrict (hvac : W.vacuum ≠ 0) : Aut W →* griessAut hvac where
  toFun α := ⟨autRestrict α 2, fun u v => griessMul_aut α u v, fun u v => griessForm_aut
  ↪  hvac α u v⟩

def GeneratedByWeightTwo (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ S : Submodule ℂ 𝒱, W.grading 2 ≤ S → (∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ, W.smul u n v ∈ S) → S
  ↪  = ⊤

noncomputable def griessForm (hvac : W.vacuum ≠ 0) (u v : W.grading 2) : ℂ :=
  RigidBridge.vacCoeff W hvac (griessThree u v)

theorem griessForm_spec (hvac : W.vacuum ≠ 0) (u v : W.grading 2) :
    W.smul (u : 𝒱) 3 (v : 𝒱) = griessForm hvac u v • W.vacuum :=
```

```
theorem vnat_griessMul_comm (u v : VnatVOA.grading 2) :
    VnatVOA.griessMul u v = VnatVOA.griessMul v u :=

noncomputable abbrev GriessAutVnat := GriessAut.griessAut VnatVOA_vacuum_ne_zero

noncomputable def vnatGriessRestrict : AutVnat →* GriessAutVnat :=
  GriessAut.griessRestrict VnatVOA_vacuum_ne_zero
```

Files `VOA/Core/GriessAutomorphism.lean`, `VOA/Moonshine/AutVnat.lean`.

Theorem 47.4 (Aut($V^\natural$) acts faithfully on the Griess algebra). *An automorphism of $V^\natural$ that fixes ${V^\natural}_2$ pointwise is the identity. Consequently restriction* $\mathrm{Aut}(V^\natural) \to \mathrm{GriessAut}(V^\natural)$, $g \mapsto g|_{{V^\natural}_2}$, *is an injective group homomorphism.*

Lean. `VOA.Moonshine.AutMonster.aut_eq_one_of_fix_frame`, `VOA.Moonshine.AutMonster.aut_eq_one_of_fix_weightTwo`, `VOA.Moonshine.AutMonster.vnatGriessRestrict_injective`

```
theorem aut_eq_one_of_fix_frame (g : AutVnat) (hg : ∀ v ∈ VnatVOA.grading 2, g v = v)
    (hx : ∀ β : leech.carrier, g ((frameX0P β, 0) : Vnat) = ((frameX0P β, 0) : Vnat)) : g =
     ↪ 1 := by

theorem aut_eq_one_of_fix_weightTwo (g : AutVnat) (hg : ∀ v ∈ VnatVOA.grading 2, g v = v) :
    g = 1 := by

theorem vnatGriessRestrict_injective : Function.Injective vnatGriessRestrict := by
```

File `VOA/Moonshine/AutFaithfulAssembly.lean`.

Lemma 47.5 (Zero mode of q_{ab} on a lattice VOA). *For every even positive-definite lattice L, frame directions a, b, polynomial q in the oscillators and charge $\gamma \in L$, formula (47.1) holds, with N_{ab} given on polynomials by the finite sum over any level bound K of q.*

Lean. `VOA.LatGen.quadState`, `VOA.LatGen.smul_quadState_one`, `VOA.HeisenbergN.nprod_currentN_neg_two`

```
noncomputable abbrev quadState (a b : Fin (rank L)) : LatModuleGen L :=
  wordStateLatGen L [(0, a), (0, b)] 0

theorem smul_quadState_one (hEven : L.Even) (hPos : L.PosDef) (a b : Fin (rank L))
    (q : FockN (rank L)) (γ : L.carrier) :
    (latticeVOAGen L hEven hPos).smul (quadState L a b) 1 (latStateGen L q γ)
      = latStateGen L
          ((chargeCoord L γ a * chargeCoord L γ b) • q
            + nprod (currentN_isField (rank L) 1 a) (currentN_isField (rank L) 1 b) (-2) q)
             ↪ γ := by
```

```
theorem nprod_currentN_neg_two (a b : Fin r) (q : FockN r) (K : ℕ)
    (hK : ∀ v ∈ q.vars, v.1 < K) :
    nprod (currentN_isField r 1 a) (currentN_isField r 1 b) (-2) q
      = ∑ k ∈ Finset.range K, ((k : ℂ) + 1) •
          (X (k, a) * pderiv (k, b) q + X (k, b) * pderiv (k, a) q) := by
```

Files `VOA/Moonshine/AutZeroModes.lean`, `VOA/Moonshine/AutFockEigen.lean`.

Lemma 47.6 (The joint-eigenvector lemma). *Let r have two distinct directions, $\mu : \mathbb{N} \to \mathbb{C}$ injective with $\mu_k + \mu_{k'} \neq 0$ for all k, k', and $N^\mu_{ab} = \sum_{k<K} \mu_k (X_{k,a}\partial_{k,b} + X_{k,b}\partial_{k,a})$ on $\mathbb{C}[X_{k,a}]$. If $q \neq 0$ has all variables of level $< K$ and $N^\mu_{ab} q = c_{ab} q$ for all a, b, then $c_{ab} = 0$ for all a, b and q is constant. Consequently (Lean:* `eigen_quadState_span`*), in a lattice VOA V_L of rank ≥ 2, a joint eigenvector of all $o(q_{ab})$ with eigenvalues $\beta_a\beta_b$ lies in $\mathbb{C}e^\beta + \mathbb{C}e^{-\beta}$; and (Lean:* `eigen_plus`*) in V^+_Λ it lies in $\mathbb{C}x_\beta$.*

Lean. `VOA.HeisenbergN.quadOp`, `VOA.HeisenbergN.quadOp_joint_eigen`, `VOA.Moonshine.AutMonster.eigen_quadState_span`, `VOA.Moonshine.AutMonster.eigen_plus`

```
noncomputable def quadOp (μ : ℕ → ℂ) (K : ℕ) (a b : Fin r) (q : FockN r) : FockN r :=
  ∑ k ∈ Finset.range K, μ k • (X (k, a) * pderiv (k, b) q + X (k, b) * pderiv (k, a) q)
```

```
theorem quadOp_joint_eigen (μ : ℕ → ℂ) (hsum : ∀ k k', μ k + μ k' ≠ 0)
    (hinj : ∀ k k', μ k = μ k' → k = k') (hr : ∀ b : Fin r, ∃ a, a ≠ b) (K : ℕ)
    (q : FockN r) (hK : ∀ v ∈ q.vars, v.1 < K) (c : Fin r → Fin r → ℂ)
    (h : ∀ a b, quadOp μ K a b q = c a b • q) (hq : q ≠ 0) :
    (∀ a b, c a b = 0) ∧ q = MvPolynomial.C (coeff 0 q) := by
```

```
theorem eigen_quadState_span (hEven : L.Even) (hPos : L.PosDef)
    (hr : ∀ b : Fin (rank L), ∃ a, a ≠ b) (β : L.carrier) (v : LatModuleGen L)
    (h : ∀ a b, (latticeVOAGen L hEven hPos).smul (quadState L a b) 1 v
      = (chargeCoord L β a * chargeCoord L β b) • v) :
    ∃ c₁ c₂ : ℂ, v = c₁ • latChargeGen L β + c₂ • latChargeGen L (-β) := by
```

```
theorem eigen_plus (β : leech.carrier) (x : VPlus)
    (h : ∀ a b, leechVOAPlus.smul (qP a b) 1 x = (chargeCoord leech β a * chargeCoord leech
    ↪ β b) • x) :
    ∃ c : ℂ, x = c • frameX0P β := by
```

Files `VOA/Moonshine/AutFockEigen.lean`, `VOA/Moonshine/AutChargeEigen.lean`, `VOA/Moonshine/AutFaithfulVnat.lean`.

Lemma 47.7 (Twisted zero modes and twisted exclusion). *On the twisted block, $(q_{ab},0)_{(1)}(0,t) = \big(0,((N^{\mathrm{tw}}_{ab}+\frac{1}{8}\delta_{ab})\otimes 1)t\big)$. If $\beta\in\mathbb{C}^{24}$ and $(u,0)_{(1)}(0,t)=\beta_a\beta_b(0,t)$ for all a,b and all $u\in V^+_\Lambda$ whose underlying state is $h_a(-1)h_b(-1)\mathbf{1}$, then $t=0$.*

Lean. `VOA.Moonshine.AutZeroTw.VnatVOA_smul_quad_one_snd`, `VOA.Moonshine.AutZeroTw.VnatVOA_twisted_outer_eigen_eq_zero`

```
theorem VnatVOA_smul_quad_one_snd (a b : Fin (rank leech)) (u : SkewVP)
    (hu : (u : LatModuleGen leech) = LatGen.quadState leech a b) (t : TP) :
    (VnatVOA.smul ((u, 0) : Vnat) 1 ((0, t) : Vnat)).1 = 0 ∧
      (((VnatVOA.smul ((u, 0) : Vnat) 1 ((0, t) : Vnat)).2 : TP) : Tw)
        = (quadNT 24 (cc a) (cc b)
            + (if a = b then (1 / 8 : ℂ) else 0) • (1 : End ℂ (FockT 24))).rTensor leechT
            ↪ (t : Tw) := by
```

```
theorem VnatVOA_twisted_outer_eigen_eq_zero (β' : Fin (rank leech) → ℂ) (t : TP)
    (h : ∀ a b : Fin (rank leech), ∀ u : SkewVP,
      (u : LatModuleGen leech) = LatGen.quadState leech a b →
        VnatVOA.smul ((u, 0) : Vnat) 1 ((0, t) : Vnat) = (β' a * β' b) • ((0, t) : Vnat)) :
    t = 0 := by
```

Files `VOA/Moonshine/AutZeroModesTwisted.lean`, `VOA/Moonshine/AutTwistedExclusion.lean`.

Lemma 47.8 (The leading product). *For $\alpha,\gamma\in\Lambda$ with $\langle\alpha,\gamma\rangle<0$, (47.2) holds in V^+_Λ, and hence $(x_\alpha,0)_{(-\langle\alpha,\gamma\rangle-1)}(x_\gamma,0)=\varepsilon(\alpha,\gamma)(x_{\alpha+\gamma},0)$ in $V^\natural$.*

Lean. `VOA.Moonshine.AutMonster.frameX0P_smul_frameX0P_of_neg`, `VOA.Moonshine.AutMonster.VnatVOA_smul_xN_of_neg`

```
theorem frameX0P_smul_frameX0P_of_neg (α γ : leech.carrier) (h : leech.form α γ < 0) :
    leechVOAPlus.smul (frameX0P α) (-(leech.form α γ) - 1) (frameX0P γ)
      = latticeCocycle leech α γ • frameX0P (α + γ) := by
```

```
theorem VnatVOA_smul_xN_of_neg (α γ : leech.carrier) (h : leech.form α γ < 0) :
    VnatVOA.smul ((frameX0P α, 0) : Vnat) (-(leech.form α γ) - 1) ((frameX0P γ, 0) : Vnat)
      = latticeCocycle leech α γ • ((frameX0P (α + γ), 0) : Vnat) := by
```

Files `VOA/Moonshine/AutFaithful.lean`, `VOA/Moonshine/AutFaithfulVnat.lean`.

Theorem 47.9 (The Leech lattice is generated by its minimal vectors). *Every $\beta \in \Lambda$ is an integral linear combination of vectors of norm 4. Equivalently: if a property P of lattice vectors holds at 0 and is preserved by $\gamma \mapsto \gamma \pm \alpha$ for every minimal α, it holds on all of Λ.*

Lean. `VOA.Moonshine.LeechSpan.leech_mem_closure_minimal`, `VOA.Moonshine.LeechSpan.leech_closure_minimal_eq_top`, `VOA.Moonshine.LeechSpan.leech_induction`

```
theorem leech_mem_closure_minimal (β : leech.carrier) :
    β ∈ AddSubgroup.closure {α : leech.carrier | leech.form α α = 4} := by

theorem leech_closure_minimal_eq_top :
    AddSubgroup.closure {α : leech.carrier | leech.form α α = 4} = ⊤ :=

theorem leech_induction (P : leech.carrier → Prop) (h0 : P 0)
    (hadd : ∀ γ α : leech.carrier, leech.form α α = 4 → P γ → P (γ + α))
    (hsub : ∀ γ α : leech.carrier, leech.form α α = 4 → P γ → P (γ - α))
    (β : leech.carrier) : P β := by
```

File `VOA/Moonshine/AutLeechSpan.lean`.

Theorem 47.11 ($\sigma^\natural \in \mathrm{Aut}(V^\natural)$). *The involution $\sigma^\natural =$ `sigNat Dnat` is an automorphism of $V^\natural$: it is a linear involution, it intertwines every product, and it fixes the vacuum and the conformal vector.*

Lean. `VOA.Moonshine.AutMonster.Dnat`, `VOA.Moonshine.AutMonster.sigmaEquiv`, `VOA.Moonshine.AutMonster.sigmaAut`, `VOA.Moonshine.AutMonster.sigmaAut_apply`

```
noncomputable abbrev Dnat : SigmaData :=
  mkData cStar0 1 (c_ne_of_norm cStar0_norm) (by norm_num)

noncomputable def sigmaEquiv : Vnat ≃ₗ[ℂ] Vnat :=
  LinearEquiv.ofInvolutive (sigNat Dnat) sigNat_invol

noncomputable def sigmaAut : AutVnat where
  toLinearEquiv := sigmaEquiv

@[simp] theorem sigmaAut_apply (v : Vnat) : sigmaAut v = sigNat Dnat v := rfl
```

Files `VOA/Moonshine/AutVnat.lean`, `VOA/Moonshine/AutSigma.lean`. **Hypotheses / status.** None; definition; used by std. theorems

Lemma 47.18 (Ising vectors determine the untwisted block). *(a) If $g \in \mathrm{Aut}(V^\natural)$ fixes, for every minimal α, both $e_{\alpha,s}$ and $e_{\alpha,-s}$ for some normalised s, then g fixes every $(q_{ab}, 0)$ and every $(x_\alpha, 0)$, $\alpha \in \Lambda_4$. (b) If g fixes every $(q_{ab}, 0)$ and every $(x_\alpha, 0)$, $\alpha \in \Lambda_4$, then g fixes $V_\Lambda^+ \times 0$. (c) If g fixes $V_\Lambda^+ \times 0$ and the twisted generator $(0, x)$, then $g = 1$.*

Lean. `VOA.Moonshine.ConwayNorton.M8.aut_fix_quad_minimal_of_fix_ising`, `VOA.Moonshine.ConwayNorton.M8.aut_fix_untwisted_of_fix_quad_minimal`, `VOA.Moonshine.ConwayNorton.M8.aut_eq_one_of_fix_untwisted_twistedGen`, `VOA.Moonshine.AutMonster.xGen`

```
theorem aut_fix_quad_minimal_of_fix_ising (g : AutVnat)
    (hg : ∀ α : leech.carrier, leech.form α α = 4 →
      ∃ s : ℂ, s ^ 2 * latticeCocycle leech α (-α) = 1 ∧
        g (isingVec α s) = isingVec α s ∧ g (isingVec α (-s)) = isingVec α (-s)) :
    (∀ a b : Fin (rank leech), g ((qP a b, 0) : Vnat) = ((qP a b, 0) : Vnat)) ∧
      (∀ α : leech.carrier, leech.form α α = 4 →
        g ((frameX0P α, 0) : Vnat) = ((frameX0P α, 0) : Vnat)) := by

theorem aut_fix_untwisted_of_fix_quad_minimal (g : AutVnat)
    (hq : ∀ a b : Fin (rank leech), g ((qP a b, 0) : Vnat) = ((qP a b, 0) : Vnat))
```

```
    (hx : ∀ α : leech.carrier, leech.form α α = 4 →
      g ((frameX0P α, 0) : Vnat) = ((frameX0P α, 0) : Vnat)) :
    ∀ x : VPlus, g ((x, 0) : Vnat) = ((x, 0) : Vnat) := by

theorem aut_eq_one_of_fix_untwisted_twistedGen (g : AutVnat)
    (hU : ∀ x : VPlus, g ((x, 0) : Vnat) = ((x, 0) : Vnat))
    (hx : g ((0, xGen) : Vnat) = ((0, xGen) : Vnat)) : g = 1 := by
```

```
noncomputable abbrev xGen : TP := x0T (tF Dnat.s)
```

Files `VOA/Moonshine/MoonshineLRFinOrderRigid.lean`, `VOA/Moonshine/AutSigma.lean`.

Theorem 47.19 (Every automorphism of $V^\natural$ has finite order). *For every $g \in \mathrm{Aut}(V^\natural)$ there is $n \geq 1$ with $g^n = 1$.*

Lean. `VOA.Moonshine.ConwayNorton.M8.aut_isingVec_periodic`, `VOA.Moonshine.ConwayNorton.M8.aut_pow_fix_untwisted`, `VOA.Moonshine.ConwayNorton.M8.autVnat_isOfFinOrder`

```
theorem aut_isingVec_periodic (g : AutVnat) {α : leech.carrier}
    (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 ∗ latticeCocycle leech α (-α) = 1) :
    ∃ K : ℕ, 1 ≤ K ∧ (g ^ K) (isingVec α s) = isingVec α s := by

theorem aut_pow_fix_untwisted (g : AutVnat) :
    ∃ K : ℕ, 1 ≤ K ∧ ∀ x : VPlus, (g ^ K) ((x, 0) : Vnat) = ((x, 0) : Vnat) := by

theorem autVnat_isOfFinOrder : ∀ g : AutVnat, IsOfFinOrder g := by
```

File `VOA/Moonshine/MoonshineLRFinOrderRigid.lean`.

Theorem 47.22 ($\mathrm{Aut}(V^\natural)$ is finite). *(a) An automorphism of $V^\natural$ that fixes every rigid idempotent of ${V^\natural}_2$ is the identity. (b) If $R(V^\natural)$ is finite, $\mathrm{Aut}(V^\natural)$ is finite. (c) $\mathrm{Aut}(V^\natural)$ is finite.*

Lean. `VOA.Moonshine.ConwayNorton.M8.eq_one_of_fix_rigid`, `VOA.Moonshine.ConwayNorton.M8.finite_autVnat_of_finite_rigid`, `VOA.Moonshine.ConwayNorton.M8.finite_autVnat`

```
theorem eq_one_of_fix_rigid (k : AutVnat)
    (hk : ∀ e ∈ vnatRigidSet, GriessAut.autRestrict k 2 e = e) : k = 1 := by

theorem finite_autVnat_of_finite_rigid (hR : vnatRigidSet.Finite) : Finite AutVnat := by

theorem finite_autVnat : Finite AutVnat := by
```

File `VOA/Moonshine/MoonshineLRFiniteAut.lean`.

Theorem 47.25 (The Monster Lie algebra is generated by its simple root spaces). *$\mathfrak{m}$ is generated by $\mathfrak{m}_0$ and the root spaces $\mathfrak{m}_{\pm(1,-1)}$, $\mathfrak{m}_{\pm(1,j)}$, $j \geq 1$.*

Lean. `VOA.Moonshine.ConwayNorton.M8.monsterLiePreData`, `VOA.Moonshine.ConwayNorton.M8.finrank_monsterLieBorcherdsWeight_cone`, `VOA.Moonshine.ConwayNorton.M8.monsterLiePreData_KNZ`, `VOA.Moonshine.ConwayNorton.M8.monsterLieSimpleRootGeneration`

```
def monsterLiePreData : BorcherdsPreData monsterLieAlgebra where
  weightSpace := monsterLieBorcherdsWeight
  weight_internal := monsterLieBorcherdsWeight_isInternal
  weight_finite := monsterLieBorcherdsWeight_finite

theorem finrank_monsterLieBorcherdsWeight_cone (m n : ℕ) (hm : 0 < m) (_hn : 0 < n) :
    (finrank ℂ (monsterLieBorcherdsWeight ((m : ℤ), (n : ℤ))) : ℂ) = cJ (m ∗ n) := by
```

```
theorem monsterLiePreData_KNZ : monsterLiePreData.KNZ := by

theorem monsterLieSimpleRootGeneration : MonsterLieSimpleRootGeneration :=
  monsterLiePreData.lieSpan_eq_top_of_KNZ monsterLiePreData_KNZ
```

File `VOA/Moonshine/MoonshineLRGenerationMonster.lean`.

Theorem 47.27 (Conway–Norton for $\mathrm{Aut}(V^\natural)$; Borcherds). *For every automorphism g of $V^\natural$ there are a subgroup $\Gamma_g \le \mathrm{SL}_2(\mathbb{R})$ and $N \ge 1$ such that $\Gamma_0(N) \le \Gamma_g$, Γ_g is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and $T_g = q^{-1} + \sum_{n\ge 1} \mathrm{tr}(g|V^\natural_{n+1})q^n$ is the normalised Hauptmodul of Γ_g. (This is the qualitative genus-zero form: Γ_g is not identified with Conway and Norton's group and N is not related to the order of g; it is weaker than the table-level conjecture.)*

Lean. `VOA.Moonshine.ConwayNorton.M8.conwayNorton_of_isOfFinOrder`, `VOA.Moonshine.ConwayNorton.M8.conwayNortonConjecture_holds`, `VOA.Moonshine.ConwayNorton.conwayNorton`

```
theorem conwayNorton_of_isOfFinOrder (hfin : ∀ g : AutVnat, IsOfFinOrder g) :
    ConwayNortonConjecture :=
  conwayNorton_of_simpleRootGeneration monsterLieSimpleRootGeneration hfin

theorem conwayNortonConjecture_holds : ConwayNortonConjecture :=
  conwayNorton_of_isOfFinOrder autVnat_isOfFinOrder

theorem conwayNorton : ConwayNortonConjecture :=
  M8.conwayNortonConjecture_holds
```

Files `VOA/Moonshine/MoonshineLRGenerationMonster.lean`, `VOA/Moonshine/MoonshineLRConwayNortonFinal.lean`, `VOA/Moonshine/MoonshineCNFinalAudit.lean`. **Hypotheses / status.** `hfin`; std.

Chapter 48

The Griess algebra of the moonshine module and AxiomMath's Monster

10 of the chapter's 59 numbered statements: its headline results and every definition they use.

Definition 48.17 (The space $\mathcal{B}$ and the form b). Sym is the space of symmetric 24×24 rational matrices, $X = \mathbb{Q}^{\mathcal{S}}$ with standard basis $(e_c)_{c\in\mathcal{S}}$, W the space of $I \times 24$ rational matrices, and $\mathcal{B} = \mathrm{Sym} \times X \times W$. For $u = (A, x, N)$, $v = (A', x', N')$,

$$b(u, v) = \operatorname{tr}(AA') + \sum_{c\in\mathcal{S}} x_c x'_c + \sum_{j\in I} \sum_{i<24} N_{ji} N'_{ji}.$$

Lean. `VOA.Moonshine.MonsterGriess.symMat`, `VOA.Moonshine.MonsterGriess.SpaceX`, `VOA.Moonshine.MonsterGriess.SpaceW`, `VOA.Moonshine.MonsterGriess.SpaceB`, `VOA.Moonshine.MonsterGriess.bform`, `VOA.Moonshine.MonsterGriess.finrank_B`

```
def symMat : Submodule ℚ (Matrix (Fin 24) (Fin 24) ℚ) where
  carrier := {A | Aᵀ = A}
```

```
abbrev SpaceX : Type := ShortIdx → ℚ

abbrev SpaceW : Type := Matrix SpinIdx (Fin 24) ℚ

abbrev SpaceB : Type := symMat × SpaceX × SpaceW
```

```
noncomputable def bform : LinearMap.BilinForm ℚ SpaceB :=
  LinearMap.mk₂ ℚ (fun u v => trace ((u.1 : Matrix (Fin 24) (Fin 24) ℚ) *
      (v.1 : Matrix (Fin 24) (Fin 24) ℚ)) +
      ∑ c, u.2.1 c * v.2.1 c + ∑ j, ∑ i, u.2.2 j i * v.2.2 j i)
```

```
theorem finrank_B : finrank ℚ SpaceB = 196884 := by
```

Files `VOA/Moonshine/MonsterGriess/Algebra.lean`, `VOA/Moonshine/MonsterGriess/TrialityBasis.lean`. **Hypotheses / status.** — (definitions)

Definition 48.18 (The Griess product of $\mathcal{B}$). The product is the bilinear map determined by the following table (and commutativity), for $A, A' \in \mathrm{Sym}$, $x, y \in X$, $N, M \in W$ ("$\cdot$" is the product, each entry lists the (Sym, X, W)-components):

product	Sym-part	X-part / W-part
$A \cdot A'$	$2(AA' + A'A)$	$0 \quad / \quad 0$
$A \cdot x$	0	$c \mapsto \frac{1}{8} x_c q_c(A) \quad / \quad 0$
$x \cdot y$	$\frac{1}{8} \sum_c x_c y_c U_c$	$c \mapsto \sum_{d,e} x_d y_e \tau(d,e,c) \quad / \quad 0$
$A \cdot N$	0	$0 \quad / \quad \frac{\operatorname{tr} A}{8} N + NA$
$x \cdot N$	0	$0 \quad / \quad \frac{1}{8} \sum_c x_c (R_c N - \frac{1}{4} R_c N U_c)$
$N \cdot M$	$\frac{1}{8} \operatorname{tr}(N^{\mathsf{T}} M) 1 + \frac{1}{2}(N^{\mathsf{T}} M + M^{\mathsf{T}} N)$	$c \mapsto \frac{1}{8}(\operatorname{tr}(N^{\mathsf{T}} R_c M) - \frac{1}{4} q_c(N^{\mathsf{T}} R_c M)) \quad / \quad 0$

Lean. `VOA.Moonshine.MonsterGriess.Umat`, `VOA.Moonshine.MonsterGriess.Rmat`, `VOA.Moonshine.MonsterGriess.qf`, `VOA.Moonshine.MonsterGriess.tau`, `VOA.Moonshine.MonsterGriess.gmulA`, `VOA.Moonshine.MonsterGriess.gmulX`, `VOA.Moonshine.MonsterGriess.gmulWHalf`, `VOA.Moonshine.MonsterGriess.gmulW`, `VOA.Moonshine.MonsterGriess.gmulFun`, `VOA.Moonshine.MonsterGriess.gmul`, `VOA.Moonshine.MonsterGriess.gmul_spinBlock_spinBlock_snd_fst`

```
noncomputable def Umat (c : ShortIdx) : Matrix (Fin 24) (Fin 24) ℚ := vecMulVec (uQ c) (uQ
↪  c)

noncomputable def Rmat (c : ShortIdx) : Matrix SpinIdx SpinIdx ℚ := qmatR ℚ 0 c.1.1 c.1.2

noncomputable def qf (c : ShortIdx) (M : Matrix (Fin 24) (Fin 24) ℚ) : ℚ :=
  uQ c ⬝ᵥ (M *ᵥ uQ c)

noncomputable def tau (d e c : ShortIdx) : ℚ :=
  open Classical in if c.1 = d.1 + e.1 then sgn (pair e.1.1 d.1.2) else 0
```

```
noncomputable def gmulA (u v : SpaceB) : Matrix (Fin 24) (Fin 24) ℚ :=
  (2 : ℚ) • (Am u * Am v + Am v * Am u) + (8 : ℚ)⁻¹ • ∑ c, (xs u c * xs v c) • Umat c +
    (trace ((Nm u)ᵀ * Nm v) / 8) • (1 : Matrix (Fin 24) (Fin 24) ℚ) +
    (2 : ℚ)⁻¹ • ((Nm u)ᵀ * Nm v + (Nm v)ᵀ * Nm u)
```

```
noncomputable def gmulX (u v : SpaceB) (c : ShortIdx) : ℚ :=
  (xs v c * qf c (Am u) + xs u c * qf c (Am v)) / 8 +
    ∑ d, ∑ e, xs u d * xs v e * tau d e c +
    (trace ((Nm u)ᵀ * Rmat c * Nm v) - qf c ((Nm u)ᵀ * Rmat c * Nm v) / 4) / 8

noncomputable def gmulWHalf (u v : SpaceB) : SpaceW :=
  (trace (Am u) / 8) • Nm v + Nm v * Am u +
    (8 : ℚ)⁻¹ • ∑ c, xs u c • (Rmat c * Nm v - (4 : ℚ)⁻¹ • (Rmat c * Nm v * Umat c))

noncomputable def gmulW (u v : SpaceB) : SpaceW := gmulWHalf u v + gmulWHalf v u

noncomputable def gmulFun (u v : SpaceB) : SpaceB :=
  (⟨gmulA u v, gmulA_symm u v⟩, gmulX u v, gmulW u v)
```

```
noncomputable def gmul : SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] SpaceB :=
  LinearMap.mk₂ ℚ gmulFun gmulFun_add_left gmulFun_smul_left gmulFun_add_right
    gmulFun_smul_right
```

```
theorem gmul_spinBlock_spinBlock_snd_fst (N M : SpaceW) :
    (gmul (spinBlock N) (spinBlock M)).2.1
      = fun c => (trace (Nᵀ * Rmat c * M) - qf c (Nᵀ * Rmat c * M) / 4) / 8 := by
```

Files `VOA/Moonshine/MonsterGriess/Algebra.lean`, `VOA/Moonshine/MoonshineCNGriessTable.lean`. **Hypotheses / status.** — (definitions)

Definition 48.23 (Algebra automorphisms; `AutO`). A $\mathbb{Q}$-linear automorphism f of $\mathcal{B}$ is an *algebra automorphism* if $f(u \cdot v) = f(u) \cdot f(v)$ for all u, v. `AutO` is the subgroup of $GL_{\mathbb{Q}}(\mathcal{B})$ of algebra automorphisms preserving b.

Lean. `VOA.Moonshine.MonsterGriess.IsAlgAut`, `VOA.Moonshine.MonsterGriess.AutO`, `VOA.Moonshine.MonsterGriess.mem_AutO`

```
def IsAlgAut (f : SpaceB ≃ₗ[ℚ] SpaceB) : Prop := ∀ u v, f (gmul u v) = gmul (f u) (f v)
```

```
def AutO : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) where
  carrier := {g | IsAlgAut g ∧ ∀ u v, bform (g u) (g v) = bform u v}
```

```
theorem mem_AutO {g : SpaceB ≃ₗ[ℚ] SpaceB} :
    g ∈ AutO ↔ (∀ u v, g (gmul u v) = gmul (g u) (g v)) ∧ ∀ u v, bform (g u) (g v) = bform
     ↪ u v :=
  Iff.rfl
```

File `VOA/Moonshine/MonsterGriess/AutO.lean`. **Hypotheses / status.** — (definition)

Theorem 48.33 (τ_B is an automorphism; $G_{\mathrm{mat}} \le$ `AutO`). *$\tau_B(u \cdot v) = \tau_B(u) \cdot \tau_B(v)$ for all u, v; hence $\tau_B \in$ `AutO` and $G_{\mathrm{mat}} \le$ `AutO`.*

Lean. `VOA.Moonshine.MonsterGriess.tauB_gmul`, `VOA.Moonshine.MonsterGriess.tauB_mem_AutO`, `VOA.Moonshine.MonsterGriess.Generated.Gmat_le_AutO`

```
theorem tauB_gmul (u v : SpaceB) : tauB (gmul u v) = gmul (tauB u) (tauB v) := by
```

```
theorem tauB_mem_AutO : tauB ∈ AutO := ⟨isAlgAut_tauB, bform_tauB⟩
```

```
theorem Gmat_le_AutO : Gmat ≤ AutO := Gmat_le_AutO_of_isAlgAut isAlgAut_tauB
```

File `VOA/Moonshine/MonsterGriess/ProductInvariance.lean`. **Hypotheses / status.** None

Theorem 48.34 (AxiomMath's rational parity-centralizer theorem). *(1) A $\mathbb{Q}$-linear automorphism f of $\mathcal{B}$ preserves the product and the form and commutes with x_B if and only if $f \in G_{\mathrm{vis}}$. (2) Every such f is θ_p for a visible pair p, unique up to $p \mapsto p \cdot (-1,-1)$. (3) $p \cdot (-1,-1) = (-s,-g)$. Equivalently,* $\mathtt{AutO} \cap C(x_B) = G_{\mathrm{vis}}$.

Lean. `VOA.Moonshine.MonsterGriess.Parity.rational_parity_centralizer_equals_visible`, `VOA.Moonshine.MonsterGriess.Parity.AutO_inf_centralizer_xB_eq_Gvis`

```
theorem rational_parity_centralizer_equals_visible :
    (∀ f : SpaceB ≃ₗ[ℚ] SpaceB, ((∀ u v, f (gmul u v) = gmul (f u) (f v)) ∧
        (∀ u v, bform (f u) (f v) = bform u v) ∧ ∀ u, f (xB u) = xB (f u)) ↔ f ∈ Gvis) ∧
    (∀ f : SpaceB ≃ₗ[ℚ] SpaceB, (∀ u v, f (gmul u v) = gmul (f u) (f v)) →
      (∀ u v, bform (f u) (f v) = bform u v) → (∀ u, f (xB u) = xB (f u)) →
      ∃ p : Pvis, thetaEquiv p = f ∧
        ∀ p' : Pvis, thetaEquiv p' = f ↔ p' = p ∨ p' = p * negPair) ∧
    ∀ p : Pvis, (((p * negPair : Pvis).1.1 : GL SpinIdx ℚ) : Matrix SpinIdx SpinIdx ℚ) =
        -((p.1.1 : GL SpinIdx ℚ) : Matrix SpinIdx SpinIdx ℚ) ∧
      mat (p * negPair : Pvis).1.2 = -mat p.1.2 := by
```

```
theorem AutO_inf_centralizer_xB_eq_Gvis :
    AutO ⊓ Subgroup.centralizer {xB} = Gvis := by
```

File `VOA/Moonshine/MonsterGriess/ParityEqualsVisible.lean`. **Hypotheses / status.** None

Theorem 48.36 (The untwisted Griess products in Conway coordinates). *For symmetric A, B and $\beta \in \Lambda$ with integer coordinates y_β: (a) $Q_C(A)_{(1)}Q_C(B) = 2\,Q_C(AB + BA)$; (b) $Q_C(A)_{(1)}x_\beta = \frac{1}{8}(y_\beta^{\mathsf T}Ay_\beta)\,x_\beta$, where $x_\beta = e^{\beta} + e^{-\beta}$; (c) $\beta(-1)^2\mathbb{1} = Q_C(\frac{1}{8}y_\beta y_\beta^{\mathsf T})$; (d) $\langle Q_C(A), Q_C(B)\rangle = 2\,\mathrm{tr}(AB)$; $\langle x_\alpha, x_\beta\rangle = 2\varepsilon(\alpha,\alpha)$ if $\beta = \pm\alpha$ and 0 otherwise (α minimal).*

Lean. `VOA.Moonshine.GriessDictionary.conwayQuad_one_conwayQuad_of_symm`, `VOA.Moonshine.GriessDictionary.conwayQuad_one_frameX0P`, `VOA.Moonshine.GriessDictionary.dV_self_eq_conwayQuad`, `VOA.Moonshine.GriessDictionary.plusForm_conwayQuad_conwayQuad_of_symm`, `VOA.Moonshine.GriessDictionary.plusForm_frameX0P_frameX0P`

```
theorem conwayQuad_one_conwayQuad_of_symm {A B : Matrix (Fin 24) (Fin 24) ℂ} (hA : Aᵀ = A)
    (hB : Bᵀ = B) :
    leechVOAPlus.smul (conwayQuad A) 1 (conwayQuad B) = (2 : ℂ) • conwayQuad (A * B + B *
     ↪ A) := by
```

```
theorem conwayQuad_one_frameX0P (A : Matrix (Fin 24) (Fin 24) ℂ) (β : leech.carrier) :
    leechVOAPlus.smul (conwayQuad A) 1 (frameX0P β)
      = ((8 : ℂ)⁻¹ * ((fun i => (vec β i : ℂ)) ·ᵥ (A *ᵥ fun i => (vec β i : ℂ)))) •
       ↪ frameX0P β := by
```

```
theorem dV_self_eq_conwayQuad (β : leech.carrier) :
    (dV β β : VPlus)
      = conwayQuad ((8 : ℂ)⁻¹ • vecMulVec (fun i => (vec β i : ℂ)) (fun i => (vec β i :
       ↪ ℂ))) := by
```

```
theorem plusForm_conwayQuad_conwayQuad_of_symm {A B : Matrix (Fin 24) (Fin 24) ℂ}
    (hB : Bᵀ = B) : plusForm (conwayQuad A) (conwayQuad B) = 2 * trace (A * B) := by
```

```
theorem plusForm_frameX0P_frameX0P {α : leech.carrier} (hα : leech.form α α = 4)
    (β : leech.carrier) :
    plusForm (frameX0P α) (frameX0P β)
      = if β = α ∨ β = -α then 2 * latticeCocycle leech α α else 0 := by
```

File `VOA/Moonshine/MoonshineCNGriessConway.lean`. **Hypotheses / status.** Displayed

Theorem 48.39 (The untwisted dictionary is an isometric algebra map). *If ζ is a sign dictionary, then for all $u, v \in \mathcal{B}$ with zero W-part, $\Phi_\zeta(u \cdot v) = \Phi_\zeta(u)_{(1)}\Phi_\zeta(v)$ and $\langle \Phi_\zeta u, \Phi_\zeta v\rangle = 2\,b(u,v)$.*

Lean. `VOA.Moonshine.GriessDictionary.untwistedDict_gmul`, `VOA.Moonshine.GriessDictionary.untwistedDict_form`

```
theorem untwistedDict_gmul (hζ : ShortSignDictionary ζ) {u v : SpaceB} (hu : u.2.2 = 0)
    (hv : v.2.2 = 0) :
    untwistedDict ζ (gmul u v) = leechVOAPlus.smul (untwistedDict ζ u) 1 (untwistedDict ζ
     ↪ v) := by
```

```
theorem untwistedDict_form (hζ : ShortSignDictionary ζ) {u v : SpaceB} (hu : u.2.2 = 0)
    (hv : v.2.2 = 0) :
    plusForm (untwistedDict ζ u) (untwistedDict ζ v) = 2 * (bform u v : ℂ) := by
```

File `VOA/Moonshine/MoonshineCNGriessUntwisted.lean`. **Hypotheses / status.** Sign dictionary, $W = 0$

Theorem 48.43 (The untwisted dictionary, unconditionally). `griessShortSign` *is a sign dictionary; hence for all u, v with zero W-part the dictionary Φ_ζ (into $V^\natural$) is multiplicative and $\langle \Phi_\zeta u, \Phi_\zeta v\rangle = 2b(u,v)$, and its $\mathbb{C}$-linear extension is an isomorphism $\mathbb{C} \otimes_{\mathbb{Q}} (\mathrm{Sym} \oplus X) \cong (V_\Lambda^+)_2$.*

Lean. `VOA.Moonshine.GriessDictionary.shortSignDictionary_griessShortSign`, `VOA.Moonshine.GriessDictionary.untwistedGriessNat_gmul`, `VOA.Moonshine.GriessDictionary.untwistedGriessNat_form`, `VOA.Moonshine.GriessDictionary.untwistedIsoC`, `VOA.Moonshine.GriessDictionary.untwistedIsoC_tmul`

```
theorem shortSignDictionary_griessShortSign : ShortSignDictionary griessShortSign :=
```

```
theorem untwistedGriessNat_gmul {u v : SpaceB} (hu : u.2.2 = 0) (hv : v.2.2 = 0) :
    untwistedDictNat griessShortSign (gmul u v)
      = VnatVOA.griessMul (untwistedDictNat griessShortSign u)
          (untwistedDictNat griessShortSign v) :=
```

```
theorem untwistedGriessNat_form {u v : SpaceB} (hu : u.2.2 = 0) (hv : v.2.2 = 0) :
    SimpleNat.natForm (untwistedDictNat griessShortSign u : Vnat)
        (untwistedDictNat griessShortSign v : Vnat) = 2 * (bform u v : ℂ) :=
```

```
noncomputable def untwistedIsoC : ℂ ⊗[ℚ] untwistedSub ≃ₗ[ℂ] leechVOAPlus.grading 2 :=
  (Algebra.TensorProduct.basis ℂ untwistedBasisQ).equiv untwistedBasisC (Equiv.refl _)
```

```
theorem untwistedIsoC_tmul (u : untwistedSub) :
    (untwistedIsoC (1 ⊗ₜ u) : VPlus) = untwistedDict griessShortSign (u : SpaceB) := by
```

Files `VOA/Moonshine/MoonshineCNGriessSign.lean`, `VOA/Moonshine/MoonshineCNGriessUntwistedBasis.lean`.
Hypotheses / status. None

Theorem 48.51 (The W-block: form, module channels, and AxiomMath's invariance). *(a)* $\langle \mathrm{spinDict}N, \mathrm{spinDict}M\rangle = 2\sum N_{ji}M_{ji}$. *(b)* $\mathrm{matDict}(A)_{(1)}\mathrm{spinDict}(N) = \mathrm{spinDict}(\frac{\mathrm{tr}A}{8}N + NA)$ *and* $\mathrm{shortDict}_{\zeta'}(x)_{(1)}\mathrm{spinDict}(N) = \mathrm{spinDict}(\frac{1}{8}\sum_c x_c(R_cN - \frac{1}{4}R_cNU_c))$. *(c) On AxiomMath's side, for u with zero W-part, $b(N\cdot M, u) = b(M, u\cdot N)$.*

Lean. `VOA.Moonshine.GriessDictionary.TPForm_spinDict`, `VOA.Moonshine.GriessDictionary.topAct_matDict_spinDict`, `VOA.Moonshine.GriessDictionary.topAct_shortDict_spinDict`, `VOA.Moonshine.GriessDictionary.bform_gmul_spinBlock_spinBlock`

```
theorem TPForm_spinDict (N M : SpaceW) :
    TPForm (spinDict N) (spinDict M) = 2 * ((∑ j, ∑ i, N j i * M j i : ℚ) : ℂ) := by
```

```
theorem topAct_matDict_spinDict (A : symMat) (N : SpaceW) :
    topAct (matDict A) (spinDict N)
      = spinDict ((trace (A : Matrix (Fin 24) (Fin 24) ℚ) / 8) • N + N * (A : Matrix (Fin
       ↪  24) (Fin 24) ℚ)) := by
```

```
theorem topAct_shortDict_spinDict (x : SpaceX) (N : SpaceW) :
    topAct (shortDict fullShortSign x) (spinDict N)
      = spinDict ((8 : ℚ)⁻¹ • ∑ c, x c • (Rmat c * N - (4 : ℚ)⁻¹ • (Rmat c * N * Umat c)))
       ↪  := by
```

```
theorem bform_gmul_spinBlock_spinBlock (N M : SpaceW) {u : SpaceB} (hu : u.2.2 = 0) :
    bform (gmul (spinBlock N) (spinBlock M)) u
      = bform (spinBlock M) (spinBlock (spinBlockAction u N)) := by
```

File `VOA/Moonshine/MoonshineCNGriessSpinDict.lean`. **Hypotheses / status.** None

Theorem 48.54 (`griessIso`: the Griess algebra of $V^\natural$ is AxiomMath's, over $\mathbb{C}$). *There is a $\mathbb{C}$-linear isomorphism $\Theta : \mathbb{C}\otimes_{\mathbb{Q}}\mathcal{B} \to {V^\natural}_2$ with $\Theta(1\otimes u) = \mathrm{griessNat}(u)$, such that Θ carries the $\mathbb{C}$-bilinear extension of the product of $\mathcal{B}$ to the Griess product of $V^\natural$ and the $\mathbb{C}$-bilinear extension of $2b$ to the invariant form of $V^\natural$.*

Lean. `VOA.Moonshine.GriessDictionary.griessIso`, `VOA.Moonshine.GriessDictionary.griessIso_tmul`, `VOA.Moonshine.GriessDictionary.griessIso_mul`, `VOA.Moonshine.GriessDictionary.griessIso_form`

```
noncomputable def griessIso : ℂ ⊗[ℚ] SpaceB ≃ₗ[ℂ] VnatVOA.grading 2 :=
  (Algebra.TensorProduct.basis ℂ griessBasisQ).equiv griessBasisC (Equiv.refl _)
```

```
theorem griessIso_tmul (u : SpaceB) : griessIso (1 ⊗ₜ u) = griessNat u := by
```

```
theorem griessIso_mul (x y : ℂ ⊗[ℚ] SpaceB) :
    griessIso (LinearMap.BilinMap.baseChange ℂ gmul x y)
      = VnatVOA.griessMul (griessIso x) (griessIso y) := by
```

```
theorem griessIso_form (x y : ℂ ⊗[ℚ] SpaceB) :
    SimpleNat.natForm (griessIso x : Vnat) (griessIso y : Vnat)
      = 2 * LinearMap.BilinForm.baseChange ℂ bform x y := by
```

File `VOA/Moonshine/MoonshineCNGriessIso.lean`. **Hypotheses / status. none**

Chapter 49

The Monster acts on the moonshine module: lifts, triality, the commutant and the converse

10 of the chapter's 34 numbered statements: its headline results and every definition they use.

Theorem 49.4 (Co_0 on T^+, covariantly). *For a surjective isometry ι of Λ, the map $\varphi_\iota = O_\iota \otimes \rho$ restricted to T^+ satisfies $\varphi_\iota(u_{(n)}t) = (\Phi_\iota u)_{(n)}\varphi_\iota(t)$ for all $u \in V_\Lambda^+$, n, t; and some nonzero multiple $\lambda_\iota\varphi_\iota$ preserves* `TPForm`.

Lean. `VOA.Moonshine.ConwayNorton.M8.liftTwPlusEquiv_mode`, `VOA.Moonshine.ConwayNorton.M8.liftTwExists_formScalar`

```
theorem liftTwPlusEquiv_mode (hsurj : Function.Surjective ι)
    (u : (orbifold (Subgroup.zpowers leechTheta)).carrier) (n : ℤ) (t : TP) :
    liftTwPlusEquiv ι hι (mode (plusModule.YM u) n t) =
      mode (plusModule.YM (leechPlusLatMorphAut ι hι hsurj u)) n (liftTwPlusEquiv ι hι t)
```

```
theorem liftTwExists_formScalar : ∃ c : ℂˣ, ∀ t t' : TP,
    TPForm (liftTwNormalizedPlusEquiv ι hι c t)
      (liftTwNormalizedPlusEquiv ι hι c t') = TPForm t t'
```

Files `VOA/Moonshine/MoonshineLRLiftTwisted2.lean`, `VOA/Moonshine/MoonshineLRLiftTwisted3.lean`. **Hypotheses / status.** Isometry, onto

Theorem 49.8 (Rational criterion for lattice lifts). *Let ι be a surjective isometry of Λ.*

(a) liftTwVnatAut(ι) *is the lift of an element of* AutO *if and only if its normalised twisted part maps rational spin-dictionary vectors to rational ones.*

(b) For every α the number $w(\iota,\alpha) = c(\alpha)Z(\iota\alpha)/Z(\alpha)$ is ± 1.

(c) If R is a rational 4096×4096 matrix with $R^{\mathsf{T}}R = 1$ and $RQ_\alpha = w(\iota,\alpha)\,Q_{\iota\alpha}R$ for all $\alpha \in \Lambda$ (equivalently, for the 24 basis vectors of Λ), then liftTwVnatAut(ι) *is the lift of an element of* AutO.

Lean. `VOA.Moonshine.ConwayNorton.M8.liftLiftableSpinRational`, `VOA.Moonshine.ConwayNorton.M8.liftLiftableIffSpinRational`, `VOA.Moonshine.ConwayNorton.M8.liftMonoPhase`, `VOA.Moonshine.ConwayNorton.M8.liftMonoPhase_sign`, `VOA.Moonshine.ConwayNorton.M8.liftMonoOfUnphasedIntertwiner`, `VOA.Moonshine.ConwayNorton.M8.liftGenOfBasisIntertwiner`

```
def liftLiftableSpinRational : Prop :=
  ∀ N : SpaceW, ∃ N' : SpaceW,
    liftTwNormalizedPlusEquiv ι hι (liftWScalar ι hι) (spinDict N) = spinDict N'
-- …
theorem liftLiftableIffSpinRational (hsurj : Function.Surjective ι) :
    (∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut ι hι hsurj) ↔
      liftLiftableSpinRational ι hι
```

```
def liftMonoPhase (α : leech.carrier) : ℂ :=
  latCob leech leech ι α * signRefinement (coordOfRed (red leech (ι α))) /
    signRefinement (coordOfRed (red leech α))
-- …
theorem liftMonoPhase_sign (α : leech.carrier) :
    liftMonoPhase ι α = 1 ∨ liftMonoPhase ι α = -1
-- …
theorem liftMonoOfUnphasedIntertwiner (hsurj : Function.Surjective ι)
    (R : Matrix SpinIdx SpinIdx ℚ) (horth : Rᵀ * R = 1)
    (hinter : ∀ α : leech.carrier,
      R * liftMonoQmat α = liftMonoWeight ι α • (liftMonoQmat (ι α) * R)) :
    ∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut ι hι hsurj
```

```
theorem liftGenOfBasisIntertwiner (ι : leech.carrier →ₗ[ℤ] leech.carrier)
    (hι : ∀ α β, leech.form (ι α) (ι β) = leech.form α β)
    (hsurj : Function.Surjective ι) (R : Matrix SpinIdx SpinIdx ℚ)
    (horth : Rᵀ * R = 1)
    (hB : ∀ j, R * liftMonoQmat (latticeBasis leech j) =
      liftMonoWeight ι (latticeBasis leech j) •
        (liftMonoQmat (ι (latticeBasis leech j)) * R)) :
    ∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut ι hι hsurj
```

Files `VOA/Moonshine/MoonshineLRLiftLiftable.lean`, `VOA/Moonshine/MoonshineLRLiftLiftable2.lean`, `VOA/Moonshine/MoonshineLRLiftLiftable3.lean`. **Hypotheses / status.** Rational orthogonal intertwiner

Theorem 49.13 (Every visible element is liftable). *Every element of G_{vis}, regarded as an element of* AutO*, is liftable. Consequently Conway–Norton holds for every element of* AutO *as soon as some liftable τ together with G_{vis} generates* AutO*.*

Lean. `VOA.Moonshine.ConwayNorton.M8.liftVisRAllVisible`, `VOA.Moonshine.ConwayNorton.M8.visibleSetAutO`, `VOA.Moonshine.ConwayNorton.M8.isVnatLiftable_of_mem_visibleSetAutO`, `VOA.Moonshine.ConwayNorton.M8.conwayNorton_autO_of_tauB`

```
theorem liftVisRAllVisible : ∀ (g : SpaceB ≃ₗ[ℚ] SpaceB) (hg : g ∈ Gvis),
    IsVnatLiftable ⟨g, Gvis_le_AutO hg⟩
```

```
def visibleSetAutO : Set AutO := {g | (g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Gvis}
-- …
theorem isVnatLiftable_of_mem_visibleSetAutO : ∀ g ∈ visibleSetAutO, IsVnatLiftable g
-- …
theorem conwayNorton_autO_of_tauB {τ : AutO} (hτ : IsVnatLiftable τ)
    (hGmat : Subgroup.closure (insert τ visibleSetAutO) = ⊤) (g : AutO) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h)
```

Files `VOA/Moonshine/MoonshineLRLiftVisible2.lean`, `VOA/Moonshine/MoonshineLRMonsterVisible.lean`. **Hypotheses / status.** None

Theorem 49.17 (τ_B is liftable). *τ_B, as an element of* AutO*, is liftable. More precisely: (a) $\sigma_B x_B \sigma_B^{-1} \in \{y_B, z_B\}$; (b) if $g \in$ AutO satisfies $g x_B g^{-1} = \sigma_B x_B \sigma_B^{-1}$, then g is liftable; (c) hence τ_B is liftable.*

Lean. VOA.Moonshine.ConwayNorton.M8.tauBLiftConjugate, VOA.Moonshine.ConwayNorton.M8.tauBLiftConjugate_dict, VOA.Moonshine.ConwayNorton.M8.tauBLiftSigma_conj_xB, VOA.Moonshine.ConwayNorton.M8.tauBLift_of_conjugate, VOA.Moonshine.ConwayNorton.M8.tauBLift_isVnatLiftable_tauB_of_mem_AutO, VOA.Moonshine.ConwayNorton.M8.isVnatLiftable_tauB

```
def tauBLiftConjugate : AutO :=
  (tauBLiftSigma : AutO) * ⟨xB, xB_mem_AutO⟩ * (tauBLiftSigma : AutO)⁻¹
-- …
theorem tauBLiftConjugate_dict (u : SpaceB) :
    (griessNat ((tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB) u) : Vnat) =
      sigmaBlockFrameSign Dnat.s (griessNat u : Vnat)
-- …
theorem tauBLiftSigma_conj_xB (hτ : tauB ∈ AutO) :
    (tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB) = yB v
      (tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB) = zB
-- …
theorem tauBLift_of_conjugate (g : AutO)
    (hg : (g : SpaceB ≃ₗ[ℚ] SpaceB) * xB * (g : SpaceB ≃ₗ[ℚ] SpaceB)⁻¹ =
      (tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB)) : IsVnatLiftable g
-- …
theorem tauBLift_isVnatLiftable_tauB_of_mem_AutO (hτ : tauB ∈ AutO) :
    IsVnatLiftable ⟨tauB, hτ⟩
```

```
theorem isVnatLiftable_tauB : IsVnatLiftable ⟨tauB, tauB_mem_AutO⟩ :=
  tauBLift_isVnatLiftable_tauB_of_mem_AutO tauB_mem_AutO
```

Files VOA/Moonshine/MoonshineLRTauB.lean, VOA/Moonshine/MoonshineLRMonsterFinal.lean. **Hypotheses / status.** hτ (a theorem)

Theorem 49.18 (Every element of G_{mat} acts on $V^\natural$; Conway–Norton for G_{mat}). *(a) G_{mat} is the image in $\mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ of the subgroup of AutO generated by τ_B and the visible elements. (b) Every $g \in \mathrm{AutO}$ with $g \in G_{\mathrm{mat}}$ is liftable. (c) For every $g \in \mathrm{AutO}$ with $g \in G_{\mathrm{mat}}$ there is an automorphism h of $V^\natural$ acting on $V^\natural{}_2 \cong \mathbb{C} \otimes \mathcal{B}$ as $1 \otimes g$, whose McKay–Thompson function T_h is the normalised Hauptmodul of a genus-zero group $\Gamma \supseteq \Gamma_0(N)$ commensurable with $\mathrm{SL}_2(\mathbb{Z})$.*

Lean. VOA.Moonshine.ConwayNorton.tauBAutO, VOA.Moonshine.ConwayNorton.gmat_eq_map_closure_insert_tauB, VOA.Moonshine.ConwayNorton.isVnatLiftable_of_mem_gmat, VOA.Moonshine.ConwayNorton.conwayNorton_gmat

```
noncomputable def tauBAutO : AutO := ⟨tauB, tauB_mem_AutO⟩
-- …
theorem gmat_eq_map_closure_insert_tauB :
    Generated.Gmat = (Subgroup.closure (insert tauBAutO M8.visibleSetAutO)).map
     ↪ AutO.subtype
-- …
theorem isVnatLiftable_of_mem_gmat (g : AutO)
    (hg : (g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Generated.Gmat) : IsVnatLiftable g
-- …
theorem conwayNorton_gmat (g : AutO) (hg : (g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Generated.Gmat) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h)
```

```
example (g : SpaceB ≃ₗ[ℚ] SpaceB)
    (hA : (∀ u v, g (gmul u v) = gmul (g u) (g v)) ∧ ∀ u v, bform (g u) (g v) = bform u v)
    (hg : g ∈ Subgroup.closure ((Visible.Gvis : Set (SpaceB ≃ₗ[ℚ] SpaceB)) ∪ {tauB})) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h
```

```
      = (griessIso.symm ≪≫ₗ LinearEquiv.baseChange ℚ ℂ SpaceB SpaceB g ≪≫ₗ griessIso) ∧
    ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
      IsNormalizedHauptmodul Γ (mckayThompsonFun h) :=
```

File `VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean`. **Hypotheses / status.** None beyond membership $g \in G_{\mathrm{mat}}$

Theorem 49.27 (The identity line is the conformal line, and is fixed). $\mathrm{griessNat}(\mathbf{e}_1) = 2\omega$; *hence every automorphism of* $V^\natural$ *fixes* $\mathrm{griessNat}(\mathbf{e}_1)$*, every element of* G_{mat} *fixes* $\mathbf{e}_1$*, and* P_0 *commutes with* G_{mat}*. Moreover* G_{mat} *is finite.*

Lean. `VOA.Moonshine.MonsterAction.griessNat_griessIdentityVec`, `VOA.Moonshine.MonsterAction.autVnat_apply_griessNat_identityVec`, `VOA.Moonshine.MonsterAction.finite_gmat`

```
theorem griessNat_griessIdentityVec :
    griessNat griessIdentityVec = (2 : ℂ) • vnatConformalTwo
-- …
theorem autVnat_apply_griessNat_identityVec (h : AutVnat) :
    h (griessNat griessIdentityVec : Vnat) = griessNat griessIdentityVec
-- …
theorem finite_gmat : Finite Generated.Gmat
```

File `VOA/Moonshine/MoonshineCNRationalGriess.lean`. **Hypotheses / status.** None

Theorem 49.32 (The converse, without Tits). *(a) Every automorphism* h *of* $V^\natural$ *maps the rational Griess lattice into itself: for each* $u \in \mathcal{B}$ *there is* $u' \in \mathcal{B}$ *with* $h(\mathrm{griessNat}\, u) = \mathrm{griessNat}\, u'$*. (b)* $\mathrm{Aut}(V^\natural)$ *is isomorphic to the subgroup of* AutO *of liftable elements; the isomorphism is inverse to* `monsterLift`*. (c) If* $G_{\mathrm{mat}} = \mathrm{AutO}$*, then* $\mathrm{Aut}(V^\natural) \cong \mathrm{AutO}$*.*

Lean. `VOA.Moonshine.ConwayNorton.M8.autVnat_mulEquiv_liftable`, `VOA.Moonshine.ConwayNorton.M8.autVnat_mulEquiv_liftable_symm_apply`, `VOA.Moonshine.ConwayNorton.autVnat_rational`, `VOA.Moonshine.ConwayNorton.nonempty_autVnat_mulEquiv_liftable`, `VOA.Moonshine.ConwayNorton.autVnat_mulEquiv_autO`

```
noncomputable def autVnat_mulEquiv_liftable : AutVnat ≃* liftableSubgroup :=
  autVnat_mulEquiv_liftable_of_commutant gmatCommutantScalar
-- …
theorem autVnat_mulEquiv_liftable_symm_apply (g : liftableSubgroup) :
    autVnat_mulEquiv_liftable.symm g = monsterLift g
```

```
theorem autVnat_rational (h : AutVnat) (u : SpaceB) :
    ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat u' :=
  autVnat_rational_of_commutant M8.gmatCommutantScalar h u
-- …
theorem nonempty_autVnat_mulEquiv_liftable : Nonempty (AutVnat ≃* liftableSubgroup) :=
  ⟨M8.autVnat_mulEquiv_liftable⟩
-- …
theorem autVnat_mulEquiv_autO (hGmat : Generated.Gmat = AutO) : Nonempty (AutVnat ≃* AutO)
 ↪  :=
  autVnat_mulEquiv_autO_of_commutant_of_Gmat M8.gmatCommutantScalar hGmat
```

Files `VOA/Moonshine/MoonshineLRCommutantFinal.lean`, `VOA/Moonshine/MoonshineCNSymmetryFinalAudit.lean`. **Hypotheses / status.** None

Definition 49.33 (The generation theorem, as a proposition). GmatEqAutO is the proposition $G_{\mathrm{mat}} = \mathrm{AutO}$: the subgroup of $\mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ generated by the visible group and τ_B is the full

group of $\mathbb{Q}$-linear automorphisms of AxiomMath's rational Griess algebra preserving its product and its form. It is AxiomMath's statement `gmat_eq_griess` after renaming, with one definition (`SpinLattice.phi0`) restated in a definitionally equal form; that statement is proved in AxiomMath's library (Lean 4.35, commit `69cb972f`); the literature milestones on its proof path that were inspected (`cl_lit_stroth_leon_sims_baby_monster_recognition`, its `_models` variant, and `cl_lit_goldschmidt_strongly_closed_abelian`) are theorems there, not axioms; its full axiom footprint has not been re-checked (that needs `#print axioms` on a Lean 4.35 build of AxiomMath's library).

Lean. `VOA.Moonshine.ConwayNorton.GmatEqAutO`

```
def GmatEqAutO : Prop :=
  Generated.Gmat = AutO
```

File `VOA/Moonshine/MoonshineLRMonsterHeadline.lean`. **Hypotheses / status.** (a `Prop`)

Theorem 49.34 ($\mathrm{AutO} \cong \mathrm{Aut}(V^\natural)$**, given** GmatEqAutO**).** *Assume* $G_{\mathrm{mat}} = \mathrm{AutO}$. *Then every element of* AutO *is liftable, the lift* $\mathrm{AutO} \to \mathrm{Aut}(V^\natural)$ *is bijective, and the resulting group isomorphism* φ *satisfies* $\varphi(g)|_{V^\natural{}_2} = \mathcal{G} \circ (1_{\mathbb{C}} \otimes g) \circ \mathcal{G}^{-1}$ *for every* g, $\mathcal{G} =$ `griessIso`.

Lean. `VOA.Moonshine.ConwayNorton.isVnatLiftable_of_gmatEqAutO`, `VOA.Moonshine.ConwayNorton.monsterLiftOfAll_bijective`, `VOA.Moonshine.ConwayNorton.monsterEquivAutVnat`, `VOA.Moonshine.ConwayNorton.monsterEquivAutVnat_restrict`

```
theorem isVnatLiftable_of_gmatEqAutO (hM : GmatEqAutO) (g : AutO) : IsVnatLiftable g
-- …
theorem monsterLiftOfAll_bijective (hM : GmatEqAutO) :
    Function.Bijective (monsterLiftOfAll (isVnatLiftable_of_gmatEqAutO hM))
-- …
noncomputable def monsterEquivAutVnat (hM : GmatEqAutO) : AutO ≃* AutVnat :=
  MulEquiv.ofBijective (monsterLiftOfAll (isVnatLiftable_of_gmatEqAutO hM))
    (monsterLiftOfAll_bijective hM)
-- …
theorem monsterEquivAutVnat_restrict (hM : GmatEqAutO) (g : AutO) :
    vnatWeightTwoRestrict (monsterEquivAutVnat hM g) =
      griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) :=
  monsterLiftOfAll_restrict (isVnatLiftable_of_gmatEqAutO hM) g
```

File `VOA/Moonshine/MoonshineLRMonsterHeadline.lean`. **Hypotheses / status.** `GmatEqAutO`

Theorem 49.35 (Monstrous moonshine for the Monster; Conway–Norton, Borcherds). *Assume* $G_{\mathrm{mat}} = \mathrm{AutO}$. *Then there is a group isomorphism* $\varphi : \mathrm{AutO} \to \mathrm{Aut}(V^\natural)$ *such that*

(i) for every $g \in \mathrm{AutO}$, $\varphi(g)$ *acts on* $V^\natural{}_2 \cong \mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B}$ *as* $1 \otimes g$; *and*

(ii) for every $g \in \mathrm{AutO}$ *there are a subgroup* $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ *(in Lean: a subgroup of* $\mathrm{GL}_2(\mathbb{R})$ *all of whose elements have determinant* 1*) and* $N \geq 1$ *with* $\Gamma_0(N) \leq \Gamma$, Γ *commensurable with* $\mathrm{SL}_2(\mathbb{Z})$, *such that*

$$T_{\varphi(g)}(\tau) = q^{-1} + \sum_{n \geq 1} \mathrm{tr}\big(\varphi(g)|V^\natural{}_{n+1}\big) q^n, \qquad q = e^{2\pi i \tau},$$

is a normalised Hauptmodul for Γ.

Lean. `VOA.Moonshine.ConwayNorton.monstrousMoonshine`

```
theorem monstrousMoonshine (hM : GmatEqAutO) :
    ∃ φ : AutO ≃* AutVnat,
      (∀ g : AutO, vnatWeightTwoRestrict (φ g) = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB)) ∧
        ConwayNortonVia (φ : AutO →* AutVnat) :=
  ⟨monsterEquivAutVnat hM, monsterEquivAutVnat_restrict hM, fun _ => conwayNorton _⟩
```

File `VOA/Moonshine/MoonshineLRMonsterHeadline.lean`. **Hypotheses / status.** **`GmatEqAutO`**

Part IX

Verification

Chapter 50

Verification, dead ends, numerics and open problems

3 of the chapter's 5 numbered statements: its headline results and every definition they use.

Theorem 50.2 (The all-indices twisted Jacobi identity forces $M = 0$). *Let V be a vertex operator algebra, $g \in \operatorname{Aut} V$, $N \geq 2$, and let Y_M be a linear map from V to fractional-mode families of endomorphisms of a vector space M such that $Y_M(\mathbf{1}, z) = \mathrm{id}$ (only the mode $\mathbf{1}_{(-1)}$, label $k = -N$, is nonzero, and it is the identity). If Y_M satisfies the twisted Jacobi identity for all $j, u \in V^j, v, x, \ell, k, m$ without restriction on k, then $M = 0$.*

Lean. `VOA.jacobiAllIndices_eq_zero`
File `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`:

```
theorem jacobiAllIndices_eq_zero {g : Aut W} {N : ℕ} (hN : 2 ≤ N) {M : Type*}
    [AddCommGroup M] [Module ℂ M] (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M))
    (hvac : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0)
    (hJ : JacobiAllIndices g N YM) (x : M) : x = 0 := by
```

Hypotheses / status. Unconditional

Theorem 50.3 (A $\mathbb{C}$-bilinear form is never positive on a nonzero $\mathbb{C}$-subspace). *Let β be a $\mathbb{C}$-bilinear form on a complex vector space E and $S \leq E$ a subspace. If $\beta(x, x)$ is a positive real number for every nonzero $x \in S$, then $S = 0$.*

Lean. `VOA.eq_bot_of_bilinear_pos`
File `VOA/Core/GradedUnitarity.lean`:

```
theorem eq_bot_of_bilinear_pos (β : BilinForm ℂ E) (S : Submodule ℂ E)
    (hpos : ∀ x ∈ S, x ≠ 0 → ∃ r : ℝ, 0 < r ∧ β x x = (r : ℂ)) : S = ⊥ := by
```

Hypotheses / status. Unconditional

Theorem 50.6 (golayE is not determined). *There are two characters e_1, e_2 of Λ_0 with values in $\mathbb{C}^\times$, both satisfying the specification $\psi = g_e\, \theta\, g_e^{-1}$, whose values at the preimage w_0 of $8e_0$ differ by a sign.*

Lean. `VOA.Moonshine.TrialityTw.golayE_not_determined`
File `VOA/Moonshine/TrialityTwSigma.lean`:

```
theorem golayE_not_determined :
    ∃ e₁ e₂ : lam0.carrier →+ Additive ℂˣ,
      (∀ x, lam0Psi x = charLat lam0 e₁ (thetaLat lam0 (charLat lam0 (-e₁) x))) ∧
      (∀ x, lam0Psi x = charLat lam0 e₂ (thetaLat lam0 (charLat lam0 (-e₂) x))) ∧
      ∃ w₀ : lam0.carrier, inclΛ w₀ = frameW ∧ charVal lam0 e₂ w₀ = -charVal lam0 e₁ w₀ :=
       ↪ by
```

Hypotheses / status. Unconditional